Fall 2026

# Probabilistic Artificial Intelligence

*Andreas Krause, Jonas Hübotter*

ETH zürich

Institute for Machine Learning
Department of Computer Science

Compiled on **September 5, 2026**.

This set of notes was written for the course Probabilistic Artificial Intelligence (263-5210-00L) at ETH Zürich. Distribution of these notes without the permission of the authors is prohibited.

© 2026 ETH Zürich. All rights reserved.

# *Preface*

Artificial intelligence commonly refers to the science and engineering of artificial systems that can carry out tasks generally associated with requiring aspects of human intelligence, such as playing games, translating languages, and driving cars. In recent years, there have been exciting advances in learning-based, data-driven approaches towards AI, and machine learning and deep learning have enabled computer systems to perceive the world in unprecedented ways. Reinforcement learning has enabled breakthroughs in complex games such as Go and challenging robotics tasks such as quadrupedal locomotion.

A key aspect of intelligence is to not only make predictions, but reason about the *uncertainty* in these predictions, and to consider this uncertainty when making decisions. This is what this course on "Probabilistic Artificial Intelligence" is about. The first part covers probabilistic approaches to machine learning. We discuss the differentiation between "epistemic" uncertainty due to lack of data and "aleatoric" uncertainty, which is irreducible and stems, e.g., from noisy observations and outcomes. We discuss concrete approaches towards probabilistic inference, such as Bayesian linear regression, Gaussian process models and Bayesian neural networks. Often, inference and making predictions with such models is intractable, and we discuss modern approaches to efficient approximate inference.

The second part of the course is about taking uncertainty into account in sequential decision tasks. We consider active learning and Bayesian optimization — approaches that collect data by proposing experiments that are informative for reducing the epistemic uncertainty. We then consider reinforcement learning, a rich formalism for modeling agents that learn to act in uncertain environments. After covering the basic formalism of Markov Decision Processes, we consider modern deep RL approaches that use neural network function approximation. We close by discussing modern approaches in model-based RL, which harness epistemic and aleatoric uncertainty to guide exploration, while also reasoning about safety.

## *Guide to the Reader*

The material covered in this manuscript may support a one semester graduate introduction to probabilistic machine learning and sequential decision-making. We welcome readers from all backgrounds. However, we assume familiarity with basic concepts in probability, calculus, linear algebra, and machine learning (e.g., neural networks) as covered in a typical introductory course to machine learning. In Chapter 1, we give a gentle introduction to probabilistic inference, which serves as the foundation for the rest of the manuscript. As part of this first chapter, we also review key concepts from probability theory. We provide a chapter reviewing key concepts of further mathematical background in the back of the manuscript.

Throughout the manuscript, we focus on key concepts and ideas rather than their historical development. We encourage you to consult the provided references for further reading and historical context to delve deeper into the covered topics.

Accompanying this manuscript, we provide an extensive set of curated examples as Jupyter notebooks which you can run and play with. You can find them at:
`https://gitlab.inf.ethz.ch/OU-KRAUSE/pai-demos`.

Finally, we have included a set of exercises at the end of each chapter. When we highlight an exercise throughout the text, we use this question mark: ⑦ — so don't be surprised when you stumble upon it. You will find solutions to all exercises in the back of the manuscript.

**Problem 1.1**

We hope you will find this resource useful.

## *Contributing*

We encourage you to raise issues and suggest fixes for anything you think can be improved. We are thankful for any such feedback!
CONTACT: pai-script@lists.inf.ethz.ch
REPOSITORY: `https://gitlab.inf.ethz.ch/OU-KRAUSE/pai-script`

## *Acknowledgements*

We are grateful to Sebastian Curi for creating the original Jupyter notebooks that accompany the course at ETH Zürich and which were instrumental in the creation of many figures. We thank Hado van Hasselt for kindly contributing Figure 12.1, and thank Tuomas Haarnoja (Haarnoja et al., 2018a) and Roberto Calandra (Chua et al., 2018) for kindly agreeing to have their figures included in this manuscript. Furthermore,

many of the exercises in these notes are adapted from iterations of the course at ETH Zürich. Special thanks to all instructors that contributed to the course material over the years. We also thank all students of the course in the Fall of 2022, 2023, and 2024 who provided valuable feedback on various iterations of this manuscript and corrected many mistakes. Finally, we thank Zhiyuan Hu, Shyam Sundhar Ramesh, Leander Diaz-Bone, Nicolas Menet, and Ido Hakimi for proofreading parts of various drafts of this text.

# *Contents*


*1 Fundamentals of Inference* 1

*1.1 Probability* 2

*1.2 Probabilistic Inference* 15

*1.3 Supervised Learning and Point Estimates* 22

*1.4 Outlook: Decision Theory* 29

*I Probabilistic Machine Learning* 35

*2 Linear Regression* 39

*2.1 Weight-space View* 40

*2.2 Aleatoric and Epistemic Uncertainty* 44

*2.3 Non-linear Features* 45

*2.4 Function-space View* 45

*3 Filtering* 51

*3.1 Conditioning and Prediction* 53

*3.2 Kalman Filters* 54

*4 Gaussian Processes* 59

*4.1 Learning and Inference* 60

*4.2 Sampling* 61

*4.3 Kernel Functions* 62

*4.4 Model Selection* 67

*4.5 Approximations* 70

5 *Variational Inference* 83
5.1 *Laplace Approximation* 83
5.2 *Predictions with a Variational Posterior* 87
5.3 *Blueprint of Variational Inference* 88
5.4 *Information Theoretic Aspects of Uncertainty* 89
5.5 *Evidence Lower Bound* 99

6 *Markov Chain Monte Carlo Methods* 113
6.1 *Markov Chains* 114
6.2 *Elementary Sampling Methods* 121
6.3 *Sampling using Gradients* 124

7 *Deep Learning* 139
7.1 *Artificial Neural Networks* 139
7.2 *Bayesian Neural Networks* 142
7.3 *Approximate Probabilistic Inference* 144
7.4 *Calibration* 152
7.5 *Diffusion Generative Models* 155

*II Sequential Decision-Making* 163

8 *Active Learning* 167
8.1 *Conditional Entropy* 167
8.2 *Mutual Information* 168
8.3 *Submodularity of Mutual Information* 172
8.4 *Maximizing Mutual Information* 173
8.5 *Learning Locally: Transductive Active Learning* 177

9 *Bayesian Optimization* 183
9.1 *Exploration-Exploitation Dilemma* 183
9.2 *Online Learning and Bandits* 184
9.3 *Acquisition Functions* 186

10 *Markov Decision Processes* 203
10.1 *Bellman Expectation Equation* 205

*10.2 Policy Evaluation* 206
*10.3 Policy Optimization* 209
*10.4 Partial Observability* 215

*11 Tabular Reinforcement Learning* 223
*11.1 The Reinforcement Learning Problem* 223
*11.2 Model-based Approaches* 225
*11.3 Balancing Exploration and Exploitation* 226
*11.4 Model-free Approaches* 230

*12 Model-free Reinforcement Learning* 239
*12.1 Tabular Reinforcement Learning as Optimization* 239
*12.2 Value Function Approximation* 241
*12.3 Policy Approximation* 244
*12.4 On-policy Actor-Critics* 250
*12.5 Off-policy Actor-Critics* 257
*12.6 Maximum Entropy Reinforcement Learning* 262
*12.7 Learning from Preferences* 266

*13 Model-based Reinforcement Learning* 279
*13.1 Planning* 280
*13.2 Learning* 287
*13.3 Exploration* 293

*A Mathematical Background* 305
*A.1 Probability* 305
*A.2 Quadratic Forms and Gaussians* 307
*A.3 Parameter Estimation* 308
*A.4 Optimization* 319
*A.5 Useful Matrix Identities and Inequalities* 325

*B Solutions* 327

*Bibliography* 391

*Syllabus* 399

*Summary of Notation* 401

*Acronyms* 407

*Index* 409

*Errata* 417

# 1

# *Fundamentals of Inference*

Boolean logic is the algebra of statements which are either true or false. Consider, for example, the statements

"If it is raining, the ground is wet." and "It is raining."

A quite remarkable property of Boolean logic is that we can combine these premises to draw logical inferences which are *new* (true) statements. In the above example, we can conclude that the ground must be wet. This is an example of logical reasoning which is commonly referred to as *logical inference*, and the study of artificial systems that are able to perform logical inference is known as *symbolic artificial intelligence*.

But is it really raining? Perhaps it is hard to tell by looking out of the window. Or we have seen it rain earlier, but some time has passed since we have last looked out of the window. And is it really true that if it rains, the ground is wet? Perhaps the rain is just light enough that it is absorbed quickly, and therefore the ground still appears dry.

This goes to show that in our experience, the real world is rarely black and white. We are frequently (if not usually) uncertain about the truth of statements, and yet we are able to reason about the world and make predictions. We will see that the principles of Boolean logic can be extended to reason in the face of uncertainty. The mathematical framework that allows us to do this is probability theory, which — as we will find in this first chapter — can be seen as a natural extension of Boolean logic from the domain of certainty to the domain of uncertainty. In fact, in the 20th century, Richard Cox and Edwin Thompson Jaynes have done early work to formalize probability theory as the "logic under uncertainty" (Cox, 1961; Jaynes, 2002).

In this first chapter, we will briefly recall the fundamentals of probability theory, and we will see how *probabilistic inference* can be used

to reason about the world. In the remaining chapters, we will then discuss how probabilistic inference can be performed efficiently given limited computational resources and limited time, which is the key challenge in *probabilistic artificial intelligence*.

## 1.1 Probability

Probability is commonly interpreted in two different ways. In the frequentist interpretation, one interprets the probability of an event (say a coin coming up "heads" when flipping it) as the limit of relative frequencies in repeated independent experiments. That is,

$$\text{Probability} = \lim_{N \to \infty} \frac{\#\text{ events happening in } N \text{ trials}}{N}.$$

This interpretation is natural, but has a few issues. It is not very difficult to conceive of settings where repeated experiments do not make sense. Consider the outcome:

"Person X will live for at least 80 years."

There is no way in which we could conduct multiple independent experiments in this case. Still, this statement is going to turn out either true or false, as humans we are just not able to determine its truth value beforehand. Nevertheless, humans commonly have *beliefs* about statements of this kind. We also commonly reason about statements such as

"The Beatles were more groundbreaking than The Monkees."

This statement does not even have an objective truth value, and yet we as humans tend to have opinions about it.

While it is natural to consider the relative frequency of the outcome in repeated experiments as our belief, if we are not able to conduct repeated experiments, our notion of probability is simply a subjective measure of uncertainty about outcomes. In the early 20th century, Bruno De Finetti has done foundational work to formalize this notion which is commonly called *Bayesian reasoning* or the Bayesian interpretation of probability (De Finetti, 1970).

We will see that modern approaches to probabilistic inference often lend themselves to a Bayesian interpretation, even if such an interpretation is not strictly necessary. For our purposes, probabilities will be a means to an end: the end usually being solving some task. This task may be to make a prediction or to take an action with an uncertain outcome, and we can evaluate methods according to how well they perform on this task. No matter the interpretation, the mathematical

framework of probability theory which we will formally introduce in the following is the same.

### 1.1.1 Probability Spaces

A probability space is a mathematical model for a random experiment. The set of all possible outcomes of the experiment $\Omega$ is called *sample space*. An *event* $A \subseteq \Omega$ of interest may be any combination of possible outcomes. The set of all events $\mathcal{A} \subseteq \mathcal{P}(\Omega)$ that we are interested in is often called the *event space* of the experiment.[1] This set of events is required to be a $\sigma$-algebra over the sample space.

[1] We use $\mathcal{P}(\Omega)$ to denote the *power set* (set of all subsets) of $\Omega$.

**Definition 1.1** ($\sigma$-algebra)**.** Given the set $\Omega$, the set $\mathcal{A} \subseteq \mathcal{P}(\Omega)$ is a *$\sigma$-algebra* over $\Omega$ if the following properties are satisfied:

1. $\Omega \in \mathcal{A}$;
2. if $A \in \mathcal{A}$, then $\overline{A} \in \mathcal{A}$ (*closedness under complements*); and
3. if we have $A_i \in \mathcal{A}$ for all $i$, then $\bigcup_{i=1}^{\infty} A_i \in \mathcal{A}$ (*closedness under countable unions*).

Note that the three properties of $\sigma$-algebras correspond to characteristics we universally expect when working with random experiments. Namely, that we are able to reason about the event $\Omega$ that any of the possible outcomes occur, that we are able to reason about an event not occurring, and that we are able to reason about events that are composed of multiple (smaller) events.

**Example 1.2: Event space of throwing a die**

The event space $\mathcal{A}$ can also be thought of as "how much information is available about the experiment". For example, if the experiment is a throw of a die and $\Omega$ is the set of possible values on the die: $\Omega = \{1, \ldots, 6\}$, then the following $\mathcal{A}$ implies that the observer cannot distinguish between 1 and 3:

$$\mathcal{A} \doteq \{\emptyset, \Omega, \{1,3,5\}, \{2,4,6\}\}.$$

Intuitively, the observer only understands the parity of the face of the die.

**Definition 1.3** (Probability measure)**.** Given the set $\Omega$ and the $\sigma$-algebra $\mathcal{A}$ over $\Omega$, the function

$$\mathbb{P} : \mathcal{A} \to \mathbb{R}$$

is a *probability measure* on $\mathcal{A}$ if the *Kolmogorov axioms* are satisfied:

1. $0 \leq \mathbb{P}(A) \leq 1$ for any $A \in \mathcal{A}$;
2. $\mathbb{P}(\Omega) = 1$; and
3. $\mathbb{P}(\bigcup_{i=1}^{\infty} A_i) = \sum_{i=1}^{\infty} \mathbb{P}(A_i)$ for any countable set of mutually disjoint events $\{A_i \in \mathcal{A}\}_i$.[2]

[2] We say that a set of sets $\{A_i\}_i$ is disjoint if for all $i \neq j$ we have $A_i \cap A_j = \emptyset$.

Remarkably, all further statements about probability follow from these three natural axioms. For an event $A \in \mathcal{A}$, we call $\mathbb{P}(A)$ the *probability* of $A$. We are now ready to define a probability space.

**Definition 1.4** (Probability space)**.** A *probability space* is a triple $(\Omega, \mathcal{A}, \mathbb{P})$ where

- $\Omega$ is a sample space,
- $\mathcal{A}$ is a $\sigma$-algebra over $\Omega$, and
- $\mathbb{P}$ is a probability measure on $\mathcal{A}$.

**Example 1.5: Borel $\sigma$-algebra over $\mathbb{R}$**

In our context, we often have that $\Omega$ is the set of real numbers $\mathbb{R}$ or a compact subset of it. In this case, a natural event space is the $\sigma$-algebra generated by the set of events

$$A_x \doteq \{x' \in \Omega : x' \leq x\}.$$

The smallest $\sigma$-algebra $\mathcal{A}$ containing all sets $A_x$ is called the *Borel $\sigma$-algebra*. $\mathcal{A}$ contains all "reasonable" subsets of $\Omega$ (except for some pathological examples). For example, $\mathcal{A}$ includes all singleton sets $\{x\}$, as well as all countable unions of intervals.

In the case of discrete $\Omega$, in fact $\mathcal{A} = \mathcal{P}(\Omega)$, i.e., the Borel $\sigma$-algebra contains *all* subsets of $\Omega$.

### 1.1.2 Random Variables

The set $\Omega$ is often rather complex. For example, take $\Omega$ to be the set of all possible graphs on $n$ vertices. Then the outcome of our experiment is a graph. Usually, we are not interested in a specific graph but rather a property such as the number of edges, which is shared by many graphs. A function that maps a graph to its number of edges is a random variable.

**Definition 1.6** (Random variable)**.** A *random variable* $X$ is a function

$$X : \Omega \to \mathcal{T}$$

where $\mathcal{T}$ is called *target space* of the random variable,[3] and where $X$ respects the information available in the $\sigma$-algebra $\mathcal{A}$.

[3] For a random variable that maps a graph to its number of edges, $\mathcal{T} = \mathbb{N}_0$. For our purposes, you can generally assume $\mathcal{T} \subseteq \mathbb{R}$.

**Remark 1.7: Respecting the information in $\mathcal{A}$**

The random variable $X$ is required to respect the information available in the $\sigma$-algebra $\mathcal{A}$.[4] Formally, for a $\sigma$-algebra $\mathcal{B}$ over $\mathcal{T}$ we must have

$$\forall S \in \mathcal{B} : \quad \{\omega \in \Omega : X(\omega) \in S\} \in \mathcal{A}. \tag{1.1}$$

[4] In our example of throwing a die, $X$ should assign the same value to the outcomes $1, 3, 5$.

If $\mathcal{T}$ is a finite set then $\mathcal{B}$ can be simply its power set $\mathcal{P}(\mathcal{T})$. If $\mathcal{T} = \mathbb{R}$ then $\mathcal{B}$ is typically taken to be the Borel $\sigma$-algebra over $\mathbb{R}$.

Concrete values $x$ of a random variable $X$ are often referred to as *states* or *realizations* of $X$. The probability that $X$ takes on a value in $S \subseteq \mathcal{T}$ is

$$\mathbb{P}(X \in S) = \mathbb{P}(\{\omega \in \Omega : X(\omega) \in S\}). \tag{1.2}$$

### 1.1.3 Distributions

Consider a random variable $X$ on a probability space $(\Omega, \mathcal{A}, \mathbb{P})$, where $\Omega$ is a compact subset of $\mathbb{R}$, and $\mathcal{A}$ the Borel $\sigma$-algebra.

In this case, we can refer to the probability that $X$ assumes a particular state or set of states by writing

$$p_X(x) \doteq \mathbb{P}(X = x) \quad \text{(in the discrete setting)}, \tag{1.3}$$

$$P_X(x) \doteq \mathbb{P}(X \leq x). \tag{1.4}$$

Note that "$X = x$" and "$X \leq x$" are merely events (that is, they characterize subsets of the sample space $\Omega$ satisfying this condition) which are in the Borel $\sigma$-algebra, and hence their probability is well-defined.

Hereby, $p_X$ and $P_X$ are referred to as the probability mass function (PMF) and cumulative distribution function (CDF) of $X$, respectively. Note that we can also *implicitly* define probability spaces through random variables and their associated PMF/CDF, which is often very convenient.

We list some common examples of discrete distributions in Section A.1.1. Further, note that for continuous variables, $\mathbb{P}(X = x) = 0$. Here, instead we typically use the probability density function (PDF), to which we (with slight abuse of notation) also refer with $p_X$. We discuss densities in greater detail in Section 1.1.4.

We call the subset $S \subseteq \mathcal{T}$ of the domain of a PMF or PDF $p_X$ such that all elements $x \in S$ have positive probability, $p_X(x) > 0$, the *support* of the distribution $p_X$. This quantity is denoted by $X(\Omega)$.

### 1.1.4 Continuous Distributions

As mentioned, a continuous random variable can be characterized by its *probability density function* (PDF). But what is a density? We can derive some intuition from physics.

Let $M$ be a (non-homogeneous) physical object, e.g., a rock. We commonly use $m(M)$ and $\mathrm{vol}(M)$ to refer to its mass and volume, respectively. Now, consider for a point $\boldsymbol{x} \in M$ and a ball $B_r(\boldsymbol{x})$ around $\boldsymbol{x}$ with

radius $r$ the following quantities:

$$\lim_{r \to 0} \operatorname{vol}(B_r(\boldsymbol{x})) = 0 \quad \lim_{r \to 0} m(B_r(\boldsymbol{x})) = 0.$$

They appear utterly uninteresting at first, yet, if we divide them, we get what is called the *density* of $M$ at $\boldsymbol{x}$.

$$\lim_{r \to 0} \frac{m(B_r(\boldsymbol{x}))}{\operatorname{vol}(B_r(\boldsymbol{x}))} \doteq \rho(\boldsymbol{x}).$$

We know that the relationship between density and mass is described by the following formula:

$$m(M) = \int_M \rho(\boldsymbol{x}) \, d\boldsymbol{x}.$$

In other words, the density is to be integrated. For a small region $I$ around $\boldsymbol{x}$, we can approximate $m(I) \approx \rho(\boldsymbol{x}) \cdot \operatorname{vol}(I)$.

Crucially, observe that even though the mass of any particular point $\boldsymbol{x}$ is zero, i.e., $m(\{\boldsymbol{x}\}) = 0$, assigning a density $\rho(\boldsymbol{x})$ to $\boldsymbol{x}$ is useful for integration and approximation. The same idea applies to continuous random variables, only that volume corresponds to intervals on the real line and mass to probability. Recall that probability density functions are normalized such that their probability mass across the entire real line integrates to one.

**Example 1.8: Normal distribution / Gaussian**

A famous example of a continuous distribution is the *normal distribution*, also called *Gaussian*. We say, a random variable $X$ is *normally distributed*, $X \sim \mathcal{N}(\mu, \sigma^2)$, if its PDF is

$$\mathcal{N}(x; \mu, \sigma^2) \doteq \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{1.5}$$

We have $\mathbb{E}[X] = \mu$ and $\operatorname{Var}[X] = \sigma^2$. If $\mu = 0$ and $\sigma^2 = 1$, this distribution is called the *standard normal distribution*. The Gaussian CDF cannot be expressed in closed-form.

Note that the mean of a Gaussian distribution coincides with the maximizer of its PDF, also called *mode* of a distribution.

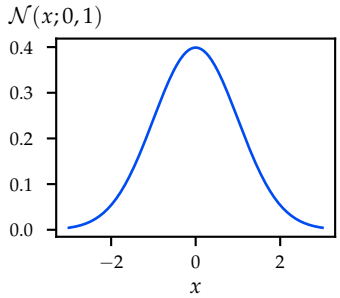



Figure 1.1: PDF of the standard normal distribution. Observe that the PDF is symmetric around the mode.

We will focus in the remainder of this chapter on continuous distributions, but the concepts we discuss extend mostly to discrete distributions simply by "replacing integrals by sums".

### 1.1.5 Joint Probability

A joint probability (as opposed to a marginal probability) is the probability of two or more events occurring simultaneously:

$$\mathbb{P}(A, B) \doteq \mathbb{P}(A \cap B). \tag{1.6}$$

In terms of random variables, this concept extends to joint distributions. Instead of characterizing a single random variable, a *joint distribution* is a function $p_{\mathbf{X}} : \mathbb{R}^n \to \mathbb{R}$, characterizing a *random vector* $\mathbf{X} \doteq [X_1 \ \cdots \ X_n]^\top$. For example, if the $X_i$ are discrete, the joint distribution characterizes joint probabilities of the form

$$\mathbb{P}(\mathbf{X} = [x_1, \ldots, x_n]) = \mathbb{P}(X_1 = x_1, \ldots, X_n = x_n),$$

and hence describes the relationship among all variables $X_i$. For this reason, a joint distribution is also called a *generative model*. We use $X_{i:j}$ to denote the random vector $[X_i \ \cdots \ X_j]^\top$.

We can "sum out" (respectively "integrate out") variables from a joint distribution in a process called "marginalization":

**Fact 1.9** (Sum rule). *We have that*

$$p(x_{1:i-1}, x_{i+1:n}) = \int_{X_i(\Omega)} p(x_{1:i-1}, x_i, x_{i+1:n}) \, dx_i. \tag{1.7}$$

### 1.1.6 Conditional Probability

Conditional probability updates the probability of an event $A$ given some new information, for example, after observing the event $B$.

**Definition 1.10** (Conditional probability). Given two events $A$ and $B$ such that $\mathbb{P}(B) > 0$, the probability of $A$ conditioned on $B$ is given as

$$\mathbb{P}(A \mid B) \doteq \frac{\mathbb{P}(A, B)}{\mathbb{P}(B)}. \tag{1.8}$$

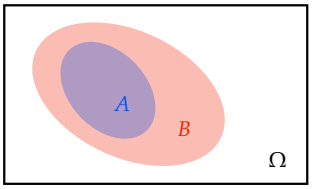



Figure 1.2: Conditioning an event $A$ on another event $B$ can be understood as replacing the universe of all possible outcomes $\Omega$ by the observed outcomes $B$. Then, the conditional probability is simply expressing the likelihood of $A$ given that $B$ occurred.

Simply rearranging the terms yields,

$$\mathbb{P}(A, B) = \mathbb{P}(A \mid B) \cdot \mathbb{P}(B) = \mathbb{P}(B \mid A) \cdot \mathbb{P}(A). \tag{1.9}$$

Thus, the probability that both $A$ and $B$ occur can be calculated by multiplying the probability of event $A$ and the probability of $B$ conditional on $A$ occurring.

We say $\mathbf{Z} \sim \mathbf{X} \mid \mathbf{Y} = \boldsymbol{y}$ (or simply $\mathbf{Z} \sim \mathbf{X} \mid \boldsymbol{y}$) if $\mathbf{Z}$ follows the *conditional distribution*

$$p_{\mathbf{X}|\mathbf{Y}}(\boldsymbol{x} \mid \boldsymbol{y}) \doteq \frac{p_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x}, \boldsymbol{y})}{p_{\mathbf{Y}}(\boldsymbol{y})}. \tag{1.10}$$

If $\mathbf{X}$ and $\mathbf{Y}$ are discrete, we have that $p_{\mathbf{X}|\mathbf{Y}}(\boldsymbol{x} \mid \boldsymbol{y}) = \mathbb{P}(\mathbf{X} = \boldsymbol{x} \mid \mathbf{Y} = \boldsymbol{y})$ as one would naturally expect.

Extending Equation (1.9) to arbitrary random vectors yields the product rule (also called the *chain rule of probability*):

**Fact 1.11** (Product rule). *Given random variables $X_{1:n}$,*

$$p(x_{1:n}) = p(x_1) \cdot \prod_{i=2}^{n} p(x_i \mid x_{1:i-1}). \tag{1.11}$$

Combining sum rule and product rule, we can compute marginal probabilities too:

$$p(\boldsymbol{x}) = \int_{\mathbf{Y}(\Omega)} p(\boldsymbol{x}, \boldsymbol{y})\, d\boldsymbol{y} = \int_{\mathbf{Y}(\Omega)} p(\boldsymbol{x} \mid \boldsymbol{y}) \cdot p(\boldsymbol{y})\, d\boldsymbol{y} \tag{1.12}$$

first using the sum rule (1.7) then the product rule (1.11)

This is called the *law of total probability* (LOTP), which is colloquially often referred to as *conditioning* on $\mathbf{Y}$. If it is difficult to compute $p(\boldsymbol{x})$ directly, conditioning can be a useful technique when $\mathbf{Y}$ is chosen such that the densities $p(\boldsymbol{x} \mid \boldsymbol{y})$ and $p(\boldsymbol{y})$ are straightforward to understand.

### 1.1.7 Independence

Two random vectors $\mathbf{X}$ and $\mathbf{Y}$ are *independent* (denoted $\mathbf{X} \perp \mathbf{Y}$) if and only if knowledge about the state of one random vector does not affect the distribution of the other random vector, namely if their conditional CDF (or in case they have a joint density, their conditional PDF) simplifies to

$$P_{\mathbf{X}|\mathbf{Y}}(\boldsymbol{x} \mid \boldsymbol{y}) = P_{\mathbf{X}}(\boldsymbol{x}), \quad p_{\mathbf{X}|\mathbf{Y}}(\boldsymbol{x} \mid \boldsymbol{y}) = p_{\mathbf{X}}(\boldsymbol{x}). \tag{1.13}$$

For the conditional probabilities to be well-defined, we need to assume that $p_{\mathbf{Y}}(\boldsymbol{y}) > 0$.

The more general characterization of independence is that $\mathbf{X}$ and $\mathbf{Y}$ are independent if and only if their joint CDF (or in case they have a joint density, their joint PDF) can be decomposed as follows:

$$P_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x}, \boldsymbol{y}) = P_{\mathbf{X}}(\boldsymbol{x}) \cdot P_{\mathbf{Y}}(\boldsymbol{y}), \quad p_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x}, \boldsymbol{y}) = p_{\mathbf{X}}(\boldsymbol{x}) \cdot p_{\mathbf{Y}}(\boldsymbol{y}). \tag{1.14}$$

The equivalence of the two characterizations (when $p_{\mathbf{Y}}(\boldsymbol{y}) > 0$) is easily proven using the product rule: $p_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x}, \boldsymbol{y}) = p_{\mathbf{Y}}(\boldsymbol{y}) \cdot p_{\mathbf{X}|\mathbf{Y}}(\boldsymbol{x} \mid \boldsymbol{y})$.

A "weaker" notion of independence is conditional independence.[5] Two random vectors $\mathbf{X}$ and $\mathbf{Y}$ are *conditionally independent* given a random vector $\mathbf{Z}$ (denoted $\mathbf{X} \perp \mathbf{Y} \mid \mathbf{Z}$) iff, given $\mathbf{Z}$, knowledge about the value of one random vector $\mathbf{Y}$ does not affect the distribution of the other random vector $\mathbf{X}$, namely if

$$P_{\mathbf{X}|\mathbf{Y},\mathbf{Z}}(\boldsymbol{x} \mid \boldsymbol{y}, \boldsymbol{z}) = P_{\mathbf{X}|\mathbf{Z}}(\boldsymbol{x} \mid \boldsymbol{z}), \tag{1.15a}$$

[5] We discuss in Remark 1.12 how "weaker" is to be interpreted in this context.

$$p_{\mathbf{X}|\mathbf{Y},\mathbf{Z}}(\boldsymbol{x} \mid \boldsymbol{y}, \boldsymbol{z}) = p_{\mathbf{X}|\mathbf{Z}}(\boldsymbol{x} \mid \boldsymbol{z}). \tag{1.15b}$$

Similarly to independence, we have that $\mathbf{X}$ and $\mathbf{Y}$ are conditionally independent given $\mathbf{Z}$ if and only if their joint CDF or joint PDF can be decomposed as follows:

$$P_{\mathbf{X},\mathbf{Y}|\mathbf{Z}}(\boldsymbol{x}, \boldsymbol{y} \mid \boldsymbol{z}) = P_{\mathbf{X}|\mathbf{Z}}(\boldsymbol{x} \mid \boldsymbol{z}) \cdot P_{\mathbf{Y}|\mathbf{Z}}(\boldsymbol{y} \mid \boldsymbol{z}), \tag{1.16a}$$

$$p_{\mathbf{X},\mathbf{Y}|\mathbf{Z}}(\boldsymbol{x}, \boldsymbol{y} \mid \boldsymbol{z}) = p_{\mathbf{X}|\mathbf{Z}}(\boldsymbol{x} \mid \boldsymbol{z}) \cdot p_{\mathbf{Y}|\mathbf{Z}}(\boldsymbol{y} \mid \boldsymbol{z}). \tag{1.16b}$$

**Remark 1.12: Common causes**
How can conditional independence be understood as a "weaker" notion of independence? Clearly, conditional independence does not imply independence: a trivial example is $X \perp X \mid X \not\Longrightarrow X \perp X$.[6] Neither does independence imply conditional independence: for example, $X \perp Y \not\Longrightarrow X \perp Y \mid X + Y$.[7]

When we say that conditional independence is a weaker notion we mean to emphasize that $X$ and $Y$ can be "made" (conditionally) independent by conditioning on the "right" $Z$ even if $X$ and $Y$ are dependent. This is known as *Reichenbach's common cause principle* which says that for any two random variables $X \not\perp Y$ there exists a random variable $Z$ (which may be $X$ or $Y$) that causally influences both $X$ and $Y$, and which is such that $X \perp Y \mid Z$.

[6] $X \perp X \mid X$ is true trivially.

[7] Knowing $X$ and $X + Y$ already implies the value of $Y$, and hence, $X \not\perp Y \mid X + Y$.

### 1.1.8 Directed Graphical Models

Directed graphical models (also called *Bayesian networks*) are often used to visually denote the (conditional) independence relationships of a large number of random variables. They are a schematic representation of the factorization of the generative model into a product of conditional distributions as a directed acyclic graph. Given the sequence of random variables $\{X_i\}_{i=1}^n$, their generative model can be expressed as

$$p(x_{1:n}) = \prod_{i=1}^{n} p(x_i \mid \text{parents}(x_i)) \tag{1.17}$$

where $\text{parents}(x_i)$ is the set of parents of the vertex $X_i$ in the directed graphical model. In other words, the parenthood relationship encodes a conditional independence of a random variable $X$ with a random variable $Y$ given their parents:[8]

$$X \perp Y \mid \text{parents}(X), \text{parents}(Y). \tag{1.18}$$

Equation (1.17) simply uses the product rule and the conditional independence relationships to factorize the generative model. This can greatly reduce the model's complexity, i.e., the length of the product.

[8] More generally, vertices $u$ and $v$ are conditionally independent given a set of vertices $Z$ if $Z$ *d-separates* $u$ and $v$, which we will not cover in depth here.

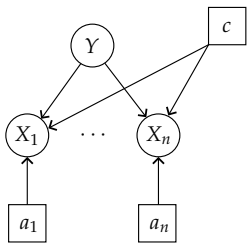



Figure 1.3: Example of a directed graphical model. The random variables $X_1, \ldots, X_n$ are mutually independent given the random variable $Y$. The squared rectangular nodes are used to represent dependencies on parameters $c, a_1, \ldots, a_n$.

An example of a directed graphical model is given in Figure 1.3. Circular vertices represent random quantities (i.e., random variables). In contrast, square vertices are commonly used to represent deterministic quantities (i.e., parameters that the distributions depend on). In the given example, we have that $X_i$ is conditionally independent of all other $X_j$ given $Y$. *Plate notation* is a condensed notation used to represent repeated variables of a graphical model. An example is given in Figure 1.4.

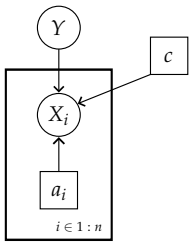



Figure 1.4: The same directed graphical model as in Figure 1.3 using plate notation.

### 1.1.9 Expectation

The *expected value* or *mean* $\mathbb{E}[\mathbf{X}]$ of a random vector $\mathbf{X}$ is the (asymptotic) arithmetic mean of an arbitrarily increasing number of independent realizations of $\mathbf{X}$. That is,[9]

$$\mathbb{E}[\mathbf{X}] \doteq \int_{\mathbf{X}(\Omega)} \boldsymbol{x} \cdot p(\boldsymbol{x}) \, d\boldsymbol{x} \tag{1.19}$$

[9] In infinite probability spaces, absolute convergence of $\mathbb{E}[\mathbf{X}]$ is necessary for the existence of $\mathbb{E}[\mathbf{X}]$.

A very special and often used property of expectations is their *linearity*, namely that for any random vectors $\mathbf{X}$ and $\mathbf{Y}$ in $\mathbb{R}^n$ and any $\boldsymbol{A} \in \mathbb{R}^{m \times n}, \boldsymbol{b} \in \mathbb{R}^m$ it holds that

$$\mathbb{E}[\boldsymbol{A}\mathbf{X} + \boldsymbol{b}] = \boldsymbol{A}\mathbb{E}[\mathbf{X}] + \boldsymbol{b} \quad \text{and} \quad \mathbb{E}[\mathbf{X} + \mathbf{Y}] = \mathbb{E}[\mathbf{X}] + \mathbb{E}[\mathbf{Y}]. \tag{1.20}$$

Note that $\mathbf{X}$ and $\mathbf{Y}$ do not necessarily have to be independent! Further, if $\mathbf{X}$ and $\mathbf{Y}$ are independent then

$$\mathbb{E}\left[\mathbf{X}\mathbf{Y}^\top\right] = \mathbb{E}[\mathbf{X}] \cdot \mathbb{E}[\mathbf{Y}]^\top. \tag{1.21}$$

The following intuitive lemma can be used to compute expectations of transformed random variables.

**Fact 1.13** (Law of the unconscious statistician, LOTUS)**.**

$$\mathbb{E}[g(\mathbf{X})] = \int_{\mathbf{X}(\Omega)} g(\boldsymbol{x}) \cdot p(\boldsymbol{x}) \, d\boldsymbol{x} \tag{1.22}$$

*where $g : \mathbf{X}(\Omega) \to \mathbb{R}^n$ is a "nice" function[10] and $\mathbf{X}$ is a continuous random vector. The analogous statement with a sum replacing the integral holds for discrete random variables.*

[10] $g$ being a continuous function, which is either bounded or absolutely integrable (i.e., $\int |g(\boldsymbol{x})| \, p(\boldsymbol{x}) \, d\boldsymbol{x} < \infty$), is sufficient. This is satisfied in most cases.

Similarly to conditional probability, we can also define conditional expectations. The expectation of a continuous random vector $\mathbf{X}$ given that $\mathbf{Y} = \boldsymbol{y}$ is defined as

$$\mathbb{E}[\mathbf{X} \mid \mathbf{Y} = \boldsymbol{y}] \doteq \int_{\mathbf{X}(\Omega)} \boldsymbol{x} \cdot p_{\mathbf{X}|\mathbf{Y}}(\boldsymbol{x} \mid \boldsymbol{y}) \, d\boldsymbol{x}. \tag{1.23}$$

Observe that $\mathbb{E}[\mathbf{X} \mid \mathbf{Y} = \cdot]$ defines a deterministic mapping from $\boldsymbol{y}$ to $\mathbb{E}[\mathbf{X} \mid \mathbf{Y} = \boldsymbol{y}]$. Therefore, $\mathbb{E}[\mathbf{X} \mid \mathbf{Y}]$ is itself a random vector:

$$\mathbb{E}[\mathbf{X} \mid \mathbf{Y}](\omega) = \mathbb{E}[\mathbf{X} \mid \mathbf{Y} = \mathbf{Y}(\omega)] \tag{1.24}$$

where $\omega \in \Omega$. This random vector $\mathbb{E}[\mathbf{X} \mid \mathbf{Y}]$ is called the *conditional expectation* of $\mathbf{X}$ given $\mathbf{Y}$.

Analogously to the law of total probability (1.12), one can condition an expectation on another random vector. This is known as the *tower rule* or the *law of total expectation* (LOTE):

**Theorem 1.14** (Tower rule)**.** *Given random vectors* $\mathbf{X}$ *and* $\mathbf{Y}$*, we have*

$$\mathbb{E}_{\mathbf{Y}}[\mathbb{E}_{\mathbf{X}}[\mathbf{X} \mid \mathbf{Y}]] = \mathbb{E}[\mathbf{X}]. \tag{1.25}$$

*Proof sketch.* We only prove the case where $\mathbf{X}$ and $\mathbf{Y}$ have a joint density. We have

$$\begin{aligned}
\mathbb{E}[\mathbb{E}[\mathbf{X} \mid \mathbf{Y}]] &= \int \left( \int \boldsymbol{x} \cdot p(\boldsymbol{x} \mid \boldsymbol{y}) \, d\boldsymbol{x} \right) p(\boldsymbol{y}) \, d\boldsymbol{y} \\
&= \int \int \boldsymbol{x} \cdot p(\boldsymbol{x}, \boldsymbol{y}) \, d\boldsymbol{x} \, d\boldsymbol{y} && \text{by definition of conditional densities (1.10)} \\
&= \int \boldsymbol{x} \int p(\boldsymbol{x}, \boldsymbol{y}) \, d\boldsymbol{y} \, d\boldsymbol{x} && \text{by Fubini's theorem} \\
&= \int \boldsymbol{x} \cdot p(\boldsymbol{x}) \, d\boldsymbol{x} && \text{using the sum rule (1.7)} \\
&= \mathbb{E}[\mathbf{X}]. && \square
\end{aligned}$$

### 1.1.10 Covariance and Variance

Given two random vectors $\mathbf{X}$ in $\mathbb{R}^n$ and $\mathbf{Y}$ in $\mathbb{R}^m$, their *covariance* is defined as

$$\begin{aligned}
\operatorname{Cov}[\mathbf{X}, \mathbf{Y}] &\doteq \mathbb{E}\left[ (\mathbf{X} - \mathbb{E}[\mathbf{X}])(\mathbf{Y} - \mathbb{E}[\mathbf{Y}])^\top \right] && (1.26) \\
&= \mathbb{E}\left[ \mathbf{X}\mathbf{Y}^\top \right] - \mathbb{E}[\mathbf{X}] \cdot \mathbb{E}[\mathbf{Y}]^\top && (1.27) \\
&= \operatorname{Cov}[\mathbf{Y}, \mathbf{X}]^\top \in \mathbb{R}^{n \times m}. && (1.28)
\end{aligned}$$

Covariance measures the linear dependence between two random vectors since a direct consequence of its definition (1.26) is that given linear maps $\boldsymbol{A} \in \mathbb{R}^{n' \times n}$, $\boldsymbol{B} \in \mathbb{R}^{m' \times m}$, vectors $\boldsymbol{c} \in \mathbb{R}^{n'}$, $\boldsymbol{d} \in \mathbb{R}^{m'}$ and random vectors $\mathbf{X}$ in $\mathbb{R}^n$ and $\mathbf{Y}$ in $\mathbb{R}^m$, we have that

$$\operatorname{Cov}[\boldsymbol{A}\mathbf{X} + \boldsymbol{c}, \boldsymbol{B}\mathbf{Y} + \boldsymbol{d}] = \boldsymbol{A} \operatorname{Cov}[\mathbf{X}, \mathbf{Y}] \boldsymbol{B}^\top. \tag{1.29}$$

Two random vectors $\mathbf{X}$ and $\mathbf{Y}$ are said to be *uncorrelated* if and only if $\operatorname{Cov}[\mathbf{X}, \mathbf{Y}] = \mathbf{0}$. Note that if $\mathbf{X}$ and $\mathbf{Y}$ are independent, then Equation (1.21) implies that $\mathbf{X}$ and $\mathbf{Y}$ are uncorrelated. The reverse does not hold in general.

**Remark 1.15: Correlation**

The *correlation* of the random vectors $\mathbf{X}$ and $\mathbf{Y}$ is a normalized covariance,

$$\operatorname{Cor}[\mathbf{X}, \mathbf{Y}](i,j) \doteq \frac{\operatorname{Cov}\left[X_i, Y_j\right]}{\sqrt{\operatorname{Var}[X_i]\operatorname{Var}\left[Y_j\right]}} \in [-1, 1]. \tag{1.30}$$

Two random vectors $\mathbf{X}$ and $\mathbf{Y}$ are therefore uncorrelated if and only if $\operatorname{Cor}[\mathbf{X}, \mathbf{Y}] = \mathbf{0}$.

There is also a nice geometric interpretation of covariance and correlation. For zero mean random variables $X$ and $Y$, $\operatorname{Cov}[X, Y]$ is an inner product.[11]

The cosine of the angle $\theta$ between $X$ and $Y$ (that are not deterministic) coincides with their correlation,

$$\cos\theta = \frac{\operatorname{Cov}[X, Y]}{\|X\| \, \|Y\|} = \operatorname{Cor}[X, Y]. \tag{1.31}$$

$\cos\theta$ is also called a *cosine similarity*. Thus,

$$\theta = \arccos \operatorname{Cor}[X, Y]. \tag{1.32}$$

For example, if $X$ and $Y$ are uncorrelated, then they are orthogonal in the inner product space. If $\operatorname{Cor}[X, Y] = -1$ then $\theta \equiv \pi$ (that is, $X$ and $Y$ "point in opposite directions"), whereas if $\operatorname{Cor}[X, Y] = 1$ then $\theta \equiv 0$ (that is, $X$ and $Y$ "point in the same direction").

[11] That is,

- $\operatorname{Cov}[X, Y]$ is symmetric,
- $\operatorname{Cov}[X, Y]$ is linear (here we use $\mathbb{E}X = \mathbb{E}Y = 0$), and
- $\operatorname{Cov}[X, X] \geq 0$.

using the Euclidean inner product formula, $\operatorname{Cov}[X, Y] = \|X\| \, \|Y\| \cos\theta$

The covariance of a random vector $\mathbf{X}$ in $\mathbb{R}^n$ with itself is called its *variance*:

$$\begin{aligned}
\operatorname{Var}[\mathbf{X}] &\doteq \operatorname{Cov}[\mathbf{X}, \mathbf{X}] && (1.33)\\
&= \mathbb{E}\left[(\mathbf{X} - \mathbb{E}[\mathbf{X}])(\mathbf{X} - \mathbb{E}[\mathbf{X}])^\top\right] && (1.34)\\
&= \mathbb{E}\left[\mathbf{X}\mathbf{X}^\top\right] - \mathbb{E}[\mathbf{X}] \cdot \mathbb{E}[\mathbf{X}]^\top && (1.35)\\
&= \begin{bmatrix} \operatorname{Cov}[X_1, X_1] & \cdots & \operatorname{Cov}[X_1, X_n] \\ \vdots & \ddots & \vdots \\ \operatorname{Cov}[X_n, X_1] & \cdots & \operatorname{Cov}[X_n, X_n] \end{bmatrix}. && (1.36)
\end{aligned}$$

The scalar variance $\operatorname{Var}[X]$ of a random variable $X$ is a measure of uncertainty about the value of $X$ since it measures the average squared deviation from $\mathbb{E}[X]$. We will see that the eigenvalue spectrum of a covariance matrix can serve as a measure of uncertainty in the multivariate setting.[12]

[12] The *multivariate* setting (as opposed to the *univariate* setting) studies the joint distribution of multiple random variables.

**Remark 1.16: Standard deviation**
The length of a random variable $X$ in the inner product space described in Remark 1.15 is called its *standard deviation*,

$$\|X\| = \sqrt{\text{Cov}[X, X]} = \sqrt{\text{Var}[X]} \doteq \sigma[X]. \tag{1.37}$$

That is, the longer a random variable is in the inner product space, the more "uncertain" we are about its value. If a random variable has length 0, then it is deterministic.

The variance of a random vector $\mathbf{X}$ is also called the *covariance matrix* of $\mathbf{X}$ and denoted by $\boldsymbol{\Sigma}_{\mathbf{X}}$ (or $\boldsymbol{\Sigma}$ if the correspondence to $\mathbf{X}$ is clear from context). A covariance matrix is symmetric by definition due to the symmetry of covariance, and is always positive semi-definite (?). **Problem 1.4**

Two useful properties of variance are the following:

- It follows from Equation (1.29) that for any linear map $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ and vector $\boldsymbol{b} \in \mathbb{R}^m$,

$$\text{Var}[\boldsymbol{A}\mathbf{X} + \boldsymbol{b}] = \boldsymbol{A}\text{Var}[\mathbf{X}]\boldsymbol{A}^\top. \tag{1.38}$$

  In particular, $\text{Var}[-\mathbf{X}] = \text{Var}[\mathbf{X}]$.
- It follows from the definition of variance (1.34) that for any two random vectors $\mathbf{X}$ and $\mathbf{Y}$,

$$\text{Var}[\mathbf{X} + \mathbf{Y}] = \text{Var}[\mathbf{X}] + \text{Var}[\mathbf{Y}] + 2\text{Cov}[\mathbf{X}, \mathbf{Y}]. \tag{1.39}$$

  In particular, if $\mathbf{X}$ and $\mathbf{Y}$ are independent then the covariance term vanishes and $\text{Var}[\mathbf{X} + \mathbf{Y}] = \text{Var}[\mathbf{X}] + \text{Var}[\mathbf{Y}]$.

Analogously to conditional probability and conditional expectation, we can also define conditional variance. The *conditional variance* of a random vector $\mathbf{X}$ given another random vector $\mathbf{Y}$ is the random vector

$$\text{Var}[\mathbf{X} \mid \mathbf{Y}] \doteq \mathbb{E}\left[(\mathbf{X} - \mathbb{E}[\mathbf{X} \mid \mathbf{Y}])(\mathbf{X} - \mathbb{E}[\mathbf{X} \mid \mathbf{Y}])^\top \;\middle|\; \mathbf{Y}\right]. \tag{1.40}$$

Intuitively, the conditional variance is the remaining variance when we use $\mathbb{E}[\mathbf{X} \mid \mathbf{Y}]$ to predict $\mathbf{X}$ rather than if we used $\mathbb{E}[\mathbf{X}]$. One can also condition a variance on another random vector, analogously to the laws of total probability (1.12) and expectation (1.25).

**Theorem 1.17** (Law of total variance, LOTV)**.**

$$\text{Var}[\mathbf{X}] = \mathbb{E}_{\mathbf{Y}}[\text{Var}_{\mathbf{X}}[\mathbf{X} \mid \mathbf{Y}]] + \text{Var}_{\mathbf{Y}}[\mathbb{E}_{\mathbf{X}}[\mathbf{X} \mid \mathbf{Y}]]. \tag{1.41}$$

Here, the first term measures the average deviation from the mean of $\mathbf{X}$ across realizations of $\mathbf{Y}$ and the second term measures the uncertainty

in the mean of $\mathbf{X}$ across realizations of $\mathbf{Y}$. In Section 2.2, we will see that both terms have a meaningful characterization in the context of probabilistic inference.

*Proof sketch of LOTV.* To simplify the notation, we present only a proof for the univariate setting.

$$\begin{aligned}
\mathrm{Var}[X] &= \mathbb{E}\Big[X^2\Big] - \mathbb{E}[X]^2 \\
&= \mathbb{E}\Big[\mathbb{E}\Big[X^2 \mid Y\Big]\Big] - \mathbb{E}[\mathbb{E}[X \mid Y]]^2 && \text{by the tower rule (1.25)} \\
&= \mathbb{E}\Big[\mathrm{Var}[X \mid Y] + \mathbb{E}[X \mid Y]^2\Big] - \mathbb{E}[\mathbb{E}[X \mid Y]]^2 && \text{by the definition of variance (1.35)} \\
&= \mathbb{E}[\mathrm{Var}[X \mid Y]] + \Big(\mathbb{E}\Big[\mathbb{E}[X \mid Y]^2\Big] - \mathbb{E}[\mathbb{E}[X \mid Y]]^2\Big) \\
&= \mathbb{E}[\mathrm{Var}[X \mid Y]] + \mathrm{Var}[\mathbb{E}[X \mid Y]]. && \square \quad \text{by the definition of variance (1.35)}
\end{aligned}$$

### 1.1.11 Change of Variables

It is often useful to understand the distribution of a transformed random variable $Y = g(X)$ that is defined in terms of a random variable $X$, whose distribution is known. Let us first consider the univariate setting. We would like to express the distribution of $Y$ in terms of the distribution of $X$, that is, we would like to find

$$P_Y(y) = \mathbb{P}(Y \leq y) = \mathbb{P}(g(X) \leq y) = \mathbb{P}\Big(X \leq g^{-1}(y)\Big). \tag{1.42}$$

When the random variables are continuous, this probability can be expressed as an integration over the domain of $X$. We can then use the substitution rule of integration to "change the variables" to an integration over the domain of $Y$. Taking the derivative yields the density $p_Y$.[13] There is an analogous change of variables formula for the multivariate setting.

[13] The full proof of the change of variables formula in the univariate setting can be found in section 6.7.2 of "Mathematics for machine learning" (Deisenroth et al., 2020).

**Fact 1.18** (Change of variables formula)**.** *Let $\mathbf{X}$ be a random vector in $\mathbb{R}^n$ with density $p_{\mathbf{X}}$ and let $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^n$ be a differentiable and invertible function. Then $\mathbf{Y} = \boldsymbol{g}(\mathbf{X})$ is another random variable, whose density can be computed based on $p_{\mathbf{X}}$ and $\boldsymbol{g}$ as follows:*

$$p_{\mathbf{Y}}(\boldsymbol{y}) = p_{\mathbf{X}}(\boldsymbol{g}^{-1}(\boldsymbol{y})) \cdot \left|\det\Big(\boldsymbol{D}\boldsymbol{g}^{-1}(\boldsymbol{y})\Big)\right| \tag{1.43}$$

*where $\boldsymbol{D}\boldsymbol{g}^{-1}(\boldsymbol{y})$ is the Jacobian of $\boldsymbol{g}^{-1}$ evaluated at $\boldsymbol{y}$.*

Here, the term $\left|\det\big(\boldsymbol{D}\boldsymbol{g}^{-1}(\boldsymbol{y})\big)\right|$ measures how much a unit volume changes when applying $\boldsymbol{g}$. Intuitively, the change of variables swaps the coordinate system over which we integrate. The factor $\left|\det\big(\boldsymbol{D}\boldsymbol{g}^{-1}(\boldsymbol{y})\big)\right|$ corrects for the change in volume that is caused by this change in coordinates.

Intuitively, you can think of the vector field $g$ as a perturbation to $\mathbf{X}$, "pushing" the probability mass around. The perturbation of a density $p_{\mathbf{X}}$ by $g$ is commonly denoted by the *pushforward*

$$g_\sharp p_{\mathbf{X}} \doteq p_{\mathbf{Y}} \quad \text{where } \mathbf{Y} = g(\mathbf{X}). \tag{1.44}$$

This concludes our quick tour of probability theory, and we are well-prepared to return to the topic of probabilistic inference.

## 1.2 *Probabilistic Inference*

Recall the logical implication "If it is raining, the ground is wet." from the beginning of this chapter. Suppose that we look outside a window and see that it is not raining: will the ground be dry? Logical reasoning does not permit drawing an inference of this kind, as there might be reasons other than rain for which the ground could be wet (e.g., sprinklers). However, intuitively, by observing that it is not raining, we have just excluded the possibility that the ground is wet because of rain, and therefore we would deem it "more likely" that the ground is dry than before. In other words, if we were to walk outside now and the ground was wet, we would be more surprised than we would have been if we had not looked outside the window before.

As humans, we are constantly making such "plausible" inferences of our beliefs: be it about the weather, the outcomes of our daily decisions, or the behavior of others. *Probabilistic inference* is the process of updating such a prior belief $\mathbb{P}(\overline{W})$ to a posterior belief $\mathbb{P}(\overline{W} \mid \overline{R})$ upon observing $\overline{R}$ where — to reduce clutter — we write $W$ for "The ground is wet" and $R$ for "It is raining".

The central principle of probabilistic inference is Bayes' rule:

**Theorem 1.19** (Bayes' rule)**.** *Given random vectors* $\mathbf{X}$ *in* $\mathbb{R}^n$ *and* $\mathbf{Y}$ *in* $\mathbb{R}^m$*, we have for any* $\boldsymbol{x} \in \mathbb{R}^n, \boldsymbol{y} \in \mathbb{R}^m$ *that*

$$p(\boldsymbol{x} \mid \boldsymbol{y}) = \frac{p(\boldsymbol{y} \mid \boldsymbol{x}) \cdot p(\boldsymbol{x})}{p(\boldsymbol{y})}. \tag{1.45}$$

*Proof.* Bayes' rule is a direct consequence of the definition of conditional densities (1.10) and the product rule (1.11). □

Let us consider the meaning of each term separately:

- the *prior* $p(\boldsymbol{x})$ is the initial belief about $\boldsymbol{x}$,
- the *(conditional) likelihood* $p(\boldsymbol{y} \mid \boldsymbol{x})$ describes how likely the observations $\boldsymbol{y}$ are under a given value $\boldsymbol{x}$,
- the *posterior* $p(\boldsymbol{x} \mid \boldsymbol{y})$ is the updated belief about $\boldsymbol{x}$ after observing $\boldsymbol{y}$,

- the *joint likelihood* $p(\boldsymbol{x}, \boldsymbol{y}) = p(\boldsymbol{y} \mid \boldsymbol{x})p(\boldsymbol{x})$ combines prior and likelihood,
- the *marginal likelihood* $p(\boldsymbol{y})$ describes how likely the observations $\boldsymbol{y}$ are across all values of $\boldsymbol{x}$.

The marginal likelihood can be computed using the sum rule (1.7) or the law of total probability (1.12),

$$p(\boldsymbol{y}) = \int_{\boldsymbol{X}(\Omega)} p(\boldsymbol{y} \mid \boldsymbol{x}) \cdot p(\boldsymbol{x})\, d\boldsymbol{x}. \tag{1.46}$$

Note, however, that the marginal likelihood is simply normalizing the conditional distribution to integrate to one, and therefore a constant with respect to $\boldsymbol{x}$. For this reason, $p(\boldsymbol{y})$ is commonly called the *normalizing constant*.

**Example 1.20: Plausible inferences**

Let us confirm our intuition from the above example. The logical implication "If it is raining, the ground is wet." (denoted $R \to W$) can be succinctly expressed as $\mathbb{P}(W \mid R) = 1$. Since $\mathbb{P}(W) \leq 1$, we know that

$$\mathbb{P}(R \mid W) = \frac{\mathbb{P}(W \mid R) \cdot \mathbb{P}(R)}{\mathbb{P}(W)} = \frac{\mathbb{P}(R)}{\mathbb{P}(W)} \geq \mathbb{P}(R).$$

That is, observing that the ground is wet makes it more likely to be raining. From $\mathbb{P}(R \mid W) \geq \mathbb{P}(R)$ we know $\mathbb{P}\big(\overline{R} \mid W\big) \leq \mathbb{P}\big(\overline{R}\big)$,[14] which leads us to follow that

[14] since $\mathbb{P}(\overline{X}) = 1 - \mathbb{P}(X)$

$$\mathbb{P}\big(W \mid \overline{R}\big) = \frac{\mathbb{P}\big(\overline{R} \mid W\big) \cdot \mathbb{P}(W)}{\mathbb{P}\big(\overline{R}\big)} \leq \mathbb{P}(W),$$

that is, having observed it not to be raining made the ground less likely to be wet.

Example 1.20 is called a *plausible inference* because the observation of $\overline{R}$ does not completely determine the truth value of $\overline{W}$, and hence, does not permit logical inference. In the case, however, that logical inference is permitted, it coincides with probabilistic inference.

**Example 1.21: Logical inferences**

For example, if we were to observe that the ground is not wet, then logical inference implies that it must not be raining: $\overline{W} \to \overline{R}$. This is called the *contrapositive* of $R \to W$.

Indeed, by probabilistic inference, we obtain analogously

$$\mathbb{P}\big(R \mid \overline{W}\big) = \frac{\mathbb{P}\big(\overline{W} \mid R\big) \cdot \mathbb{P}(R)}{\mathbb{P}\big(\overline{W}\big)} = \frac{(1 - \mathbb{P}(W \mid R)) \cdot \mathbb{P}(R)}{\mathbb{P}\big(\overline{W}\big)} = 0.$$

as $\mathbb{P}(W \mid R) = 1$

Observe that a logical inference does not depend on the prior $\mathbb{P}(R)$: Even if the prior was $\mathbb{P}(R) = 1$ in Example 1.21, after observing that the ground is not wet, we are forced to conclude that it is not raining to maintain logical consistency. The examples highlight that while *logical* inference does not require the notion of a prior, plausible (*probabilistic*!) inference does.

### 1.2.1 *Where do priors come from?*

Bayes' rule necessitates the specification of a prior $p(\boldsymbol{x})$. Different priors can lead to the deduction of dramatically different posteriors, as one can easily see by considering the extreme cases of a prior that is a point density at $\boldsymbol{x} = \boldsymbol{x}_0$ and a prior that is "uniform" over $\mathbb{R}^n$.[15] In the former case, the posterior will be a point density at $\boldsymbol{x}_0$ regardless of the likelihood. In other words, no evidence can alter the "prior belief" the learner ascribed to $\boldsymbol{x}$. In the latter case, the learner has "no prior belief", and therefore the posterior will be proportional to the likelihood. Both steps of probabilistic inference are perfectly valid, though one might debate which prior is more reasonable.

[15] The latter is not a valid probability distribution, but we can still derive meaning from the posterior as we discuss in Remark 1.23.

Someone who follows the Bayesian interpretation of probability might argue that everything is conditional, meaning that the prior is simply a posterior of all former observations. While this might seem natural ("my world view from today is the combination of my world view from yesterday and the observations I made today"), this lacks an explanation for "the first day". Someone else who is more inclined towards the frequentist interpretation might also object to the existence of a prior belief altogether, arguing that a prior is *subjective* and therefore not a valid or desirable input to a learning algorithm. Put differently, a frequentist "has the belief not to have any belief". This is perfectly compatible with probabilistic inference, as long as the prior is chosen to be *noninformative*:

$$p(\boldsymbol{x}) \propto \text{const.} \tag{1.47}$$

Choosing a noninformative prior in the absence of any evidence is known as the *principle of indifference* or the *principle of insufficient reason*, which dates back to the famous mathematician Pierre-Simon Laplace.

**Example 1.22: Why be indifferent?**

Consider a criminal trial with three suspects, A, B, and C. The collected evidence shows that suspect C can not have committed the crime, however it does not yield any information about suspects A and B. Clearly, any distribution respecting the data must assign zero probability of having committed the crime to suspect C. However, any distribution interpolating between $(1, 0, 0)$ and $(0, 1, 0)$ respects the data. The principle of indifference suggests that the desired distribution is $(\frac{1}{2}, \frac{1}{2}, 0)$, and indeed, any alternative distribution seems unreasonable.

**Remark 1.23: Noninformative and improper priors**

It is not necessarily required that the prior $p(\boldsymbol{x})$ is a valid distribution (i.e., integrates to 1). Consider for example, the noninformative prior $p(\boldsymbol{x}) \propto \mathbb{1}\{\boldsymbol{x} \in I\}$ where $I \subseteq \mathbb{R}^n$ is an infinitely large interval. Such a prior which is not a valid distribution is called an *improper prior*. We can still derive meaning from the posterior of a given likelihood and (improper) prior as long as the posterior is a valid distribution.

Laplace's principle of indifference can be generalized to cases where *some* evidence is available. The *maximum entropy principle*, originally proposed by Jaynes (1968), states that one should choose as prior from all possible distributions that are *consistent* with prior knowledge, the one that makes the *least* "additional assumptions", i.e., is the least "informative". In philosophy, this principle is known as *Occam's razor* or the *principle of parsimony*. The "informativeness" of a distribution $p$ is quantified by its *entropy* which is defined as

$$\mathrm{H}[p] \doteq \mathbb{E}_{\boldsymbol{x} \sim p}[-\log p(\boldsymbol{x})]. \tag{1.48}$$

The more concentrated $p$ is, the less is its entropy; the more diffuse $p$ is, the greater is its entropy.[16]

In the absence of any prior knowledge, the uniform distribution has the highest entropy,[17] and hence, the maximum entropy principle suggests a noninformative prior (as does Laplace's principle of indifference). In contrast, if the evidence perfectly determines the value of $\boldsymbol{x}$, then the only consistent explanation is the point density at $\boldsymbol{x}$. The maximum entropy principle characterizes a reasonable choice of prior for these two extreme cases and all cases in between. Bayes' rule can in fact be derived as a consequence of the maximum entropy principle in the sense that the posterior is the least "informative" distribution among all distributions that are consistent with the prior and the ob-

[16] We give a thorough introduction to entropy in Section 5.4.

[17] This only holds true when the set of possible outcomes of $\boldsymbol{x}$ finite (or a bounded continuous interval), as in this case, the noninformative prior is a proper distribution — the uniform distribution. In the "infinite case", there is no uniform distribution and the noninformative prior can be attained from the maximum entropy principle as the limiting solution as the number of possible outcomes of $\boldsymbol{x}$ is increased.

servations (?).

**Problem 5.7**

### 1.2.2 *Conjugate Priors*

If the prior $p(\boldsymbol{x})$ and posterior $p(\boldsymbol{x} \mid \boldsymbol{y})$ are of the same family of distributions, the prior is called a *conjugate prior* to the likelihood $p(\boldsymbol{y} \mid \boldsymbol{x})$. This is a very desirable property, as it allows us to recursively apply the same learning algorithm implementing probabilistic inference. We will see in Chapter 2 that under some conditions the Gaussian is *self-conjugate*. That is, if we have a Gaussian prior and a Gaussian likelihood then our posterior will also be Gaussian. This will provide us with the first *efficient* implementation of probabilistic inference.

**Example 1.24: Conjugacy of beta and binomial distribution**

As an example for conjugacy, we will show that the beta distribution is a conjugate prior to a binomial likelihood. Recall the PMF of the binomial distribution

$$\text{Bin}(k; n, \theta) = \binom{n}{k} \theta^k (1-\theta)^{n-k} \tag{1.49}$$

and the PDF of the beta distribution,

$$\text{Beta}(\theta; \alpha, \beta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}, \tag{1.50}$$

We assume the prior $\theta \sim \text{Beta}(\alpha, \beta)$ and likelihood $k \mid \theta \sim \text{Bin}(n, \theta)$. Let $n_H = k$ be the number of heads and $n_T = n - k$ the number of tails in the binomial trial $k$. Then,

$$\begin{aligned} p(\theta \mid k) &\propto p(k \mid \theta) p(\theta) && \text{using Bayes' rule (1.45)} \\ &\propto \theta^{n_H}(1-\theta)^{n_T} \theta^{\alpha-1}(1-\theta)^{\beta-1} \\ &= \theta^{\alpha+n_H-1}(1-\theta)^{\beta+n_T-1}. \end{aligned}$$

Thus, $\theta \mid k \sim \text{Beta}(\alpha + n_H, \beta + n_T)$.

This same conjugacy can be shown for the multivariate generalization of the beta distribution, the *Dirichlet distribution*, and the multivariate generalization of the binomial distribution, the *multinomial distribution*.

### 1.2.3 *Tractable Inference with the Normal Distribution*

Using arbitrary distributions for learning and inference is computationally very expensive when the number of dimensions is large — even in the discrete setting. For example, computing marginal distributions using the sum rule yields an exponentially long sum in the

size of the random vector. Similarly, the normalizing constant of the conditional distribution is a sum of exponential length. Even to represent any discrete joint probability distribution requires space that is exponential in the number of dimensions (cf. Figure 1.5).

| $X_1$ | $\cdots$ | $X_{n-1}$ | $X_n$ | $\mathbb{P}(X_{1:n})$ |
|---|---|---|---|---|
| 0 | $\cdots$ | 0 | 0 | 0.01 |
| 0 | $\cdots$ | 0 | 1 | 0.001 |
| 0 | $\cdots$ | 1 | 0 | 0.213 |
| $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | |
| 1 | $\cdots$ | 1 | 1 | 0.0003 |

Figure 1.5: A table representing a joint distribution of $n$ binary random variables. The table has $2^n$ rows. The number of parameters is $2^n - 1$ since the final probability is determined by all other probabilities as they must sum to one.

One strategy to get around this computational blowup is to restrict the class of distributions. Gaussians are a popular choice for this purpose since they have extremely useful properties: they have a compact representation and — as we will see in Chapter 2 — they allow for closed-form probabilistic inference.

In Equation (1.5), we have already seen the PDF of the univariate Gaussian distribution. A random vector $\mathbf{X}$ in $\mathbb{R}^n$ is *normally distributed*, $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, if its PDF is

$$\mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \doteq \frac{1}{\sqrt{\det(2\pi\boldsymbol{\Sigma})}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right) \tag{1.51}$$

where $\boldsymbol{\mu} \in \mathbb{R}^n$ is the mean vector and $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$ the covariance matrix (?). We call $\boldsymbol{\Lambda} \doteq \boldsymbol{\Sigma}^{-1}$ the *precision matrix*. $\mathbf{X}$ is also called a *Gaussian random vector* (GRV). $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ is the multivariate *standard normal distribution*. We call a Gaussian *isotropic* if its covariance matrix is of the form $\boldsymbol{\Sigma} = \sigma^2 \boldsymbol{I}$ for some $\sigma^2 \in \mathbb{R}$. In this case, the sublevel sets of the PDF are perfect spheres as can be seen in Figure 1.6.

**Problem 1.11**

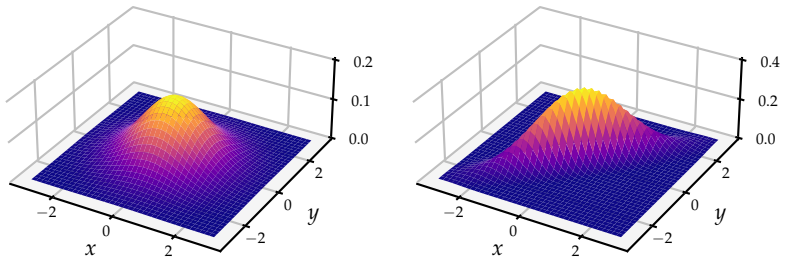



Figure 1.6: Shown are the PDFs of two-dimensional Gaussians with mean $\mathbf{0}$ and covariance matrices

$$\boldsymbol{\Sigma}_1 \doteq \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \boldsymbol{\Sigma}_2 \doteq \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}$$

respectively.

Note that a Gaussian can be represented using only $O(n^2)$ parameters. In the case of a diagonal covariance matrix, which corresponds to $n$ independent univariate Gaussians (?), we just need $O(n)$ parameters.

**Problem 1.8**

In Equation (1.51), we assume that the covariance matrix $\boldsymbol{\Sigma}$ is invertible, i.e., does not have the eigenvalue 0. This is not a restriction since it can be shown that a covariance matrix has a zero eigenvalue if and only if there exists a deterministic linear relationship between some variables in the joint distribution (?). As we have already seen that a

**Problem 1.6**

covariance matrix does not have negative eigenvalues (?), this ensures that $\boldsymbol{\Sigma}$ and $\boldsymbol{\Lambda}$ are positive definite.[18]

**Problem 1.4**

[18] The inverse of a positive definite matrix is also positive definite.

An important property of the normal distribution is that it is closed under marginalization and conditioning.

**Theorem 1.25** (Marginal and conditional distribution)**.** (?) *Consider the Gaussian random vector* $\mathbf{X}$ *and fix index sets* $A \subseteq [n]$ *and* $B \subseteq [n]$*. Then, we have that for any such* marginal distribution*,*

$$\mathbf{X}_A \sim \mathcal{N}(\boldsymbol{\mu}_A, \boldsymbol{\Sigma}_{AA}), \tag{1.52}$$

*and that for any such* conditional distribution*,*

$$\mathbf{X}_A \mid \mathbf{X}_B = \boldsymbol{x}_B \sim \mathcal{N}(\boldsymbol{\mu}_{A|B}, \boldsymbol{\Sigma}_{A|B}) \quad \textit{where} \tag{1.53a}$$

$$\boldsymbol{\mu}_{A|B} \doteq \boldsymbol{\mu}_A + \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}(\boldsymbol{x}_B - \boldsymbol{\mu}_B), \tag{1.53b}$$

$$\boldsymbol{\Sigma}_{A|B} \doteq \boldsymbol{\Sigma}_{AA} - \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}\boldsymbol{\Sigma}_{BA}. \tag{1.53c}$$

**Problem 1.9**

By $\boldsymbol{\mu}_A$ we denote $[\mu_{i_1}, \ldots, \mu_{i_k}]$ where $A = \{i_1, \ldots i_k\}$. $\boldsymbol{\Sigma}_{AA}$ is defined analogously.

Here, $\boldsymbol{\mu}_A$ characterizes the prior belief and $\boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}(\boldsymbol{x}_B - \boldsymbol{\mu}_B)$ represents "how different" $\boldsymbol{x}_B$ is from what was expected.

Theorem 1.25 provides a closed-form characterization of probabilistic inference for the case that random variables are jointly Gaussian. We will discuss in Chapter 2, how this can be turned into an efficient inference algorithm.

Observe that upon inference, the variance can only shrink! Moreover, how much the variance is reduced depends purely on *where* the observations are made (i.e., the choice of $B$) but not on *what* the observations are. In contrast, the posterior mean $\boldsymbol{\mu}_{A|B}$ depends affinely on $\boldsymbol{\mu}_B$. These are special properties of the Gaussian and do not generally hold true for other distributions.

It can be shown that Gaussians are additive and closed under affine transformations (?). The closedness under affine transformations (1.78) implies that a Gaussian $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is equivalently characterized as

**Problem 1.10**

$$\mathbf{X} = \boldsymbol{\Sigma}^{1/2}\mathbf{Y} + \boldsymbol{\mu}. \tag{1.54}$$

where $\mathbf{Y} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and $\boldsymbol{\Sigma}^{1/2}$ is the square root of $\boldsymbol{\Sigma}$.[19] Importantly, this implies together with Theorem 1.25 and additivity (1.79) that:

[19] More details on the square root of a symmetric and positive definite matrix can be found in Section A.2.

*Any affine transformation of a Gaussian random vector is a Gaussian random vector.*

A consequence of this is that given any jointly Gaussian random vectors $\mathbf{X}_A$ and $\mathbf{X}_B$, $\mathbf{X}_A$ can be expressed as an affine function of $\mathbf{X}_B$ with added independent Gaussian noise. Formally, we define

$$\mathbf{X}_A \doteq \boldsymbol{A}\mathbf{X}_B + \boldsymbol{b} + \boldsymbol{\varepsilon} \quad \text{where} \tag{1.55a}$$

$$\boldsymbol{A} \doteq \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}, \tag{1.55b}$$
$$\boldsymbol{b} \doteq \boldsymbol{\mu}_A - \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_{BB}^{-1}\boldsymbol{\mu}_B, \tag{1.55c}$$
$$\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_{A|B}). \tag{1.55d}$$

It directly follows from the closedness of Gaussians under affine transformations (1.78) that the characterization of $\mathbf{X}_A$ via Equation (1.55d) is equivalent to $\mathbf{X}_A \sim \mathcal{N}(\boldsymbol{\mu}_A, \boldsymbol{\Sigma}_{AA})$, and hence, *any* Gaussian $\mathbf{X}_A$ can be modeled as a so-called *conditional linear Gaussian*, i.e., an affine function of another Gaussian $\mathbf{X}_B$ with additional independent Gaussian noise. We will use this fact frequently to represent Gaussians in a compact form.

## 1.3 Supervised Learning and Point Estimates

Throughout the first part of this manuscript, we will focus mostly on the *supervised learning* problem where we want to learn a function

$$f^\star : \mathcal{X} \to \mathcal{Y}$$

from labeled training data. That is, we are given a collection of labeled examples, $\mathcal{D}_n \doteq \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, where the $\boldsymbol{x}_i \in \mathcal{X}$ are *inputs* and the $y_i \in \mathcal{Y}$ are *outputs* (called *labels*), and we want to find a function $\hat{f}$ that best-approximates $f^\star$. It is common to choose $\hat{f}$ from a parameterized *function class* $\mathcal{F}(\Theta)$, where each function $f_{\boldsymbol{\theta}}$ is described by some parameters $\boldsymbol{\theta} \in \Theta$.

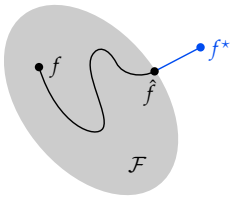



Figure 1.7: Illustration of **estimation error** and **approximation error**. $f^\star$ denotes the true function and $\hat{f}$ is the best approximation from the function class $\mathcal{F}$. We do not specify here, how one could quantify "error". For more details, see Section A.3.5.

**Remark 1.26: What this manuscript is about and not about**

As illustrated in Figure 1.7, the restriction to a function class leads to two sources of error: the *estimation error* of having "incorrectly" determined $\hat{f}$ within the function class, and the *approximation error* of the function class itself. Choosing a "good" function class / architecture with small approximation error is therefore critical for any practical application of machine learning. We will discuss various function classes, from linear models to deep neural networks, however, determining the "right" function class will not be the focus of this manuscript. To keep the exposition simple, we will assume in the following that $f^\star \in \mathcal{F}(\Theta)$ with parameters $\boldsymbol{\theta}^\star \in \Theta$.

Instead, we will focus on the problem of estimation/inference within a given function class. We will see that inference in smaller function classes is often more computationally efficient since the search space is smaller or — in the case of Gaussians — has a known tractable structure. On the other hand, larger function

classes are more expressive and therefore can typically better approximate the ground truth $f^\star$.

We differentiate between the task of *regression* where $\mathcal{Y} \doteq \mathbb{R}^k$,[20] and the task of *classification* where $\mathcal{Y} \doteq \mathcal{C}$ and $\mathcal{C}$ is an $m$-element set of classes. In other words, regression is the task of predicting a continuous label, whereas classification is the task of predicting a discrete class label. These two tasks are intimately related: in fact, we can think of classification tasks as a regression problem where we learn a probability distribution over class labels. In this regression problem, $\mathcal{Y} \doteq \Delta^{\mathcal{C}}$ where $\Delta^{\mathcal{C}}$ denotes the set of all probability distributions over the set of classes $\mathcal{C}$ which is an $(m-1)$-dimensional convex polytope in the $m$-dimensional space of probabilities $[0,1]^m$ (cf. Section A.1.2).

[20] The labels are usually scalar, so $k = 1$.

For now, let us stick to the regression setting. We will assume that the observations are noisy, that is, $y_i \overset{\text{iid}}{\sim} p(\cdot \mid \boldsymbol{x}_i, \boldsymbol{\theta}^\star)$ for some *known* conditional distribution $p(\cdot \mid \boldsymbol{x}_i, \boldsymbol{\theta})$ but *unknown* parameter $\boldsymbol{\theta}^\star$.[21] Our assumption can equivalently be formulated as

$$y_i = \underbrace{f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)}_{\text{signal}} + \underbrace{\varepsilon_i(\boldsymbol{x}_i)}_{\text{noise}} \tag{1.56}$$

[21] The case where the labels are deterministic is the special case of $p(\cdot \mid \boldsymbol{x}_i, \boldsymbol{\theta}^\star)$ being a point density at $f^\star(\boldsymbol{x}_i)$.

where $f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$ is the mean of $p(\cdot \mid \boldsymbol{x}_i, \boldsymbol{\theta})$ and $\varepsilon_i(\boldsymbol{x}_i) = y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$ is some independent zero-mean noise, for example (but not necessarily) Gaussian.[22] When the noise distribution may depend on $\boldsymbol{x}_i$, the noise is said to be *heteroscedastic* and otherwise the noise is called *homoscedastic*.

[22] It is crucial that the assumed noise distribution accurately reflects the noise of the data. For example, using a (light-tailed) Gaussian noise model in the presence of heavy-tailed noise will fail! We discuss the distinction between light and heavy tails in Section A.3.2.

### 1.3.1 Maximum Likelihood Estimation

A common approach to finding $\hat{f}$ is to select the model $f \in \mathcal{F}(\Theta)$ under which the training data is most likely. This is called the *maximum likelihood estimate* (or MLE):

$$\begin{aligned} \hat{\boldsymbol{\theta}}_{\text{MLE}} &\doteq \arg\max_{\boldsymbol{\theta} \in \Theta} p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}) && (1.57) \\ &= \arg\max_{\boldsymbol{\theta} \in \Theta} \prod_{i=1}^{n} p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}). && \text{using the independence of the training data (1.56)} \end{aligned}$$

Such products of probabilities are often numerically unstable, which is why one typically takes the logarithm:

$$= \arg\max_{\boldsymbol{\theta} \in \Theta} \underbrace{\sum_{i=1}^{n} \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta})}_{\textit{log-likelihood}}. \tag{1.58}$$

We will denote the *negative log-likelihood* by $\ell_{\text{nll}}(\boldsymbol{\theta}; \mathcal{D}_n)$.

The MLE is often used in practice due to its desirable asymptotic properties as the sample size $n$ increases. We give a brief summary here and provide additional background and definitions in Section A.3. To give any guarantees on the convergence of the MLE, we necessarily need to assume that $\boldsymbol{\theta}^\star$ is identifiable.[23] If additionally, $\ell_{\text{nll}}$ is "well-behaved" then standard results say that the MLE is *consistent* and *asymptotically normal* (Van der Vaart, 2000):

$$\hat{\boldsymbol{\theta}}_{\text{MLE}} \xrightarrow{\mathbb{P}} \boldsymbol{\theta}^\star \quad \text{and} \quad \hat{\boldsymbol{\theta}}_{\text{MLE}} \xrightarrow{\mathcal{D}} \mathcal{N}(\boldsymbol{\theta}^\star, \boldsymbol{S}_n) \quad \text{as } n \to \infty. \tag{1.59}$$

[23] That is, $\boldsymbol{\theta}^\star \neq \boldsymbol{\theta} \implies f^\star \neq f_{\boldsymbol{\theta}}$ for any $\boldsymbol{\theta} \in \Theta$. In words, there is no other parameter $\boldsymbol{\theta}$ that yields the same function $f_{\boldsymbol{\theta}}$ as $\boldsymbol{\theta}^\star$.

Here, we denote by $\boldsymbol{S}_n$ the asymptotic variance of the MLE which can be understood as measuring the "quality" of the estimate.[24] This implies in some sense that the MLE is asymptotically unbiased. Moreover, the MLE can be shown to be *asymptotically efficient* which is to say that there exists no other consistent estimator with a "smaller" asymptotic variance.[25]

[24] A "smaller" variance means that we can be more confident that the MLE is close to the true parameter.

[25] see Section A.3.4.

The situation is quite different in the finite sample regime. Here, the MLE need not be unbiased, and it is susceptible to *overfitting* to the (finite) training data as we discuss in more detail in Section A.3.5.

### 1.3.2 *Using Priors: Maximum a Posteriori Estimation*

We can incorporate prior assumptions about the parameters $\boldsymbol{\theta}^\star$ into the estimation procedure. One approach of this kind is to find the mode of the posterior distribution, called the *maximum a posteriori estimate* (or MAP estimate):

$$\begin{aligned}
\hat{\boldsymbol{\theta}}_{\text{MAP}} &\doteq \arg\max_{\boldsymbol{\theta} \in \Theta} p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) && (1.60) \\
&= \arg\max_{\boldsymbol{\theta} \in \Theta} p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}) \cdot p(\boldsymbol{\theta}) && (1.61) && \text{by Bayes' rule (1.45)} \\
&= \arg\max_{\boldsymbol{\theta} \in \Theta} \log p(\boldsymbol{\theta}) + \sum_{i=1}^{n} \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}) && (1.62) && \text{taking the logarithm} \\
&= \arg\min_{\boldsymbol{\theta} \in \Theta} \underbrace{-\log p(\boldsymbol{\theta})}_{\text{regularization}} + \underbrace{\ell_{\text{nll}}(\boldsymbol{\theta}; \mathcal{D}_n)}_{\text{quality of fit}}. && (1.63)
\end{aligned}$$

Here, the *log-prior* $\log p(\boldsymbol{\theta})$ acts as a regularizer. Common regularizers are given, for example, by

- $p(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, (2\lambda)^{-1}\boldsymbol{I})$ which yields $-\log p(\boldsymbol{\theta}) = \lambda \left\|\boldsymbol{\theta}\right\|_2^2 + \text{const}$,
- $p(\boldsymbol{\theta}) = \text{Laplace}(\boldsymbol{\theta}; \mathbf{0}, \lambda^{-1})$ which yields $-\log p(\boldsymbol{\theta}) = \lambda \left\|\boldsymbol{\theta}\right\|_1 + \text{const}$,
- a uniform prior (cf. Section 1.2.1) for which the MAP is equivalent to the MLE. In other words, the MLE is merely the mode of the posterior distribution under a uniform prior.

The Gaussian and Laplace regularizers act as simplicity biases, preferring simpler models over more complex ones, which empirically

tends to reduce the risk of overfitting. However, one may also encode more nuanced information about the (assumed) structure of $\boldsymbol{\theta}^\star$ into the prior.

An alternative way of encoding a prior is by restricting the function class to some $\widetilde{\Theta} \subset \Theta$, for example to rotation- and translation-invariant models as often done when the inputs are images. This effectively sets $p(\boldsymbol{\theta}) = 0$ for all $\boldsymbol{\theta} \in \Theta \setminus \widetilde{\Theta}$ but is better suited for numerical optimization than to impose this constraint directly on the prior.

Encoding prior assumptions into the function class or into the parameter estimation can accelerate learning and improve generalization performance dramatically, yet importantly, incorporating a prior can also inhibit learning in case the prior is "wrong". For example, when the learning task is to differentiate images of cats from images of dogs, consider the (stupid) prior that only permits models that exclusively use the upper-left pixel for prediction. No such model will be able to solve the task, and therefore starting from this prior makes the learning problem effectively unsolvable which illustrates that priors have to be chosen with care.

### *1.3.3 When does the prior matter?*

We have seen that the MLE has desirable asymptotic properties, and that MAP estimation can be seen as a regularized MLE where the type of regularization is encoded by the prior. Is it possible to derive similar asymptotic results for the MAP estimate?

To answer this question, we will look at the asymptotic effect of the prior on the posterior more generally. *Doob's consistency theorem* states that assuming parameters are identifiable,[26] there exists $\widetilde{\Theta} \subseteq \Theta$ with $p(\widetilde{\Theta}) = 1$ such that the posterior is consistent for any $\boldsymbol{\theta}^\star \in \widetilde{\Theta}$ (Doob, 1949; Miller, 2016, 2018):

$$\boldsymbol{\theta} \mid \mathcal{D}_n \xrightarrow{\mathbb{P}} \boldsymbol{\theta}^\star \qquad \text{as } n \to \infty. \tag{1.64}$$

In words, Doob's consistency theorem tells us that for *any* prior distribution, the posterior is guaranteed to converge to a point density in the (small) neighborhood $\boldsymbol{\theta}^\star \in B$ of the true parameter as long as $p(B) > 0$.[27] We call such a prior a *well-specified prior*.

[26] This is akin to the assumption required for consistency of the MLE, cf. Equation (1.59).

[27] $B$ can for example be a ball of radius $\epsilon$ around $\boldsymbol{\theta}^\star$ (with respect to some geometry of $\Theta$).

**Remark 1.27: Cromwell's rule**

In the case where $|\Theta|$ is finite, Doob's consistency theorem strongly suggests that the prior should not assign 0 probability (or probability 1 for that matter) to any individual parameter $\boldsymbol{\theta} \in \Theta$, unless we know with certainty that $\boldsymbol{\theta}^\star \neq \boldsymbol{\theta}$. This is called *Cromwell's rule*,

and a prior obeying by this rule is always well-specified.

Under the same assumption that the prior is well-specified (and regularity conditions[28]), the *Bernstein-von Mises theorem*, which was first discovered by Pierre-Simon Laplace in the early 19th century, establishes the asymptotic normality of the posterior distribution (Van der Vaart, 2000; Miller, 2016):

$$\boldsymbol{\theta} \mid \mathcal{D}_n \xrightarrow{\mathcal{D}} \mathcal{N}(\boldsymbol{\theta}^\star, \boldsymbol{S}_n) \qquad \text{as } n \to \infty \tag{1.65}$$

and where $\boldsymbol{S}_n$ is the same as the asymptotic variance of the MLE.[29]

These results link probabilistic inference to maximum likelihood estimation in the asymptotic limit of infinite data. Intuitively, in the limit of infinite data, the prior is "overwhelmed" by the observations and the posterior becomes equivalent to the limiting distribution of the MLE.[30] One can interpret the regime of infinite data as the regime where computational resources and time are unlimited and plausible inferences evolve into logical inferences. This transition signifies a shift from the realm of uncertainty to that of certainty. The importance of the prior surfaces precisely in the non-asymptotic regime where plausible inferences are necessary due to *limited computational resources and limited time*.

[28] These regularity conditions are akin to the assumptions required for asymptotic normality of the MLE, cf. Equation (1.59).

[29] This has also been called the "Bayesian central limit theorem", which is a bit of a misnomer since the theorem also applies to likelihoods (when the prior is noninformative) which are often used in frequentist statistics.

[30] More examples and discussion can be found in section 17.8 of Le Cam (1986), chapter 8 of Le Cam and Yang (2000), chapter 10 of Van der Vaart (2000), and in Tanner (1991).

### 1.3.4 Estimation vs Inference

You can interpret a single parameter vector $\boldsymbol{\theta} \in \Theta$ as "one possible explanation" of the data. Maximum likelihood and maximum a posteriori estimation are examples of *estimation* algorithms which return a one such parameter vector — called a *point estimate*. That is, given the training set $\mathcal{D}_n$, they return a single parameter vector $\hat{\boldsymbol{\theta}}_n$. We give a more detailed account of estimation in Section A.3.

**Example 1.28: Point estimates and invalid logical inferences**

To see why point estimates can be problematic, recall Example 1.20. We have seen that the logical implication

"If it is raining, the ground is wet."

can be expressed as $\mathbb{P}(W \mid R) = 1$. Observing that "The ground is wet." does not permit logical inference, yet, the maximum likelihood estimate of $R$ is $\hat{R}_{\text{MLE}} = 1$. This is logically inconsistent since there might be other *explanations* for the ground to be wet, such as a sprinkler! With only a finite sample (say independently observing $n$ times that the ground is wet), we cannot rule out with

certainty that the ground is wet for other reasons than rain.

In practice, we never observe an infinite amount of data. Example 1.28 demonstrates that on a finite sample, point estimates may perform invalid logical inferences, and can therefore lure us into a false sense of certainty.

**Remark 1.29: MLE and MAP are approximations of inference**

The MLE and MAP estimate can be seen as a naive approximation of probabilistic inference, represented by a point density which "collapses" all probability mass at the mode of the posterior distribution. This can be a relatively decent — even if overly simple — approximation when the distribution is unimodal, symmetric, and light-tailed as in Figure 1.8, but is usually a very poor approximation for practical posteriors that are complex and multimodal.

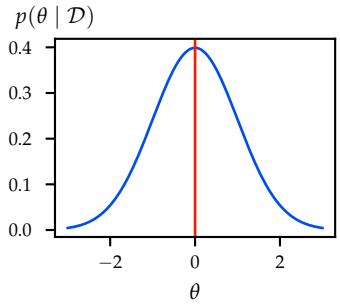



Figure 1.8: A the MLE/MAP are point estimates at the mode $\hat{\theta}$ of the posterior distribution $p(\theta \mid \mathcal{D})$.

In this manuscript, we will focus mainly on algorithms for *probabilistic inference* which compute or approximate the distribution $p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n})$ over parameters. Returning a distribution over parameters is natural since this acknowledges that given a finite sample with noisy observations, more than one parameter vector can explain the data.

### 1.3.5 Probabilistic Inference and Prediction

The prior distribution $p(\boldsymbol{\theta})$ can be interpreted as the degree of our belief that the model parameterized by $\boldsymbol{\theta}$ "describes the (previously seen) data best". The likelihood captures how likely the training data is under a particular model:

$$p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}) = \prod_{i=1}^{n} p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}). \tag{1.66}$$

The posterior then represents our belief about the best model after seeing the training data. Using Bayes' rule (1.45), we can write it as[31]

$$p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) = \frac{1}{Z} p(\boldsymbol{\theta}) \prod_{i=1}^{n} p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}) \quad \text{where} \tag{1.67a}$$

$$Z \doteq \int_{\Theta} p(\boldsymbol{\theta}) \prod_{i=1}^{n} p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}) \, d\boldsymbol{\theta} \tag{1.67b}$$

is the *normalizing constant*. We refer to this process of learning a model from data as *learning*. We can then use our learned model for *prediction* at a new input $\boldsymbol{x}^\star$ by conditioning on $\boldsymbol{\theta}$,

$$p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}) = \int_{\Theta} p(y^\star, \boldsymbol{\theta} \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}) \, d\boldsymbol{\theta}$$

by the sum rule (1.7)

[31] We generally assume that

$$p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}) = p(\boldsymbol{\theta}).$$

For our purposes, you can think of the inputs $\boldsymbol{x}_{1:n}$ as fixed deterministic parameters, but one can also consider inputs drawn from a distribution over $\mathcal{X}$.

$$= \int_\Theta p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}) \cdot p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) \, d\boldsymbol{\theta}. \tag{1.68}$$

by the product rule (1.11) and $y^\star \perp \boldsymbol{x}_{1:n}, y_{1:n} \mid \boldsymbol{\theta}$

Here, the distribution over models $p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n})$ is called the *posterior* and the distribution over predictions $p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n})$ is called the *predictive posterior*. The predictive posterior quantifies our posterior uncertainty about the "prediction" $y^\star$, however, since this is typically a complex distribution, it is difficult to communicate this uncertainty to a human. One statistic that can be used for this purpose is the smallest set $\mathcal{C}_\delta(\boldsymbol{x}^\star) \subseteq \mathbb{R}$ for a fixed $\delta \in (0, 1)$ such that

$$\mathbb{P}(y^\star \in \mathcal{C}_\delta(\boldsymbol{x}^\star) \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}) \geq 1 - \delta. \tag{1.69}$$

That is, we believe with "confidence" at least $1 - \delta$ that the true value of $y^\star$ lies in $\mathcal{C}_\delta(\boldsymbol{x}^\star)$. Such a set $\mathcal{C}_\delta(\boldsymbol{x}^\star)$ is called a *credible set*.

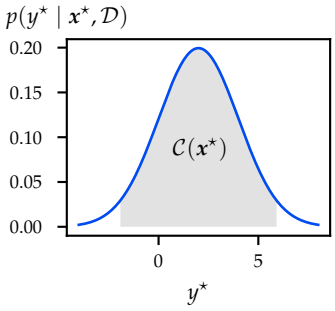



Figure 1.9: Example of a 95% credible set at $\boldsymbol{x}^\star$ where the predictive posterior is Gaussian with mean $\mu(\boldsymbol{x}^\star)$ and standard deviation $\sigma(\boldsymbol{x}^\star)$. In this case, the gray area integrates to $\approx 0.95$ for

$$\mathcal{C}_{0.05}(\boldsymbol{x}^\star) = [\mu(\boldsymbol{x}^\star) \pm 1.96\sigma(\boldsymbol{x}^\star)].$$

We have seen here that the tasks of learning and prediction are intimately related. Indeed, "prediction" can be seen in many ways as a natural by-product of "reasoning" (i.e., probabilistic inference), where we evaluate the likelihood of outcomes given our learned explanations for the world. This intuition can be read off directly from Equation (1.68) where $p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta})$ corresponds to the likelihood of an outcome given the explanation $\boldsymbol{\theta}$ and $p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n})$ corresponds to our inferred belief about the world. We will see many more examples of this link between probabilistic inference and prediction throughout this manuscript.

The high-dimensional integrals of Equations (1.67b) and (1.68) are typically intractable, and represent the main computational challenge in probabilistic inference. Throughout the first part of this manuscript, we will describe settings where exact inference is tractable, as well as modern approximate inference algorithms that can be used when exact inference is intractable.

### 1.3.6 *Recursive Probabilistic Inference and Memory*

We have already alluded to the fact that probabilistic inference has a recursive structure, which lends itself to continual learning and which often leads to efficient algorithms. Let us denote by

$$p^{(t)}(\boldsymbol{\theta}) \doteq p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:t}, y_{1:t}) \tag{1.70}$$

the posterior after the first $t$ observations with $p^{(0)}(\boldsymbol{\theta}) = p(\boldsymbol{\theta})$. Now, suppose that we have already computed $p^{(t)}(\boldsymbol{\theta})$ and observe $y_{t+1}$. We can recursively update the posterior as follows,

$$\begin{aligned} p^{(t+1)}(\boldsymbol{\theta}) &= p(\boldsymbol{\theta} \mid y_{1:t+1}) \\ &\propto p(\boldsymbol{\theta} \mid y_{1:t}) \cdot p(y_{t+1} \mid \boldsymbol{\theta}, y_{1:t}) \end{aligned}$$

using Bayes' rule (1.45)

$$= p^{(t)}(\boldsymbol{\theta}) \cdot p(y_{t+1} \mid \boldsymbol{\theta}). \tag{1.71}$$

using $y_{t+1} \perp y_{1:t} \mid \boldsymbol{\theta}$, see Figure 2.3

Intuitively, the posterior distribution at time $t$ "absorbs" or "summarizes" all seen data.

By unrolling the recursion of Equation (1.71), we see that regardless of the philosophical interpretation of probability, probabilistic inference is a fundamental mechanism of learning. Even the MLE which performs naive approximate inference without a prior (i.e., a uniform prior), is based on $p^{(n)}(\boldsymbol{\theta}) \propto p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})$ which is the result of $n$ individual plausible inferences, where the $(t+1)$-st inference uses the posterior of the $t$-th inference as its prior.

So far we have been considering the supervised learning setting, where all data is available a-priori. However, by sequentially obtaining the new posterior and replacing our prior, we can also perform probabilistic inference as data arrives online (i.e., in "real-time"). This is analogous to recursive logical inference where derived consequences are repeatedly added to the set of propositions to derive new consequences. This also highlights the intimate connection between "reasoning" and "memory". Indeed, the posterior distribution $p^{(t)}(\boldsymbol{\theta})$ can be seen as a form of memory that evolves with time $t$.

## 1.4 Outlook: Decision Theory

How can we use our predictions to make concrete decisions under uncertainty? We will study this question extensively in Part II of this manuscript, but briefly introduce some fundamental concepts here. Making decisions using a probabilistic model $p(y \mid \boldsymbol{x})$ of output $y \in \mathcal{Y}$ given input $\boldsymbol{x} \in \mathcal{X}$, such as the ones we have discussed in the previous section, is commonly formalized by

- a set of possible actions $\mathcal{A}$, and
- a reward function $r(y, a) \in \mathbb{R}$ that computes the reward or utility of taking action $a \in \mathcal{A}$, assuming the true output is $y \in \mathcal{Y}$.

Standard decision theory recommends picking the action with the largest expected utility:

$$a^\star(\boldsymbol{x}) \doteq \arg\max_{a \in \mathcal{A}} \mathbb{E}_{y|\boldsymbol{x}}[r(y, a)]. \tag{1.72}$$

Here, $a^\star$ is called the *optimal decision rule* because, under the given probabilistic model, no other rule can yield a higher expected utility.

Let us consider some examples of reward functions and their corresponding optimal decisions:

**Example 1.30: Reward functions**

Under the decision rule from Equation (1.72), different reward functions $r$ can lead to different decisions. Let us examine two reward functions for the case where $\mathcal{Y} = \mathcal{A} = \mathbb{R}$ (?).

Problem 1.13

- Alternatively to considering $r$ as a reward function, we can interpret $-r$ as the loss of taking action $a$ when the true output is $y$. If our goal is for our actions $a$ to "mimic" the output $y$, a natural choice is the squared loss, $-r(y, a) = (y - a)^2$. It turns out that under the squared loss, the optimal decision is simply the mean: $a^\star(\boldsymbol{x}) = \mathbb{E}[y \mid \boldsymbol{x}]$.
- To contrast this, we consider the asymmetric loss,

$$-r(y, a) = c_1 \underbrace{\max\{y - a, 0\}}_{\text{underestimation error}} + c_2 \underbrace{\max\{a - y, 0\}}_{\text{overestimation error}},$$

  which penalizes underestimation and overestimation differently. When $y \mid \boldsymbol{x} \sim \mathcal{N}(\mu_{\boldsymbol{x}}, \sigma_{\boldsymbol{x}}^2)$ then the optimal decision is

$$a^\star(\boldsymbol{x}) = \mu_{\boldsymbol{x}} + \underbrace{\sigma_{\boldsymbol{x}} \cdot \Phi^{-1}\left(\frac{c_1}{c_1 + c_2}\right)}_{\text{pessimism / optimism}}$$

  where $\Phi$ is the CDF of the standard normal distribution.[32] Note that if $c_1 = c_2$, then the second term vanishes and the optimal decision is the same as under the squared loss. If $c_1 > c_2$, the second term is positive (i.e., *optimistic*) to avoid underestimation, and if $c_1 < c_2$, the second term is negative (i.e., *pessimistic*) to avoid overestimation. We will find these notions of optimism and pessimism to be useful in many decision-making scenarios.

[32] Recall that the CDF $\Phi$ of the standard normal distribution is a sigmoid with its inverse satisfying

$$\Phi^{-1}(u) \begin{cases} < 0 & \text{if } u < 0.5, \\ = 0 & \text{if } u = 0.5, \\ > 0 & \text{if } u > 0.5. \end{cases}$$

While Equation (1.72) describes how to make optimal decisions given a (posterior) probabilistic model, it does not tell us how to learn or improve this model in the first place. That is, these decisions are only optimal under the assumption that we cannot use their outcomes and our resulting observations to update our model and inform future decisions. When we start to consider the effect of our decisions on future data and future posteriors, answering "how do I make optimal decisions?" becomes more complex, and we will study this in Part II on sequential decision-making.

## *Discussion*

In this chapter, we have learned about the fundamental concepts of probabilistic inference. We have seen that probabilistic inference is the

natural extension of logical reasoning to domains with uncertainty. We have also derived the central principle of probabilistic inference, Bayes' rule, which is simple to state but often computationally challenging. In the next part of this manuscript, we will explore settings where exact inference is tractable, as well as modern approaches to approximate probabilistic inference.

## *Overview of Mathematical Background*

We have included brief summaries of the fundamentals of **parameter estimation** (mean estimation in particular) and **optimization** in Sections A.3 and A.4, respectively, which we will refer back to throughout the manuscript. Section A.2 discusses the **correspondence of Gaussians and quadratic forms**. Section A.5 comprises a list of useful matrix identities and inequalities.

## *Problems*

**1.1. Properties of probability.**

Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space. Derive the following properties of probability from the Kolmogorov axioms:

1. For any $A, B \in \mathcal{A}$, if $A \subseteq B$ then $\mathbb{P}(A) \leq \mathbb{P}(B)$.
2. For any $A \in \mathcal{A}$, $\mathbb{P}(\overline{A}) = 1 - \mathbb{P}(A)$.
3. For any countable set of events $\{A_i \in \mathcal{A}\}_i$,

$$\mathbb{P}\left(\bigcup_{i=1}^{\infty} A_i\right) \leq \sum_{i=1}^{\infty} \mathbb{P}(A_i). \tag{1.73}$$

   which is called a *union bound*.

**1.2. Random walks on graphs.**

Let $G$ be a simple connected finite graph. We start at a vertex $u$ of $G$. At every step, we move to one of the neighbors of the current vertex uniformly at random, e.g., if the vertex has 3 neighbors, we move to one of them, each with probability $1/3$. What is the probability that the walk visits a given vertex $v$ eventually?

**1.3. Law of total expectation.**

Show that if $\{A_i\}_{i=1}^k$ are a partition of $\Omega$ and $\mathbf{X}$ is a random vector,

$$\mathbb{E}[\mathbf{X}] = \sum_{i=1}^{k} \mathbb{E}[\mathbf{X} \mid A_i] \cdot \mathbb{P}(A_i). \tag{1.74}$$

**1.4. Covariance matrices are positive semi-definite.**

Prove that a covariance matrix $\boldsymbol{\Sigma}$ is always positive semi-definite. That is, all of its eigenvalues are greater or equal to zero, or equivalently, $\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x} \geq 0$ for any $\boldsymbol{x} \in \mathbb{R}^n$.

**1.5. Probabilistic inference.**

As a result of a medical screening, one of the tests revealed a serious disease in a person. The test has a high accuracy of 99% (the probability of a positive response in the presence of a disease is 99% and the probability of a negative response in the absence of a disease is also 99%). However, the disease is quite rare and occurs only in one person per 10 000. Calculate the probability of the examined person having the identified disease.

**1.6. Zero eigenvalues of covariance matrices.**

Assume that $\mathbf{X}$ is centered, i.e., $\mathbb{E}[\mathbf{X}] = \mathbf{0}$. We say that a random vector $\mathbf{X}$ in $\mathbb{R}^n$ is not linearly independent if for some $\boldsymbol{\alpha} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$, $\boldsymbol{\alpha}^\top \mathbf{X} = 0$.

1. Show that if $\mathbf{X}$ is not linearly independent, then $\mathrm{Var}[\mathbf{X}]$ has a zero eigenvalue.
2. Show that if $\mathrm{Var}[\mathbf{X}]$ has a zero eigenvalue, then $\mathbf{X}$ is not linearly independent.
   *Hint: Consider the variance of $\boldsymbol{\lambda}^\top \mathbf{X}$ where $\boldsymbol{\lambda}$ is the eigenvector corresponding to the zero eigenvalue.*

Thus, we have shown that $\mathrm{Var}[\mathbf{X}]$ has a zero eigenvalue if and only if $\mathbf{X}$ is not linearly independent.

**1.7. Product of Gaussian PDFs.**

Let $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2 \in \mathbb{R}^n$ be mean vectors and $\boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2 \in \mathbb{R}^{n \times n}$ be covariance matrices. Prove that

$$\mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \propto \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) \cdot \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2) \tag{1.75}$$

for some mean vector $\boldsymbol{\mu} \in \mathbb{R}^n$ and covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$. That is, show that the product of two Gaussian PDFs is proportional to the PDF of a Gaussian.

**1.8. Independence of Gaussians.**

Show that two jointly Gaussian random vectors, $\mathbf{X}$ and $\mathbf{Y}$, are independent if and only if $\mathbf{X}$ and $\mathbf{Y}$ are uncorrelated.

**1.9. Marginal / conditional distribution of a Gaussian.**

Prove Theorem 1.25. That is, show that

1. every marginal of a Gaussian is Gaussian; and

2. conditioning on a subset of variables of a joint Gaussian is Gaussian

by finding their corresponding PDFs.

*Hint: You may use that for matrices $\boldsymbol{\Sigma}$ and $\boldsymbol{\Lambda}$ such that $\boldsymbol{\Sigma}^{-1} = \boldsymbol{\Lambda}$,*

- *if $\boldsymbol{\Sigma}$ and $\boldsymbol{\Lambda}$ are symmetric,*

$$\begin{aligned}
&\begin{bmatrix} \boldsymbol{x}_A \\ \boldsymbol{x}_B \end{bmatrix}^\top \begin{bmatrix} \boldsymbol{\Lambda}_{AA} & \boldsymbol{\Lambda}_{AB} \\ \boldsymbol{\Lambda}_{BA} & \boldsymbol{\Lambda}_{BB} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_A \\ \boldsymbol{x}_B \end{bmatrix} \\
&= \boldsymbol{x}_A^\top \boldsymbol{\Lambda}_{AA} \boldsymbol{x}_A + \boldsymbol{x}_A^\top \boldsymbol{\Lambda}_{AB} \boldsymbol{x}_B + \boldsymbol{x}_B^\top \boldsymbol{\Lambda}_{BA} \boldsymbol{x}_A + \boldsymbol{x}_B^\top \boldsymbol{\Lambda}_{BB} \boldsymbol{x}_B \\
&= \boldsymbol{x}_A^\top (\boldsymbol{\Lambda}_{AA} - \boldsymbol{\Lambda}_{AB} \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA}) \boldsymbol{x}_A + \\
&\qquad (\boldsymbol{x}_B + \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} \boldsymbol{x}_A)^\top \boldsymbol{\Lambda}_{BB} (\boldsymbol{x}_B + \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} \boldsymbol{x}_A),
\end{aligned}$$

- $\boldsymbol{\Lambda}_{BB}^{-1} = \boldsymbol{\Sigma}_{BB} - \boldsymbol{\Sigma}_{BA} \boldsymbol{\Sigma}_{AA}^{-1} \boldsymbol{\Sigma}_{AB}$,
- $\boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} = -\boldsymbol{\Sigma}_{BA} \boldsymbol{\Sigma}_{AA}^{-1}$.

*The final two equations follow from the general characterization of the inverse of a block matrix (Petersen et al., 2008, section 9.1.3).*

**1.10. Closedness properties of Gaussians.**

Recall the notion of a *moment-generating function* (MGF) of a random vector $\mathbf{X}$ in $\mathbb{R}^n$ which is defined as

$$\varphi_{\mathbf{X}}(\boldsymbol{t}) \doteq \mathbb{E} \exp\left(\boldsymbol{t}^\top \mathbf{X}\right), \quad \text{for all } \boldsymbol{t} \in \mathbb{R}^n. \tag{1.76}$$

An MGF uniquely characterizes a distribution. The MGF of the multivariate Gaussian $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is

$$\varphi_{\mathbf{X}}(\boldsymbol{t}) = \exp\left(\boldsymbol{t}^\top \boldsymbol{\mu} + \frac{1}{2} \boldsymbol{t}^\top \boldsymbol{\Sigma} \boldsymbol{t}\right). \tag{1.77}$$

This generalizes the MGF of the univariate Gaussian from Equation (A.40).

Prove the following facts.

1. *Closedness under affine transformations:* Given an $n$-dimensional Gaussian $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, and $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ and $\boldsymbol{b} \in \mathbb{R}^m$,

$$\boldsymbol{A}\mathbf{X} + \boldsymbol{b} \sim \mathcal{N}(\boldsymbol{A}\boldsymbol{\mu} + \boldsymbol{b}, \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\top). \tag{1.78}$$

2. *Additivity:* Given two independent Gaussian random vectors $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $\mathbf{X}' \sim \mathcal{N}(\boldsymbol{\mu}', \boldsymbol{\Sigma}')$ in $\mathbb{R}^n$,

$$\mathbf{X} + \mathbf{X}' \sim \mathcal{N}(\boldsymbol{\mu} + \boldsymbol{\mu}', \boldsymbol{\Sigma} + \boldsymbol{\Sigma}'). \tag{1.79}$$

These properties are unique to Gaussians and a reason for why they are widely used for learning and inference.

**1.11. Expectation and variance of Gaussians.**

Derive that $\mathbb{E}[\mathbf{X}] = \boldsymbol{\mu}$ and $\mathrm{Var}[\mathbf{X}] = \boldsymbol{\Sigma}$ when $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.
*Hint: First derive the expectation and variance of a univariate standard normal random variable.*

**1.12. Non-affine transformations of Gaussians.**

Answer the following questions with **yes** or **no**.

1. Does there exist any non-affine transformation of a Gaussian random vector which is Gaussian? If yes, give an example.
2. Let $X, Y, Z$ be independent standard normal random variables. Is $\frac{X+YZ}{\sqrt{1+Z^2}}$ Gaussian?

**1.13. Decision theory.**

Derive the optimal decisions under the squared loss and the asymmetric loss from Example 1.30.

PART I

# *Probabilistic Machine Learning*

# *Preface to Part I*

As humans, we constantly learn about the world around us. We learn to interact with our physical surroundings. We deepen our understanding of the world by establishing relationships between actors, objects, and events. And we learn about ourselves by observing how we interact with the world and with ourselves. We then continuously use this knowledge to make inferences and predictions, be it about the weather, the movement of a ball, or the behavior of a friend.

With limited computational resources, limited genetic information, and limited life experience, we are not able to learn everything about the world to complete certainty. We saw in Chapter 1 that probability theory is the mathematical framework for reasoning with uncertainty in the same way that logic is the mathematical framework for reasoning with certainty. We will discuss two kinds of uncertainty: "aleatoric" uncertainty which cannot be reduced under computational constraints, and "epistemic" uncertainty which can be reduced by observing more data.

An important aspect of learning is that we do not just learn once, but continually. Bayes' rule allows us to update our beliefs and reduce our uncertainty as we observe new data — a process that is called *probabilistic inference*. By taking the former posterior as the new prior, probabilistic inference can be performed continuously and repeated indefinitely as we observe more and more data.

Our sensory information is often noisy and imperfect, which is another source of uncertainty. The same is true for machines, even if they can sometimes sense aspects of the world more accurately than humans. We discuss how one can infer latent structure of the world from sensed data, such as the state of a dynamical system like a car, in a process that is called *filtering*.

In this first part of the course, we examine how we can build machines that are capable of (continual) learning and inference. First, we introduce probabilistic inference in the context of linear models which

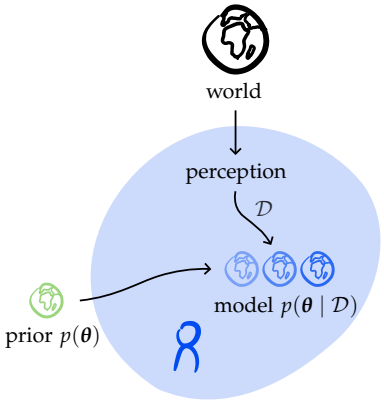



Figure 1.10: A schematic illustration of probabilistic inference in the context of the (supervised) learning of a model $\boldsymbol{\theta}$ from perceived data $\mathcal{D}$. The prior model $p(\boldsymbol{\theta})$ can equip the model with anything from substantial, to little, to no prior knowledge.

make predictions based on fixed (often hand-designed) features. We then discuss how probabilistic inference can be scaled to kernel methods and Gaussian processes which use a large (potentially infinite) number of features, and to deep neural networks which learn features dynamically from data. In these models, exact inference is typically intractable, and we discuss modern methods for approximate inference such as variational inference and Markov chain Monte Carlo. We highlight a tradeoff between curiosity (i.e., extrapolating beyond the given data) and conformity (i.e., fitting the given data), which surfaces as a fundamental principle of probabilistic inference in the regime where the data and our computational resources are limited.

# 2

# *Linear Regression*

As a first example of probabilistic inference, we will study linear models for regression[1] which assume that the output $y \in \mathbb{R}$ is a linear function of the input $\boldsymbol{x} \in \mathbb{R}^d$:

$$y \approx \boldsymbol{w}^\top \boldsymbol{x} + w_0$$

where $\boldsymbol{w} \in \mathbb{R}^d$ are the weights and $w_0 \in \mathbb{R}$ is the intercept. Observe that if we define the extended inputs $\boldsymbol{x}' \doteq (\boldsymbol{x}, 1)$ and $\boldsymbol{w}' \doteq (\boldsymbol{w}, w_0)$, then $\boldsymbol{w}'^\top \boldsymbol{x}' = \boldsymbol{w}^\top \boldsymbol{x} + w_0$, implying that without loss of generality it suffices to study linear functions without the intercept term $w_0$. We will therefore consider the following function class of linear models

$$f(\boldsymbol{x}; \boldsymbol{w}) \doteq \boldsymbol{w}^\top \boldsymbol{x}.$$

We will consider the supervised learning task of learning weights $\boldsymbol{w}$ from labeled training data $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$. We define the *design matrix*,

$$\boldsymbol{X} \doteq \begin{bmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \tag{2.1}$$

as the collection of inputs and the vector $\boldsymbol{y} \doteq [y_1 \cdots y_n]^\top \in \mathbb{R}^n$ as the collection of labels. For each noisy observation $(\boldsymbol{x}_i, y_i)$, we define the value of the approximation of our model, $f_i \doteq \boldsymbol{w}^\top \boldsymbol{x}_i$. Our model at the inputs $\boldsymbol{X}$ is described by the vector $\boldsymbol{f} \doteq [f_1 \ \cdots \ f_n]^\top$ which can be expressed succinctly as $\boldsymbol{f} = \boldsymbol{X}\boldsymbol{w}$.

The most common way of estimating $\boldsymbol{w}$ from data is the *least squares estimator*,

$$\hat{\boldsymbol{w}}_{\text{ls}} \doteq \arg\min_{\boldsymbol{w} \in \mathbb{R}^d} \sum_{i=1}^n (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 = \arg\min_{\boldsymbol{w} \in \mathbb{R}^d} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2, \tag{2.2}$$

[1] As we have discussed in Section 1.3, regression models can also be used for classification. The canonical example of a linear model for classification is logistic regression, which we will discuss in Section 5.1.1.

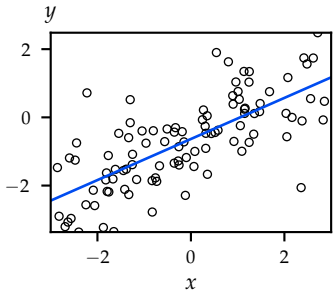



Figure 2.1: Example of linear regression with the least squares estimator (shown in blue).

minimizing the squared difference between the labels and predictions of the model. A slightly different estimator is used for *ridge regression*,

$$\hat{\boldsymbol{w}}_{\text{ridge}} \doteq \arg\min_{\boldsymbol{w}\in\mathbb{R}^d} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2 + \lambda \|\boldsymbol{w}\|_2^2 \tag{2.3}$$

where $\lambda > 0$. The squared $L_2$ regularization term $\lambda \|\boldsymbol{w}\|_2^2$ penalizes large $\boldsymbol{w}$ and thus reduces the "complexity" of the resulting model.[2] It can be shown that the unique solutions to least squares and ridge regression are given by

$$\hat{\boldsymbol{w}}_{\text{ls}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y} \qquad \text{and} \tag{2.4}$$

$$\hat{\boldsymbol{w}}_{\text{ridge}} = (\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I})^{-1} \boldsymbol{X}^\top \boldsymbol{y}, \tag{2.5}$$

respectively if the Hessian of the loss is positive definite (i.e., the loss is strictly convex) (?) which is the case as long as the columns of $\boldsymbol{X}$ are not linearly dependent. Least squares regression can be seen as finding the orthogonal projection of $\boldsymbol{y}$ onto the column space of $\boldsymbol{X}$, as is illustrated in Figure 2.2 (?).

[2] Ridge regression is more robust to multicollinearity than standard linear regression. *Multicollinearity* occurs when multiple independent inputs are highly correlated. In this case, their individual effects on the predicted variable cannot be estimated well. Classical linear regression is highly volatile to small input changes. The regularization of ridge regression reduces this volatility by introducing a bias on the weights towards 0.

**Problem 2.1 (1)**

**Problem 2.1 (2)**

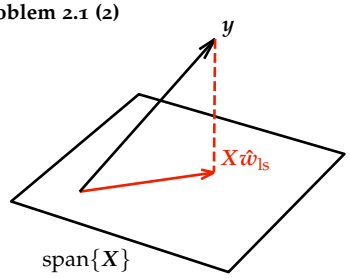



Figure 2.2: Least squares regression finds the orthogonal projection of $\boldsymbol{y}$ onto $\text{span}\{\boldsymbol{X}\}$ (here illustrated as the plane).

### 2.0.1 *Maximum Likelihood Estimation*

Since our function class comprises linear functions of the form $\boldsymbol{w}^\top \boldsymbol{x}$, the observation model from Equation (1.56) simplifies to

$$y_i = \boldsymbol{w}^{\star\top} \boldsymbol{x}_i + \varepsilon_i \tag{2.6}$$

for some weight vector $\boldsymbol{w}$, where for the purpose of this chapter we will additionally assume that $\varepsilon_i \sim \mathcal{N}(0, \sigma_{\text{n}}^2)$ is homoscedastic Gaussian noise.[3] This observation model is equivalently characterized by the Gaussian likelihood,

$$y_i \mid \boldsymbol{x}_i, \boldsymbol{w} \sim \mathcal{N}(\boldsymbol{w}^\top \boldsymbol{x}_i, \sigma_{\text{n}}^2). \tag{2.7}$$

[3] $\varepsilon_i$ is called *additive white Gaussian noise*.

Based on this likelihood we can compute the MLE (1.57) of the weights:

$$\hat{\boldsymbol{w}}_{\text{MLE}} = \arg\max_{\boldsymbol{w}\in\mathbb{R}^d} \sum_{i=1}^n \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{w}) = \arg\min_{\boldsymbol{w}\in\mathbb{R}^d} \sum_{i=1}^n (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2.$$

using Equation (1.55d)

plugging in the Gaussian likelihood and simplifying

Note that therefore $\hat{\boldsymbol{w}}_{\text{MLE}} = \hat{\boldsymbol{w}}_{\text{ls}}$.

In practice, the noise variance $\sigma_{\text{n}}^2$ is typically unknown and also has to be determined, for example, through maximum likelihood estimation. It is a straightforward exercise to check that the MLE of $\sigma_{\text{n}}^2$ given fixed weights $\boldsymbol{w}$ is $\hat{\sigma}_{\text{n}}^2 = \frac{1}{n}\sum_{i=1}^n (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2$ (?).

**Problem 2.2**

## 2.1 *Weight-space View*

The most immediate and natural probabilistic interpretation of linear regression is to quantify uncertainty about the weights $\boldsymbol{w}$. Recall

that probabilistic inference requires specification of a generative model comprised of prior and likelihood. Throughout this chapter, we will use the Gaussian prior,

$$\boldsymbol{w} \sim \mathcal{N}(\boldsymbol{0}, \sigma_{\mathrm{p}}^2 \boldsymbol{I}), \tag{2.8}$$

and the Gaussian likelihood from Equation (2.7). We will discuss possible (probabilistic) strategies for choosing hyperparameters such as the prior variance $\sigma_{\mathrm{p}}^2$ and the noise variance $\sigma_{\mathrm{n}}^2$ in Section 4.4.

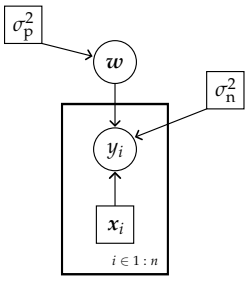



Figure 2.3: Directed graphical model of Bayesian linear regression in plate notation.

**Remark 2.1: Why a Gaussian prior?**

The choice of using a Gaussian prior may seem somewhat arbitrary at first sight, except perhaps for the nice analytical properties of Gaussians that we have seen in Section 1.2.3 and which will prove useful. The maximum entropy principle (cf. Section 1.2.1) provides a more fundamental justification for Gaussian priors since turns out that $\mathcal{N}$ has the maximum entropy among all distributions on $\mathbb{R}^d$ with known mean and variance (?).

**Problem 5.6**

Next, let us derive the posterior distribution over the weights.

$$\begin{aligned}
&\log p(\boldsymbol{w} \mid \boldsymbol{x}_{1:n}, y_{1:n}) \\
&= \log p(\boldsymbol{w}) + \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{w}) + \text{const} && \text{by Bayes' rule (1.45)} \\
&= \log p(\boldsymbol{w}) + \sum_{i=1}^{n} \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{w}) + \text{const} && \text{using independence of the samples} \\
&= -\frac{1}{2}\left[\sigma_{\mathrm{p}}^{-2} \|\boldsymbol{w}\|_2^2 + \sigma_{\mathrm{n}}^{-2} \sum_{i=1}^{n} (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2\right] + \text{const} && \text{using the Gaussian prior and likelihood} \\
&= -\frac{1}{2}\left[\sigma_{\mathrm{p}}^{-2} \|\boldsymbol{w}\|_2^2 + \sigma_{\mathrm{n}}^{-2} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2\right] + \text{const} && \text{using } \textstyle\sum_{i=1}^{n} (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 = \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2 \\
&= -\frac{1}{2}\left[\sigma_{\mathrm{p}}^{-2} \boldsymbol{w}^\top \boldsymbol{w} + \sigma_{\mathrm{n}}^{-2}\left(\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X} \boldsymbol{w} - 2\boldsymbol{y}^\top \boldsymbol{X} \boldsymbol{w} + \boldsymbol{y}^\top \boldsymbol{y}\right)\right] + \text{const} \\
&= -\frac{1}{2}\left[\boldsymbol{w}^\top (\sigma_{\mathrm{n}}^{-2} \boldsymbol{X}^\top \boldsymbol{X} + \sigma_{\mathrm{p}}^{-2} \boldsymbol{I}) \boldsymbol{w} - 2\sigma_{\mathrm{n}}^{-2} \boldsymbol{y}^\top \boldsymbol{X} \boldsymbol{w}\right] + \text{const}. && (2.9)
\end{aligned}$$

Observe that the log-posterior is a quadratic form in $\boldsymbol{w}$, so the posterior distribution must be Gaussian:

$$\boldsymbol{w} \mid \boldsymbol{x}_{1:n}, y_{1:n} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \tag{2.10a}$$

see Equation (A.12)

where we can read off the mean and variance to be

$$\boldsymbol{\mu} \doteq \sigma_{\mathrm{n}}^{-2} \boldsymbol{\Sigma} \boldsymbol{X}^\top \boldsymbol{y}, \tag{2.10b}$$

$$\boldsymbol{\Sigma} \doteq \left(\sigma_{\mathrm{n}}^{-2} \boldsymbol{X}^\top \boldsymbol{X} + \sigma_{\mathrm{p}}^{-2} \boldsymbol{I}\right)^{-1}. \tag{2.10c}$$

This also shows that Gaussians with known variance and linear likelihood are self-conjugate, a property that we had hinted at in Section 1.2.2. It can be shown more generally that Gaussians with known

variance are self-conjugate to any Gaussian likelihood (Murphy, 2007). For other generative models, the posterior can typically not be expressed in closed-form — this is a very special property of Gaussians!

### 2.1.1 *Maximum a Posteriori Estimation*

Computing the MAP estimate for the weights,

$$\begin{aligned}\hat{\boldsymbol{w}}_{\text{MAP}} &= \arg\max_{\boldsymbol{w}} \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{w}) + \log p(\boldsymbol{w}) \\ &= \arg\min_{\boldsymbol{w}} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2 + \frac{\sigma_{\text{n}}^2}{\sigma_{\text{p}}^2} \|\boldsymbol{w}\|_2^2 , \end{aligned} \tag{2.11}$$

using that the likelihood and prior are Gaussian

we observe that this is identical to ridge regression with weight decay $\lambda \doteq \sigma_{\text{n}}^2/\sigma_{\text{p}}^2$: $\hat{\boldsymbol{w}}_{\text{MAP}} = \hat{\boldsymbol{w}}_{\text{ridge}}$. Equation (2.11) is simply the MLE loss with an additional $L_2$-regularization (originating from the prior) that encourages keeping weights small. Recall that the MAP estimate corresponds to the mode of the posterior distribution, which in the case of a Gaussian is simply its mean $\boldsymbol{\mu}$. As to be expected, $\boldsymbol{\mu}$ coincides with the analytical solution to ridge regression from Equation (2.5).

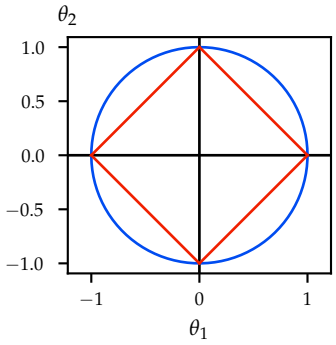



Figure 2.4: Level sets of $L_2$- (blue) and $L_1$-regularization (red), corresponding to Gaussian and Laplace priors, respectively. It can be seen that $L_1$-regularization is more effective in encouraging sparse solutions (that is, solutions where many components are set to exactly 0).

**Example 2.2: Lasso as the MAP estimate with a Laplace prior**

One problem with ridge regression is that the contribution of nearly-zero weights to the $L_2$-regularization term is negligible. Thus, $L_2$-regularization is typically not sufficient to perform variable selection (that is, set some weights to zero entirely), which is often desirable for interpretability of the model.

A commonly used alternative to ridge regression is the *least absolute shrinkage and selection operator* (or *lasso*), which regularizes with the $L_1$-norm:

$$\hat{\boldsymbol{w}}_{\text{lasso}} \doteq \arg\min_{\boldsymbol{w} \in \mathbb{R}^d} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2 + \lambda \|\boldsymbol{w}\|_1 . \tag{2.12}$$

It turns out that lasso can also be viewed as probabilistic inference, using a Laplace prior $\boldsymbol{w} \sim \text{Laplace}(\boldsymbol{0}, h)$ with length scale $h$ instead of a Gaussian prior.

Computing the MAP estimate for the weights yields,

$$\begin{aligned}\hat{\boldsymbol{w}}_{\text{MAP}} &= \arg\max_{\boldsymbol{w}} \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{w}) + \log p(\boldsymbol{w}) \\ &= \arg\min_{\boldsymbol{w}} \sum_{i=1}^{n} (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 + \frac{\sigma_{\text{n}}^2}{h} \|\boldsymbol{w}\|_1 \end{aligned} \tag{2.13}$$

using that the likelihood is Gaussian and the prior is Laplacian

which coincides with the lasso with weight decay $\lambda \doteq \sigma_{\text{n}}^2/h$.

To make predictions at a test point $\boldsymbol{x}^\star$, we define the (model-)predicted point $f^\star \doteq \hat{\boldsymbol{w}}_{\text{MAP}}^\top \boldsymbol{x}^\star$ and obtain the label prediction

$$y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n} \sim \mathcal{N}(f^\star, \sigma_{\text{n}}^2). \tag{2.14}$$

Here we observe that using point estimates such as the MAP estimate does not quantify uncertainty in the weights. The MAP estimate simply collapses all mass of the posterior around its mode. This can be harmful when we are highly unsure about the best model, e.g., because we have observed insufficient data.

### 2.1.2 *Probabilistic Inference*

Rather than selecting a single weight vector $\hat{\boldsymbol{w}}$ to make predictions, we can use the full posterior distribution. This is known as *Bayesian linear regression* (BLR) and illustrated with an example in Figure 2.5.

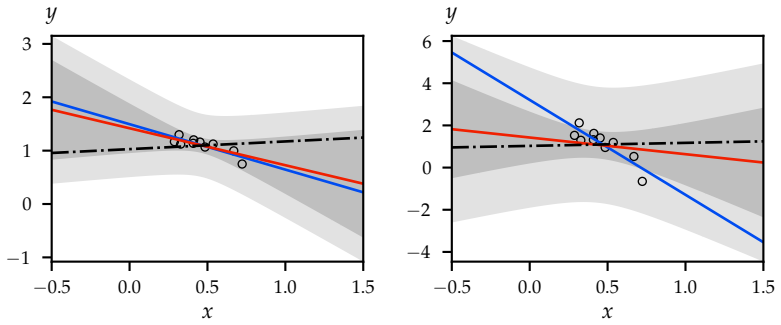



Figure 2.5: Comparison of **linear regression (MLE)**, **ridge regression (MAP estimate)**, and **Bayesian linear regression** when the data is generated according to

$$y \mid \boldsymbol{w}, \boldsymbol{x} \sim \mathcal{N}(\boldsymbol{w}^\top \boldsymbol{x}, \sigma_{\text{n}}^2).$$

The **true mean** is shown in black, the MLE in blue, and the MAP estimate in red. The dark gray area denotes the epistemic uncertainty of Bayesian linear regression and the light gray area the additional homoscedastic noise. On the left, $\sigma_{\text{n}} = 0.15$. On the right, $\sigma_{\text{n}} = 0.7$.

To make predictions at a test point $\boldsymbol{x}^\star$, we let $f^\star \doteq \boldsymbol{w}^\top \boldsymbol{x}^\star$ which has the distribution

$$f^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n} \sim \mathcal{N}(\boldsymbol{\mu}^\top \boldsymbol{x}^\star, \boldsymbol{x}^{\star\top} \boldsymbol{\Sigma} \boldsymbol{x}^\star). \tag{2.15}$$

using the closedness of Gaussians under linear transformations (1.78)

Note that this does not take into account the noise in the labels $\sigma_{\text{n}}^2$. For the label prediction $y^\star$, we obtain

$$y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n} \sim \mathcal{N}(\boldsymbol{\mu}^\top \boldsymbol{x}^\star, \boldsymbol{x}^{\star\top} \boldsymbol{\Sigma} \boldsymbol{x}^\star + \sigma_{\text{n}}^2). \tag{2.16}$$

using additivity of Gaussians (1.79)

### 2.1.3 *Recursive Probabilistic Inference*

We have already discussed the recursive properties of probabilistic inference in Section 1.3.6. For Bayesian linear regression with a Gaussian prior and likelihood, this principle can be used to derive an efficient online algorithm since also the posterior is a Gaussian,

$$p^{(t)}(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}; \boldsymbol{\mu}^{(t)}, \boldsymbol{\Sigma}^{(t)}), \tag{2.17}$$

which can be stored efficiently using only $O(d^2)$ parameters. This leads to an efficient online algorithm for Bayesian linear regression

with time-independent(!) memory complexity $O(d)$ and round complexity $O(d^2)$ (?). The interpretation of Bayesian linear regression as an online algorithm also highlights similarities to other sequential models such as Kalman filters, which we discuss in Chapter 3. In Example 3.5, we will learn that online Bayesian linear regression is, in fact, an example of a Kalman filter. **Problem 2.5**

## 2.2 *Aleatoric and Epistemic Uncertainty*

The predictive posterior distribution from Equation (2.16) highlights a decomposition of uncertainty wherein $\boldsymbol{x}^{\star\top}\boldsymbol{\Sigma}\boldsymbol{x}^\star$ corresponds to the uncertainty about our model due to the lack of data (commonly referred to as the *epistemic uncertainty*) and $\sigma_\text{n}^2$ corresponds to the uncertainty about the labels that cannot be explained by the inputs and any model from the model class (commonly referred to as the *aleatoric uncertainty*, "irreducible noise", or simply "(label) noise") (?). **Problem 2.6**

A natural probabilistic approach is to represent epistemic uncertainty with a probability distribution over models. Intuitively, the variance of this distribution measures our uncertainty about the model and its mode corresponds to our current best (point) estimate. The distribution over weights of a linear model is one example, and we will continue to explore this approach for other models in the following chapters.

It is a practical modeling choice how much inaccuracy to attribute to epistemic or aleatoric uncertainty. Generally, when a poor model is used to explain a process, more inaccuracy has to be attributed to irreducible noise. For example, when a linear model is used to "explain" a nonlinear process, most uncertainty is aleatoric as the model cannot explain the data well. As we use more expressive models, a larger portion of the uncertainty can be explained by the data.

Epistemic and aleatoric uncertainty can be formally defined in terms of the law of total variance (1.41),

$$\operatorname{Var}[y^\star \mid \boldsymbol{x}^\star] = \underbrace{\mathbb{E}_{\boldsymbol{\theta}}\left[\operatorname{Var}_{y^\star}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}]\right]}_{\text{aleatoric uncertainty}} + \underbrace{\operatorname{Var}_{\boldsymbol{\theta}}\left[\mathbb{E}_{y^\star}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}]\right]}_{\text{epistemic uncertainty}}. \tag{2.18}$$

Here, the mean variability of predictions $y^\star$ averaged across all models $\boldsymbol{\theta}$ is the estimated *aleatoric uncertainty*. In contrast, the variability of the mean prediction $y^\star$ under each model $\boldsymbol{\theta}$ is the estimated *epistemic uncertainty*. This decomposition of uncertainty will appear frequently throughout this manuscript.

## 2.3 Non-linear Features

We can use linear regression not only to learn linear functions. The trick is to apply a nonlinear transformation $\boldsymbol{\phi} : \mathbb{R}^d \to \mathbb{R}^e$ to the features $\boldsymbol{x}_i$, where $d$ is the dimension of the input space and $e$ is the dimension of the designed *feature space*. We denote the design matrix comprised of transformed features by $\boldsymbol{\Phi} \in \mathbb{R}^{n \times e}$. Note that if the feature transformation $\boldsymbol{\phi}$ is the identity function then $\boldsymbol{\Phi} = \boldsymbol{X}$.

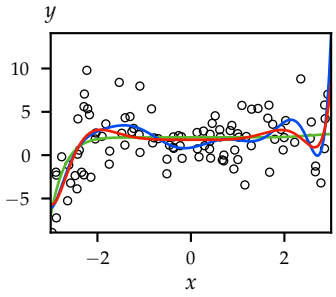



Figure 2.6: Applying linear regression with a feature space of polynomials of degree 10. The **least squares estimate** is shown in blue, **ridge regression** in red, and **lasso** in green.

**Example 2.3: Polynomial regression**

Let $\boldsymbol{\phi}(x) \doteq [x^2, x, 1]$ and $\boldsymbol{w} \doteq [a, b, c]$. Then the function that our model learns is given as

$$f = ax^2 + bx + c.$$

Thus, our model can exactly represent all polynomials up to degree 2.

However, to learn polynomials of degree $m$ in $d$ input dimensions, we need to apply the nonlinear transformation

$$\begin{aligned}\boldsymbol{\phi}(\boldsymbol{x}) = [1, x_1, \ldots, x_d, x_1^2, \ldots, x_d^2, x_1 \cdot x_2, \ldots, x_{d-1} \cdot x_d,\\ \ldots,\\ x_{d-m+1} \cdot \cdots \cdot x_d].\end{aligned}$$

Note that the feature dimension $e$ is $\sum_{i=0}^{m} \binom{d+i-1}{i} = \Theta(d^m)$.[4] Thus, the dimension of the feature space grows exponentially in the degree of polynomials and input dimensions. Even for relatively small $m$ and $d$, this becomes completely unmanageable.

The example of polynomials highlights that it may be inefficient to keep track of the weights $\boldsymbol{w} \in \mathbb{R}^e$ when $e$ is large, and that it may be useful to instead consider a reparameterization which is of dimension $n$ rather than of the feature dimension.

[4] Observe that the vector contains $\binom{d+i-1}{i}$ monomials of degree $i$ as this is the number of ways to choose $i$ times from $d$ items with replacement and without consideration of order. To see this, consider the following encoding: We take a sequence of $d + i - 1$ spots. Selecting any subset of $i$ spots, we interpret the remaining $d - 1$ spots as "barriers" separating each of the $d$ items. The selected spots correspond to the number of times each item has been selected. For example, if 2 items are to be selected out of a total of 4 items with replacement, one possible configuration is "$\circ \,||\, \circ \,|$" where $\circ$ denotes a selected spot and $|$ denotes a barrier. This configuration encodes that the first and third item have each been chosen once. The number of possible configurations — each encoding a unique outcome — is therefore $\binom{d+i-1}{i}$.

## 2.4 Function-space View

Let us now look at Bayesian linear regression through a different lens. Previously, we have been interpreting it as a distribution over the weights $\boldsymbol{w}$ of a linear function $\boldsymbol{f} = \boldsymbol{\Phi}\boldsymbol{w}$. The key idea is that for a finite set of inputs (ensuring that the design matrix is well-defined), we can equivalently consider a distribution directly over the estimated function values $\boldsymbol{f}$. We call this the *function-space view* of Bayesian linear regression.

Instead of considering a prior over the weights $\boldsymbol{w} \sim \mathcal{N}(\mathbf{0}, \sigma_\text{p}^2 \boldsymbol{I})$ as we have done previously, we now impose a prior directly on the values of our model at the observations. Using that Gaussians are closed under linear maps (1.78), we obtain the equivalent prior

$$\boldsymbol{f} \mid \boldsymbol{X} \sim \mathcal{N}(\boldsymbol{\Phi}\mathbb{E}[\boldsymbol{w}], \boldsymbol{\Phi}\text{Var}[\boldsymbol{w}]\boldsymbol{\Phi}^\top) = \mathcal{N}(\mathbf{0}, \underbrace{\sigma_\text{p}^2 \boldsymbol{\Phi}\boldsymbol{\Phi}^\top}_{\boldsymbol{K}}) \tag{2.19}$$

where $\boldsymbol{K} \in \mathbb{R}^{n \times n}$ is the so-called *kernel matrix*. Observe that the entries of the kernel matrix can be expressed as $\boldsymbol{K}(i,j) = \sigma_\text{p}^2 \cdot \boldsymbol{\phi}(\boldsymbol{x}_i)^\top \boldsymbol{\phi}(\boldsymbol{x}_j)$.

You may say that nothing has changed, and you would be right — that is precisely the point. Note, however, that the shape of the kernel matrix is $n \times n$ rather than the $e \times e$ covariance matrix over weights, which becomes unmanageable when $e$ is large. The kernel matrix $\boldsymbol{K}$ has entries only for the finite set of observed inputs. However, in principle, we could have observed any input, and this motivates the definition of the *kernel function*

$$k(\boldsymbol{x}, \boldsymbol{x}') \doteq \sigma_\text{p}^2 \cdot \boldsymbol{\phi}(\boldsymbol{x})^\top \boldsymbol{\phi}(\boldsymbol{x}') \tag{2.20}$$

for arbitrary inputs $\boldsymbol{x}$ and $\boldsymbol{x}'$. A kernel matrix is simply a finite "view" of the kernel function,

$$\boldsymbol{K} = \begin{bmatrix} k(\boldsymbol{x}_1, \boldsymbol{x}_1) & \cdots & k(\boldsymbol{x}_1, \boldsymbol{x}_n) \\ \vdots & \ddots & \vdots \\ k(\boldsymbol{x}_n, \boldsymbol{x}_1) & \cdots & k(\boldsymbol{x}_n, \boldsymbol{x}_n) \end{bmatrix} \tag{2.21}$$

Observe that by definition of the kernel matrix in Equation (2.19), the kernel matrix is a covariance matrix and the kernel function measures the covariance of the function values $f(\boldsymbol{x})$ and $f(\boldsymbol{x}')$ given inputs $\boldsymbol{x}$ and $\boldsymbol{x}'$:

$$k(\boldsymbol{x}, \boldsymbol{x}') = \text{Cov}\big[f(\boldsymbol{x}), f(\boldsymbol{x}')\big]. \tag{2.22}$$

Moreover, note that we have reformulated[5] the learning algorithm such that the feature space is now *implicit* in the choice of kernel, and the kernel is defined by inner products of (nonlinearly transformed) inputs. In other words, the choice of kernel implicitly determines the class of functions that $\boldsymbol{f}$ is sampled from (without expressing the functions explicitly in closed-form), which encodes our prior beliefs. This is known as the *kernel trick*.

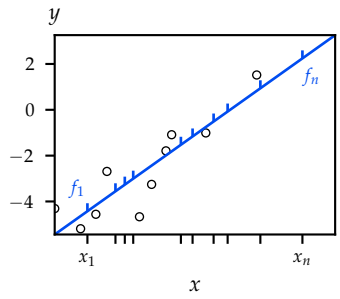



Figure 2.7: An illustration of the function-space view. The model is described by the points $(x_i, f_i)$.

[5] we often say "kernelized"

### 2.4.1 *Learning and Predictions*

We have already kernelized the Bayesian linear regression prior. The posterior distribution $\boldsymbol{f} \mid \boldsymbol{X}, \boldsymbol{y}$ is again Gaussian due to the closedness

properties of Gaussians, analogously to our derivation of the prior kernel matrix in Equation (2.19).

It remains to show that we can also rely on the kernel trick for predictions. Given the test point $\boldsymbol{x}^\star$, we define

$$\tilde{\boldsymbol{\Phi}} \doteq \begin{bmatrix} \boldsymbol{\Phi} \\ \boldsymbol{\phi}(\boldsymbol{x}^\star)^\top \end{bmatrix}, \quad \tilde{\boldsymbol{y}} \doteq \begin{bmatrix} \boldsymbol{y} \\ y^\star \end{bmatrix}, \quad \tilde{\boldsymbol{f}} \doteq \begin{bmatrix} \boldsymbol{f} \\ f^\star \end{bmatrix}.$$

We immediately obtain $\tilde{\boldsymbol{f}} = \tilde{\boldsymbol{\Phi}}\boldsymbol{w}$. Analogously to our analysis of predictions from the weight-space view, we add the label noise to obtain the estimate $\tilde{\boldsymbol{y}} = \tilde{\boldsymbol{f}} + \tilde{\boldsymbol{\varepsilon}}$ where $\tilde{\boldsymbol{\varepsilon}} \doteq [\varepsilon_1 \ \cdots \ \varepsilon_n \ \varepsilon^\star]^\top \sim \mathcal{N}(\boldsymbol{0}, \sigma_\mathrm{n}^2 \boldsymbol{I})$ is the independent label noise. Applying the same reasoning as we did for the prior, we obtain

$$\tilde{\boldsymbol{f}} \mid \boldsymbol{X}, \boldsymbol{x}^\star \sim \mathcal{N}(\boldsymbol{0}, \tilde{\boldsymbol{K}}) \tag{2.23}$$

where $\tilde{\boldsymbol{K}} \doteq \sigma_\mathrm{p}^2 \tilde{\boldsymbol{\Phi}} \tilde{\boldsymbol{\Phi}}^\top$. Adding the label noise yields

$$\tilde{\boldsymbol{y}} \mid \boldsymbol{X}, \boldsymbol{x}^\star \sim \mathcal{N}(\boldsymbol{0}, \tilde{\boldsymbol{K}} + \sigma_\mathrm{n}^2 \boldsymbol{I}). \tag{2.24}$$

Finally, we can conclude from the closedness of Gaussian random vectors under conditional distributions (1.53) that the predictive posterior $y^\star \mid \boldsymbol{x}^\star, \boldsymbol{X}, \boldsymbol{y}$ follows again a normal distribution. We will do a full derivation of the posterior and predictive posterior in Section 4.1.

### 2.4.2 *Efficient Polynomial Regression*

But how does the kernel trick address our concerns about efficiency raised in Section 2.3? After all, computing the kernel for a feature space of dimension $e$ still requires computing sums of length $e$ which is prohibitive when $e$ is large. The kernel trick opens up a couple of new doors for us:

1. For certain feature transformations $\boldsymbol{\phi}$, we may be able to find an easier to compute expression equivalent to $\boldsymbol{\phi}(\boldsymbol{x})^\top \boldsymbol{\phi}(\boldsymbol{x}')$.
2. If this is not possible, we could approximate the inner product by an easier to compute expression.
3. Or, alternatively, we may decide not to care very much about the exact feature transformation and simply experiment with kernels that induce *some* feature space (which may even be infinitely dimensional).

We will explore the third approach when we revisit kernels in Section 4.3. A polynomial feature transformation can be computed efficiently in closed-form.

**Fact 2.4.** *For the polynomial feature transformation $\boldsymbol{\phi}$ up to degree $m$ from Example 2.3, it can be shown that up to constant factors,*

$$\boldsymbol{\phi}(\boldsymbol{x})^\top \boldsymbol{\phi}(\boldsymbol{x}') = (1 + \boldsymbol{x}^\top \boldsymbol{x}')^m. \tag{2.25}$$

For example, for input dimension 2, the kernel $(1 + \boldsymbol{x}^\top \boldsymbol{x}')^2$ corresponds to the feature vector $\boldsymbol{\phi}(\boldsymbol{x}) = [1 \;\; \sqrt{2}x_1 \;\; \sqrt{2}x_2 \;\; \sqrt{2}x_1x_2 \;\; x_1^2 \;\; x_2^2]^\top$.

## *Discussion*

We have explored a probabilistic perspective on linear models, and seen that classical approaches such as least squares and ridge regression can be interpreted as approximate probabilistic inference. We then saw that we can even perform *exact* probabilistic inference efficiently if we adopt a Gaussian prior and Gaussian noise assumption. These are already powerful tools, which are often applied also to nonlinear models if we treat the latent feature space — which was either human-designed or learned via deep learning — as fixed. In the next chapter, we will digress briefly from the storyline on "learning" to see how we can adopt a similar probabilistic perspective to track latent states over time. Then, in Chapter 4, we will see how we can use the function-space view and kernel trick to learn flexible nonlinear models with exact probabilistic inference, without ever explicitly representing the feature space.

## *Problems*

**2.1. Closed-form linear regression.**

1. Derive the unique solutions to least squares and ridge regression from Equations (2.4) and (2.5).
2. For an $n \times m$ matrix $\boldsymbol{A}$ and vector $\boldsymbol{x} \in \mathbb{R}^m$, we call $\boldsymbol{\Pi}_{\boldsymbol{A}}\boldsymbol{x}$ the orthogonal projection of $\boldsymbol{x}$ onto $\operatorname{span}\{\boldsymbol{A}\} = \{\boldsymbol{A}\boldsymbol{x}' \mid \boldsymbol{x}' \in \mathbb{R}^m\}$. In particular, an orthogonal projection satisfies $\boldsymbol{x} - \boldsymbol{\Pi}_{\boldsymbol{A}}\boldsymbol{x} \perp \boldsymbol{A}\boldsymbol{x}'$ for all $\boldsymbol{x}' \in \mathbb{R}^m$. Show that $\hat{\boldsymbol{w}}_{\text{ls}}$ from Equation (2.4) is such that $\boldsymbol{X}\hat{\boldsymbol{w}}_{\text{ls}}$ is the unique closest point to $\boldsymbol{y}$ on $\operatorname{span}\{\boldsymbol{X}\}$, i.e., it satisfies $\boldsymbol{X}\hat{\boldsymbol{w}}_{\text{ls}} = \boldsymbol{\Pi}_{\boldsymbol{X}}\boldsymbol{y}$.

**2.2. MLE of noise variance.**

Show that the MLE of $\sigma_{\text{n}}^2$ given fixed weights $\boldsymbol{w}$ is

$$\hat{\sigma}_n^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2. \tag{2.26}$$

**2.3. Variance of least squares around training data.**

Show that the variance of a prediction at the point $[1 \;\; x^\star]^\top$ is smallest when $x^\star$ is the mean of the training data. More formally, show that if inputs are of the form $\boldsymbol{x}_i = [1 \;\; x_i]^\top$ where $x_i \in \mathbb{R}$ and $\hat{\boldsymbol{w}}_{\text{ls}}$ is the least squares estimate, then $\operatorname{Var}\left[y^\star \mid [1 \;\; x^\star]^\top, \hat{\boldsymbol{w}}_{\text{ls}}\right]$ is minimized for $x^\star = \frac{1}{n}\sum_{i=1}^{n} x_i$.

**2.4. Bayesian linear regression.**

Suppose you are given the following observations

$$\boldsymbol{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 2 & 1 \\ 2 & 2 \end{bmatrix}, \quad \boldsymbol{y} = \begin{bmatrix} 2.4 \\ 4.3 \\ 3.1 \\ 4.9 \end{bmatrix}$$

and assume the data follows a linear model with homoscedastic noise $\mathcal{N}(0, \sigma_{\mathrm{n}}^2)$ where $\sigma_{\mathrm{n}}^2 = 0.1$.

1. Find the maximum likelihood estimate $\hat{\boldsymbol{w}}_{\mathrm{MLE}}$ given the data.
2. Now assume that we have a prior $p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}; \boldsymbol{0}, \sigma_{\mathrm{p}}^2 \boldsymbol{I})$ with $\sigma_{\mathrm{p}}^2 = 0.05$. Find the MAP estimate $\hat{\boldsymbol{w}}_{\mathrm{MAP}}$ given the data and the prior.
3. Use the posterior $p(\boldsymbol{w} \mid \boldsymbol{X}, \boldsymbol{y})$ to get a posterior prediction for the label $y^\star$ at $\boldsymbol{x}^\star = [3 \quad 3]^\top$. Report the mean and the variance of this prediction.
4. How would you have to change the prior $p(\boldsymbol{w})$ such that

$$\hat{\boldsymbol{w}}_{\mathrm{MAP}} \to \hat{\boldsymbol{w}}_{\mathrm{MLE}}?$$

**2.5. Online Bayesian linear regression.**

1. Can you design an algorithm that updates the posterior (as opposed to recalculating it from scratch using Equation (2.10c)) in a smarter way? The requirement is that the memory should not grow as $O(t)$.
2. If $d$ is large, computing the inverse every round is very expensive. Can you use the recursive structure you found in the previous question to bring down the computational complexity of every round to $O(d^2)$?

The resulting efficient online algorithm is known as *online Bayesian linear regression*.

**2.6. Aleatoric and epistemic uncertainty of BLR.**

Prove for Bayesian linear regression that $\boldsymbol{x}^{\star\top} \boldsymbol{\Sigma} \boldsymbol{x}^\star$ is the epistemic uncertainty and $\sigma_{\mathrm{n}}^2$ the aleatoric uncertainty in $y^\star$ under the decomposition of Equation (2.18).

**2.7. Hyperpriors.**

We consider a dataset $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ of size $n$, where $\boldsymbol{x}_i \in \mathbb{R}^d$ denotes the feature vector and $y_i \in \mathbb{R}$ denotes the label of the $i$-th data point. Let $\varepsilon_i$ be i.i.d. samples from the Gaussian distribution $\mathcal{N}(0, \lambda^{-1})$ for a given $\lambda > 0$. We collect the labels in a vector $\boldsymbol{y} \in \mathbb{R}^n$, the features in a matrix $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, and the noise in a vector $\boldsymbol{\varepsilon} \in \mathbb{R}^n$. The labels are generated according to $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{w} + \boldsymbol{\varepsilon}$.

To perform Bayesian Linear Regression, we consider the prior distribution over the parameter vector $\boldsymbol{w}$ to be $\mathcal{N}(\boldsymbol{\mu}, \lambda^{-1}\boldsymbol{I}_d)$, where $\boldsymbol{I}_d$ denotes the $d$-dimensional identity matrix and $\boldsymbol{\mu} \in \mathbb{R}^d$ is a hyperparameter.

1. Given this Bayesian data model, what is the conditional covariance matrix $\boldsymbol{\Sigma}_{\boldsymbol{y}} \doteq \mathrm{Var}[\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\mu}, \lambda]$?
2. Calculate the maximum likelihood estimate of the hyperparameter $\boldsymbol{\mu}$.
3. Since we are unsure about the hyperparameter $\boldsymbol{\mu}$, we decide to model our uncertainty about $\boldsymbol{\mu}$ by placing the "*hyperprior*" $\boldsymbol{\mu} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_d)$. Is the posterior distribution $p(\boldsymbol{\mu} \mid \boldsymbol{X}, \boldsymbol{y}, \lambda)$ a Gaussian distribution? If yes, what are its mean vector and covariance matrix?
4. What is the posterior distribution $p(\lambda \mid \boldsymbol{X}, \boldsymbol{y}, \boldsymbol{\mu})$?
   *Hint: For any $a \in \mathbb{R}, \boldsymbol{A} \in \mathbb{R}^{n \times n}$ it holds that* $\det(a\boldsymbol{A}) = a^n \det(\boldsymbol{A})$.

# 3

# *Filtering*

Before we continue in Chapter 4 with the function-space view of regression, we want to look at a seemingly different but very related problem. We will study Bayesian learning and inference in the *state space model*, where we want to keep track of the state of an agent over time based on noisy observations. In this model, we have a sequence of (hidden) states $(\mathbf{X}_t)_{t \in \mathbb{N}_0}$ where $\mathbf{X}_t$ is in $\mathbb{R}^d$ and a sequence of observations $(\mathbf{Y}_t)_{t \in \mathbb{N}_0}$ where $\mathbf{Y}_t$ is in $\mathbb{R}^m$.

The process of keeping track of the state using noisy observations is also known as *Bayesian filtering* or *recursive Bayesian estimation*. Figure 3.1 illustrates this process, where an agent perceives the current state of the world and then updates its beliefs about the state based on this observation.

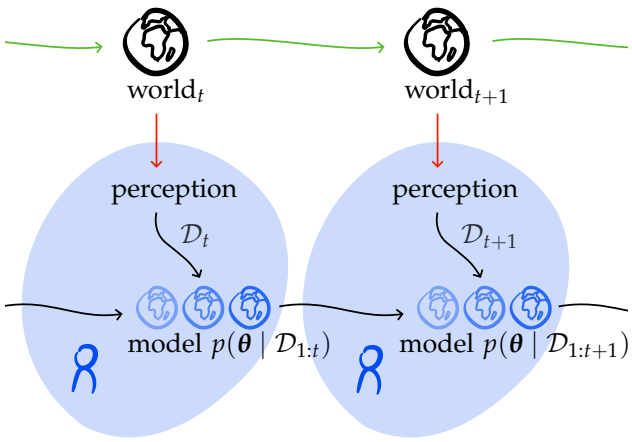



Figure 3.1: Schematic view of Bayesian filtering: An agent perceives the current state of the world and updates its belief accordingly.

We will discuss Bayesian filtering more broadly in the next section. A Kalman filter is an important special case of a Bayes' filter, which uses a Gaussian prior over the states and conditional linear Gaussians to describe the evolution of states and observations. Analogously to the

previous chapter, we will see that inference in this model is tractable due to the closedness properties of Gaussians.

**Definition 3.1** (Kalman filter)**.** A *Kalman filter* is specified by a Gaussian prior over the states,

$$\mathbf{X}_0 \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}), \tag{3.1}$$

and a conditional linear Gaussian *motion model* and *sensor model*,

$$\mathbf{X}_{t+1} \doteq \boldsymbol{F}\mathbf{X}_t + \boldsymbol{\varepsilon}_t \qquad \boldsymbol{F} \in \mathbb{R}^{d\times d}, \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_x), \tag{3.2}$$

$$\mathbf{Y}_t \doteq \boldsymbol{H}\mathbf{X}_t + \boldsymbol{\eta}_t \qquad \boldsymbol{H} \in \mathbb{R}^{m\times d}, \boldsymbol{\eta}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_y), \tag{3.3}$$

respectively. The motion model is sometimes also called *transition model* or *dynamics model*. Crucially, Kalman filters assume that $\boldsymbol{F}$ and $\boldsymbol{H}$ are known. In general, $\boldsymbol{F}$ and $\boldsymbol{H}$ may depend on $t$. Also, $\boldsymbol{\varepsilon}$ and $\boldsymbol{\eta}$ may have a non-zero mean, commonly called a "drift".

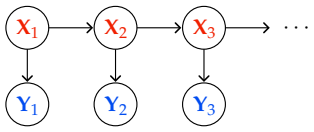



Figure 3.2: Directed graphical model of a Kalman filter with hidden states $\mathbf{X}_t$ and observables $\mathbf{Y}_t$.

Because Kalman filters use conditional linear Gaussians, which we have already seen in Equation (1.55d), their joint distribution (over all variables) is also Gaussian. This means that predicting the future states of a Kalman filter is simply inference with multivariate Gaussians. In Bayesian filtering, however, we do not only want to make predictions occasionally. In Bayesian filtering, we want to *keep track* of states, that is, predict the current state of an agent online.[1] To do this efficiently, we need to update our *belief* about the state of the agent recursively, similarly to our recursive Bayesian updates in Bayesian linear regression (see Section 2.1.3).

[1] Here, *online* is common terminology to say that we want to perform inference at time $t$ without exposure to times $t+1, t+2, \ldots$, so in "real-time".

From the directed graphical model of a Kalman filter shown in Figure 3.2, we can immediately gather the following conditional independence relations,[2]

$$\mathbf{X}_{t+1} \perp \mathbf{X}_{1:t-1}, \mathbf{Y}_{1:t-1} \mid \mathbf{X}_t, \tag{3.4}$$

$$\mathbf{Y}_t \perp \mathbf{X}_{1:t-1} \mid \mathbf{X}_t \tag{3.5}$$

$$\mathbf{Y}_t \perp \mathbf{Y}_{1:t-1} \mid \mathbf{X}_{t-1}. \tag{3.6}$$

[2] Alternatively, they follow from the definition of the motion and sensor models as linear updates.

The first conditional independence property is also known as the *Markov property*, which we will return to later in our discussion of Markov chains and Markov decision processes. This characterization of the Kalman filter, yields the following factorization of the joint distribution:

$$\begin{aligned} p(\boldsymbol{x}_{1:t}, \boldsymbol{y}_{1:t}) &= \prod_{i=1}^{t} p(\boldsymbol{x}_i \mid \boldsymbol{x}_{1:i-1}) p(\boldsymbol{y}_i \mid \boldsymbol{x}_{1:t}, \boldsymbol{y}_{1:i-1}) && \text{using the product rule (1.11)} \\ &= p(\boldsymbol{x}_1) p(\boldsymbol{y}_1 \mid \boldsymbol{x}_1) \prod_{i=2}^{t} p(\boldsymbol{x}_i \mid \boldsymbol{x}_{i-1}) p(\boldsymbol{y}_i \mid \boldsymbol{x}_i). && \text{using the conditional independence properties from (3.4), (3.5), and (3.6)} \end{aligned} \tag{3.7}$$

## 3.1 Conditioning and Prediction

We can describe Bayesian filtering by the following recursive scheme with the two phases, *conditioning* (also called "update") and *prediction*:

**Algorithm 3.2:** Bayesian filtering

start with a prior over initial states $p(\boldsymbol{x}_0)$
**for** $t = 1$ **to** $\infty$ **do**
  assume we have $p(\boldsymbol{x}_t \mid \boldsymbol{y}_{1:t-1})$
  **conditioning**: compute $p(\boldsymbol{x}_t \mid \boldsymbol{y}_{1:t})$ using the new observation $\boldsymbol{y}_t$
  **prediction**: compute $p(\boldsymbol{x}_{t+1} \mid \boldsymbol{y}_{1:t})$

Let us consider the conditioning step first:

$$\begin{aligned} p(\boldsymbol{x}_t \mid \boldsymbol{y}_{1:t}) &= \frac{1}{Z} p(\boldsymbol{x}_t \mid \boldsymbol{y}_{1:t-1}) p(\boldsymbol{y}_t \mid \boldsymbol{x}_t, \boldsymbol{y}_{1:t-1}) && \text{using Bayes' rule (1.45)} \\ &= \frac{1}{Z} p(\boldsymbol{x}_t \mid \boldsymbol{y}_{1:t-1}) p(\boldsymbol{y}_t \mid \boldsymbol{x}_t). \quad (3.8) && \text{using the conditional independence structure (3.6)} \end{aligned}$$

For the prediction step, we obtain,

$$\begin{aligned} p(\boldsymbol{x}_{t+1} \mid \boldsymbol{y}_{1:t}) &= \int p(\boldsymbol{x}_{t+1}, \boldsymbol{x}_t \mid \boldsymbol{y}_{1:t}) \, d\boldsymbol{x}_t && \text{using the sum rule (1.7)} \\ &= \int p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{y}_{1:t}) p(\boldsymbol{x}_t \mid \boldsymbol{y}_{1:t}) \, d\boldsymbol{x}_t && \text{using the product rule (1.11)} \\ &= \int p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t) p(\boldsymbol{x}_t \mid \boldsymbol{y}_{1:t}) \, d\boldsymbol{x}_t. \quad (3.9) && \text{using the conditional independence structure (3.4)} \end{aligned}$$

In general, these distributions can be very complicated, but for Gaussians (i.e., Kalman filters) they can be expressed in closed-form.

**Remark 3.3: Bayesian smoothing**

*Bayesian smoothing* is a closely related task to Bayesian filtering. While Bayesian filtering methods estimate the current state based only on observations obtained before and at the current time step, Bayesian smoothing computes the distribution of $\mathbf{X}_k \mid \boldsymbol{y}_{1:t}$ where $t > k$. That is Bayesian smoothing estimates $\mathbf{X}_k$ based on data until *and beyond* time $k$. Note that if $k = t$, then Bayesian smoothing coincides with Bayesian filtering.

Analogously to Equation (3.8),

$$p(\boldsymbol{x}_k \mid \boldsymbol{y}_{1:t}) \propto p(\boldsymbol{x}_k \mid \boldsymbol{y}_{1:k}) p(\boldsymbol{y}_{k+1:t} \mid \boldsymbol{x}_k). \quad (3.10)$$

If we assume a Gaussian prior and conditional Gaussian transition and dynamics models (this is called *Kalman smoothing*), then

by the closedness properties of Gaussians, $\mathbf{X}_k \mid \boldsymbol{y}_{1:t}$ is a Gaussian. Indeed, all terms of Equation (3.10) are Gaussian PDFs and as seen in Equation (1.75), the product of two Gaussian PDFs is again proportional to a Gaussian PDF.

The first term, $\mathbf{X}_k \mid \boldsymbol{y}_{1:k}$, is the marginal posterior of the hidden states of the Kalman filter which can be obtained with Bayesian filtering.

By conditioning on $\mathbf{X}_{k+1}$, we have for the second term,

$$
\begin{aligned}
p(\boldsymbol{y}_{k+1:t} \mid \boldsymbol{x}_k) &= \int p(\boldsymbol{y}_{k+1:t} \mid \boldsymbol{x}_k, \boldsymbol{x}_{k+1}) p(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k) \, d\boldsymbol{x}_{k+1} && \text{using the sum rule (1.7) and product rule (1.11)} \\
&= \int p(\boldsymbol{y}_{k+1:t} \mid \boldsymbol{x}_{k+1}) p(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k) \, d\boldsymbol{x}_{k+1} && \text{using the conditional independence structure (3.5)} \\
&= \int p(\boldsymbol{y}_{k+1} \mid \boldsymbol{x}_{k+1}) p(\boldsymbol{y}_{k+2:t} \mid \boldsymbol{x}_{k+1}) p(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k) \, d\boldsymbol{x}_{k+1} && \text{using the conditional independence structure (3.6)}
\end{aligned}
\tag{3.11}
$$

Let us have a look at the terms in the product:

- $p(\boldsymbol{y}_{k+1} \mid \boldsymbol{x}_{k+1})$ is obtained from the sensor model,
- $p(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k)$ is obtained from the transition model, and
- $p(\boldsymbol{y}_{k+2:t} \mid \boldsymbol{x}_{k+1})$ can be computed recursively backwards in time.

This recursion results in linear equations resembling a Kalman filter running backwards in time.

Thus, in the setting of Kalman smoothing, both factors of Equation (3.10) can be computed efficiently: one using a (forward) Kalman filter; the other using a "backward" Kalman filter. More concretely, in time $O(t)$, we can compute the two factors for all $k \in [t]$. This approach is known as *two-filter smoothing* or the *forward-backward algorithm*.

## 3.2 Kalman Filters

Let us return to the setting of Kalman filters where priors and likelihoods are Gaussian. Here, we will see that the update and prediction steps can be computed in closed form.

### 3.2.1 Conditioning

The conditioning operation in Kalman filters is also called the Kalman update. Before introducing the general Kalman update, let us consider a simpler example:

**Example 3.4: Random walk in 1d**

We use the simple motion and sensor models,[3]

$$X_{t+1} \mid x_t \sim \mathcal{N}(x_t, \sigma_x^2), \tag{3.12a}$$
$$Y_t \mid x_t \sim \mathcal{N}(x_t, \sigma_y^2). \tag{3.12b}$$

Let $X_t \mid y_{1:t} \sim \mathcal{N}(\mu_t, \sigma_t^2)$ be our belief at time $t$. It can be shown that Bayesian filtering yields the belief $X_{t+1} \mid y_{1:t+1} \sim \mathcal{N}(\mu_{t+1}, \sigma_{t+1}^2)$ at time $t+1$ where (?)

$$\mu_{t+1} \doteq \frac{\sigma_y^2 \mu_t + (\sigma_t^2 + \sigma_x^2) y_{t+1}}{\sigma_t^2 + \sigma_x^2 + \sigma_y^2}, \quad \sigma_{t+1}^2 \doteq \frac{(\sigma_t^2 + \sigma_x^2)\sigma_y^2}{\sigma_t^2 + \sigma_x^2 + \sigma_y^2}. \tag{3.13}$$

Although looking intimidating at first, this update has a very natural interpretation. Let us define the following quantity,

$$\lambda \doteq \frac{\sigma_t^2 + \sigma_x^2}{\sigma_t^2 + \sigma_x^2 + \sigma_y^2} = 1 - \frac{\sigma_y^2}{\sigma_t^2 + \sigma_x^2 + \sigma_y^2} \in [0, 1]. \tag{3.14}$$

Using $\lambda$, we can write the updated mean as a convex combination of the previous mean and the observation,

$$\mu_{t+1} = (1 - \lambda)\mu_t + \lambda y_{t+1} \tag{3.15}$$
$$= \mu_t + \lambda(y_{t+1} - \mu_t). \tag{3.16}$$

Intuitively, $\lambda$ is a form of "gain" that influences how much of the new information should be incorporated into the updated mean. For this reason, $\lambda$ is also called *Kalman gain*.

The updated variance can similarly be rewritten,

$$\sigma_{t+1}^2 = \lambda \sigma_y^2 = (1 - \lambda)(\sigma_t^2 + \sigma_x^2). \tag{3.17}$$

In particular, observe that if $\mu_t = y_{t+1}$ (i.e., we observe our prediction), we have $\mu_{t+1} = \mu_t$ as there is no new information. Similarly, for $\sigma_y^2 \to \infty$ (i.e., we do not trust our observations), we have

$$\lambda \to 0, \quad \mu_{t+1} = \mu_t, \quad \sigma_{t+1}^2 = \sigma_t^2 + \sigma_x^2.$$

In contrast, for $\sigma_y^2 \to 0$, we have

$$\lambda \to 1, \quad \mu_{t+1} = y_{t+1}, \quad \sigma_{t+1}^2 = 0.$$

The general formulas for the *Kalman update* follow the same logic as in the above example of a one-dimensional random walk. Given the

[3] This corresponds to $\boldsymbol{F} = \boldsymbol{H} = \boldsymbol{I}$ and a drift of 0.

**Problem 3.1**

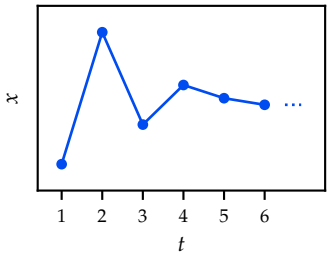



Figure 3.3: Hidden states during a random walk in one dimension.

prior belief $\mathbf{X}_t \mid \boldsymbol{y}_{1:t} \sim \mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t)$, we have

$$\mathbf{X}_{t+1} \mid \boldsymbol{y}_{1:t+1} \sim \mathcal{N}(\boldsymbol{\mu}_{t+1}, \boldsymbol{\Sigma}_{t+1}) \quad \text{where} \tag{3.18a}$$

$$\boldsymbol{\mu}_{t+1} \doteq \boldsymbol{F}\boldsymbol{\mu}_t + \boldsymbol{K}_{t+1}(\boldsymbol{y}_{t+1} - \boldsymbol{H}\boldsymbol{F}\boldsymbol{\mu}_t), \tag{3.18b}$$

$$\boldsymbol{\Sigma}_{t+1} \doteq (\boldsymbol{I} - \boldsymbol{K}_{t+1}\boldsymbol{H})(\boldsymbol{F}\boldsymbol{\Sigma}_t\boldsymbol{F}^\top + \boldsymbol{\Sigma}_x). \tag{3.18c}$$

Hereby, $\boldsymbol{K}_{t+1}$ is the *Kalman gain*,

$$\boldsymbol{K}_{t+1} \doteq (\boldsymbol{F}\boldsymbol{\Sigma}_t\boldsymbol{F}^\top + \boldsymbol{\Sigma}_x)\boldsymbol{H}^\top(\boldsymbol{H}(\boldsymbol{F}\boldsymbol{\Sigma}_t\boldsymbol{F}^\top + \boldsymbol{\Sigma}_x)\boldsymbol{H}^\top + \boldsymbol{\Sigma}_y)^{-1} \in \mathbb{R}^{d\times m}. \tag{3.18d}$$

Note that $\boldsymbol{\Sigma}_t$ and $\boldsymbol{K}_t$ can be computed offline as they are independent of the observation $\boldsymbol{y}_{t+1}$. $\boldsymbol{F}\boldsymbol{\mu}_t$ represents the expected state at time $t+1$, and hence, $\boldsymbol{H}\boldsymbol{F}\boldsymbol{\mu}_t$ corresponds to the expected observation. Therefore, the term $\boldsymbol{y}_{t+1} - \boldsymbol{H}\boldsymbol{F}\boldsymbol{\mu}_t$ measures the error in the predicted observation and the Kalman gain $\boldsymbol{K}_{t+1}$ appears as a measure of relevance of the new observation compared to the prediction.

**Example 3.5: Bayesian linear regression as a Kalman filter**

Even though they arise from a rather different setting, it turns out that Kalman filters are a generalization of Bayesian linear regression! To see this, recall the online Bayesian linear regression algorithm from Section 2.1.3. Observe that by keeping attempting to estimate the (hidden) weights $\boldsymbol{w}^\star$ from sequential noisy observations $y_t$, this algorithm performs Bayesian filtering! Moreover, we have used a Gaussian prior and likelihood. This is precisely the setting of a Kalman filter!

Concretely, we are estimating the constant (i.e., $\boldsymbol{F} = \boldsymbol{I}$, $\boldsymbol{\varepsilon} = \boldsymbol{0}$) hidden state $\boldsymbol{x}_t = \boldsymbol{w}^{(t)}$ with prior $\boldsymbol{w}^{(0)} \sim \mathcal{N}(\boldsymbol{0}, \sigma_\mathrm{p}^2\boldsymbol{I})$.

Our sensor model is time-dependent, since in each iteration we observe a different input $\boldsymbol{x}_t$. Furthermore, we only observe a scalar-valued label $y_t$.[4] Formally, our sensor model is characterized by $\boldsymbol{h}_t = \boldsymbol{x}_t^\top$ and noise $\eta_t = \varepsilon_t$ with $\varepsilon_t \sim \mathcal{N}(0, \sigma_\mathrm{n}^2)$.

You will show in (?) that the Kalman update (3.18) is the online equivalent to computing the posterior of the weights in Bayesian linear regression.

[4] That is, $m = 1$ in our general Kalman filter formulation from above.

**Problem 3.2**

### 3.2.2 *Predicting*

Using now that the marginal posterior of $\mathbf{X}_t$ is a Gaussian due to the closedness properties of Gaussians, we have

$$\mathbf{X}_{t+1} \mid \boldsymbol{y}_{1:t} \sim \mathcal{N}(\hat{\boldsymbol{\mu}}_{t+1}, \hat{\boldsymbol{\Sigma}}_{t+1}), \tag{3.19}$$

and it suffices to compute the prediction mean $\hat{\boldsymbol{\mu}}_{t+1}$ and covariance matrix $\hat{\boldsymbol{\Sigma}}_{t+1}$.

For the mean,

$$\begin{aligned}
\hat{\boldsymbol{\mu}}_{t+1} &= \mathbb{E}[\boldsymbol{x}_{t+1} \mid \boldsymbol{y}_{1:t}] \\
&= \mathbb{E}[\boldsymbol{F}\boldsymbol{x}_t + \boldsymbol{\varepsilon}_t \mid \boldsymbol{y}_{1:t}] && \text{using the motion model (3.2)} \\
&= \boldsymbol{F}\mathbb{E}[\boldsymbol{x}_t \mid \boldsymbol{y}_{1:t}] && \text{using linearity of expectation (1.20) and } \mathbb{E}[\boldsymbol{\varepsilon}_t] = \mathbf{0} \\
&= \boldsymbol{F}\boldsymbol{\mu}_t. && \text{using the mean of the Kalman update} \qquad (3.20)
\end{aligned}$$

For the covariance matrix,

$$\begin{aligned}
\hat{\boldsymbol{\Sigma}}_{t+1} &= \mathbb{E}\left[(\boldsymbol{x}_{t+1} - \hat{\boldsymbol{\mu}}_{t+1})(\boldsymbol{x}_{t+1} - \hat{\boldsymbol{\mu}}_{t+1})^\top \;\middle|\; \boldsymbol{y}_{1:t}\right] && \text{using the definition of the covariance matrix (1.36)} \\
&= \boldsymbol{F}\mathbb{E}\left[(\boldsymbol{x}_t - \boldsymbol{\mu}_t)(\boldsymbol{x}_t - \boldsymbol{\mu}_t)^\top \;\middle|\; \boldsymbol{y}_{1:t}\right]\boldsymbol{F}^\top + \mathbb{E}\left[\boldsymbol{\varepsilon}_t\boldsymbol{\varepsilon}_t^\top\right] && \text{using (3.20), the motion model (3.2) and that } \boldsymbol{\varepsilon}_t \text{ is independent of the observations} \\
&= \boldsymbol{F}\boldsymbol{\Sigma}_t\boldsymbol{F}^\top + \boldsymbol{\Sigma}_x. && \qquad (3.21)
\end{aligned}$$

**Optional Readings**

Kalman filters and related models are often called *temporal models*. For a broader look at such models, read chapter 15 of "Artificial intelligence: a modern approach" (Russell and Norvig, 2002).

## *Discussion*

In this chapter, we have introduced Kalman filters as a special case of probabilistic filtering where probabilistic inference can be performed in closed form. Similarly to Bayesian linear regression, probabilistic inference is tractable due to assuming Gaussian priors and likelihoods. Indeed, learning linear models and Kalman filters are very closely related as seen in Example 3.5, and we will further explore this relationship in Problem 4.3. We will refer back to filtering in the second part of this manuscript when we discuss sequential decision-making with partial observability of the state space. Next, we return to the storyline on "learning" using exact probabilistic inference.

## *Problems*

**3.1. Kalman update.**

Derive the predictive distribution $X_{t+1} \mid y_{1:t+1}$ (3.13) of the Kalman filter described in the above example using your knowledge about multivariate Gaussians from Section 1.2.3.
*Hint: First compute the predictive distribution $X_{t+1} \mid y_{1:t}$.*

**3.2. Bayesian linear regression as a Kalman filter.**

Recall the specific Kalman filter from Example 3.5. With this model the Kalman update (3.18) simplifies to

$$\boldsymbol{k}_t = \frac{\boldsymbol{\Sigma}_{t-1}\boldsymbol{x}_t}{\boldsymbol{x}_t^\top \boldsymbol{\Sigma}_{t-1}\boldsymbol{x}_t + \sigma_\mathrm{n}^2}, \tag{3.22a}$$

$$\boldsymbol{\mu}_t = \boldsymbol{\mu}_{t-1} + \boldsymbol{k}_t(y_t - \boldsymbol{x}_t^\top \boldsymbol{\mu}_{t-1}), \tag{3.22b}$$

$$\boldsymbol{\Sigma}_t = \boldsymbol{\Sigma}_{t-1} - \boldsymbol{k}_t \boldsymbol{x}_t^\top \boldsymbol{\Sigma}_{t-1}, \tag{3.22c}$$

with $\boldsymbol{\mu}_0 = \mathbf{0}$ and $\boldsymbol{\Sigma}_0 = \sigma_\mathrm{p}^2 \boldsymbol{I}$. Note that the Kalman gain $\boldsymbol{k}_t$ is a vector in $\mathbb{R}^d$. We assume $\sigma_\mathrm{n}^2 = \sigma_\mathrm{p}^2 = 1$ for simplicity.

Prove by induction that the $(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t)$ produced by the Kalman update are equivalent to $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ from the posterior of Bayesian linear regression (2.10) given $\boldsymbol{x}_{1:t}, y_{1:t}$. You may use that $\boldsymbol{\Sigma}_t^{-1}\boldsymbol{k}_t = \boldsymbol{x}_t$.

*Hint: In the inductive step, first prove the equivalence of $\boldsymbol{\Sigma}_t$ and then expand $\boldsymbol{\Sigma}_t^{-1}\boldsymbol{\mu}_t$ to prove the equivalence of $\boldsymbol{\mu}_t$.*

**3.3. Parameter estimation using Kalman filters.**

Suppose that we want to estimate the value of an unknown constant $\pi$ using uncorrelated measurements

$$y_t = \pi + \eta_t, \quad \eta_t \sim \mathcal{N}(0, \sigma_y^2).$$

1. How can this problem be formulated as a Kalman filter? Compute closed form expressions for the Kalman gain and the variance of the estimation error $\sigma_t^2$ in terms of $t$, $\sigma_y^2$, and $\sigma_0^2$.
2. What is the Kalman filter when $t \to \infty$?
3. Suppose that one has no prior assumptions on $\pi$, meaning that $\mu_0 = 0$ and $\sigma_0^2 \to \infty$. Which well-known estimator does the Kalman filter reduce to in this case?

# 4

# *Gaussian Processes*

Let us remember our first attempt from Chapter 2 at scaling up Bayesian linear regression to nonlinear functions. We saw that we can model nonlinear functions by transforming the input space to a suitable higher-dimensional feature space, but found that this approach scales poorly if we require a large number of features. We then found something remarkable: by simply changing our perspective from a weight-space view to a function-space view, we could implement Bayesian linear regression without ever needing to compute the features explicitly. Under the function-space view, the key object describing the class of functions we can model is not the features $\boldsymbol{\phi}(\boldsymbol{x})$, but instead the kernel function which only implicitly defines a feature space. Our key observation in this chapter is that we can therefore stop reasoning about feature spaces, and instead directly work with kernel functions that describe "reasonable" classes of functions.

We are still concerned with the problem of estimating the value of a function $f : \mathcal{X} \to \mathbb{R}$ at arbitrary points $\boldsymbol{x}^\star \in \mathcal{X}$ given training data $\{\boldsymbol{x}_i, y_i\}_{i=1}^n$, where the labels are assumed to be corrupted by homoscedastic Gaussian noise with variance $\sigma_{\mathrm{n}}^2$,

$$y_i = f(\boldsymbol{x}_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_{\mathrm{n}}^2).$$

As in Chapter 2 on Bayesian linear regression, we denote by $\boldsymbol{X}$ the design matrix (collection of training inputs) and by $\boldsymbol{y}$ the vector of training labels. We will represent the unknown function value at a point $\boldsymbol{x} \in \mathcal{X}$ by the random variable $f_{\boldsymbol{x}} \doteq f(\boldsymbol{x})$. The collection of these random variables is then called a Gaussian process if any finite subset of them is jointly Gaussian:

**Definition 4.1** (Gaussian process, GP)**.** A *Gaussian process* is an infinite set of random variables such that any finite number of them are jointly Gaussian and such that they are consistent under marginalization.[1]

[1] That is, if you take a joint distribution for $n$ variables and marginalize out one of them, you should recover the joint distribution for the remaining $n-1$ variables.

The fact that with a Gaussian process, any finite subset of the random variables is jointly Gaussian is the key property allowing us to perform exact probabilistic inference. Intuitively, a Gaussian process can be interpreted as a normal distribution over functions — and is therefore often called an "infinite-dimensional Gaussian".

A Gaussian process is characterized by a *mean function* $\mu : \mathcal{X} \to \mathbb{R}$ and a *covariance function* (or *kernel function*) $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ such that for any set of points $A \doteq \{x_1, \dots, x_m\} \subseteq \mathcal{X}$, we have

$$f_A \doteq [f_{x_1} \ \cdots \ f_{x_m}]^\top \sim \mathcal{N}(\boldsymbol{\mu}_A, \boldsymbol{K}_{AA}) \tag{4.1}$$

where

$$\boldsymbol{\mu}_A \doteq \begin{bmatrix} \mu(x_1) \\ \vdots \\ \mu(x_m) \end{bmatrix}, \quad \boldsymbol{K}_{AA} \doteq \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_m) \\ \vdots & \ddots & \vdots \\ k(x_m, x_1) & \cdots & k(x_m, x_m) \end{bmatrix}. \tag{4.2}$$

We write $f \sim \mathcal{GP}(\mu, k)$. In particular, given a mean function, covariance function, and using the homoscedastic noise assumption,

$$y^\star \mid x^\star \sim \mathcal{N}(\mu(x^\star), k(x^\star, x^\star) + \sigma_{\text{n}}^2), \tag{4.3}$$

with $\sigma^2(x^\star) = k(x^\star, x^\star)$ as the epistemic uncertainty and $\sigma_{\text{n}}^2$ as the aleatoric uncertainty. Commonly, for notational simplicity, the mean function is taken to be zero. Note that for a fixed mean this is not a restriction, as we can simply apply the zero-mean Gaussian process to the difference between the mean and the observations.[2]

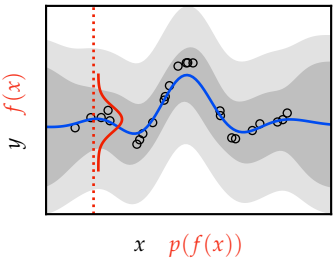



Figure 4.1: A Gaussian process can be interpreted as an infinite-dimensional Gaussian over functions. At any location $x$ in the domain, this yields a distribution over values $f(x)$ shown in red. The blue line corresponds to the MAP estimate (i.e., mean function of the Gaussian process), the dark gray region corresponds to the epistemic uncertainty and the light gray region denotes the additional aleatoric uncertainty.

[2] For alternative ways of representing a mean function, refer to section 2.7 of "Gaussian processes for machine learning" (Williams and Rasmussen, 2006).

## 4.1 Learning and Inference

First, let us look at learning and inference in the context of Gaussian processes. With slight abuse of our previous notation, let us denote the set of observed points by $A \doteq \{x_1, \dots, x_n\}$. Given a prior $f \sim \mathcal{GP}(\mu, k)$ and the noisy observations $y_i = f(x_i) + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma_{\text{n}}^2)$, we can then write the joint distribution of the observations $y_{1:n}$ and the noise-free prediction $f^\star$ at a test point $x^\star$ as

$$\begin{bmatrix} \boldsymbol{y} \\ f^\star \end{bmatrix} \mid x^\star, \boldsymbol{X} \sim \mathcal{N}(\tilde{\boldsymbol{\mu}}, \tilde{\boldsymbol{K}}), \quad \text{where} \tag{4.4}$$

$$\tilde{\boldsymbol{\mu}} \doteq \begin{bmatrix} \boldsymbol{\mu}_A \\ \mu(x^\star) \end{bmatrix}, \quad \tilde{\boldsymbol{K}} \doteq \begin{bmatrix} \boldsymbol{K}_{AA} + \sigma_{\text{n}}^2 \boldsymbol{I} & \boldsymbol{k}_{x^\star, A} \\ \boldsymbol{k}_{x^\star, A}^\top & k(x^\star, x^\star) \end{bmatrix}, \quad \boldsymbol{k}_{x, A} \doteq \begin{bmatrix} k(x, x_1) \\ \vdots \\ k(x, x_n) \end{bmatrix}. \tag{4.5}$$

Deriving the conditional distribution using (1.53), we obtain that the Gaussian process posterior is given by

$$f \mid x_{1:n}, y_{1:n} \sim \mathcal{GP}(\mu', k'), \quad \text{where} \tag{4.6}$$

$$\mu'(\boldsymbol{x}) \doteq \mu(\boldsymbol{x}) + \boldsymbol{k}_{\boldsymbol{x},A}^\top (\boldsymbol{K}_{AA} + \sigma_\mathrm{n}^2 \boldsymbol{I})^{-1} (\boldsymbol{y}_A - \boldsymbol{\mu}_A), \tag{4.7}$$

$$k'(\boldsymbol{x}, \boldsymbol{x}') \doteq k(\boldsymbol{x}, \boldsymbol{x}') - \boldsymbol{k}_{\boldsymbol{x},A}^\top (\boldsymbol{K}_{AA} + \sigma_\mathrm{n}^2 \boldsymbol{I})^{-1} \boldsymbol{k}_{\boldsymbol{x}',A}. \tag{4.8}$$

Observe that analogously to Bayesian linear regression, the posterior covariance can only decrease when conditioning on additional data, and is independent of the observations $y_i$.

We already studied inference in the function-space view of Bayesian linear regression, but did not make the predictive posterior explicit. Using Equation (4.6), the predictive posterior at $\boldsymbol{x}^\star$ is simply

$$f^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n} \sim \mathcal{N}(\mu'(\boldsymbol{x}^\star), k'(\boldsymbol{x}^\star, \boldsymbol{x}^\star)). \tag{4.9}$$

## 4.2 *Sampling*

Often, we are not interested in the full predictive posterior distribution, but merely want to obtain samples of our Gaussian process model. We will briefly examine two approaches.

1. For the first approach, consider a discretized subset of points

   $$\boldsymbol{f} \doteq [f_1, \ldots, f_n]$$

   that we want to sample.[3] Note that $\boldsymbol{f} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{K})$. We have already seen in Equation (1.54) that

   $$\boldsymbol{f} = \boldsymbol{K}^{1/2} \boldsymbol{\varepsilon} + \boldsymbol{\mu} \tag{4.10}$$

   where $\boldsymbol{K}^{1/2}$ is the square root of $\boldsymbol{K}$ and $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ is standard Gaussian noise.[4] However, computing the square root of $\boldsymbol{K}$ takes $O(n^3)$ time.

2. For the second approach, recall the product rule (1.11),

   $$p(f_1, \ldots, f_n) = \prod_{i=1}^{n} p(f_i \mid f_{1:i-1}).$$

   That is the joint distribution factorizes neatly into a product where each factor only depends on the "outcomes" of preceding factors. We can therefore obtain samples one-by-one, each time conditioning on one more observation:

   $$\begin{aligned} f_1 &\sim p(f_1) \\ f_2 &\sim p(f_2 \mid f_1) \\ f_3 &\sim p(f_3 \mid f_1, f_2) \\ &\vdots \end{aligned} \tag{4.11}$$

   This general approach is known as *forward sampling*. Due to the matrix inverse in the formula of the GP posterior (4.6), this approach also takes $O(n^3)$ time.

We will discuss more efficient approximate sampling methods in Section 4.5.

[3] For example, if we want to render the function, the length of this vector could be guided by the screen resolution.

[4] We discuss square roots of matrices in Section A.2.

## 4.3 Kernel Functions

We have seen that kernel functions are the key object describing the class of functions a Gaussian process can model. Depending on the kernel function, the "shape" of functions that are realized from a Gaussian process varies greatly. Let us recap briefly from Section 2.4 what a kernel function is:

**Definition 4.2** (Kernel function)**.** A *kernel function* $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ satisfies

- $k(\boldsymbol{x}, \boldsymbol{x}') = k(\boldsymbol{x}', \boldsymbol{x})$ for any $\boldsymbol{x}, \boldsymbol{x}' \in \mathcal{X}$ (symmetry), and
- $\boldsymbol{K}_{AA}$ is positive semi-definite for any $A \subseteq \mathcal{X}$.

The two defining conditions ensure that for any $A \subseteq \mathcal{X}$, $\boldsymbol{K}_{AA}$ is a valid covariance matrix. We say that a kernel function is *positive definite* if $\boldsymbol{K}_{AA}$ is positive definite for any $A \subseteq \mathcal{X}$.

Intuitively, the kernel function evaluated at locations $\boldsymbol{x}$ and $\boldsymbol{x}'$ describes how $f(\boldsymbol{x})$ and $f(\boldsymbol{x}')$ are related, which we can express formally as

$$k(\boldsymbol{x}, \boldsymbol{x}') = \operatorname{Cov}\big[f(\boldsymbol{x}), f(\boldsymbol{x}')\big]. \tag{4.12}$$

If $\boldsymbol{x}$ and $\boldsymbol{x}'$ are "close", then $f(\boldsymbol{x})$ and $f(\boldsymbol{x}')$ are usually taken to be positively correlated, encoding a "smooth" function.

In the following, we will discuss some of the most common kernel functions, how they can be combined to create "new" kernels, and how we can characterize the class of functions they can model.

### 4.3.1 Common Kernels

First, we look into some of the most commonly used kernels. Often an additional factor $\sigma^2$ (*output scale*) is added, which we assume here to be 1 for simplicity.

1. The *linear kernel* is defined as

$$k(\boldsymbol{x}, \boldsymbol{x}'; \boldsymbol{\phi}) \doteq \boldsymbol{\phi}(\boldsymbol{x})^\top \boldsymbol{\phi}(\boldsymbol{x}') \tag{4.13}$$

where $\boldsymbol{\phi}$ is a nonlinear transformation as introduced in Section 2.3 or the identity.

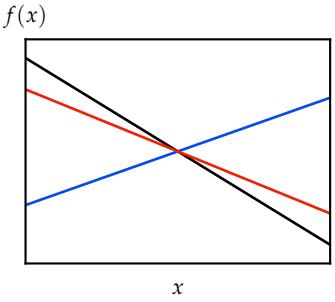



Figure 4.2: Functions sampled according to a Gaussian process with a linear kernel and $\phi = \mathrm{id}$.

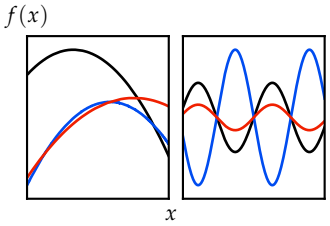



Figure 4.3: Functions sampled according to a Gaussian process with a linear kernel and $\boldsymbol{\phi}(x) = [1, x, x^2]$ (left) and $\phi(x) = \sin(x)$ (right).

**Remark 4.3: GPs with linear kernel and BLR**

A Gaussian process with a linear kernel is equivalent to Bayesian linear regression. This follows directly from the function-space view of Bayesian linear regression (see Section 2.4) and comparing the derived kernel function (2.20) with the definition of the linear kernel (4.13).

2. The *Gaussian kernel* (also known as *squared exponential kernel* or *radial basis function (RBF) kernel*) is defined as

$$k(\boldsymbol{x}, \boldsymbol{x}'; h) \doteq \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{x}'\|_2^2}{2h^2}\right) \tag{4.14}$$

where $h$ is its *length scale*. The larger the length scale $h$, the smoother the resulting functions.[5] Furthermore, it turns out that the feature space (think back to Section 2.4!) corresponding to the Gaussian kernel is "infinitely dimensional", as you will show in (?). So the Gaussian kernel already encodes a function class that we were not able to model under the weight-space view of Bayesian linear regression.

[5] As the length scale is increased, the exponent of the exponential increases, resulting in a higher dependency between locations.

**Problem 4.1**

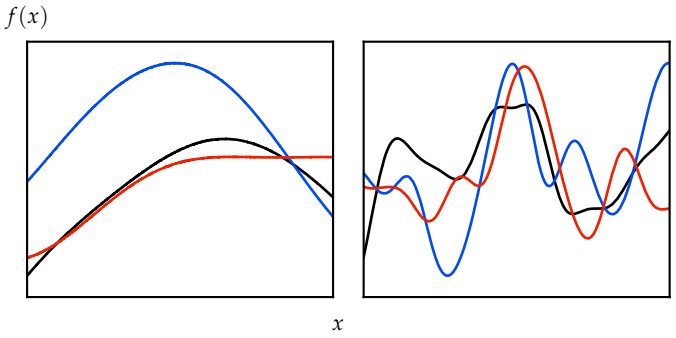



Figure 4.4: Functions sampled according to a Gaussian process with a Gaussian kernel and length scales $h = 5$ (left) and $h = 1$ (right).

3. The *Laplace kernel* (also known as *exponential kernel*) is defined as

$$k(\boldsymbol{x}, \boldsymbol{x}'; h) \doteq \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{x}'\|_2}{h}\right). \tag{4.15}$$

As can be seen in Figure 4.7, samples from a GP with Laplace kernel are non-smooth as opposed to the samples from a GP with Gaussian kernel.

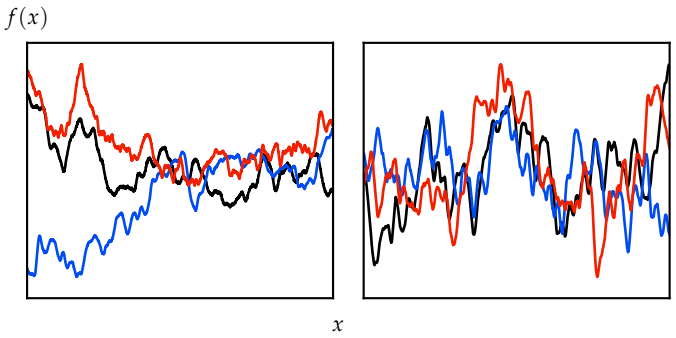



Figure 4.7: Functions sampled according to a Gaussian process with a Laplace kernel and length scales $h = 10\,000$ (left) and $h = 10$ (right).

4. The *Matérn kernel* trades the smoothness of the Gaussian and the Laplace kernels. As such, it is frequently used in practice to model

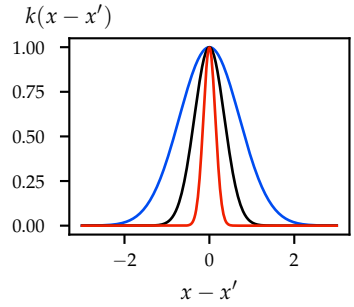



Figure 4.5: Gaussian kernel with length scales $h = 1$, $h = 0.5$, and $h = 0.2$.

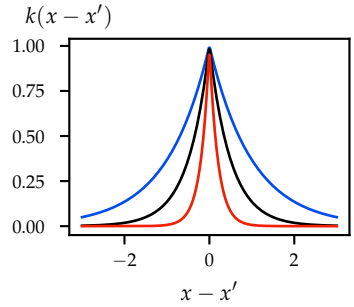



Figure 4.6: Laplace kernel with length scales $h = 1$, $h = 0.5$, and $h = 0.2$.

"real world" functions that are relatively smooth. It is defined as

$$k(\boldsymbol{x}, \boldsymbol{x}'; \nu, h) \doteq \frac{2^{1-\nu}}{\Gamma(\nu)} \left( \frac{\sqrt{2\nu}\, \|\boldsymbol{x} - \boldsymbol{x}'\|_2}{h} \right)^{\nu} K_\nu \left( \frac{\sqrt{2\nu}\, \|\boldsymbol{x} - \boldsymbol{x}'\|_2}{h} \right) \tag{4.16}$$

where $\Gamma$ is the Gamma function, $K_\nu$ the modified Bessel function of the second kind, and $h$ a length scale parameter. For $\nu = 1/2$, the Matérn kernel is equivalent to the Laplace kernel. For $\nu \to \infty$, the Matérn kernel is equivalent to the Gaussian kernel. The resulting functions are $\lceil \nu \rceil - 1$ times mean square differentiable.[6] In particular, GPs with a Gaussian kernel are infinitely many times mean square differentiable whereas GPs with a Laplace kernel are mean square continuous but not mean square differentiable.

[6] Refer to Remark A.12 for the definitions of mean square continuity and differentiability.

### 4.3.2 *Composing Kernels*

Given two kernels $k_1 : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ and $k_2 : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, they can be composed to obtain a new kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ in the following ways:

- $k(\boldsymbol{x}, \boldsymbol{x}') \doteq k_1(\boldsymbol{x}, \boldsymbol{x}') + k_2(\boldsymbol{x}, \boldsymbol{x}')$,
- $k(\boldsymbol{x}, \boldsymbol{x}') \doteq k_1(\boldsymbol{x}, \boldsymbol{x}') \cdot k_2(\boldsymbol{x}, \boldsymbol{x}')$,
- $k(\boldsymbol{x}, \boldsymbol{x}') \doteq c \cdot k_1(\boldsymbol{x}, \boldsymbol{x}')$ for any $c > 0$,
- $k(\boldsymbol{x}, \boldsymbol{x}') \doteq f(k_1(\boldsymbol{x}, \boldsymbol{x}'))$ for any polynomial $f$ with positive coefficients or $f = \exp$.

For example, the additive structure of a function $f(\boldsymbol{x}) \doteq f_1(\boldsymbol{x}) + f_2(\boldsymbol{x})$ can be easily encoded in GP models. Suppose that $f_1 \sim \mathcal{GP}(\mu_1, k_1)$ and $f_2 \sim \mathcal{GP}(\mu_2, k_2)$, then the distribution of the sum of those two functions $f = f_1 + f_2 \sim \mathcal{GP}(\mu_1 + \mu_2, k_1 + k_2)$ is another GP.[7]

[7] We use $f \doteq f_1 + f_2$ to denote the function $f(\cdot) = f_1(\cdot) + f_2(\cdot)$.

Whereas the addition of two kernels $k_1$ and $k_2$ can be thought of as an *OR* operation (i.e., the kernel has high value if either $k_1$ or $k_2$ have high value), the multiplication of $k_1$ and $k_2$ can be thought of as an *AND* operation (i.e., the kernel has high value if both $k_1$ and $k_2$ have high value). For example, the product of two linear kernels results in functions which are quadratic.

As mentioned previously, the constant $c$ of a scaled kernel function $k'(\boldsymbol{x}, \boldsymbol{x}') \doteq c \cdot k(\boldsymbol{x}, \boldsymbol{x}')$ is generally called the *output scale* of a kernel, and it scales the variance $\mathrm{Var}[f(\boldsymbol{x})] = c \cdot k(\boldsymbol{x}, \boldsymbol{x})$ of the predictions $f(\boldsymbol{x})$ from $\mathcal{GP}(\mu, k')$.

**Optional Readings**

For a broader introduction to how kernels can be used and combined to model certain classes of functions, read

- chapter 2 of "Automatic model construction with Gaussian processes" (Duvenaud, 2014) also known as the "kernel cookbook",
- chapter 4 of "Gaussian processes for machine learning" (Williams and Rasmussen, 2006).

### 4.3.3 *Stationarity and Isotropy*

Kernel functions are commonly classified according to two properties:

**Definition 4.4** (Stationarity and isotropy)**.** A kernel $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is called

- *stationary* (or *shift-invariant*) if there exists a function $\tilde{k}$ such that $\tilde{k}(\boldsymbol{x} - \boldsymbol{x}') = k(\boldsymbol{x}, \boldsymbol{x}')$, and
- *isotropic* if there exists a function $\tilde{k}$ such that $\tilde{k}(\|\boldsymbol{x} - \boldsymbol{x}'\|) = k(\boldsymbol{x}, \boldsymbol{x}')$ with $\|\cdot\|$ any norm.

Note that stationarity is a necessary condition for isotropy. In other words, isotropy implies stationarity.

**Example 4.5: Stationarity and isotropy of kernels**

| | stationary | isotropic |
|---|---|---|
| linear kernel | no | no |
| Gaussian kernel | yes | yes |
| $k(\boldsymbol{x}, \boldsymbol{x}') \doteq \exp(-\|\boldsymbol{x} - \boldsymbol{x}'\|_{\boldsymbol{M}}^2)$ where $\boldsymbol{M}$ is positive semi-definite | yes | no |

$\|\cdot\|_{\boldsymbol{M}}$ denotes the Mahalanobis norm induced by matrix $\boldsymbol{M}$

For $\boldsymbol{x}' = \boldsymbol{x}$, stationarity implies that the kernel must only depend on $\boldsymbol{0}$. In other words, a stationary kernel must depend on relative locations only. This is clearly not the case for the linear kernel, which depends on the absolute locations of $\boldsymbol{x}$ and $\boldsymbol{x}'$. Therefore, the linear kernel cannot be isotropic either.

For the Gaussian kernel, isotropy follows immediately from its definition.

The last kernel is clearly stationary by definition, but not isotropic for general matrices $\boldsymbol{M}$. Note that for $\boldsymbol{M} = \boldsymbol{I}$ it is indeed isotropic.

Stationarity encodes the idea that relative location matters more than absolute location: the process "looks the same" no matter where we shift it in the input space. This is often appropriate when we believe the same statistical behavior holds across the entire domain (e.g., no region is special). Isotropy goes one step further by requiring that the kernel depends only on the distance between points, so that all

directions in the space are treated equally. In other words, there is no preferred orientation or axis. This is especially useful in settings where we expect uniform behavior in every direction (as with the Gaussian kernel). Such kernels are simpler to specify and interpret since we only need a single "scale" (like a length scale) rather than multiple parameters or directions.

### 4.3.4 *Reproducing Kernel Hilbert Spaces*

We can characterize the precise class of functions that can be modeled by a Gaussian process with a given kernel function. This corresponding function space is called a *reproducing kernel Hilbert space* (RKHS), and we will discuss it briefly in this section.

Recall that Gaussian processes keep track of a posterior distribution $f \mid \boldsymbol{x}_{1:n}, y_{1:n}$ over functions. We will in fact show later that the corresponding MAP estimate $\hat{f}$ corresponds to the solution to a regularized optimization problem in the RKHS space of functions. This duality is similar to the duality between the MAP estimate of Bayesian linear regression and ridge regression we observed in Chapter 2. So what is the reproducing kernel Hilbert space of a kernel function $k$?

**Definition 4.6** (Reproducing kernel Hilbert space, RKHS)**.** Given a kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, its corresponding *reproducing kernel Hilbert space* is the space of functions $f$ defined as

$$\mathcal{H}_k(\mathcal{X}) \doteq \left\{ f(\cdot) = \sum_{i=1}^{n} \alpha_i k(\boldsymbol{x}_i, \cdot) : n \in \mathbb{N}, \boldsymbol{x}_i \in \mathcal{X}, \alpha_i \in \mathbb{R} \right\}. \tag{4.17}$$

The inner product of the RKHS is defined as

$$\langle f, g \rangle_k \doteq \sum_{i=1}^{n} \sum_{j=1}^{n'} \alpha_i \alpha'_j k(\boldsymbol{x}_i, \boldsymbol{x}'_j), \tag{4.18}$$

where $g(\cdot) = \sum_{j=1}^{n'} \alpha'_j k(\boldsymbol{x}'_j, \cdot)$, and induces the norm $\|f\|_k = \sqrt{\langle f, f \rangle_k}$. You can think of the norm as measuring the "smoothness" or "complexity" of $f$. (?) **Problem 4.4 (2)**

It is straightforward to check that for all $\boldsymbol{x} \in \mathcal{X}$, $k(\boldsymbol{x}, \cdot) \in \mathcal{H}_k(\mathcal{X})$. Moreover, the RKHS inner product $\langle \cdot, \cdot \rangle_k$ satisfies for all $\boldsymbol{x} \in \mathcal{X}$ and $f \in \mathcal{H}_k(\mathcal{X})$ that $f(\boldsymbol{x}) = \langle f(\cdot), k(\boldsymbol{x}, \cdot) \rangle_k$ which is also known as the *reproducing property* (?). That is, evaluations of RKHS functions $f$ are inner products in $\mathcal{H}_k(\mathcal{X})$ parameterized by the "feature map" $k(\boldsymbol{x}, \cdot)$. **Problem 4.4 (1)**

The *representer theorem* (Schölkopf et al., 2001) characterizes the solution to regularized optimization problems in RKHSs:

**Theorem 4.7** (Representer theorem)**.** (?) *Let $k$ be a kernel and let $\lambda > 0$. For $f \in \mathcal{H}_k(\mathcal{X})$ and training data $\{(\boldsymbol{x}_i, f(\boldsymbol{x}_i))\}_{i=1}^n$, let $\mathcal{L}(f(\boldsymbol{x}_1), \ldots, f(\boldsymbol{x}_n)) \in$* **Problem 4.5**

$\mathbb{R} \cup \{\infty\}$ *denote any loss function which depends on* $f$ *only through its evaluation at the training points. Then, any minimizer*

$$\hat{f} \in \operatorname*{arg\,min}_{f \in \mathcal{H}_k(\mathcal{X})} \mathcal{L}(f(\boldsymbol{x}_1), \ldots, f(\boldsymbol{x}_n)) + \lambda \left\|f\right\|_k^2 \tag{4.19}$$

*admits a representation of the form*

$$\hat{f}(\boldsymbol{x}) = \hat{\boldsymbol{\alpha}}^\top \boldsymbol{k}_{\boldsymbol{x},\{\boldsymbol{x}_i\}_{i=1}^n} = \sum_{i=1}^n \hat{\alpha}_i k(\boldsymbol{x}, \boldsymbol{x}_i) \quad \textit{for some } \hat{\boldsymbol{\alpha}} \in \mathbb{R}^n. \tag{4.20}$$

This statement is remarkable: the solutions to general regularized optimization problems over the generally infinite-dimensional space of functions $\mathcal{H}_k(\mathcal{X})$ can be represented as a linear combination of the kernel functions evaluated at the training points. The representer theorem can be used to show that the MAP estimate of a Gaussian process corresponds to the solution of a regularized linear regression problem in the RKHS of the kernel function, namely, (?)

**Problem 4.6**

$$\hat{f} \doteq \operatorname*{arg\,min}_{f \in \mathcal{H}_k(\mathcal{X})} -\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, f) + \frac{1}{2} \left\|f\right\|_k^2. \tag{4.21}$$

Here, the first term corresponds to the likelihood, measuring the "quality of fit". The regularization term limits the "complexity" of $\hat{f}$. Regularization is necessary to prevent overfitting since in an expressive RKHSs, there may be many functions that interpolate the training data perfectly. This shows the close link between Gaussian process regression and Bayesian linear regression, with the kernel function $k$ generalizing the inner product of feature maps to feature spaces of possibly "infinite dimensionality". Because solutions can be represented as linear combinations of kernel evaluations at the training points, Gaussian processes remain computationally tractable even though they can model functions over "infinite-dimensional" feature spaces.

## 4.4 Model Selection

We have not yet discussed how to pick the hyperparameters $\boldsymbol{\theta}$ (e.g., parameters of kernels). A common technique in supervised learning is to select hyperparameters $\boldsymbol{\theta}$, such that the resulting function estimate $\hat{f}_{\boldsymbol{\theta}}$ leads to the most accurate predictions on hold-out validation data. After reviewing this approach, we contrast it with a probabilistic approach to model selection, which avoids using point estimates of $\hat{f}_{\boldsymbol{\theta}}$ and rather utilizes the full posterior.

### 4.4.1 Optimizing Validation Set Performance

A common approach to model selection is to split our data $\mathcal{D}$ into separate training set $\mathcal{D}^{\text{train}} \doteq \{(\boldsymbol{x}_i^{\text{train}}, y_i^{\text{train}})\}_{i=1}^n$ and validation sets

$\mathcal{D}^{\text{val}} \doteq \{(\boldsymbol{x}_i^{\text{val}}, y_i^{\text{val}})\}_{i=1}^m$. We then optimize the model for a parameter candidate $\boldsymbol{\theta}_j$ using the training set. This is usually done by picking a point estimate (like the MAP estimate),

$$\hat{f}_j \doteq \arg\max_f p(f \mid \boldsymbol{x}_{1:n}^{\text{train}}, y_{1:n}^{\text{train}}). \tag{4.22}$$

Then, we score $\boldsymbol{\theta}_j$ according to the performance of $\hat{f}_j$ on the validation set,

$$\hat{\boldsymbol{\theta}} \doteq \arg\max_{\boldsymbol{\theta}_j} p(y_{1:m}^{\text{val}} \mid \boldsymbol{x}_{1:m}^{\text{val}}, \hat{f}_j). \tag{4.23}$$

This ensures that $\hat{f}_j$ does not depend on $\mathcal{D}^{\text{val}}$.

**Remark 4.8: Approximating population risk**

Why is it useful to separate the data into a training and a validation set? Recall from Section A.3.5 that minimizing the empirical risk without separating training and validation data may lead to overfitting as both the loss and $\hat{f}_j$ depend on the same data $\mathcal{D}$. In contrast, using independent training and validation sets, $\hat{f}_j$ does not depend on $\mathcal{D}^{\text{val}}$, and we have that

$$\frac{1}{m}\sum_{i=1}^m \ell(y_i^{\text{val}} \mid \boldsymbol{x}_i^{\text{val}}, \hat{f}_j) \approx \mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{P}}\left[\ell(y \mid \boldsymbol{x}, \hat{f}_j)\right], \tag{4.24}$$

using Monte Carlo sampling.[8] In words, for reasonably large $m$, minimizing the empirical risk as we do in Equation (4.23) approximates minimizing the population risk.

[8] We generally assume $\mathcal{D} \overset{\text{iid}}{\sim} \mathcal{P}$, in particular, we assume that the individual samples of the data are i.i.d.. Recall that in this setting, Hoeffding's inequality (A.41) can be used to gauge how large $m$ should be.

While this approach often is quite effective at preventing overfitting as compared to using the same data for training and picking $\hat{\boldsymbol{\theta}}$, it still collapses the uncertainty in $f$ into a point estimate. Can we do better?

### 4.4.2 *Maximizing the Marginal Likelihood*

We have already seen for Bayesian linear regression, that picking a point estimate loses a lot of information. Instead of optimizing the effects of $\boldsymbol{\theta}$ for a specific point estimate $\hat{f}$ of the model $f$, *maximizing the marginal likelihood* optimizes the effects of $\boldsymbol{\theta}$ across all realizations of $f$. In this approach, we obtain our hyperparameter estimate via

$$\begin{aligned}
\hat{\boldsymbol{\theta}}_{\text{MLE}} &\doteq \arg\max_{\boldsymbol{\theta}} p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}) && \text{(4.25)} && \text{using the definition of marginal likelihood in Bayes' rule (1.45)} \\
&= \arg\max_{\boldsymbol{\theta}} \int p(y_{1:n}, f \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})\, df && && \text{by conditioning on } f \text{ using the sum rule (1.7)} \\
&= \arg\max_{\boldsymbol{\theta}} \int p(y_{1:n} \mid \boldsymbol{x}_{1:n}, f, \boldsymbol{\theta}) p(f \mid \boldsymbol{\theta})\, df. && \text{(4.26)} && \text{using the product rule (1.11)}
\end{aligned}$$

Remarkably, this approach typically avoids overfitting even though we do not use a separate training and validation set. The following table provides an intuitive argument for why maximizing the marginal likelihood is a good strategy.

| | likelihood | prior |
|---|---|---|
| "underfit" model (too simple $\boldsymbol{\theta}$) | small for "almost all" $f$ | large |
| "overfit" model (too complex $\boldsymbol{\theta}$) | large for "few" $f$<br>small for "most" $f$ | small |
| "just right" | moderate for "many" $f$ | moderate |

Table 4.1: The table gives an intuitive explanation of effects of parameter choices $\boldsymbol{\theta}$ on the marginal likelihood. Note that words in quotation marks refer to intuitive quantities, as we have infinitely many realizations of $f$.

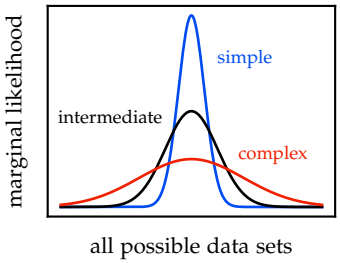



Figure 4.8: A schematic illustration of the marginal likelihood of a simple, intermediate, and complex model across all possible data sets.

For an "underfit" model, the likelihood is mostly small as the data cannot be well described, while the prior is large as there are "fewer" functions to choose from. For an "overfit" model, the likelihood is large for "some" functions (which would be picked if we were only minimizing the training error and not doing cross validation) but small for "most" functions. The prior is small, as the probability mass has to be distributed among "more" functions. Thus, in both cases, one term in the product will be small. Hence, maximizing the marginal likelihood naturally encourages trading between a large likelihood and a large prior.

In the context of Gaussian process regression, recall from Equation (4.3) that

$$y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{K}_{f,\boldsymbol{\theta}} + \sigma_{\mathrm{n}}^2 \boldsymbol{I}) \tag{4.27}$$

where $\boldsymbol{K}_{f,\boldsymbol{\theta}}$ denotes the kernel matrix at the inputs $\boldsymbol{x}_{1:n}$ depending on the kernel function parameterized by $\boldsymbol{\theta}$. We write $\boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}} \doteq \boldsymbol{K}_{f,\boldsymbol{\theta}} + \sigma_{\mathrm{n}}^2 \boldsymbol{I}$. Continuing from Equation (4.25), we obtain

$$\begin{aligned}
\hat{\boldsymbol{\theta}}_{\mathrm{MLE}} &= \arg\max_{\boldsymbol{\theta}} \mathcal{N}(\boldsymbol{y}; \mathbf{0}, \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}) \\
&= \arg\min_{\boldsymbol{\theta}} \frac{1}{2} \boldsymbol{y}^\top \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \boldsymbol{y} + \frac{1}{2} \log\det(\boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}) + \frac{n}{2} \log 2\pi && \text{(4.28)} \quad \text{taking the negative logarithm} \\
&= \arg\min_{\boldsymbol{\theta}} \frac{1}{2} \boldsymbol{y}^\top \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \boldsymbol{y} + \frac{1}{2} \log\det(\boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}) && \text{(4.29)} \quad \text{the last term is independent of } \boldsymbol{\theta}
\end{aligned}$$

The first term of the optimization objective describes the "goodness of fit" (i.e., the "alignment" of $\boldsymbol{y}$ with $\boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}$). The second term characterizes the "volume" of the model class. Thus, this optimization naturally trades the aforementioned objectives.

Marginal likelihood maximization is an empirical Bayes method. Often it is simply referred to as *empirical Bayes*. It also has the nice property

that the gradient of its objective (the MLL loss) can be expressed in closed-from (?),

$$\frac{\partial}{\partial \theta_j} \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}) = \frac{1}{2} \operatorname{tr}\left( (\boldsymbol{\alpha}\boldsymbol{\alpha}^\top - \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1}) \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} \right) \tag{4.30}$$

where $\boldsymbol{\alpha} \doteq \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \boldsymbol{y}$ and $\operatorname{tr}(\boldsymbol{M})$ is the trace of a matrix $\boldsymbol{M}$. This optimization problem is, in general, non-convex. Figure 4.10 gives an example of two local optima according to empirical Bayes.

Taking a step back, observe that taking a probabilistic perspective on model selection naturally led us to consider all realizations of our model $f$ instead of using point estimates. However, we are still using point estimates for our model parameters $\boldsymbol{\theta}$. Continuing on our probabilistic adventure, we could place a prior $p(\boldsymbol{\theta})$ on them too.[9] We could use it to obtain the MAP estimate (still a point estimate!) which adds an additional regularization term

$$\hat{\boldsymbol{\theta}}_{\text{MAP}} \doteq \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) \tag{4.31}$$

$$= \arg\min_{\boldsymbol{\theta}} -\log p(\boldsymbol{\theta}) - \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}). \tag{4.32}$$

An alternative approach is to consider the full posterior distribution over parameters $\boldsymbol{\theta}$. The resulting predictive distribution is, however, intractable,

$$p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}) = \tag{4.33}$$
$$\int \int p(y^\star \mid \boldsymbol{x}^\star, f) \cdot p(f \mid \boldsymbol{x}_{1:n}, y_{1:n}, \boldsymbol{\theta}) \cdot p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) \, df \, d\boldsymbol{\theta}.$$

Recall that as the mode of Gaussians coincides with their mean, the MAP estimate corresponds to the mean of the predictive posterior.

As a final note, observe that in principle, there is nothing stopping us from descending deeper in the probabilistic hierarchy. The prior on the model parameters $\boldsymbol{\theta}$ is likely to have parameters too. Ultimately, we need to break out of this hierarchy of dependencies and choose a prior.

## 4.5 *Approximations*

To learn a Gaussian process, we need to invert $n \times n$ matrices, hence the computational cost is $O(n^3)$. Compare this to Bayesian linear regression which allows us to learn a regression model in $O(nd^2)$ time (even online) where $d$ is the feature dimension. It is therefore natural to look for ways of approximating a Gaussian process.

**Problem 4.7**

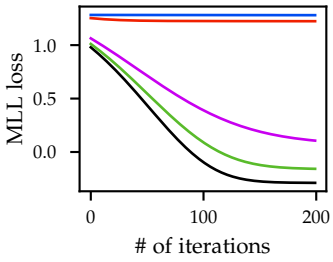



Figure 4.9: An example of model selection by maximizing the log likelihood (without hyperpriors) using a **linear**, **quadratic**, **Laplace**, **Matérn** ($\nu = 3/2$), and **Gaussian** kernel, respectively. They are used to learn the function

$$x \mapsto \frac{\sin(x)}{x} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.01)$$

using SGD with learning rate 0.1.

[9] Such a prior is called *hyperprior*.

using Bayes' rule (1.45) and then taking the negative logarithm

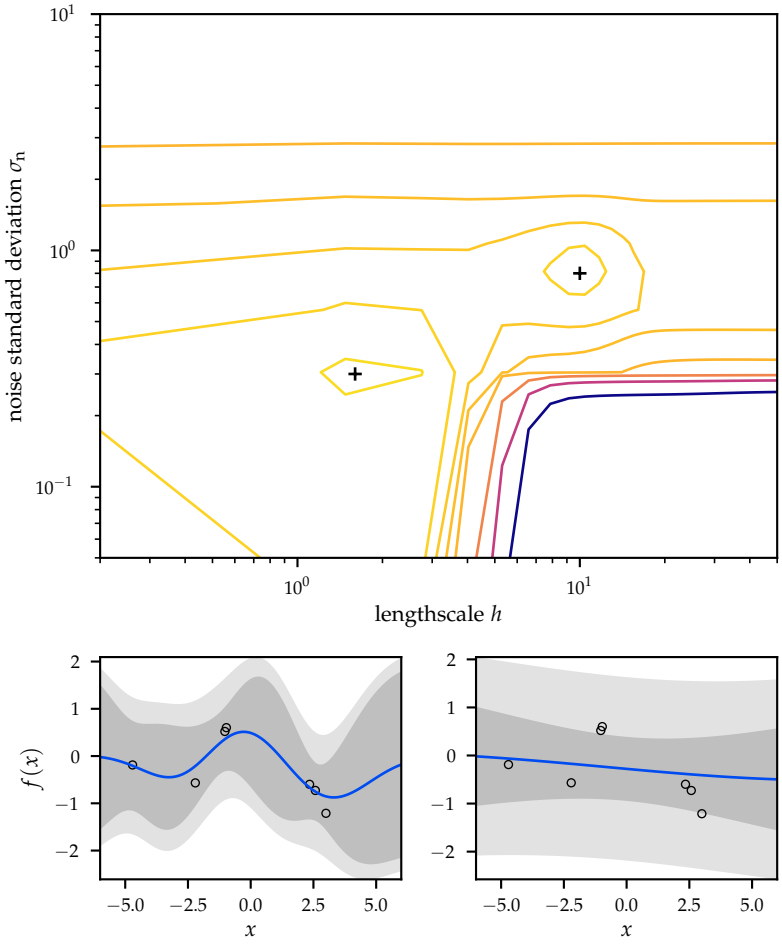



Figure 4.10: The top plot shows contour lines of an empirical Bayes with two local optima. The bottom two plots show the Gaussian processes corresponding to the two optimal models. The left model with smaller lengthscale is chosen within a more flexible class of models, while the right model explains more observations through noise. Adapted from figure 5.5 of "Gaussian processes for machine learning" (Williams and Rasmussen, 2006).

### 4.5.1 Local Methods

Recall that during forward sampling, we had to condition on a larger and larger number of previous samples. When sampling at a location $\boldsymbol{x}$, a very simple approximation is to only condition on those samples $\boldsymbol{x}'$ that are "close" (where $|k(\boldsymbol{x}, \boldsymbol{x}')| \geq \tau$ for some $\tau > 0$). Essentially, this method "cuts off the tails" of the kernel function $k$. However, $\tau$ has to be chosen carefully as if $\tau$ is chosen too large, samples become essentially independent.

This is one example of a *sparse approximation* of a Gaussian process. We will discuss more advanced sparse approximations known as "inducing point methods" in Section 4.5.3.

### 4.5.2 Kernel Function Approximation

Another method is to approximate the kernel function directly. The idea is to construct a "low-dimensional" feature map $\boldsymbol{\phi} : \mathbb{R}^d \to \mathbb{R}^m$ that approximates the kernel,

$$k(\boldsymbol{x}, \boldsymbol{x}') \approx \boldsymbol{\phi}(\boldsymbol{x})^\top \boldsymbol{\phi}(\boldsymbol{x}'). \tag{4.34}$$

Then, we can apply Bayesian linear regression, resulting in a time complexity of $O(nm^2 + m^3)$.

One example of this approach are *random Fourier features*, which we will discuss in the following.

**Remark 4.9: Fourier transform**

First, let us remind ourselves of Fourier transformations. The Fourier transform is a method of decomposing frequencies into their individual components.

Recall *Euler's formula* which states that for any $x \in \mathbb{R}$,

$$e^{ix} = \cos x + i \sin x \tag{4.35}$$

where $i$ is the imaginary unit of complex numbers. The formula is illustrated in Figure 4.11. Note that $e^{-i2\pi x}$ corresponds to rotating clockwise around the unit circle in $\mathbb{R}^2$ — completing a rotation whenever $x \in \mathbb{R}$ reaches the next natural number.

We can scale $x$ by a frequency $\xi$: $e^{-i2\pi\xi x}$. If $\boldsymbol{x} \in \mathbb{R}^d$, we can also scale each component $j$ of $\boldsymbol{x}$ by a different frequency $\boldsymbol{\xi}(j)$. Multiplying a function $f : \mathbb{R}^d \to \mathbb{R}$ with the rotation around the unit circle with given frequencies $\boldsymbol{\xi}$, yields a quantity that describes the

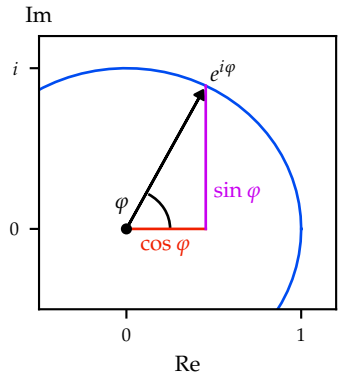



Figure 4.11: Illustration of Euler's formula. It can be seen that $e^{i\varphi}$ corresponds to a (counter-clockwise) rotation on the unit circle as $\varphi$ varies from 0 to $2\pi$.

amplitude of the frequencies $\boldsymbol{\xi}$,

$$\hat{f}(\boldsymbol{\xi}) \doteq \int_{\mathbb{R}^d} f(\boldsymbol{x}) e^{-i2\pi\boldsymbol{\xi}^\top \boldsymbol{x}} \, d\boldsymbol{x}. \tag{4.36}$$

$\hat{f}$ is called the *Fourier transform* of $f$. $f$ is called the *inverse Fourier transform* of $\hat{f}$, and can be computed using

$$f(\boldsymbol{x}) = \int_{\mathbb{R}^d} \hat{f}(\boldsymbol{\xi}) e^{i2\pi\boldsymbol{\xi}^\top \boldsymbol{x}} \, d\boldsymbol{\xi}. \tag{4.37}$$

It is common to write $\boldsymbol{\omega} \doteq 2\pi\boldsymbol{\xi}$. See Figure 4.12 for an example.

Refer to "But what is the Fourier Transform? A visual introduction" (Sanderson, 2018) for a visual introduction.

Because a stationary kernel $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ can be interpreted as a function in one variable, it has an associated Fourier transform which we denote by $p(\boldsymbol{\omega})$. That is,

$$k(\boldsymbol{x} - \boldsymbol{x}') = \int_{\mathbb{R}^d} p(\boldsymbol{\omega}) e^{i\boldsymbol{\omega}^\top (\boldsymbol{x} - \boldsymbol{x}')} \, d\boldsymbol{\omega}. \tag{4.38}$$

**Fact 4.10** (Bochner's theorem)**.** *A continuous stationary kernel on $\mathbb{R}^d$ is positive definite if and only if its Fourier transform $p(\boldsymbol{\omega})$ is non-negative.*

Bochner's theorem implies that when a continuous and stationary kernel is positive definite and scaled appropriately, its Fourier transform $p(\boldsymbol{\omega})$ is a proper probability distribution. In this case, $p(\boldsymbol{\omega})$ is called the *spectral density* of the kernel $k$.

**Remark 4.11: Eigenvalue spectrum of stationary kernels**

When a kernel $k$ is stationary (i.e., a univariate function of $\boldsymbol{x} - \boldsymbol{x}'$), its eigenfunctions (with respect to the usual Lebesgue measure) turn out to be the complex exponentials $\exp(i\boldsymbol{\omega}^\top(\boldsymbol{x} - \boldsymbol{x}'))$. In simpler terms, you can think of these exponentials as "building blocks" at different frequencies $\boldsymbol{\omega}$. The spectral density $p(\boldsymbol{\omega})$ associated with the kernel tells you how strongly each frequency contributes, i.e., how large the corresponding eigenvalue is.

A key insight of this analysis is that the rate at which these magnitudes $p(\boldsymbol{\omega})$ decay with increasing frequency $\boldsymbol{\omega}$ reveals the smoothness of the processes governed by the kernel. If a kernel allocates more "power" to high frequencies (meaning the spectral density decays slowly), the resulting processes will appear "rougher". Conversely, if high-frequency components are suppressed, the process will appear "smoother".

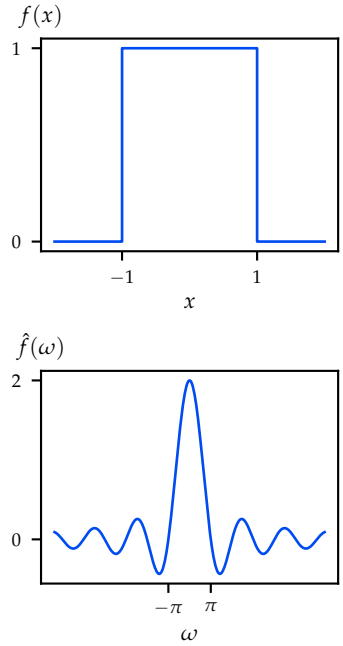



Figure 4.12: The Fourier transform of a rectangular pulse,

$$f(x) \doteq \begin{cases} 1 & x \in [-1, 1] \\ 0 & \text{otherwise,} \end{cases}$$

is given by

$$\hat{f}(\omega) = \int_{-1}^{1} e^{-i\omega x} \, dx = \frac{1}{i\omega}\left(e^{i\omega} - e^{-i\omega}\right) = \frac{2\sin(\omega)}{\omega}.$$

For an in-depth introduction to the eigenfunction analysis of kernels, refer to section 4.3 of "Gaussian processes for machine learning" (Williams and Rasmussen, 2006).

**Example 4.12: Spectral density of the Gaussian kernel**

The Gaussian kernel with length scale $h$ has the spectral density

$$\begin{aligned} p(\boldsymbol{\omega}) &= \int_{\mathbb{R}^d} k(\boldsymbol{x}-\boldsymbol{x}';h) e^{-i\boldsymbol{\omega}^\top(\boldsymbol{x}-\boldsymbol{x}')}\, d(\boldsymbol{x}-\boldsymbol{x}') && \text{using the definition of the Fourier transform (4.36)} \\ &= \int_{\mathbb{R}^d} \exp\left(-\frac{\|\boldsymbol{x}\|_2^2}{2h^2} - i\boldsymbol{\omega}^\top \boldsymbol{x}\right) d\boldsymbol{x} && \text{using the definition of the Gaussian kernel (4.14)} \\ &= (2h^2\pi)^{d/2} \exp\left(-h^2 \frac{\|\boldsymbol{\omega}\|_2^2}{2}\right). && (4.39) \end{aligned}$$

The key idea is now to interpret the kernel as an expectation,

$$\begin{aligned} k(\boldsymbol{x}-\boldsymbol{x}') &= \int_{\mathbb{R}^d} p(\boldsymbol{\omega}) e^{i\boldsymbol{\omega}^\top(\boldsymbol{x}-\boldsymbol{x}')}\, d\boldsymbol{\omega} && \text{from Equation (4.38)} \\ &= \mathbb{E}_{\boldsymbol{\omega}\sim p}\left[e^{i\boldsymbol{\omega}^\top(\boldsymbol{x}-\boldsymbol{x}')}\right] && \text{by the definition of expectation (1.19)} \\ &= \mathbb{E}_{\boldsymbol{\omega}\sim p}\left[\cos(\boldsymbol{\omega}^\top\boldsymbol{x} - \boldsymbol{\omega}^\top\boldsymbol{x}') + i\sin(\boldsymbol{\omega}^\top\boldsymbol{x} - \boldsymbol{\omega}^\top\boldsymbol{x}')\right]. && \text{using Euler's formula (4.35)} \end{aligned}$$

Observe that as both $k$ and $p$ are real, convergence of the integral implies $\mathbb{E}_{\boldsymbol{\omega}\sim p}\left[\sin(\boldsymbol{\omega}^\top\boldsymbol{x} - \boldsymbol{\omega}^\top\boldsymbol{x}')\right] = 0$. Hence,

$$\begin{aligned} &= \mathbb{E}_{\boldsymbol{\omega}\sim p}\left[\cos(\boldsymbol{\omega}^\top\boldsymbol{x} - \boldsymbol{\omega}^\top\boldsymbol{x}')\right] \\ &= \mathbb{E}_{\boldsymbol{\omega}\sim p}\mathbb{E}_{b\sim\mathrm{Unif}([0,2\pi])}\left[\cos((\boldsymbol{\omega}^\top\boldsymbol{x}+b) - (\boldsymbol{\omega}^\top\boldsymbol{x}'+b))\right] && \text{expanding with } b-b \\ &= \mathbb{E}_{\boldsymbol{\omega}\sim p}\mathbb{E}_{b\sim\mathrm{Unif}([0,2\pi])}\left[\cos(\boldsymbol{\omega}^\top\boldsymbol{x}+b)\cos(\boldsymbol{\omega}^\top\boldsymbol{x}'+b)\right. \\ &\qquad\qquad \left. + \sin(\boldsymbol{\omega}^\top\boldsymbol{x}+b)\sin(\boldsymbol{\omega}^\top\boldsymbol{x}'+b)\right] && \text{using the angle subtraction identity, } \cos(\alpha-\beta) = \cos\alpha\cos\beta + \sin\alpha\sin\beta \\ &= \mathbb{E}_{\boldsymbol{\omega}\sim p}\mathbb{E}_{b\sim\mathrm{Unif}([0,2\pi])}\left[2\cos(\boldsymbol{\omega}^\top\boldsymbol{x}+b)\cos(\boldsymbol{\omega}^\top\boldsymbol{x}'+b)\right] && \text{using} \\ &= \mathbb{E}_{\boldsymbol{\omega}\sim p, b\sim\mathrm{Unif}([0,2\pi])}\left[z_{\boldsymbol{\omega},b}(\boldsymbol{x})\cdot z_{\boldsymbol{\omega},b}(\boldsymbol{x}')\right] && (4.40) \end{aligned}$$

$$\mathbb{E}_b[\cos(\alpha+b)\cos(\beta+b)] = \mathbb{E}_b[\sin(\alpha+b)\sin(\beta+b)]$$

for $b \sim \mathrm{Unif}([0,2\pi])$

where $z_{\boldsymbol{\omega},b}(\boldsymbol{x}) \doteq \sqrt{2}\cos(\boldsymbol{\omega}^\top\boldsymbol{x}+b)$,

$$\approx \frac{1}{m}\sum_{i=1}^{m} z_{\boldsymbol{\omega}^{(i)},b^{(i)}}(\boldsymbol{x})\cdot z_{\boldsymbol{\omega}^{(i)},b^{(i)}}(\boldsymbol{x}') \qquad (4.41)$$

using Monte Carlo sampling to estimate the expectation, see Example A.6

for independent samples $\boldsymbol{\omega}^{(i)} \overset{\text{iid}}{\sim} p$ and $b^{(i)} \overset{\text{iid}}{\sim} \mathrm{Unif}([0,2\pi])$,

$$= \boldsymbol{z}(\boldsymbol{x})^\top \boldsymbol{z}(\boldsymbol{x}') \qquad (4.42)$$

where the (randomized) feature map of random Fourier features is

$$\boldsymbol{z}(\boldsymbol{x}) \doteq \frac{1}{\sqrt{m}}\left[z_{\boldsymbol{\omega}^{(1)},b^{(1)}}(\boldsymbol{x}), \ldots, z_{\boldsymbol{\omega}^{(m)},b^{(m)}}(\boldsymbol{x})\right]^\top. \qquad (4.43)$$

Intuitively, each component of the feature map $\boldsymbol{z}(\boldsymbol{x})$ projects $\boldsymbol{x}$ onto a random direction $\boldsymbol{\omega}$ drawn from the (inverse) Fourier transform $p(\boldsymbol{\omega})$ of $k(\boldsymbol{x}-\boldsymbol{x}')$, and wraps this line onto the unit circle in $\mathbb{R}^2$. After transforming two points $\boldsymbol{x}$ and $\boldsymbol{x}'$ in this way, their inner product is an unbiased estimator of $k(\boldsymbol{x}-\boldsymbol{x}')$. The mapping $z_{\boldsymbol{\omega},b}(\boldsymbol{x}) = \sqrt{2}\cos(\boldsymbol{\omega}^\top \boldsymbol{x} + b)$ additionally rotates the circle by a random amount $b$ and projects the points onto the interval $[0, 1]$.

Rahimi et al. (2007) show that Bayesian linear regression with the feature map $\boldsymbol{z}$ approximates Gaussian processes with a stationary kernel:

**Theorem 4.13** (Uniform convergence of Fourier features)**.** *Suppose $\mathcal{M}$ is a compact subset of $\mathbb{R}^d$ with diameter $\operatorname{diam}(\mathcal{M})$. Then for a stationary kernel $k$, the random Fourier features $\boldsymbol{z}$, and any $\epsilon > 0$ it holds that*

$$\mathbb{P}\left(\sup_{\boldsymbol{x},\boldsymbol{x}'\in\mathcal{M}} \left|\boldsymbol{z}(\boldsymbol{x})^\top \boldsymbol{z}(\boldsymbol{x}') - k(\boldsymbol{x}-\boldsymbol{x}')\right| \geq \epsilon\right) \leq 2^8\left(\frac{\sigma_{\mathrm{p}}\operatorname{diam}(\mathcal{M})}{\epsilon}\right)^2 \exp\left(-\frac{m\epsilon^2}{8(d+2)}\right) \tag{4.44}$$

*where $\sigma_{\mathrm{p}}^2 \doteq \mathbb{E}_{\boldsymbol{\omega}\sim p}\left[\boldsymbol{\omega}^\top\boldsymbol{\omega}\right]$ is the second moment of $p$, $m$ is the dimension of $\boldsymbol{z}(\boldsymbol{x})$, and $d$ is the dimension of $\boldsymbol{x}$.* (?)

Note that the error probability decays exponentially fast in the dimension of the Fourier feature space.

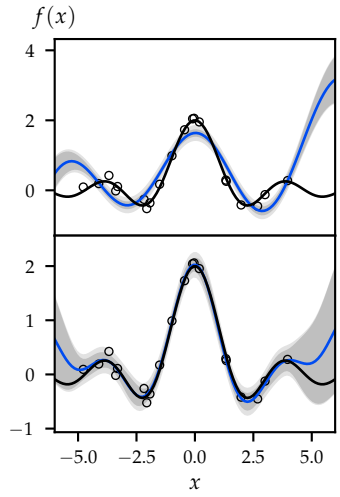



Figure 4.13: Example of random Fourier features with where the number of features $m$ is 5 (top) and 10 (bottom), respectively. The noise-free true function is shown in black and the mean of the Gaussian process is shown in blue.

### 4.5.3 *Data Sampling*

Another natural approach is to only consider a (random) subset of the training data during learning. The naive approach is to subsample uniformly at random. Not very surprisingly, we can do much better.

One subsampling method is the *inducing points method* (Quinonero-Candela and Rasmussen, 2005). The idea is to summarize the data around so-called inducing points.[10] For now, let us consider an arbitrary set of inducing points,

$$U \doteq \{\bar{\boldsymbol{x}}_1, \ldots, \bar{\boldsymbol{x}}_k\}.$$

Then, the original Gaussian process can be recovered using marginalization,

$$p(f^\star, \boldsymbol{f}) = \int_{\mathbb{R}^k} p(f^\star, \boldsymbol{f}, \boldsymbol{u})\, d\boldsymbol{u} = \int_{\mathbb{R}^k} p(f^\star, \boldsymbol{f} \mid \boldsymbol{u}) p(\boldsymbol{u})\, d\boldsymbol{u}, \tag{4.45}$$

where $\boldsymbol{f} \doteq [f(\boldsymbol{x}_1) \;\cdots\; f(\boldsymbol{x}_n)]^\top$ and $f^\star \doteq f(\boldsymbol{x}^\star)$ at some evaluation point $\boldsymbol{x}^\star \in \mathcal{X}$. We use $\boldsymbol{u} \doteq [f(\bar{\boldsymbol{x}}_1) \;\cdots\; f(\bar{\boldsymbol{x}}_k)]^\top \in \mathbb{R}^k$ to denote the predictions of the model at the inducing points $U$. Due to the marginalization property of Gaussian processes (4.1), we have that $\boldsymbol{u} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{K}_{UU})$.

**Problem 4.8**

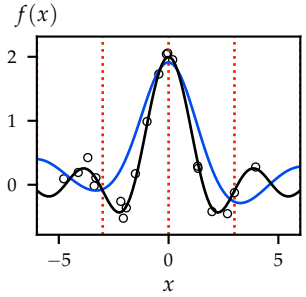



Figure 4.14: Inducing points $\boldsymbol{u}$ are shown as vertical dotted red lines. The noise-free true function is shown in black and the mean of the Gaussian process is shown in blue. Observe that the true function is approximated "well" around the inducing points.

[10] The inducing points can be treated as hyperparameters.

using the sum rule (1.7) and product rule (1.11)

The key idea is to approximate the joint prior, assuming that $f^\star$ and $\boldsymbol{f}$ are conditionally independent given $\boldsymbol{u}$,

$$p(f^\star, \boldsymbol{f}) \approx \int_{\mathbb{R}^k} p(f^\star \mid \boldsymbol{u}) p(\boldsymbol{f} \mid \boldsymbol{u}) p(\boldsymbol{u}) \, d\boldsymbol{u}. \tag{4.46}$$

Here, $p(\boldsymbol{f} \mid \boldsymbol{u})$ and $p(f^\star \mid \boldsymbol{u})$ are commonly called the *training conditional* and the *testing conditional*, respectively. Still denoting the observations by $A = \{\boldsymbol{x}_1, \dots, \boldsymbol{x}_n\}$ and defining $\star \doteq \{\boldsymbol{x}^\star\}$, we know, using the closed-form expression for conditional Gaussians (1.53),

$$p(\boldsymbol{f} \mid \boldsymbol{u}) \sim \mathcal{N}(\boldsymbol{f}; \boldsymbol{K}_{AU} \boldsymbol{K}_{UU}^{-1} \boldsymbol{u}, \boldsymbol{K}_{AA} - \boldsymbol{Q}_{AA}), \tag{4.47a}$$
$$p(f^\star \mid \boldsymbol{u}) \sim \mathcal{N}(f^\star; \boldsymbol{K}_{\star U} \boldsymbol{K}_{UU}^{-1} \boldsymbol{u}, \boldsymbol{K}_{\star\star} - \boldsymbol{Q}_{\star\star}) \tag{4.47b}$$

where $\boldsymbol{Q}_{ab} \doteq \boldsymbol{K}_{aU} \boldsymbol{K}_{UU}^{-1} \boldsymbol{K}_{Ub}$. Intuitively, $\boldsymbol{K}_{AA}$ represents the prior covariance and $\boldsymbol{Q}_{AA}$ represents the covariance "explained" by the inducing points.[11]

[11] For more details, refer to section 2 of "A unifying view of sparse approximate Gaussian process regression" (Quinonero-Candela and Rasmussen, 2005).

Computing the full covariance matrix is expensive. In the following, we mention two approximations to the covariance of the training conditional (and testing conditional).

**Example 4.14: Subset of regressors**

The *subset of regressors* (SoR) approximation is defined as

$$q_{\text{SoR}}(\boldsymbol{f} \mid \boldsymbol{u}) \doteq \mathcal{N}(\boldsymbol{f}; \boldsymbol{K}_{AU} \boldsymbol{K}_{UU}^{-1} \boldsymbol{u}, \boldsymbol{0}), \tag{4.48a}$$
$$q_{\text{SoR}}(f^\star \mid \boldsymbol{u}) \doteq \mathcal{N}(f^\star; \boldsymbol{K}_{\star U} \boldsymbol{K}_{UU}^{-1} \boldsymbol{u}, \boldsymbol{0}). \tag{4.48b}$$

Comparing to Equation (4.47b), SoR simply forgets about all variance and covariance.

**Example 4.15: Fully independent training conditional**

The *fully independent training conditional* (FITC) approximation is defined as

$$q_{\text{FITC}}(\boldsymbol{f} \mid \boldsymbol{u}) \doteq \mathcal{N}(\boldsymbol{f}; \boldsymbol{K}_{AU} \boldsymbol{K}_{UU}^{-1} \boldsymbol{u}, \operatorname{diag}\{\boldsymbol{K}_{AA} - \boldsymbol{Q}_{AA}\}), \tag{4.49a}$$
$$q_{\text{FITC}}(f^\star \mid \boldsymbol{u}) \doteq \mathcal{N}(f^\star; \boldsymbol{K}_{\star U} \boldsymbol{K}_{UU}^{-1} \boldsymbol{u}, \operatorname{diag}\{\boldsymbol{K}_{\star\star} - \boldsymbol{Q}_{\star\star}\}). \tag{4.49b}$$

In contrast to SoR, FITC keeps track of the variances but forgets about the covariance.

The computational cost for inducing point methods SoR and FITC is dominated by the cost of inverting $\boldsymbol{K}_{UU}$. Thus, the time complexity is cubic in the number of inducing points, but only linear in the number of data points.

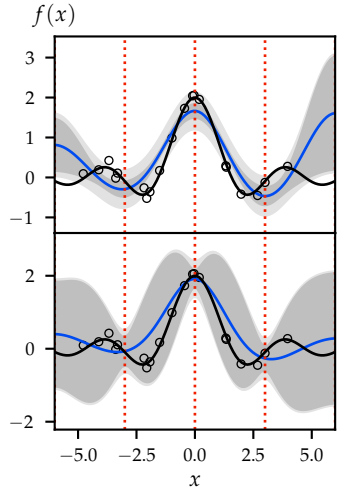



Figure 4.15: Comparison of SoR (top) and FITC (bottom). The inducing points $\boldsymbol{u}$ are shown as vertical dotted red lines. The noise-free true function is shown in black and the mean of the Gaussian process is shown in blue.

## *Discussion*

This chapter introduced Gaussian processes which leverage the function-space view on linear regression to perform exact probabilistic inference with flexible, nonlinear models. A Gaussian process can be seen as a non-parametric model since it can represent an infinite-dimensional parameter space. Instead, as we saw with the representer theorem, such non-parametric (i.e., "function-space") models are directly represented as functions of the data points. While this can make these models more flexible than a simple linear parametric model in input space, it also makes them computationally expensive as the number of data points grows. To this end, we discussed several ways of approximating Gaussian processes.

Nevertheless, for today's internet-scale datasets, modern machine learning typically relies on large parametric models that learn features from data. These models can effectively amortize the cost of inference during training by encoding information into a fixed set of parameters. In the following chapters, we will start to explore approaches to approximate probabilistic inference that can be applied to such models.

## *Problems*

**4.1. Feature space of Gaussian kernel.**

1. Show that the univariate Gaussian kernel with length scale $h = 1$ implicitly defines a feature space with basis vectors

$$\boldsymbol{\phi}(x) = \begin{bmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \end{bmatrix} \quad \text{with} \quad \phi_j(x) = \frac{1}{\sqrt{j!}} e^{-\frac{x^2}{2}} x^j.$$

   *Hint: Use the Taylor series expansion of the exponential function,* $e^x = \sum_{j=0}^{\infty} \frac{x^j}{j!}$.
2. Note that the vector $\boldsymbol{\phi}(x)$ is $\infty$-dimensional. Thus, taking the function-space view allows us to perform regression in an infinite-dimensional feature space. What is the effective dimension when regressing $n$ univariate data points with a Gaussian kernel?

**4.2. Kernels on the circle.**

Consider a dataset $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ with labels $y_i \in \mathbb{R}$ and inputs $\boldsymbol{x}_i$ which lie on the unit circle $\mathbb{S} \subset \mathbb{R}^2$. In particular, any element of $\mathbb{S}$ can be identified with points in $\mathbb{R}^2$ of form $(\cos(\theta), \sin(\theta))$ or with the respective angles $\theta \in [0, 2\pi)$.

You now want to use GP regression to learn an unknown mapping from $\mathbb{S}$ to $\mathbb{R}$ using this dataset. Thus, you need a valid kernel $k$ :

$\mathbb{S} \times \mathbb{S} \to \mathbb{R}$. First, we look at kernels $k$ which can be understood as analogous to the Gaussian kernel.

1. You think of the "extrinsic" kernel $k_e : \mathbb{S} \times \mathbb{S} \to \mathbb{R}$ defined by
$$k_e(\theta, \theta') \doteq \exp\left(-\frac{\|\boldsymbol{x}(\theta) - \boldsymbol{x}(\theta')\|_2^2}{2\kappa^2}\right),$$
where $\boldsymbol{x}(\theta) \doteq (\cos(\theta), \sin(\theta))$. Is $k_e$ positive semi-definite for all values of $\kappa > 0$?
2. Then, you think of an "intrinsic" kernel $k_i : \mathbb{S} \times \mathbb{S} \to \mathbb{R}$ defined by
$$k_i(\theta, \theta') \doteq \exp\left(-\frac{d(\theta, \theta')^2}{2\kappa^2}\right)$$
where $d(\theta, \theta') \doteq \min(|\theta - \theta'|, |\theta - \theta' - 2\pi|, |\theta - \theta' + 2\pi|)$ is the standard arc length distance on the circle $\mathbb{S}$.
You would now like to test whether this kernel is positive semi-definite. We pick $\kappa = 2$ and compute the kernel matrix $\boldsymbol{K}$ for the points corresponding to the angles $\{0, \pi/2, \pi, 3\pi/2\}$. This kernel matrix $\boldsymbol{K}$ has eigenvectors $(1, 1, 1, 1)$ and $(-1, 1, -1, 1)$.
Now compute the eigenvalue corresponding to the eigenvector $(-1, 1, -1, 1)$.
3. Is $k_i$ positive semi-definite for $\kappa = 2$?
4. A mathematician friend of yours suggests to you yet another kernel for points on the circle $\mathbb{S}$, called the *heat kernel*. The kernel itself has a complicated expression but can be accurately approximated by
$$k_h(\theta, \theta') \doteq \frac{1}{C_\kappa}\left(1 + \sum_{l=1}^{L-1} e^{-\frac{\kappa^2}{2}l^2} 2\cos(l(\theta - \theta'))\right),$$
where $L \in \mathbb{N}$ controls the quality of approximation and $C_\kappa > 0$ is a normalizing constant that depends only on $\kappa$.
Is $k_h$ is positive semi-definite for all values of $\kappa > 0$ and $L \in \mathbb{N}$?
*Hint: Recall that* $\cos(a - b) = \cos(a)\cos(b) + \sin(a)\sin(b)$.

### 4.3. A Kalman filter as a Gaussian process.

Next we will show that the Kalman filter from Example 3.4 can be seen as a Gaussian process. To this end, we define

$$f : \mathbb{N}_0 \to \mathbb{R}, \quad t \mapsto X_t. \tag{4.50}$$

Assuming that $X_0 \sim \mathcal{N}(0, \sigma_0^2)$ and $X_{t+1} \doteq X_t + \varepsilon_t$ with independent noise $\varepsilon_t \sim \mathcal{N}(0, \sigma_x^2)$, show that

$$f \sim \mathcal{GP}(0, k_{\mathrm{KF}}) \quad \text{where} \tag{4.51}$$
$$k_{\mathrm{KF}}(t, t') \doteq \sigma_0^2 + \sigma_x^2 \min\{t, t'\}. \tag{4.52}$$

This particular kernel $k(t, t') \doteq \min\{t, t'\}$ but over the continuous-time domain defines the *Wiener process* (also known as Brownian motion).

**4.4. Reproducing property and RKHS norm.**

1. Derive the reproducing property.
   *Hint: Use* $k(\boldsymbol{x}, \boldsymbol{x}') = \langle k(\boldsymbol{x}, \cdot), k(\boldsymbol{x}', \cdot) \rangle_k$.
2. Show that the RKHS norm $\|\cdot\|_k$ is a measure of smoothness by proving that for any $f \in \mathcal{H}_k(\mathcal{X})$ and $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$ it holds that

$$|f(\boldsymbol{x}) - f(\boldsymbol{y})| \leq \|f\|_k \, \|k(\boldsymbol{x}, \cdot) - k(\boldsymbol{y}, \cdot)\|_k \,.$$

**4.5. Representer theorem.**

With this, we can now derive the representer theorem (4.20).

*Hint: Recall*

1. *the reproducing property* $f(\boldsymbol{x}) = \langle f, k(\boldsymbol{x}, \cdot) \rangle_k$ *with* $k(\boldsymbol{x}, \cdot) \in \mathcal{H}_k(\mathcal{X})$ *which holds for all* $f \in \mathcal{H}_k(\mathcal{X})$ *and* $\boldsymbol{x} \in \mathcal{H}_k(\mathcal{X})$*, and*
2. *that the norm after projection is smaller or equal the norm before projection.*

*Then decompose* $f$ *into parallel and orthogonal components with respect to* $\operatorname{span}\{k(\boldsymbol{x}_1, \cdot), \ldots, k(\boldsymbol{x}_n, \cdot)\}$.

**4.6. MAP estimate of Gaussian processes.**

Let us denote by $A = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ the set of training points. We will now show that the MAP estimate of GP regression corresponds to the solution of the regularized linear regression problem in the RKHS stated in Equation (4.21):

$$\hat{f} \doteq \operatorname*{arg\,min}_{f \in \mathcal{H}_k(\mathcal{X})} - \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, f) + \frac{1}{2} \|f\|_k^2 \,.$$

In the following, we abbreviate $\boldsymbol{K} = \boldsymbol{K}_{AA}$. We will also assume that the GP has a zero mean function.

1. Show that Equation (4.21) is equivalent to

$$\hat{\boldsymbol{\alpha}} \doteq \operatorname*{arg\,min}_{\boldsymbol{\alpha} \in \mathbb{R}^n} \|\boldsymbol{y} - \boldsymbol{K}\boldsymbol{\alpha}\|_2^2 + \lambda \, \|\boldsymbol{\alpha}\|_{\boldsymbol{K}}^2 \tag{4.53}$$

   for some $\lambda > 0$ which is also known as *kernel ridge regression*. Determine $\lambda$.
2. Show that Equation (4.53) with the $\lambda$ determined in (1) is equivalent to the MAP estimate of GP regression.
   *Hint: Recall from Equation* (4.6) *that the MAP estimate at a point* $\boldsymbol{x}^\star$ *is* $\mathbb{E}[f^\star \mid \boldsymbol{x}^\star, \boldsymbol{X}, \boldsymbol{y}] = \boldsymbol{k}_{\boldsymbol{x}^\star, A}^\top (\boldsymbol{K} + \sigma_{\mathrm{n}}^2 \boldsymbol{I})^{-1} \boldsymbol{y}$.

**4.7. Gradient of the marginal likelihood.**

In this exercise, we derive Equation (4.30).

Recall that we were considering a dataset $(\boldsymbol{X}, \boldsymbol{y})$ of noise-perturbed evaluations $y_i = f(\boldsymbol{x}_i) + \varepsilon_i$ where $\varepsilon_i \sim \mathcal{N}(0, \sigma_{\mathrm{n}}^2)$ and $f$ is an unknown

function. We make the hypothesis $f \sim \mathcal{GP}(0, k_{\boldsymbol{\theta}})$ with a zero mean function and the covariance function $k_{\boldsymbol{\theta}}$. We are interested in finding the hyperparameters $\boldsymbol{\theta}$ that maximize the marginal likelihood $p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\theta})$.

1. Derive Equation (4.30).
   *Hint: You can use the following identities:*
   *(a) for any invertible matrix $\boldsymbol{M}$,*

$$\frac{\partial}{\partial \theta_j} \boldsymbol{M}^{-1} = -\boldsymbol{M}^{-1} \frac{\partial \boldsymbol{M}}{\partial \theta_j} \boldsymbol{M}^{-1} \quad \textit{and} \tag{4.54}$$

   *(b) for any symmetric positive definite matrix $\boldsymbol{M}$,*

$$\frac{\partial}{\partial \theta_j} \log \det(\boldsymbol{M}) = \operatorname{tr}\left( \boldsymbol{M}^{-1} \frac{\partial \boldsymbol{M}}{\partial \theta_j} \right). \tag{4.55}$$

2. Assume now that the covariance function for the noisy targets (i.e., including the noise contribution) can be expressed as

$$k_{\boldsymbol{y},\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{x}') = \theta_0 \tilde{k}(\boldsymbol{x}, \boldsymbol{x}')$$

   where $\tilde{k}$ is a valid kernel independent of $\theta_0$.[12]
   Show that $\frac{\partial}{\partial \theta_0} \log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\theta}) = 0$ admits a closed-form solution for $\theta_0$ which we denote by $\theta_0^\star$.
3. How should the optimal parameter $\theta_0^\star$ be scaled if we scale the labels $\boldsymbol{y}$ by a scalar $s$?

[12] That is, $\boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}(i, j) = k_{\boldsymbol{y},\boldsymbol{\theta}}(\boldsymbol{x}_i, \boldsymbol{x}_j)$.

**4.8. Uniform convergence of Fourier features.**

In this exercise, we will prove Theorem 4.13.

Let $s(\boldsymbol{x}, \boldsymbol{x}') \doteq \boldsymbol{z}(\boldsymbol{x})^\top \boldsymbol{z}(\boldsymbol{x}')$ and $f(\boldsymbol{x}, \boldsymbol{x}') \doteq s(\boldsymbol{x}, \boldsymbol{x}') - k(\boldsymbol{x}, \boldsymbol{x}')$. Observe that both functions are shift invariant, and we will therefore denote them as univariate functions with argument $\boldsymbol{\Delta} \equiv \boldsymbol{x} - \boldsymbol{x}' \in \mathcal{M}_\Delta$. Notice that our goal is to bound the probability of the event $\sup_{\boldsymbol{\Delta} \in \mathcal{M}_\Delta} |f(\boldsymbol{\Delta})| \geq \epsilon$.

1. Show that for all $\boldsymbol{\Delta} \in \mathcal{M}_\Delta$, $\mathbb{P}(|f(\boldsymbol{\Delta})| \geq \epsilon) \leq 2 \exp\left( -\frac{m\epsilon^2}{4} \right)$.

What we have derived in (1) is known as a *pointwise convergence* guarantee. However, we are interested in bounding the *uniform convergence* over the compact set $\mathcal{M}_\Delta$.

Our approach will be to "cover" the compact set $\mathcal{M}_\Delta$ using $T$ balls of radius $r$ whose centers we denote by $\{\boldsymbol{\Delta}_i\}_{i=1}^T$. It can be shown that this is possible for some $T \leq (4 \operatorname{diam}(\mathcal{M})/r)^d$. It can furthermore be shown that

$$\forall i.\ |f(\boldsymbol{\Delta}_i)| < \frac{\epsilon}{2} \text{ and } \|\nabla f(\boldsymbol{\Delta}^\star)\|_2 < \frac{\epsilon}{2r} \implies \sup_{\boldsymbol{\Delta} \in \mathcal{M}_\Delta} |f(\boldsymbol{\Delta})| < \epsilon$$

where $\boldsymbol{\Delta}^\star = \arg\max_{\boldsymbol{\Delta} \in \mathcal{M}_\Delta} \|\nabla f(\boldsymbol{\Delta})\|_2$.

2. Prove $\mathbb{P}\left(\|\nabla f(\boldsymbol{\Delta}^\star)\|_2 \geq \frac{\epsilon}{2r}\right) \leq \left(\frac{2r\sigma_\mathrm{p}}{\epsilon}\right)^2$.
   *Hint: Recall that the random Fourier feature approximation is unbiased, i.e.,* $\mathbb{E}[s(\boldsymbol{\Delta})] = k(\boldsymbol{\Delta})$.
3. Prove $\mathbb{P}\left(\bigcup_{i=1}^T |f(\boldsymbol{\Delta}_i)| \geq \frac{\epsilon}{2}\right) \leq 2T \exp\left(-\frac{m\epsilon^2}{16}\right)$.
4. Combine the results from (2) and (3) to prove Theorem 4.13.
   *Hint: You may use that*
   *(a)* $\alpha r^{-d} + \beta r^2 = 2\beta^{\frac{d}{d+2}}\alpha^{\frac{2}{d+2}}$ *for* $r = (\alpha/\beta)^{\frac{1}{d+2}}$ *and*
   *(b)* $\frac{\sigma_\mathrm{p}\mathrm{diam}(\mathcal{M})}{\epsilon} \geq 1$.
5. Show that for the Gaussian kernel (4.14), $\sigma_\mathrm{p}^2 = \frac{d}{h^2}$.
   *Hint: First show* $\sigma_\mathrm{p}^2 = -\mathrm{tr}(\boldsymbol{H}_{\boldsymbol{\Delta}} k(\mathbf{0}))$.

**4.9. Subset of regressors.**

1. Using an SoR approximation, prove the following:

$$q_{\mathrm{SoR}}(\boldsymbol{f}, f^\star) = \mathcal{N}\left(\begin{bmatrix}\boldsymbol{f}\\ f^\star\end{bmatrix}; \mathbf{0}, \begin{bmatrix}\boldsymbol{Q}_{AA} & \boldsymbol{Q}_{A\star}\\ \boldsymbol{Q}_{\star A} & \boldsymbol{Q}_{\star\star}\end{bmatrix}\right) \tag{4.56}$$

$$q_{\mathrm{SoR}}(f^\star \mid \boldsymbol{y}) = \mathcal{N}(f^\star; \boldsymbol{Q}_{\star A}\tilde{\boldsymbol{Q}}_{AA}^{-1}\boldsymbol{y}, \boldsymbol{Q}_{\star\star} - \boldsymbol{Q}_{\star A}\tilde{\boldsymbol{Q}}_{AA}^{-1}\boldsymbol{Q}_{A\star}) \tag{4.57}$$

   where $\tilde{\boldsymbol{Q}}_{ab} \doteq \boldsymbol{Q}_{ab} + \sigma_\mathrm{n}^2$.
2. Derive that the resulting model is a degenerate Gaussian process with covariance function

$$k_{\mathrm{SoR}}(\boldsymbol{x}, \boldsymbol{x}') \doteq \boldsymbol{k}_{\boldsymbol{x},U}^\top \boldsymbol{K}_{UU}^{-1} \boldsymbol{k}_{\boldsymbol{x}',U}. \tag{4.58}$$

# 5
# *Variational Inference*

We have seen how to perform (efficient) probabilistic inference with Gaussians, exploiting their closed-form formulas for marginal and conditional distributions. But what if we work with other distributions?

In this and the following chapter, we will discuss two methods of approximate inference. We begin by discussing variational (probabilistic) inference, which aims to find a good approximation of the posterior distribution from which it is easy to sample. In Chapter 6, we discuss Markov chain Monte Carlo methods, which approximate the sampling from the posterior distribution directly.

The fundamental idea behind variational inference is to approximate the true posterior distribution using a "simpler" posterior that is as close as possible to the true posterior:

$$p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) = \frac{1}{Z} p(\boldsymbol{\theta}, y_{1:n} \mid \boldsymbol{x}_{1:n}) \approx q(\boldsymbol{\theta} \mid \boldsymbol{\lambda}) \doteq q_{\boldsymbol{\lambda}}(\boldsymbol{\theta}) \tag{5.1}$$

where $\boldsymbol{\lambda}$ represents the parameters of the *variational posterior* $q_{\boldsymbol{\lambda}}$, also called *variational parameters*. In doing so, variational inference reduces probabilistic inference — where the fundamental difficulty lies in solving high-dimensional integrals — to an optimization problem. Optimizing (stochastic) objectives is a well-understood problem with efficient algorithms that perform well in practice.[1]

[1] We provide an overview of first-order methods such as stochastic gradient descent in Section A.4.

## 5.1 Laplace Approximation

Before introducing a general framework of variational inference, we discuss a simpler method of approximate inference known as *Laplace's method*. This method was proposed as a method of approximating integrals as early as 1774 by Pierre-Simon Laplace. The idea is to use a Gaussian approximation (that is, a second-order Taylor approximation) of the posterior distribution around its mode. Let

$$\psi(\boldsymbol{\theta}) \doteq \log p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) \tag{5.2}$$

denote the log-posterior. Then, using a second-order Taylor approximation (A.53) around the mode $\hat{\boldsymbol{\theta}}$ of $\psi$ (i.e., the MAP estimate), we obtain the approximation $\hat{\psi}$ which is accurate for $\boldsymbol{\theta} \approx \hat{\boldsymbol{\theta}}$:

$$\begin{aligned}\psi(\boldsymbol{\theta}) \approx \hat{\psi}(\boldsymbol{\theta}) &\doteq \psi(\hat{\boldsymbol{\theta}}) + (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^\top \boldsymbol{\nabla}\psi(\hat{\boldsymbol{\theta}}) + \frac{1}{2}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^\top \boldsymbol{H}_\psi(\hat{\boldsymbol{\theta}})(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}) \\ &= \psi(\hat{\boldsymbol{\theta}}) + \frac{1}{2}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^\top \boldsymbol{H}_\psi(\hat{\boldsymbol{\theta}})(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}). \qquad (5.3)\end{aligned}$$

using $\boldsymbol{\nabla}\psi(\hat{\boldsymbol{\theta}}) = 0$

Compare this expression to the log-PDF of a Gaussian:

$$\log \mathcal{N}(\boldsymbol{\theta}; \hat{\boldsymbol{\theta}}, \boldsymbol{\Lambda}^{-1}) = -\frac{1}{2}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^\top \boldsymbol{\Lambda}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}) + \text{const.} \qquad (5.4)$$

Since $\psi(\hat{\boldsymbol{\theta}})$ is constant with respect to $\boldsymbol{\theta}$,

$$\hat{\psi}(\boldsymbol{\theta}) = \log \mathcal{N}(\boldsymbol{\theta}; \hat{\boldsymbol{\theta}}, -\boldsymbol{H}_\psi(\hat{\boldsymbol{\theta}})^{-1}) + \text{const.} \qquad (5.5)$$

The *Laplace approximation* $q$ of $p$ is

$$q(\boldsymbol{\theta}) \doteq \mathcal{N}(\boldsymbol{\theta}; \hat{\boldsymbol{\theta}}, \boldsymbol{\Lambda}^{-1}) \propto \exp(\hat{\psi}(\boldsymbol{\theta})) \qquad \text{where} \qquad (5.6a)$$

$$\boldsymbol{\Lambda} \doteq -\boldsymbol{H}_\psi(\hat{\boldsymbol{\theta}}) = -\boldsymbol{H}_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n})\big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}. \qquad (5.6b)$$

Recall that for this approximation to be well-defined, the covariance matrix $\boldsymbol{\Lambda}^{-1}$ (or equivalently the precision matrix $\boldsymbol{\Lambda}$) needs to be symmetric and positive semi-definite. Let us verify that this is indeed the case for sufficiently smooth $\psi$.[2] In this case, the Hessian $\boldsymbol{\Lambda}$ is symmetric since the order of differentiation does not matter. Moreover, by the second-order optimality condition, $\boldsymbol{H}_\psi(\hat{\boldsymbol{\theta}})$ is negative semi-definite since $\hat{\boldsymbol{\theta}}$ is a maximum of $\psi$, which implies that $\boldsymbol{\Lambda}$ is positive semi-definite.

[2] $\psi$ being twice continuously differentiable around $\hat{\boldsymbol{\theta}}$ is sufficient.

**Example 5.1: Laplace approximation of a Gaussian**

Consider approximating the Gaussian density $p(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ using a Laplace approximation.

We know that the mode of $p$ is $\boldsymbol{\mu}$, which we can verify by computing the gradient,

$$\boldsymbol{\nabla}_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}) = -\frac{1}{2}(2\boldsymbol{\Sigma}^{-1}\boldsymbol{\theta} - 2\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}) \overset{!}{=} 0 \iff \boldsymbol{\theta} = \boldsymbol{\mu}. \qquad (5.7)$$

For the Hessian of $\log p(\boldsymbol{\theta})$, we get

$$\boldsymbol{H}_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}) = (\boldsymbol{D}_{\boldsymbol{\theta}}(\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} - \boldsymbol{\Sigma}^{-1}\boldsymbol{\theta}))^\top = -(\boldsymbol{\Sigma}^{-1})^\top = -\boldsymbol{\Sigma}^{-1}. \qquad (5.8)$$

using $(\boldsymbol{A}^{-1})^\top = (\boldsymbol{A}^\top)^{-1}$ and symmetry of $\boldsymbol{\Sigma}$

We see that the Laplace approximation of a Gaussian $p(\boldsymbol{\theta})$ is exact, which should not come as a surprise since the second-order Taylor approximation of $\log p(\boldsymbol{\theta})$ is exact for Gaussians.

The Laplace approximation matches the shape of the true posterior around its mode but may not represent it accurately elsewhere — often leading to extremely overconfident predictions. An example is given in Figure 5.1. Nevertheless, the Laplace approximation has some desirable properties such as being relatively easy to apply in a post-hoc manner, that is, after having already computed the MAP estimate. It preserves the MAP point estimate as its mean and just "adds" a little uncertainty around it. However, the fact that it can be arbitrarily different from the true posterior makes it unsuitable for approximate probabilistic inference.

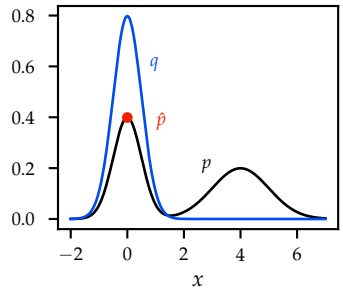



Figure 5.1: The Laplace approximation $q$ greedily selects the mode of the true posterior distribution $p$ and matches the curvature around the mode $\hat{p}$. As shown here, the Laplace approximation can be extremely overconfident when $p$ is not approximately Gaussian.

### 5.1.1 *Example: Bayesian Logistic Regression*

As an example, we will look at Laplace approximation in the context of Bayesian logistic regression. Logistic regression learns a classifier that decides for a given input whether it belongs to one of two classes. A sigmoid function, typically the *logistic function*,

$$\sigma(z) \doteq \frac{1}{1+\exp(-z)} \in (0,1), \qquad z = \boldsymbol{w}^\top \boldsymbol{x}, \tag{5.9}$$

is used to obtain the class probabilities. *Bayesian logistic regression* corresponds to Bayesian linear regression with a Bernoulli likelihood,

$$y \mid \boldsymbol{x}, \boldsymbol{w} \sim \text{Bern}(\sigma(\boldsymbol{w}^\top \boldsymbol{x})), \tag{5.10}$$

where $y \in \{-1, 1\}$ is the binary class label.[3] Observe that given a data point $(\boldsymbol{x}, y)$, the probability of a correct classification is

$$p(y \mid \boldsymbol{x}, \boldsymbol{w}) = \begin{cases} \sigma(\boldsymbol{w}^\top \boldsymbol{x}) & \text{if } y = 1 \\ 1 - \sigma(\boldsymbol{w}^\top \boldsymbol{x}) & \text{if } y = -1 \end{cases} = \sigma(y \boldsymbol{w}^\top \boldsymbol{x}), \tag{5.11}$$

as the logistic function $\sigma$ is symmetric around 0. Also, recall that Bayesian linear regression used the prior

$$p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}; \boldsymbol{0}, \sigma_\text{p}^2 \boldsymbol{I}) \propto \exp\left(-\frac{1}{2\sigma_\text{p}^2} \|\boldsymbol{w}\|_2^2\right).$$

Let us first find the posterior mode, that is, the MAP estimate of the weights:

$$\begin{aligned}
\hat{\boldsymbol{w}} &= \arg\max_{\boldsymbol{w}} p(\boldsymbol{w} \mid \boldsymbol{x}_{1:n}, y_{1:n}) \\
&= \arg\max_{\boldsymbol{w}} p(\boldsymbol{w}) p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{w}) && \text{using Bayes' rule (1.45)} \\
&= \arg\max_{\boldsymbol{w}} \log p(\boldsymbol{w}) + \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{w}) && \text{taking the logarithm} \\
&= \arg\max_{\boldsymbol{w}} -\frac{1}{2\sigma_\text{p}^2} \|\boldsymbol{w}\|_2^2 + \sum_{i=1}^{n} \log \sigma(y_i \boldsymbol{w}^\top \boldsymbol{x}_i) && \text{using independence of the observations and Equation (5.11)}
\end{aligned}$$

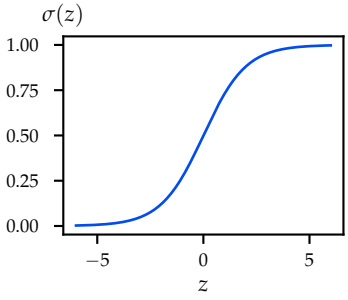



Figure 5.2: The logistic function squashes the linear function $\boldsymbol{w}^\top \boldsymbol{x}$ onto the interval $(0, 1)$.

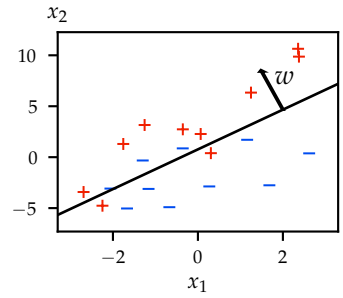



Figure 5.3: Logistic regression classifies data into two classes with a linear decision boundary.

[3] The same approach extends to Gaussian processes where it is known as *Gaussian process classification*, see Problem 5.2 and Hensman et al. (2015).

$$= \arg\min_{\boldsymbol{w}} \frac{1}{2\sigma_{\mathrm{p}}^2} \|\boldsymbol{w}\|_2^2 + \sum_{i=1}^{n} \log(1 + \exp(-y_i \boldsymbol{w}^\top \boldsymbol{x}_i)). \tag{5.12}$$

using the definition of $\sigma$ (5.9)

Note that for $\lambda = 1/2\sigma_{\mathrm{p}}^2$, the above optimization is equivalent to standard (regularized) logistic regression where

$$\ell_{\log}(\boldsymbol{w}^\top \boldsymbol{x}; y) \doteq \log(1 + \exp(-y \boldsymbol{w}^\top \boldsymbol{x})) \tag{5.13}$$

is called *logistic loss*. The gradient of the logistic loss is given by ⓘ

**Problem 5.1 (1)**

$$\nabla_{\boldsymbol{w}} \ell_{\log}(\boldsymbol{w}^\top \boldsymbol{x}; y) = -y\boldsymbol{x} \cdot \sigma(-y \boldsymbol{w}^\top \boldsymbol{x}). \tag{5.14}$$

Recall that due to the symmetry of $\sigma$ around 0, $\sigma(-y\boldsymbol{w}^\top \boldsymbol{x})$ is the probability that $\boldsymbol{x}$ was *not* classified as $y$. Intuitively, if the model is "surprised" by the label, the gradient is large.

We can therefore use SGD with the (regularized) gradient step and with batch size 1,

$$\boldsymbol{w} \leftarrow \boldsymbol{w}(1 - 2\lambda\eta_t) + \eta_t y \boldsymbol{x} \sigma(-y \boldsymbol{w}^\top \boldsymbol{x}), \tag{5.15}$$

for the data point $(\boldsymbol{x}, y)$ picked uniformly at random from the training data. Here, $2\lambda\eta_t$ is due to the gradient of the regularization term, in effect, performing weight decay.

**Example 5.2: Laplace approx. of Bayesian logistic regression**

We have already found the mode of the posterior distribution, $\hat{\boldsymbol{w}}$.

Let us denote by

$$\pi_i \doteq \mathbb{P}(y_i = 1 \mid \boldsymbol{x}_i, \hat{\boldsymbol{w}}) = \sigma(\hat{\boldsymbol{w}}^\top \boldsymbol{x}_i) \tag{5.16}$$

the probability of $\boldsymbol{x}_i$ belonging to the positive class under the model given by the MAP estimate of the weights. For the precision matrix, we then have

$$\begin{aligned}
\boldsymbol{\Lambda} &= -\boldsymbol{H}_{\boldsymbol{w}} \log p(\boldsymbol{w} \mid \boldsymbol{x}_{1:n}, y_{1:n})|_{\boldsymbol{w}=\hat{\boldsymbol{w}}} \\
&= -\boldsymbol{H}_{\boldsymbol{w}} \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{w})|_{\boldsymbol{w}=\hat{\boldsymbol{w}}} - \boldsymbol{H}_{\boldsymbol{w}} \log p(\boldsymbol{w})|_{\boldsymbol{w}=\hat{\boldsymbol{w}}} \\
&= \sum_{i=1}^{n} \boldsymbol{H}_{\boldsymbol{w}} \ell_{\log}(\boldsymbol{w}^\top \boldsymbol{x}_i; y_i)\Big|_{\boldsymbol{w}=\hat{\boldsymbol{w}}} + \sigma_{\mathrm{p}}^{-2} \boldsymbol{I} && \text{using the definition of the logistic loss (5.13)} \\
&= \sum_{i=1}^{n} \boldsymbol{x}_i \boldsymbol{x}_i^\top \pi_i (1 - \pi_i) + \sigma_{\mathrm{p}}^{-2} \boldsymbol{I} \\
&= \boldsymbol{X}^\top \mathrm{diag}_{i \in [n]}\{\pi_i(1 - \pi_i)\} \boldsymbol{X} + \sigma_{\mathrm{p}}^{-2} \boldsymbol{I}. && \text{using the Hessian of the logistic loss (5.73) which you derive in } \textbf{Problem 5.1 (2)}
\end{aligned} \tag{5.17}$$

Observe that $\pi_i(1 - \pi_i) \approx 0$ if $\pi_i \approx 1$ or $\pi_i \approx 0$. That is, if a training example is "well-explained" by $\hat{\boldsymbol{w}}$, then its contribution to the precision matrix is small. In contrast, we have $\pi_i(1 - \pi_i) = 0.25$

for $\pi_i = 0.5$. Importantly, $\boldsymbol{\Lambda}$ does not depend on the normalization constant of the posterior distribution which is hard to compute.

In summary, we have that $\mathcal{N}(\hat{\boldsymbol{w}}, \boldsymbol{\Lambda}^{-1})$ is the Laplace approximation of $p(\boldsymbol{w} \mid \boldsymbol{x}_{1:n}, y_{1:n})$.

## 5.2 *Predictions with a Variational Posterior*

How can we make predictions using our variational approximation? We simply approximate the (intractable) true posterior with our variational posterior:

$$\begin{aligned} p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}) &= \int p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) \, d\boldsymbol{\theta} && \text{using the sum rule (1.7)} \\ &\approx \int p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}) q_{\boldsymbol{\lambda}}(\boldsymbol{\theta}) \, d\boldsymbol{\theta}. && (5.18) \end{aligned}$$

A straightforward approach is to observe that Equation (5.18) can be viewed as an expectation over the variational posterior $q_{\boldsymbol{\lambda}}$ and approximated via Monte Carlo sampling:

$$\begin{aligned} &= \mathbb{E}_{\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}}[p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta})] && (5.19) \\ &\approx \frac{1}{m} \sum_{j=1}^{m} p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}_j) && (5.20) \end{aligned}$$

where $\boldsymbol{\theta}_j \overset{\text{iid}}{\sim} q_{\boldsymbol{\lambda}}$.

**Example 5.3: Predictions in Bayesian logistic regression**

In the case of Bayesian logistic regression with a Gaussian approximation of the posterior, we can obtain more accurate predictions.

Observe that the final prediction $y^\star$ is conditionally independent of the model parameters $\boldsymbol{w}$ given the "latent value" $f^\star = \boldsymbol{w}^\top \boldsymbol{x}^\star$:

$$\begin{aligned} p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}) &\approx \int p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{w}) q_{\boldsymbol{\lambda}}(\boldsymbol{w}) \, d\boldsymbol{w} \\ &= \int \int p(y^\star \mid f^\star) p(f^\star \mid \boldsymbol{x}^\star, \boldsymbol{w}) q_{\boldsymbol{\lambda}}(\boldsymbol{w}) \, d\boldsymbol{w} \, df^\star && \text{once more, using the sum rule (1.7)} \\ &= \int p(y^\star \mid f^\star) \int p(f^\star \mid \boldsymbol{x}^\star, \boldsymbol{w}) q_{\boldsymbol{\lambda}}(\boldsymbol{w}) \, d\boldsymbol{w} \, df^\star. && \text{rearranging terms} \end{aligned} \tag{5.21}$$

The outer integral can be readily approximated since it is only one-dimensional! The challenging part is the inner integral, which is a high-dimensional integral over the model weights $\boldsymbol{w}$. Since the posterior over weights $q_{\boldsymbol{\lambda}}(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}; \hat{\boldsymbol{w}}, \boldsymbol{\Lambda}^{-1})$ is a Gaussian, we

have due to the closedness properties of Gaussians (1.78) that

$$\int p(f^\star \mid \boldsymbol{x}^\star, \boldsymbol{w}) q_\lambda(\boldsymbol{w})\, d\boldsymbol{w} = \mathcal{N}(\hat{\boldsymbol{w}}^\top \boldsymbol{x}^\star, \boldsymbol{x}^{\star\top} \boldsymbol{\Lambda}^{-1} \boldsymbol{x}^\star). \tag{5.22}$$

Crucially, this is a one-dimensional Gaussian in function-space as opposed to the $d$-dimensional Gaussian $q_\lambda$ in weight-space!

As we have seen in Equation (5.11), for Bayesian logistic regression, the prediction $y^\star$ depends deterministically on the predicted latent value $f^\star$: $p(y^\star \mid f^\star) = \sigma(y^\star f^\star)$. Combining Equations (5.21) and (5.22), we obtain

$$p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}) \approx \int \sigma(y^\star f^\star) \cdot \mathcal{N}(f^\star; \hat{\boldsymbol{w}}^\top \boldsymbol{x}^\star, \boldsymbol{x}^{\star\top} \boldsymbol{\Lambda}^{-1} \boldsymbol{x}^\star)\, df^\star. \tag{5.23}$$

We have replaced the high-dimensional integral over the model parameters $\boldsymbol{\theta}$ by the one-dimensional integral over the prediction of our variational posterior $f^\star$. While this integral is generally still intractable, it can be approximated efficiently using numerical quadrature methods such as the Gauss-Legendre quadrature or alternatively with Monte Carlo sampling.

## 5.3 *Blueprint of Variational Inference*

General probabilistic inference poses the challenge of approximating the posterior distribution with limited memory and computation, resource constraints also present in humans and other intelligent systems. These resource constraints require information to be compressed, and as we will see, such a compression poses a fundamental tradeoff between model accuracy (on the observed data) and model complexity (to avoid overfitting).

Laplace approximation approximates the true (intractable) posterior with a simpler one, by greedily matching mode and curvature around it. Can we find "less greedy" approaches? We can view variational probabilistic inference more generally as a family of approaches aiming to approximate the true posterior distribution by one that is closest (according to some criterion) among a "simpler" class of distributions. To this end, we need to fix a class of distributions and define suitable criteria, which we can then optimize numerically. The key benefit is that we can reduce the (generally intractable) problem of high-dimensional integration to the (often much more tractable) problem of optimization.

**Definition 5.4** (Variational family)**.** Let $\mathcal{P}$ be the class of all probability

distributions. A *variational family* $\mathcal{Q} \subseteq \mathcal{P}$ is a class of distributions such that each distribution $q \in \mathcal{Q}$ is characterized by unique variational parameters $\boldsymbol{\lambda} \in \Lambda$.

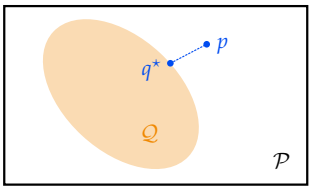



Figure 5.4: An illustration of variational inference in the space of distributions $\mathcal{P}$. The variational distribution $q^\star \in \mathcal{Q}$ is the optimal approximation of the true posterior $p$.

**Example 5.5: Family of independent Gaussians**

A straightforward example for a variational family is the family of independent Gaussians,

$$\mathcal{Q} \doteq \left\{ q(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \operatorname{diag}_{i \in [d]}\{\sigma_i^2\}) \right\}, \tag{5.24}$$

which is parameterized by $\boldsymbol{\lambda} \doteq [\mu_{1:d}, \sigma^2_{1:d}]$. Such a multivariate distribution where all variables are independent is called a *mean-field distribution*. Importantly, this family of distributions is characterized by only $2d$ parameters!

Note that Figure 5.4 is a generalization of the canonical distinction between estimation error and approximation error from Figure 1.7, only that here, we operate in the space of distributions over functions as opposed the space of functions. A common notion of distance between two distributions $q$ and $p$ is the Kullback-Leibler divergence $\mathrm{KL}(q \| p)$ which we will define in the next section. Using this notion of distance, we need to solve the following optimization problem:

$$q^\star \doteq \arg\min_{q \in \mathcal{Q}} \mathrm{KL}(q \| p) = \arg\min_{\lambda \in \Lambda} \mathrm{KL}(q_\lambda \| p). \tag{5.25}$$

In Section 5.4, we introduce information theory and the Kullback-Leibler divergence. Then, in Section 5.5, we discuss how the optimization problem of Equation (5.25) can be solved efficiently.

## 5.4 *Information Theoretic Aspects of Uncertainty*

One of our main objectives throughout this manuscript is to capture the "uncertainty" about events $A$ in an appropriate probability space. One very natural measure of uncertainty is their probability, $\mathbb{P}(A)$. In this section, we will introduce an alternative measure of uncertainty, namely the so-called "surprise" about the event $A$.

### 5.4.1 *Surprise*

The *surprise* about an event with probability $u$ is defined as

$$\mathrm{S}[u] \doteq -\log u. \tag{5.26}$$

Observe that the surprise is a function from $\mathbb{R}_{\geq 0}$ to $\mathbb{R}$, where we let $\mathrm{S}[0] \equiv \infty$. Moreover, for a discrete random variable $X$, we have

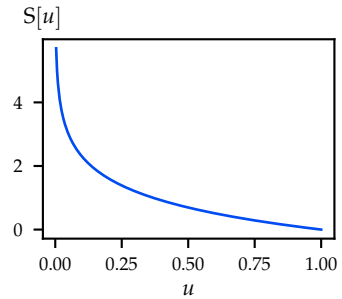



Figure 5.5: Surprise $\mathrm{S}[u]$ associated with an event of probability $u$.

that $p(x) \leq 1$, and hence, $\mathrm{S}[p(x)] \geq 0$. But why is it reasonable to measure surprise by $-\log u$?

Remarkably, it can be shown that the following natural axiomatic characterization leads to exactly this definition of surprise.

**Theorem 5.6** (Axiomatic characterization of surprise)**.** *The axioms*

1. $\mathrm{S}[u] > \mathrm{S}[v] \implies u < v$ *(anti-monotonicity)* — *we are more surprised by unlikely events*
2. S *is continuous,* — *no jumps in surprise for infinitesimal changes of probability*
3. $\mathrm{S}[uv] = \mathrm{S}[u] + \mathrm{S}[v]$ *for independent events,* — *the surprise of independent events is additive*

*characterize* S *up to a positive constant factor.*

*Proof.* Observe that the third condition looks similar to the product rule of logarithms: $\log(uv) = \log v + \log v$. We can formalize this intuition by remembering Cauchy's functional equation, $f(x+y) = f(x) + f(y)$, which has the unique family of solutions $\{f : x \mapsto cx : c \in \mathbb{R}\}$ if $f$ is required to be continuous. Such a solution is called an "additive function". Consider the function $g(x) \doteq f(e^x)$. Then, $g$ is additive if and only if

$$f(e^x e^y) = f(e^{x+y}) = g(x+y) = g(x) + g(y) = f(e^x) + f(e^y).$$

This is precisely the third axiom of surprise for $f = \mathrm{S}$ and $e^x = u$! Hence, the second and third axioms of surprise imply that $g$ must be additive and that $g(x) = \mathrm{S}[e^x] = cx$ for any $c \in \mathbb{R}$. If we replace $e^x$ by $u$, we obtain $\mathrm{S}[u] = c \log u$. The first axiom of surprise implies that $c < 0$, and thus, $\mathrm{S}[u] = -c' \log u$ for any $c' > 0$. □

Importantly, surprise offers a different perspective on uncertainty as opposed to probability: the uncertainty about an event can either be interpreted in terms of its probability or in terms of its surprise, and the two "spaces of uncertainty" are related by a log-transform. This relationship is illustrated in Figure 5.6. Information theory is the study of uncertainty in terms of surprise.

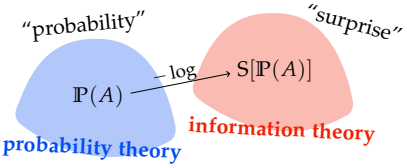



Figure 5.6: Illustration of the probability space and the corresponding "surprise space".

Throughout this manuscript we will see many examples where modeling uncertainty in terms of surprise (i.e., the information-theoretic interpretation of uncertainty) is useful. One example where we have already encountered the "surprise space" was in the context of likelihood maximization (cf. Section 1.3.1) where we used that the log-transform linearizes products of probabilities. We will see later in Chapter 6 that in many cases the surprise $\mathrm{S}[p(x)]$ can also be interpreted as a "cost" or "energy" associated with the state $x$.

### 5.4.2 *Entropy*

The *entropy* of a distribution $p$ is the average surprise about samples from $p$. In this way, entropy is a notion of uncertainty associated with the distribution $p$: if the entropy of $p$ is large, we are more uncertain about $x \sim p$ than if the entropy of $p$ were low. Formally,

$$\mathrm{H}[p] \doteq \mathbb{E}_{x\sim p}[\mathrm{S}[p(x)]] = \mathbb{E}_{x\sim p}[-\log p(x)]. \tag{5.27}$$

When $\mathbf{X} \sim p$ is a random vector distributed according to $p$, we write $\mathrm{H}[\mathbf{X}] \doteq \mathrm{H}[p]$. Observe that by definition, if $p$ is discrete then $\mathrm{H}[p] \geq 0$ as $p(x) \leq 1$ $(\forall x)$.[4] For discrete distributions it is common to use the logarithm with base 2 rather than the natural logarithm:[5]

$$\mathrm{H}[p] = -\sum_x p(x) \log_2 p(x) \qquad \text{(if } p \text{ is discrete)}, \tag{5.28a}$$

$$\mathrm{H}[p] = -\int p(\boldsymbol{x}) \log p(\boldsymbol{x})\, d\boldsymbol{x} \qquad \text{(if } p \text{ is continuous)}. \tag{5.28b}$$

Let us briefly recall Jensen's inequality, which is a useful tool when working with expectations of convex functions such as entropy:[6]

**Fact 5.7** (Jensen's Inequality). (?) *Given a random variable $X$ and a convex function $g : \mathbb{R} \to \mathbb{R}$, we have*

$$g(\mathbb{E}[X]) \leq \mathbb{E}[g(X)]. \tag{5.29}$$

**Example 5.8: Examples of entropy**

- *Fair Coin* $\mathrm{H}[\mathrm{Bern}(0.5)] = -2(0.5 \log_2 0.5) = 1.$
- *Unfair Coin*

$$\mathrm{H}[\mathrm{Bern}(0.1)] = -0.1 \log_2 0.1 - 0.9 \log_2 0.9 \approx 0.469.$$

- *Uniform Distribution*

$$\mathrm{H}[\mathrm{Unif}(\{1,\dots,n\})] = -\sum_{i=1}^{n} \frac{1}{n} \log_2 \frac{1}{n} = \log_2 n.$$

The uniform distribution has the maximum entropy among all discrete distributions supported on $\{1,\dots,n\}$ (?). Note that a fair coin corresponds to a uniform distribution with $n = 2$. Also observe that $\log_2 n$ corresponds to the number of bits required to encode the outcome of the experiment.

In general, the entropy $\mathrm{H}[p]$ of a discrete distribution $p$ can be interpreted as the average number of bits required to encode a sample $x \sim p$, or in other words, the average "information" carried by a sample $x$.

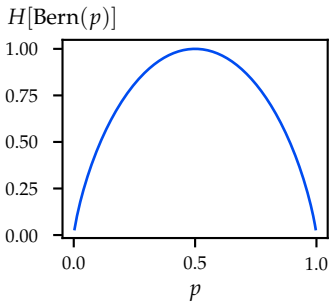



Figure 5.7: Entropy of a Bernoulli experiment with success probability $p$.

[4] The entropy of a continuous distribution can be negative. For example,

$$\mathrm{H}[\mathrm{Unif}([a,b])] = -\int \frac{1}{b-a} \log \frac{1}{b-a}\, dx = \log(b-a)$$

which is negative if $b - a < 1$.

[5] Recall that $\log_2 x = \frac{\log x}{\log 2}$, that is, logarithms to a different base only differ by a constant factor.

[6] The surprise $\mathrm{S}[u]$ is convex in $u$.

**Problem 5.3 (1)**

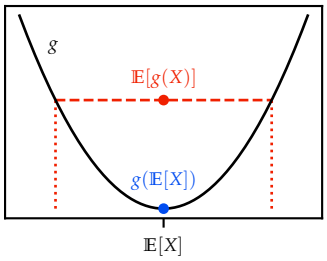



Figure 5.8: An illustration of Jensen's inequality. Due to the convexity of $g$, we have that $g$ evaluated at $\mathbb{E}[X]$ will always be below the average of evaluations of $g$.

**Problem 5.3 (2)**

**Example 5.9: Entropy of a Gaussian**

Let us derive the entropy of a univariate Gaussian. Recall the PDF,

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{Z} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where $Z = \sqrt{2\pi\sigma^2}$. Using the definition of entropy (5.28b), we obtain,

$$\begin{aligned}
\mathrm{H}\Big[\mathcal{N}(\mu, \sigma^2)\Big] &= -\int \frac{1}{Z} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \\
&\qquad \cdot \log\left(\frac{1}{Z} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)\right) dx \\
&= \log Z \underbrace{\int \frac{1}{Z} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) dx}_{1} \\
&\qquad + \int \frac{1}{Z} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \frac{(x-\mu)^2}{2\sigma^2} \, dx \\
&= \log Z + \frac{1}{2\sigma^2} \mathbb{E}\Big[(x-\mu)^2\Big] && \text{using LOTUS (1.22)} \\
&= \log(\sigma\sqrt{2\pi}) + \frac{1}{2} && \text{using } \mathbb{E}\big[(x-\mu)^2\big] = \mathrm{Var}[x] = \sigma^2 \text{ (1.34)} \\
&= \log(\sigma\sqrt{2\pi e}). && \text{using } \log\sqrt{e} = 1/2 \qquad (5.30)
\end{aligned}$$

In general, the entropy of a Gaussian is

$$\mathrm{H}[\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})] = \frac{1}{2} \log\det(2\pi e \boldsymbol{\Sigma}) = \frac{1}{2} \log\Big((2\pi e)^d \det(\boldsymbol{\Sigma})\Big). \quad (5.31)$$

Note that the entropy is a function of the determinant of the covariance matrix $\boldsymbol{\Sigma}$. In general, there are various ways of "scalarizing" the notion of uncertainty for a multivariate distribution. The determinant of $\boldsymbol{\Sigma}$ measures the volume of the credible sets around the mean $\boldsymbol{\mu}$, and is also called the *generalized variance*. Next to entropy and generalized variance (which are closely related for Gaussians), a common scalarization is the trace of $\boldsymbol{\Sigma}$, which is also called the *total variance*.

### 5.4.3 Cross-Entropy

How can we use entropy to measure our average surprise when assuming the data follows some distribution $q$ but in reality the data follows a different distribution $p$?

**Definition 5.10** (Cross-entropy)**.** The *cross-entropy* of a distribution $q$ relative to the distribution $p$ is

$$\mathrm{H}[p\|q] \doteq \mathbb{E}_{x\sim p}[\mathrm{S}[q(x)]] = \mathbb{E}_{x\sim p}[-\log q(x)]. \quad (5.32)$$

Cross-entropy can also be expressed in terms of the KL-divergence (cf. Section 5.4.4) $\mathrm{KL}(p\|q)$ which measures how "different" the distribution $q$ is from a reference distribution $p$,

$$\mathrm{H}[p\|q] = \mathrm{H}[p] + \mathrm{KL}(p\|q) \geq \mathrm{H}[p]. \tag{5.33}$$

$\mathrm{KL}(p\|q) \geq 0$ is shown in Problem 5.5

Quite intuitively, the average surprise in samples from $p$ with respect to the distribution $q$ is given by the inherent uncertainty in $p$ and the additional surprise that is due to us assuming the wrong data distribution $q$. The "closer" $q$ is to the true data distribution $p$, the smaller is the additional average surprise.

### 5.4.4 *Kullback-Leibler Divergence*

As mentioned, the Kullback-Leibler divergence is a (non-metric) measure of distance between distributions. It is defined as follows.

**Definition 5.11** (Kullback-Leibler divergence, KL-divergence)**.** Given two distributions $p$ and $q$, the *Kullback-Leibler divergence* (or *relative entropy*) of $q$ with respect to $p$,

$$\begin{aligned}\mathrm{KL}(p\|q) &\doteq \mathrm{H}[p\|q] - \mathrm{H}[p] && (5.34)\\ &= \mathbb{E}_{\boldsymbol{\theta}\sim p}[\mathrm{S}[q(\boldsymbol{\theta})] - \mathrm{S}[p(\boldsymbol{\theta})]] && (5.35)\\ &= \mathbb{E}_{\boldsymbol{\theta}\sim p}\left[\log \frac{p(\boldsymbol{\theta})}{q(\boldsymbol{\theta})}\right], && (5.36)\end{aligned}$$

measures how different $q$ is from a reference distribution $p$.

In words, $\mathrm{KL}(p\|q)$ measures the *additional* expected surprise when observing samples from $p$ that is due to assuming the (wrong) distribution $q$ and which is not inherent in the distribution $p$ already.[7]

[7] The KL-divergence only captures the additional expected surprise since the surprise inherent in $p$ (as measured by $\mathrm{H}[p]$) is subtracted.

The KL-divergence has the following properties:

- $\mathrm{KL}(p\|q) \geq 0$ for any distributions $p$ and $q$ (?), **Problem 5.5 (1)**
- $\mathrm{KL}(p\|q) = 0$ if and only if $p = q$ almost surely (?), and **Problem 5.5 (2)**
- there exist distributions $p$ and $q$ such that $\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$.

The KL-divergence can simply be understood as a shifted version of cross-entropy, which is zero if we consider the divergence between two identical distributions.

We will briefly look at another interpretation for how KL-divergence measures "distance" between distributions. Suppose we are presented with a sequence $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_n$ of independent samples from either a distribution $p$ or a distribution $q$, both of which are known. Which of $p$ or $q$ was used to generate the data is, however, unknown to us, and we would like to find out. A natural approach is to choose the distribution whose data likelihood is larger. That is, we choose $p$ if

$p(\boldsymbol{\theta}_{1:n}) > q(\boldsymbol{\theta}_{1:n})$ and vice versa. Assuming that the samples are independent and rewriting the inequality slightly, we choose $p$ if

$$\prod_{i=1}^{n} \frac{p(\boldsymbol{\theta}_i)}{q(\boldsymbol{\theta}_i)} > 1, \quad \text{or equivalently if} \quad \sum_{i=1}^{n} \log \frac{p(\boldsymbol{\theta}_i)}{q(\boldsymbol{\theta}_i)} > 0. \tag{5.37}$$

taking the logarithm

Assume without loss of generality that $\boldsymbol{\theta}_i \sim p$. Then, using the law of large numbers (A.36),

$$\frac{1}{n} \sum_{i=1}^{n} \log \frac{p(\boldsymbol{\theta}_i)}{q(\boldsymbol{\theta}_i)} \xrightarrow{a.s.} \mathbb{E}_{\boldsymbol{\theta} \sim p} \left[ \log \frac{p(\boldsymbol{\theta})}{q(\boldsymbol{\theta})} \right] = \mathrm{KL}(p \| q) \tag{5.38}$$

as $n \to \infty$. Plugging this into our decision criterion from Equation (5.37), we find that

$$\mathbb{E}\left[ \sum_{i=1}^{n} \log \frac{p(\boldsymbol{\theta}_i)}{q(\boldsymbol{\theta}_i)} \right] = n \mathrm{KL}(p \| q). \tag{5.39}$$

In this way, $\mathrm{KL}(p\|q)$ measures the observed "distance" between $p$ and $q$. Recall that assuming $p \neq q$ we have that $\mathrm{KL}(p\|q) > 0$ with probability 1, and therefore we correctly choose $p$ with probability 1 as $n \to \infty$. Moreover, Hoeffding's inequality (A.41) can be used to determine "how quickly" samples converge to this limit, that is, how quickly we can distinguish between $p$ and $q$.

**Example 5.12: KL-divergence of Bernoulli random variables**

Suppose we are given two Bernoulli distributions $\mathrm{Bern}(p)$ and $\mathrm{Bern}(q)$. Then, their KL-divergence is

$$\begin{aligned} \mathrm{KL}(\mathrm{Bern}(p) \| \mathrm{Bern}(q)) &= \sum_{x \in \{0,1\}} \mathrm{Bern}(x; p) \log \frac{\mathrm{Bern}(x; p)}{\mathrm{Bern}(x; q)} \\ &= p \log \frac{p}{q} + (1-p) \log \frac{(1-p)}{(1-q)}. \end{aligned} \tag{5.40}$$

Observe that $\mathrm{KL}(\mathrm{Bern}(p)\|\mathrm{Bern}(q)) = 0$ if and only if $p = q$.

**Example 5.13: KL-divergence of Gaussians**

Suppose we are given two Gaussian distributions $p \doteq \mathcal{N}(\boldsymbol{\mu}_p, \boldsymbol{\Sigma}_p)$ and $q \doteq \mathcal{N}(\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)$ with dimension $d$. The KL-divergence of $p$ and $q$ is given by (?)

**Problem 5.8**

$$\begin{aligned} \mathrm{KL}(p\|q) = \frac{1}{2} \Bigg( & \mathrm{tr}(\boldsymbol{\Sigma}_q^{-1} \boldsymbol{\Sigma}_p) + (\boldsymbol{\mu}_p - \boldsymbol{\mu}_q)^\top \boldsymbol{\Sigma}_q^{-1} (\boldsymbol{\mu}_p - \boldsymbol{\mu}_q) \\ & - d + \log \frac{\det(\boldsymbol{\Sigma}_q)}{\det(\boldsymbol{\Sigma}_p)} \Bigg). \end{aligned} \tag{5.41}$$

For independent Gaussians with unit variance, $\boldsymbol{\Sigma}_p = \boldsymbol{\Sigma}_q = \boldsymbol{I}$, the expression simplifies to the squared Euclidean distance,

$$\mathrm{KL}(p\|q) = \frac{1}{2}\left\|\boldsymbol{\mu}_q - \boldsymbol{\mu}_p\right\|_2^2. \tag{5.42}$$

If we approximate independent Gaussians with variances $\sigma_i^2$,

$$p \doteq \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}\{\sigma_1^2, \ldots, \sigma_d^2\}),$$

by a standard normal distribution, $q \doteq \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, the expression simplifies to

$$\mathrm{KL}(p\|q) = \frac{1}{2}\sum_{i=1}^{d}(\sigma_i^2 + \mu_i^2 - 1 - \log\sigma_i^2). \tag{5.43}$$

Here, the term $\mu_i^2$ penalizes a large mean of $p$, the term $\sigma_i^2$ penalizes a large variance of $p$, and the term $-\log\sigma_i^2$ penalizes a small variance of $p$. As expected, $\mathrm{KL}(p\|q)$ is proportional to the amount of information we lose by approximating $p$ with the simpler distribution $q$.

### 5.4.5 *Forward and Reverse KL-divergence*

$\mathrm{KL}(p\|q)$ is also called the *forward* (or *inclusive*) KL-divergence. In contrast, $\mathrm{KL}(q\|p)$ is called the *reverse* (or *exclusive*) KL-divergence. Figure 5.9 shows the approximations of a general Gaussian obtained when $\mathcal{Q}$ is the family of diagonal (independent) Gaussians. Thereby,

$$q_1^\star \doteq \arg\min_{q\in\mathcal{Q}} \mathrm{KL}(p\|q) \quad \text{and} \quad q_2^\star \doteq \arg\min_{q\in\mathcal{Q}} \mathrm{KL}(q\|p).$$

$q_1^\star$ is the result when using the forward KL-divergence and $q_2^\star$ is the result when using reverse KL-divergence. It can be seen that the reverse KL-divergence tends to greedily select the mode and underestimating the variance which, in this case, leads to an overconfident prediction. The forward KL-divergence, in contrast, is more conservative and yields what one could consider the "desired" approximation.

Recall that in the blueprint of variational inference (5.25) we used the reverse KL-divergence. This is for computational reasons. Observe that to approximate the KL-divergence $\mathrm{KL}(p\|q)$ using Monte Carlo sampling, we would need to obtain samples from $p$ yet $p$ is the intractable posterior distribution which we were trying to approximate in the first place. Crucially, observe that if the true posterior

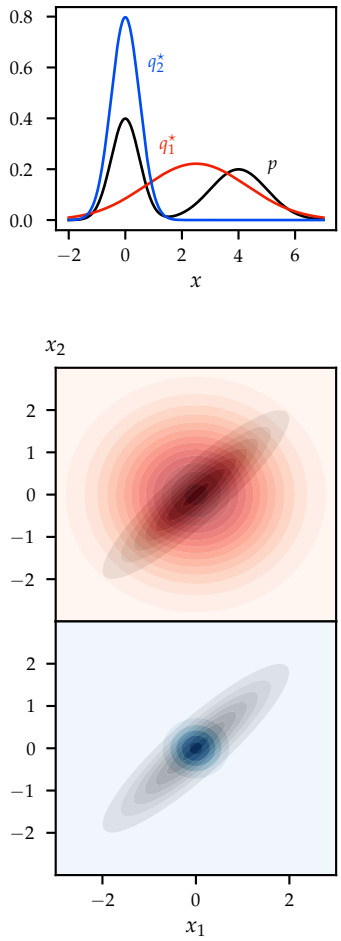



Figure 5.9: Comparison of the **forward** KL-divergence $q_1^\star$ and the **reverse** KL-divergence $q_2^\star$ when used to approximate the **true posterior** $p$. The first plot shows the PDFs in a one-dimensional feature space where $p$ is a mixture of two univariate Gaussians. The second plot shows contour lines of the PDFs in a two-dimensional feature space where the non-diagonal Gaussian $p$ is approximated by diagonal Gaussians $q_1^\star$ and $q_2^\star$. It can be seen that $q_1^\star$ selects the variance and $q_2^\star$ selects the mode of $p$. The approximation $q_1^\star$ is more conservative than the (overconfident) approximation $q_2^\star$.

$p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n})$ is in the variational family $\mathcal{Q}$, then

$$\arg\min_{q \in \mathcal{Q}} \mathrm{KL}(q \| p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n})) \overset{a.s.}{=} p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n}), \tag{5.44}$$

as $\min_{q \in \mathcal{Q}} \mathrm{KL}(q \| p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n})) \overset{a.s.}{=} 0$

so minimizing reverse-KL still recovers the true posterior almost surely.

**Remark 5.14: Greediness of reverse-KL**

As in the previous example, consider the independent Gaussians

$$p \doteq \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}_{i \in [d]}\{\sigma_i^2\}),$$

which we seek to approximate by a standard normal distribution $q \doteq \mathcal{N}(\mathbf{0}, \boldsymbol{I})$. Using (5.41), we obtain for the reverse KL-divergence,

$$\begin{aligned} \mathrm{KL}(q \| p) &= \frac{1}{2}\left(\mathrm{tr}\Big(\mathrm{diag}\{\sigma_i^{-2}\}\Big) + \boldsymbol{\mu}^\top \mathrm{diag}\{\sigma_i^{-2}\}\boldsymbol{\mu} - d \right. \\ &\qquad \left. + \log\det\Big(\mathrm{diag}\{\sigma_i^2\}\Big)\right) \\ &= \frac{1}{2}\sum_{i=1}^{d}\left(\sigma_i^{-2} + \frac{\mu_i^2}{\sigma_i^2} - 1 + \log\sigma_i^2\right). \end{aligned} \tag{5.45}$$

Here, $\sigma_i^{-2}$ penalizes small variance, $\mu_i^2/\sigma_i^2$ penalizes a large mean, and $\log\sigma_i^2$ penalizes large variance. Compare this to the expression for the forward KL-divergence $\mathrm{KL}(p\|q)$ that we have seen in Equation (5.43). In particular, observe that reverse-KL penalizes large variance less strongly than forward-KL.

Note, however, that reverse-KL is not greedy in the same sense as Laplace approximation, as it does still take the variance into account and does not *purely* match the mode of $p$.

### 5.4.6 *Interlude: Minimizing Forward KL-Divergence*

Before completing the blueprint of variational inference in Section 5.5 by showing how *reverse-KL* can be efficiently minimized, we will digress briefly and relate minimizing *forward-KL* to two other well-known inference algorithms. This discussion will deepen our understanding of the KL-divergence and its role in probabilistic inference, but feel free to skip ahead to Section 5.5 if you are eager to complete the blueprint.

*Minimizing forward-KL as maximum likelihood estimation:* First, we observe that minimizing the forward KL-divergence is equivalent to maximum likelihood estimation on an infinitely large sample size. The classical application of this result is in the setting where $p(\boldsymbol{x})$ is a gen-

erative model, and we aim to estimate its density with the parameterized model $q_{\boldsymbol{\lambda}}$.

**Lemma 5.15** (Forward KL-divergence as MLE)**.** *Given some generative model $p(\boldsymbol{x})$ and a likelihood $q_{\boldsymbol{\lambda}}(\boldsymbol{x}) = q(\boldsymbol{x} \mid \boldsymbol{\lambda})$ (that we use to approximate the true data distribution), we have*

$$\arg\min_{\boldsymbol{\lambda}\in\Lambda} \mathrm{KL}(p\|q_{\boldsymbol{\lambda}}) \overset{a.s.}{=} \arg\max_{\boldsymbol{\lambda}\in\Lambda} \lim_{n\to\infty} \frac{1}{n}\sum_{i=1}^{n} \log q(\boldsymbol{x}_i \mid \boldsymbol{\lambda}), \tag{5.46}$$

*where $\boldsymbol{x}_i \overset{\text{iid}}{\sim} p$ are independent samples from the true data distribution.*

*Proof.*

$$\begin{aligned}
\mathrm{KL}(p\|q_{\boldsymbol{\lambda}}) &= \mathrm{H}[p\|q_{\boldsymbol{\lambda}}] - \mathrm{H}[p] && \text{using the definition of KL-divergence (5.34)}\\
&= \mathbb{E}_{(\boldsymbol{x},y)\sim p}[-\log q(\boldsymbol{x} \mid \boldsymbol{\lambda})] + \text{const} && \text{dropping } \mathrm{H}[p] \text{ and using the definition of cross-entropy (5.32)}\\
&\overset{a.s.}{=} -\lim_{n\to\infty} \frac{1}{n}\sum_{i=1}^{n} \log q(\boldsymbol{x}_i \mid \boldsymbol{\lambda}) + \text{const} && \text{using Monte Carlo sampling, i.e., the law of large numbers (A.36)}
\end{aligned}$$

where $\boldsymbol{x}_i \overset{\text{iid}}{\sim} p$ are independent samples. □

This tells us that *any* maximum likelihood estimate $q_{\boldsymbol{\lambda}}$ minimizes the forward KL-divergence to the empirical data distribution. Note that here, we aim to learn model parameters $\boldsymbol{\lambda}$ for estimating the probability of $\boldsymbol{x}$, whereas in the setting of variational probabilistic inference, we want to learn parameters $\boldsymbol{\lambda}$ of a distribution over $\boldsymbol{\theta}$ and $\boldsymbol{\theta}$ parameterizes a *distribution over* $\boldsymbol{x}$. This interpretation is therefore not immediately useful for probabilistic inference (i.e., in the setting where $p$ is a posterior distribution over model parameters $\boldsymbol{\theta}$) as a maximum likelihood estimate requires i.i.d. samples from $p$ which we cannot easily obtain in this case.[8]

[8] It is possible to obtain "approximate" samples using Markov chain Monte Carlo (MCMC) methods which we discuss in Chapter 6.

**Example 5.16: Minimizing cross-entropy**

Minimizing the KL-divergence between $p$ and $q_{\boldsymbol{\lambda}}$ is equivalent to minimizing cross-entropy since $\mathrm{KL}(p\|q_{\boldsymbol{\lambda}}) = \mathrm{H}[p\|q_{\boldsymbol{\lambda}}] - \mathrm{H}[p]$ and $\mathrm{H}[p]$ is constant with respect to $p$.

Lets consider an example in a binary classification problem with the label $y \in \{0, 1\}$ and predicted class probability $\hat{y} \in [0, 1]$ for some fixed input. It is natural to use cross-entropy as a measure of dissimilarity between $y$ and $\hat{y}$,

$$\begin{aligned}
\ell_{\text{bce}}(\hat{y}; y) &\doteq \mathrm{H}[\mathrm{Bern}(y)\|\mathrm{Bern}(\hat{y})]\\
&= -\sum_{x\in\{0,1\}} \mathrm{Bern}(x; y) \log \mathrm{Bern}(x; \hat{y})\\
&= -y \log \hat{y} - (1-y)\log(1-\hat{y}).
\end{aligned} \tag{5.47}$$

This loss function is also known as the *binary cross-entropy loss* and we will discuss it in more detail in Section 7.1.3 in the context of neural networks.

*Minimizing forward-KL as moment matching:* Now to a second interpretation of minimizing forward-KL. *Moment matching* (also known as the *method of moments*) is a technique for approximating an unknown distribution $p$ with a parameterized distribution $q_\lambda$ where $\boldsymbol{\lambda}$ is chosen such that $q_\lambda$ matches the (estimated) moments of $p$. For example, given the estimates $\boldsymbol{a}$ and $\boldsymbol{B}$ of the first and second moment of $p$,[9] and if $q_\lambda$ is a Gaussian with parameters $\boldsymbol{\lambda} = \{\boldsymbol{\mu}, \boldsymbol{\Sigma}\}$, then moment matching chooses $\boldsymbol{\lambda}$ as the solution to

$$\mathbb{E}_p[\boldsymbol{\theta}] \approx \boldsymbol{a} \overset{!}{=} \boldsymbol{\mu} = \mathbb{E}_{q_\lambda}[\boldsymbol{\theta}]$$
$$\mathbb{E}_p\left[\boldsymbol{\theta}\boldsymbol{\theta}^\top\right] \approx \boldsymbol{B} \overset{!}{=} \boldsymbol{\Sigma} + \boldsymbol{\mu}\boldsymbol{\mu}^\top = \mathbb{E}_{q_\lambda}\left[\boldsymbol{\theta}\boldsymbol{\theta}^\top\right].$$

[9] These estimates are computed using the samples from $p$. For example, using a sample mean and a sample variance to compute the estimates of the first and second moment.

using the definition of variance (1.35)

In general the number of moments to be matched (i.e., the number of equations) is adjusted such that it is equal to the number of parameters to be estimated. We will see now that the "matching" of moments is also ensured when $q_\lambda$ is obtained by minimizing the forward KL-divergence within the family of Gaussians.

The Gaussian PDF can be expressed as

$$\mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{Z(\boldsymbol{\lambda})} \exp(\boldsymbol{\lambda}^\top \boldsymbol{s}(\boldsymbol{\theta})) \quad \text{where} \tag{5.48}$$
$$\boldsymbol{\lambda} \doteq \begin{bmatrix} \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} \\ \text{vec}[\boldsymbol{\Sigma}^{-1}] \end{bmatrix} \tag{5.49}$$
$$\boldsymbol{s}(\boldsymbol{\theta}) \doteq \begin{bmatrix} \boldsymbol{\theta} \\ \text{vec}[-\frac{1}{2}\boldsymbol{\theta}\boldsymbol{\theta}^\top] \end{bmatrix} \tag{5.50}$$

and $Z(\boldsymbol{\lambda}) \doteq \int \exp(\boldsymbol{\lambda}^\top \boldsymbol{s}(\boldsymbol{\theta}))\, d\boldsymbol{\theta}$, and we will confirm this in just a moment.[10] The family of distributions with densities of the form (5.48) — with an additional scaling constant $h(\boldsymbol{\theta})$ which is often 1 — is called the *exponential family* of distributions. Here, $\boldsymbol{s}(\boldsymbol{\theta})$ are the *sufficient statistics*, $\boldsymbol{\lambda}$ are called the *natural parameters*, and $Z(\boldsymbol{\lambda})$ is the normalizing constant. In this context, $Z(\boldsymbol{\lambda})$ is often called the *partition function*.

[10] Given a matrix $\boldsymbol{A} \in \mathbb{R}^{n \times m}$, we use

$$\text{vec}[\boldsymbol{A}] \in \mathbb{R}^{n \cdot m}$$

to denote the row-by-row concatenation of $\boldsymbol{A}$ yielding a vector of length $n \cdot m$.

To see that the Gaussian is indeed part of the exponential family as promised in Equations (5.49) and (5.50), consider

$$\mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \propto \exp\left(-\frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\theta} - \boldsymbol{\mu})\right)$$
$$\propto \exp\left(\text{tr}\left(-\frac{1}{2}\boldsymbol{\theta}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\theta}\right) + \boldsymbol{\theta}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right)$$

expanding the inner product and using that $\text{tr}(x) = x$ for all $x \in \mathbb{R}$

$$= \exp\left(\operatorname{tr}\left(-\frac{1}{2}\boldsymbol{\theta}\boldsymbol{\theta}^\top \boldsymbol{\Sigma}^{-1}\right) + \boldsymbol{\theta}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right)$$

using that the trace is invariant under cyclic permutations

$$= \exp\left(\operatorname{vec}[-\tfrac{1}{2}\boldsymbol{\theta}\boldsymbol{\theta}^\top]^\top \operatorname{vec}[\boldsymbol{\Sigma}^{-1}] + \boldsymbol{\theta}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right).$$

using $\operatorname{tr}(\boldsymbol{A}\boldsymbol{B}) = \operatorname{vec}[\boldsymbol{A}]^\top \operatorname{vec}[\boldsymbol{B}]$ for any $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{n\times n}$

This allows us to express the forward KL-divergence as

$$\begin{aligned}\mathrm{KL}(p\|q_\lambda) &= \int p(\boldsymbol{\theta}) \log \frac{p(\boldsymbol{\theta})}{q_\lambda(\boldsymbol{\theta})}\, d\boldsymbol{\theta} \\ &= -\int p(\boldsymbol{\theta}) \cdot \boldsymbol{\lambda}^\top \boldsymbol{s}(\boldsymbol{\theta})\, d\boldsymbol{\theta} + \log Z(\boldsymbol{\lambda}) + \text{const.}\end{aligned}$$

using that $\int p(\boldsymbol{\theta}) \log p(\boldsymbol{\theta})\, d\boldsymbol{\theta}$ is constant

Differentiating with respect to the natural parameters $\boldsymbol{\lambda}$ gives

$$\begin{aligned}\nabla_\lambda \mathrm{KL}(p\|q_\lambda) &= -\int p(\boldsymbol{\theta})\boldsymbol{s}(\boldsymbol{\theta})\, d\boldsymbol{\theta} + \frac{1}{Z(\boldsymbol{\lambda})}\int \boldsymbol{s}(\boldsymbol{\theta}) \exp(\boldsymbol{\lambda}^\top \boldsymbol{s}(\boldsymbol{\theta}))\, d\boldsymbol{\theta} \\ &= -\mathbb{E}_{\boldsymbol{\theta}\sim p}[\boldsymbol{s}(\boldsymbol{\theta})] + \mathbb{E}_{\boldsymbol{\theta}\sim q_\lambda}[\boldsymbol{s}(\boldsymbol{\theta})].\end{aligned}$$

Hence, for any minimizer of $\mathrm{KL}(p\|q_\lambda)$, we have that the sufficient statistics under $p$ and $q_\lambda$ match:

$$\mathbb{E}_p[\boldsymbol{s}(\boldsymbol{\theta})] = \mathbb{E}_{q_\lambda}[\boldsymbol{s}(\boldsymbol{\theta})]. \tag{5.51}$$

Therefore, in the Gaussian case,

$$\mathbb{E}_p[\boldsymbol{\theta}] = \mathbb{E}_{q_\lambda}[\boldsymbol{\theta}] \quad \text{and} \quad \mathbb{E}_p\left[-\tfrac{1}{2}\boldsymbol{\theta}\boldsymbol{\theta}^\top\right] = \mathbb{E}_{q_\lambda}\left[-\tfrac{1}{2}\boldsymbol{\theta}\boldsymbol{\theta}^\top\right],$$

implying that

$$\mathbb{E}_p[\boldsymbol{\theta}] = \boldsymbol{\mu} \quad \text{and} \quad \operatorname{Var}_p[\boldsymbol{\theta}] = \mathbb{E}_p\left[\boldsymbol{\theta}\boldsymbol{\theta}^\top\right] - \mathbb{E}_p[\boldsymbol{\theta}] \cdot \mathbb{E}_p[\boldsymbol{\theta}]^\top = \boldsymbol{\Sigma} \tag{5.52}$$

using Equation (1.35)

where $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are the mean and variance of the approximation $q_\lambda$, respectively. That is, a Gaussian $q_\lambda$ minimizing $\mathrm{KL}(p\|q_\lambda)$ has the same first and second moment as $p$. Combining this insight with our observation from Equation (5.46) that minimizing forward-KL is equivalent to maximum likelihood estimation, we see that if we use MLE to fit a Gaussian to given data, this Gaussian will eventually match the first and second moments of the data distribution.

## 5.5 *Evidence Lower Bound*

Let us return to the blueprint of variational inference from Section 5.3. To complete this blueprint, it remains to show that the reverse KL-divergence can be minimized efficiently. We have

$$\begin{aligned}\mathrm{KL}(q\|p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n})) &= \mathbb{E}_{\boldsymbol{\theta}\sim q}\left[\log \frac{q(\boldsymbol{\theta})}{p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n})}\right] \\ &= \mathbb{E}_{\boldsymbol{\theta}\sim q}\left[\log \frac{p(y_{1:n} \mid \boldsymbol{x}_{1:n}) q(\boldsymbol{\theta})}{p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})}\right]\end{aligned}$$

using the definition of the KL-divergence (5.34)

using the definition of conditional probability (1.8)

$$= \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}) \underbrace{- \mathbb{E}_{\boldsymbol{\theta} \sim q}[\log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})] - \mathrm{H}[q]}_{-L(q,p;\mathcal{D}_n)}$$

using linearity of expectation (1.20)

where $L(q, p; \mathcal{D}_n)$ is called the *evidence lower bound* (ELBO) given the data $\mathcal{D}_n = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$. This gives the relationship

$$L(q, p; \mathcal{D}_n) = \underbrace{\log p(y_{1:n} \mid \boldsymbol{x}_{1:n})}_{\text{const}} - \mathrm{KL}(q \| p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n})). \tag{5.53}$$

Thus, maximizing the ELBO coincides with minimizing reverse-KL.

Maximizing the ELBO,

$$L(q, p; \mathcal{D}_n) \doteq \mathbb{E}_{\boldsymbol{\theta} \sim q}[\log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})] + \mathrm{H}[q], \tag{5.54}$$

selects $q$ that has large joint likelihood $p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})$ and large entropy $\mathrm{H}[q]$. The ELBO can also be expressed in various other forms:

$$L(q, p; \mathcal{D}_n) = \mathbb{E}_{\boldsymbol{\theta} \sim q}[\log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}) - \log q(\boldsymbol{\theta})] \tag{5.55a}$$

using the definition of entropy (5.27)

$$= \mathbb{E}_{\boldsymbol{\theta} \sim q}[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) - \log q(\boldsymbol{\theta})] \tag{5.55b}$$

using the product rule (1.11)

$$= \underbrace{\mathbb{E}_{\boldsymbol{\theta} \sim q}[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})]}_{\text{log-likelihood}} \underbrace{- \mathrm{KL}(q \| p(\cdot))}_{\text{proximity to prior}}. \tag{5.55c}$$

using the definition of KL-divergence (5.34)

where we denote by $p(\cdot)$ the prior distribution. Equation (5.55c) highlights the connection to probabilistic inference, namely that maximizing the ELBO selects a variational distribution $q$ that is close to the prior distribution $p(\cdot)$ while also maximizing the *average* likelihood of the data $p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})$ for $\boldsymbol{\theta} \sim q$. This is in contrast to maximum a posteriori estimation, which picks a *single* model $\boldsymbol{\theta}$ that maximizes the likelihood and proximity to the prior. As an example, let us look at the case where the prior is noninformative, i.e., $p(\cdot) \propto 1$. In this case, the ELBO simplifies to $\mathbb{E}_{\boldsymbol{\theta} \sim q}[\log p(y_{1:n}, \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})] + \mathrm{H}[q] + \text{const}$. That is, maximizing the ELBO maximizes the average likelihood of the data under the variational distribution $q$ while regularizing $q$ to have high entropy. Why is it reasonable to maximize the entropy of $q$? Consider two distributions $q_1$ and $q_2$ under which the data is "equally" likely and which are "equally" close to the prior. Maximizing the entropy selects the distribution that exhibits the most uncertainty which is in accordance with the maximum entropy principle.[11]

Recalling that KL-divergence is non-negative, it follows from Equation (5.53) that the evidence lower bound is a (uniform[12]) lower bound to the *evidence* $\log p(y_{1:n} \mid \boldsymbol{x}_{1:n})$:

$$\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}) \geq L(q, p; \mathcal{D}_n). \tag{5.56}$$

[11] In Section 1.2.1, we discussed the maximum entropy principle and the related principle of indifference at length. In simple terms, the maximum entropy principle states that without information, we should choose the distribution that is maximally uncertain.

[12] That is, the bound holds for any variational distribution $q$ (with full support).

This indicates that maximizing the evidence lower bound is an adequate method of model selection which can be used instead of maximizing the evidence (marginal likelihood) directly (as was discussed in Section 4.4.2). Note that this inequality lower bounds the logarithm of an integral by an expectation of a logarithm over some variational distribution $q$. Hence, the ELBO is a family of lower bounds — one for each variational distribution. Such inequalities are called variational inequalities.

**Example 5.17: Gaussian VI vs Laplace approximation**

Consider maximizing the ELBO within the variational family of Gaussians $\mathcal{Q} \doteq \{q(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma})\}$. How does this relate to Laplace approximation which also fits a Gaussian approximation to the posterior

$$p(\boldsymbol{\theta} \mid \mathcal{D}) \propto p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})?$$

It turns out that both approximations are closely related. Indeed, it can be shown that while the Laplace approximation is fitted *locally* at the MAP estimate $\hat{\boldsymbol{\theta}}$, satisfying

$$\begin{aligned} \mathbf{0} &= \boldsymbol{\nabla}_{\boldsymbol{\theta}} \log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}) \\ \boldsymbol{\Sigma}^{-1} &= -\boldsymbol{H}_{\boldsymbol{\theta}} \log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}, \end{aligned} \tag{5.57}$$

Gaussian variational inference satisfies the conditions of the Laplace approximation *on average* with respect to the approximation $q$ (?):

**Problem 5.10**

$$\begin{aligned} \mathbf{0} &= \mathbb{E}_{\boldsymbol{\theta}\sim q}[\boldsymbol{\nabla}_{\boldsymbol{\theta}} \log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})] \\ \boldsymbol{\Sigma}^{-1} &= -\mathbb{E}_{\boldsymbol{\theta}\sim q}[\boldsymbol{H}_{\boldsymbol{\theta}} \log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})]. \end{aligned} \tag{5.58}$$

For this reason, the Gaussian variational approximation does not suffer from the same overconfidence as the Laplace approximation.[13]

[13] see Figure 5.1

**Example 5.18: ELBO for Bayesian logistic regression**

Recall that Bayesian logistic regression uses the prior distribution $\boldsymbol{w} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.[14]

[14] We omit the scaling factor $\sigma_{\mathrm{p}}^2$ here for simplicity.

Suppose we use the variational family $\mathcal{Q}$ of all diagonal Gaussians from Example 5.5. We have already seen in Equation (5.43) that for a prior $p \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and a variational distribution

$$q_{\boldsymbol{\lambda}} \sim \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}_{i\in[d]}\{\sigma_i^2\}),$$

we have

$$\mathrm{KL}(q \| p(\cdot)) = \frac{1}{2} \sum_{i=1}^{d} (\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2).$$

It remains to find the expected likelihood under models from our approximate posterior:

$$\begin{aligned} \mathbb{E}_{\boldsymbol{w} \sim q_{\boldsymbol{\lambda}}}[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{w})] &= \mathbb{E}_{\boldsymbol{w} \sim q_{\boldsymbol{\lambda}}}\left[\sum_{i=1}^{n} \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{w})\right] && \text{using independence of the data} \\ &= \mathbb{E}_{\boldsymbol{w} \sim q_{\boldsymbol{\lambda}}}\left[-\sum_{i=1}^{n} \ell_{\log}(\boldsymbol{w}; \boldsymbol{x}_i, y_i)\right]. && \text{substituting the logistic loss (5.13)} \end{aligned} \tag{5.59}$$

### 5.5.1 *Gradient of Evidence Lower Bound*

We have yet to discuss how the optimization problem of maximizing the ELBO can be solved efficiently. A suitable tool is stochastic gradient descent (SGD), however, SGD requires unbiased gradient estimates of the loss $\ell(\boldsymbol{\lambda}; \mathcal{D}_n) \doteq -L(q_{\boldsymbol{\lambda}}, p; \mathcal{D}_n)$. That is, we need to obtain gradient estimates of

$$\nabla_{\boldsymbol{\lambda}} L(q_{\boldsymbol{\lambda}}, p; \mathcal{D}_n) = \nabla_{\boldsymbol{\lambda}} \mathbb{E}_{\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}}[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})] - \nabla_{\boldsymbol{\lambda}} \mathrm{KL}(q_{\boldsymbol{\lambda}} \| p(\cdot)). \qquad \text{using the definition of the ELBO (5.55c)} \tag{5.60}$$

Typically, the KL-divergence (and its gradient) can be computed exactly for commonly used variational families. For example, we have already seen a closed-form expression of the KL-divergence for Gaussians in Equation (5.41).

Obtaining the gradient of the expected log-likelihood is more difficult. This is because the expectation integrates over the measure $q_{\boldsymbol{\lambda}}$, which depends on the variational parameters $\boldsymbol{\lambda}$. Thus, we cannot move the gradient operator inside the expectation as commonly done (cf. Section A.1.5). There are two main techniques which are used to rewrite the gradient in such a way that Monte Carlo sampling becomes possible.

One approach is to use *score gradients* via the "score function trick":

$$\begin{aligned} &\nabla_{\boldsymbol{\lambda}} \mathbb{E}_{\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}}[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})] \\ &\quad = \mathbb{E}_{\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}}[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}) \underbrace{\nabla_{\boldsymbol{\lambda}} \log q_{\boldsymbol{\lambda}}(\boldsymbol{\theta})}_{\textit{score function}}], \end{aligned} \tag{5.61}$$

which we introduce in Section 12.3.2 in the context of reinforcement learning. More common in the context of variational inference is the so-called "reparameterization trick".

**Theorem 5.19** (Reparameterization trick)**.** *Given a random variable $\varepsilon \sim \phi$ (which is independent of $\lambda$) and given a differentiable and invertible function $g : \mathbb{R}^d \to \mathbb{R}^d$. We let $\theta \doteq g(\varepsilon; \lambda)$. Then,*

$$q_\lambda(\theta) = \phi(\varepsilon) \cdot |\det(D_\varepsilon g(\varepsilon; \lambda))|^{-1}, \tag{5.62}$$

$$\mathbb{E}_{\theta \sim q_\lambda}[f(\theta)] = \mathbb{E}_{\varepsilon \sim \phi}[f(g(\varepsilon; \lambda))] \tag{5.63}$$

*for a "nice" function $f : \mathbb{R}^d \to \mathbb{R}^e$.*

*Proof.* By the change of variables formula (1.43) and using $\varepsilon = g^{-1}(\theta; \lambda)$,

$$\begin{aligned} q_\lambda(\theta) &= \phi(\varepsilon) \cdot \left|\det\Big(D_\theta g^{-1}(\theta; \lambda)\Big)\right| \\ &= \phi(\varepsilon) \cdot \left|\det\Big((D_\varepsilon g(\varepsilon; \lambda))^{-1}\Big)\right| && \text{by the inverse function theorem, } Dg^{-1}(y) = Dg(x)^{-1} \\ &= \phi(\varepsilon) \cdot |\det(D_\varepsilon g(\varepsilon; \lambda))|^{-1}. && \text{using } \det(A^{-1}) = \det(A)^{-1} \end{aligned}$$

Equation (5.63) is a direct consequence of the law of the unconscious statistician (1.22). □

In other words, the reparameterization trick allows a change of "densities" by finding a function $g(\cdot; \lambda)$ and a reference density $\phi$ such that $q_\lambda = g(\cdot; \lambda)_\sharp \phi$ is the pushforward of $\phi$ under perturbation $g$. Applying the reparameterization trick, we can swap the order of gradient and expectation,

$$\nabla_\lambda \, \mathbb{E}_{\theta \sim q_\lambda}[f(\theta)] = \mathbb{E}_{\varepsilon \sim \phi}[\nabla_\lambda \, f(g(\varepsilon; \lambda))]. \tag{5.64}$$ using Equation (A.5)

We call a distribution $q_\lambda$ *reparameterizable* if it admits reparameterization, i.e., if we can find $g$ and a suitable reference density $\phi$ which is independent of $\lambda$.

**Example 5.20: Reparameterization trick for Gaussians**

Suppose we use a Gaussian variational approximation,

$$q_\lambda(\theta) \doteq \mathcal{N}(\theta; \mu, \Sigma),$$

where we assume $\Sigma$ to have full rank (i.e., be invertible). We have seen in Equation (1.78) that a Gaussian random vector $\varepsilon \sim \mathcal{N}(0, I)$ following a standard normal distribution can be transformed to follow the Gaussian distribution $q_\lambda$ by using the linear transformation,

$$\theta = g(\varepsilon; \lambda) \doteq \Sigma^{1/2}\varepsilon + \mu. \tag{5.65}$$

In particular, we have

$$\varepsilon = g^{-1}(\theta; \lambda) = \Sigma^{-1/2}(\theta - \mu) \quad \text{and} \tag{5.66}$$ by solving Equation (5.65) for $\varepsilon$

$$\phi(\boldsymbol{\varepsilon}) = q_{\boldsymbol{\lambda}}(\boldsymbol{\theta}) \cdot \left|\det\left(\boldsymbol{\Sigma}^{1/2}\right)\right|. \tag{5.67}$$

using the reparameterization trick (i.e., the change of variables formula) (5.62)

In the following, we write $\boldsymbol{C} \doteq \boldsymbol{\Sigma}^{1/2}$. Let us now derive the gradient estimate for the evidence lower bound assuming the Gaussian variational approximation from Example 5.20. This approach extends to any reparameterizable distribution.

$$\begin{aligned}
&\nabla_{\boldsymbol{\lambda}} \mathbb{E}_{\boldsymbol{\theta}\sim q_{\boldsymbol{\lambda}}}[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})] \\
&= \nabla_{\boldsymbol{C},\boldsymbol{\mu}} \mathbb{E}_{\boldsymbol{\varepsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})}\left[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{C}\boldsymbol{\varepsilon}+\boldsymbol{\mu}}\right] && (5.68) && \text{using the reparameterization trick (5.63)} \\
&= n \cdot \nabla_{\boldsymbol{C},\boldsymbol{\mu}} \mathbb{E}_{\boldsymbol{\varepsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})}\left[\frac{1}{n}\sum_{i=1}^{n} \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{C}\boldsymbol{\varepsilon}+\boldsymbol{\mu}}\right] && && \text{using independence of the data and extending with } {}^{n}/_{n} \\
&= n \cdot \nabla_{\boldsymbol{C},\boldsymbol{\mu}} \mathbb{E}_{\boldsymbol{\varepsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \mathbb{E}_{i\sim\mathrm{Unif}([n])}\left[\log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{C}\boldsymbol{\varepsilon}+\boldsymbol{\mu}}\right] && && \text{interpreting the sum as an expectation} \\
&= n \cdot \mathbb{E}_{\boldsymbol{\varepsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \mathbb{E}_{i\sim\mathrm{Unif}([n])}\left[\nabla_{\boldsymbol{C},\boldsymbol{\mu}}\ \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{C}\boldsymbol{\varepsilon}+\boldsymbol{\mu}}\right] && (5.69) && \text{using Equation (A.5)} \\
&\approx n \cdot \frac{1}{m}\sum_{j=1}^{m} \nabla_{\boldsymbol{C},\boldsymbol{\mu}}\ \log p(y_{i_j} \mid x_{i_j}, \boldsymbol{\theta})\Big|_{\boldsymbol{\theta}=\boldsymbol{C}\boldsymbol{\varepsilon}_j+\boldsymbol{\mu}} && (5.70) && \text{using Monte Carlo sampling}
\end{aligned}$$

where $\boldsymbol{\varepsilon}_j \overset{\text{iid}}{\sim} \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and $i_j \overset{\text{iid}}{\sim} \mathrm{Unif}([n])$. This yields an unbiased gradient estimate, which we can use with stochastic gradient descent to maximize the evidence lower bound. We have successfully recast the difficult problems of learning and inference as an optimization problem!

The procedure of approximating the true posterior using a variational posterior by maximizing the evidence lower bound using stochastic optimization is also called *black box stochastic variational inference* (Ranganath et al., 2014; Titsias and Lázaro-Gredilla, 2014; Duvenaud and Adams, 2015). The only requirement is that we can obtain unbiased gradient estimates from the evidence lower bound (and the likelihood). We have just discussed one of many approaches to obtain such gradient estimates (Mohamed et al., 2020). If we use the variational family of diagonal Gaussians, we only require twice as many parameters as other inference techniques like MAP estimation. The performance can be improved by using natural gradients and variance reduction techniques for the gradient estimates such as control variates.

### 5.5.2 *Minimizing Surprise via Exploration and Exploitation*

Now that we have established a way to optimize the ELBO, let us dwell a bit more on its interpretation. Observe that the evidence can also be interpreted as the negative surprise about the observations under the prior distribution $p(\boldsymbol{\theta})$, and by negating Equation (5.56), we obtain the

variational upper bound

$$\begin{aligned} \mathrm{S}[p(y_{1:n} \mid \boldsymbol{x}_{1:n})] &\leq \underbrace{\mathbb{E}_{\boldsymbol{\theta}\sim q}[\mathrm{S}[p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})]]}_{\text{called } \textit{energy}} - \mathrm{H}[q] \\ &= -L(q, p; \mathcal{D}_n). \end{aligned} \tag{5.71}$$

Here, $-L(q, p; \mathcal{D}_n)$ is commonly called the *(variational) free energy* with respect to $q$. Free energy can also be characterized as

$$-L(q, p; \mathcal{D}_n) = \underbrace{\mathbb{E}_{\boldsymbol{\theta}\sim q}[\mathrm{S}[p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})]]}_{\text{average surprise}} + \underbrace{\mathrm{KL}(q \| p(\cdot))}_{\text{proximity to prior}}, \tag{5.72}$$

analogously to Equation (5.55c)

and therefore its minimization is minimizing the average surprise about the data under the variational distribution $q$ while maximizing proximity to the prior $p(\cdot)$. Systems that minimize the surprise in their observations are widely studied in many areas of science.[15]

[15] Refer to the *free energy principle* which was originally introduced by the neuroscientist Karl Friston (Friston, 2010).

To minimize surprise, the free energy makes apparent a natural tradeoff between two extremes: On the one hand, models $q(\boldsymbol{\theta})$ that "overfit" to observations (e.g., by using a point estimate of $\boldsymbol{\theta}$), and hence, result in a large surprise when new observations deviate from this specific belief. On the other hand, models $q(\boldsymbol{\theta})$ that "underfit" to observations (e.g., by expecting outcomes with equal probability), and hence, any observation results in a non-negligible surprise. Either of these extremes is undesirable.

As we have alluded to previously when introducing the ELBO, the two terms constituting free energy map neatly onto this tradeoff. Therein, the *entropy* (which is maximized) encourages $q$ to have uncertainty, in other words, to "explore" beyond the finite data. In contrast, the *energy* (which is minimized) encourages $q$ to fit the observed data closely, in other words, to "exploit" the finite data. This tradeoff is ubiquitous in approximations to probabilistic inference that deal with limited computational resources and limited time, and we will encounter it many times more. We will point out these connections as we go along.

Since this tradeoff is so fundamental and appears in many branches of science under different names, it is difficult to give it an appropriate unifying name. The essence of this tradeoff can be captured as a *principle of curiosity and conformity*, which suggests that reasoning under uncertainty requires curiosity to entertain and pursue alternative explanations of the data *and* conformity to make consistent predictions.

## *Discussion*

We have explored variational inference, where we approximate the intractable posterior distribution of probabilistic inference with a sim-

pler distribution. We operationalized this idea by turning the inference problem, which requires computing high-dimensional integrals, into a tractable optimization problem. Gaussians are frequently used as variational distributions due to their versatility and compact representation.

Nevertheless, recall from Figure 5.4 that while the estimation error in variational inference can be small, choosing a variational family that is too simple can lead to a large approximation error. We have seen that for posteriors that are multimodal or have heavy tails, Gaussians may not provide a good approximation. In the next chapter, we will explore alternative techniques for approximate inference that can handle more complex posteriors.

## *Problems*

**5.1. Logistic loss.**

1. Derive the gradient of $\ell_{\log}$ as given in Equation (5.14).
2. Show that

$$\boldsymbol{H}_{\boldsymbol{w}} \ell_{\log}(\boldsymbol{w}^\top \boldsymbol{x}; y) = \boldsymbol{x}\boldsymbol{x}^\top \cdot \sigma(\boldsymbol{w}^\top \boldsymbol{x}) \cdot (1 - \sigma(\boldsymbol{w}^\top \boldsymbol{x})). \tag{5.73}$$

   *Hint: Begin by deriving the first derivative of the logistic function, and use the chain rule of multivariate calculus,*

$$\underbrace{\boldsymbol{D}_{\boldsymbol{x}}(\boldsymbol{f} \circ \boldsymbol{g})}_{\mathbb{R}^n \to \mathbb{R}^{m \times n}} = \underbrace{(\boldsymbol{D}_{\boldsymbol{g}(\boldsymbol{x})} \boldsymbol{f} \circ \boldsymbol{g}) \cdot \boldsymbol{D}_{\boldsymbol{x}} \boldsymbol{g}}_{\mathbb{R}^n \to \mathbb{R}^{m \times k} . \mathbb{R}^{k \times n}} \tag{5.74}$$

   *where* $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^k$ *and* $\boldsymbol{f} : \mathbb{R}^k \to \mathbb{R}^m$.
3. Is the logistic loss $\ell_{\log}$ convex in $\boldsymbol{w}$?

**5.2. Gaussian process classification.**

In this exercise, we will study the use of Gaussian processes for classification tasks, commonly called *Gaussian process classification* (GPC). Linear logistic regression is extended to GPC by replacing the Gaussian prior over weights with a GP prior on $f$,

$$f \sim \mathcal{GP}(0, k), \quad y \mid \boldsymbol{x}, f \sim \text{Bern}(\sigma(f(\boldsymbol{x}))) \tag{5.75}$$

where $\sigma : \mathbb{R} \to (0, 1)$ is some logistic-type function. Note that Bayesian logistic regression is the special case where $k$ is the linear kernel and $\sigma$ is the logistic function. This is analogous to the relationship of Bayesian linear regression and Gaussian process regression.

In the GP regression setting of Chapter 4, $y_i$ was assumed to be a noisy observation of $f(\boldsymbol{x}_i)$. In the classification setting, we now have

that $y_i \in \{-1, +1\}$ is a binary class label and $f(\boldsymbol{x}_i) \in \mathbb{R}$ is a latent value. We study the setting where $\sigma(z) = \Phi(z; 0, \sigma_\text{n}^2)$ is the CDF of a univariate Gaussian with mean 0 and variance $\sigma_\text{n}^2$, also called a *probit likelihood*.

To make probabilistic predictions for a query point $\boldsymbol{x}^\star$, we first compute the distribution of the latent variable $f^\star$,

$$p(f^\star \mid \boldsymbol{x}_{1:n}, y_{1:n}, \boldsymbol{x}^\star) = \int p(f^\star \mid \boldsymbol{x}_{1:n}, \boldsymbol{x}^\star, \boldsymbol{f}) p(\boldsymbol{f} \mid x_{1:n}, y_{1:n}) \, d\boldsymbol{f} \tag{5.76}$$

using sum rule (1.7) and product rule (1.11), and $f^\star \perp y_{1:n} \mid \boldsymbol{f}$

where $p(\boldsymbol{f} \mid \boldsymbol{x}_{1:n}, y_{1:n})$ is the posterior over the latent variables.

1. Assuming that we can efficiently compute $p(f^\star \mid \boldsymbol{x}_{1:n}, y_{1:n}, \boldsymbol{x}^\star)$ (approximately), describe how we can find the predictive posterior $p(y^\star = +1 \mid \boldsymbol{x}_{1:n}, y_{1:n}, \boldsymbol{x}^\star)$.
2. The posterior over the latent variables is not a Gaussian as we used a non-Gaussian likelihood, and hence, the integral of the latent predictive posterior (5.76) is analytically intractable. A common technique is to approximate the latent posterior $p(\boldsymbol{f} \mid \boldsymbol{x}_{1:n}, y_{1:n})$ with a Gaussian using a Laplace approximation $q \doteq \mathcal{N}(\hat{\boldsymbol{f}}, \boldsymbol{\Lambda}^{-1})$. It is generally not possible to obtain an analytical representation of the mode of the Laplace approximation $\hat{\boldsymbol{f}}$. Instead, $\hat{\boldsymbol{f}}$ is commonly found using a second-order optimization scheme such as Newton's method.
   (a) Find the precision matrix $\boldsymbol{\Lambda}$ of the Laplace approximation.
   *Hint: Observe that for a label $y_i \in \{-1, +1\}$, the probability of a correct classification given the latent value $f_i$ is $p(y_i \mid f_i) = \sigma(y_i f_i)$, where we use the symmetry of the probit likelihood around 0.*
   (b) Assume that $k(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{x}^\top \boldsymbol{x}'$ is the linear kernel ($\sigma_\text{p} = 1$) and that $\sigma$ is the logistic function (5.9). Show for this setting that the matrix $\boldsymbol{\Lambda}$ derived in (a) is equivalent to the precision matrix $\boldsymbol{\Lambda}'$ of the Laplace approximation of Bayesian logistic regression (5.17).[16] You may assume that $\hat{f}_i = \hat{\boldsymbol{w}}^\top \boldsymbol{x}_i$.
   *Hint: First derive under which condition $\boldsymbol{\Lambda}$ and $\boldsymbol{\Lambda}'$ are "equivalent".*
   (c) Observe that the (approximate) latent predictive posterior

$$q(f^\star \mid \boldsymbol{x}_{1:n}, y_{1:n}, \boldsymbol{x}^\star) \doteq \int p(f^\star \mid \boldsymbol{x}_{1:n}, \boldsymbol{x}^\star, \boldsymbol{f}) q(\boldsymbol{f} \mid x_{1:n}, y_{1:n}) \, d\boldsymbol{f}$$

   which uses the Laplace approximation of the latent posterior is Gaussian.[17] Determine its mean and variance.
   *Hint: Condition on the latent variables $\boldsymbol{f}$ using the laws of total expectation and variance.*
   (d) Compare the prediction $p(f^\star \mid \boldsymbol{x}_{1:n}, y_{1:n}, \boldsymbol{x}^\star)$ you obtained in (1) (but now using the Laplace-approximated latent predictive posterior) to the prediction $\sigma(\mathbb{E}_{f^\star \sim q}[f^\star])$. Are they identical? If not, describe how they are different.

[16] This should not be surprising since — as already mentioned — Gaussian process classification is a generalization of Bayesian logistic regression.

[17] Using the Laplace-approximated latent posterior, $[f^\star \ \boldsymbol{f}]$ are jointly Gaussian. Thus, it directly follows from Theorem 1.25 that the marginal distribution over $f^\star$ is also a Gaussian.

3. The use of the probit likelihood may seem arbitrary. Consider the following model which may be more natural,

$$f \sim \mathcal{GP}(0, k), \quad y = \mathbb{1}\{\underbrace{f(\boldsymbol{x}) + \varepsilon}_{\text{GP regression}} \geq 0\}, \quad \varepsilon \sim \mathcal{N}(0, \sigma_{\text{n}}^2). \tag{5.77}$$

Show that the model from Equation (5.75) using a noise-free latent process with probit likelihood $\Phi(z; 0, \sigma_{\text{n}}^2)$ is equivalent (in expectation over $\varepsilon$) to the model from Equation (5.77).

**5.3. Jensen's inequality.**

1. Prove the finite form of Jensen's inequality.
**Theorem 5.21** (Jensen's inequality, finite form)**.** *Let $f : \mathbb{R}^n \to \mathbb{R}$ be a convex function. Suppose that $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k \in \mathbb{R}^n$ and $\theta_1, \ldots, \theta_k \geq 0$ with $\theta_1 + \cdots + \theta_k = 1$. Then,*

$$f(\theta_1 \boldsymbol{x}_1 + \cdots + \theta_k \boldsymbol{x}_k) \leq \theta_1 f(\boldsymbol{x}_1) + \cdots + \theta_k f(\boldsymbol{x}_k). \tag{5.78}$$

Observe that if $X$ is a random variable with finite support, the above two versions of Jensen's inequality are equivalent.
2. Show that for any discrete distribution $p$ supported on a finite domain of size $n$, $\mathrm{H}[p] \leq \log_2 n$. This implies that the uniform distribution has maximum entropy.

**5.4. Binary cross-entropy loss.**

Show that the logistic loss (5.13) is equivalent to the binary cross-entropy loss with $\hat{y} = \sigma(\hat{f})$. That is,

$$\ell_{\log}(\hat{f}; y) = \ell_{\text{bce}}(\hat{y}; y). \tag{5.79}$$

**5.5. Gibbs' inequality.**

1. Prove $\mathrm{KL}(p \| q) \geq 0$ which is also known as *Gibbs' inequality*.
2. Let $p$ and $q$ be discrete distributions with finite identical support $A$. Show that $\mathrm{KL}(p \| q) = 0$ if and only if $p \equiv q$.
*Hint: Use that if a function $f : \mathbb{R}^n \to \mathbb{R}$ is strictly convex and $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k \in \mathbb{R}^n$, $\theta_1, \ldots, \theta_k \geq 0$, $\theta_1 + \cdots + \theta_k = 1$, we have that*

$$f(\theta_1 \boldsymbol{x}_1 + \cdots + \theta_k \boldsymbol{x}_k) = \theta_1 f(\boldsymbol{x}_1) + \cdots + \theta_k f(\boldsymbol{x}_k) \tag{5.80}$$

*iff $\boldsymbol{x}_1 = \cdots = \boldsymbol{x}_k$. This is a slight adaptation of Jensen's inequality in finite-form, which you proved in Problem 5.3.*

**5.6. Maximum entropy principle.**

In this exercise we will prove that the normal distribution is the distribution with maximal entropy among all (univariate) distributions supported on $\mathbb{R}$ with fixed mean $\mu$ and variance $\sigma^2$. Let $g(x) \doteq \mathcal{N}(x; \mu, \sigma^2)$, and $f(x)$ be any distribution on $\mathbb{R}$ with mean $\mu$ and variance $\sigma^2$.

1. Prove that $\mathrm{KL}(f\|g) = \mathrm{H}[g] - \mathrm{H}[f]$.
   *Hint: Equivalently, show that* $\mathrm{H}[f\|g] = \mathrm{H}[g]$. *That is, the expected surprise evaluated based on the Gaussian $g$ is invariant to the true distribution $f$.*
2. Conclude that $\mathrm{H}[g] \geq \mathrm{H}[f]$.

**5.7. Probabilistic inference as a consequence of the maximum entropy principle.**

Consider the family of generative models of the random vectors $\mathbf{X}$ in $\mathcal{X}$ and $\mathbf{Y}$ in $\mathcal{Y}$:

$$\Delta^{\mathcal{X}\times\mathcal{Y}} = \left\{ q : \mathcal{X}\times\mathcal{Y} \to \mathbb{R}_{\geq 0} \;\middle|\; \int_{\mathcal{X}\times\mathcal{Y}} q(\boldsymbol{x},\boldsymbol{y})\, d\boldsymbol{x}\, d\boldsymbol{y} = 1 \right\}.$$

Suppose that we observe $\mathbf{Y}$ to be $\boldsymbol{y}'$, and are looking for a (new) generative model that is consistent with this information, that is,

$$q(\boldsymbol{y}) = \int_{\mathcal{X}} q(\boldsymbol{x},\boldsymbol{y})\, d\boldsymbol{x} = \delta_{\boldsymbol{y}'}(\boldsymbol{y}) \qquad \text{using the sum rule (1.7)}$$

where $\delta_{\boldsymbol{y}'}$ denotes the point density at $\boldsymbol{y}'$. The product rule (1.11) implies that $q(\boldsymbol{x},\boldsymbol{y}) = \delta_{\boldsymbol{y}'}(\boldsymbol{y}) \cdot q(\boldsymbol{x} \mid \boldsymbol{y})$, but any choice of $q(\boldsymbol{x} \mid \boldsymbol{y})$ is possible.

We will derive that given any fixed generative model $p_{\mathbf{X},\mathbf{Y}}$, the "posterior" distribution $q_{\mathbf{X}}(\cdot) = p_{\mathbf{X}|\mathbf{Y}}(\cdot \mid \boldsymbol{y}')$ minimizes the relative entropy $\mathrm{KL}(q_{\mathbf{X},\mathbf{Y}}\|p_{\mathbf{X},\mathbf{Y}})$ subject to the constraint $\mathbf{Y} = \boldsymbol{y}'$. In other words, among all distributions $q_{\mathbf{X},\mathbf{Y}}$ that are consistent with the observation $\mathbf{Y} = \boldsymbol{y}'$, the posterior distribution $q_{\mathbf{X}}(\cdot) = p_{\mathbf{X}|\mathbf{Y}}(\cdot \mid \boldsymbol{y}')$ is the one with "maximum entropy".

1. Show that the optimization problem

$$\begin{aligned} &\operatorname*{arg\,min}_{q\in\Delta^{\mathcal{X}\times\mathcal{Y}}} \mathrm{KL}(q_{\mathbf{X},\mathbf{Y}}\|p_{\mathbf{X},\mathbf{Y}}) \\ &\text{subject to } q(\boldsymbol{y}) = \delta_{\boldsymbol{y}'}(\boldsymbol{y}) \quad \forall \boldsymbol{y} \in \mathcal{Y} \end{aligned}$$

   is solved by $q(\boldsymbol{x},\boldsymbol{y}) = \delta_{\boldsymbol{y}'}(\boldsymbol{y}) \cdot p(\boldsymbol{x} \mid \boldsymbol{y})$.
   *Hint: Solve the dual problem.*
2. Conclude that $q(\boldsymbol{x}) = p(\boldsymbol{x} \mid \boldsymbol{y}')$.

**5.8. KL-divergence of Gaussians.**

Derive Equation (5.41).

*Hint: If* $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu},\boldsymbol{\Sigma})$ *in d dimensions, then we have that for any* $\boldsymbol{m} \in \mathbb{R}^d$ *and* $\boldsymbol{A} \in \mathbb{R}^{d\times d}$,

$$\mathbb{E}\Big[(\mathbf{X}-\boldsymbol{m})^\top \boldsymbol{A}(\mathbf{X}-\boldsymbol{m})\Big] = (\boldsymbol{\mu}-\boldsymbol{m})^\top \boldsymbol{A}(\boldsymbol{\mu}-\boldsymbol{m}) + \mathrm{tr}(\boldsymbol{A}\boldsymbol{\Sigma}) \qquad (5.81)$$

**5.9. Forward vs reverse KL.**

1. Consider a factored approximation $q(x, y) = q(x)q(y)$ to a joint distribution $p(x, y)$. Show that to minimize the forward $\mathrm{KL}(p\|q)$ we should set $q(x) = p(x)$ and $q(y) = p(y)$, i.e., the optimal approximation is a product of marginals.
2. Consider the following joint distribution $p$, where the rows represent $y$ and the columns $x$:

| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 1/8 | 1/8 | 0 | 0 |
| 2 | 1/8 | 1/8 | 0 | 0 |
| 3 | 0 | 0 | 1/4 | 0 |
| 4 | 0 | 0 | 0 | 1/4 |

   Show that the reverse $\mathrm{KL}(q\|p)$ for this $p$ has three distinct minima. Identify those minima and evaluate $\mathrm{KL}(q\|p)$ at each of them.
3. What is the value of $\mathrm{KL}(q\|p)$ if we use the approximation $q(x, y) = p(x)p(y)$?

**5.10. Gaussian VI vs Laplace approximation.**

In this exercise, we compare the Laplace approximation from Section 5.1 to variational inference with the variational family of Gaussians,

$$\mathcal{Q} \doteq \{q(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma})\}.$$

1. Let $p$ be any distribution on $\mathbb{R}$, and let $q^\star = \arg\min_{q\in\mathcal{Q}} \mathrm{KL}(p\|q)$. Show that $q^\star$ differs from the Laplace approximation of $p$.

Minimizing forward-KL is typically intractable, and we have seen that it is therefore common to minimize the reverse-KL instead:

$$\tilde{q} = \arg\min_{q\in\mathcal{Q}} \mathrm{KL}(q\|p(\cdot \mid \mathcal{D}_n)).$$

2. Show that $\tilde{q} = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ satisfies Equation (5.58):

$$\begin{aligned} \mathbf{0} &= \mathbb{E}_{\boldsymbol{\theta}\sim\tilde{q}}[\nabla_{\boldsymbol{\theta}} \log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})] \\ \boldsymbol{\Sigma}^{-1} &= -\mathbb{E}_{\boldsymbol{\theta}\sim\tilde{q}}[\boldsymbol{H}_{\boldsymbol{\theta}} \log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})]. \end{aligned}$$

   *Hint 1: For any positive definite and symmetric matrix $\boldsymbol{A}$, it holds that $\nabla_{\boldsymbol{A}} \log\det(\boldsymbol{A}) = \boldsymbol{A}^{-1}$.*
   *Hint 2: For any function $f$ and Gaussian $p = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$,*

$$\begin{aligned} \nabla_{\boldsymbol{\mu}} \mathbb{E}_{\boldsymbol{x}\sim p}[f(\boldsymbol{x})] &= \mathbb{E}_{\boldsymbol{x}\sim p}[\nabla_{\boldsymbol{x}} f(\boldsymbol{x})] \\ \nabla_{\boldsymbol{\Sigma}} \mathbb{E}_{\boldsymbol{x}\sim p}[f(\boldsymbol{x})] &= \frac{1}{2}\mathbb{E}_{\boldsymbol{x}\sim p}[\boldsymbol{H}_{\boldsymbol{x}} f(\boldsymbol{x})]. \end{aligned} \tag{5.82}$$

Recall the conditions satisfied by the Laplace approximation of the posterior $p(\boldsymbol{\theta} \mid \mathcal{D}) \propto p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})$ as detailed in Equation (5.57). The

Laplace approximation is fitted *locally* at the MAP estimate $\hat{\boldsymbol{\theta}}$. Comparing Equation (5.57) to Equation (5.58), we see that Gaussian variational inference satisfies the conditions of the Laplace approximation *on average*. For more details, refer to Opper and Archambeau (2009).

**5.11. Gradient of reverse-KL.**

Suppose $p \doteq \mathcal{N}(\mathbf{0}, \sigma_{\mathrm{p}}^2 \boldsymbol{I})$ and a tractable distribution described by

$$q_{\boldsymbol{\lambda}} \doteq \mathcal{N}(\boldsymbol{\mu}, \operatorname{diag}\{\sigma_1^2, \dots, \sigma_d^2\})$$

where $\boldsymbol{\mu} \doteq [\mu_1 \ \cdots \ \mu_d]$ and $\boldsymbol{\lambda} \doteq [\mu_1 \ \cdots \ \mu_d \ \sigma_1 \ \cdots \ \sigma_d]$. Show that the gradient of $\mathrm{KL}(q_{\boldsymbol{\lambda}} \| p(\cdot))$ with respect to $\boldsymbol{\lambda}$ is given by

$$\nabla_{\boldsymbol{\mu}} \mathrm{KL}(q_{\boldsymbol{\lambda}} \| p(\cdot)) = \sigma_{\mathrm{p}}^{-2} \boldsymbol{\mu}, \quad \text{and} \tag{5.83a}$$

$$\nabla_{[\sigma_1 \ \cdots \ \sigma_d]} \mathrm{KL}(q_{\boldsymbol{\lambda}} \| p(\cdot)) = \begin{bmatrix} \frac{\sigma_1}{\sigma_{\mathrm{p}}^2} - \frac{1}{\sigma_1} & \cdots & \frac{\sigma_d}{\sigma_{\mathrm{p}}^2} - \frac{1}{\sigma_d} \end{bmatrix}. \tag{5.83b}$$

**5.12. Reparameterizable distributions.**

1. Let $X \sim \mathrm{Unif}([a, b])$ for any $a \leq b$. That is,

$$p_X(x) = \begin{cases} \frac{1}{b-a} & \text{if } x \in [a, b] \\ 0 & \text{otherwise.} \end{cases} \tag{5.84}$$

   Show that $X$ can be reparameterized in terms of $\mathrm{Unif}([0, 1])$. *Hint: You may use that for any $Y \sim \mathrm{Unif}([a, b])$ and $c \in \mathbb{R}$,*
   - $Y + c \sim \mathrm{Unif}([a + c, b + c])$ *and*
   - $cY \sim \mathrm{Unif}([c \cdot a, c \cdot b])$.
2. Let $Z \sim \mathcal{N}(\mu, \sigma^2)$ and $X \doteq e^Z$. That is, $X$ is logarithmically normal distributed with parameters $\mu$ and $\sigma^2$. Show that $X$ can be reparameterized in terms of $\mathcal{N}(0, 1)$.
3. Show that $\mathrm{Cauchy}(0, 1)$ can be reparameterized in terms of $\mathrm{Unif}([0, 1])$.

Finally, let us apply the reparameterization trick to compute the gradient of an expectation.

4. Let $\mathrm{ReLU}(z) \doteq \max\{0, z\}$ and $w > 0$. Show that

$$\frac{d}{d\mu} \mathbb{E}_{x \sim \mathcal{N}(\mu, 1)} \mathrm{ReLU}(wx) = w\Phi(\mu)$$

   where $\Phi$ denotes the CDF of the standard normal distribution.

# 6

# *Markov Chain Monte Carlo Methods*

Variational inference approximates the entire posterior distribution. However, note that the key challenge in probabilistic inference is not learning the posterior distribution, but using the posterior distribution for predictions,

$$p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}) = \int p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n})\, d\boldsymbol{\theta}. \tag{6.1}$$

This integral can be interpreted as an expectation over the posterior distribution,

$$= \mathbb{E}_{\boldsymbol{\theta} \sim p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n})}[p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta})]. \tag{6.2}$$

Observe that the likelihood $f(\boldsymbol{\theta}) \doteq p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta})$ is easy to evaluate. The difficulty lies in sampling from the posterior distribution. Assuming we can obtain independent samples from the posterior distribution, we can use Monte Carlo sampling to obtain an unbiased estimate of the expectation,

$$\approx \frac{1}{m} \sum_{i=1}^{m} f(\boldsymbol{\theta}^{(i)}) \tag{6.3}$$

for independent $\boldsymbol{\theta}^{(i)} \overset{\text{iid}}{\sim} p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n})$. The law of large numbers (A.36) and Hoeffding's inequality (A.41) imply that this estimator is consistent and sharply concentrated.[1]

[1] For more details, see Section A.3.3.

Obtaining samples of the posterior distribution is therefore sufficient to perform approximate inference. Recall that the difficulty of computing the posterior $p$ exactly, was in finding the normalizing constant $Z$,

$$p(x) = \frac{1}{Z} q(x). \tag{6.4}$$

The joint likelihood $q$ is typically easy to obtain. Note that $q(x)$ is proportional to the probability density associated with $x$, but $q$ does not

integrate to 1. Such functions are also called a *finite measure*. Without normalizing $q$, we cannot directly sample from it.

**Remark 6.1: The difficulty of sampling — even with a PDF**

Even a decent approximation of $Z$ does not yield a general efficient sampling method. For example, one very common approach to sampling is *inverse transform sampling* (cf. Section A.1.3) which requires an (approximate) quantile function. Computing the quantile function given an arbitrary PDF requires solving integrals over the domain of the PDF which is what we were trying to avoid in the first place.

The key idea of Markov chain Monte Carlo methods is to construct a Markov chain, which is efficient to simulate and has the stationary distribution $p$.

## 6.1 Markov Chains

To start, let us revisit the fundamental theory behind Markov chains.

**Definition 6.2** (Markov chain)**.** A *(finite and discrete-time) Markov chain* over the state space

$$S \doteq \{0, \dots, n-1\} \tag{6.5}$$

is a stochastic process[2] $(X_t)_{t \in \mathbb{N}_0}$ valued in $S$ such that the *Markov property* is satisfied:

$$X_{t+1} \perp X_{0:t-1} \mid X_t. \tag{6.6}$$

[2] A *stochastic process* is a sequence of random variables.

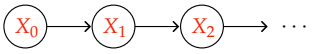



Figure 6.1: Directed graphical model of a Markov chain. The random variable $X_{t+1}$ is conditionally independent of the random variables $X_{0:t-1}$ given $X_t$.

Intuitively, the Markov property states that future behavior is independent of past states given the present state.

**Remark 6.3: Generalizations of Markov chains**

One can also define continuous-state Markov chains (for example, where states are vectors in $\mathbb{R}^d$) and the results which we state for (finite) Markov chains will generally carry over. For a survey, refer to "General state space Markov chains and MCMC algorithms" (Roberts and Rosenthal, 2004).

Moreover, one can also consider continuous-time Markov chains. One example of such a continuous-space and continuous-time Markov chain is the *Wiener process* (cf. Remark 6.23).

We restrict our attention to *time-homogeneous* Markov chains,[3] which

[3] That is, the transition probabilities do not change over time.

can be characterized by a *transition function*,

$$p(x' \mid x) \doteq \mathbb{P}\big(X_{t+1} = x' \mid X_t = x\big). \tag{6.7}$$

As the state space is finite, we can describe the transition function by the *transition matrix*,

$$\boldsymbol{P} \doteq \begin{bmatrix} p(x_1 \mid x_1) & \cdots & p(x_n \mid x_1) \\ \vdots & \ddots & \vdots \\ p(x_1 \mid x_n) & \cdots & p(x_n \mid x_n) \end{bmatrix} \in \mathbb{R}^{n \times n}. \tag{6.8}$$

Note that each row of $\boldsymbol{P}$ must always sum to 1. Such matrices are also called *stochastic*.

The *transition graph* of a Markov chain is a directed graph consisting of vertices $S$ and weighted edges represented by the adjacency matrix $\boldsymbol{P}$.

The current state of the Markov chain at time $t$ is denoted by the probability distribution $q_t$ over states $S$, that is, $X_t \sim q_t$. In the finite setting, $q_t$ is a PMF, which is often written explicitly as the row vector $\boldsymbol{q}_t \in \mathbb{R}^{1 \times |S|}$. The initial state (or prior) of the Markov chain is given as $X_0 \sim q_0$. One iteration of the Markov chain can then be expressed as follows: (?) **Problem 6.1**

$$\boldsymbol{q}_{t+1} = \boldsymbol{q}_t \boldsymbol{P}. \tag{6.9}$$

It is implied directly that we can write the state of the Markov chain at time $t + k$ as

$$\boldsymbol{q}_{t+k} = \boldsymbol{q}_t \boldsymbol{P}^k. \tag{6.10}$$

The entry $\boldsymbol{P}^k(x, x')$ corresponds to the probability of transitioning from state $x \in S$ to state $x' \in S$ in exactly $k$ steps (?). We denote this entry by $p^{(k)}(x' \mid x)$. **Problem 6.2**

In the analysis of Markov chains, there are two main concepts of interest: stationarity and convergence. We begin by introducing stationarity.

### 6.1.1 Stationarity

**Definition 6.4** (Stationary distribution)**.** A distribution $\pi$ is *stationary* with respect to the transition function $p$ iff

$$\pi(x) = \sum_{x' \in S} p(x \mid x')\pi(x') \tag{6.11}$$

holds for all $x \in S$. It follows from Equation (6.9) that equivalently, $\pi$ is stationary w.r.t. a transition matrix $\boldsymbol{P}$ iff

$$\boldsymbol{\pi} = \boldsymbol{\pi} \boldsymbol{P}. \tag{6.12}$$

After entering a stationary distribution $\pi$, a Markov chain will always remain in the stationary distribution. In particular, suppose that $X_t$ is distributed according to $\pi$, then for all $k \geq 0$, $X_{t+k} \sim \pi$.

**Remark 6.5: When does a stationary distribution exist?**

In general, there are Markov chains with infinitely many stationary distributions or no stationary distribution at all. You can find some examples in Figure 6.2.

It can be shown that there exists a unique stationary distribution $\pi$ if the Markov chain is *irreducible*, that is, if every state is reachable from every other state with a positive probability when the Markov chain is run for enough steps. Formally,

$$\forall x, x' \in S. \exists k \in \mathbb{N}. p^{(k)}(x' \mid x) > 0. \tag{6.13}$$

Equivalently, a Markov chain is irreducible iff its transition graph is strongly connected.

### 6.1.2 *Convergence*

Let us now consider Markov chains with a unique stationary distribution.[4] A natural next question is whether this Markov chain converges to its stationary distribution. We say that a Markov chain converges to its stationary distribution iff we have

$$\lim_{t \to \infty} q_t = \pi, \tag{6.14}$$

irrespectively of the initial distribution $q_0$.

[4] Observe that the stationary distribution of an irreducible Markov chain must have full support, that is, assign positive probability to every state.

**Remark 6.6: When does a Markov chain converge?**

Even if a Markov chain has a unique stationary distribution, it does not have to converge to it. Consider example (3) in Figure 6.2. Clearly, $\pi = (\frac{1}{2}, \frac{1}{2})$ is the unique stationary distribution. However, observe that if we start with a suitable initial distribution such as $q_0 = (1, 0)$, at no point in time will the probability of all states be positive, and in particular, the chain will not converge to $\pi$. Instead, the chain behaves periodically, i.e., its state distributions are $q_{2t} = (0, 1)$ and $q_{2t+1} = (1, 0)$ for all $t \in \mathbb{N}$. It turns out that if we exclude such "periodic" Markov chains, then the remaining (irreducible) Markov chains will always converge to their stationary distribution.

Formally, a Markov chain is *aperiodic* if for all states $x \in S$,

$$\exists k_0 \in \mathbb{N}. \forall k \geq k_0. p^{(k)}(x \mid x) > 0. \tag{6.15}$$

In words, a Markov chain is aperiodic iff for every state $x$, the transition graph has a closed path from $x$ to $x$ with length $k$ for all $k \in \mathbb{N}$ greater than some $k_0 \in \mathbb{N}$.

This additional property leads to the concept of ergodicity.

**Definition 6.7** (Ergodicity)**.** A Markov chain is *ergodic* iff there exists a $t \in \mathbb{N}_0$ such that for any $x, x' \in S$ we have

$$p^{(t)}(x' \mid x) > 0, \tag{6.16}$$

whereby $p^{(t)}(x' \mid x)$ is the probability to reach $x'$ from $x$ in exactly $t$ steps. Equivalent conditions are

1. that there exists some $t \in \mathbb{N}_0$ such that all entries of $\boldsymbol{P}^t$ are strictly positive; and
2. that it is irreducible and aperiodic.

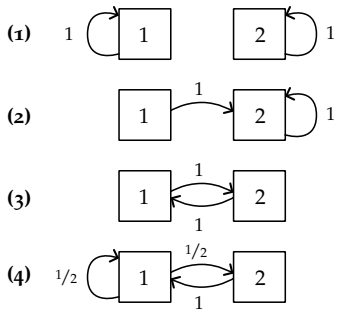



Figure 6.2: Transition graphs of Markov chains: (1) is not ergodic as its transition diagram is not strongly connected; (2) is not ergodic for the same reason; (3) is irreducible but periodic and therefore not ergodic; (4) is ergodic with stationary distribution $\pi(1) = 2/3, \pi(2) = 1/3$.

**Example 6.8: Making a Markov chain ergodic**

A commonly used strategy to ensure that a Markov chain is ergodic is to add "self-loops" to every vertex in the transition graph. That is, to ensure that at any point in time, the Markov chain remains with positive probability in its current state.

Take a (not necessarily ergodic) but irreducible Markov chain with transition matrix $\boldsymbol{P}$. We define the new Markov chain

$$\boldsymbol{P}' \doteq \frac{1}{2}\boldsymbol{P} + \frac{1}{2}\boldsymbol{I}. \tag{6.17}$$

It is a simple exercise to confirm that $\boldsymbol{P}'$ is stochastic, and hence a valid transition matrix. Also, it follows directly that $\boldsymbol{P}'$ is irreducible (as $\boldsymbol{P}$ is irreducible) and aperiodic as every vertex has a closed path of length 1 to itself, and therefore the chain is ergodic.

Take now $\boldsymbol{\pi}$ to be a stationary distribution of $\boldsymbol{P}$. We have that $\boldsymbol{\pi}$ is also a stationary distribution of $\boldsymbol{P}'$ as

$$\boldsymbol{\pi}\boldsymbol{P}' = \frac{1}{2}\boldsymbol{\pi}\boldsymbol{P} + \frac{1}{2}\boldsymbol{\pi}\boldsymbol{I} = \frac{1}{2}\boldsymbol{\pi} + \frac{1}{2}\boldsymbol{\pi} = \boldsymbol{\pi}. \tag{6.18}$$

using (6.12)

**Fact 6.9** (Fundamental theorem of ergodic Markov chains, theorem 4.9 of Levin and Peres (2017))**.** *An ergodic Markov chain has a unique stationary distribution $\pi$ (with full support) and*

$$\lim_{t \to \infty} q_t = \pi \tag{6.19}$$

*irrespectively of the initial distribution $q_0$.*

This naturally suggests constructing an ergodic Markov chain such that its stationary distribution coincides with the posterior distribution. If we then sample "sufficiently long", $X_t$ is drawn from a distribution that is "very close" to the posterior distribution.

**Remark 6.10: How quickly does a Markov chain converge?**

The convergence speed of Markov chains is a rich field of research. "Sufficiently long" and "very close" are commonly made precise by the notions of *rapidly mixing* Markov chains and *total variation distance*.

**Definition 6.11** (Total variation distance)**.** The total variation distance between two probability distributions $\mu$ and $\nu$ on $\mathcal{A}$ is defined by

$$\|\mu - \nu\|_{\mathrm{TV}} \doteq 2 \sup_{A \subseteq \mathcal{A}} |\mu(A) - \nu(A)| . \tag{6.20}$$

It defines the distance between $\mu$ and $\nu$ to be the maximum difference between the probabilities that $\mu$ and $\nu$ assign to the same event.

As opposed to the KL-divergence (5.34), the total variation distance is a metric. In particular, it is symmetric and satisfies the triangle inequality. It can be shown that

$$\|\mu - \nu\|_{\mathrm{TV}} \leq \sqrt{2\mathrm{KL}(\mu \| \nu)} \tag{6.21}$$

which is known as *Pinsker's inequality*. Moreover, if $\mu$ and $\nu$ are discrete distributions over the set $S$, it can be shown that

$$\|\mu - \nu\|_{\mathrm{TV}} = \sum_{i \in S} |\mu(i) - \nu(i)| . \tag{6.22}$$

**Definition 6.12** (Mixing time)**.** For a Markov chain with stationary distribution $\pi$, its *mixing time* with respect to the total variation distance for any $\epsilon > 0$ is

$$\tau_{\mathrm{TV}}(\epsilon) \doteq \min\{t \mid \forall q_0 : \|q_t - \pi\|_{\mathrm{TV}} \leq \epsilon\}. \tag{6.23}$$

Thus, the mixing time measures the time required by a Markov chain for the distance to stationarity to be small. A Markov chain is typically said to be *rapidly mixing* if for any $\epsilon > 0$,

$$\tau_{\mathrm{TV}}(\epsilon) \in O(\mathrm{poly}(n, \log(1/\epsilon))). \tag{6.24}$$

That is, a rapidly mixing Markov chain on $n$ states needs to be simulated for at most $\mathrm{poly}(n)$ steps to obtain a "good" sample from its stationary distribution $\pi$.

You can find a thorough introduction to mixing times in chapter 4 of "Markov chains and mixing times" (Levin and Peres, 2017). Later chapters introduce methods for showing that a Markov chain is rapidly mixing.

### 6.1.3 *Detailed Balance Equation*

How can we confirm that the stationary distribution of a Markov chain coincides with the posterior distribution? The detailed balance equation yields a very simple method.

**Definition 6.13** (Detailed balance equation / reversibility)**.** A Markov chain satisfies the *detailed balance equation* with respect to a distribution $\pi$ iff

$$\pi(x)p(x' \mid x) = \pi(x')p(x \mid x') \tag{6.25}$$

holds for any $x, x' \in S$. A Markov chain that satisfies the detailed balance equation with respect to $\pi$ is called *reversible* with respect to $\pi$.

**Lemma 6.14.** *Given a finite Markov chain, if the Markov chain is reversible with respect to $\pi$ then $\pi$ is a stationary distribution.*[5]

[5] Note that reversibility of $\pi$ is only a sufficient condition for stationarity of $\pi$, it is not necessary! In particular, there are irreversible ergodic Markov chains.

*Proof.* Let $\pi \doteq q_t$. We have,

$$\begin{aligned}
q_{t+1}(x) &= \sum_{x' \in S} p(x \mid x') q_t(x') && \text{using the Markov property (6.6)} \\
&= \sum_{x' \in S} p(x \mid x') \pi(x') \\
&= \sum_{x' \in S} p(x' \mid x) \pi(x) && \text{using the detailed balance equation (6.25)} \\
&= \pi(x) \sum_{x' \in S} p(x' \mid x) \\
&= \pi(x). && \text{using that } \textstyle\sum_{x' \in S} p(x' \mid x) = 1
\end{aligned}$$

□

That is, if we can show that the detailed balance equation (6.25) holds for some distribution $q$, then we know that $q$ is the stationary distribution of the Markov chain.

Next, reconsider our posterior distribution $p(x) = \frac{1}{Z} q(x)$ from Equation (6.4). If we substitute the posterior for $\pi$ in the detailed balance equation, we obtain

$$\frac{1}{Z} q(x) p(x' \mid x) = \frac{1}{Z} q(x') p(x \mid x'), \tag{6.26}$$

or equivalently,

$$q(x)p(x' \mid x) = q(x')p(x \mid x'). \tag{6.27}$$

In words, we do not need to know the true posterior $p$ to check that the stationary distribution of our Markov chain coincides with $p$, it suffices to know the finite measure $q$!

### 6.1.4 Ergodic Theorem

If we now suppose that we can construct a Markov chain whose stationary distribution coincides with the posterior distribution — we will see later that this is possible — it is not apparent that this allows us to estimate expectations over the posterior distribution. Note that although constructing such a Markov chain allows us to obtain samples from the posterior distribution, they are *not* independent. In fact, due to the structure of a Markov chain, by design, they are strongly dependent. Thus, the law of large numbers and Hoeffding's inequality do not apply. By itself, it is not even clear that an estimator relying on samples from a single Markov chain will be unbiased.

Theoretically, we could simulate many Markov chains separately and obtain one sample from each of them. This, however, is extremely inefficient. It turns out that there is a way to generalize the (strong) law of large numbers to Markov chains.

**Theorem 6.15** (Ergodic theorem, appendix C of Levin and Peres (2017))**.** *Given an ergodic Markov chain $(X_t)_{t\in\mathbb{N}_0}$ over a finite state space $S$ with stationary distribution $\pi$ and a function $f : S \to \mathbb{R}$,*

$$\frac{1}{n}\sum_{i=1}^{n} f(x_i) \overset{a.s.}{\to} \sum_{x\in S} \pi(x) f(x) = \mathbb{E}_{x\sim\pi}[f(x)] \tag{6.28}$$

*as $n \to \infty$ where $x_i \sim X_i \mid x_{i-1}$.*

This result is the fundamental reason for why Markov chain Monte Carlo methods are possible. There are analogous results for continuous domains.

Note, however, that the ergodic theorem only tells us that simulating a single Markov chain yields an unbiased estimator. It does not tell us anything about the rate of convergence and variance of such an estimator. The convergence rate depends on the mixing time of the Markov chain, which is difficult to establish in general.

In practice, one observes that Markov chain Monte Carlo methods have a so-called *"burn-in" time* during which the distribution of the Markov chain does not yet approximate the posterior distribution well. Typically, the first $t_0$ samples are therefore discarded,

$$\mathbb{E}[f(X)] \approx \frac{1}{T - t_0} \sum_{t=t_0+1}^{T} f(X_t). \tag{6.29}$$

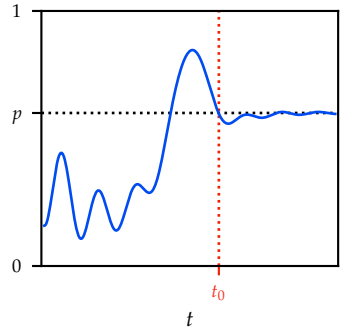



Figure 6.3: Illustration of the "burn-in" time $t_0$ of a Markov chain approximating the posterior $p(y^\star = 1 \mid \boldsymbol{X}, \boldsymbol{y})$ of Bayesian logistic regression. The true posterior $p$ is shown in gray. The distribution of the Markov chain at time $t$ is shown in red.

It is not clear in general how $T$ and $t_0$ should be chosen such that the estimator is unbiased, rather they have to be tuned.

Another widely used heuristic is to first find the mode of the posterior distribution and then start the Markov chain at that point. This tends to increase the rate of convergence drastically, as the Markov chain does not have to "walk to the location in the state space where most probability mass will be located".

## 6.2 *Elementary Sampling Methods*

We will now examine methods for constructing and sampling from a Markov chain with the goal of approximating samples from the posterior distribution $p$. Note that in this setting the state space of the Markov chain is $\mathbb{R}^n$ and a single state at time $t$ is described by the random vector $\mathbf{X} \doteq [X_1, \dots, X_n]$.

### 6.2.1 *Metropolis-Hastings Algorithm*

Suppose we are given a *proposal distribution* $r(\boldsymbol{x}' \mid \boldsymbol{x})$ which, given we are in state $\boldsymbol{x}$, proposes a new state $\boldsymbol{x}'$. Metropolis and Hastings showed that using the *acceptance distribution* $\text{Bern}(\alpha(\boldsymbol{x}' \mid \boldsymbol{x}))$ where

$$\alpha(\boldsymbol{x}' \mid \boldsymbol{x}) \doteq \min\left\{1, \frac{p(\boldsymbol{x}')r(\boldsymbol{x} \mid \boldsymbol{x}')}{p(\boldsymbol{x})r(\boldsymbol{x}' \mid \boldsymbol{x})}\right\} \tag{6.30}$$

$$= \min\left\{1, \frac{q(\boldsymbol{x}')r(\boldsymbol{x} \mid \boldsymbol{x}')}{q(\boldsymbol{x})r(\boldsymbol{x}' \mid \boldsymbol{x})}\right\} \tag{6.31}$$

similarly to the detailed balance equation, the normalizing constant $Z$ cancels

to decide whether to follow the proposal yields a Markov chain with stationary distribution $p(\boldsymbol{x}) = \frac{1}{Z}q(\boldsymbol{x})$.

**Algorithm 6.16:** Metropolis-Hastings algorithm

```
initialize x ∈ ℝ^n
for t = 1 to T do
    sample x' ~ r(x' | x)
    sample u ~ Unif([0, 1])
    if u ≤ α(x' | x) then update x ← x'
    else update x ← x
```

Intuitively, the acceptance distribution corrects for the bias in the proposal distribution. That is, if the proposal distribution $r$ is likely to propose states with low probability under $p$, the acceptance distribution will reject these proposals frequently. The following theorem formalizes this intuition.

**Theorem 6.17** (Metropolis-Hastings theorem)**.** *Given an arbitrary proposal distribution $r$ whose support includes the support of $q$, the stationary distribution of the Markov chain simulated by the Metropolis-Hastings algorithm is $p(\boldsymbol{x}) = \frac{1}{Z} q(\boldsymbol{x})$.*

*Proof.* First, let us define the transition probabilities of the Markov chain. The probability of transitioning from a state $\boldsymbol{x}$ to a state $\boldsymbol{x}'$ is given by $r(\boldsymbol{x}' \mid \boldsymbol{x})\alpha(\boldsymbol{x}' \mid \boldsymbol{x})$ if $\boldsymbol{x} \neq \boldsymbol{x}'$ and the probability of proposing to remain in state $\boldsymbol{x}$, $r(\boldsymbol{x} \mid \boldsymbol{x})$, plus the probability of denying the proposal, otherwise.

$$p(\boldsymbol{x}' \mid \boldsymbol{x}) = \begin{cases} r(\boldsymbol{x}' \mid \boldsymbol{x})\alpha(\boldsymbol{x}' \mid \boldsymbol{x}) & \text{if } \boldsymbol{x} \neq \boldsymbol{x}' \\ r(\boldsymbol{x} \mid \boldsymbol{x}) + \sum_{\boldsymbol{x}'' \neq \boldsymbol{x}} r(\boldsymbol{x}'' \mid \boldsymbol{x})(1 - \alpha(\boldsymbol{x}'' \mid \boldsymbol{x})) & \text{otherwise.} \end{cases} \tag{6.32}$$

We will show that the stationary distribution is $p$ by showing that $p$ satisfies the detailed balance equation (6.25). Let us fix arbitrary states $\boldsymbol{x}$ and $\boldsymbol{x}'$. First, observe that if $\boldsymbol{x} = \boldsymbol{x}'$, then the detailed balance equation is trivially satisfied. Without loss of generality we assume

$$\alpha(\boldsymbol{x} \mid \boldsymbol{x}') = 1, \quad \alpha(\boldsymbol{x}' \mid \boldsymbol{x}) = \frac{q(\boldsymbol{x}')r(\boldsymbol{x} \mid \boldsymbol{x}')}{q(\boldsymbol{x})r(\boldsymbol{x}' \mid \boldsymbol{x})}.$$

For $\boldsymbol{x} \neq \boldsymbol{x}'$, we then have,

$$\begin{aligned}
p(\boldsymbol{x}) \cdot p(\boldsymbol{x}' \mid \boldsymbol{x}) &= \frac{1}{Z} q(\boldsymbol{x}) p(\boldsymbol{x}' \mid \boldsymbol{x}) && \text{using the definition of the distribution } p \\
&= \frac{1}{Z} q(\boldsymbol{x}) r(\boldsymbol{x}' \mid \boldsymbol{x}) \alpha(\boldsymbol{x}' \mid \boldsymbol{x}) && \text{using the transition probabilities of the Markov chain} \\
&= \frac{1}{Z} q(\boldsymbol{x}) r(\boldsymbol{x}' \mid \boldsymbol{x}) \frac{q(\boldsymbol{x}')r(\boldsymbol{x} \mid \boldsymbol{x}')}{q(\boldsymbol{x})r(\boldsymbol{x}' \mid \boldsymbol{x})} && \text{using the definition of the acceptance distribution } \alpha \\
&= \frac{1}{Z} q(\boldsymbol{x}') r(\boldsymbol{x} \mid \boldsymbol{x}') \\
&= \frac{1}{Z} q(\boldsymbol{x}') r(\boldsymbol{x} \mid \boldsymbol{x}') \alpha(\boldsymbol{x} \mid \boldsymbol{x}') && \text{using the definition of the acceptance distribution } \alpha \\
&= \frac{1}{Z} q(\boldsymbol{x}') p(\boldsymbol{x} \mid \boldsymbol{x}') && \text{using the transition probabilities of the Markov chain} \\
&= p(\boldsymbol{x}') \cdot p(\boldsymbol{x} \mid \boldsymbol{x}'). && \text{using the definition of the distribution } p
\end{aligned}$$

□

Note that by the fundamental theorem of ergodic Markov chains (6.19), for convergence to the stationary distribution, it is sufficient for the Markov chain to be ergodic. Ergodicity follows immediately when the transition probabilities $p(\cdot \mid \boldsymbol{x})$ have full support. For example, if the proposal distribution $r(\cdot \mid \boldsymbol{x})$ has full support, the full support of $p(\cdot \mid \boldsymbol{x})$ follows immediately from Equation (6.32). The rate of convergence of Metropolis-Hastings depends strongly on the choice of the proposal distribution, and we will explore different choices of proposal distribution in the following.

### 6.2.2 *Gibbs Sampling*

A popular example of a Metropolis-Hastings algorithm is Gibbs sampling as presented in Algorithm 6.18.

**Algorithm 6.18:** Gibbs sampling

initialize $\boldsymbol{x} = [x_1, \dots, x_n] \in \mathbb{R}^n$
**for** $t = 1$ **to** $T$ **do**
  pick a variable $i$ uniformly at random from $\{1, \dots, n\}$
  set $\boldsymbol{x}_{-i} \doteq [x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n]$
  update $x_i$ by sampling according to the posterior distribution $p(x_i \mid \boldsymbol{x}_{-i})$

Intuitively, by re-sampling single coordinates according to the posterior distribution given the other coordinates, Gibbs sampling finds states that are successively "more" likely. Selecting the index $i$ uniformly at random ensures that the underlying Markov chain is ergodic provided the conditional distributions $p(\cdot \mid \boldsymbol{x}_{-i})$ have full support.

**Theorem 6.19** (Gibbs sampling as Metropolis-Hastings)**.** *Gibbs sampling is a Metropolis-Hastings algorithm. For any fixed $i \in [n]$, it has proposal distribution*

$$r_i(\boldsymbol{x}' \mid \boldsymbol{x}) \doteq \begin{cases} p(x'_i \mid \boldsymbol{x}'_{-i}) & \boldsymbol{x}' \text{ differs from } \boldsymbol{x} \text{ only in entry } i \\ 0 & \text{otherwise} \end{cases} \tag{6.33}$$

*and acceptance distribution $\alpha_i(\boldsymbol{x}' \mid \boldsymbol{x}) \doteq 1$.*

*Proof.* We show that $\alpha_i(\boldsymbol{x}' \mid \boldsymbol{x}) = 1$ follows from the definition of an acceptance distribution in Metropolis-Hastings (6.31) and the choice of proposal distribution (6.33).

By (6.31),

$$\alpha_i(\boldsymbol{x}' \mid \boldsymbol{x}) = \min\left\{1, \frac{p(\boldsymbol{x}')r_i(\boldsymbol{x} \mid \boldsymbol{x}')}{p(\boldsymbol{x})r_i(\boldsymbol{x}' \mid \boldsymbol{x})}\right\}$$

Note that $p(\boldsymbol{x}) = p(x_i, \boldsymbol{x}_{-i}) = p(x_i \mid \boldsymbol{x}_{-i})p(\boldsymbol{x}_{-i})$ using the product rule (1.11). Therefore,

$$\begin{aligned} &= \min\left\{1, \frac{p(x'_i \mid \boldsymbol{x}'_{-i})p(\boldsymbol{x}'_{-i})r_i(\boldsymbol{x} \mid \boldsymbol{x}')}{p(x_i \mid \boldsymbol{x}_{-i})p(\boldsymbol{x}_{-i})r_i(\boldsymbol{x}' \mid \boldsymbol{x})}\right\} \\ &= \min\left\{1, \frac{p(x'_i \mid \boldsymbol{x}'_{-i})p(\boldsymbol{x}'_{-i})p(x_i \mid \boldsymbol{x}_{-i})}{p(x_i \mid \boldsymbol{x}_{-i})p(\boldsymbol{x}_{-i})p(x'_i \mid \boldsymbol{x}'_{-i})}\right\} && \text{using the proposal distribution (6.33)} \\ &= \min\left\{1, \frac{p(\boldsymbol{x}'_{-i})}{p(\boldsymbol{x}_{-i})}\right\} \end{aligned}$$

$= 1.$ □ using that $p(\boldsymbol{x}'_{-i}) = p(\boldsymbol{x}_{-i})$

If the index $i$ is chosen uniformly at random as in Algorithm 6.18, then the proposal distribution is $r(\boldsymbol{x}' \mid \boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n} r_i(\boldsymbol{x}' \mid \boldsymbol{x})$, which analogously to Theorem 6.19 has the associated acceptance distribution

$$\alpha(\boldsymbol{x}' \mid \boldsymbol{x}) = \min\left\{1, \frac{p(\boldsymbol{x}')r(\boldsymbol{x} \mid \boldsymbol{x}')}{p(\boldsymbol{x})r(\boldsymbol{x}' \mid \boldsymbol{x})}\right\} = \min\left\{1, \frac{\sum_{i=1}^{n} p(\boldsymbol{x}')r_i(\boldsymbol{x} \mid \boldsymbol{x}')}{\sum_{i=1}^{n} p(\boldsymbol{x})r_i(\boldsymbol{x}' \mid \boldsymbol{x})}\right\} = 1.$$

**Corollary 6.20** (Convergence of Gibbs sampling)**.** *As Gibbs sampling is a specific example of an MH-algorithm, the stationary distribution of the simulated Markov chain is* $p(\boldsymbol{x})$.

Note that for the proposals of Gibbs sampling, we have

$$p(x_i \mid \boldsymbol{x}_{-i}) = \frac{p(x_i, \boldsymbol{x}_{-i})}{\sum_{x_i} p(x_i, \boldsymbol{x}_{-i})} = \frac{q(x_i, \boldsymbol{x}_{-i})}{\sum_{x_i} q(x_i, \boldsymbol{x}_{-i})}. \tag{6.34}$$

using the definition of condition probability (1.8) and the sum rule (1.7)

Under many models, this probability can be efficiently evaluated due to the conditioning on the remaining coordinates $\boldsymbol{x}_{-i}$. If $X_i$ has finite support, the normalizer can be computed exactly.

## 6.3 Sampling using Gradients

In this section, we discuss more advanced sampling methods. The main idea that we will study is the interpretation of sampling as an optimization problem. We will build towards an optimization view of sampling step-by-step, and first introduce what is commonly called the "energy" of a distribution.

### 6.3.1 Gibbs Distributions

Gibbs distributions are a special class of distributions that are widely used in machine learning, and which are characterized by an energy.

**Definition 6.21** (Gibbs distribution)**.** Formally, a *Gibbs distribution* (also called a *Boltzmann distribution*) is a continuous distribution $p$ whose PDF is of the form

$$p(\boldsymbol{x}) = \frac{1}{Z}\exp(-f(\boldsymbol{x})). \tag{6.35}$$

$f : \mathbb{R}^n \to \mathbb{R}$ is also called an *energy function*. When the energy function $f$ is convex, its Gibbs distribution is called *log-concave*.

A useful property is that Gibbs distributions always have full support.[6] It is often easier to reason about "energies" rather than probabilities as they are neither restricted to be non-negative nor do they have to integrate to 1. Note that the Gibbs distribution belongs to the exponential family (5.48) with sufficient statistic $-f(\boldsymbol{x})$.

[6] This can easily be seen as $\exp(\cdot) > 0$.

Observe that the posterior distribution can always be interpreted as a Gibbs distribution as long as prior and likelihood have full support,

$$p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) = \frac{1}{Z} p(\boldsymbol{\theta}) p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}) \qquad \text{using Bayes' rule (1.45)}$$
$$= \frac{1}{Z} \exp(-[-\log p(\boldsymbol{\theta}) - \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})]). \quad (6.36)$$

Thus, defining the energy function

$$f(\boldsymbol{\theta}) \doteq -\log p(\boldsymbol{\theta}) - \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta}) \quad (6.37)$$
$$= -\log p(\boldsymbol{\theta}) - \sum_{i=1}^{n} \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}), \quad (6.38)$$

yields

$$p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) = \frac{1}{Z} \exp(-f(\boldsymbol{\theta})). \quad (6.39)$$

Note that $f$ coincides with the loss function used for MAP estimation (1.62). For a noninformative prior, the regularization term vanishes and the energy reduces to the negative log-likelihood $\ell_{\text{nll}}(\boldsymbol{\theta}; \mathcal{D})$ (i.e., the loss function of maximum likelihood estimation (1.57)).

Using that the posterior is a Gibbs distribution, we can rewrite the acceptance distribution of Metropolis-Hastings,

$$\alpha(\boldsymbol{x}' \mid \boldsymbol{x}) = \min\left\{1, \frac{r(\boldsymbol{x} \mid \boldsymbol{x}')}{r(\boldsymbol{x}' \mid \boldsymbol{x})} \exp(f(\boldsymbol{x}) - f(\boldsymbol{x}'))\right\}. \quad (6.40)$$

this is obtained by substituting the PDF of a Gibbs distribution for the posterior

**Example 6.22: Metropolis-Hastings with Gaussian proposals**

Let us consider Metropolis-Hastings with the Gaussian proposal distribution,

$$r(\boldsymbol{x}' \mid \boldsymbol{x}) \doteq \mathcal{N}(\boldsymbol{x}'; \boldsymbol{x}, \tau \boldsymbol{I}). \quad (6.41)$$

Due to the symmetry of Gaussians, we have

$$\frac{r(\boldsymbol{x} \mid \boldsymbol{x}')}{r(\boldsymbol{x}' \mid \boldsymbol{x})} = \frac{\mathcal{N}(\boldsymbol{x}; \boldsymbol{x}', \tau \boldsymbol{I})}{\mathcal{N}(\boldsymbol{x}'; \boldsymbol{x}, \tau \boldsymbol{I})} = 1.$$

Hence, the acceptance distribution is defined by

$$\alpha(\boldsymbol{x}' \mid \boldsymbol{x}) = \min\{1, \exp(f(\boldsymbol{x}) - f(\boldsymbol{x}'))\}. \quad (6.42)$$

Intuitively, when a state with lower energy is proposed, that is $f(\boldsymbol{x}') \leq f(\boldsymbol{x})$, then the proposal will always be accepted. In contrast, if the energy of the proposed state is higher, the acceptance probability decreases exponentially in the difference of energies $f(\boldsymbol{x}) - f(\boldsymbol{x}')$. Thus, Metropolis-Hastings minimizes the en-

ergy function, which corresponds to minimizing the negative log-likelihood and negative log-prior. The variance in the proposals $\tau$ helps in getting around local optima, but the search direction is uniformly random (i.e., "uninformed").

### 6.3.2 *From Energy to Surprise (and back)*

Energy-based models are a well-known class of models in machine learning where an energy function $f$ is learned from data. These energy functions do not need to originate from a probability distribution, yet they induce a probability distribution via their Gibbs distribution $p(\boldsymbol{x}) \propto \exp(-f(\boldsymbol{x}))$. As we will see in Problem 6.7, this Gibbs distribution is the associated maximum entropy distribution. Observe that the surprise about $\boldsymbol{x}$ under this distribution is given by

$$\mathrm{S}[p(\boldsymbol{x})] = f(\boldsymbol{x}) + \log Z. \tag{6.43}$$

That is, up to a constant shift, the energy of $\boldsymbol{x}$ *coincides* with the surprise about $\boldsymbol{x}$. Energies are therefore sufficient for comparing the "likelihood" of points, and they do not require normalization.[7]

What kind of energies could we use? In Section 6.3.1, we discussed the use of the negative log-posterior or negative log-likelihood as energies. In general, *any* loss function $\ell(\boldsymbol{x})$ can be thought of as an energy function with an associated maximum entropy distribution $p(\boldsymbol{x}) \propto \exp(-\ell(\boldsymbol{x}))$.

### 6.3.3 *Langevin Dynamics*

Until now, we have looked at Metropolis-Hastings algorithms with proposal distributions that do not explicitly take into account the curvature of the energy function around the current state. Langevin dynamics adapts the Gaussian proposals of the Metropolis-Hastings algorithm we have seen in Example 6.22 to search the state space in an "informed" direction. The simple idea is to bias the sampling towards states with lower energy, thereby making it more likely that a proposal is accepted.

A natural idea is to shift the proposal distribution perpendicularly to the gradient of the energy function. This yields the following proposal distribution,

$$r(\boldsymbol{x}' \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}'; \boldsymbol{x} - \eta_t \nabla f(\boldsymbol{x}), 2\eta_t \boldsymbol{I}). \tag{6.44}$$

The resulting variant of Metropolis-Hastings is known as the *Metropolis adjusted Langevin algorithm* (MALA) or *Langevin Monte Carlo* (LMC).

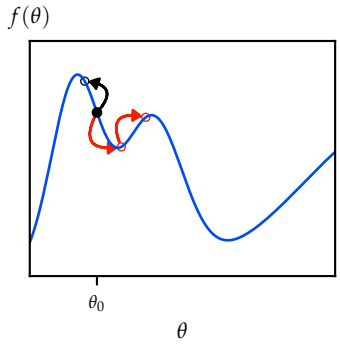



Figure 6.4: Metropolis-Hastings and Langevin dynamics minimize the energy function $f(\theta)$ shown in blue. Suppose we start at the black dot $\theta_0$, then the black and red arrows denote possible subsequent samples. Metropolis-Hastings uses an "uninformed" search direction, whereas Langevin dynamics uses the gradient of $f(\theta)$ to make "more promising" proposals. The random proposals help get past local optima.

[7] Intuitively, an energy can be used to compare the "likelihood" of two points $\boldsymbol{x}$ and $\boldsymbol{x}'$ whereas the probability $\boldsymbol{x}$ makes a statement about the "likelihood" of $\boldsymbol{x}$ relative to *all* other points.

It can be shown that, as $\eta_t \to 0$, we have for the acceptance probability $\alpha(\boldsymbol{x}' \mid \boldsymbol{x}) \to 1$ using that the acceptance probability is 1 if $\boldsymbol{x}' = \boldsymbol{x}$. Hence, the Metropolis-Hastings acceptance step can be omitted once the rejection probability becomes negligible. The algorithm which always accepts the proposal of Equation (6.44) is known as the *unadjusted Langevin algorithm* (ULA).

Observe that if the stationary distribution is the posterior distribution

$$p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n}) = \frac{1}{Z} \exp \underbrace{\left( \log p(\boldsymbol{\theta}) + \sum_{i=1}^{n} \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}) \right)}_{-f(\boldsymbol{\theta})}$$

with energy $f$ as we discussed in Section 6.3.1, then the proposal $\boldsymbol{\theta}'$ of MALA/LMC can be equivalently formulated as

$$\begin{aligned} \boldsymbol{\theta}' &\leftarrow \boldsymbol{\theta} - \eta_t \nabla f(\boldsymbol{\theta}) + \boldsymbol{\varepsilon} \\ &= \boldsymbol{\theta} + \eta_t \left( \nabla \log p(\boldsymbol{\theta}) + \sum_{i=1}^{n} \nabla \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}) \right) + \boldsymbol{\varepsilon} \end{aligned} \tag{6.45}$$

where $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, 2\eta_t \boldsymbol{I})$.

**Remark 6.23: ULA is a discretized diffusion**

The unadjusted Langevin algorithm can be seen as a discretization of Langevin dynamics, which is a continuous-time stochastic process with a drift and with random stationary and independent Gaussian increments. The randomness is modeled by a Wiener process.

**Definition 6.24** (Wiener process)**.** The *Wiener process* (also known as *Brownian motion*) is a sequence of random vectors $\{\mathbf{W}_t\}_{t \geq 0}$ such that

1. $\mathbf{W}_0 = \mathbf{0}$,
2. with probability 1, $\mathbf{W}_t$ is continuous in $t$,
3. the process has independent increments,[8] and
4. $\mathbf{W}_{t+u} - \mathbf{W}_t \sim \mathcal{N}(\mathbf{0}, u\boldsymbol{I})$.

Consider the continuous-time stochastic process $\boldsymbol{\theta}$ defined by the stochastic differential equation (SDE)

$$d\boldsymbol{\theta}_t = \underbrace{-\nabla f(\boldsymbol{\theta}_t)\, dt}_{\text{drift}} + \underbrace{\sqrt{2}\, d\mathbf{W}_t}_{\text{noise}}, \tag{6.46}$$

Such a stochastic process is also called a *diffusion (process)* and Equation (6.46) specifically is called *Langevin dynamics*. Here, the first term is called the "drift" of the process, and the second term is called its "noise". Note that if the noise term is zero then Equa-

[8] That is, the "future" increments $\mathbf{W}_{t+u} - \mathbf{W}_t$ for $u \geq 0$ are independent of past values $\mathbf{W}_s$ for $s < t$.

tion (6.46) is simply an ordinary differential equation (ODE).

A diffusion can be discretized using the Euler-Maruyama method (also called "forward Euler") to obtain a discrete approximation $\boldsymbol{\theta}_k \approx \boldsymbol{\theta}(\tau_k)$ where $\tau_k$ denotes the $k$-th time step. Choosing the time steps such that $\Delta t_k \doteq \tau_{k+1} - \tau_k = \eta_k$ yields the approximation

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \boldsymbol{\nabla} f(\boldsymbol{\theta}_k)\, \Delta t_k + \sqrt{2}\, \Delta \mathbf{W}_k \tag{6.47}$$

where $\Delta \mathbf{W}_k \doteq \mathbf{W}_{\tau_{k+1}} - \mathbf{W}_{\tau_k} \sim \mathcal{N}(\mathbf{0}, \Delta t_k \boldsymbol{I})$. Observe that this coincides with the update rule of Langevin dynamics from Equation (6.45).

The appearance of drift and noise is closely related to the *principle of curiosity and conformity*, we encountered in the previous chapter on variational inference. The noise term (also called the *diffusion*) leads to exploration of the state space (i.e., curiosity about alternative explanations of the data), whereas the drift term (also called the *distillation*) leads to minimizing the energy or loss (i.e., conformity to the data). Interestingly, this same principle appears in both variational inference and MCMC, two very different approaches to approximate probabilistic inference. In the remainder of this manuscript we will find this to be a reoccurring theme.

For log-concave distributions, the mixing time of the Markov chain underlying Langevin dynamics can be shown to be polynomial in the dimension $n$ (Vempala and Wibisono, 2019). You will prove this result for strongly log-concave distributions in ⓘ and see that the analysis is analogous to the canonical convergence analysis of classical optimization schemes.

**Problem 6.9**

### 6.3.4 Stochastic Gradient Langevin Dynamics

Note that computing the gradient of the energy function, which corresponds to computing exact gradients of the log-prior and log-likelihood, in every step can be expensive. The proposal step of MALA/LMC can be made more efficient by approximating the gradient with an unbiased gradient estimate, leading to *stochastic gradient Langevin dynamics* (SGLD) shown in Algorithm 6.25 (Welling and Teh, 2011). Observe that SGLD (6.48) differs from MALA/LMC (6.45) only by using a sample-based approximation of the gradient.

Intuitively, in the initial phase of the algorithm, the stochastic gradient term dominates, and therefore, SGLD corresponds to a variant of stochastic gradient ascent. In the later phase, the update rule is dominated by the injected noise $\boldsymbol{\varepsilon}$, and will effectively be Langevin

**Algorithm 6.25:** Stochastic gradient Langevin dynamics, SGLD

initialize $\boldsymbol{\theta}$
**for** $t = 1$ **to** $T$ **do**
  sample $i_1, \dots, i_m \sim \text{Unif}(\{1, \dots, n\})$ independently
  sample $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, 2\eta_t \boldsymbol{I})$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta_t \Big( \boldsymbol{\nabla} \log p(\boldsymbol{\theta}) + \frac{n}{m} \sum_{j=1}^{m} \boldsymbol{\nabla} \log p(y_{i_j} \mid \boldsymbol{x}_{i_j}, \boldsymbol{\theta}) \Big) + \boldsymbol{\varepsilon}$ // (6.48)

dynamics. SGLD transitions smoothly between the two phases.

Under additional assumptions, SGLD is guaranteed to converge to the posterior distribution for decreasing learning rates $\eta_t = \Theta(t^{-1/3})$ (Raginsky et al., 2017; Xu et al., 2018). SGLD does not use the acceptance step from Metropolis-Hastings as asymptotically, SGLD corresponds to Langevin dynamics and the Metropolis-Hastings rejection probability goes to zero for a decreasing learning rate.

### 6.3.5 *Hamiltonian Monte Carlo*

As MALA and SGLD can be seen as a sampling-based analogue of GD and SGD, a similar analogue for (stochastic) gradient descent with momentum is the (stochastic gradient) *Hamiltonian Monte Carlo* (HMC) algorithm, which we discuss in the following (Duane et al., 1987; Chen et al., 2014).

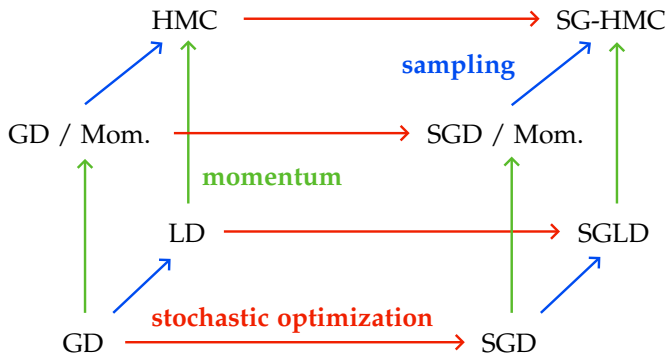



Figure 6.5: A commutative diagram of sampling and optimization algorithms. Langevin dynamics (LD) is the non-stochastic variant of SGLD.

We have seen that if we want to sample from a distribution

$$p(\boldsymbol{x}) \propto \exp(-f(\boldsymbol{x}))$$

with energy function $f$, we can construct a Markov chain whose distribution converges to $p$. We have also seen that for this approach to

work, the chain must move through all areas of significant probability with reasonable speed.

If one is faced with a distribution $p$ which is multimodal (i.e., that has several "peaks"), one has to ensure that the chain will explore all modes, and can therefore "jump between different areas of the space".

So in general, *local* updates are doomed to fail. Methods such as Metropolis-Hastings with Gaussian proposals, or even Langevin Monte Carlo might face this issue, as they do not jump to distant areas of the state space with significant acceptance probability. It will therefore take a long time to move from one peak to another.

The HMC algorithm is an instance of Metropolis-Hastings which uses momentum to propose distant points that conserve energy, with high acceptance probability. The general idea of HMC is to *lift* samples $\boldsymbol{x}$ to a higher-order space by considering an auxiliary variable $\boldsymbol{y}$ with the same dimension as $\boldsymbol{x}$. We also lift the distribution $p$ to a distribution on the $(\boldsymbol{x}, \boldsymbol{y})$-space by defining a distribution $p(\boldsymbol{y} \mid \boldsymbol{x})$ and setting $p(\boldsymbol{x}, \boldsymbol{y}) \doteq p(\boldsymbol{y} \mid \boldsymbol{x}) p(\boldsymbol{x})$. It is common to pick $p(\boldsymbol{y} \mid \boldsymbol{x})$ to be a Gaussian with zero mean and variance $m\boldsymbol{I}$. Hence,

$$p(\boldsymbol{x}, \boldsymbol{y}) \propto \exp\left(-\frac{1}{2m} \|\boldsymbol{y}\|_2^2 - f(\boldsymbol{x})\right). \tag{6.49}$$

Physicists might recognize the above as the canonical distribution of a Newtonian system if one takes $\boldsymbol{x}$ as the position and $\boldsymbol{y}$ as the momentum. $H(\boldsymbol{x}, \boldsymbol{y}) \doteq \frac{1}{2m} \|\boldsymbol{y}\|_2^2 + f(\boldsymbol{x})$ is called the *Hamiltonian*. HMC then takes a step in this higher-order space according to the Hamiltonian dynamics,[9]

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{\nabla}_{\boldsymbol{y}} H, \quad \frac{d\boldsymbol{y}}{dt} = -\boldsymbol{\nabla}_{\boldsymbol{x}} H, \tag{6.50}$$

reaching some new point $(\boldsymbol{x}', \boldsymbol{y}')$ and *projecting* back to the state space by selecting $\boldsymbol{x}'$ as the new sample. This is illustrated in Figure 6.6. In the next iteration, we resample the momentum $\boldsymbol{y}' \sim p(\cdot \mid \boldsymbol{x}')$ and repeat the procedure.

[9] That is, HMC follows the trajectory of these dynamics for some time.

In an implementation of this algorithm, one has to solve Equation (6.50) numerically rather than exactly. Typically, this is done using the *Leapfrog method*, which for a step size $\tau$ computes

$$\boldsymbol{y}(t + \tau/2) = \boldsymbol{y}(t) - \frac{\tau}{2} \boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x}(t)) \tag{6.51a}$$

$$\boldsymbol{x}(t + \tau) = \boldsymbol{x}(t) + \frac{\tau}{m} \boldsymbol{y}(t + \tau/2) \tag{6.51b}$$

$$\boldsymbol{y}(t + \tau) = \boldsymbol{y}(t + \tau/2) - \frac{\tau}{2} \boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x}(t + \tau)). \tag{6.51c}$$

Then, one repeats this procedure $L$ times to arrive at a point $(\boldsymbol{x}', \boldsymbol{y}')$. To correct for the resulting discretization error, the proposal is either

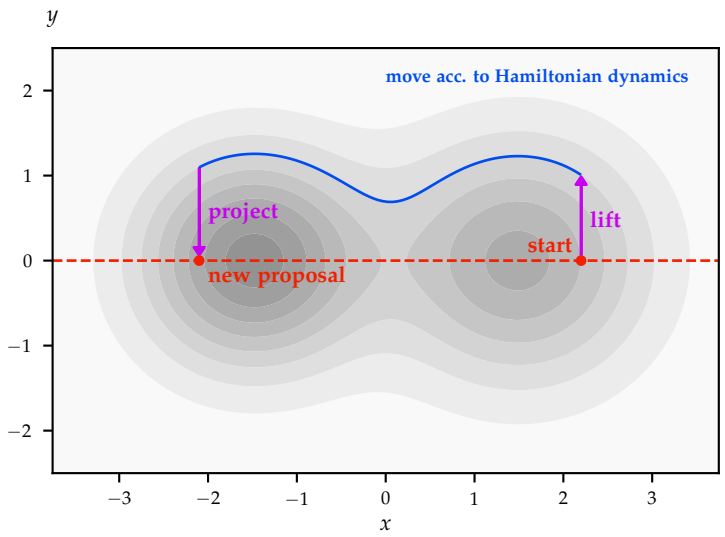



Figure 6.6: Illustration of Hamiltonian Monte Carlo. Shown is the contour plot of a distribution $p$, which is a mixture of two Gaussians, in the $(x, y)$-space.

First, the initial point in the state space is lifted to the $(x, y)$-space. Then, we move according to Hamiltonian dynamics and finally project back onto the state space.

accepted or rejected in a final Metropolis-Hastings acceptance step. If the proposal distribution is symmetric (which we will confirm in a moment), the acceptance probability is

$$\alpha((\boldsymbol{x}', \boldsymbol{y}') \mid (\boldsymbol{x}, \boldsymbol{y})) \doteq \min\{1, \exp(H(\boldsymbol{x}, \boldsymbol{y}) - H(\boldsymbol{x}', \boldsymbol{y}'))\}. \tag{6.52}$$

It follows that $p(\boldsymbol{x}, \boldsymbol{y})$ is the stationary distribution of the Markov chain underlying HMC. Due to the independence of $\boldsymbol{x}$ and $\boldsymbol{y}$, this also implies that the projection to $\boldsymbol{x}$ yields a Markov chain with stationary distribution $p(\boldsymbol{x})$.

So why is the proposal distribution symmetric? This follows from the time-reversibility of Hamiltonian dynamics. It is straightforward to check that the dynamics from Equation (6.50) are identical if we replace $t$ with $-t$ and $\boldsymbol{y}$ with $-\boldsymbol{y}$. Intuitively, unlike the position $\boldsymbol{x}$, the momentum $\boldsymbol{y}$ is reversed when time is reversed as it depends on the velocity which is the time-derivative of the position.[10] In simpler terms, time-reversibility states that if we observe the evolution of a system (e.g., two billiard balls colliding), we cannot distinguish whether we are observing the system evolve forward or backward in time. The Leapfrog method maintains the time-reversibility of the dynamics.

Symmetry of the proposal distribution is ensured by proposing the point $(\boldsymbol{x}', -\boldsymbol{y}')$.[11] Intuitively, this is simply to ensure that the system is run backward in time as often as it is run forward in time. Recall that the momentum is resampled before each iteration (i.e., the proposed momentum is "discarded") and observe that $p(\boldsymbol{x}', -\boldsymbol{y}') = p(\boldsymbol{x}', \boldsymbol{y}')$,[12] so we can safely disregard the direction of time when computing the acceptance probability in Equation (6.52).

[10] The momentum of the $i$-th coordinate is $y_i = m_i v_i$ where $m_i$ is the mass and $v_i = \frac{dx_i}{dt}$ is the velocity.

[11] More formally, the proposal distribution is the Dirac delta at $(\boldsymbol{x}', -\boldsymbol{y}')$.

[12] We use here that $p(\boldsymbol{y} \mid \boldsymbol{x})$ was chosen to be symmetric around zero.

## *Discussion*

To summarize, Markov chain Monte Carlo methods use a Markov chain to approximately sample from an intractable distribution. Note that unlike for variational inference, the convergence of many methods can be guaranteed. Moreover, for log-concave distributions (e.g., with Bayesian logistic regression), the underlying Markov chain converges quickly to the stationary distribution. Methods such as Langevin dynamics and Hamiltonian Monte Carlo aim to accelerate mixing by proposing points with a higher acceptance probability than Metropolis-Hastings with "undirected" Gaussian proposals. Nevertheless, in general, the convergence (mixing time) may be slow, meaning that, in practice, accuracy and efficiency have to be traded.

**Optional Readings**

- Ma, Chen, Jin, Flammarion, and Jordan (2019).
  *Sampling can be faster than optimization*.
- Teh, Thiery, and Vollmer (2016).
  *Consistency and fluctuations for stochastic gradient Langevin dynamics*.
- Chen, Fox, and Guestrin (2014).
  *Stochastic gradient hamiltonian monte carlo*.

## *Problems*

**6.1. Markov chain update.**

Prove Equation (6.9), i.e., that one iteration of the Markov chain can be expressed as $\boldsymbol{q}_{t+1} = \boldsymbol{q}_t \boldsymbol{P}$.

**6.2. $k$-step transitions.**

Prove that the entry $\boldsymbol{P}^k(x, x')$ corresponds to the probability of transitioning from state $x \in S$ to state $x' \in S$ in exactly $k$ steps.

**6.3. Finding stationary distributions.**

A news station classifies each day as "good", "fair", or "poor" based on its daily ratings which fluctuate with what is occurring in the news. Moreover, the following table shows the probabilistic relationship between the type of the current day and the probability of the type of the next day conditioned on the type of the current day.

| | | next day | | |
|---|---|---|---|---|
| | | good | fair | poor |
| current day | good | 0.60 | 0.30 | 0.10 |
| | fair | 0.50 | 0.25 | 0.25 |
| | poor | 0.20 | 0.40 | 0.40 |

In the long run, what percentage of news days will be classified as "good"?

**6.4. Example of Metropolis-Hastings.**

Consider the state space $\{0,1\}^n$ of binary strings having length $n$. Let the proposal distribution be $r(x' \mid x) = 1/n$ if $x'$ differs from $x$ in exactly one bit and $r(x' \mid x) = 0$ otherwise. Suppose we desire a stationary distribution $p$ for which $p(x)$ is proportional to the number of ones that occur in the bit string $x$. For example, in the long run, a random walk should visit a string having five 1s five times as often as it visits a string having only a single 1. Provide a general formula for the acceptance probability $\alpha(x' \mid x)$ that would be used if we were to obtain the desired stationary distribution used the Metropolis-Hastings algorithm.

**6.5. Practical examples of Gibbs sampling.**

In this exercise, we look at some examples where Gibbs sampling is useful.

1. Consider the distribution

$$p(x,y) \doteq \binom{n}{x} y^{x+\alpha-1}(1-y)^{n-x+\beta-1}, \quad x \in [n], y \in [0,1].$$

   Convince yourself that it is hard to sample directly from $p$ and prove that it is an easy task if one uses Gibbs sampling. That is, show that the conditional distributions $p(x \mid y)$ and $p(y \mid x)$ are easy to sample from.
   *Hint: Take a look at the Beta distribution* (1.50).

2. Consider the following generative model $p(\mu, \lambda, x_{1:n})$ given by the likelihood $x_{1:n} \mid \mu, \lambda \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \lambda^{-1})$ and the independent priors

$$\mu \sim \mathcal{N}(\mu_0, \lambda_0^{-1}) \quad \text{and} \quad \lambda \sim \text{Gamma}(\alpha, \beta).$$

   We would like to sample from the posterior $p(\mu, \lambda \mid x_{1:n})$. Show that

$$\mu \mid \lambda, x_{1:n} \sim \mathcal{N}(m_\lambda, l_\lambda^{-1}) \quad \text{and} \quad \lambda \mid \mu, x_{1:n} \sim \text{Gamma}(a_\mu, b_\mu),$$

   and derive $m_\lambda, l_\lambda, a_\mu, b_\mu$. Such a prior is called a *semi-conjugate prior* to the likelihood, as the prior on $\mu$ is conjugate for any fixed value of $\lambda$ and vice-versa.

**Gamma distribution** The PDF of the *gamma distribution* $\text{Gamma}(\alpha, \beta)$ is defined as

$$\text{Gamma}(x; \alpha, \beta) \propto x^{\alpha-1}e^{-\beta x}, \quad x \in \mathbb{R}_{>0}.$$

A random variable $X \sim \text{Gamma}(\alpha, \beta)$ measures the waiting time until $\alpha > 0$ events occur in a Poisson process with rate $\beta > 0$. In particular, when $\alpha = 1$ then the gamma distribution coincides with the exponential distribution with rate $\beta$.

3. Let us assume that $x_{1:n} \mid \alpha, c \overset{\text{iid}}{\sim} \text{Pareto}(\alpha, c)$ and assume the improper prior $p(\alpha, c) \propto \mathbb{1}\{\alpha, c > 0\}$ which corresponds to a noninformative prior. Derive the posterior $p(\alpha, c \mid x_{1:n})$. Then, also derive the conditional distributions $p(\alpha \mid c, x_{1:n})$ and $p(c \mid \alpha, x_{1:n})$, and observe that they correspond to known distributions / are easy to sample from.

**6.6. Energy function of Bayesian logistic regression.**

Recall from Equation (5.12) that the energy function of Bayesian logistic regression is

$$f(\boldsymbol{w}) = \lambda \|\boldsymbol{w}\|_2^2 + \sum_{i=1}^{n} \log(1 + \exp(-y_i \boldsymbol{w}^\top \boldsymbol{x}_i)), \tag{6.53}$$

which coincided with the standard optimization objective of (regularized) logistic regression.

Show that the posterior distribution of Bayesian logistic regression is log-concave.

**6.7. Maximum entropy property of Gibbs distribution.**

1. Let $X$ be a random variable supported on the finite set $\mathcal{T} \subset \mathbb{R}$.[13] Show that the Gibbs distribution with energy function $\frac{1}{T} f(x)$ for some temperature scalar $T \in \mathbb{R}$ is the distribution with maximum entropy of all distributions supported on $\mathcal{T}$ that satisfy the constraint $\mathbb{E}[f(X)] < \infty$.
   *Hint: Solve the dual problem (analogously to Problem 5.7).*
2. What happens for $T \to \{0, \infty\}$?

[13] The same result can be shown to hold for arbitrary compact subsets.

**6.8. Energy reduction of Gibbs sampling.**

Let $p(\boldsymbol{x})$ be a probability density over $\mathbb{R}^d$, which we want to sample from. Assume that $p$ is a Gibbs distribution with energy function $f : \mathbb{R}^d \to \mathbb{R}$.

In this exercise, we will study a single round of Gibbs sampling with initial state $\boldsymbol{x}$ and final state $\boldsymbol{x}'$ where

$$x'_j = \begin{cases} x'_i & \text{if } j = i \\ x_j & \text{otherwise} \end{cases}$$

for some fixed index $i$ and $x'_i \sim p(\cdot \mid \boldsymbol{x}_{-i})$.

Show that

$$\mathbb{E}_{x'_i \sim p(\cdot \mid \boldsymbol{x}_{-i})}\left[f(\boldsymbol{x}')\right] \leq f(\boldsymbol{x}) - \mathrm{S}[p(x_i \mid \boldsymbol{x}_{-i})] + \mathrm{H}[p(\cdot \mid \boldsymbol{x}_{-i})]. \tag{6.54}$$

That is, the energy is expected to decrease if the surprise of $x_i$ given $\boldsymbol{x}_{-i}$ is larger than the expected surprise of the new $x'_i$ given $\boldsymbol{x}_{-i}$, i.e., $\mathrm{S}[p(x_i \mid \boldsymbol{x}_{-i})] \geq \mathrm{H}[p(\cdot \mid \boldsymbol{x}_{-i})]$.

*Hint: Recall the framing of Gibbs sampling as a variant of Metropolis-Hastings and relate this to the acceptance distribution of Metropolis-Hastings when $p$ is a Gibbs distribution.*

**6.9. Mixing time of Langevin dynamics.**

In this exercise, we will show that for certain Gibbs distributions, $p(\boldsymbol{\theta}) \propto \exp(-f(\boldsymbol{\theta}))$, Langevin dynamics is rapidly mixing. To do this, we will observe that Langevin dynamics can be seen as a continuous-time optimization algorithm in the space of distributions.

First, we consider a simpler and more widely-known optimization algorithm, namely the *gradient flow*

$$d\boldsymbol{x}_t = -\boldsymbol{\nabla} f(\boldsymbol{x}_t)\, dt. \tag{6.55}$$

Note that gradient descent is simply the discrete-time approximation of gradient flow just as ULA is the discrete-time approximation of Langevin dynamics. In the analysis of ODEs such as the gradient flow, so-called *Lyapunov functions* are commonly used to prove convergence of $\boldsymbol{x}_t$ to a fixed point (also called an *equilibrium*).

Let us assume that $f$ is $\alpha$-strongly convex for some $\alpha > 0$, that is,

$$f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \boldsymbol{\nabla} f(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x}) + \frac{\alpha}{2} \|\boldsymbol{y} - \boldsymbol{x}\|_2^2 \quad \forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n. \tag{6.56}$$

In words, $f$ is lower bounded by a quadratic function with curvature $\alpha$. Moreover, assume w.l.o.g. that $f$ minimized at $f(\mathbf{0}) = 0$.[14]

[14] This can always be achieved by shifting the coordinate system and subtracting a constant from $f$.

1. Show that $f$ satisfies the *Polyak-Łojasiewicz (PL) inequality*, i.e.,

$$f(\boldsymbol{x}) \leq \frac{1}{2\alpha} \|\boldsymbol{\nabla} f(\boldsymbol{x})\|_2^2 \quad \forall \boldsymbol{x} \in \mathbb{R}^n. \tag{6.57}$$

2. Prove $\frac{d}{dt} f(\boldsymbol{x}_t) \leq -2\alpha f(\boldsymbol{x}_t)$.

Thus, $\mathbf{0}$ is the fixed point of Equation (6.55) and the Lyapunov function $f$ is monotonically decreasing along the trajectory of $\boldsymbol{x}_t$. We recall *Grönwall's inequality* which states that for any real-valued continuous functions $g(t)$ and $\beta(t)$ on the interval $[0, T] \subset \mathbb{R}$ such that $\frac{d}{dt} g(t) \leq \beta(t) g(t)$ for all $t \in [0, T]$ we have

$$g(t) \leq g(0) \exp\left( \int_0^t \beta(s)\, ds \right) \quad \forall t \in [0, T]. \tag{6.58}$$

3. Conclude that $f(\boldsymbol{x}_t) \leq e^{-2\alpha t} f(\boldsymbol{x}_0)$.

Now that we have proven the convergence of gradient flow using $f$ as Lyapunov function, we will follow the same template to prove the convergence of Langevin dynamics to the distribution $p(\boldsymbol{\theta}) \propto \exp(-f(\boldsymbol{\theta}))$.

We will use that the evolution of $\{\boldsymbol{\theta}_t\}_{t\geq 0}$ following the Langevin dynamics (6.46) is equivalently characterized by their densities $\{q_t\}_{t\geq 0}$ following the *Fokker-Planck equation*

$$\frac{\partial q_t}{\partial t} = \boldsymbol{\nabla} \cdot (q_t \boldsymbol{\nabla} f) + \Delta q_t. \tag{6.59}$$

Here, $\boldsymbol{\nabla}\cdot$ and $\Delta$ are the divergence and Laplacian operators, respectively.[15] Intuitively, the first term of the Fokker-Planck equation corresponds to the drift and its second term corresponds to the diffusion (i.e., the Gaussian noise).

[15] For ease of notation, we omit the explicit dependence of $q_t$, $p$, and $f$ on $\boldsymbol{\theta}$.

**Remark 6.26: Intuition on vector calculus**
Recall that the divergence $\boldsymbol{\nabla} \cdot \mathbf{F}$ of a vector field $\mathbf{F}$ measures the change of volume under the flow of $\mathbf{F}$. That is, if in the small neighborhood of a point $\boldsymbol{x}$, $\mathbf{F}$ points towards $\boldsymbol{x}$, then the divergence at $\boldsymbol{x}$ is negative as the volume shrinks. If $\mathbf{F}$ points away from $\boldsymbol{x}$, then the divergence at $\boldsymbol{x}$ is positive as the volume increases.
The Laplacian $\Delta\varphi = \boldsymbol{\nabla} \cdot (\boldsymbol{\nabla}\varphi)$ of a scalar field $\varphi$ can be understood intuitively as measuring "heat dissipation". That is, if $\varphi(\boldsymbol{x})$ is smaller than the average value of $\varphi$ in a small neighborhood of $\boldsymbol{x}$, then the Laplacian at $\boldsymbol{x}$ is positive.

Regarding the Fokker-Planck equation (6.59), the second term $\Delta q_t$ can therefore be understood as locally dissipating the probability mass of $q_t$ (which is due to the diffusion term in the SDE). On the other hand, the term $\boldsymbol{\nabla} \cdot (q_t \boldsymbol{\nabla} f)$ can be understood as a Laplacian of $f$ "weighted" by $q_t$. Intuitively, the vector field $\boldsymbol{\nabla} f$ moves flow in the direction of high energy, and hence, its divergence is larger in regions of lower energy and smaller in regions of higher energy. This term therefore corresponds to a drift from regions of high energy to regions of low energy.

4. Show that $\Delta q_t = \boldsymbol{\nabla} \cdot (q_t \boldsymbol{\nabla} \log q_t)$, implying that the Fokker-Planck equation simplifies to

$$\frac{\partial q_t}{\partial t} = \boldsymbol{\nabla} \cdot \left( q_t \boldsymbol{\nabla} \log \frac{q_t}{p} \right). \tag{6.60}$$

   *Hint: The Laplacian of a scalar field $\varphi$ is $\Delta\varphi \doteq \boldsymbol{\nabla} \cdot (\boldsymbol{\nabla}\varphi)$.*

Observe that the Fokker-Planck equation already implies that $p$ is indeed a stationary distribution, as if $q_t = p$ then $\frac{\partial q_t}{\partial t} = 0$. Moreover, note the similarity of the integrand of $\mathrm{KL}(q_t \| p)$, $q_t \log \frac{q_t}{p}$, to Equation (6.60). We will therefore use the KL-divergence as Lyapunov function.

5. Prove $\frac{d}{dt}\mathrm{KL}(q_t \| p) = -\mathrm{J}(q_t \| p)$. Here,

$$\mathrm{J}(q_t \| p) \doteq \mathbb{E}_{\boldsymbol{\theta}\sim q_t}\left[ \left\| \boldsymbol{\nabla} \log \frac{q_t(\boldsymbol{\theta})}{p(\boldsymbol{\theta})} \right\|_2^2 \right] \tag{6.61}$$

denotes the *relative Fisher information* of $p$ with respect to $q_t$. *Hint: For any distribution $q$ on $\mathbb{R}^n$,*

$$\int_{\mathbb{R}^n} (\boldsymbol{\nabla} \cdot q\mathbf{F})\varphi \, d\boldsymbol{x} = -\int_{\mathbb{R}^n} q \, \boldsymbol{\nabla}\varphi \cdot \mathbf{F} \, d\boldsymbol{x} \tag{6.62}$$

*follows for any vector field $\mathbf{F}$ and scalar field $\varphi$ from the divergence theorem and the product rule of the divergence operator.*

Thus, the relative Fisher information can be seen as the negated time-derivative of the KL-divergence, and as $\mathrm{J}(q_t \| p) \geq 0$ it follows that the KL-divergence is decreasing along the trajectory.

The *log-Sobolev inequality* (LSI) is satisfied by a distribution $p$ with a constant $\alpha > 0$ if for all $q$:

$$\mathrm{KL}(q\|p) \leq \frac{1}{2\alpha}\mathrm{J}(q\|p). \tag{6.63}$$

It is a classical result that if $f$ is $\alpha$-strongly convex then $p$ satisfies the LSI with constant $\alpha$ (Bakry and Émery, 2006).

6. Show that if $f$ is $\alpha$-strongly convex for some $\alpha > 0$ (we say that $p$ is "strongly log-concave"), then $\mathrm{KL}(q_t\|p) \leq e^{-2\alpha t}\mathrm{KL}(q_0\|p)$.
7. Conclude that under the same assumption on $f$, Langevin dynamics is rapidly mixing, i.e., $\tau_{\mathrm{TV}}(\epsilon) \in O(\mathrm{poly}(n, \log(1/\epsilon)))$.

To summarize, we have seen that Langevin dynamics is an optimization scheme in the space of distributions, and that its convergence can be analyzed analogously to classical optimization schemes. Notably, in this exercise we have studied continuous-time Langevin dynamics. Convergence guarantees for discrete-time approximations can be derived using the same techniques. If this interests you, refer to "Rapid convergence of the unadjusted Langevin algorithm: Isoperimetry suffices" (Vempala and Wibisono, 2019).

**6.10. Hamiltonian Monte Carlo.**

1. Prove that if the dynamics are solved exactly (as opposed to numerically using the Leapfrog method), then the acceptance probability of the MH-step is always 1.
2. Prove that the Langevin Monte Carlo algorithm from (6.44) can be seen as a special case of HMC if only one Leapfrog step is used ($L = 1$) and $m = 1$.

# 7
# *Deep Learning*

We began our journey through probabilistic machine learning with linear models. In most practical applications, however, it is seen that models perform better when labels may nonlinearly depend on the inputs, and for this reason linear models are often used in conjunction with "hand-designed" nonlinear features. Designing these features is costly, time-consuming, and requires expert knowledge. Moreover, the design of such features is inherently limited by human comprehension and ingenuity.

## *7.1 Artificial Neural Networks*

One widely used family of nonlinear functions are *artificial "deep" neural networks*,[1]

$$f : \mathbb{R}^d \to \mathbb{R}^k, \quad f(x; \theta) \doteq \varphi(W_L \varphi(W_{L-1}(\cdots \varphi(W_1 x)))) \tag{7.1}$$

where $\theta \doteq [W_1, \ldots, W_L]$ is a vector of *weights* (written as matrices $W_l \in \mathbb{R}^{n_l \times n_{l-1}}$)[2] and $\varphi : \mathbb{R} \to \mathbb{R}$ is a component-wise nonlinear function. Thus, a deep neural network can be seen as nested ("deep") linear functions composed with nonlinearities. This simple kind of neural network is also called a *multilayer perceptron*.

[1] In the following, we will refrain from using the characterizations "artificial" and "deep" for better readability.

[2] where $n_0 = d$ and $n_L = k$

In this chapter, we will be focusing mostly on performing probabilistic inference with a *given* neural network architecture. To this end, understanding the basic architecture of a multilayer perceptron will be sufficient for us. For a more thorough introduction to the field of deep learning and various architectures, you may refer to the books of Goodfellow et al. (2016) and Prince (2023).

A neural network can be visualized by a *computation graph*. An example for such a computation graph is given in Figure 7.1. The columns of the computation graph are commonly called *layers*, whereby the left-most column is the *input layer*, the right-most column is the *output*

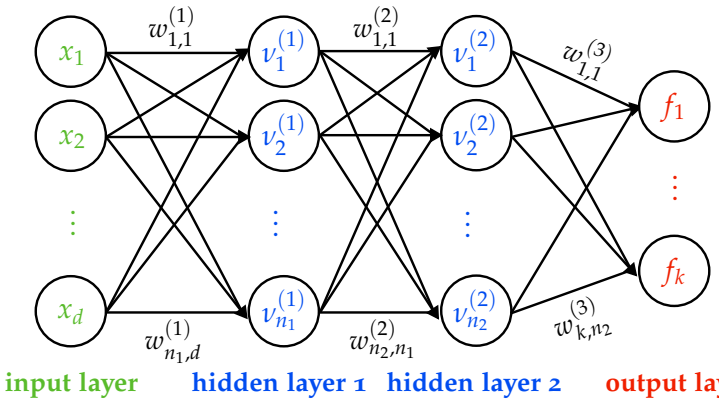



Figure 7.1: Computation graph of a neural network with two hidden layers.

*layer*, and the remaining columns are the *hidden layers*. The inputs are (as we have previously) referred to as $\boldsymbol{x} \doteq [x_1, \dots, x_d]$. The outputs (i.e., vertices of the output layer) are often referred to as *logits* and named $\boldsymbol{f} \doteq [f_1, \dots, f_k]$. The *activations* of an individual (hidden) layer $l$ of the neural network are described by

$$\boldsymbol{v}^{(l)} \doteq \boldsymbol{\varphi}(\boldsymbol{W}_l \boldsymbol{v}^{(l-1)}) \tag{7.2}$$

where $\boldsymbol{v}^{(0)} = \boldsymbol{x}$. The activation of the $i$-th node is $v_i^{(l)} = \boldsymbol{v}^{(l)}(i)$.

### 7.1.1 Activation Functions

The nonlinearity $\varphi$ is called an *activation function*. The following two activation functions are particularly common:

1. The *hyperbolic tangent* (Tanh) is defined as

$$\operatorname{Tanh}(z) \doteq \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)} \in (-1, 1). \tag{7.3}$$

   Tanh is a scaled and shifted variant of the sigmoid function (5.9) which we have previously seen in the context of logistic regression as $\operatorname{Tanh}(z) = 2\sigma(2z) - 1$.
2. The *rectified linear unit* (ReLU) is defined as

$$\operatorname{ReLU}(z) \doteq \max\{z, 0\} \in [0, \infty). \tag{7.4}$$

   In particular, the ReLU activation function leads to "sparser" gradients as it selects the halfspace of inputs with positive sign. Moreover, the gradients of ReLU do not "vanish" as $z \to \pm\infty$ which can lead to faster training.

It is important that the activation function is nonlinear because otherwise, any composition of layers would still represent a linear function. Non-linear activation functions allow the network to represent arbitrary functions. This is known as the *universal approximation theorem*, and it states that any artificial neural network with just a single hidden

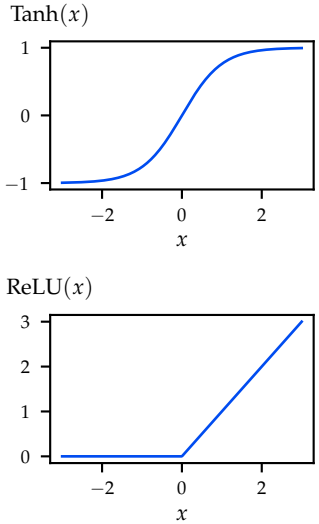



Figure 7.2: The Tanh and ReLU activation functions, respectively.

layer (with arbitrary width) and non-polynomial activation function $\varphi$ can approximate any continuous function to an arbitrary accuracy.

### 7.1.2 Classification

Although we mainly focus on regression, neural networks can equally well be used for classification. If we want to classify inputs into $c$ separate classes, we can simply construct a neural network with $c$ outputs, $\boldsymbol{f} = [f_1, \dots, f_c]$, and normalize them into a probability distribution. Often, the *softmax function* is used for normalization,

$$\sigma_i(\boldsymbol{f}) \doteq \frac{\exp(f_i)}{\sum_{j=1}^{c} \exp(f_j)} \tag{7.5}$$

where $\sigma_i(\boldsymbol{f})$ corresponds to the probability mass of class $i$. The softmax is a generalization of the logistic function (5.9) to more than two classes (?). Note that the softmax corresponds to a Gibbs distribution with energies $-\boldsymbol{f}$.

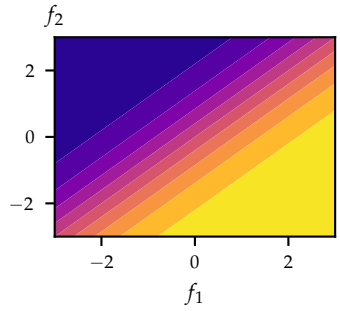



Figure 7.3: Softmax $\sigma_1(f_1, f_2)$ for a binary classification problem. Blue denotes a small probability and yellow denotes a large probability of belonging to class 1, respectively.

**Problem 7.1**

### 7.1.3 Maximum Likelihood Estimation

We will study neural networks under the lens of supervised learning (cf. Section 1.3) where we are provided some independently-sampled (noisy) data $\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{n}$ generated according to an unknown process $\boldsymbol{y} \sim p(\cdot \mid \boldsymbol{x}, \boldsymbol{\theta}^\star)$, which we wish to approximate.

Upon initialization, the network does generally not approximate this process well, so a key element of deep learning is "learning" a parameterization $\boldsymbol{\theta}$ that is a good approximation. To this end, one typically considers a loss function $\ell(\boldsymbol{\theta}; \boldsymbol{y})$ which quantifies the "error" of the network outputs $\boldsymbol{f}(\boldsymbol{x}; \boldsymbol{\theta})$. In the classical setting of regression, i.e., $y \in \mathbb{R}$ and $k = 1$, $\ell$ is often taken to be the *(empirical) mean squared error*,

$$\ell_{\text{mse}}(\boldsymbol{\theta}; \mathcal{D}) \doteq \frac{1}{n} \sum_{i=1}^{n} (f(\boldsymbol{x}_i; \boldsymbol{\theta}) - y_i)^2. \tag{7.6}$$

As we have already seen in Section 2.0.1 in the context of linear regression, minimizing mean squared error corresponds to maximum likelihood estimation under a Gaussian likelihood.

In the setting of classification where $\boldsymbol{y} \in \{0, 1\}^c$ is a one-hot encoding of class membership,[3] it is instead common to interpret the outputs of a neural network as probabilities akin to our discussion in Section 7.1.2. We denote by $q_{\boldsymbol{\theta}}(\cdot \mid \boldsymbol{x})$ the resulting probability distribution over classes with PMF $[\sigma_1(\boldsymbol{f}(\boldsymbol{x}; \boldsymbol{\theta})), \dots, \sigma_c(\boldsymbol{f}(\boldsymbol{x}; \boldsymbol{\theta}))]$, and aim to find $\boldsymbol{\theta}$ such that $q_{\boldsymbol{\theta}}(\boldsymbol{y} \mid \boldsymbol{x}) \approx p(\boldsymbol{y} \mid \boldsymbol{x})$. In this context, it is common to minimize the cross-entropy,

$$\mathrm{H}[p \| q_{\boldsymbol{\theta}}] = \mathbb{E}_{(\boldsymbol{x}, \boldsymbol{y}) \sim p}[-\log q_{\boldsymbol{\theta}}(\boldsymbol{y} \mid \boldsymbol{x})]$$

[3] That is, exactly one component of $\boldsymbol{y}$ is 1 and all others are 0, indicating to which class the given example belongs.

using the definition of cross-entropy (5.32)

$$\approx \underbrace{-\frac{1}{n}\sum_{i=1}^{n}\log q_{\boldsymbol{\theta}}(\boldsymbol{y}_i \mid \boldsymbol{x}_i)}_{\doteq \ell_{\text{ce}}(\boldsymbol{\theta};\mathcal{D})} \tag{7.7}$$

using Monte Carlo sampling

which can be understood as minimizing the surprise about the training data under the model. $\ell_{\text{ce}}$ is called the *cross-entropy loss*. Disregarding the constant $1/n$, we can rewrite the cross-entropy loss as

$$\propto -\sum_{i=1}^{n}\log q_{\boldsymbol{\theta}}(\boldsymbol{y}_i \mid \boldsymbol{x}_i) = \ell_{\text{nll}}(\boldsymbol{\theta};\mathcal{D}) \tag{7.8}$$

Recall that $\ell_{\text{nll}}(\boldsymbol{\theta};\mathcal{D})$ is the *negative log-likelihood* of the training data, and thus, empirically minimizing cross-entropy can equivalently be interpreted as maximum likelihood estimation.[4] Furthermore, recall from Problem 5.1 that for a two-class classification problem the cross-entropy loss is equivalent to the logistic loss.

[4] We have previously seen this equivalence of MLE and empirically minimizing KL-divergence in Section 5.4.6 (minimizing the cross-entropy $\mathrm{H}[p\|q_{\boldsymbol{\theta}}]$ is equivalent to minimizing forward-KL $\mathrm{KL}(p\|q_{\boldsymbol{\theta}})$). Note that this interpretation is not exclusive to the canonical cross-entropy loss from Equation (7.7), but holds for any MLE. For example, minimizing mean squared error corresponds to empirically minimizing the KL-divergence with a Gaussian likelihood.

### 7.1.4 *Backpropagation*

A crucial property of neural networks is that they are differentiable. That is, we can compute gradients $\nabla_{\boldsymbol{\theta}}\ell$ of $\boldsymbol{f}$ with respect to the parameterization of the model $\boldsymbol{\theta} = \boldsymbol{W}_{1:L}$ and some loss function $\ell(\boldsymbol{f};\boldsymbol{y})$. Being able to obtain these gradients efficiently allows for "learning" a particular function from data using first-order optimization methods.

The algorithm for computing gradients of a neural network is called *backpropagation* and is essentially a repeated application of the chain rule. Note that using the chain rule for every path through the network is computationally infeasible, as this quickly leads to a combinatorial explosion as the number of hidden layers is increased. The key insight of backpropagation is that we can use the *feed-forward* structure of our neural network to memoize computations of the gradient, yielding a linear time algorithm. Obtaining gradients by backpropagation is often called *automatic differentiation (auto-diff)*. For more details, refer to Goodfellow et al. (2016).

Computing the exact gradient for each data point is still fairly expensive when the size of the neural network is large. Typically, stochastic gradient descent is used to obtain unbiased gradient estimates using batches of only $m$ of the $n$ data points, where $m \ll n$.

## 7.2 *Bayesian Neural Networks*

How can we perform probabilistic inference in neural networks? We adopt the same strategy which we already used for Bayesian linear regression, we impose a Gaussian prior on the weights,

$$\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{0}, \sigma_{\text{p}}^2\boldsymbol{I}). \tag{7.9}$$

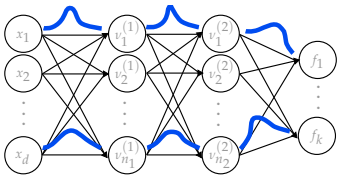



Figure 7.4: Bayesian neural networks model a distribution over the weights of a neural network.

Similarly, we can use a Gaussian likelihood to describe how well the data is described by the model $\boldsymbol{f}$,

$$y \mid \boldsymbol{x}, \boldsymbol{\theta} \sim \mathcal{N}(f(\boldsymbol{x}; \boldsymbol{\theta}), \sigma_{\mathrm{n}}^2). \tag{7.10}$$

Thus, instead of considering weights as point estimates which are learned exactly, *Bayesian neural networks* learn a distribution over the weights of the network. In principle, other priors and likelihoods can be used, yet Gaussians are typically chosen due to their closedness properties, which we have seen in Section 1.2.3 and many times since.

### 7.2.1 Maximum a Posteriori Estimation

Before studying probabilistic inference, let us first consider MAP estimation in the context of neural networks.

The MAP estimate of the weights is obtained by

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}) + \sum_{i=1}^{n} \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}). \tag{7.11}$$

In Section 7.1.3, we have seen that the negative log-likelihood under a Gaussian likelihood (7.10) is the squared error between label and prediction,

$$\log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}) = -\frac{(y_i - f(\boldsymbol{x}_i; \boldsymbol{\theta}))^2}{2\sigma_{\mathrm{n}}^2} + \text{const.} \tag{7.12}$$

Obtaining the MAP estimate instead, simply corresponds to adding an $L_2$-regularization term to the squared error loss,

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{2\sigma_{\mathrm{p}}^2} \|\boldsymbol{\theta}\|_2^2 + \frac{1}{2\sigma_{\mathrm{n}}^2} \sum_{i=1}^{n} (y_i - f(\boldsymbol{x}_i; \boldsymbol{\theta}))^2. \tag{7.13}$$

using that for an isotropic Gaussian prior, $\log p(\boldsymbol{\theta}) = -\frac{1}{2\sigma_{\mathrm{p}}^2} \|\boldsymbol{\theta}\|_2^2 + \text{const}$

As we have already seen in Remark A.31 and the context of Bayesian linear regression (and ridge regression), using a Gaussian prior is equivalent to applying weight decay.[5] Using gradient ascent, we obtain the following update rule,

[5] Recall that weight decay regularizes weights by shrinking them towards zero.

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}(1 - \lambda \eta_t) + \eta_t \sum_{i=1}^{n} \boldsymbol{\nabla} \log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}) \tag{7.14}$$

where $\lambda \doteq 1/\sigma_{\mathrm{p}}^2$. The gradients of the likelihood can be obtained using automatic differentiation.

### 7.2.2 Heteroscedastic Noise

Equation (7.10) uses the scalar parameter $\sigma_{\mathrm{n}}^2$ to model the aleatoric uncertainty (label noise), similarly to how we modeled the label noise

in Bayesian linear regression and Gaussian processes. Such a noise model is called *homoscedastic noise* as the noise is assumed to be uniform across the domain. In many settings, however, the noise is inherently *heteroscedastic noise*. That is, the noise varies depending on the input and which "region" of the domain the input is from. This behavior is visualized in Figure 7.5.

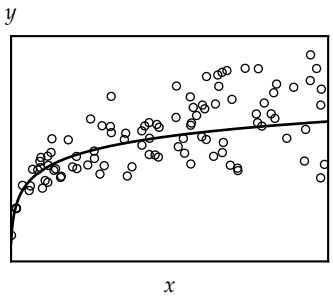



Figure 7.5: Illustration of data with variable (heteroscedastic) noise. The noise increases as the inputs increase in magnitude. The noise-free function is shown in black.

There is a natural way of modeling heteroscedastic noise with Bayesian neural networks. We use a neural network with two outputs $f_1$ and $f_2$ and define

$$y \mid \boldsymbol{x}, \boldsymbol{\theta} \sim \mathcal{N}(\mu(\boldsymbol{x}; \boldsymbol{\theta}), \sigma^2(\boldsymbol{x}; \boldsymbol{\theta})) \quad \text{where} \tag{7.15a}$$

$$\mu(\boldsymbol{x}; \boldsymbol{\theta}) \doteq f_1(\boldsymbol{x}; \boldsymbol{\theta}), \tag{7.15b}$$

$$\sigma^2(\boldsymbol{x}; \boldsymbol{\theta}) \doteq \exp(f_2(\boldsymbol{x}; \boldsymbol{\theta})). \tag{7.15c}$$

we exponentiate $f_2$ to ensure non-negativity of the variance

Using this model, the likelihood term from Equation (7.11) is

$$\begin{aligned}
\log p(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}) &= \log \mathcal{N}(y_i; \mu(\boldsymbol{x}_i; \boldsymbol{\theta}), \sigma^2(\boldsymbol{x}_i; \boldsymbol{\theta})) \\
&= \log \frac{1}{\sqrt{2\pi\sigma^2(\boldsymbol{x}_i; \boldsymbol{\theta})}} - \frac{(y_i - \mu(\boldsymbol{x}_i; \boldsymbol{\theta}))^2}{2\sigma^2(\boldsymbol{x}_i; \boldsymbol{\theta})} \\
&= \underbrace{\log \frac{1}{\sqrt{2\pi}}}_{\text{const}} - \frac{1}{2}\left[\log \sigma^2(\boldsymbol{x}_i; \boldsymbol{\theta}) + \frac{(y_i - \mu(\boldsymbol{x}_i; \boldsymbol{\theta}))^2}{\sigma^2(\boldsymbol{x}_i; \boldsymbol{\theta})}\right].
\end{aligned} \tag{7.16}$$

note that the normalizing constant depends on the noise model!

Hence, the model can either explain the label $y_i$ by an accurate model $\mu(\boldsymbol{x}_i; \boldsymbol{\theta})$ or by a large variance $\sigma^2(\boldsymbol{x}_i; \boldsymbol{\theta})$, yet, it is penalized for choosing large variances. Intuitively, this allows to attenuate losses for some data points by attributing them to large variance (when no model reflecting all data points simultaneously can be found). This allows the model to "learn" its aleatoric uncertainty. However, recall that the MAP estimate still corresponds to a point estimate of the weights, so we forgo modeling the epistemic uncertainty.

## 7.3 Approximate Probabilistic Inference

Naturally, we want to understand the epistemic uncertainty of our model too. However, learning and inference in Bayesian neural networks are generally intractable (even when using a Gaussian prior and likelihood) when the noise is not assumed to be homoscedastic and known.[6] Thus, we are led to the techniques of approximate inference, which we discussed in the previous two chapters.

[6] In this case, the conjugate prior to a Gaussian likelihood is not a Gaussian. See, e.g., `https://en.wikipedia.org/wiki/Conjugate_prior`.

### 7.3.1 Variational Inference

As we have discussed in Chapter 5, we can apply black box stochastic variational inference which — in the context of neural networks — is also known as *Bayes by Backprop*. As variational family, we use the family of independent Gaussians which we have already encountered in Example 5.5.[7] Recall the fundamental objective of variational inference (5.25),

[7] Independent Gaussians are useful because they can be encoded using only $2d$ parameters, which is crucial when the size of the neural network is large.

$$\begin{aligned}
&\arg\min_{q\in\mathcal{Q}} \mathrm{KL}(q\|p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n})) \\
&= \arg\max_{q\in\mathcal{Q}} L(q, p; \mathcal{D}) && \text{using Equation (5.53)} \\
&= \arg\max_{q\in\mathcal{Q}} \mathbb{E}_{\boldsymbol{\theta}\sim q}[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})] - \mathrm{KL}(q\|p(\cdot)). && \text{using Equation (5.55c)}
\end{aligned}$$

The KL-divergence $\mathrm{KL}(q\|p(\cdot))$ can be expressed in closed-form for Gaussians.[8] Recall that we can obtain unbiased gradient estimates of the expectation using the reparameterization trick (5.68),

[8] see Equation (5.41)

$$\mathbb{E}_{\boldsymbol{\theta}\sim q}[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})] = \mathbb{E}_{\boldsymbol{\varepsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})}\Big[\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\Sigma}^{1/2}\boldsymbol{\varepsilon}+\boldsymbol{\mu}}\Big].$$

As $\boldsymbol{\Sigma}$ is the diagonal matrix $\mathrm{diag}\{\sigma_1^2, \ldots, \sigma_d^2\}$, $\boldsymbol{\Sigma}^{1/2} = \mathrm{diag}\{\sigma_1, \ldots, \sigma_d\}$. The gradients of the likelihood can be obtained using backpropagation. We can now use the variational posterior $q_\lambda$ to perform approximate inference,

$$\begin{aligned}
p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}) &= \int p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n})\, d\boldsymbol{\theta} && \text{using the sum rule (1.7) and product rule (1.11)} \\
&= \mathbb{E}_{\boldsymbol{\theta}\sim p(\cdot|\boldsymbol{x}_{1:n}, y_{1:n})}[p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta})] && \text{interpreting the integral as an expectation over the posterior} \\
&\approx \mathbb{E}_{\theta\sim q_\lambda}[p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta})] && \text{approximating the posterior with the variational posterior } q_\lambda \\
&\approx \frac{1}{m}\sum_{i=1}^{m} p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}^{(i)}) && \text{using Monte Carlo sampling} \qquad (7.17)
\end{aligned}$$

for independent samples $\boldsymbol{\theta}^{(i)} \overset{\text{iid}}{\sim} q_\lambda$,

$$= \frac{1}{m}\sum_{i=1}^{m} \mathcal{N}(y^\star; \mu(\boldsymbol{x}^\star; \boldsymbol{\theta}^{(i)}), \sigma^2(\boldsymbol{x}^\star; \boldsymbol{\theta}^{(i)})). \qquad (7.18) \quad \text{using the neural network}$$

Intuitively, variational inference in Bayesian neural networks can be interpreted as averaging the predictions of multiple neural networks drawn according to the variational posterior $q_\lambda$.

Using the Monte Carlo samples $\boldsymbol{\theta}^{(i)}$, we can also estimate the mean of our predictions,

$$\mathbb{E}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}] \approx \frac{1}{m}\sum_{i=1}^{m} \mu(\boldsymbol{x}^\star; \boldsymbol{\theta}^{(i)}) \doteq \overline{\mu}(\boldsymbol{x}^\star), \qquad (7.19)$$

and the variance of our predictions,

$$\begin{aligned}\operatorname{Var}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}] &= \mathbb{E}_{\boldsymbol{\theta}}\left[\operatorname{Var}_{y^\star}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}]\right] + \operatorname{Var}_{\boldsymbol{\theta}}\left[\mathbb{E}_{y^\star}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}]\right] && \text{using the law of total variance (1.41)} \\ &= \mathbb{E}_{\boldsymbol{\theta}}\left[\sigma^2(\boldsymbol{x}^\star; \boldsymbol{\theta})\right] + \operatorname{Var}_{\boldsymbol{\theta}}[\mu(\boldsymbol{x}^\star; \boldsymbol{\theta})]. && \text{using the likelihood (7.15a)}\end{aligned}$$

Recall from Equation (2.18) that the first term corresponds to the aleatoric uncertainty of the data and the second term corresponds to the epistemic uncertainty of the model. We can approximate them using the Monte Carlo samples $\boldsymbol{\theta}^{(i)}$,

$$\begin{aligned}\operatorname{Var}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}] \approx \frac{1}{m}\sum_{i=1}^{m} \sigma^2(\boldsymbol{x}^\star; \boldsymbol{\theta}^{(i)}) \\ + \frac{1}{m-1}\sum_{i=1}^{m}(\mu(\boldsymbol{x}^\star; \boldsymbol{\theta}^{(i)}) - \overline{\mu}(\boldsymbol{x}^\star))^2\end{aligned} \tag{7.20}$$

using a sample mean (A.14) and sample variance (A.16).

### 7.3.2 *Markov Chain Monte Carlo*

As we have discussed in Chapter 6, an alternative method to approximating the full posterior distribution is to sample from it directly. By the ergodic theorem (6.28), we can use any of the discussed sampling strategies to obtain samples $\boldsymbol{\theta}^{(t)}$ such that

$$p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}) \approx \frac{1}{T}\sum_{t=1}^{T} p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}^{(t)}). \qquad \text{see (6.29)}$$

Here, we omit the offset $t_0$ which is commonly used to avoid the "burn-in" period for simplicity. Algorithms such as SGLD or SG-HMC are often used as they rely only on stochastic gradients of the loss function which can be computed efficiently using automatic differentiation.

Typically, for large networks, we cannot afford to store all $T$ samples of models. Thus, we need to summarize the iterates.[9] One approach is to keep track of $m$ *snapshots* of weights $[\boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(m)}]$ according to some schedule and use those for inference (e.g., by averaging the predictions of the corresponding neural networks). This approach of sampling a subset of some data is generally called *subsampling*.

[9] That is, combine the individual samples of weights $\boldsymbol{\theta}^{(i)}$.

Another approach is to summarize (that is, approximate) the weights using sufficient statistics (e.g., a Gaussian).[10] In other words, we learn the Gaussian approximation,

$$\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}), \quad \text{where} \tag{7.21a}$$

$$\boldsymbol{\mu} \doteq \frac{1}{T}\sum_{i=1}^{T} \boldsymbol{\theta}^{(i)}, \qquad \text{using a sample mean (A.14)} \tag{7.21b}$$

[10] A statistic is *sufficient* for a family of probability distributions if the samples from which it is calculated yield no more information than the statistic with respect to the learned parameters. We provide a formal definition in Section 8.2.

$$\boldsymbol{\Sigma} \doteq \frac{1}{T-1}\sum_{i=1}^{T}(\boldsymbol{\theta}^{(i)}-\boldsymbol{\mu})(\boldsymbol{\theta}^{(i)}-\boldsymbol{\mu})^\top, \tag{7.21c}$$

using a sample variance (A.16)

using sample means and sample (co)variances. This can be implemented efficiently using running averages of the first and second moments,

$$\boldsymbol{\mu} \leftarrow \frac{1}{T+1}(T\boldsymbol{\mu}+\boldsymbol{\theta}) \quad \text{and} \quad \boldsymbol{A} \leftarrow \frac{1}{T+1}(T\boldsymbol{A}+\boldsymbol{\theta}\boldsymbol{\theta}^\top) \tag{7.22}$$

upon observing the fresh sample $\boldsymbol{\theta}$. $\boldsymbol{\Sigma}$ can easily be calculated from these moments,

$$\boldsymbol{\Sigma} = \frac{T}{T-1}(\boldsymbol{A}-\boldsymbol{\mu}\boldsymbol{\mu}^\top). \tag{7.23}$$

using the characterization of sample variance in terms of estimators of the first and second moment (A.17)

To predict, we can sample weights $\boldsymbol{\theta}$ from the learned Gaussian.

**Remark 7.1: Stochastic weight averaging**

It turns out that this approach works well even without injecting additional Gaussian noise during training, e.g., when using SGD rather than SGLD. Simply taking the mean of the iterates of SGD is called *stochastic weight averaging* (SWA). Describing the iterates of SGD by Gaussian sufficient statistics (analogously to Equation (7.21c)), is known as the *stochastic weight averaging-Gaussian* (SWAG) method (Izmailov et al., 2018).

### 7.3.3 *Dropout and Dropconnect*

We will now discuss two approximate inference techniques that are tailored to neural networks. The first is "dropout"/"dropconnect" regularization. Traditionally, *dropout regularization* (Hinton et al., 2012; Srivastava et al., 2014) randomly omits vertices of the computation graph during training, see Figure 7.7. In contrast, *dropconnect regularization* (Wan et al., 2013) randomly omits edges of the computation graph. The key idea that we will present here is to interpret this type of regularization as performing variational inference.

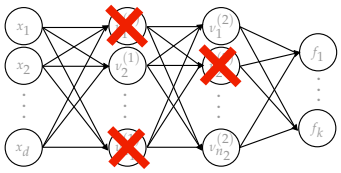



Figure 7.6: Illustration of dropout regularization. Some vertices of the computation graph are randomly omitted. In contrast, dropconnect regularization randomly omits edges of the computation graph.

In our exposition, we will focus on dropconnect, but the same approach also applies to dropout (Gal and Ghahramani, 2016). Suppose that we omit an edge of the computation graph (i.e., set its weight to zero) with probability $p$. Then our variational posterior is given by

$$q(\boldsymbol{\theta} \mid \boldsymbol{\lambda}) \doteq \prod_{j=1}^{d} q_j(\theta_j \mid \lambda_j) \tag{7.24}$$

where $d$ is the number of weights of the neural network and

$$q_j(\theta_j \mid \lambda_j) \doteq p\delta_0(\theta_j) + (1-p)\delta_{\lambda_j}(\theta_j). \tag{7.25}$$

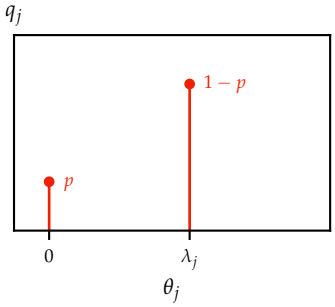



Figure 7.7: Interpretation of dropconnect regularization as variational inference. The only coordinates where the variational posterior $q_j$ has positive probability are 0 and $\lambda_j$.

Here, $\delta_\alpha$ is the Dirac delta function with point mass at $\alpha$.[11] The variational parameters $\boldsymbol{\lambda}$ correspond to the "original" weights of the network. In words, the variational posterior expresses that the $j$-th weight has value 0 with probability $p$ and value $\lambda_j$ with probability $1 - p$. For fixed weights $\boldsymbol{\lambda}$, sampling from the variational posterior $q_{\boldsymbol{\lambda}}$ corresponds to sampling a vector $\boldsymbol{z}$ with entries $\boldsymbol{z}(i) \sim \text{Bern}(p)$, yielding $\boldsymbol{z} \odot \boldsymbol{\lambda}$ which is one of $2^d$ possible subnetworks.[12]

[11] see Section A.1.4

[12] $\boldsymbol{A} \odot \boldsymbol{B}$ denotes the Hadamard (element-wise) product.

The weights $\boldsymbol{\lambda}$ can be learned by maximizing the ELBO, analogously to black-box variational inference. The KL-divergence term of the ELBO is not tractable for the variational family described by Equation (7.25), instead a common approach is to use a mixture of Gaussians:

$$q_j(\theta_j \mid \lambda_j) \doteq p\mathcal{N}(\theta_j; 0, 1) + (1 - p)\mathcal{N}(\theta_j; \lambda_j, 1). \tag{7.26}$$

In this case, it can be shown that $\text{KL}(q_{\boldsymbol{\lambda}} \| p(\cdot)) \approx \frac{p}{2} \|\boldsymbol{\lambda}\|_2^2$ for sufficiently large $d$ (Gal and Ghahramani, 2015, proposition 1). Thus,

$$\begin{aligned}
&\arg\max_{\boldsymbol{\lambda} \in \Lambda} L(q_{\boldsymbol{\lambda}}, p; \mathcal{D}) \\
&= \arg\max_{\boldsymbol{\lambda} \in \Lambda} \mathbb{E}_{\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}}[\log p(\boldsymbol{y}_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})] - \text{KL}(q \| p(\cdot)) && \text{using Equation (5.55c)} \\
&\approx \arg\min_{\boldsymbol{\lambda} \in \Lambda} -\frac{1}{m} \sum_{i=1}^{m} \log p(\boldsymbol{y}_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})\big|_{\boldsymbol{\theta} = \boldsymbol{z}^{(i)} \odot \boldsymbol{\lambda} + \boldsymbol{\varepsilon}^{(i)}} + \frac{p}{2} \|\boldsymbol{\lambda}\|_2^2 && \text{using Monte Carlo sampling} \quad (7.27)
\end{aligned}$$

where we reparameterize $\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}$ by $\boldsymbol{\theta} = \boldsymbol{z} \odot \boldsymbol{\lambda} + \boldsymbol{\varepsilon}$ with $\boldsymbol{z}(i) \sim \text{Bern}(p)$ and $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. Equation (7.27) is the standard $L_2$-regularized loss function of a neural network with weights $\boldsymbol{\lambda}$ and dropconnect, and it is straightforward to obtain unbiased gradient estimates by automatic differentiation.

Crucially, for the interpretation of dropconnect regularization as variational inference to be valid, we also need to perform dropconnect regularization during inference,

$$\begin{aligned}
p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}) &\approx \mathbb{E}_{\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}}[p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta})] \\
&\approx \frac{1}{m} \sum_{i=1}^{m} p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}^{(i)}) && \text{using Monte Carlo sampling} \quad (7.28)
\end{aligned}$$

where $\boldsymbol{\theta}^{(i)} \overset{\text{iid}}{\sim} q_{\boldsymbol{\lambda}}$ are independent samples. This coincides with our earlier discussion of variational inference for Bayesian neural networks in Equation (7.17). In words, we average the predictions of $m$ neural networks for each of which we randomly "drop out" weights.

**Remark 7.2: Masksembles**

A practical problem of dropout is that for any reasonable choice of dropout probability, the dropout masks $z^{(i)}$ will overlap significantly, which tends to make the predictions $p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}^{(i)})$ highly correlated. This can lead to underestimation of epistemic uncertainty. *Masksembles* (Durasov et al., 2021) mitigate this issue by choosing a fixed set of pre-defined dropout masks, which have controlled overlap, and alternating between them during training. In the extreme case of "infinitely many" masks, masksembles are equivalent to dropout since each mask is only seen once.

### 7.3.4 Probabilistic Ensembles

We have seen that variational inference in the context of Bayesian neural networks can be interpreted as averaging the predictions of $m$ neural networks drawn according to the variational posterior.

A natural adaptation of this idea is to immediately learn the weights of $m$ neural networks. The idea is to randomly choose $m$ training sets by sampling uniformly from the data with replacement. Then, using our analysis from Section 7.2.1, we obtain $m$ MAP estimates of the weights $\boldsymbol{\theta}^{(i)}$, yielding the approximation

$$\begin{aligned} p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}) &= \mathbb{E}_{\boldsymbol{\theta} \sim p(\cdot \mid \boldsymbol{x}_{1:n}, y_{1:n})}[p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta})] \\ &\approx \frac{1}{m} \sum_{i=1}^{m} p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta}^{(i)}). \end{aligned} \tag{7.29}$$

using bootstrapping

Here, the randomly chosen $m$ training sets induce "diversity" of the models. In practice, in the context of deep neural networks where the global minimizer of the loss can rarely be identified, it is common to use the full training data to train each of the $m$ neural networks. Random initialization and random shuffling of the training data is typically enough to ensure some degree of diversity between the individual models (Lakshminarayanan et al., 2017). We can connect ensembles to the other approximate inference techniques we have discussed: First, ensembles can be seen as a specific kind of masksemble, where the masks are non-overlapping,[13] which mitigates the issue of correlated predictions from Remark 7.2. Second, ensembling can be combined with other approximate inference techniques such as variational inference, Laplace approximation, or SWAG to get a mixture of Gaussians as the posterior approximation.

[13] That is, the $m$ models do not share any of their parameters. Ensembles and dropout lie on opposite ends of this spectrum.

Note that Equation (7.29) is not equivalent to Monte Carlo sampling, although it looks very similar. The key difference is that this approach does not sample from the true posterior distribution $p$, but instead

from the empirical posterior distribution $\hat{p}$ given the (re-sampled) MAP estimates. Intuitively, this can be understood as the difference between sampling from a distribution $p$ directly (Monte Carlo sampling) versus sampling from an approximate (empirical) distribution $\hat{p}$ (corresponding to the training data), which itself is constructed from samples of the true distribution $p$. This approach is known as *bootstrapping* or *bagging* (short for *b*ootstrap *agg*regat*ing*) and plays a central role in model-free reinforcement learning. We will return to this concept in Section 11.4.1.

### 7.3.5 Diverse Probabilistic Ensembles

Probabilistic ensembles can be loosely interpreted as randomly initializing $m$ "particles" $\{\boldsymbol{\theta}^{(i)}\}_{i=1}^m$ and then pushing each particle towards regions of high posterior probability. A potential issue with this approach is that if the initialization of particles is not sufficiently diverse, the particles may "collapse" since every particle eventually converges to a local optimum of the loss function. A natural approach to mitigate this issue is to alter the objective of each particle from simply aiming to minimize the loss $-\log p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n})$, which we will abbreviate by $-\log p(\boldsymbol{\theta})$, to also "push" the particles away from each other. We will see next that this can be interpreted as a form of variational inference under a very flexible variational family.

In our discussion of *variational inference*, we have seen that minimizing reverse-KL is equivalent to maximizing the evidence lower bound, and used this to derive an optimization algorithm to compute approximate posteriors. We will discuss the alternative approach of directly computing the gradient of the KL-divergence. Consider the variational family of all densities that can be expressed as smooth transformations of points sampled from a reference density $\phi$. That is, we consider $\mathcal{Q}_\phi \doteq \{\mathbf{T}_\sharp \phi \mid \mathbf{T} : \Theta \to \Theta \text{ is smooth}\}$ where $\mathbf{T}_\sharp \phi$ is the density of the random vector $\boldsymbol{\theta}' = \mathbf{T}(\boldsymbol{\theta})$ with $\boldsymbol{\theta} \sim \phi$.[14] The density $\phi$ can be thought of as the initial distribution of the particles, and the smooth map $\mathbf{T}$ as the dynamics that push the particles towards the target density $p$. It can be shown that for "almost any" reference density $\phi$, this variational family $\mathcal{Q}_\phi$ is expressive enough to closely approximate "almost arbitrary" distributions.[15] A natural approach is therefore to *learn* the appropriate smooth map $\mathbf{T}$ between the reference density $\phi$ and the target density $p$.

[14] Refer back to Section 1.1.11 for a recap on pushforwards.

[15] For a more detailed discussion, refer to "Stein variational gradient descent: A general purpose Bayesian inference algorithm" (Liu and Wang, 2016).

**Example 7.3: Gaussian variational inference**

We have seen in Section 5.5 that if $\phi$ is standard normal and $\mathbf{T}(\boldsymbol{x}; \{\boldsymbol{\mu}, \boldsymbol{\Sigma}^{1/2}\}) = \boldsymbol{\mu} + \boldsymbol{\Sigma}^{1/2}\boldsymbol{x}$ an affine transformation then the ELBO

can be maximized efficiently using stochastic gradient descent. However, in this case, $\mathcal{Q}_\phi$ can only approximate Gaussian-like distributions since the expressivity of the map $\mathbf{T}$ is limited under the fixed reference density $\phi$.

An alternative approach to Gaussian variational inference is the following algorithm known as *Stein variational gradient descent* (SVGD), where we recursively apply carefully chosen smooth maps to the current variational approximation:

$$q_0 \xrightarrow{\mathbf{T}_0^\star} q_1 \xrightarrow{\mathbf{T}_1^\star} q_2 \xrightarrow{\mathbf{T}_2^\star} \cdots \qquad \text{where } q_{t+1} \doteq \mathbf{T}_{t\,\sharp}^\star q_t. \tag{7.30}$$

We consider maps $\mathbf{T} = \mathbf{id} + \boldsymbol{f}$ where $\mathbf{id}(\boldsymbol{\theta}) \doteq \boldsymbol{\theta}$ denotes the identity map and $\boldsymbol{f}(\boldsymbol{x})$ represents a (small) perturbation. Recall that at time $t$ we seek to minimize $\mathrm{KL}(\mathbf{T}_\sharp q_t \| p)$, so we choose the smooth map as

$$\mathbf{T}_t^\star \doteq \mathbf{id} - \eta_t \, \boldsymbol{\nabla}_{\boldsymbol{f}} \, \mathrm{KL}(\mathbf{T}_\sharp q_t \| p)\big|_{\boldsymbol{f}=\mathbf{0}} \tag{7.31}$$

where $\eta_t$ is a step size. In this way, the SVGD update (7.31) can be interpreted as a step of "functional" gradient descent.

To be able to compute the gradient of the KL-divergence, we need to make some structural assumptions on the perturbation $\boldsymbol{f}$. SVGD assumes that $\boldsymbol{f} = [f_1 \ \cdots \ f_d]^\top$ with $f_i \in \mathcal{H}_k(\Theta)$ is from some reproducing kernel Hilbert space $\mathcal{H}_k(\Theta)$ of a positive definite kernel $k$; we say $\boldsymbol{f} \in \mathcal{H}_k^d(\Theta)$. Within the RKHS, we can compute the gradient of the KL-divergence exactly. Liu and Wang (2016) show that in this case, the functional gradient of the KL-divergence can be expressed in closed-form as $-\boldsymbol{\nabla}_{\boldsymbol{f}} \, \mathrm{KL}(\mathbf{T}_\sharp q \| p)\big|_{\boldsymbol{f}=\mathbf{0}} = \boldsymbol{\varphi}_{q,p}^\star$ where

$$\boldsymbol{\varphi}_{q,p}^\star(\cdot) \doteq \mathbb{E}_{\boldsymbol{\theta}\sim q}[k(\cdot, \boldsymbol{\theta}) \boldsymbol{\nabla}_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}) + \boldsymbol{\nabla}_{\boldsymbol{\theta}} \, k(\cdot, \boldsymbol{\theta})]. \tag{7.32}$$

SVGD then approximates $q$ using the particles $\{\boldsymbol{\theta}^{(i)}\}_{i=1}^m$ as follows:

**Algorithm 7.4:** Stein variational gradient descent, SVGD

initialize particles $\{\boldsymbol{\theta}^{(i)}\}_{i=1}^m$
**repeat**
  **for** each particle $i \in [m]$ **do**
    $\boldsymbol{\theta}^{(i)} \leftarrow \boldsymbol{\theta}^{(i)} + \eta_t \hat{\boldsymbol{\varphi}}_{q,p}^\star(\boldsymbol{\theta}^{(i)})$ where
    $\hat{\boldsymbol{\varphi}}_{q,p}^\star(\boldsymbol{\theta}) \doteq \frac{1}{m} \sum_{j=1}^m \left[ k(\boldsymbol{\theta}, \boldsymbol{\theta}^{(j)}) \boldsymbol{\nabla}_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}) + \boldsymbol{\nabla}_{\boldsymbol{\theta}^{(j)}} \, k(\boldsymbol{\theta}, \boldsymbol{\theta}^{(j)}) \right]$
**until** converged

Often, a Gaussian kernel (4.14) with length scale $h$ is used to model the perturbations, in which case the repulsion term is

$$\nabla_{\boldsymbol{\theta}^{(j)}} k(\boldsymbol{\theta}, \boldsymbol{\theta}^{(j)}) = \frac{1}{h^2}(\boldsymbol{\theta} - \boldsymbol{\theta}^{(j)}) k(\boldsymbol{\theta}, \boldsymbol{\theta}^{(j)}) \tag{7.33}$$

and the negative functional gradient simplifies to

$$\hat{\boldsymbol{\varphi}}^{\star}_{q,p}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{j=1}^{m} k(\boldsymbol{\theta}, \boldsymbol{\theta}^{(j)}) \Big[ \underbrace{\nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta})}_{\text{drift}} + \underbrace{h^{-2}(\boldsymbol{\theta} - \boldsymbol{\theta}^{(j)})}_{\text{repulsion}} \Big]. \tag{7.34}$$

Note that SVGD has similarities to Langevin dynamics, which as seen in (?) can also be interpreted as following a gradient of the KL-divergence. Whereas Langevin dynamics perturbs particles according to a drift towards regions of high posterior probability and some random diffusion (cf. Equation (6.46)), the first term of $\hat{\boldsymbol{\varphi}}^{\star}_{q,p}(\boldsymbol{\theta})$ perturbs particles to drift towards regions of high posterior probability while the second term leads to a mutual "repulsion" of particles. Notably, the perturbations of Langevin dynamics are noisy, while SVGD perturbs particles deterministically and the randomness is exclusively in the initialization of particles. The repulsion term prevents particles from collapsing to a single mode of the posterior distribution, which is a possible failure mode of other particle-based posterior approximations such as ensembles.

**Problem 6.9**

Note that the above decomposition of $\hat{\boldsymbol{\varphi}}^{\star}_{q,p}(\boldsymbol{\theta})$ is once more an example of the *principle of curiosity and conformity* which we have seen to be a recurring theme in approaches to approximate inference. The repulsion term leads to exploration of the particles (i.e., "curiosity" about alternative explanations), while the drift term leads to minimization of the loss (i.e., "conformity" to the data).

Lu et al. (2019) show that under some assumptions, SVGD converges asymptotically to the target density $p$ as $\eta_t \to 0$. SVGD's name originates from *Stein's method* which is a general-purpose approach for characterizing convergence in distribution.[16]

[16] For an introduction to Stein's method, read "Measuring sample quality with Stein's method" (Gorham and Mackey, 2015).

## 7.4 Calibration

A key challenge of Bayesian deep learning (and also other probabilistic methods) is the calibration of models. We say that a model is *well-calibrated* if its confidence coincides with its accuracy across many predictions. Consider a classification model that predicts that the label of a given input belongs to some class with probability 80%. If the model is well-calibrated, then the prediction is correct about 80% of the time. In other words, during calibration, we adjust the probability estimation of the model.

We will first mention two methods of estimating the calibration of a model, namely the marginal likelihood and reliability diagrams. Then, in Section 7.4.3, we survey commonly used heuristics for empirically improving the calibration.

### 7.4.1 Evidence

A popular method (which we already encountered multiple times) is to use the evidence of a validation set $\boldsymbol{x}_{1:m}^{\text{val}}$ of size $m$ given the training set $\boldsymbol{x}_{1:n}^{\text{train}}$ of size $n$ for estimating the model calibration. Here, the evidence can be understood as describing how well the validation set is described by the model trained on the training set. We obtain,

$$
\begin{aligned}
&\log p(y_{1:m}^{\text{val}} \mid \boldsymbol{x}_{1:m}^{\text{val}}, \boldsymbol{x}_{1:n}^{\text{train}}, y_{1:n}^{\text{train}}) \\
&= \log \int p(y_{1:m}^{\text{val}} \mid \boldsymbol{x}_{1:m}^{\text{val}}, \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}^{\text{train}}, y_{1:n}^{\text{train}}) \, d\boldsymbol{\theta} && \text{using the sum rule (1.7) and product rule (1.11)} \\
&\approx \log \int p(y_{1:m}^{\text{val}} \mid \boldsymbol{x}_{1:m}^{\text{val}}, \boldsymbol{\theta}) q_{\boldsymbol{\lambda}}(\boldsymbol{\theta}) \, d\boldsymbol{\theta} && \text{approximating with the variational posterior} \\
&= \log \int \prod_{i=1}^{m} p(y_i^{\text{val}} \mid \boldsymbol{x}_i^{\text{val}}, \boldsymbol{\theta}) q_{\boldsymbol{\lambda}}(\boldsymbol{\theta}) \, d\boldsymbol{\theta} && \text{using the independence of the data} \qquad (7.35)
\end{aligned}
$$

The resulting integrals are typically very small which leads to numerical instabilities. Therefore, it is common to maximize a lower bound to the evidence instead,

$$
\begin{aligned}
&= \log \mathbb{E}_{\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}} \left[ \prod_{i=1}^{m} p(y_i^{\text{val}} \mid \boldsymbol{x}_i^{\text{val}}, \boldsymbol{\theta}) \right] && \text{interpreting the integral as an expectation over the variational posterior} \\
&\geq \mathbb{E}_{\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}} \left[ \sum_{i=1}^{m} \log p(y_i^{\text{val}} \mid \boldsymbol{x}_i^{\text{val}}, \boldsymbol{\theta}) \right] \qquad (7.36) && \text{using Jensen's inequality (5.29)} \\
&\approx \frac{1}{k} \sum_{j=1}^{k} \sum_{i=1}^{m} \log p(y_i^{\text{val}} \mid \boldsymbol{x}_i^{\text{val}}, \boldsymbol{\theta}^{(j)}) \qquad (7.37) && \text{using Monte Carlo sampling}
\end{aligned}
$$

for independent samples $\boldsymbol{\theta}^{(j)} \overset{\text{iid}}{\sim} q_{\boldsymbol{\lambda}}$.

### 7.4.2 Reliability Diagrams

Reliability diagrams take a frequentist perspective to estimate the calibration of a model. For simplicity, we assume a calibration problem with two classes, 1 and $-1$ (similarly to logistic regression).[17]

[17] Reliability diagrams generalize beyond this restricted example.

We group the predictions of a validation set into $M$ interval bins of size $1/M$ according to the class probability predicted by the model, $\mathbb{P}(Y_i = 1 \mid \boldsymbol{x}_i)$. We then compare within each bin, how often the model thought the inputs belonged to the class (confidence) with how often the inputs actually belonged to the class (frequency). Formally, we

define $B_m$ as the set of samples falling into bin $m$ and let

$$\text{freq}(B_m) \doteq \frac{1}{|B_m|} \sum_{i \in B_m} \mathbb{1}\{Y_i = 1\} \tag{7.38}$$

be the proportion of samples in bin $m$ that belong to class 1 and let

$$\text{conf}(B_m) \doteq \frac{1}{|B_m|} \sum_{i \in B_m} \mathbb{P}(Y_i = 1 \mid \boldsymbol{x}_i) \tag{7.39}$$

be the average confidence of samples belonging to class 1 within the bin $m$.

Thus, a model is well calibrated if $\text{freq}(B_m) \approx \text{conf}(B_m)$ for each bin $m \in [M]$. There are two common metrics of calibration that quantify how "close" a model is to being well calibrated.

1. The *expected calibration error* (ECE) is the average deviation of a model from perfect calibration,

$$\ell_{\text{ECE}} \doteq \sum_{m=1}^{M} \frac{|B_m|}{n} |\text{freq}(B_m) - \text{conf}(B_m)| \tag{7.40}$$

   where $n$ is the size of the validation set.
2. The *maximum calibration error* (MCE) is the maximum deviation of a model from perfect calibration among all bins,

$$\ell_{\text{MCE}} \doteq \max_{m \in [M]} |\text{freq}(B_m) - \text{conf}(B_m)| \,. \tag{7.41}$$

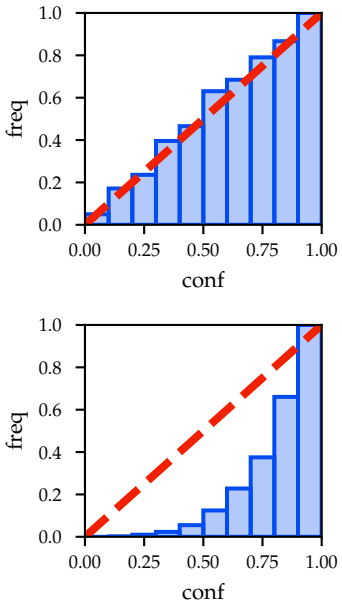



Figure 7.8: Examples of reliability diagrams with ten bins. A perfectly calibrated model approximates the diagonal dashed red line. The first reliability diagram shows a well-calibrated model. In contrast, the second reliability diagram shows an overconfident model.

### 7.4.3 *Heuristics for Improving Calibration*

We now survey a few heuristics which can be used empirically to improve model calibration.

1. *Histogram binning* assigns a calibrated score $q_m \doteq \text{freq}(B_m)$ to each bin during validation. Then, during inference, we return the calibrated score $q_m$ of the bin corresponding to the prediction of the model.
2. *Isotonic regression* extends histogram binning by using variable bin boundaries. We find a piecewise constant function $\boldsymbol{f} \doteq [f_1, \dots, f_M]$ that minimizes the bin-wise squared loss,

$$\min_{M, \boldsymbol{f}, \boldsymbol{a}} \quad \sum_{m=1}^{M} \sum_{i=1}^{n} \mathbb{1}\{a_m \leq \mathbb{P}(Y_i = 1 \mid \boldsymbol{x}_i) < a_{m+1}\}(f_m - y_i)^2 \tag{7.42a}$$

$$\text{subject to} \quad 0 = a_1 \leq \cdots \leq a_{M+1} = 1, \tag{7.42b}$$

$$f_1 \leq \cdots \leq f_M \tag{7.42c}$$

   where $\boldsymbol{f}$ are the calibrated scores and $\boldsymbol{a} \doteq [a_1, \dots, a_{M+1}]$ are the bin boundaries. We then return the calibrated score $f_m$ of the bin corresponding to the prediction of the model.

3. *Platt scaling* adjusts the logits $z_i$ of the output layer to

$$q_i \doteq \sigma(az_i + b) \tag{7.43}$$

and then learns parameters $a, b \in \mathbb{R}$ to maximize the likelihood.

4. *Temperature scaling* is a special and widely used instance of Platt scaling where $a \doteq 1/T$ and $b \doteq 0$ for some temperature scalar $T > 0$,

$$q_i \doteq \sigma\left(\frac{z_i}{T}\right). \tag{7.44}$$

Intuitively, for a larger temperature $T$, the probability is distributed more evenly among the classes (without changing the ranking), yielding a more uncertain prediction. In contrast, for a lower temperature $T$, the probability is concentrated more towards the top choices, yielding a less uncertain prediction. As seen in Problem 6.7, temperature scaling can be motivated as tuning the mean of the softmax distribution.

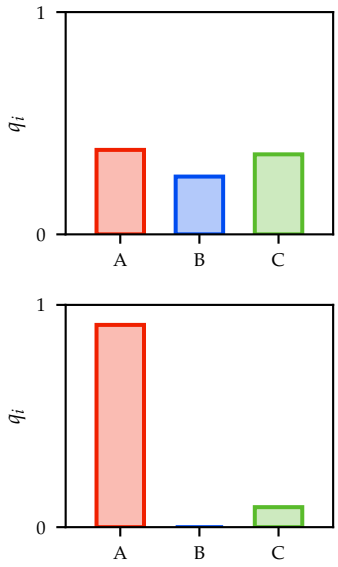



Figure 7.9: Illustration of temperature scaling for a classifier with three classes. On the top, we have a prediction with a high temperature, yielding a very uncertain prediction (in favor of class $A$). Below, we have a prediction with a low temperature, yielding a prediction that is strongly in favor of class $A$. Note that the ranking ($A \succ C \succ B$) is preserved.

**Optional Readings**

- Guo, Pleiss, Sun, and Weinberger (2017). *On calibration of modern neural networks*.
- Blundell, Cornebise, Kavukcuoglu, and Wierstra (2015). *Weight uncertainty in neural network*.
- Kendall and Gal (2017). *What uncertainties do we need in Bayesian deep learning for computer vision?*.

## 7.5 Diffusion Generative Models

### 7.5.1 Problem setup and intuition

We now turn our attention to a different task: generative modelling. We are given a set of points $\{x^{(i)}\}_{i=1}^N$ sampled *i.i.d.* from a distribution $p$, and our task is to learn a model that can be used to sample new points (approximately) from $p$. Diffusion models are built on the following intuition: it is simple to define a gradual process transforming data into noise, thus it should be possible to learn to gradually reverse that process. Formally, one can easily construct a Markov chain $\{X_t\}_{t=1}^\infty$ with Gaussian transitions starting from data, i.e. $X_0 \sim p$. This Markov chain is commonly called the "forward process". A diffusion model learns a Markovian generative process that maps Gaussian noise to data via a learned *backward* Markov chain; training is done by matching that backward chain to the reverse of a fixed *forward* noising chain via variational inference. The result is a powerful composition of

simple Gaussian transitions, resulting in a model capable of capturing many complex real-world data distributions. In this section we will see how to precisely define Denoising Diffusion Probabilistic Models (DDPMs), a specific type of diffusion model implementing these high level ideas.

### 7.5.2 Forward (noising) process

Define a discrete time index $t = 0, \dots, T$ with $x_0$ the data and $x_T$ near isotropic Gaussian noise. The forward process (fixed) is a Markov chain:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \qquad q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t; \sqrt{\alpha_t}\, x_{t-1},\ \beta_t I\big),$$

where we typically set $\alpha_t = 1 - \beta_t$ and choose a small schedule $\beta_t \in (0, 1)$ (often increasing). Define the cumulative product

$$\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s.$$

Because the transitions are linear-Gaussian, we have a closed form marginal:

$$\boxed{q(x_t \mid x_0) = \mathcal{N}\big(x_t; \sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t) I\big).}$$

This formula is useful: to sample $x_t$ given $x_0$ we may draw $\varepsilon \sim \mathcal{N}(0, I)$ and compute

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon.$$

instead of simulating every transition along the process. Notice that for $\beta_t$ increasing we have that $\bar{\alpha}_t \to 0$, and thus $q(x_T)$ is close to $\mathcal{N}(0, I)$ for large $T$. The forward process is said to *diffuse* data into Gaussian noise.

### 7.5.3 Goal: learn a backward (denoising) process

We learn a parameterized backward chain

$$p_\lambda(x_{0:T}) = p_\lambda(x_T) \prod_{t=1}^{T} p_\lambda(x_{t-1} \mid x_t), \quad p_\lambda(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1}; \mu_\lambda(x_t, t), \Sigma_\lambda(x_t, t)\big),$$

with $p_\lambda(x_T)$ usually set to $\mathcal{N}(0, I)$. Note that the backwards transitions $p_\lambda(x_{t-1}|x_t)$ are once again Gaussian, this time with learned mean $\mu_\lambda$ and covariance $\Sigma_\lambda$. We fit $p_\lambda$ by maximizing the data marginal log-likelihood $\log p_\lambda(x_0)$:

$$\log p_\lambda(x_0) = \log \int p_\lambda(x_{0:T}) dx_{1:T} \tag{7.45}$$

As before, this integral is intractable. In the past, we used variational inference to find a posterior $q_\lambda$ maximising a lower bound on the evidence above. Note that in the case of diffusion models $q$ is fixed, so we derive a variational lower bound (ELBO) using the fixed forward process $q$ as the variational/exact posterior family.

### 7.5.4 *ELBO derivation*

As in previous subsections, we aim to maximize the ELBO, leading to a tractable loss function. Recall that maximizing the ELBO amounts to minizing KL divergence between our model's posterior and the "true" data posterior:

$$\mathrm{KL}(q|p_\lambda(\cdot|x_0)) = -\mathcal{L}(\lambda; x_0) + \text{const} \tag{7.46}$$

where

$$\mathcal{L}(\lambda; x_0) = \mathbb{E}_{x_{1:n}\sim q(\cdot|x_0)}[\log p_\lambda(x_{0:n})] - \log q(x_{1:n}|x_0) \tag{7.47}$$

(see equation 5.55a). Note that whereas before we had a fixed inference model and were trying to approximate the posterior, we now instead fix the posterior $q$ and try to find a model $p_\lambda(x_{0:T})$ minimizing the same KL divergence. As before, the ELBO can be interpreted as a lower bound on log likelihood, which can be derived as follows:

Start from

$$\log p_\lambda(x_0) = \log \int p_\lambda(x_{0:T})\, dx_{1:T} = \log \int \frac{p_\lambda(x_{0:T})}{q(x_{1:T} \mid x_0)} q(x_{1:T} \mid x_0)\, dx_{1:T}.$$

Applying Jensen (or standard VI), we get the following ELBO

$$\mathcal{L}(\lambda; x_0) \coloneqq \mathbb{E}_{q(x_{1:T}|x_0)}\Big[\log p_\lambda(x_{0:T}) - \log q(x_{1:T} \mid x_0)\Big] \leq \log p_\lambda(x_0).$$

Expanding both chains and using the Markov structure yields a telescoping sum; a convenient rearrangement (as in Ho et al. 2020) is:

$$\begin{aligned}\mathcal{L}(\lambda; x_0) = \mathbb{E}_q\big[\log p_\lambda(x_T)\big] - \sum_{t=2}^{T} \mathbb{E}_q\Big[\mathrm{KL}\big(q(x_{t-1} \mid x_t, x_0) \parallel p_\lambda(x_{t-1} \mid x_t)\big)\Big] \\ - \mathbb{E}_q\big[-\log p_\lambda(x_0 \mid x_1)\big].\end{aligned} \tag{7.48}$$

Thus maximizing the ELBO (equivalently minimizing the negative ELBO) encourages the learned local conditional $p_\lambda(x_{t-1} \mid x_t)$ to match the exact reverse conditional $q(x_{t-1} \mid x_t, x_0)$ at each $t$.

### 7.5.5 *The exact reverse conditional*

Because the forward process is linear-Gaussian, the posterior

$$q(x_{t-1} \mid x_t, x_0)$$

of the forward process transitions is Gaussian and admits closed form mean and variance. One convenient form is:

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(x_{t-1}; \tilde{\mu}_t(x_t, x_0),\ \tilde{\beta}_t I\big),$$

with

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t,$$

and

$$\boxed{\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\, x_t.}$$

### 7.5.6 *Closed-form KLs and training objective*

Each KL term in (7.48) is between two Gaussians, which admits the following closed form in general:

$$\mathrm{KL}\big(\mathcal{N}(\mu_q, \Sigma_q) \,\|\, \mathcal{N}(\mu_p, \Sigma_p)\big) = \frac{1}{2}\Big[\mathrm{tr}(\Sigma_p^{-1}\Sigma_q) + (\mu_p - \mu_q)^\top \Sigma_p^{-1}(\mu_p - \mu_q) - d + \log \frac{\det(\Sigma)_p}{\det(\Sigma)_q}\Big].$$

For isotropic covariances with $\Sigma_p = \sigma_p^2 I$ and $\Sigma_q = \sigma_q^2 I$ this simplifies (up to additive constants independent of $\mu_p$) to

$$\mathrm{KL}(\cdot) \propto \frac{1}{2\sigma_p^2} \|\mu_p - \mu_q\|^2 + (\text{const}).$$

In practice one often fixes the model covariance $\Sigma_\lambda(x_t, t) = \sigma_t^2 I$ (for all $x_t$) and trains only the mean $\mu_\lambda$. Then each KL term reduces to a squared error on the means, weighted by $1/(2\sigma_t^2)$:

$$\mathrm{KL}\big(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\lambda(x_{t-1} \mid x_t) \propto \frac{1}{2\sigma_t^2} \|\mu_\lambda(x_t, t)) - \bar{\mu}(x_t, t)\|^2 + (\text{const}) \tag{7.49}$$

which informs the choice of our first loss function:

$$\mathcal{L}(\lambda) = \mathbb{E}_{t \sim U[1,T],\, x_0 \sim p,\, x_t \sim q(x_t \mid x_0)} \Big[\frac{1}{2\sigma_t^2} \|\mu_\lambda(x_t, t)) - \bar{\mu}(x_t, t)\|^2\Big] \tag{7.50}$$

### 7.5.7 *Mean prediction vs. noise prediction*

The derivations above suggest to minimise the squared error $\|\mu_\lambda - \bar{\mu}_t\|^2$. This is essentially equivalent to learning an estimator $\hat{x}_0$ for $x_0$, and predicting $\bar{\mu}_t(x_t, x_0)$ as $\bar{\mu}_t(x_t, \hat{x}_0)$. A widely used reparametrization predicts the noise $\varepsilon$ used to construct $x_t$ instead:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

Let $\varepsilon_\lambda(x_t, t)$ be the neural network's prediction of $\varepsilon$. One can recover an estimator $\hat{x}_0$:

$$\hat{x}_0(x_t) = \frac{1}{\sqrt{\bar{\alpha}_t}}\Big(x_t - \sqrt{1-\bar{\alpha}_t}\, \varepsilon_\lambda(x_t, t)\Big).$$

Plugging $\hat{x}_0$ into the formula for $\tilde{\mu}_t$ yields a model mean

$$\mu_\lambda(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\Big(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon_\lambda(x_t, t)\Big).$$

With algebra one shows the squared mean error becomes proportional to a squared error on $\varepsilon$:

$$\boxed{\mathcal{L}_{\text{simple}}(\lambda) = \mathbb{E}_{t\sim U[1,T],\, x_0\sim p,\, \varepsilon\sim\mathcal{N}(0,I)}\Big[\lambda_t\, \big\|\varepsilon - \varepsilon_\lambda(x_t, t)\big\|^2\Big],}$$

for appropriate weights $\lambda_t$ that depend on $\{\alpha_s\}$ and chosen $\sigma_t^2$. In practice, choosing a simple unweighted MSE (or fixed per-step weights) works extremely well and is the standard training objective used in implementations (often called the $\ell_{\text{simple}}$ loss). Empirically noise prediction is more stable and easier to optimize than direct mean prediction.

**Algorithm 7.5:** Training step (single SGD update)

**repeat**
- Sample minibatch $\{x_0^{(i)}\}$ from data.
- Sample $t \sim \text{Uniform}\{1, \dots, T\}$ for each data point.
- Sample $\varepsilon \sim \mathcal{N}(0, I)$ and form $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$.
- Compute $\varepsilon_\lambda(x_t, t)$ and loss $\|\varepsilon - \varepsilon_\lambda(x_t, t)\|^2$ (possibly weighted).
- SGD step on $\lambda$ to minimize loss.

**until** converged

### 7.5.8 *Connections to score matching and Langevin dynamics*

In this subsection we make explicit the algebraic and conceptual connections between the DDPM *noise-prediction* parameterization and *score*

*matching*, and we show how the discrete-time backward sampling step resembles a single step of annealed Langevin dynamics. This subsection is written as a direct extension of the variational-derivation and sampling algorithm presented earlier.

The noise-prediction parametrization hints at an interpretation of sampling according to annealed Langevin dynamics. Indeed, we will see that the noise-prediction loss $\mathcal{L}_{\text{simple}}$ greatly resembles a score-matching loss.

### 7.5.9 Noise prediction as a score estimator

Recall the forward marginal used in DDPMs:

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t; \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I\big), \tag{7.51}$$

the conditional score (gradient of the log conditional density with respect to $x_t$) is

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{1-\bar{\alpha}_t}. \tag{7.52}$$

Using the reparameterization

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{7.53}$$

we get the identity

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{\sqrt{1-\bar{\alpha}_t}\, \varepsilon}{1-\bar{\alpha}_t} = -\frac{\varepsilon}{\sqrt{1-\bar{\alpha}_t}}. \tag{7.54}$$

The marginal (true) score at time $t$ is the expectation of the conditional score under the posterior $q(x_0 \mid x_t)$:

$$s_t(x_t) \equiv \nabla_{x_t} \log q_t(x_t) = \mathbb{E}_{q(x_0 \mid x_t)}\big[\nabla_{x_t} \log q(x_t \mid x_0)\big] = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\, \mathbb{E}[\varepsilon \mid x_t]. \tag{7.55}$$

DDPM-style training minimizes an MSE loss to predict $\varepsilon$ from $x_t$, so a trained network $\varepsilon_\lambda(x_t, t)$ approximates the posterior mean $\mathbb{E}[\varepsilon \mid x_t]$. Hence we obtain an explicit score estimator:

$$s_\lambda(x_t, t) \approx -\frac{\varepsilon_\lambda(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}. \tag{7.56}$$

This formula is the algebraic bridge between noise prediction in DDPMs and denoising score matching: a single network serves both as a noise predictor and (after rescaling) as a score function at noise level $t$.

### 7.5.10 DDPM backward update rewritten using the score

Recall the standard noise-prediction parameterization relation between the model mean and the predicted noise:

$$\mu_\lambda(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\Big(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon_\lambda(x_t, t)\Big). \tag{7.57}$$

Using (7.56) we substitute $\varepsilon_\lambda = -\sqrt{1-\bar{\alpha}_t}\, s_\lambda$ to get the compact form

$$\mu_\lambda(x_t, t) = \frac{x_t + \beta_t\, s_\lambda(x_t, t)}{\sqrt{\alpha_t}}. \tag{7.58}$$

The stochastic backward update with additive Gaussian noise (with chosen variance $\sigma_t^2$) is

$$x_{t-1} = \mu_\lambda(x_t, t) + \sigma_t z = \frac{x_t + \beta_t s_\lambda(x_t, t)}{\sqrt{\alpha_t}} + \sigma_t z, \qquad z \sim \mathcal{N}(0, I). \tag{7.59}$$

To make the Langevin-like structure explicit, rearrange (7.59) to isolate the increment from $x_t$:

$$x_{t-1} = x_t + \Big(\frac{1}{\sqrt{\alpha_t}} - 1\Big) x_t + \frac{\beta_t}{\sqrt{\alpha_t}}\, s_\lambda(x_t, t) + \sigma_t z. \tag{7.60}$$

For small $\beta_t$ (hence $\alpha_t = 1 - \beta_t$ close to 1) we may use the expansions

$$\frac{1}{\sqrt{\alpha_t}} - 1 = \frac{1}{\sqrt{1-\beta_t}} - 1 \approx \frac{\beta_t}{2} + \mathcal{O}(\beta_t^2), \qquad \frac{\beta_t}{\sqrt{\alpha_t}} \approx \beta_t + \mathcal{O}(\beta_t^2). \tag{7.61}$$

If additionally $\sigma_t^2 = \mathcal{O}(\beta_t)$ (as is common), then to leading order in $\beta_t$ the update becomes

$$\boxed{x_{t-1} \approx x_t + \beta_t\, s_\lambda(x_t, t) + \sqrt{\beta_t}\, z, \qquad z \sim \mathcal{N}(0, I).}$$

This is the familiar form of an Euler–Maruyama discretization of a reverse-time SDE (or a single step of annealed Langevin dynamics, see e.g. (6.47)): a small drift in the direction of the score (uphill in log-density) plus a random Gaussian perturbation whose magnitude scales with the step size.

## *Discussion*

This chapter concludes our discussion of (approximate) probabilistic inference. Across the last three chapters, we have seen numerous methods for approximating the posterior distributions of deep neural networks:

- Methods such as dropout and stochastic weight averaging are frequently used in practice. Other particle-based approaches such as ensembles and SVGD are used less frequently since they are computationally more expensive to train, but are some of the most effective methods in estimating uncertainty.
- Recently, Laplace approximations regained interest since they can be applied "post-hoc" after training simply by computing or approximating the Hessian of the loss function (Daxberger et al., 2021; Antorán et al., 2022). Still, Laplace approximations come with the limitations inherent to unimodal Gaussian approximations.

- Other work, particularly in fine-tuning, has explored approximating the posterior distribution of deep neural networks by treating them as linear functions in a fixed learned feature space, in which case one can use the tools for exact probabilistic inference from Chapters 2 and 4 (e.g., Hübotter et al., 2025).

Despite large progress in approximate inference over the past decade, efficient and reliable uncertainty estimation of large models remains an important open challenge.

## *Problems*

**7.1. Softmax is a generalization of the logistic function.**

Show that for a two-class classification problem (i.e., $c = 2$), the softmax function is equivalent to the logistic function (5.9) for the univariate model $f \doteq f_1 - f_0$. That is, $\sigma_1(\boldsymbol{f}) = \sigma(f)$ and $\sigma_0(\boldsymbol{f}) = 1 - \sigma(f)$.

Thus, the softmax function is a generalization of the logistic function to more than two classes.

PART II

# *Sequential Decision-Making*

# *Preface to Part II*

In the first part of the course, we have learned about how we can build machines that are capable of updating their beliefs and reducing their epistemic uncertainty through probabilistic inference. We have also discussed ways of keeping track of the world through noisy sensory information by filtering. An important aspect of intelligence is to use this acquired knowledge for making decisions and taking actions that have a positive impact on the world.

Already today, we are surrounded by machines that make decisions and take actions; that is, exhibit some degree of agency. Be it a search engine producing a list of search results, a chatbot answering a question, or a driving-assistance system steering a car: these systems are all perceiving the world, making decisions, and then taking actions that in turn have an effect on the world. Figure 7.10 illustrates this *perception-action loop*.

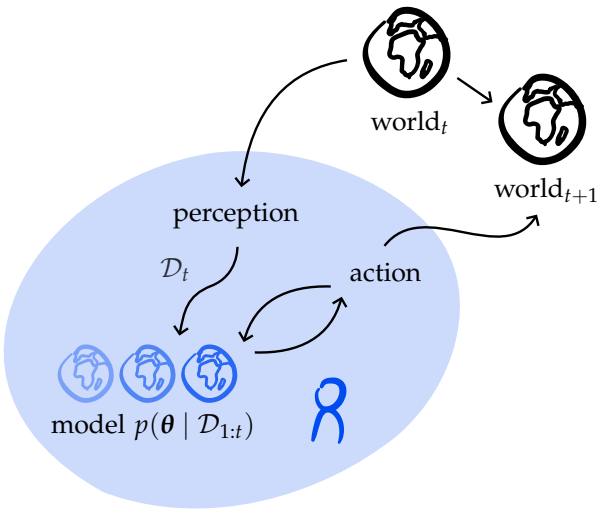



Figure 7.10: An illustration of the perception-action loop. This is a straightforward extension of our view of probabilistic inference from Figure 1.10 with the addition of an "action" component which is capable of "adaptively" interacting with the outside world and the internal world model.

In the second part of this course, we will discuss the underpinning principles of building machines that are capable of making sequential

decisions. We will see that decision-making itself can be cast as probabilistic inference, obeying the same mechanisms that we used in the first part to build learning systems.

We discuss various ways of addressing the question:

*How to act, given that computational resources and time are limited?*

One approach is to act with the aim to reduce epistemic uncertainty, which is the topic of active learning. Another approach is to act with the aim to maximize some reward signal, which is the topic of bandits, Bayesian optimization, and reinforcement learning.

This surfaces the exploration-exploitation dilemma where the agent has to prioritize either maximizing its immediate rewards or reducing its uncertainty about the world which might pay off in the future. We discuss that this dilemma is, in fact, in direct correspondence to the principle of curiosity and conformity which we discussed extensively throughout Part I.

Since time is limited, it is critical to be sample-efficient when learning the most important aspects of the world. At the same time, interactions with the world are often complex, and some interactions might even be harmful. We discuss how an agent can use its epistemic uncertainty to guide the exploration of its environment while mitigating risks and reasoning about safety.

# 8

# *Active Learning*

By now, we have seen that probabilistic machine learning is very useful for estimating the uncertainty in our models (epistemic uncertainty) and in the data (aleatoric uncertainty). We have been focusing on the setting of supervised learning where we are given a set $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ of labeled data, yet we often encounter settings where we have only little data and acquiring new data is costly.

In this chapter — and in the following chapter on Bayesian optimization — we will discuss how one can use uncertainty to effectively collect more data. In other words, we want to figure out where in the domain we should sample to obtain the most useful information. Throughout most of this chapter, we focus on the most common way of quantifying "useful information", namely the expected reduction in entropy which is also called the *mutual information*.

## *8.1 Conditional Entropy*

We begin by introducing the notion of conditional entropy. Recall that the entropy $\mathrm{H}[\mathbf{X}]$ of a random vector $\mathbf{X}$ can be interpreted as our average surprise when observing realizations $\boldsymbol{x} \sim \mathbf{X}$. Thus, entropy can be considered as a quantification of the uncertainty about a random vector (or equivalently, its distribution).[1]

[1] We discussed entropy extensively in Section 5.4.

A natural extension is to consider the entropy of $\mathbf{X}$ given the occurrence of an event corresponding to another random variable (e.g., $\mathbf{Y} = \boldsymbol{y}$ for a random vector $\mathbf{Y}$),

$$\mathrm{H}[\mathbf{X} \mid \mathbf{Y} = \boldsymbol{y}] \doteq \mathbb{E}_{\boldsymbol{x} \sim p(\boldsymbol{x} \mid \boldsymbol{y})}[-\log p(\boldsymbol{x} \mid \boldsymbol{y})]. \tag{8.1}$$

Instead of averaging over the surprise of samples from the distribution $p(\boldsymbol{x})$ (like the entropy $\mathrm{H}[\mathbf{X}]$), this quantity simply averages over the surprise of samples from the conditional distribution $p(\boldsymbol{x} \mid \boldsymbol{y})$.

**Definition 8.1** (Conditional entropy)**.** The *conditional entropy* of a random vector $\mathbf{X}$ given the random vector $\mathbf{Y}$ is defined as

$$\mathrm{H}[\mathbf{X} \mid \mathbf{Y}] \doteq \mathbb{E}_{\boldsymbol{y} \sim p(\boldsymbol{y})}[\mathrm{H}[\mathbf{X} \mid \mathbf{Y} = \boldsymbol{y}]] \tag{8.2}$$

$$= \mathbb{E}_{(\boldsymbol{x}, \boldsymbol{y}) \sim p(\boldsymbol{x}, \boldsymbol{y})}[-\log p(\boldsymbol{x} \mid \boldsymbol{y})]. \tag{8.3}$$

Intuitively, the conditional entropy of $\mathbf{X}$ given $\mathbf{Y}$ describes our average surprise about realizations of $\mathbf{X}$ given a particular realization of $\mathbf{Y}$, averaged over all such possible realizations of $\mathbf{Y}$. In other words, conditional entropy corresponds to the expected remaining uncertainty in $\mathbf{X}$ after we observe $\mathbf{Y}$. Note that, in general, $\mathrm{H}[\mathbf{X} \mid \mathbf{Y}] \neq \mathrm{H}[\mathbf{Y} \mid \mathbf{X}]$.

It is crucial to stress the difference between $\mathrm{H}[\mathbf{X} \mid \mathbf{Y} = \boldsymbol{y}]$ and the conditional entropy $\mathrm{H}[\mathbf{X} \mid \mathbf{Y}]$. The former simply corresponds to a probabilistic update of our uncertainty in $\mathbf{X}$ after we have observed the realization $\boldsymbol{y} \sim \mathbf{Y}$. In contrast, conditional entropy *predicts* how much uncertainty will remain about $\mathbf{X}$ (in expectation) after we *will* observe $\mathbf{Y}$.

**Definition 8.2** (Joint entropy)**.** One can also define the *joint entropy* of random vectors $\mathbf{X}$ and $\mathbf{Y}$,

$$\mathrm{H}[\mathbf{X}, \mathbf{Y}] \doteq \mathbb{E}_{(\boldsymbol{x}, \boldsymbol{y}) \sim p(\boldsymbol{x}, \boldsymbol{y})}[-\log p(\boldsymbol{x}, \boldsymbol{y})], \tag{8.4}$$

as the combined uncertainty about $\mathbf{X}$ and $\mathbf{Y}$. Observe that joint entropy is symmetric.

This gives the *chain rule for entropy*,

$$\mathrm{H}[\mathbf{X}, \mathbf{Y}] = \mathrm{H}[\mathbf{Y}] + \mathrm{H}[\mathbf{X} \mid \mathbf{Y}] \tag{8.5}$$

using the product rule (1.11) and the definition of conditional entropy (8.2)

$$= \mathrm{H}[\mathbf{X}] + \mathrm{H}[\mathbf{Y} \mid \mathbf{X}]. \tag{8.6}$$

using symmetry of joint entropy

That is, the joint entropy of $\mathbf{X}$ and $\mathbf{Y}$ is given by the uncertainty about $\mathbf{X}$ and the additional uncertainty about $\mathbf{Y}$ given $\mathbf{X}$. Moreover, this also yields *Bayes' rule for entropy*,

$$\mathrm{H}[\mathbf{X} \mid \mathbf{Y}] = \mathrm{H}[\mathbf{Y} \mid \mathbf{X}] + \mathrm{H}[\mathbf{X}] - \mathrm{H}[\mathbf{Y}]. \tag{8.7}$$

using the chain rule for entropy (8.5) twice

A very intuitive property of entropy is its monotonicity: when conditioning on additional observations the entropy can never increase,

$$\mathrm{H}[\mathbf{X} \mid \mathbf{Y}] \leq \mathrm{H}[\mathbf{X}]. \tag{8.8}$$

Colloquially, this property is also called the "*information never hurts*" principle. We will derive a proof in the following section.

## 8.2 Mutual Information

Recall that our fundamental objective is to reduce entropy, as this corresponds to reduced uncertainty in the variables, which we want to

predict. Thus, we are interested in how much information we "gain" about the random vector $\mathbf{X}$ by choosing to observe a random vector $\mathbf{Y}$. By our interpretation of conditional entropy from the previous section, this is described by the following quantity.

**Definition 8.3** (Mutual information, MI)**.** The *mutual information* of $\mathbf{X}$ and $\mathbf{Y}$ (also known as the *information gain*) is defined as

$$\begin{aligned} \mathrm{I}(\mathbf{X};\mathbf{Y}) &\doteq \mathrm{H}[\mathbf{X}] - \mathrm{H}[\mathbf{X} \mid \mathbf{Y}] && (8.9)\\ &= \mathrm{H}[\mathbf{X}] + \mathrm{H}[\mathbf{Y}] - \mathrm{H}[\mathbf{X},\mathbf{Y}]. && (8.10) \end{aligned}$$

In words, we subtract the uncertainty left about $\mathbf{X}$ after observing $\mathbf{Y}$ from our initial uncertainty about $\mathbf{X}$. This measures the reduction in our uncertainty in $\mathbf{X}$ (as measured by entropy) upon observing $\mathbf{Y}$. Unlike conditional entropy, it follows from the definition that mutual information is symmetric. That is,

$$\mathrm{I}(\mathbf{X};\mathbf{Y}) = \mathrm{I}(\mathbf{Y};\mathbf{X}). \tag{8.11}$$

Thus, the mutual information between $\mathbf{X}$ and $\mathbf{Y}$ can be understood as the approximation error (or information loss) when assuming that $\mathbf{X}$ and $\mathbf{Y}$ are independent.

In particular, using Gibbs' inequality (cf. Problem 5.5), this relationship shows that $\mathrm{I}(\mathbf{X};\mathbf{Y}) \geq 0$ with equality when $\mathbf{X}$ and $\mathbf{Y}$ are independent, and also proves the *information never hurts* principle (8.8) as

$$0 \leq \mathrm{I}(\mathbf{X};\mathbf{Y}) = \mathrm{H}[\mathbf{X}] - \mathrm{H}[\mathbf{X} \mid \mathbf{Y}]. \tag{8.12}$$

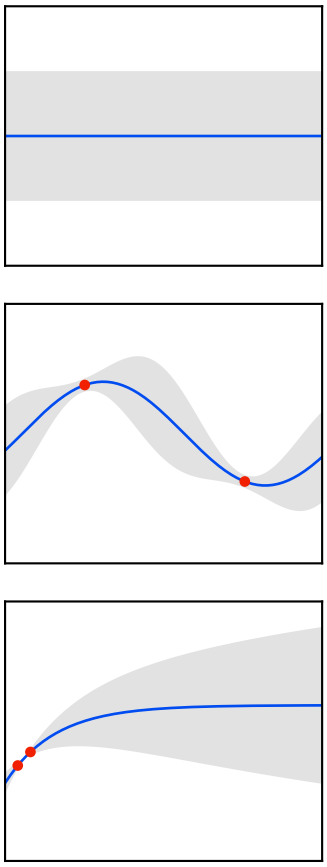


Figure 8.1: Information gain. The first graph shows the prior. The second graph shows a selection of samples with large information gain (large uncertainty reduction). The third graph shows a selection of samples with small information gain (small uncertainty reduction).

**Example 8.4: Mutual information of Gaussians**

Given the Gaussian random vector $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and the noisy observation $\mathbf{Y} = \mathbf{X} + \boldsymbol{\varepsilon}$ where $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_\mathrm{n}^2 \boldsymbol{I})$, we want to find the information gain of $\mathbf{X}$ when observing $\mathbf{Y}$. Using our definitions from this chapter, we obtain

$$\begin{aligned} \mathrm{I}(\mathbf{X};\mathbf{Y}) &= \mathrm{I}(\mathbf{Y};\mathbf{X}) && \text{using symmetry (8.11)}\\ &= \mathrm{H}[\mathbf{Y}] - \mathrm{H}[\mathbf{Y} \mid \mathbf{X}] && \text{by mutual information (8.9)}\\ &= \mathrm{H}[\mathbf{Y}] - \mathrm{H}[\boldsymbol{\varepsilon}] && \text{given } \mathbf{X}\text{, the only randomness in } \mathbf{Y} \text{ originates from } \boldsymbol{\varepsilon}\\ &= \frac{1}{2}\log\Big((2\pi e)^d \det\Big(\boldsymbol{\Sigma} + \sigma_\mathrm{n}^2 \boldsymbol{I}\Big)\Big) - \frac{1}{2}\log\Big((2\pi e)^d \det\Big(\sigma_\mathrm{n}^2 \boldsymbol{I}\Big)\Big) && \text{using the entropy of Gaussians (5.31)}\\ &= \frac{1}{2}\log\frac{\det\big(\boldsymbol{\Sigma} + \sigma_\mathrm{n}^2 \boldsymbol{I}\big)}{\det(\sigma_\mathrm{n}^2 \boldsymbol{I})}\\ &= \frac{1}{2}\log\det\Big(\boldsymbol{I} + \sigma_\mathrm{n}^{-2}\boldsymbol{\Sigma}\Big). && (8.13) \end{aligned}$$

Intuitively, the larger the noise $\sigma_\mathrm{n}^2$ in relation to the covariance

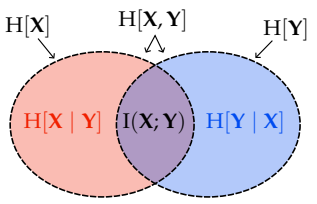



Figure 8.2: Relationship between mutual information and entropy, expressed as a Venn diagram.

of $\mathbf{X}$, the smaller the information gain.

### 8.2.1 Synergy and Redundancy

It is sometimes useful to write down the mutual information of $\mathbf{X}$ and $\mathbf{Y}$ conditioned (in expectation) on a third random vector $\mathbf{Z}$. This leads us to the following definition.

**Definition 8.5** (Conditional mutual information)**.** The *conditional mutual information* of $\mathbf{X}$ and $\mathbf{Y}$ given $\mathbf{Z}$ is defined as

$$\mathrm{I}(\mathbf{X};\mathbf{Y} \mid \mathbf{Z}) \doteq \mathrm{H}[\mathbf{X} \mid \mathbf{Z}] - \mathrm{H}[\mathbf{X} \mid \mathbf{Y},\mathbf{Z}]. \tag{8.14}$$

$$= \mathrm{H}[\mathbf{X},\mathbf{Z}] + \mathrm{H}[\mathbf{Y},\mathbf{Z}] - \mathrm{H}[\mathbf{Z}] - \mathrm{H}[\mathbf{X},\mathbf{Y},\mathbf{Z}] \tag{8.15}$$

using the relationship of joint and conditional entropy (8.5)

$$= \mathrm{I}(\mathbf{X};\mathbf{Y},\mathbf{Z}) - \mathrm{I}(\mathbf{X};\mathbf{Z}). \tag{8.16}$$

Thus, the conditional mutual information corresponds to the reduction of uncertainty in $\mathbf{X}$ when observing $\mathbf{Y}$, given we already observed $\mathbf{Z}$. It also follows that conditional mutual information is symmetric:

$$\mathrm{I}(\mathbf{X};\mathbf{Y} \mid \mathbf{Z}) = \mathrm{I}(\mathbf{Y};\mathbf{X} \mid \mathbf{Z})\,. \tag{8.17}$$

We have seen in this chapter that entropy is monotonically decreasing as we condition on new information, and called this the "information never hurts" principle (8.8). However, the same does not hold for mutual information! That is, information about a random vector $\mathbf{Z}$ may reduce the mutual information between random vectors $\mathbf{X}$ and $\mathbf{Y}$ (?).

**Problem 8.2**

**Remark 8.6: Sufficient statistics and data processing inequality**

A related concept is the data processing inequality (8.42) which you prove in (?) and which allows us to formalize a concept which we have seen multiple times already, namely the notion of a sufficient statistic. Consider the Markov chain $\lambda \to \mathbf{X} \to s(\mathbf{X})$, for example, $\lambda$ may be parameters of the distribution of $\mathbf{X}$. By the data processing inequality (8.42), $\mathrm{I}(\lambda; s(\mathbf{X})) \leq \mathrm{I}(\lambda;\mathbf{X})$. If the data processing inequality is satisfied with equality then $s(\mathbf{X})$ is called a *sufficient statistic* of $\mathbf{X}$ for the inference of $\lambda$.

**Problem 8.2 (3)**

To understand the behavior of mutual information under conditioning, it is helpful to consider the *interaction information*

$$\mathrm{I}(\mathbf{X};\mathbf{Y};\mathbf{Z}) \doteq \mathrm{I}(\mathbf{X};\mathbf{Y}) - \mathrm{I}(\mathbf{X};\mathbf{Y} \mid \mathbf{Z}). \tag{8.18}$$

If the interaction is positive then some information about $\mathbf{X}$ that is provided by $\mathbf{Y}$ is also provided by $\mathbf{Z}$ (i.e., conditioning on $\mathbf{Z}$ decreases MI between $\mathbf{X}$ and $\mathbf{Y}$), and we say that there is *redundancy* between $\mathbf{Y}$

and $\mathbf{Z}$ (with respect to $\mathbf{X}$). Conversely, if the interaction is negative then learning about $\mathbf{Z}$ increases what can be learned from $\mathbf{Y}$ about $\mathbf{X}$, and we say that there is *synergy* between $\mathbf{Y}$ and $\mathbf{Z}$. We will see later in this chapter that the absence of synergies can lead to efficient algorithms for maximizing mutual information.

### 8.2.2 Mutual Information as Utility Function

Following our introduction of mutual information, it is natural to answer the question "where should I collect data?" by saying "wherever mutual information is maximized". More concretely, assume we are given a set $\mathcal{X}$ of possible observations of $f$, where $y_{\boldsymbol{x}}$ denotes a single such observation at $\boldsymbol{x} \in \mathcal{X}$,

$$y_{\boldsymbol{x}} \doteq f_{\boldsymbol{x}} + \varepsilon_{\boldsymbol{x}}, \tag{8.19}$$

$f_{\boldsymbol{x}} \doteq f(\boldsymbol{x})$, and $\varepsilon_{\boldsymbol{x}}$ is some zero-mean Gaussian noise. For a set of observations $S = \{\boldsymbol{x}_1, \dots, \boldsymbol{x}_n\}$, we can write $\boldsymbol{y}_S = \boldsymbol{f}_S + \boldsymbol{\varepsilon}$ where

$$\boldsymbol{y}_S \doteq \begin{bmatrix} y_{\boldsymbol{x}_1} \\ \vdots \\ y_{\boldsymbol{x}_n} \end{bmatrix}, \quad \boldsymbol{f}_S \doteq \begin{bmatrix} f_{\boldsymbol{x}_1} \\ \vdots \\ f_{\boldsymbol{x}_n} \end{bmatrix}, \quad \text{and} \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_{\mathrm{n}}^2 \boldsymbol{I}).$$

Note that both $\boldsymbol{y}_S$ and $\boldsymbol{f}_S$ are random vectors. Our goal is then to find a subset $S \subseteq \mathcal{X}$ of size $n$ maximizing the information gain between our model $f$ and $\boldsymbol{y}_S$.

This yields the maximization objective,

$$I(S) \doteq \mathrm{I}(\boldsymbol{f}_S; \boldsymbol{y}_S) = \mathrm{H}[\boldsymbol{f}_S] - \mathrm{H}[\boldsymbol{f}_S \mid \boldsymbol{y}_S]. \tag{8.20}$$

Here, $\mathrm{H}[\boldsymbol{f}_S]$ corresponds to the uncertainty about $\boldsymbol{f}_S$ before obtaining the observations $\boldsymbol{y}_S$ and $\mathrm{H}[\boldsymbol{f}_S \mid \boldsymbol{y}_S]$ corresponds to the uncertainty about $\boldsymbol{f}_S$, in expectation, after obtaining the observations $\boldsymbol{y}_S$.

**Remark 8.7: Making optimal decisions with intrinsic rewards**

Note that this objective function maps neatly onto our initial consideration of making optimal decisions under uncertainty from Section 1.4. In fact, you can think of maximizing $I(S)$ simply as computing the optimal decision rule for the utility

$$r(\boldsymbol{y}_S, S) = \mathrm{H}[\boldsymbol{f}_S] - \mathrm{H}[\boldsymbol{f}_S \mid \mathbf{Y}_S = \boldsymbol{y}_S], \tag{8.21}$$

with $\mathbf{Y}_S = \boldsymbol{y}_S$ denoting an event

with $I(S) = \mathbb{E}_{\boldsymbol{y}_S}[r(\boldsymbol{y}_S, S)]$ measuring the expected utility of observations $\boldsymbol{y}_S$. Such a utility or reward function is often called an *intrinsic reward* since it does not measure an "objective" external quantity, but instead a "subjective" quantity that is internal to the

model of $f$.

Observe that picking a subset of points $S \subseteq \mathcal{X}$ from the domain $\mathcal{X}$ is a combinatorial problem. That is to say, we are optimizing a function over discrete sets. In general, such combinatorial optimization problems tend to be very difficult. It can be shown that maximizing mutual information is $\mathcal{NP}$-hard.

## 8.3 Submodularity of Mutual Information

We will look at optimizing mutual information in the following section. First, we want to introduce the notion of submodularity which is important in the analysis of discrete functions.

**Definition 8.8** (Marginal gain)**.** Given a (discrete) function $F : \mathcal{P}(\mathcal{X}) \to \mathbb{R}$, the *marginal gain* of $\boldsymbol{x} \in \mathcal{X}$ given $A \subseteq \mathcal{X}$ is defined as

$$\Delta_F(\boldsymbol{x} \mid A) \doteq F(A \cup \{\boldsymbol{x}\}) - F(A). \tag{8.22}$$

Intuitively, the marginal gain describes how much "adding" the additional $\boldsymbol{x}$ to $A$ increases the value of $F$.

When maximizing mutual information, the marginal gain is (?)

**Problem 8.4**

$$\begin{aligned} \Delta_I(\boldsymbol{x} \mid A) &= \mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{y}_A) && (8.23)\\ &= \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{y}_A] - \mathrm{H}[\varepsilon_{\boldsymbol{x}}]. && (8.24) \end{aligned}$$

That is, when maximizing mutual information, the marginal gain corresponds to the difference between the uncertainty after observing $\boldsymbol{y}_A$ and the entropy of the noise $\mathrm{H}[\varepsilon_{\boldsymbol{x}}]$. Altogether, the marginal gain represents the reduction in uncertainty by observing $\{\boldsymbol{x}\}$.

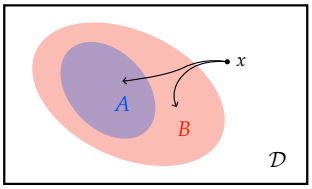



Figure 8.3: Monotone submodularity. The effect of "adding" $\boldsymbol{x}$ to the smaller set $A$ is larger than the effect of adding $\boldsymbol{x}$ to the larger set $B$.

**Definition 8.9** (Submodularity)**.** A (discrete) function $F : \mathcal{P}(\mathcal{X}) \to \mathbb{R}$ is *submodular* iff for any $\boldsymbol{x} \in \mathcal{X}$ and any $A \subseteq B \subseteq \mathcal{X}$ it satisfies

$$F(A \cup \{\boldsymbol{x}\}) - F(A) \geq F(B \cup \{\boldsymbol{x}\}) - F(B). \tag{8.25}$$

Equivalently, using our definition of marginal gain, we have that $F$ is submodular iff for any $\boldsymbol{x} \in \mathcal{X}$ and any $A \subseteq B \subseteq \mathcal{X}$,

$$\Delta_F(\boldsymbol{x} \mid A) \geq \Delta_F(\boldsymbol{x} \mid B). \tag{8.26}$$

That is, "adding" $\boldsymbol{x}$ to the smaller set $A$ yields more marginal gain than adding $\boldsymbol{x}$ to the larger set $B$. In other words, the function $F$ has "diminishing returns". In this way, submodularity can be interpreted as a notion of "concavity" for discrete functions.

**Definition 8.10** (Monotone submodularity)**.** A function $F : \mathcal{P}(\mathcal{X}) \to \mathbb{R}$ is called *monotone* iff for any $A \subseteq B \subseteq \mathcal{X}$ it satisfies

$$F(A) \leq F(B). \tag{8.27}$$

If $F$ is also submodular, then $F$ is called *monotone submodular*.

**Theorem 8.11.** *The objective $I$ is monotone submodular.*

*Proof.* We fix arbitrary subsets $A \subseteq B \subseteq \mathcal{X}$ and any $\boldsymbol{x} \in \mathcal{X}$. We have,

$$
\begin{aligned}
I \text{ is submodular} &\iff \Delta_I(\boldsymbol{x} \mid A) \geq \Delta_I(\boldsymbol{x} \mid B) && \text{by submodularity in terms of marginal gain (8.26)} \\
&\iff \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{y}_A] - \mathrm{H}[\varepsilon_{\boldsymbol{x}}] \geq \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{y}_B] - \mathrm{H}[\varepsilon_{\boldsymbol{x}}] && \text{using Equation (8.24)} \\
&\iff \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{y}_A] \geq \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{y}_B]. && \mathrm{H}[\varepsilon_{\boldsymbol{x}}] \text{ cancels}
\end{aligned}
$$

Due to the "information never hurts" principle (8.8) of entropy and as $A \subseteq B$, this is always true. Moreover,

$$
\begin{aligned}
I \text{ is monotone} &\iff I(A) \leq I(B) && \text{by the definition of monotinicity (8.27)} \\
&\iff \mathrm{I}(\boldsymbol{f}_A; \boldsymbol{y}_A) \leq \mathrm{I}(\boldsymbol{f}_B; \boldsymbol{y}_B) && \text{using the definition of } I \text{ (8.20)} \\
&\iff \mathrm{I}(\boldsymbol{f}_B; \boldsymbol{y}_A) \leq \mathrm{I}(\boldsymbol{f}_B; \boldsymbol{y}_B) && \text{using } \mathrm{I}(\boldsymbol{f}_B; \boldsymbol{y}_A) = \mathrm{I}(\boldsymbol{f}_A; \boldsymbol{y}_A) \text{ as } \boldsymbol{y}_A \perp \boldsymbol{f}_B \mid \boldsymbol{f}_A \\
&\iff \mathrm{H}[\boldsymbol{f}_B] - \mathrm{H}[\boldsymbol{f}_B \mid \boldsymbol{y}_A] \leq \mathrm{H}[\boldsymbol{f}_B] - \mathrm{H}[\boldsymbol{f}_B \mid \boldsymbol{y}_B] && \text{using the definition of MI (8.9)} \\
&\iff \mathrm{H}[\boldsymbol{f}_B \mid \boldsymbol{y}_A] \geq \mathrm{H}[\boldsymbol{f}_B \mid \boldsymbol{y}_B], && \mathrm{H}[\boldsymbol{f}_B] \text{ cancels}
\end{aligned}
$$

which is also satisfied due to the "information never hurts" principle (8.8). □

The submodularity of $I$ can be interpreted from the perspective of information theory. It turns out that submodularity is equivalent to the absence of synergy between observations (?). Intuitively, without synergies, acting greedily is enough to find a near-optimal solution. If there are synergies, then the combinatorial search problem is much harder, because single-step optimal actions do not necessarily lead us to the optimal solution. Consider the extreme case of having to solve a "needle in a haystack" problem, where only a single subset of $\mathcal{X}$ with size $k$ achieves objective value 1, with all other subsets achieving objective value 0. In this case, we can do nothing but exhaustively search through all $|\mathcal{X}|^k$ combinations to find the optimal solution.

**Problem 8.5**

## 8.4 Maximizing Mutual Information

As we cannot efficiently pick a set $S \subseteq \mathcal{X}$ to maximize mutual information but know that $I$ is submodular, a natural approach is to maximize mutual information greedily. That is, we pick the locations $\boldsymbol{x}_1$ through $\boldsymbol{x}_n$ individually by greedily finding the location with the maximal mutual information. The following general result for monotone submodular function maximization shows that, indeed, this greedy approach provides a good approximation.

**Theorem 8.12** (Greedy submodular function maximization)**.** *If the set function $F : \mathcal{P}(\mathcal{X}) \to \mathbb{R}_{\geq 0}$ is monotone submodular, then greedily maximizing $F$ is a $(1 - 1/e)$-approximation:*[2]

[2] $1 - 1/e \approx 0.632$

$$F(S_n) \geq \left(1 - \frac{1}{e}\right) \max_{\substack{S \subseteq \mathcal{X} \\ |S| = n}} F(S). \tag{8.28}$$

*Proof.* Fix any $n \geq 1$. Let $S^\star \in \arg\max\{F(S) \mid S \in \mathcal{X}, |S| \leq n\}$. We can assume $|S^\star| = n$ due to the monotonicity (8.27) of $F$. We write, $\{\boldsymbol{x}_1^\star, \dots, \boldsymbol{x}_n^\star\} \doteq S^\star$. We have,

$$\begin{aligned}
F(S^\star) &\leq F(S^\star \cup S_t) && \text{using monotonicity (8.27)} \\
&= F(S_t) + \sum_{i=1}^{n} \Delta_F(\boldsymbol{x}_i^\star \mid S_t \cup \{\boldsymbol{x}_1^\star, \dots, \boldsymbol{x}_{i-1}^\star\}) && \text{using the definition of marginal gain (8.22)} \\
&\leq F(S_t) + \sum_{\boldsymbol{x}^\star \in S^\star} \Delta_F(\boldsymbol{x}^\star \mid S_t) && \text{using submodularity (8.26)} \\
&\leq F(S_t) + n(F(S_{t+1}) - F(S_t)). && \text{using that } S_{t+1} = S_t \cup \{\boldsymbol{x}\} \text{ is chosen such that } \Delta_F(\boldsymbol{x} \mid S_t) \text{ is maximized (8.29)}
\end{aligned}$$

Let $\delta_t \doteq F(S^\star) - F(S_t)$. Then,

$$\delta_t = F(S^\star) - F(S_t) \leq n(F(S_{t+1}) - F(S_t)) = n(\delta_t - \delta_{t+1}).$$

This implies $\delta_{t+1} \leq (1 - 1/n)\delta_t$ and $\delta_n \leq (1 - 1/n)^n \delta_0 \leq \delta_0/e$, using the well-known inequality $1 - x \leq e^{-x}$ for all $x \in \mathbb{R}$.

Finally, observe that $\delta_0 = F(S^\star) - F(\emptyset) \leq F(S^\star)$ due to the non-negativity of $F$. We obtain,

$$\delta_n = F(S^\star) - F(S_n) \leq \frac{\delta_0}{e} \leq \frac{F(S^\star)}{e}.$$

Rearranging the terms yields the theorem. □

**Optional Readings**

The original proof of greedy maximization for submodular functions was given by "An analysis of approximations for maximizing submodular set functions" (Nemhauser et al., 1978).

For more background on maximizing submodular functions, see "Submodular function maximization" (Krause and Golovin, 2014).

Now that we have established that greedy maximization of mutual information is a decent approximation to maximizing the joint information of data, we will look at how this optimization problem can be solved in practice.

### 8.4.1 Uncertainty Sampling

When maximizing mutual information, at time $t$ when we have already picked $S_t = \{\boldsymbol{x}_1, \dots, \boldsymbol{x}_t\}$, we need to solve the following optimization problem,

$$\boldsymbol{x}_{t+1} \doteq \arg\max_{\boldsymbol{x} \in \mathcal{X}} \Delta_I(\boldsymbol{x} \mid S_t) \tag{8.29}$$

$$= \arg\max_{\boldsymbol{x} \in \mathcal{X}} \mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{y}_{S_t}). \tag{8.30}$$

using Equation (8.23)

Note that $f_{\boldsymbol{x}}$ and $y_{\boldsymbol{x}}$ are univariate random variables. Thus, using our formula for the mutual information of conditional linear Gaussians (8.13), we can simplify to,

$$= \arg\max_{\boldsymbol{x} \in \mathcal{X}} \frac{1}{2} \log\left(1 + \frac{\sigma_t^2(\boldsymbol{x})}{\sigma_{\mathrm{n}}^2}\right) \tag{8.31}$$

where $\sigma_t^2(\boldsymbol{x})$ is the (remaining) variance at $\boldsymbol{x}$ after observing $S_t$. Assuming the label noise is independent of $\boldsymbol{x}$ (i.e., homoscedastic),

$$= \arg\max_{\boldsymbol{x} \in \mathcal{X}} \sigma_t^2(\boldsymbol{x}). \tag{8.32}$$

Therefore, if $f$ is modeled by a Gaussian and we assume homoscedastic noise, greedily maximizing mutual information corresponds to simply picking the point $\boldsymbol{x}$ with the largest variance. This algorithm is also called *uncertainty sampling*.

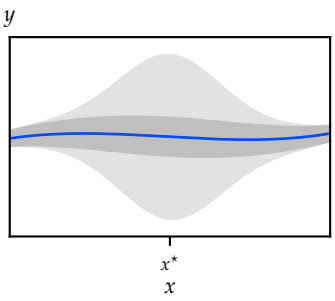



Figure 8.4: Uncertainty sampling with heteroscedastic noise. The epistemic uncertainty of the model is shown in a dark gray. The aleatoric uncertainty of the data is shown in a light gray. Uncertainty sampling would repeatedly pick points around $x^\star$ as they maximize the epistemic uncertainty, even though the aleatoric uncertainty at $x^\star$ is much larger than at the boundary.

### 8.4.2 Heteroscedastic Noise

Uncertainty sampling is clearly problematic if the noise is heteroscedastic. If there are a particular set of inputs with a large aleatoric uncertainty dominating the epistemic uncertainty, uncertainty sampling will continuously choose those points even though the epistemic uncertainty will not be reduced substantially (cf. Figure 8.4).

Looking at Equation (8.31) suggests a natural fix. Instead of only considering the epistemic uncertainty $\sigma_t^2(\boldsymbol{x})$, we can also consider the aleatoric uncertainty $\sigma_{\mathrm{n}}^2(\boldsymbol{x})$ by modeling heteroscedastic noise, yielding

$$\boldsymbol{x}_{t+1} \doteq \arg\max_{\boldsymbol{x} \in \mathcal{X}} \frac{1}{2} \log\left(1 + \frac{\sigma_t^2(\boldsymbol{x})}{\sigma_{\mathrm{n}}^2(\boldsymbol{x})}\right) = \arg\max_{\boldsymbol{x} \in \mathcal{X}} \frac{\sigma_t^2(\boldsymbol{x})}{\sigma_{\mathrm{n}}^2(\boldsymbol{x})}. \tag{8.33}$$

Thus, we choose locations that trade large epistemic uncertainty with large aleatoric uncertainty. Ideally, we find a location where the epistemic uncertainty is large, and the aleatoric uncertainty is low, which promises a significant reduction of uncertainty around this location.

### 8.4.3 *Classification*

While we focused on regression, one can apply active learning also for other settings, such as (probabilistic) classification. In this setting, for any input $\boldsymbol{x}$, a model produces a categorical distribution over labels $y_{\boldsymbol{x}}$.[3] Here, uncertainty sampling corresponds to selecting samples that maximize the entropy of the predicted label $y_{\boldsymbol{x}}$,

[3] see Section 1.3

$$\boldsymbol{x}_{t+1} \doteq \arg\max_{\boldsymbol{x} \in \mathcal{X}} \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{x}_{1:t}, y_{1:t}]. \tag{8.34}$$

The entropy of a categorical distribution is simply a finite sum of weighted surprise terms.

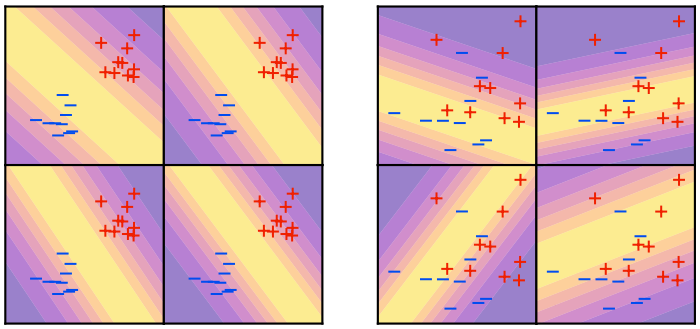


Figure 8.5: Uncertainty sampling in classification. The area with high uncertainty (as measured by entropy) is highlighted in yellow. The shown figures display each a sequence of model updates, each after one new observation. In the left figure, the classes are well-separated and uncertainty is dominated by epistemic uncertainty, whereas in the right figure the uncertainty is dominated by noise. In the latter case, if we mostly choose points $\boldsymbol{x}$ in the area of highest uncertainty (i.e., close to the decision boundary) to make observations, the label noise results in frequently changing models.

This approach generally leads to sampling points that are close to the decision boundary. Often, the uncertainty is mainly dominated by label noise rather than epistemic uncertainty, and hence, we do not learn much from our observations. This is a similar problem to the one we encountered with uncertainty sampling in the setting of heteroscedastic noise.

This naturally suggests distinguishing between the aleatoric and epistemic uncertainty of the model $f$ (parameterized by $\boldsymbol{\theta}$). To this end, mutual information can be used similarly as we have done with uncertainty sampling for regression,

$$\begin{aligned}
\boldsymbol{x}_{t+1} &\doteq \arg\max_{\boldsymbol{x} \in \mathcal{X}} \mathrm{I}(\boldsymbol{\theta}; y_{\boldsymbol{x}} \mid \boldsymbol{x}_{1:t}, y_{1:t}) && (8.35)\\
&= \arg\max_{\boldsymbol{x} \in \mathcal{X}} \mathrm{I}(y_{\boldsymbol{x}}; \boldsymbol{\theta} \mid \boldsymbol{x}_{1:t}, y_{1:t}) && \text{using symmetry (8.11)}\\
&= \arg\max_{\boldsymbol{x} \in \mathcal{X}} \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{x}_{1:t}, y_{1:t}] - \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{\theta}, \boldsymbol{x}_{1:t}, y_{1:t}] && \text{using the definition of mutual information (8.9)}\\
&= \arg\max_{\boldsymbol{x} \in \mathcal{X}} \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{x}_{1:t}, y_{1:t}] - \mathbb{E}_{\boldsymbol{\theta} \mid \boldsymbol{x}_{1:t}, y_{1:t}}[\mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{\theta}, \boldsymbol{x}_{1:t}, y_{1:t}]] && (8.36)\ \text{using the definition of conditional entropy (8.2)}\\
&= \arg\max_{\boldsymbol{x} \in \mathcal{X}} \underbrace{\mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{x}_{1:t}, y_{1:t}]}_{\substack{\text{entropy of}\\ \text{predictive posterior}}} - \mathbb{E}_{\boldsymbol{\theta} \mid \boldsymbol{x}_{1:t}, y_{1:t}} \underbrace{\mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{\theta}]}_{\substack{\text{entropy of}\\ \text{likelihood}}}. && (8.37)\ \text{using the definition of entropy (5.27) and assuming } y_{\boldsymbol{x}} \perp \boldsymbol{x}_{1:t}, y_{1:t} \mid \boldsymbol{\theta}
\end{aligned}$$

The first term measures the entropy of the averaged prediction while the second term measures the average entropy of predictions. Thus, the first term looks for points where the average prediction is not confident. In contrast, the second term penalizes points where many of the sampled models are not confident about their prediction, and thus looks for points where the models are confident in their predictions. This identifies those points $\boldsymbol{x}$ where the models *disagree* about the label $y_{\boldsymbol{x}}$ (that is, each model is "confident" but the models predict different labels). For this reason, this approach is known as *Bayesian active learning by disagreement* (BALD).

Note that the second term of the difference acts as a regularizer when compared to Equation (8.34). The second term mirrors our description of aleatoric uncertainty from Section 2.2. Recall that we interpreted aleatoric uncertainty as the average uncertainty for all models. Crucially, here we use entropy to "measure" uncertainty, whereas previously we have been using variance. Therefore, intuitively, Equation (8.36) subtracts the aleatoric uncertainty from the total uncertainty about the label.

Observe that both terms require approximate forms of the posterior distribution. In Chapters 5 and 6, we have seen various approaches from variational inference and MCMC methods which can be used here. The first term can be approximated by the predictive distribution of an approximated posterior which is obtained, for example, using variational inference. The nested entropy of the second term is typically easy to compute, as it corresponds to the entropy of the (discrete) likelihood distribution $p(y \mid \boldsymbol{x}, \boldsymbol{\theta})$ of the model $\boldsymbol{\theta}$. The outer expectation of the second term may be approximated using (approximate) samples from the posterior distribution obtained via variational inference, MCMC, or some other method.

**Optional Readings**

- Gal, Islam, and Ghahramani (2017).
  *Deep Bayesian active learning with image data*.

## 8.5 Learning Locally: Transductive Active Learning

So far we have explored how to select observations that provide us with the best predictor $f(\boldsymbol{x})$ across the entire domain $\boldsymbol{x} \in \mathcal{X}$. However, we typically utilize predictors to make predictions at a particular location $\boldsymbol{x}^\star$. It is therefore a natural question to ask how to select observations that lead to the best individual prediction $f(\boldsymbol{x}^\star)$ at the prediction target $\boldsymbol{x}^\star$. The distinction between these two settings is closely

related to the distinction between two general approaches to learning: *inductive learning* and *transductive learning*. Inductive learning aims to extract general rules from the data, that is, to extract and compress the most bits of information. In contrast, transductive learning aims to make the best prediction at a particular location, that is, to extract the most bits of information *relevant to the prediction* at $\boldsymbol{x}^\star$. The concept of transduction was developed by Vladimir Vapnik in the 1980s who described its essence as follows:

> "When solving a problem of interest, do not solve a more general problem as an intermediate step. Try to get the answer that you really need but not a more general one." — Vladimir Vapnik

**Remark 8.13: What are the prediction targets $x^\star$?**

Typically, in transductive learning, we cannot directly observe the value $f(\boldsymbol{x}^\star)$ at the prediction target $\boldsymbol{x}^\star$, for example, when learning from a fixed dataset $\mathcal{X}' \subset \mathcal{X}$. If we *could* observe $f(\boldsymbol{x}^\star)$ directly (or perturbed by noise), solving the learning task would only require memorization. Instead, most interesting learning tasks require generalizing $f(\boldsymbol{x}^\star)$ from the behavior of $f$ at other locations. Therefore, transductive learning becomes interesting precisely when we cannot directly observe $f(\boldsymbol{x}^\star)$.[4]

Note that this is similar to the inductive setting where we, in principle, could observe $f(\boldsymbol{x})$ at any location $\boldsymbol{x}$, but practically, we can only observe $f(\boldsymbol{x})$ at a finite number of locations. Since such an inductive model is then used to make predictions at any location $\boldsymbol{x}^\star$, it also needs to generalize from the observations.

[4] We will discuss an example of this kind in Problem 8.6.

Following the transductive paradigm, when already knowing that our goal is to predict $f(\boldsymbol{x}^\star)$ our objective is to select observations that provide the most information about $f(\boldsymbol{x}^\star)$. We can express this objective elegantly using the probabilistic framework from this chapter (MacKay, 1992; Hübotter et al., 2024):

$$\boldsymbol{x}_{t+1} \doteq \arg\max_{\boldsymbol{x} \in \mathcal{X}'} \mathrm{I}(f_{\boldsymbol{x}^\star}; y_{\boldsymbol{x}} \mid \boldsymbol{x}_{1:t}, y_{1:t}) = \arg\min_{\boldsymbol{x} \in \mathcal{X}'} \mathrm{H}[f_{\boldsymbol{x}^\star} \mid \boldsymbol{x}_{1:t}, y_{1:t}, y_{\boldsymbol{x}}]. \tag{8.38}$$

Hübotter et al. (2024) show that this objective leads to a remarkably different selection of observations compared to the inductive active learning objective we discussed earlier. Indeed, while the inductive objective focuses on selecting a *diverse* set of examples, the transductive objective also takes into account the *relevance* of the examples to the prediction target $\boldsymbol{x}^\star$. We can see this tradeoff between diversity and

relevance by rewriting the transductive objective as

$$\mathrm{I}(f_{x^\star}; y_x \mid \boldsymbol{x}_{1:t}, y_{1:t}) = \underbrace{\mathrm{I}(f_{x^\star}; y_x)}_{\text{relevance}} - \underbrace{\mathrm{I}(f_{x^\star}; y_x; y_{1:t} \mid \boldsymbol{x}_{1:t})}_{\text{redundancy}} \tag{8.39}$$

using the definition of interaction information (8.18)

where the first term measures the information gain of $y_x$ about $f_{x^\star}$ while the second term is the interaction information which measures the redundancy of the information in $y_x$ and $y_{1:t}$ about $f_{x^\star}$. In this way, transductive active learning describes a middle ground between traditional search and retrieval methods that focus on relevance on the one hand and inductive active learning which focuses on diversity on the other hand.

**Optional Readings**

- Hübotter, Sukhija, Treven, As, and Krause (2024). *Transductive active learning: Theory and applications*.

In modern machine learning, one often differentiates between a "pre-training" and a "fine-tuning" stage. During pre-training, a model is trained on a large dataset to extract general knowledge without a specific task in mind. Then, during fine-tuning, the model is adapted to a specific task by training on a smaller dataset. Whereas (inductive) active learning is closely linked to the pre-training stage, transductive active learning has been shown to be useful for task-specific fine-tuning:

- Hübotter, Bongni, Hakimi, and Krause (2025). *Efficiently Learning at Test-Time: Active Fine-Tuning of LLMs*.
- Bagatella, Hübotter, Martius, and Krause (2024). *Active Fine-Tuning of Generalist Policies*.

## *Discussion*

We have discussed how to select the most informative data. Thereby, we focused mostly on *inductive* active learning which is applicable to "pre-training" when we aim to extract general knowledge from data, but also explored *transductive* active learning which is useful for "fine-tuning" when we aim to adapt a model to a specific task.

We focused on quantifying the "informativeness" of data by its information gain, which is a common approach, though many other viable criteria exist:

**Remark 8.14: Beyond mutual information**

The problem of identifying which experiments to conduct in order to maximize learning is studied extensively in the field of *experimental design* where a set of observations $S$ is commonly called a *design*. In the presence of a prior and likelihood, and a different possible posterior distribution for each design $S$, the field is also called *Bayesian experimental design* (Chaloner and Verdinelli, 1995; Rainforth et al., 2024; Mutnỳ, 2024).

As we highlighted in the beginning of this chapter, how we measure the utility (i.e., the informativeness) of a design $S$ is crucial. Such a measure is called a *design criterion* and a design which is optimal with respect to a design criterion is called an *optimal design*. The literature studies various design criteria beyond maximizing mutual information (i.e., minimizing posterior entropy). One popular alternative is to select the observations $S$ that minimize the trace of the posterior covariance matrix, which corresponds to minimizing the posterior average variance.

Next, we will move to the topic of optimization and ask which data we should select to find the optimum of an unknown function as quickly as possible. In the following chapter, we will focus on "Bayesian optimization" (also called "bandit optimization") where our aim is to *find and sample* the optimal point. A related task that is slightly more related to active learning is the "best-arm identification" problem where we aim only to *identify* the optimal point without sampling it. This problem is closely related transductive active learning (with the local task being defined by the location of the maximum) and so-called *entropy search* methods that minimize the entropy of the posterior distribution over the location or value of the maximum (akin to Equation (8.38)) are often used to solve this problem (Hennig and Schuler, 2012; Wang and Jegelka, 2017; Hvarfner et al., 2022).

## *Problems*

**8.1. Mutual information and KL divergence.**

Show that by expanding the definition of mutual information,

$$\begin{aligned} \mathrm{I}(\mathbf{X}; \mathbf{Y}) &= \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim p}\left[\log \frac{p(\boldsymbol{x},\boldsymbol{y})}{p(\boldsymbol{x})p(\boldsymbol{y})}\right] \\ &= \mathrm{KL}(p(\boldsymbol{x},\boldsymbol{y}) \| p(\boldsymbol{x})p(\boldsymbol{y})) \qquad (8.40) \\ &= \mathbb{E}_{\boldsymbol{y}\sim p}[\mathrm{KL}(p(\boldsymbol{x} \mid \boldsymbol{y}) \| p(\boldsymbol{x}))], \qquad (8.41) \end{aligned}$$

where $p(\boldsymbol{x}, \boldsymbol{y})$ denotes the joint distribution and $p(\boldsymbol{x}), p(\boldsymbol{y})$ denote the marginal distributions of $\mathbf{X}$ and $\mathbf{Y}$.

**8.2. Non-monotonicity of cond. mutual information.**

1. Show that if $\mathbf{X} \perp \mathbf{Z}$ then $\mathrm{I}(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}) \geq \mathrm{I}(\mathbf{X}; \mathbf{Y})$.
2. Show that if $\mathbf{X} \perp \mathbf{Z} \mid \mathbf{Y}$ then $\mathrm{I}(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}) \leq \mathrm{I}(\mathbf{X}; \mathbf{Y})$.
3. Note that the condition $\mathbf{X} \perp \mathbf{Z} \mid \mathbf{Y}$ is the Markov property, namely, $\{\mathbf{X}, \mathbf{Y}, \mathbf{Z}\}$ form a Markov chain with graphical model $\mathbf{X} \to \mathbf{Y} \to \mathbf{Z}$. This situation often occurs when data is processed sequentially. Prove

$$\mathrm{I}(\mathbf{X}; \mathbf{Z}) \leq \mathrm{I}(\mathbf{X}; \mathbf{Y}). \tag{8.42}$$

   which is also known as the *data processing inequality*, and which says that processing cannot increase the information contained in a signal.

**8.3. Interaction information.**

1. Show that interaction information is symmetric.
2. Let $X_1, X_2 \sim \mathrm{Bern}(p)$ for some $p \in (0, 1)$ and independent. We denote by $Y \doteq X_1 \oplus X_2$ their XOR. Compute $\mathrm{I}(Y; X_1; X_2)$.

**8.4. Marginal gain of maximizing mutual information.**

Show that in the context of maximizing mutual information, the marginal gain is

$$\Delta_I(\boldsymbol{x} \mid A) = \mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{y}_A) = \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{y}_A] - \mathrm{H}[\varepsilon_{\boldsymbol{x}}].$$

**8.5. Submodularity means no synergy.**

Show that submodularity is equivalent to the absence of synergy between observations. That is, show that for all $A \subseteq B$,

$$\mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}}; \boldsymbol{y}_{B \setminus A} \mid \boldsymbol{y}_A) \geq 0. \tag{8.43}$$

**8.6. Transductive active learning.**

Consider the prior distribution $X_i \sim \mathcal{N}(0, 1)$ with all $X_i$ independent and consider the "output" variable

$$Z \doteq \sum_{i=1}^{100} i \cdot X_i.$$

Our observation when $X_{i_t}$ is selected in round $t$ is generated by

$$Y_t \doteq X_{i_t} + \varepsilon_t \quad \text{with } \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, 1).$$

For a set of inputs $S = \{i_1, \ldots, i_t\} \subseteq \{1, \ldots, 100\}$, we define the *information gain* of $S$ as

$$F(S) \doteq \mathrm{I}(Z; Y_{1:t}) = \mathrm{H}[Z] - \mathrm{H}[Z \mid Y_{1:t}]$$

where $\mathrm{H}[Z]$ is the entropy of $Z$ according to the prior and $\mathrm{H}[Z \mid Y_{1:t}]$ is the conditional entropy after the observations $Y_1, \ldots, Y_t$.

*Note: The random vector $(X_1, \ldots, X_{100}, Z)$ is jointly Gaussian due to the closedness of Gaussians under linear transformations.*

1. We define the *marginal information gain* $\Delta(j \mid S)$ for a new observation $Y_j$ as
$$\Delta(j \mid S) \doteq F(S \cup \{j\}) - F(S).$$
Does $\Delta(j \mid S) \geq 0$ hold for all $j$ and $S$?
2. Is maximizing the marginal information gain $\Delta(i \mid S)$ equivalent to picking the point $i \in \{1, \ldots, 100\}$ with maximum variance under the current posterior over $X_i$, that is, "equivalent to uncertainty sampling"?
3. Let us consider the alternative prior where $X_i \sim \mathrm{Bern}(0.5)$ are fair coin flips which we observe directly (i.e., $Y_t = X_{i_t}$). For which of the following definitions of $Z$ is the acquisition function $S \mapsto F(S)$ submodular?
   (a) $Z \doteq X_1 \wedge \cdots \wedge X_{100}$ with $\wedge$ denoting the logical AND.
   (b) $Z \doteq X_1 \vee \cdots \vee X_{100}$ with $\vee$ denoting the logical OR.
   (c) $Z \doteq X_1 \oplus \cdots \oplus X_{100}$ with $\oplus$ denoting the logical XOR, i.e., the exclusive OR which returns 1 iff exactly one of the $X_i$ is 1 and 0 otherwise.

# 9

# *Bayesian Optimization*

Often, obtaining data is costly. In the previous chapter, this led us to investigate how we can optimally improve our understanding (i.e., reduce uncertainty) of the process we are trying to model. However, purely improving our understanding is often not good enough. In many cases, we want to use our improving understanding *simultaneously* to reach certain goals. This is a very common problem in artificial intelligence and will concern us for the rest of this manuscript. One common instance of this problem is the setting of optimization.

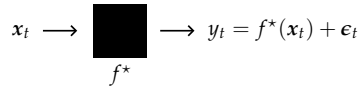



Figure 9.1: Illustration of Bayesian optimization. We pass an input $x_t$ into the unknown function $f^\star$ to obtain noisy observations $y_t$.

Given some function $f^\star : \mathcal{X} \to \mathbb{R}$, suppose we want to find the

$$\arg\max_{x \in \mathcal{X}} f^\star(\boldsymbol{x}). \tag{9.1}$$

Now, contrary to classical optimization, we are interested in the setting where the function $f^\star$ is unknown to us (like a "black-box"). We are only able to obtain noisy observations of $f^\star$,

$$y_t = f^\star(\boldsymbol{x}_t) + \varepsilon_t. \tag{9.2}$$

Moreover, these noise-perturbed evaluations are costly to obtain. We will assume that similar alternatives yield similar results,[1] which is commonly encoded by placing a Gaussian process prior on $f^\star$. This assumed correlation is fundamentally what will allow us to learn a model of $f^\star$ from relatively few samples.[2]

## 9.1 Exploration-Exploitation Dilemma

In Bayesian optimization, we want to learn a model of $f^\star$ and use this model to optimize $f^\star$ simultaneously. These goals are somewhat contrary. Learning a model of $f^\star$ requires us to explore the input space while using the model to optimize $f^\star$ requires us to focus on the most promising well-explored areas. This trade-off is commonly known as the *exploration-exploitation dilemma*, whereby

[1] That is, $f^\star$ is "smooth". We will be more precise in the subsequent parts of this chapter. If this were not the case, optimizing the function without evaluating it everywhere would not be possible. Fortunately, many interesting functions obey by this relatively weak assumption.

[2] There are countless examples of this problem in the "real world". Instances are

- drug trials
- chemical engineering — the development of physical products
- recommender systems
- automatic machine learning — automatic tuning of model & hyperparameters
- and many more...

- *exploration* refers to choosing points that are "informative" with respect to the unknown function. For example, points that are far away from previously observed points (i.e., have high posterior variance);[3] and
- *exploitation* refers to choosing promising points where we expect the function to have high values. For example, points that have a high posterior mean and a low posterior variance.

In other words, the exploration-exploitation dilemma refers to the challenge of learning enough to understand $f^\star$, but not learning too much to lose track of the objective — optimizing $f^\star$.

[3] We explored this topic (with strategies like uncertainty sampling) in the previous chapter.

The exploration-exploitation dilemma is yet another example of the *principle of curiosity and conformity* which we introduced in Section 5.5.2 and encountered many times since in our study of approximate probabilistic inference. We will see in subsequent chapters that sequential decision-making is intimately related to probabilistic inference, and there we will also make this correspondence more precise.

## 9.2 Online Learning and Bandits

Bayesian optimization is closely related to a form of online learning. In *online learning* we are given a set of possible inputs $\mathcal{X}$ and an unknown function $f^\star : \mathcal{X} \to \mathbb{R}$. We are now asked to choose a sequence of inputs $x_1, \dots x_T$ online,[4] and our goal is to maximize our cumulative reward $\sum_{t=1}^{T} f^\star(x_t)$. Depending on what we observe about $f^\star$, there are different variants of online learning. Bayesian optimization is closest to the so-called (stochastic) "bandit" setting.

[4] *Online* is best translated as "sequential". That is, we need to pick $x_{t+1}$ based only on our prior observations $y_1, \dots, y_t$.

### 9.2.1 Multi-Armed Bandits

The "*multi-armed bandits*" (MAB) problem is a classical, canonical formalization of the exploration-exploitation dilemma. In the MAB problem, we are provided with $k$ possible actions (arms) and want to maximize our reward online within the time horizon $T$. We do not know the reward distributions of the actions in advance, however, so we need to trade learning the reward distribution with following the most promising action. Bayesian optimization can be interpreted as a variant of the MAB problem where there can be a potentially infinite number of actions (arms), but their rewards are correlated (because of the smoothness of the Gaussian process prior).

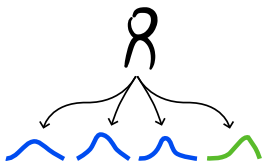


Figure 9.2: Illustration of a multi-armed bandit with four arms, each with a different reward distribution. The agent tries to identify the arm with the most beneficial reward distribution shown in green.

There exists a large body of work on this and similar problems in online decision-making. Much of this work develops theory on how to explore and exploit in the face of uncertainty. The shared prevalence of the exploration-exploitation dilemma signals a deep connection be-

tween online learning and Bayesian optimization (and — as we will later come to see — reinforcement learning). Many of the approaches which we will encounter in the context of these topics are strongly related to methods in online learning.

One of the key principles of the theory on multi-armed bandits and reinforcement learning is the principle of *"optimism in the face of uncertainty"*, which suggests that it is a good guideline to explore where we can hope for the best outcomes. We will frequently come back to this general principle in our discussion of algorithms for Bayesian optimization and reinforcement learning.

### 9.2.2 *Regret*

The key performance metric in online learning is the regret.

**Definition 9.1** (Regret)**.** The *(cumulative) regret* for a time horizon $T$ associated with choices $\{\boldsymbol{x}_t\}_{t=1}^T$ is defined as

$$R_T \doteq \sum_{t=1}^{T} \underbrace{\left(\max_{\boldsymbol{x}} f^\star(\boldsymbol{x}) - f^\star(\boldsymbol{x}_t)\right)}_{\textit{instantaneous regret}} \tag{9.3}$$

$$= T \max_{\boldsymbol{x}} f^\star(\boldsymbol{x}) - \sum_{t=1}^{T} f^\star(\boldsymbol{x}_t). \tag{9.4}$$

The regret can be interpreted as the additive loss with respect to the *static* optimum $\max_{\boldsymbol{x}} f^\star(\boldsymbol{x})$.

The goal is to find algorithms that achieve *sublinear* regret,

$$\lim_{T\to\infty} \frac{R_T}{T} = 0. \tag{9.5}$$

Importantly, if we use an algorithm which explores *forever*, e.g., by going to a random point $\tilde{\boldsymbol{x}}$ with a constant probability $\epsilon$ in each round, then the regret will grow linearly with time. This is because the instantaneous regret is at least $\epsilon(\max_{\boldsymbol{x}} f^\star(\boldsymbol{x}) - f^\star(\tilde{\boldsymbol{x}}))$ and non-decreasing. Conversely, if we use an algorithm which *never* explores, then we might never find the static optimum, and hence, also incur constant instantaneous regret in each round, implying that regret grows linearly with time. Thus, achieving sublinear regret requires *balancing* exploration and exploitation.

Typically, online learning (and Bayesian optimization) consider stationary environments, hence the comparison to the static optimum. Dynamic environments are studied in *online algorithms* (see metrical task systems[5], convex function chasing[6], and generalizations of multi-armed bandits to changing reward distributions) and reinforcement

[5] Metrical task systems are a classical example in online algorithms. Suppose we are moving in a (finite) decision space $\mathcal{X}$. In each round, we are given a "task" $f_t : \mathcal{X} \to \mathbb{R}$ which is more or less costly depending on our state $\boldsymbol{x}_t \in \mathcal{X}$. In many contexts, it is natural to assume that it is also costly to move around in the decision space. This cost is modeled by a metric $d(\cdot,\cdot)$ on $\mathcal{X}$. In *metrical task systems*, we want to minimize our total cost,

$$\sum_{t=1}^{T} f_t(\boldsymbol{x}_t) + d(\boldsymbol{x}_t, \boldsymbol{x}_{t-1}).$$

That is, we want to trade completing our tasks optimally with moving around in the state space. Crucially, we do not know the sequence of tasks $f_t$ in advance. Due to the cost associated with moving in the decision space, previous choices affect the future!

[6] *Convex function chasing* (or *convex body chasing*) generalize metrical task systems to continuous domains $\mathcal{X}$. To make any guarantees about the performance in these settings, one typically has to assume that the tasks $f_t$ are convex. Note that this mirrors our assumption in Bayesian optimization that similar alternatives yield similar results.

learning. When operating in dynamic environments, other metrics such as the competitive ratio,[7] which compares against the best *dynamic* choice, are useful. As we will later come to see in Section 13.1 in the context of reinforcement learning, operating in dynamic environments is deeply connected to a rich field of research called *control*.

[7] To assess the performance in dynamic environments, we typically compare to a dynamic optimum. As these problems are difficult (we are usually not able to guarantee convergence to the dynamic optimum), one considers a multiplicative performance metric similar to the approximation ratio, the *competitive ratio*,

$$\text{cost}(\text{ALG}) \leq \alpha \cdot \text{cost}(\text{OPT}),$$

where OPT corresponds to the dynamic optimal choice (in hindsight).

## 9.3 Acquisition Functions

It is common to use a so-called *acquisition function* to greedily pick the next point to sample based on the current model.

Throughout our description of acquisition functions, we will focus on a setting where we model $f^\star$ using a Gaussian process which we denote by $f$. The methods generalize to other means of learning $f^\star$ such as Bayesian deep learning. The various acquisition functions $F$ are used in the same way as is illustrated in Algorithm 9.2.

**Algorithm 9.2:** Bayesian optimization (with GPs)

initialize $f \sim \mathcal{GP}(\mu_0, k_0)$
**for** $t = 1$ **to** $T$ **do**
  choose $\boldsymbol{x}_t = \arg\max_{\boldsymbol{x} \in \mathcal{X}} F(\boldsymbol{x}; \mu_{t-1}, k_{t-1})$
  observe $y_t = f(\boldsymbol{x}_t) + \epsilon_t$
  perform a probabilistic update to obtain $\mu_t$ and $k_t$

**Remark 9.3: Model selection**

Selecting a model of $f^\star$ in sequential decision-making is much harder than in the i.i.d. data setting of supervised learning. There are mainly the following two dangers:

- the data sets collected in active learning and Bayesian optimization are *small*; and
- the data points are selected *dependently* on prior observations.

This leads to a specific danger of overfitting. In particular, due to feedback loops between the model and the acquisition function, one may end up sampling the same point repeatedly.

One approach to reduce the chance of overfitting is the use of hyperpriors which we mentioned previously in Section 4.4.2. Another approach that often works fairly well is to occasionally (according to some schedule) select points uniformly at random instead of using the acquisition function. This tends to prevent getting stuck in suboptimal parts of the state space.

One possible acquisition function is uncertainty sampling (8.31), which we discussed in the previous chapter. However, this acquisition function does not at all take into account the objective of maximizing $f^\star$ and focuses solely on exploration.

Suppose that our model $f$ of $f^\star$ is well-calibrated, in the sense that the true function lies within its confidence bounds. Consider the best lower bound, that is, the maximum of the lower confidence bound. Now, if the true function is really contained in the confidence bounds, it must hold that the optimum is somewhere above this best lower bound. In particular, we can exclude all regions of the domain where the upper confidence bound (the optimistic estimate of the function value) is lower than the best lower bound. This is visualized in Figure 9.3.

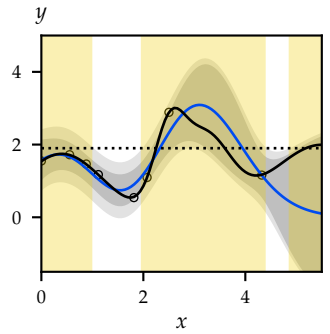



Figure 9.3: Optimism in Bayesian optimization. The **unknown function** is shown in black, our **model** in blue with gray confidence bounds. The dotted black line denotes the maximum lower bound. We can therefore focus our exploration to the yellow regions where the upper confidence bound is higher than the maximum lower bound.

Therefore, we only really care how the function looks like in the regions where the upper confidence bound is larger than the best lower bound. The key idea behind the methods that we will explore is to focus exploration on these plausible maximizers.

Note that it is crucial that our uncertainty about $f$ reflects the "fit" of our model to the unknown function. If the model is not well calibrated or does not describe the underlying function at all, these methods will perform poorly. This is where we can use the Bayesian philosophy by imposing a prior belief that may be conservative.

### 9.3.1 *Upper Confidence Bound*

The principle of *optimism in the face of uncertainty* suggests picking the point where we can hope for the optimal outcome. In this setting, this corresponds to simply maximizing the *upper confidence bound* (UCB),

$$\boldsymbol{x}_{t+1} \doteq \arg\max_{\boldsymbol{x}\in\mathcal{X}} \mu_t(\boldsymbol{x}) + \beta_{t+1}\sigma_t(\boldsymbol{x}), \tag{9.6}$$

where $\sigma_t(\boldsymbol{x}) \doteq \sqrt{k_t(\boldsymbol{x},\boldsymbol{x})}$ is the standard deviation at $\boldsymbol{x}$ and $\beta_t$ regulates how confident we are about our model $f$ (i.e., the choice of confidence interval).

This acquisition function naturally trades exploitation by preferring a large posterior mean with exploration by preferring a large posterior variance. Note that if $\beta_t = 0$ then UCB is purely exploitative, whereas, if $\beta_t \to \infty$, UCB recovers uncertainty sampling (i.e., is purely explorative).[8] UCB is an example of an optimism-based method, as it greedily picks the point where we can hope for the best outcome.

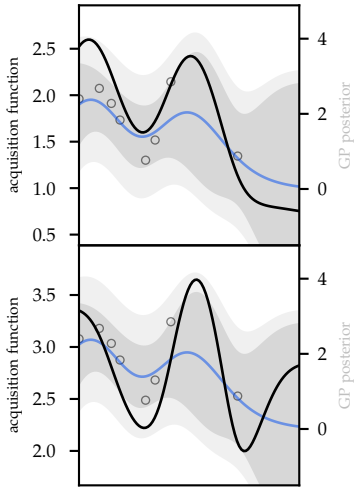



Figure 9.4: Plot of the UCB acquisition function for $\beta = 0.25$ and $\beta = 1$, respectively.

As can be seen in Figure 9.4, the UCB acquisition function is generally non-convex. For selecting the next point, we can use approximate

[8] Due to the monotonicity of $(\cdot)^2$, it does not matter whether we optimize the variance or standard deviation at $\boldsymbol{x}$.

global optimization techniques like Lipschitz optimization (in low dimensions) and gradient ascent with random initialization (in high dimensions). Another widely used technique is to sample some random points from the domain, score them according to this criterion, and simply take the best one.

The choice of $\beta_t$ is crucial for the performance of UCB. Intuitively, for UCB to work even if the unknown function $f^\star$ is not contained in the confidence bounds, we use $\beta_t$ to re-scale the confidence bounds to enclose $f^\star$ as shown in Figure 9.5. A theoretical analysis requires that $\beta_t$ is chosen "correctly". Formally, we say that the sequence $\beta_t$ is chosen correctly if it leads to *well-calibrated confidence intervals*, that is, if with probability at least $1-\delta$,

$$\forall t \geq 1,\ \forall \boldsymbol{x} \in \mathcal{X}: \quad f^\star(\boldsymbol{x}) \in \mathcal{C}_t(\boldsymbol{x}) \doteq [\mu_{t-1}(\boldsymbol{x}) \pm \beta_t(\delta) \cdot \sigma_{t-1}(\boldsymbol{x})]. \tag{9.7}$$

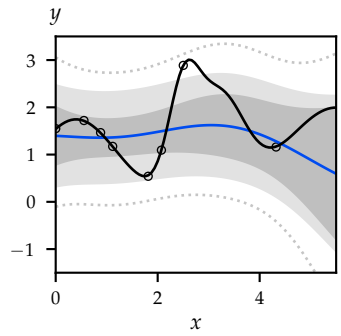



Figure 9.5: Re-scaling the confidence bounds. The dotted gray lines represent updated confidence bounds.

Bounds on $\beta_t(\delta)$ can be derived both in a "Bayesian" and in a "frequentist" setting. In the Bayesian setting, it is assumed that $f^\star$ is drawn from the prior GP, i.e., $f^\star \sim \mathcal{GP}(\mu_0, k_0)$. However, in many cases this may be an unrealistic assumption. In the frequentist setting, it is assumed instead that $f^\star$ is a fixed element of a reproducing kernel Hilbert space $\mathcal{H}_k(\mathcal{X})$ which depending on the kernel $k$ can encompass a large class of functions. We will discuss the Bayesian setting first and later return to the frequentist setting.

**Theorem 9.4** (Bayesian confidence intervals, lemma 5.5 of Srinivas et al. (2010))**.** (?) *Let $\delta \in (0,1)$. Assuming $f^\star \sim \mathcal{GP}(\mu_0, k_0)$ and Gaussian observation noise $\epsilon_t \sim \mathcal{N}(0, \sigma_\mathrm{n}^2)$, the sequence*

$$\beta_t(\delta) = O\left(\sqrt{\log(|\mathcal{X}|t/\delta)}\right) \tag{9.8}$$

*satisfies $\mathbb{P}(\forall t \geq 1, \boldsymbol{x} \in \mathcal{X} : f^\star(\boldsymbol{x}) \in \mathcal{C}_t(\boldsymbol{x})) \geq 1-\delta$.*

**Problem 9.2**

Under the assumption of well-calibrated confidence intervals, we can bound the regret of UCB.

**Theorem 9.5** (Regret of GP-UCB, theorem 2 of Srinivas et al. (2010))**.** (?) *If $\beta_t(\delta)$ is chosen "correctly" for a fixed $\delta \in (0,1)$, with probability at least $1-\delta$, greedily choosing the upper confidence bound yields cumulative regret*

$$R_T = O\left(\beta_T(\delta)\sqrt{\gamma_T T}\right) \tag{9.9}$$

*where*

$$\gamma_T \doteq \max_{\substack{S \subseteq \mathcal{X} \\ |S|=T}} \mathrm{I}(\boldsymbol{f}_S; \boldsymbol{y}_S) = \max_{\substack{S \subseteq \mathcal{X} \\ |S|=T}} \frac{1}{2} \log\det\left(\boldsymbol{I} + \sigma_\mathrm{n}^{-2} \boldsymbol{K}_{SS}\right) \tag{9.10}$$

*is the maximum information gain after $T$ rounds.*

**Problem 9.3**

Observe that if the information gain is sublinear in $T$ then we achieve sublinear regret and, in particular, converge to the true optimum. The information gain $\gamma_T$ measures how much can be learned about $f^\star$ within $T$ rounds. If the function is assumed to be smooth (perhaps even linear), then the information gain is smaller than if the function was assumed to be "rough". Intuitively, the smoother the functions encoded by the prior, the smaller is the class of functions to choose from and the more can be learned from a single observation about "neighboring" points.

**Theorem 9.6** (Information gain of common kernels, theorem 5 of Srinivas et al. (2010) and remark 2 of Vakili et al. (2021))**.** *Due to submodularity, we have the following bounds on the information gain of common kernels:*

- linear kernel

$$\gamma_T = O(d \log T), \tag{9.11}$$

- Gaussian kernel

$$\gamma_T = O\Big((\log T)^{d+1}\Big), \tag{9.12}$$

- Matérn kernel *for* $\nu > \frac{1}{2}$

$$\gamma_T = O\Big(T^{\frac{d}{2\nu+d}} (\log T)^{\frac{2\nu}{2\nu+d}}\Big). \tag{9.13}$$

The information gain of common kernels is illustrated in Figure 9.6. Notably, when all points in the domain are independent, the information gain is linear in $T$. This is because when the function $f^\star$ may be arbitrarily "rough", we cannot generalize from a single observation to "neighboring" points, and as there are infinitely many points in the domain $\mathcal{X}$ there are no diminishing returns. As one would expect, in this case, Theorem 9.5 does not yield sublinear regret. However, we can see from Theorem 9.6 that the information gain is sublinear for linear, Gaussian, and most Matérn kernels. Moreover, observe that unless the function is linear, the information gain grows exponentially with the dimension $d$. This is because the number of "neighboring" points (with respect to Euclidean geometry) decreases exponentially with the dimension which is also known as the *curse of dimensionality*.

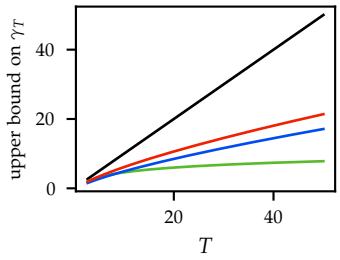



Figure 9.6: Information gain of **independent**, **linear**, **Gaussian**, and **Matérn** ($\nu \approx 0.5$) kernels with $d = 2$ (up to constant factors). The kernels with sublinear information gain have strong diminishing returns (due to their strong dependence between "close" points). In contrast, the independent kernel has no dependence between points in the domain, and therefore no diminishing returns. Intuitively, the "smoother" the class of functions modeled by the kernel, the stronger are the diminishing returns.

As mentioned, the size of the confidence intervals can also be analyzed under a frequentist assumption on $f^\star$.

**Theorem 9.7** (Frequentist confidence intervals, theorem 2 of Chowdhury and Gopalan (2017))**.** *Let* $\delta \in (0, 1)$*. Assuming* $f^\star \in \mathcal{H}_k(\mathcal{X})$*, we have that with probability at least* $1 - \delta$*, the sequence*

$$\beta_t(\delta) = \|f^\star\|_k + \sigma_n \sqrt{2(\gamma_t + \log(1/\delta))} \tag{9.14}$$

*satisfies* $\mathbb{P}(\forall t \geq 1, \boldsymbol{x} \in \mathcal{X} : f^\star(\boldsymbol{x}) \in \mathcal{C}_t(\boldsymbol{x})) \geq 1 - \delta$.

That is, $\beta_t$ depends on the information gain of the kernel as well as on the "complexity" of $f^\star$ which is measured in terms of its norm in the underlying reproducing kernel Hilbert space $\mathcal{H}_k(\mathcal{X})$.

**Remark 9.8: Bayesian vs frequentist assumption**

Theorems 9.4 and 9.7 provide different bounds on $\beta_t(\delta)$ based on fundamentally different assumptions on the ground truth $f^\star$: The Bayesian assumption is that $f^\star$ is drawn from the prior GP, whereas the frequentist assumption is that $f^\star$ is an element of a reproducing kernel Hilbert space $\mathcal{H}_k(\mathcal{X})$. The frequentist assumption holds uniformly for all functions $f^\star$ with $\|f^\star\|_k < \infty$, whereas the Bayesian assumption holds only under the Bayesian "belief" that $f^\star$ is drawn from the prior GP.

Interestingly, neither assumption encompasses the other. This is because if $f \sim \mathcal{GP}(0, k)$ then it can be shown that almost surely $\|f\|_k = \infty$, which implies that $f \notin \mathcal{H}_k(\mathcal{X})$ (Srinivas et al., 2010).

We remark that Theorem 9.7 holds also under the looser assumption that observations are perturbed by $\sigma_{\mathrm{n}}$-sub-Gaussian noise (cf. Equation (A.39)) instead of Gaussian noise. The bound on $\gamma_T$ from Equation (9.13) for the Matérn kernel does not yield sublinear regret when combined with the standard regret bound from Theorem 9.5, however, Whitehouse et al. (2024) show that the regret of GP-UCB is sublinear also in this case provided $\sigma_{\mathrm{n}}^2$ is chosen carefully.

This concludes our discussion of the UCB algorithm. We have seen that its regret can be analyzed under both Bayesian and frequentist assumptions on $f^\star$.

### 9.3.2 *Improvement*

Another well-known family of methods is based on keeping track of a running optimum $\hat{f}_t$, and scoring points according to their improvement upon the running optimum. The *improvement* of $\boldsymbol{x}$ after round $t$ is measured by

$$I_t(\boldsymbol{x}) \doteq (f(\boldsymbol{x}) - \hat{f}_t)_+ \tag{9.15}$$

where we use $(\cdot)_+$ to denote $\max\{0, \cdot\}$.

The *probability of improvement* (PI) picks the point that maximizes the probability to improve upon the running optimum,

$$\begin{aligned} \boldsymbol{x}_{t+1} &\doteq \arg\max_{x \in \mathcal{X}} \mathbb{P}(I_t(\boldsymbol{x}) > 0 \mid \boldsymbol{x}_{1:t}, y_{1:t}) && (9.16) \\ &= \arg\max_{x \in \mathcal{X}} \mathbb{P}(f(\boldsymbol{x}) > \hat{f}_t \mid \boldsymbol{x}_{1:t}, y_{1:t}) && (9.17) \end{aligned}$$

$$= \arg\max_{x \in \mathcal{X}} \Phi\left(\frac{\mu_t(x) - \hat{f}_t}{\sigma_t(x)}\right) \tag{9.18}$$

using linear transformations of Gaussians (1.78)

where $\Phi$ denotes the CDF of the standard normal distribution and we use that $f(x) \mid x_{1:t}, y_{1:t} \sim \mathcal{N}(\mu_t(x), \sigma_t^2(x))$. Probability of improvement tends to be biased in favor of exploitation, as it prefers points with large posterior mean and small posterior variance which is typically true "close" to the previously observed maximum $\hat{f}_t$.

Probability of improvement looks at *how likely* a point is to improve upon the running optimum. An alternative is to look at *how much* a point is expected to improve upon the running optimum. This acquisition function is called the *expected improvement* (EI),

$$x_{t+1} \doteq \arg\max_{x \in \mathcal{X}} \mathbb{E}[I_t(x) \mid x_{1:t}, y_{1:t}]\,. \tag{9.19}$$

Intuitively, EI seeks a large expected improvement (exploitation) while also preferring states with a large variance (exploration). Expected improvement yields the same regret bound as UCB (Nguyen et al., 2017).

The expected improvement acquisition function is often flat which makes it difficult to optimize in practice due to vanishing gradients. One approach addressing this is to instead optimize the logarithm of EI (Ament et al., 2024).

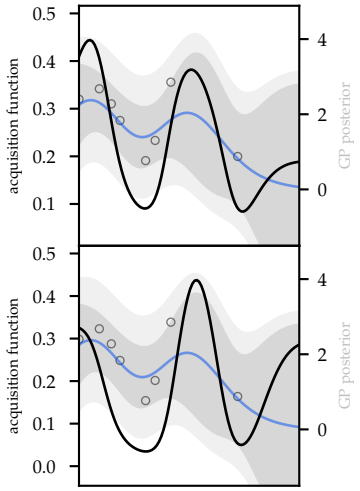



Figure 9.7: Plot of the PI and EI acquisition functions, respectively.

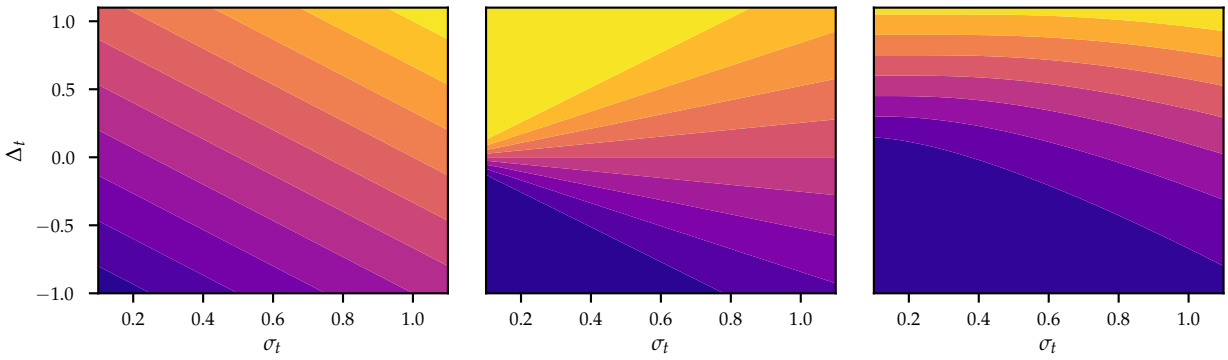



Figure 9.8: Contour lines of acquisition functions for varying $\Delta_t = \mu_t(x) - \hat{f}_t$ and $\sigma_t$. A brighter color corresponds to a larger acquisition function value. The first graph shows contour lines of UCB with $\beta_t = 0.75$, the second of PI, and the third of EI.

### 9.3.3 *Thompson Sampling*

We can also interpret the principle of *optimism in the face of uncertainty* in a slightly different way than we did with UCB (and EI). Suppose we select the next point according to the probability that it is optimal (assuming that the posterior distribution is an accurate representation of the uncertainty),

$$\pi(x \mid x_{1:t}, y_{1:t}) \doteq \mathbb{P}_{f \mid x_{1:t}, y_{1:t}}\left(f(x) = \max_{x'} f(x')\right) \tag{9.20}$$

$$\boldsymbol{x}_{t+1} \sim \pi(\cdot \mid \boldsymbol{x}_{1:t}, y_{1:t}). \tag{9.21}$$

This approach of sampling according to the *probability of maximality* $\pi$ is called *probability matching*. Probability matching is exploratory as it prefers points with larger variance (as they automatically have a larger chance of being optimal), but at the same time exploitative as it effectively discards points with low posterior mean and low posterior variance. Unfortunately, it is generally difficult to compute $\pi$ analytically given a posterior.

Instead, it is common to use a sampling-based approximation of $\pi$. Observe that the density $\pi$ can be expressed as an expectation,

$$\pi(\boldsymbol{x} \mid \boldsymbol{x}_{1:t}, y_{1:t}) = \mathbb{E}_{f \mid \boldsymbol{x}_{1:t}, y_{1:t}}\left[\mathbb{1}\{f(\boldsymbol{x}) = \max_{\boldsymbol{x}'} f(\boldsymbol{x}')\}\right], \tag{9.22}$$

which we can approximate using Monte Carlo sampling (typically using a single sample),

$$\approx \mathbb{1}\{\tilde{f}_{t+1}(\boldsymbol{x}) = \max_{\boldsymbol{x}'} \tilde{f}_{t+1}(\boldsymbol{x}')\} \tag{9.23}$$

where $\tilde{f}_{t+1} \sim p(\cdot \mid \boldsymbol{x}_{1:t}, y_{1:t})$ is a sample from our posterior distribution. Observe that this approximation of $\pi$ coincides with a point density at the maximizer of $\tilde{f}_{t+1}$.

The resulting algorithm is known as *Thompson sampling*. At time $t+1$, we sample a function $\tilde{f}_{t+1} \sim p(\cdot \mid \boldsymbol{x}_{1:t}, y_{1:t})$ from our posterior distribution. Then, we simply maximize $\tilde{f}_{t+1}$,

$$\boldsymbol{x}_{t+1} \doteq \arg\max_{\boldsymbol{x} \in \mathcal{X}} \tilde{f}_{t+1}(\boldsymbol{x}). \tag{9.24}$$

In many cases, the randomness in the realizations of $\tilde{f}_{t+1}$ is already sufficient to effectively trade exploration and exploitation. Similar regret bounds to those of UCB can also be established for Thompson sampling (Russo and Van Roy, 2016; Kandasamy et al., 2018).

### 9.3.4 *Information-Directed Sampling*

After having looked at multiple methods that aim to balance exploitation of the current posterior distribution over $f$ for immediate returns with exploration to reduce uncertainty about $f$ for future returns, we will next discuss a method that makes this tradeoff explicit.

Denoting the instantaneous regret of choosing $\boldsymbol{x}$ as $\Delta(\boldsymbol{x}) \doteq \max_{\boldsymbol{x}'} f^\star(\boldsymbol{x}') - f^\star(\boldsymbol{x})$ and by $I_t(\boldsymbol{x})$ some function capturing the "information gain" associated with observing $\boldsymbol{x}$ in iteration $t+1$, we can define the *information ratio*,

$$\Psi_t(\boldsymbol{x}) \doteq \frac{\Delta(\boldsymbol{x})^2}{I_t(\boldsymbol{x})}, \tag{9.25}$$

which was originally introduced by Russo and Van Roy (2016). Here exploitation reduces regret while exploration increases information gain, and hence, points $\boldsymbol{x}$ that minimize the information ratio are those that most effectively balance exploration and exploitation. We can make the key observation that the regret $\Delta(\cdot)$ decreases when $I_t(\cdot)$ decreases, as a small $I_t(\cdot)$ implies that the algorithm has already learned a lot about the function $f^\star$. The strength of this relationship is quantified by the information ratio:

**Theorem 9.9** (Proposition 1 of Russo and Van Roy (2014) and theorem 8 of Kirschner and Krause (2018))**.** *For any iteration $T \geq 1$, let $\sum_{t=1}^{T} I_{t-1}(\boldsymbol{x}_t) \leq \gamma_T$ and suppose that $\Psi_{t-1}(\boldsymbol{x}_t) \leq \overline{\Psi}_T$ for all $t \in [T]$. Then, the cumulative regret is bounded by*

$$R_T \leq \sqrt{\gamma_T \overline{\Psi}_T T}. \tag{9.26}$$

*Proof.* By Equation (9.25), $r_t = \Delta(\boldsymbol{x}_t) = \sqrt{\Psi_{t-1}(\boldsymbol{x}_t) \cdot I_{t-1}(\boldsymbol{x}_t)}$. Hence,

$$\begin{aligned}
R_T &= \sum_{t=1}^{T} r_t \\
&= \sum_{t=1}^{T} \sqrt{\Psi_{t-1}(\boldsymbol{x}_t) \cdot I_{t-1}(\boldsymbol{x}_t)} \\
&\leq \sqrt{\sum_{t=1}^{T} \Psi_{t-1}(\boldsymbol{x}_t) \cdot \sum_{t=1}^{T} I_{t-1}(\boldsymbol{x}_t)} && \text{using the Cauchy-Schwarz inequality} \\
&\leq \sqrt{\gamma_T \overline{\Psi}_T T}. && \text{using the assumptions on } I_t(\cdot) \text{ and } \Psi_t(\cdot)
\end{aligned}$$

□

**Example 9.10: How to measure "information gain"?**

One possibility of measuring the "information gain" is

$$I_t(\boldsymbol{x}) \doteq \mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{x}_{1:t}, y_{1:t}) \tag{9.27}$$

which — as you may recall — is precisely the marginal gain of the utility $I(S) = \mathrm{I}(\boldsymbol{f}_S; \boldsymbol{y}_S)$ we were studying in Chapter 8. In this case,

$$\begin{aligned}
\sum_{t=1}^{T} I_{t-1}(\boldsymbol{x}_t) &= \sum_{t=1}^{T} \mathrm{I}(f_{\boldsymbol{x}_t}; y_{\boldsymbol{x}_t} \mid \boldsymbol{x}_{1:t-1}, y_{1:t-1}) \\
&= \sum_{t=1}^{T} \mathrm{I}(\boldsymbol{f}_{\boldsymbol{x}_{1:T}}; y_{\boldsymbol{x}_t} \mid \boldsymbol{x}_{1:t-1}, y_{1:t-1}) && \text{using } \mathrm{I}(\mathbf{X}, \mathbf{Z}; \mathbf{Y}) \geq \mathrm{I}(\mathbf{X}; \mathbf{Y}) \text{ which follows from Equations (8.12) and (8.16) and is called } \textit{monotonicity of MI} \\
&= \mathrm{I}(\boldsymbol{f}_{\boldsymbol{x}_{1:T}}; \boldsymbol{y}_{\boldsymbol{x}_{1:T}}) && \text{by repeated application of Equation (8.16), also called the } \textit{chain rule of MI} \\
&\leq \gamma_T. && \text{by definition of } \gamma_T \text{ (9.10)}
\end{aligned}$$

The regret bound from Theorem 9.9 suggests an algorithm which in each iteration chooses the point which minimizes the information ratio (9.25). However, this is not possible since $\Delta(\cdot)$ is unknown due to its dependence on $f^\star$. Kirschner and Krause (2018) propose to use a surrogate to the regret which is based on the current model of $f^\star$,

$$\hat{\Delta}_t(\boldsymbol{x}) \doteq \max_{\boldsymbol{x}' \in \mathcal{X}} u_t(\boldsymbol{x}') - l_t(\boldsymbol{x}). \tag{9.28}$$

Here, $u_t(\boldsymbol{x}) \doteq \mu_t(\boldsymbol{x}) + \beta_{t+1}\sigma_t(\boldsymbol{x})$ and $l_t(\boldsymbol{x}) \doteq \mu_t(\boldsymbol{x}) - \beta_{t+1}\sigma_t(\boldsymbol{x})$ are the upper and lower confidence bounds of the confidence interval $\mathcal{C}_t(\boldsymbol{x})$ of $f^\star(\boldsymbol{x})$, respectively. Similarly to our discussion of UCB, we make the assumption that the sequence $\beta_t$ is chosen "correctly" (cf. Equation (9.7)) so that the confidence interval is well-calibrated and $\Delta(\boldsymbol{x}) \leq \hat{\Delta}_t(\boldsymbol{x})$ with high probability. The resulting algorithm

$$\boldsymbol{x}_{t+1} \doteq \arg\min_{\boldsymbol{x} \in \mathcal{X}} \left\{ \widehat{\Psi}_t(\boldsymbol{x}) \doteq \frac{\hat{\Delta}_t(\boldsymbol{x})^2}{I_t(\boldsymbol{x})} \right\} \tag{9.29}$$

is known as *information-directed sampling* (IDS).

**Theorem 9.11** (Regret of IDS, lemma 8 of Kirschner and Krause (2018))**.** (?) *Let $\beta_t(\delta)$ be chosen "correctly" for a fixed $\delta \in (0, 1)$. Then, if the measure of information gain is $I_t(\boldsymbol{x}) = \mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{x}_{1:t}, y_{1:t})$, with probability at least $1 - \delta$, IDS has cumulative regret*

$$R_T = O\left(\beta_T(\delta)\sqrt{\gamma_T T}\right). \tag{9.30}$$

Regret bounds such as Theorem 9.11 can be derived also for different measures of information gain. For example, the argument of Problem 9.6 also goes through for the "greedy" measure

$$I_t(\boldsymbol{x}) \doteq \mathrm{I}(f_{\boldsymbol{x}_t^{\mathrm{UCB}}}; y_{\boldsymbol{x}} \mid \boldsymbol{x}_{1:t}, y_{1:t}) \tag{9.31}$$

which focuses exclusively on reducing the uncertainty at $\boldsymbol{x}_t^{\mathrm{UCB}}$ rather than globally. We compare the two measures of information gain in Figure 9.9. Observe that the acquisition function depends critically on the choice of $I_t(\cdot)$ and is less sensitive to the scaling of confidence intervals.

IDS trades exploitation and exploration by balancing the (exploitative) regret surrogate with a measure of information gain (such as those studied in Chapter 8) that is purely explorative. In this way, IDS can account for kinds of information which are not addressed by alternative algorithms such as UCB or EI (Russo and Van Roy, 2014): Depending on the measure of information gain, IDS can select points to obtain *indirect information* about other points or *cumulating information* that does not immediately lead to a higher reward but only when combined with subsequent observations. Moreover, IDS avoids selecting points which yield *irrelevant information*.

**Problem 9.6**

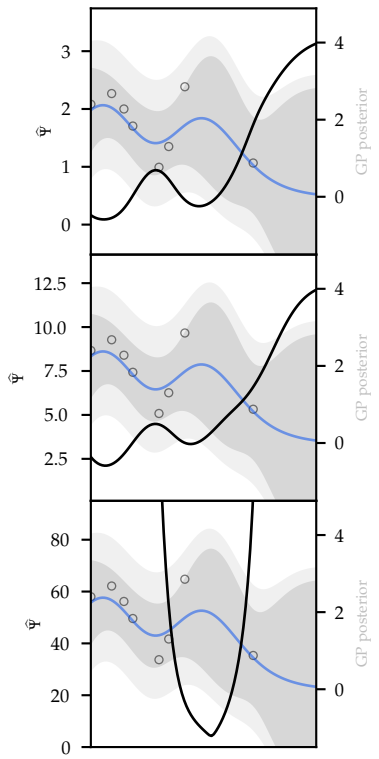



Figure 9.9: Plot of the surrogate information ratio $\widehat{\Psi}$: IDS selects its minimizer. The first two plots use the "global" information gain measure from Example 9.10 with $\beta = 0.25$ and $\beta = 0.5$, respectively. The third plot uses the "greedy" information gain measure from Equation (9.31) and $\beta = 1$.

### 9.3.5 *Probabilistic Inference*

As we mentioned before, computing the probability of maximality $\pi$ is generally intractable. You can think of computing $\pi$ as attempting to fully solve the probabilistic inference problem associated with determining the optimum of $f \mid \boldsymbol{x}_{1:t}, y_{1:t}$. In many cases, it is useful to determine the probability that a point is optimal under the current posterior distribution, and we will consider one particular example in the following.

**Example 9.12: Maximizing recall**

In domains such as molecular design we often use machine learning to screen candidates for further manual testing. The goal here is to suggest a small set $E$ from a large domain of molecules $\mathcal{X}$, so that the probability of $E$ containing the optimal molecule, i.e., the *recall*, is maximized. Note that this task is quite different from online Bayesian optimization, for example, in BO we get sequential feedback that we can use to decide which inputs to query next. Nevertheless, we will see in this section that both tasks turn out to be closely related.

Let us assume that the maximizer is unique almost surely, which with GPs is automatically the case if there are no same-mean, perfectly-correlated entries. The recall task is then intimately related to the task of determining the probability of maximality, since

$$\begin{aligned}\operatorname*{arg\,max}_{E \subseteq \mathcal{X} : |E| = k} \mathbb{P}_{f \mid \boldsymbol{x}_{1:t}, y_{1:t}} \left( \max_{\boldsymbol{x} \in E} f(\boldsymbol{x}) = \max_{\boldsymbol{x}'} f(\boldsymbol{x}') \right) \\ = \operatorname*{arg\,max}_{E \subseteq \mathcal{X} : |E| = k} \sum_{\boldsymbol{x} \in E} \pi(\boldsymbol{x} \mid \boldsymbol{x}_{1:t}, y_{1:t}).\end{aligned} \tag{9.32}$$

by noting that for $\boldsymbol{x} \neq \boldsymbol{y}$, the events $\{f(\boldsymbol{x}) = \max_{\boldsymbol{x}'} f(\boldsymbol{x}')\}$ and $\{f(\boldsymbol{y}) = \max_{\boldsymbol{x}'} f(\boldsymbol{x}')\}$ are disjoint

Thus, to obtain the recall-optimal candidates for further testing, we need to find the probability of maximality. However, as mentioned, computing the probability of maximality $\pi$ is generally intractable.

Figure 9.10 depicts an example of a recall task, where the black line shows the optimal recall achieved by knowing the probability of maximality $\pi$ exactly. LITE (Menet et al., 2025) is an almost-linear time approximation of $\pi$, whereas TS selects points via Thompson sampling and MEANS selects the points with the highest posterior mean. Selecting $E$ according to probability of maximality achieves much higher recall than the other intuitive heuristics.

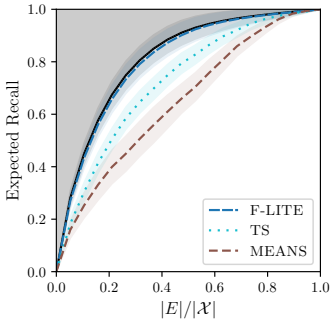



Figure 9.10: Menet et al. (2025) show in an experiment that selecting $E \subseteq \mathcal{X}$ according to the estimates of probability of maximality from LITE (here called F-LITE) is near-optimal. Intuitive heuristics such as Thompson sampling (TS) or selecting points with the highest posterior mean (MEANS) perform worse.

So how can we estimate the probability of maximality? LITE approximates $\pi$ by

$$\pi(\boldsymbol{x} \mid \boldsymbol{x}_{1:t}, y_{1:t}) = \mathbb{P}_{f|\boldsymbol{x}_{1:t}, y_{1:t}}\left(f(\boldsymbol{x}) \geq \max_{\boldsymbol{x}'} f(\boldsymbol{x}')\right)$$

$$\approx \mathbb{P}_{f|\boldsymbol{x}_{1:t}, y_{1:t}}(f(\boldsymbol{x}) \geq \kappa^*) \tag{9.33}$$

$$= \Phi\left(\frac{\mu_t(\boldsymbol{x}) - \kappa^*}{\sigma_t(\boldsymbol{x})}\right) \tag{9.34}$$

with $\Phi$ denoting the CDF of the standard normal distribution and $\kappa^*$ chosen such that the approximation of $\pi$ integrates to 1, so that it is a valid distribution.

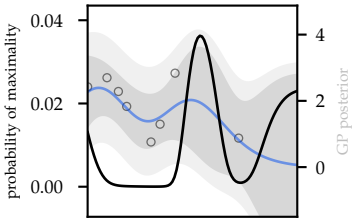



Figure 9.11: Plot of the probability of maximality as estimated by LITE.

Remarkably, LITE is intimately related to many of the BO methods we discussed in this chapter. First, and most obviously, Equation (9.33) looks similar to the PI acquisition function (9.17). But note that $\kappa^*$ is not equal to the best observed value $\hat{f}_t$ as in PI, but instead typically larger. If $\kappa^* > \hat{f}_t$, then Equation (9.33) is more exploratory than PI: it puts additional emphasis on points with large posterior variance and comparatively less emphasis on points with large posterior mean, which we illustrate in Figure 9.11.

An insightful interpretation of LITE is that it balances two different kinds of exploration: *optimism in the face of uncertainty* and *entropy regularization*. To see this, let us define the following variational objective:

$$\mathcal{W}(\pi) \doteq \sum_{\boldsymbol{x} \in \mathcal{X}} \pi(\boldsymbol{x}) \cdot \Big( \underbrace{\mu_t(\boldsymbol{x})}_{\text{exploitation}} + \underbrace{\sqrt{2\,\mathrm{S}'(\pi(\boldsymbol{x}))} \cdot \sigma_t(\boldsymbol{x})}_{\text{exploration}} \Big) \tag{9.35}$$

with the "quasi-surprise" $\mathrm{S}'(u) \doteq \frac{1}{2}(\phi(\Phi^{-1}(u))/u)^2$. The quasi-surprise $\mathrm{S}'(\cdot)$ behaves similarly to the surprise $-\ln(\cdot)$. In fact, their asymptotics coincide:

$$\mathrm{S}'(1) = 0 = -\ln(1) \quad \text{and} \quad \mathrm{S}'(u) \to -\ln(u) \text{ as } u \to 0^+.$$

The objective $\mathcal{W}$ is maximized for those probability distributions $\pi$ that are concentrated around points with large mean $\mu_t(\boldsymbol{x})$ and points with large exploration bonus. We have seen that the uncertainty $\sigma_t(\boldsymbol{x})$ about $f(\boldsymbol{x})$ is the standard exploration bonus of UCB-style algorithms. In Equation (9.35), $\sigma_t(\boldsymbol{x})$ is weighted by the quasi-surprise, which acts as "entropy regularization": it increases the entropy of $\pi$ by uniformly pushing $\pi(\boldsymbol{x})$ away from zero. You can think of entropy regularization as encouraging a different kind of exploration than optimism by not making deterministic decisions like in UCB.

Menet et al. (2025) show that LITE (9.34) is the solution to the varia-

tional problem (9.35) among valid probability distributions ⓘ:

**Problem 9.7**

$$\arg\max_{\pi \in \Delta^{\mathcal{X}}} \mathcal{W}(\pi) = \Phi\left(\frac{\mu_t(\cdot) - \kappa^*}{\sigma_t(\cdot)}\right) \tag{9.36}$$

with $\kappa^*$ such that the right-hand side sums to 1.[9] This indicates that LITE and Thompson sampling, which samples from probability of maximality, achieve exploration through two means:

1. **Optimism:** by preferring points with large uncertainty $\sigma_t(\boldsymbol{x})$ about the reward value $f(\boldsymbol{x})$.
2. **Decision uncertainty:** by assigning some probability mass to all $\boldsymbol{x}$, that is, by remaining uncertain about which $\boldsymbol{x}$ is the maximizer.

In our discussion of balancing exploration and exploitation in reinforcement learning, we will return to this dichotomy of exploration strategies.

[9] $\Delta^{\mathcal{X}}$ is the probability simplex on $\mathcal{X}$.

**Remark 9.13: But why is "exploration" useful for recall?**

Let us pause for a moment and reflect on why this interpretation of LITE is remarkable. This interpretation shows that LITE is more "exploratory" than simply taking the highest posterior means. However, the recall task (9.32) differs from the standard BO task in that we do not collect any further observations, which we may use downstream to make better decisions. In the recall task, we are only interested in having the best shot at including the maximizer in the set $E$, without obtaining any further information or making any subsequent decisions. At first sight, this seems to suggest that we should be as "exploitative" as possible. Then how can it be that "exploration" is useful?

We will explore this question with the following example: You are observing a betting game. When placing a bet, players can either place a safe bet ("playing safe") or a risky bet ("playing risky"). You now have to place a bet on their bets, and estimate which of the players will win the most money: those that play safe or those that play risky? Note that you do not care whether your guess ends up in 2nd place or last place among all players — you only care about whether your guess wins. That is, you are interested in *recalling* the best player, not in "ranking" the players.

Consider three players: one that plays safe and two that play risky. Suppose that the safe bet has payoff $S = 1$ while each risky bet has payoff $R \sim \mathcal{N}(0, 100)$. In expectation, the safe player will win the most money. However, one can see with just a little bit of algebra that the probability of either of the risky players winning the most money is $\approx$35%, whereas the safe player only wins with

probability $\approx 29\%$ (?). That is, it is in fact *optimal* to bet on either of the risky players since the best player might have vastly outperformed their expected winnings, and performed closer to their upper confidence bound. In summary, maximizing recall requires us to be "exploratory" since it is likely that the optimum among inputs is one that has performed better than expected, not simply the one with the highest expected performance.

**Problem 9.8**

## *Discussion*

In this chapter, we have explored the exploration-exploitation dilemma in the context of optimizing black-box functions. To this end, we have explored various methods to balance exploration and exploitation. While computing the precise probability of maximality is generally intractable, we found that it can be understood approximately as balancing two sources of exploration: optimism in the face of uncertainty and entropy regularization.

In the following chapters, we will begin to discuss stateful settings where the black-box optimization task is known as "reinforcement learning". Naturally, we will see that the exploration-exploitation dilemma is also a central challenge in reinforcement learning, and we will revisit many of the concepts we have discussed in this chapter.

**Optional Readings**

- Srinivas, Krause, Kakade, and Seeger (2010).
  *Gaussian process optimization in the bandit setting: No regret and experimental design*.
- Golovin, Solnik, Moitra, Kochanski, Karro, and Sculley (2017).
  *Google vizier: A service for black-box optimization*.
- Romero, Krause, and Arnold (2013).
  *Navigating the protein fitness landscape with Gaussian processes*.
- Chowdhury and Gopalan (2017).
  *On kernelized multi-armed bandits*.

## *Problems*

**9.1. Convergence to the static optimum.**

Show that any algorithm where $\lim_{t\to\infty} f^\star(\boldsymbol{x}_t)$ exists achieves sublinear regret if and only if it converges to the static optimum, that is,

$$\lim_{t\to\infty} f^\star(\boldsymbol{x}_t) = \max_{\boldsymbol{x}} f^\star(\boldsymbol{x}). \tag{9.37}$$

*Hint: Use that if a sequence $a_n$ converges to $a$ as $n \to \infty$, then we have for the sequence*

$$b_n \doteq \frac{1}{n}\sum_{i=1}^{n} a_i \tag{9.38}$$

*that $\lim_{n\to\infty} b_n = a$. This is also known as the* Cesàro mean.

**9.2. Bayesian confidence intervals.**

In this exercise, we derive Theorem 9.4.

1. For fixed $t \geq 1$ and $\boldsymbol{x} \in \mathcal{X}$, prove

$$\mathbb{P}(f^\star(\boldsymbol{x}) \notin \mathcal{C}_t(\boldsymbol{x}) \mid \boldsymbol{x}_{1:t-1}, y_{1:t-1}) \leq e^{-\beta_t^2/2}. \tag{9.39}$$

   *Hint: Bound $\mathbb{P}(Z > c)$ for $Z \sim \mathcal{N}(0,1)$ and $c > 0$.*
2. Prove Theorem 9.4.

**9.3. Regret of GP-UCB.**

To develop some intuition, we will derive Theorem 9.5.

1. Show that if Equation (9.7) holds, then for a fixed $t \geq 1$ the instantaneous regret $r_t$ is bounded by $2\beta_t\sigma_{t-1}(\boldsymbol{x}_t)$.
2. Let $S_T \doteq \{\boldsymbol{x}_t\}_{t=1}^T$, and define $\boldsymbol{f}_T \doteq \boldsymbol{f}_{S_T}$ and $\boldsymbol{y}_T \doteq \boldsymbol{y}_{s_T}$. Prove

$$\mathrm{I}(\boldsymbol{f}_T; \boldsymbol{y}_T) = \frac{1}{2}\sum_{t=1}^{T} \log\left(1 + \frac{\sigma_{t-1}^2(\boldsymbol{x}_t)}{\sigma_{\mathrm{n}}^2}\right). \tag{9.40}$$

3. Combine (1) and (2) to show Theorem 9.5. We assume w.l.o.g. that the sequence $\{\beta_t\}_t$ is monotonically increasing.
   *Hint: If $s \in [0, M]$ for some $M > 0$ then $s \leq C \cdot \log(1+s)$ with $C \doteq M/\log(1+M)$.*

**9.4. Sublinear regret of GP-UCB for a linear kernel.**

Assume that $f^\star \sim \mathcal{GP}(0, k)$ where $k$ is the linear kernel

$$k(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{x}^\top \boldsymbol{x}'.$$

In addition, we assume that for any $\boldsymbol{x} \in \mathcal{X}$, $\|\boldsymbol{x}\|_2 \leq 1$. Moreover, recall that the points in a finite set $S \subseteq \mathcal{X}$ can be written in a matrix form (the "design matrix") which we denote by $\boldsymbol{X}_S \in \mathbb{R}^{d\times|S|}$.

1. Prove that $\gamma_T = O(d \log T)$.
2. Deduce from (1) and Theorem 9.5 that $\lim_{T \to \infty} R_T / T = 0$.

**9.5. Closed-form expected improvement.**

Let us denote the acquisition function of EI from Equation (9.19) by $\mathrm{EI}_t(\boldsymbol{x})$. In this exercise, we derive a closed-form expression.

1. Show that

$$\mathrm{EI}_t(\boldsymbol{x}) = \int_{(\hat{f}_t - \mu_t(\boldsymbol{x}))/\sigma_t(\boldsymbol{x})}^{+\infty} (\mu_t(\boldsymbol{x}) + \sigma_t(\boldsymbol{x})\varepsilon - \hat{f}_t) \cdot \phi(\varepsilon)\, d\varepsilon \tag{9.41}$$

   where $\phi$ is the PDF of the univariate standard normal distribution (1.5).
   *Hint: Reparameterize the posterior distribution*

$$f(\boldsymbol{x}) \mid \boldsymbol{x}_{1:t}, y_{1:t} \sim \mathcal{N}(\mu_t(\boldsymbol{x}), \sigma_t^2(\boldsymbol{x}))$$

   *using a standard normal distribution.*
2. Using the above expression, show that

$$\mathrm{EI}_t(\boldsymbol{x}) = (\mu_t(\boldsymbol{x}) - \hat{f}_t)\Phi\left(\frac{\mu_t(\boldsymbol{x}) - \hat{f}_t}{\sigma_t(\boldsymbol{x})}\right) + \sigma_t(\boldsymbol{x})\phi\left(\frac{\mu_t(\boldsymbol{x}) - \hat{f}_t}{\sigma_t(\boldsymbol{x})}\right) \tag{9.42}$$

   where $\Phi(u) \doteq \int_{-\infty}^{u} \phi(\varepsilon)\, d\varepsilon$ denotes the CDF of the standard normal distribution.

Note that the first term of Equation (9.42) encourages exploitation while the second term encourages exploration. EI can be seen as a special case of UCB where the confidence bounds are scaled depending on $\boldsymbol{x}$:

$$\beta_t = \phi(z_t(\boldsymbol{x}))/\Phi(z_t(\boldsymbol{x}))$$

for $z_t(\boldsymbol{x}) \doteq (\mu_t(\boldsymbol{x}) - \hat{f}_t)/\sigma_t(\boldsymbol{x})$.

**9.6. Regret of IDS.**

We derive Theorem 9.11.

1. Prove that for all $t \geq 1$, $\hat{\Delta}_t(\boldsymbol{x}_t^{\mathrm{UCB}}) \leq 2\beta_{t+1}\sigma_t(\boldsymbol{x}_t^{\mathrm{UCB}})$ where we denote by $\boldsymbol{x}_t^{\mathrm{UCB}} \doteq \arg\max_{\boldsymbol{x} \in \mathcal{X}} u_t(\boldsymbol{x})$ the point maximizing the upper confidence bound after iteration $t$.
2. Using (1) and the assumption on $I_t$, bound $\widehat{\Psi}_t(\boldsymbol{x}_t^{\mathrm{UCB}})$.
   *Hint: You may find the hint of Problem 9.3 (3) useful.*
3. Complete the proof of Theorem 9.11.

**9.7. Variational form of LITE.**

Let $|\mathcal{X}| < \infty$ be such that we can write $\mathcal{W}$ as an objective function over the probability simplex $\Delta^{|\mathcal{X}|} \subset \mathbb{R}^{|\mathcal{X}|}$. Derive Equation (9.36).

*Hint: First show that $\mathcal{W}(\cdot)$ is concave (i.e., minimizing $-\mathcal{W}(\cdot)$ is a convex optimization problem) and then use Lagrange multipliers to find the optimum.*

**9.8. Finding the winning player.**

Consider the betting game from Remark 9.13 with two risky players, $R_1 \sim \mathcal{N}(0, 100)$ and $R_2 \sim \mathcal{N}(0, 100)$, and one safe player $S = 1$. Prove that the individual probability of any of the risky players winning the most money is larger than the winning probability of the safe player. We assume that players payoffs are mutually independent.

*Hint: Compute the CDF of $R \doteq \max\{R_1, R_2\}$.*

# 10

# *Markov Decision Processes*

We will now turn to the topic of probabilistic planning. Planning deals with the problem of deciding which action an agent should play in a (stochastic) environment.[1] A key formalism for probabilistic planning in *known* environments are so-called Markov decision processes. Starting from the next chapter, we will look at reinforcement learning, which extends probabilistic planning to unknown environments.

[1] An environment is *stochastic* as opposed to deterministic, when the outcome of actions is random.

Consider the setting where we have a sequence of states $(X_t)_{t\in\mathbb{N}_0}$ similarly to Markov chains. But now, the next state $X_{t+1}$ of an agent does not only depend on the previous state $X_t$ but also depends on the last action $A_t$ of this agent.

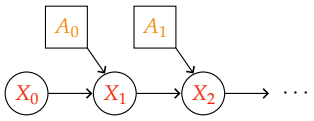



Figure 10.1: Directed graphical model of a Markov decision process with hidden states $X_t$ and actions $A_t$.

**Definition 10.1** ((Finite) Markov decision process, MDP)**.** A *(finite) Markov decision process* is specified by

- a (finite) set of *states* $X \doteq \{1, \dots, n\}$,
- a (finite) set of *actions* $A \doteq \{1, \dots, m\}$,
- *transition probabilities*

$$p(x' \mid x, a) \doteq \mathbb{P}\big(X_{t+1} = x' \mid X_t = x, A_t = a\big) \tag{10.1}$$

  which is also called the *dynamics model*, and
- a *reward function* $r : X \times A \to \mathbb{R}$ which maps the current state $x$ and an action $a$ to some reward.

The reward function may also depend on the next state $x'$, however, we stick to the above model for simplicity. Also, the reward function can be random with mean $r$. Observe that $r$ induces the sequence of rewards $(R_t)_{t\in\mathbb{N}_0}$, where

$$R_t \doteq r(X_t, A_t), \tag{10.2}$$

which is sometimes used in the literature instead of $r$.

Crucially, we assume the dynamics model $p$ and the reward function $r$ to be known. That is, we operate in a known environment. For now,

we also assume that the environment is *fully observable*. In other words, we assume that our agent knows its current state. In Section 10.4, we discuss how this method can be extended to the partially observable setting.

Our fundamental objective is to learn how the agent should behave to optimize its reward. In other words, given its current state, the agent should decide (optimally) on the action to play. Such a decision map — whether optimal or not — is called a policy.

**Definition 10.2** (Policy)**.** A *policy* is a function that maps each state $x \in X$ to a probability distribution over the actions. That is, for any time $t$,

$$\pi(a \mid x) \doteq \mathbb{P}(A_t = a \mid X_t = x). \tag{10.3}$$

In other words, a policy assigns to each action $a \in A$, a probability of being played given the current state $x \in X$.

We assume that policies are stationary, that is, do not change over time.

**Remark 10.3: Stochastic policies**

We will see later in this chapter that in fully observable environments optimal policies are always deterministic. Thus, there is no need to consider stochastic policies in the context of Markov decision processes. For this chapter, you can think of a policy $\pi$ simply as a deterministic mapping $\pi : X \to A$ from current state to played action. In the context of reinforcement learning, we will later see in Section 12.5.1 that randomized policies are important in trading exploration and exploitation.

Observe that a policy induces a Markov chain $(X_t^\pi)_{t \in \mathbb{N}_0}$ with transition probabilities,

$$p^\pi(x' \mid x) \doteq \mathbb{P}\big(X_{t+1}^\pi = x' \mid X_t^\pi = x\big) = \sum_{a \in A} \pi(a \mid x) p(x' \mid x, a). \tag{10.4}$$

This is crucial: if our agent follows a fixed policy (i.e., decision-making protocol) then the evolution of the process is described fully by a Markov chain.

As mentioned, we want to maximize the reward. There are many models of calculating a score from the infinite sequence of rewards $(R_t)_{t \in \mathbb{N}_0}$. For the purpose of our discussion of Markov decision processes and reinforcement learning, we will focus on a very common reward called discounted payoff.

**Definition 10.4** (Discounted payoff)**.** The *discounted payoff* (also called *discounted total reward*) from time $t$ is defined as the random variable,

$$G_t \doteq \sum_{m=0}^{\infty} \gamma^m R_{t+m} \tag{10.5}$$

where $\gamma \in [0, 1)$ is the *discount factor*.

**Remark 10.5: Other reward models**

Other well-known methods for combining rewards into a score are

$$\underset{\text{(instantaneous)}}{G_t \doteq R_t} \quad , \quad \underset{\text{(finite-horizon)}}{G_t \doteq \sum_{m=0}^{T-1} R_{t+m}}, \quad \text{and} \quad \underset{\text{(mean payoff)}}{G_t \doteq \liminf_{T\to\infty} \frac{1}{T} \sum_{m=0}^{T-1} R_{t+m}}.$$

The methods that we will discuss can also be analyzed using these or other alternative reward models.

We now want to understand the effect of the starting state and initial action on our optimization objective $G_t$. To analyze this, it is common to use the following two functions:

**Definition 10.6** (State value function)**.** The *state value function*,[2]

$$v_t^\pi(x) \doteq \mathbb{E}_\pi[G_t \mid X_t = x], \tag{10.7}$$

measures the average discounted payoff from time $t$ starting from state $x \in X$.

[2] Recall that following a fixed policy $\pi$ induces a Markov chain $(X_t^\pi)_{t\in\mathbb{N}_0}$. We define

$$\mathbb{E}_\pi[\cdot] \doteq \mathbb{E}_{(X_t^\pi)_{t\in\mathbb{N}_0}}[\cdot] \tag{10.6}$$

as an expectation over all possible sequences of states $(x_t)_{t\in\mathbb{N}_0}$ within this Markov chain.

**Definition 10.7** (State-action value function)**.** The *state-action value function* (also called *Q-function*),

$$\begin{aligned} q_t^\pi(x,a) &\doteq \mathbb{E}_\pi[G_t \mid X_t = x, A_t = a] && (10.8)\\ &= r(x,a) + \gamma \sum_{x'\in X} p(x' \mid x,a) \cdot v_{t+1}^\pi(x'), && (10.9) \end{aligned}$$

(10.9): by expanding the defition of the discounted payoff (10.5); corresponds to one step in the induced Markov chain

measures the average discounted payoff from time $t$ starting from state $x \in X$ and with playing action $a \in A$. In other words, it combines the immediate return with the value of the next states.

Note that both $v_t^\pi(x)$ and $q_t^\pi(x,a)$ are deterministic scalar-valued functions. Because we assumed stationary dynamics, rewards, and policies, the discounted payoff starting from a given state $x$ will be independent of the start time $t$. Thus, we write $v^\pi(x) \doteq v_0^\pi(x)$ and $q^\pi(x,a) \doteq q_0^\pi(x,a)$ without loss of generality.

## 10.1 Bellman Expectation Equation

Let us now see how we can compute the value function,

$$\begin{aligned} v^\pi(x) &= \mathbb{E}_\pi[G_0 \mid X_0 = x] && \text{using the definition of the value function (10.7)}\\ &= \mathbb{E}_\pi\left[\sum_{m=0}^{\infty} \gamma^m R_m \,\middle|\, X_0 = x\right] && \text{using the definition of the discounted payoff (10.5)} \end{aligned}$$

$$= \mathbb{E}_\pi\left[\gamma^0 R_0 \;\middle|\; X_0 = x\right] + \gamma \mathbb{E}_\pi\left[\sum_{m=0}^{\infty} \gamma^m R_{m+1} \;\middle|\; X_0 = x\right]$$ using linearity of expectation (1.20)

$$= r(x, \pi(x)) + \gamma \mathbb{E}_{x'}\left[\mathbb{E}_\pi\left[\sum_{m=0}^{\infty} \gamma^m R_{m+1} \;\middle|\; X_1 = x'\right] \;\middle|\; X_0 = x\right]$$ by simplifying the first expectation and conditioning the second expectation on $X_1$

$$= r(x, \pi(x)) + \gamma \sum_{x' \in X} p(x' \mid x, \pi(x)) \mathbb{E}_\pi\left[\sum_{m=0}^{\infty} \gamma^m R_{m+1} \;\middle|\; X_1 = x'\right]$$ expanding the expectation on $X_1$ and using conditional independence of the discounted payoff of $X_0$ given $X_1$

$$= r(x, \pi(x)) + \gamma \sum_{x' \in X} p(x' \mid x, \pi(x)) \mathbb{E}_\pi\left[\sum_{m=0}^{\infty} \gamma^m R_m \;\middle|\; X_0 = x'\right]$$ shifting the start time of the discounted payoff using stationarity

$$= r(x, \pi(x)) + \gamma \sum_{x' \in X} p(x' \mid x, \pi(x)) \mathbb{E}_\pi\left[G_0 \mid X_0 = x'\right]$$ using the definition of the discounted payoff (10.5)

$$= r(x, \pi(x)) + \gamma \sum_{x' \in X} p(x' \mid x, \pi(x)) \cdot v^\pi(x'). \tag{10.10}$$ using the definition of the value function (10.7)

$$= r(x, \pi(x)) + \gamma \mathbb{E}_{x' \mid x, \pi(x)}\left[v^\pi(x')\right]. \tag{10.11}$$ interpreting the sum as an expectation

This equation is known as the *Bellman expectation equation*, and it shows a recursive dependence of the value function on itself. The intuition is clear: the value of the current state corresponds to the reward from the next action plus the discounted sum of all future rewards obtained from the subsequent states.

For stochastic policies, the above calculation can be extended to yield,

$$v^\pi(x) = \sum_{a \in A} \pi(a \mid x)\left(r(x, a) + \gamma \sum_{x' \in X} p(x' \mid x, a) v^\pi(x')\right) \tag{10.12}$$

$$= \mathbb{E}_{a \sim \pi(x)}\left[r(x, a) + \gamma \mathbb{E}_{x' \mid x, a}\left[v^\pi(x')\right]\right] \tag{10.13}$$

$$= \mathbb{E}_{a \sim \pi(x)}[q^\pi(x, a)]. \tag{10.14}$$

For stochastic policies, by also conditioning on the first action, one can obtain an analogous equation for the state-action value function,

$$q^\pi(x, a) = r(x, a) + \gamma \sum_{x' \in X} p(x' \mid x, a) \sum_{a' \in A} \pi(a' \mid x') q^\pi(x', a') \tag{10.15}$$

$$= r(x, a) + \gamma \mathbb{E}_{x' \mid x, a} \mathbb{E}_{a' \sim \pi(x')}\left[q^\pi(x', a')\right]. \tag{10.16}$$

Note that it does not make sense to consider a similar recursive formula for the state-action value function in the setting of deterministic policies as the action played when in state $x \in X$ is uniquely determined as $\pi(x)$. In particular,

$$v^\pi(x) = q^\pi(x, \pi(x)). \tag{10.17}$$

## 10.2 Policy Evaluation

Bellman's expectation equation tells us how we can find the value function $v^\pi$ of a fixed policy $\pi$ using a system of linear equations!

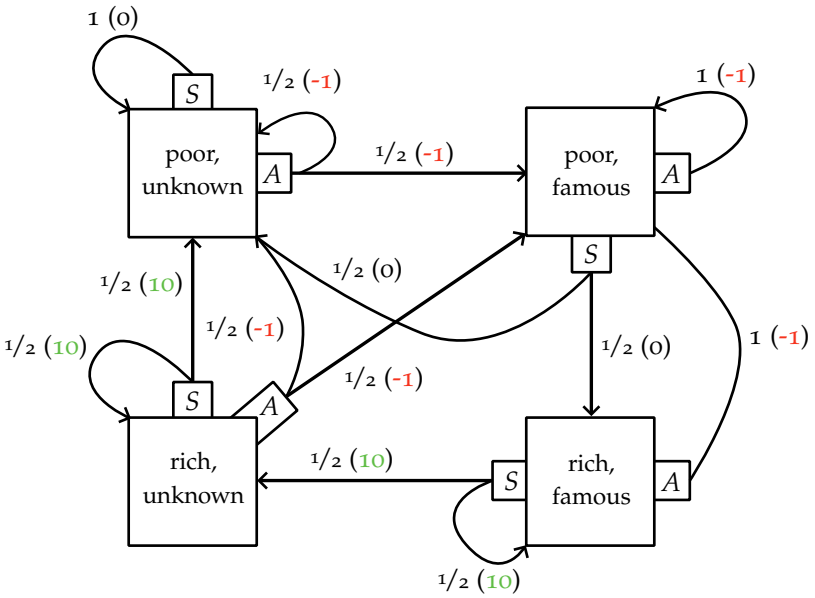



Figure 10.2: Example of an MDP, which we study in Problem 10.1. Suppose you are building a company. The shown MDP models "how to become rich and famous". Here, the action $S$ is short for *saving* and the action $A$ is short for *advertising*.

Suppose you begin by being "poor and unknown". Then, the greedy action (i.e., the action maximizing instantaneous reward) is to save. However, within this simplified environment, saving when you are poor and unknown means that you will remain poor and unknown forever. As the potential rewards in other states are substantially larger, this simple example illustrates that following the greedy choice is generally not optimal.

The example is adapted from Andrew Moore's lecture notes on MDPs (Moore, 2002).

Using,

$$\boldsymbol{v}^\pi \doteq \begin{bmatrix} v^\pi(1) \\ \vdots \\ v^\pi(n) \end{bmatrix}, \quad \boldsymbol{r}^\pi \doteq \begin{bmatrix} r(1,\pi(1)) \\ \vdots \\ r(n,\pi(n)) \end{bmatrix}, \quad \text{and}$$
$$\boldsymbol{P}^\pi \doteq \begin{bmatrix} p(1 \mid 1,\pi(1)) & \cdots & p(n \mid 1,\pi(1)) \\ \vdots & \ddots & \vdots \\ p(1 \mid n,\pi(n)) & \cdots & p(n \mid n,\pi(n)) \end{bmatrix} \tag{10.18}$$

and a little bit of linear algebra, the Bellman expectation equation (10.10) is equivalent to

$$\boldsymbol{v}^\pi = \boldsymbol{r}^\pi + \gamma \boldsymbol{P}^\pi \boldsymbol{v}^\pi \tag{10.19}$$
$$\iff \quad (\boldsymbol{I} - \gamma \boldsymbol{P}^\pi)\boldsymbol{v}^\pi = \boldsymbol{r}^\pi$$
$$\iff \quad \boldsymbol{v}^\pi = (\boldsymbol{I} - \gamma \boldsymbol{P}^\pi)^{-1}\boldsymbol{r}^\pi. \tag{10.20}$$

Solving this linear system of equations (i.e., performing matrix inversion) takes cubic time in the size of the state space.

### 10.2.1 *Fixed-point Iteration*

To obtain an (approximate) solution of $\boldsymbol{v}^\pi$, we can use that it is the unique fixed-point of the affine mapping $\boldsymbol{B}^\pi : \mathbb{R}^n \to \mathbb{R}^n$,

$$\boldsymbol{B}^\pi \boldsymbol{v} \doteq \boldsymbol{r}^\pi + \gamma \boldsymbol{P}^\pi \boldsymbol{v}. \tag{10.21}$$

Using this fact (which we will prove in just a moment), we can use fixed-point iteration of $\boldsymbol{B}^\pi$.

**Algorithm 10.8:** Fixed-point iteration

initialize $\boldsymbol{v}^\pi$ (e.g., as $\mathbf{0}$)
**for** $t = 1$ **to** $T$ **do**
  $\boldsymbol{v}^\pi \leftarrow \boldsymbol{B}^\pi \boldsymbol{v}^\pi = \boldsymbol{r}^\pi + \gamma \boldsymbol{P}^\pi \boldsymbol{v}^\pi$

Fixed-point iteration has computational advantages, for example, for sparse transitions.

**Theorem 10.9.** *$\boldsymbol{v}^\pi$ is the unique fixed-point of $\boldsymbol{B}^\pi$.*

*Proof.* It is immediate from Bellman's expectation equation (10.19) and the definition of $\boldsymbol{B}^\pi$ (10.21) that $\boldsymbol{v}^\pi$ is a fixed-point of $\boldsymbol{B}^\pi$. To prove uniqueness, we will show that $\boldsymbol{B}^\pi$ is a contraction.

**Remark 10.10: Contractions**
A *contraction* is a concept from topology. In a Banach space $(\mathcal{X}, \|\cdot\|)$ (a metric space with a norm), $f : \mathcal{X} \to \mathcal{X}$ is a contraction iff there exists a $k < 1$ such that

$$\|f(\boldsymbol{x}) - f(\boldsymbol{y})\| \leq k \cdot \|\boldsymbol{x} - \boldsymbol{y}\| \tag{10.22}$$

for any $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$. By the *Banach fixed-point theorem*, a contraction admits a unique fixed-point. Intuitively, by iterating the function $f$, the distance to any fixed-point shrinks by a factor $k$ in each iteration, hence, converges to 0. As we cannot converge to multiple fixed-points simultaneously, the fixed-point of a contraction $f$ must be unique.

Let $\boldsymbol{v} \in \mathbb{R}^n$ and $\boldsymbol{v}' \in \mathbb{R}^n$ be arbitrary initial guesses. We use the $L_\infty$ space,[3]

$$\begin{aligned}
\left\|\boldsymbol{B}^\pi \boldsymbol{v} - \boldsymbol{B}^\pi \boldsymbol{v}'\right\|_\infty &= \left\|\boldsymbol{r}^\pi + \gamma \boldsymbol{P}^\pi \boldsymbol{v} - \boldsymbol{r}^\pi - \gamma \boldsymbol{P}^\pi \boldsymbol{v}'\right\|_\infty && \\
&= \gamma \left\|\boldsymbol{P}^\pi (\boldsymbol{v} - \boldsymbol{v}')\right\|_\infty && \\
&\leq \gamma \max_{\boldsymbol{x} \in X} \sum_{x' \in X} p(x' \mid x, \pi(x)) \cdot \left|\boldsymbol{v}(x') - \boldsymbol{v}'(x')\right| . && \text{using the definition of the } L_\infty \text{ norm (10.23), expanding the multiplication, and using } \left|\textstyle\sum_i a_i\right| \leq \textstyle\sum_i |a_i| \\
&\leq \gamma \left\|\boldsymbol{v} - \boldsymbol{v}'\right\|_\infty . && \text{using } \textstyle\sum_{x' \in X} p(x' \mid x, \pi(x)) = 1 \text{ and } |\boldsymbol{v}(x') - \boldsymbol{v}'(x')| \leq \|\boldsymbol{v} - \boldsymbol{v}'\|_\infty
\end{aligned} \tag{10.24}$$

Thus, by Equation (10.22), $\boldsymbol{B}^\pi$ is a contraction and by Banach's fixed-point theorem $\boldsymbol{v}^\pi$ is its unique fixed-point. □

Let $\boldsymbol{v}_t^\pi$ be the value function estimate after $t$ iterations. Then, we have

[3] The $L_\infty$ *norm* (also called *supremum norm*) is defined as

$$\|\boldsymbol{x}\|_\infty \doteq \max_i |\boldsymbol{x}(i)| . \tag{10.23}$$

using the definition of $\boldsymbol{B}^\pi$ (10.21)

for the convergence of fixed-point iteration,

$$\begin{aligned} \|\boldsymbol{v}_t^\pi - \boldsymbol{v}^\pi\|_\infty &= \left\|\boldsymbol{B}^\pi \boldsymbol{v}_{t-1}^\pi - \boldsymbol{B}^\pi \boldsymbol{v}^\pi\right\|_\infty && \text{using the update rule of fixed-point iteration and } \boldsymbol{B}^\pi \boldsymbol{v}^\pi = \boldsymbol{v}^\pi \\ &\leq \gamma \left\|\boldsymbol{v}_{t-1}^\pi - \boldsymbol{v}^\pi\right\|_\infty && \text{using (10.24)} \\ &\leq \gamma^t \left\|\boldsymbol{v}_0^\pi - \boldsymbol{v}^\pi\right\|_\infty . && \text{by induction} \end{aligned} \tag{10.25}$$

This shows that fixed-point iteration converges to $\boldsymbol{v}^\pi$ exponentially fast.

## *10.3 Policy Optimization*

Recall that our goal was to find an optimal policy,

$$\pi^\star \doteq \arg\max_\pi \mathbb{E}_\pi[G_0]. \tag{10.26}$$

We can alternatively characterize an optimal policy as follows: We order policies by saying

$$\pi \geq \pi' \iff v^\pi(x) \geq v^{\pi'}(x) \quad \text{for all } x \in X. \tag{10.27}$$

Strictly speaking, this is a preorder on individual policies.[4] We can then look at the equivalence classes of policies under this preorder, i.e., the sets of policies that share the same value function. Over these equivalence classes, the above preorder is a partial order.[5] An optimal policy $\pi^\star$ is then simply a policy whose equivalence class is maximal according to this partial order. Note that it is not clear a priori whether such an optimal policy exists, since there may in general be policies that dominate each other on different subsets of states. Those policies are incomparable under the relation from Equation (10.27). In the following, we will see that an optimal policy does exist, and that we can find it.

[4] That is, it is reflexive ($\pi \geq \pi$ for all policies $\pi$) and transitive ($\pi \geq \pi'$ and $\pi' \geq \pi''$ implies $\pi \geq \pi''$).

[5] That is, for any two classes of policies $[\pi]$ and $[\pi']$, if $[\pi] \geq [\pi']$ and $[\pi'] \geq [\pi]$ then $[\pi] = [\pi']$, which is called "antisymmetry".

It follows from the above that all optimal policies have identical value functions. Subsequently, we use $v^\star \doteq v^{\pi^\star}$ and $q^\star \doteq q^{\pi^\star}$ to denote the state value function and state-action value function arising from an optimal policy, respectively. As an optimal policy maximizes the value of each state, we have that

$$v^\star(x) = \max_\pi v^\pi(x), \quad q^\star(x,a) = \max_\pi q^\pi(x,a). \tag{10.28}$$

Simply optimizing over each policy is not a good idea as there are $m^n$ deterministic policies in total. It turns out that we can do much better.

### 10.3.1 Greedy Policies

Consider a policy that acts greedily according to the immediate return. It is fairly obvious that this policy will not perform well because the agent might never get to high-reward states. But what if someone could tell us not just the immediate return, but the long-term value of the states our agent can reach in a single step? If we knew the value of each state our agent can reach, then we can simply pick the action that maximizes the expected value. We will make this approach precise in the next section.

This thought experiment suggests the definition of a greedy policy with respect to a value function.

**Definition 10.11** (Greedy policy)**.** The *greedy policy* with respect to a state-action value function $q$ is defined as

$$\pi_q(x) \doteq \arg\max_{a \in A} q(x, a). \tag{10.29}$$

Analogously, we define the *greedy policy* with respect to a state value function $v$,

$$\pi_v(x) \doteq \arg\max_{a \in A} r(x, a) + \gamma \sum_{x' \in X} p(x' \mid x, a) \cdot v(x'). \tag{10.30}$$

We can use $v$ and $q$ interchangeably (?).

**Problem 10.2**

### 10.3.2 Bellman Optimality Equation

Observe that following the greedy policy $\pi_v$, will lead us to a new value function $v^{\pi_v}$. With respect to this value function, we can again obtain a greedy policy, of which we can then obtain a new value function. In this way, the correspondence between greedy policies and value functions induces a cyclic dependency, which is visualized in Figure 10.3.

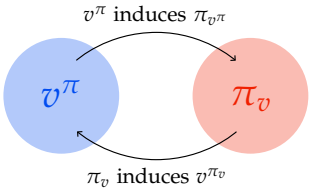



Figure 10.3: Cyclic dependency between **value function** and **greedy policy**.

It turns out that the optimal policy $\pi^\star$ is a fixed-point of this dependency. This is made precise by the following theorem.

**Theorem 10.12** (Bellman's theorem)**.** *A policy $\pi^\star$ is optimal iff it is greedy with respect to its own value function. In other words, $\pi^\star$ is optimal iff $\pi^\star(x)$ is a distribution over the set* $\arg\max_{a \in A} q^\star(x, a)$.

In particular, if for every state there is a unique action that maximizes the state-action value function, the policy $\pi^\star$ is deterministic and unique,

$$\pi^\star(x) = \arg\max_{a \in A} q^\star(x, a). \tag{10.31}$$

*Proof.* It is a direct consequence of Equation (10.28) that a policy is optimal iff it is greedy with respect to $q^\star$. □

This theorem confirms our intuition from the previous section that greedily following an optimal value function is itself optimal. In particular, Bellman's theorem shows that there always exists an optimal policy which is deterministic and stationary.

We have seen, that $\pi^\star$ is a fixed-point of greedily picking the best action according to its state-action value function. The converse is also true:

**Corollary 10.13.** *The optimal value functions $v^\star$ and $q^\star$ are a fixed-point of the so-called* Bellman update*,*

$$v^\star(x) = \max_{a \in A} q^\star(x, a), \tag{10.32}$$

$$= \max_{a \in A} r(x, a) + \gamma \mathbb{E}_{x'|x,a}\left[v^\star(x')\right] \tag{10.33}$$

*using the definition of the q-function* (10.9)

$$q^\star(x, a) = r(x, a) + \gamma \mathbb{E}_{x'|x,a}\left[\max_{a' \in A} q^\star(x', a')\right]. \tag{10.34}$$

*Proof.* It follows from Equation (10.14) that

$$v^\star(x) = \mathbb{E}_{a \sim \pi^\star(x)}[q^\star(x, a)]. \tag{10.35}$$

Thus, as $\pi^\star$ is greedy with respect to $q^\star$, $v^\star(x) = \max_{a \in A} q^\star(x, a)$.

Equation (10.34) follows analogously from Equation (10.15). □

These equations are also called the *Bellman optimality equations*. Intuitively, the Bellman optimality equations express that the value of a state under an optimal policy must equal the expected return for the best action from that state. Bellman's theorem is also known as *Bellman's optimality principle*, which is a more general concept.

**Bellman's optimality principle** Bellman's optimality equations for MDPs are one of the main settings of Bellman's optimality principle. However, Bellman's optimality principle has many other important applications, for example in dynamic programming. Broadly speaking, Bellman's optimality principle says that optimal solutions to decision problems can be decomposed into optimal solutions to sub-problems.

The two perspectives of Bellman's theorem naturally suggest two separate ways of finding the optimal policy. Policy iteration uses the perspective from Equation (10.31) of $\pi^\star$ as a fixed-point of the dependency between greedy policy and value function. In contrast, value iteration uses the perspective from Equation (10.32) of $v^\star$ as the fixed-point of the Bellman update. Another approach which we will not discuss here is to use a linear program where the Bellman update is interpreted as a set of linear inequalities.

### *10.3.3 Policy Iteration*

Starting from an arbitrary initial policy, policy iteration as shown in Algorithm 10.14 uses the Bellman expectation equation to compute

**Algorithm 10.14:** Policy iteration

```
initialize π (arbitrarily)
repeat
    compute v^π
    compute π_{v^π}
    π ← π_{v^π}
until converged
```

the value function of that policy (as we have discussed in Section 10.2) and then chooses the greedy policy with respect to that value function as its next iterate.

Let $\pi_t$ be the policy after $t$ iterations. We will now show that policy iteration converges to the optimal policy. The proof is split into two parts. First, we show that policy iteration improves policies monotonically. Then, we will use this fact to show that policy iteration converges.

**Lemma 10.15** (Monotonic improvement of policy iteration)**.** *We have,*

- $v^{\pi_{t+1}}(x) \geq v^{\pi_t}(x)$ *for all* $x \in X$*; and*
- $v^{\pi_{t+1}}(x) > v^{\pi_t}(x)$ *for at least one* $x \in X$*, unless* $v^{\pi_t} \equiv v^\star$*.*

*Proof.* We consider the Bellman update from (10.32) as the mapping $\boldsymbol{B}^\star : \mathbb{R}^n \to \mathbb{R}^n$,

$$(\boldsymbol{B}^\star \boldsymbol{v})(x) \doteq \max_{a \in A} q(x, a), \tag{10.36}$$

where $q$ is the state-action value function corresponding to the state value function $\boldsymbol{v} \in \mathbb{R}^n$. Recall that after obtaining $v^{\pi_t}$, policy iteration first computes the greedy policy w.r.t. $v^{\pi_t}$, $\pi_{t+1} \doteq \pi_{v^{\pi_t}}$, and then computes its value function $v^{\pi_{t+1}}$.

To establish the (weak) monotonic improvement of policy iteration, we consider a fixed-point iteration (cf. Algorithm 10.8) of $\boldsymbol{v}^{\pi_{t+1}}$ initialized by $\boldsymbol{v}^{\pi_t}$. We denote the iterates by $\tilde{\boldsymbol{v}}_\tau$, in particular, we have that $\tilde{\boldsymbol{v}}_0 = \boldsymbol{v}^{\pi_t}$ and $\lim_{\tau \to \infty} \tilde{\boldsymbol{v}}_\tau = \boldsymbol{v}^{\pi_{t+1}}$.[6] First, observe that for the first iteration of fixed-point iteration,

[6] using the convergence of fixed-point iteration (10.25)

$$\begin{aligned}
\tilde{v}_1(x) &= (\boldsymbol{B}^\star \boldsymbol{v}^{\pi_t})(x) && \text{using that } \pi_{t+1} \text{ is greedy wrt. } v^{\pi_t} \\
&= \max_{a \in A} q^{\pi_t}(x, a) && \text{using the definition of the Bellman update (10.36)} \\
&\geq q^{\pi_t}(x, \pi_t(x)) \\
&= v^{\pi_t}(x) \\
&= \tilde{v}_0(x). && \text{using (10.17)}
\end{aligned}$$

Let us now consider a single iteration of fixed-point iteration. We have,

$$\tilde{v}_{\tau+1}(x) = r(x, \pi_{t+1}(x)) + \gamma \sum_{x' \in X} p(x' \mid x, \pi_{t+1}(x)) \cdot \tilde{v}_\tau(x'). \qquad \text{using the definition of } \tilde{v}_{\tau+1} \text{ (10.21)}$$

Using an induction on $\tau$, we conclude,

$$\geq r(x, \pi_{t+1}(x)) + \gamma \sum_{x' \in X} p(x' \mid x, \pi_{t+1}(x)) \cdot \tilde{v}_{\tau-1}(x') \qquad \text{using the induction hypothesis, } \tilde{v}_\tau(x') \geq \tilde{v}_{\tau-1}(x')$$
$$= \tilde{v}_\tau(x).$$

This establishes the first claim,

$$\boldsymbol{v}^{\pi_{t+1}} = \lim_{\tau \to \infty} \tilde{\boldsymbol{v}}_\tau \geq \tilde{\boldsymbol{v}}_0 = \boldsymbol{v}^{\pi_t}. \tag{10.37}$$

For the second claim, recall from Bellman's theorem (10.32) that $v^\star$ is a (unique) fixed-point of the Bellman update $\boldsymbol{B}^\star$.[7] In particular, we have $v^{\pi_{t+1}} \equiv v^{\pi_t}$ if and only if $v^{\pi_{t+1}} \equiv v^{\pi_t} \equiv v^\star$. In other words, if $v^{\pi_t} \not\equiv v^\star$ then Equation (10.37) is strict for at least one $x \in X$ and $v^{\pi_{t+1}} \not\equiv v^{\pi_t}$. This proves the strict monotonic improvement of policy iteration. □

[7] We will show in Equation (10.38) that $\boldsymbol{B}^\star$ is a contraction, implying that $v^\star$ is the *unique* fixed-point of $\boldsymbol{B}^\star$.

**Theorem 10.16** (Convergence of policy iteration)**.** *For finite Markov decision processes, policy iteration converges to an optimal policy.*

*Proof.* Finite Markov decision processes only have a finite number of deterministic policies (albeit exponentially many). Observe that policy iteration only considers deterministic policies, and recall that there is an optimal policy that is deterministic. As the value of policies strictly increase in each iteration until an optimal policy is found, policy iteration must converge in finite time. □

It can be shown that policy iteration converges to an exact solution in a polynomial number of iterations (Ye, 2011). Each iteration of policy iteration requires computing the value function, which we have seen to be of cubic complexity in the number of states.

### *10.3.4 Value Iteration*

As we have mentioned, another natural approach of finding the optimal policy is to interpret $v^\star$ as the fixed point of the Bellman update. Recall our definition of the Bellman update from Equation (10.36),

$$(\boldsymbol{B}^\star \boldsymbol{v})(x) = \max_{a \in A} q(x, a),$$

where $q$ was the state-action value function associated with the state value function $\boldsymbol{v}$. The value iteration algorithm is shown in Algorithm 10.17.

We will now prove the convergence of value iteration using the fixed-point interpretation.

**Algorithm 10.17:** Value iteration

initialize $v(x) \leftarrow \max_{a \in A} r(x, a)$ for each $x \in X$
**for** $t = 1$ **to** $\infty$ **do**
  $v(x) \leftarrow (\boldsymbol{B}^\star \boldsymbol{v})(x) = \max_{a \in A} q(x, a)$ for each $x \in X$
choose $\pi_v$

**Theorem 10.18** (Convergence of value iteration)**.** *Value iteration converges asymptotically to an optimal policy.*

*Proof.* Clearly, value iteration converges if $v^\star$ is the unique fixed-point of $\boldsymbol{B}^\star$. We already know from Bellman's theorem (10.32) that $v^\star$ is a fixed-point of $\boldsymbol{B}^\star$. It remains to show that it is indeed the unique fixed-point.

Analogously to our proof of the convergence of fixed-point iteration to the value function $v^\pi$, we show that $\boldsymbol{B}^\star$ is a contraction. Fix arbitrary $\boldsymbol{v}, \boldsymbol{v}' \in \mathbb{R}^n$, then

$$
\begin{aligned}
\left\| \boldsymbol{B}^\star \boldsymbol{v} - \boldsymbol{B}^\star \boldsymbol{v}' \right\|_\infty &= \max_{x \in X} \left| (\boldsymbol{B}^\star \boldsymbol{v})(x) - (\boldsymbol{B}^\star \boldsymbol{v}')(x) \right| && \text{using the definition of the } L_\infty \text{ norm (10.23)} \\
&= \max_{x \in X} \left| \max_{a \in A} q(x, a) - \max_{a \in A} q'(x, a) \right| && \text{using the definition of the Bellman update (10.36)} \\
&\leq \max_{x \in X} \max_{a \in A} \left| q(x, a) - q'(x, a) \right| && \text{using } |\max_x f(x) - \max_x g(x)| \leq \max_x |f(x) - g(x)| \\
&\leq \gamma \max_{x \in X} \max_{a \in A} \sum_{x' \in X} p(x' \mid x, a) \left| \boldsymbol{v}(x') - \boldsymbol{v}'(x') \right| && \text{using the definition of the Q-function (10.9) and } |\textstyle\sum_i a_i| \leq \sum_i |a_i| \\
&\leq \gamma \left\| \boldsymbol{v} - \boldsymbol{v}' \right\|_\infty && \text{using } \textstyle\sum_{x' \in X} p(x' \mid x, a) = 1 \text{ and } |\boldsymbol{v}(x') - \boldsymbol{v}'(x')| \leq \|\boldsymbol{v} - \boldsymbol{v}'\|_\infty
\end{aligned} \tag{10.38}
$$

where $q$ and $q'$ are the state-action value functions associated with $\boldsymbol{v}$ and $\boldsymbol{v}'$, respectively. By Equation (10.22), $\boldsymbol{B}^\star$ is a contraction and by Banach's fixed-point theorem $v^\star$ is its unique fixed-point. □

**Remark 10.19: Value iteration as a dynamic program**

Let us denote by $v_t$ the value function estimate after the $t$-th iteration. Observe that $v_t(x)$ corresponds to the maximum expected reward when starting in state $x$ and the "world ends" after $t$ time steps. In particular, $v_0$ corresponds to the maximum immediate reward. This suggests a different perspective on value iteration (akin to dynamic programming) where in each iteration we extend the time horizon of our approximation by one time step.

For any $\epsilon > 0$, value iteration converges to an $\epsilon$-optimal solution in polynomial time. However, unlike policy iteration, value iteration does

not generally reach the *exact* optimum in a finite number of iterations. Recalling the update rule of value iteration, its main benefit is that each iteration only requires a sum over all possible actions $a$ in state $x$ and a sum over all reachable states $x'$ from $x$. In sparse Markov decision processes,[8] an iteration of value iteration can be performed in (virtually) constant time.

[8] Sparsity refers to the interconnectivity of the state space. When only few states are reachable from any state, we call an MDP sparse.

## 10.4 Partial Observability

So far we have focused on the fully observable setting. That is, at any time, our agent knows its current state. We have seen that we can efficiently find the optimal policy (as long as the Markov decision process is finite).

We have already encountered the partially observable setting in Chapter 3, where we discussed filtering. In this section, we consider how Markov decision processes can be extended to a partially observable setting where the agent can only access noisy observations $Y_t$ of its state $X_t$.

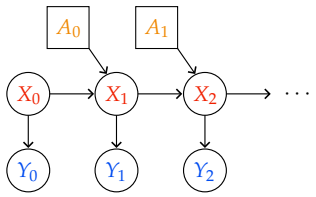



Figure 10.4: Directed graphical model of a partially observable Markov decision process with hidden states $X_t$, observables $Y_t$, and actions $A_t$.

**Definition 10.20** (Partially observable Markov decision process, POMDP)**.** Similarly to a Markov decision process, a *partially observable Markov decision process* is specified by

- a set of *states* $X$,
- a set of *actions* $A$,
- *transition probabilities* $p(x' \mid x, a)$, and
- a *reward function* $r : X \times A \to \mathbb{R}$.

Additionally, it is specified by

- a set of *observations* $Y$, and
- *observation probabilities*

$$o(y \mid x) \doteq \mathbb{P}(Y_t = y \mid X_t = x). \tag{10.39}$$

Whereas MDPs are controlled Markov chains, POMDPs are controlled hidden Markov models.

**Remark 10.21: Hidden Markov models**
A hidden Markov model is a Markovian process with unobservable states $X_t$ and observations $Y_t$ that depend on $X_t$ in a known way.

**Definition 10.22** (Hidden Markov model, HMM)**.** A *hidden Markov model* is specified by

- a set of *states* $X$,
- *transition probabilities* $p(x' \mid x) \doteq \mathbb{P}(X_{t+1} = x' \mid X_t = x)$ (also called *motion model*), and

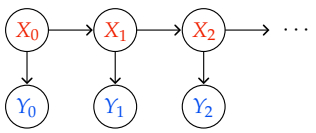



Figure 10.5: Directed graphical model of a hidden Markov model with hidden states $X_t$ and observables $Y_t$.

- a *sensor model* $o(y \mid x) \doteq \mathbb{P}(Y_t = y \mid X_t = x)$.

Following from its directed graphical model shown in Figure 10.5, its joint probability distribution factorizes into

$$\mathbb{P}(x_{0:t}, y_{0:t}) = \mathbb{P}(x_0) \cdot o(y_0 \mid x_0) \cdot \prod_{i=1}^{t} p(x_i \mid x_{i-1}) \cdot o(y_i \mid x_i). \tag{10.40}$$

Observe that a Kalman filter can be viewed as a hidden Markov model with conditional linear Gaussian motion and sensor models and a Gaussian prior on the initial state. In particular, the tasks of *filtering*, *smoothing*, and *predicting* which we discussed extensively in Chapter 3 are also of interest for hidden Markov models.

A widely used application of hidden Markov models is to find the most likely sequence (also called *most likely explanation*) of hidden states $x_{0:t}$ given a series of observations $y_{0:t}$,[9] that is, to find

$$\arg\max_{x_{0:t}} \mathbb{P}(x_{0:t} \mid y_{0:t}). \tag{10.41}$$

This task can be solved in linear time by a simple backtracking algorithm known as the *Viterbi algorithm*.

[9] This is useful in many applications such as speech recognition, decoding data that was transmitted over a noisy channel, beat detection, and many more.

POMDPs are a very powerful model, but hard to solve in general. A POMDP can be reduced to a Markov decision process with an enlarged state space. The key insight is to consider an MDP whose states are the *beliefs*,

$$b_t(x) \doteq \mathbb{P}(X_t = x \mid y_{0:t}, a_{0:t-1}), \tag{10.42}$$

about the current state in the POMDP. In other words, the states of the MDP are probability distributions over the states of the POMDP. We will make this more precise in the following.

Let us assume that our prior belief about the state of our agent is given by $\mathbb{P}(X_0 = x)$. Then, by Bayes' rule (1.45), the initial belief of our agent is $b_0(x) \propto o(y_0 \mid x) \cdot \mathbb{P}(X_0 = x)$. Keeping track of how beliefs change over time is known as *filtering*, which we already encountered in Section 3.1. Given a prior belief $b_t$, an action taken $a_t$, and a new observation $y_{t+1}$, the belief state can be updated as follows,

$$\begin{aligned} b_{t+1}(x) &= \mathbb{P}(X_{t+1} = x \mid y_{0:t+1}, a_{0:t}) && \text{by the definition of beliefs (10.42)} \\ &\propto \mathbb{P}(y_{t+1} \mid X_{t+1} = x) \cdot \mathbb{P}(X_{t+1} = x \mid y_{0:t}, a_{0:t}) && \text{using Bayes' rule (1.45)} \\ &= o(y_{t+1} \mid x) \cdot \mathbb{P}(X_{t+1} = x \mid y_{0:t}, a_{0:t}) && \text{using the definition of observation probabilities (10.39)} \end{aligned}$$

$$= o(y_{t+1} \mid x) \sum_{x' \in X} p(x \mid x', a_t) \cdot \mathbb{P}\big(X_t = x' \mid y_{0:t}, a_{0:t-1}\big)$$ by conditioning on the previous state $x'$, noting $a_t$ does not influence $X_t$

$$= o(y_{t+1} \mid x) \sum_{x' \in X} p(x \mid x', a_t) \cdot b_t(x'). \tag{10.43}$$ using the definition of beliefs (10.42)

Thus, the updated belief state is a deterministic mapping from the previous belief state depending only on the (random) observation $y_{t+1}$ and the taken action $a_t$. Note that this obeys a Markovian structure of transition probabilities with respect to the beliefs $b_t$.

The sequence of belief-states defines the sequence of random variables $(B_t)_{t \in \mathbb{N}_0}$,

$$B_t \doteq X_t \mid y_{0:t}, a_{0:t-1}, \tag{10.44}$$

where the (state-)space of all beliefs is the (infinite) space of all probability distributions over $X$,[10]

$$\mathcal{B} \doteq \Delta^X \doteq \Big\{ \boldsymbol{b} \in \mathbb{R}^{|X|} : \boldsymbol{b} \geq \mathbf{0}, \textstyle\sum_{i=1}^{|X|} \boldsymbol{b}(i) = 1 \Big\}. \tag{10.45}$$

[10] This definition naturally extends to continuous state spaces $\mathcal{X}$.

A Markov decision process, where every belief corresponds to a state is called a belief-state MDP.

**Definition 10.23** (Belief-state Markov decision process)**.** Given a POMDP, the corresponding *belief-state Markov decision process* is a Markov decision process specified by

- the *belief space* $\mathcal{B} \doteq \Delta^X$ depending on the *hidden states* $X$,
- the set of *actions* $A$,
- *transition probabilities*

$$\tau(b' \mid b, a) \doteq \mathbb{P}\big(B_{t+1} = b' \mid B_t = b, A_t = a\big), \tag{10.46}$$

- and *rewards*

$$\rho(b, a) \doteq \mathbb{E}_{x \sim b}[r(x, a)] = \sum_{x \in X} b(x) r(x, a). \tag{10.47}$$

It remains to derive the transition probabilities $\tau$ in terms of the original POMDP. We have,

$$\begin{aligned} \tau(b_{t+1} \mid b_t, a_t) &= \mathbb{P}(b_{t+1} \mid b_t, a_t) \\ &= \sum_{y_{t+1} \in Y} \mathbb{P}(b_{t+1} \mid b_t, a_t, y_{t+1}) \mathbb{P}(y_{t+1} \mid b_t, a_t). \end{aligned} \tag{10.48}$$ by conditioning on $y_{t+1} \in Y$

Using the Markovian structure of the belief updates, we naturally set,

$$\mathbb{P}(b_{t+1} \mid b_t, a_t, y_{t+1}) \doteq \begin{cases} 1 & \text{if } b_{t+1} \text{ matches the belief update} \\ & \text{of Equation (10.42) given } b_t, a_t, \text{ and} \\ & y_{t+1}, \\ 0 & \text{otherwise.} \end{cases} \tag{10.49}$$

The final missing piece is the likelihood of an observation $y_{t+1}$ given the prior belief $b_t$ and action $a_t$, which using our interpretation of beliefs corresponds to

$$\begin{aligned}\mathbb{P}(y_{t+1} \mid b_t, a_t) &= \mathbb{E}_{x \sim b_t}\Big[\mathbb{E}_{x' \mid x, a_t}\big[\mathbb{P}\big(y_{t+1} \mid X_{t+1} = x'\big)\big]\Big] \\ &= \mathbb{E}_{x \sim b_t}\Big[\mathbb{E}_{x' \mid x, a_t}\big[o(y_{t+1} \mid x')\big]\Big] \\ &= \sum_{x \in X} b_t(x) \sum_{x' \in X} p(x' \mid x, a_t) \cdot o(y_{t+1} \mid x'). \end{aligned} \tag{10.50}$$

using the definition of observation probabilities (10.39)

In principle, we can now apply arbitrary algorithms for planning in MDPs to POMDPs. Of course, the problem is that there are infinitely many beliefs, even for a finite state space $X$.[11] The belief-state MDP has therefore an infinitely large belief space $\mathcal{B}$. Even when only planning over finite horizons, exponentially many beliefs can be reached. So the belief space blows-up very quickly.

We will study MDPs with large state spaces (where transition dynamics and rewards are unknown) in Chapters 12 and 13. Similar methods can also be used to approximately solve POMDPs.

A key idea in approximate solutions to POMDPs is that most belief states are never reached. A common approach is to discretize the belief space by sampling or by applying a dimensionality reduction. Examples are *point-based value iteration* (PBVI) and *point-based policy iteration* (PBPI) (Shani et al., 2013).

[11] You can think of an $|X|$-dimensional space. Here, all points whose coordinates sum to 1 correspond to probability distributions (i.e., beliefs) over the hidden states $X$. The convex hull of these points is also known as the $(|X|-1)$-dimensional *probability simplex* (cf. Section A.1.2). Now, by definition of the $(|X|-1)$-dimensional probability simplex as a polytope in $|X|-1$ dimensions, we can conclude that its boundary consists of infinitely many points in $|X|$ dimensions. Noting that these points corresponded to the probability distributions on $|X|$, we conclude that there are infinitely many such distributions.

## *Discussion*

Even though we focus on the fully observed setting throughout this manuscript, the partially observed setting can be reduced to the fully observed setting with very large state spaces. In the next chapter, we will consider learning and planning in unknown Markov decision processes (i.e., reinforcement learning) for small state spaces. The setting of small state and action spaces is also known as the *tabular setting*. Then, in the final two chapters, we will consider approximate methods for large state and action spaces. In particular, in Section 13.1, we will revisit the problem of probabilistic planning in known Markov decision processes, but with continuous state and action spaces.

## *Problems*

**10.1. Value functions.**

Recall the example of "becoming rich and famous" from Figure 10.2. Consider the policy, $\pi \equiv S$ (i.e., to always *save*) and let $\gamma = 1/2$. Show that the (rounded) state-action value function $q^\pi$ is as follows:

| | save | advertise |
|---|---|---|
| poor, unknown | **0** | 0.1 |
| poor, famous | **4.4** | 1.2 |
| rich, famous | **17.8** | 1.2 |
| rich, unknown | **13.3** | 0.1 |

Shown in bold is the state value function $v^\pi$.

**10.2. Greedy policies.**

Show that if $q$ and $v$ arise from the same policy, that is, $q$ is defined in terms of $v$ as per Equation (10.9), then

$$\pi_v \equiv \pi_q. \tag{10.51}$$

This implies that we can use $v$ and $q$ interchangeably.

**10.3. Optimal policies.**

Again, recall the example of "becoming rich and famous" from Figure 10.2.

1. Show that the policy $\pi \equiv S$, which we considered in Problem 10.1, is not optimal.
2. Instead, consider the policy

$$\pi' \equiv \begin{cases} A & \text{if poor and unknown} \\ S & \text{otherwise} \end{cases}$$

   and let $\gamma = 1/2$. Show that the (rounded) state-action value function $q^{\pi'}$ is as follows:

| | save | advertise |
|---|---|---|
| poor, unknown | 0.8 | **1.6** |
| poor, famous | **4.5** | 1.2 |
| rich, famous | **17.8** | 1.2 |
| rich, unknown | **13.4** | 0.2 |

   Shown in bold is the state value function $v^{\pi'}$.
3. Is the policy $\pi'$ optimal?

**10.4. Linear convergence of policy iteration.**

Denote by $\pi_t$ the policy obtained by policy iteration after $t$ iterations. Use that the Bellman operator $\boldsymbol{B}^\star$ is a contraction with the unique fixed-point $\boldsymbol{v}^\star$ to show that

$$\|\boldsymbol{v}^{\pi_t} - \boldsymbol{v}^\star\|_\infty \leq \gamma^t \|\boldsymbol{v}^{\pi_0} - \boldsymbol{v}^\star\|_\infty \tag{10.52}$$

where $\boldsymbol{v}^\pi$ and $\boldsymbol{v}^\star$ are vector representations of the functions $v^\pi$ and $v^\star$, respectively.

*Hint: Recall from Theorem 10.15 that* $\boldsymbol{v}^{\pi_{t+1}} \geq \boldsymbol{B}^\star \boldsymbol{v}^{\pi_t} \geq \boldsymbol{v}^{\pi_t}$.

**10.5. Reward modification.**

A key technique for solving sequential decision problems is the modification of reward functions that leaves the optimal policy unchanged while improving sample efficiency or convergence rates. This exercise looks at simple ways of modifying rewards and understanding how these modifications affect the optimal policy.

Consider two Markov decision processes $M \doteq (X, A, p, r)$ and $M' \doteq (X, A, p, r')$ where the reward function $r$ is modified to obtain $r'$, and the rewards are bounded and discounted by the discount factor $\gamma \in [0, 1)$. Let $\pi^\star_M$ be the optimal policy for $M$.

1. Suppose $r'(x) = \alpha r(x)$, where $\alpha > 0$. Show that the optimal policy $\pi^\star$ of $M$ is also an optimal policy of $M'$.
2. Given a modification of the form $r'(x) = r(x) + c$, where $c > 0$ is a constant scalar, show that the optimal policy $\pi^\star_M$ can be different from $\pi^\star_{M'}$.
3. Another way of modifying the reward function is through *reward shaping* where one supplies additional rewards to the agent to guide the learning process. When one has no knowledge of the underlying transition dynamics $p$, a commonly used transformation is $r'(x, x') = r(x, x') + f(x, x')$ where $f$ is a *potential-based* shaping function defined as

$$f(x, x') \doteq \gamma\phi(x') - \phi(x), \quad \phi : X \to \mathbb{R}. \tag{10.53}$$

   Show that the optimal policy remains unchanged under this definition of $f$.

**10.6. A partially observable fishing problem.**

We model an angler's decisions while fishing, where the states are partially observable. There are two states: (1) **Fish** ($F$): A fish is hooked on the line. (2) **No fish** ($\overline{F}$): No fish is hooked on the line.

The angler can choose between two actions:

- **Pull up the rod** ($P$): If there is a fish on the line ($F$), there is a 90% chance of catching it (reward $+10$, transitioning to $\overline{F}$) and a 10% chance of it escaping (reward $-1$, transitioning to $\overline{F}$). If there is no fish ($\overline{F}$), pulling up the rod results in no catch, staying in $\overline{F}$ with a reward of $-5$.
- **Waiting** ($W$): All waiting actions result in a reward of $-1$. In state $F$, there is a 60% chance of the fish staying (remaining in $F$) and a 40% chance of it escaping (transitioning to $\overline{F}$). In state $\overline{F}$, there is a 50% chance of a fish biting (transitioning to $F$) and a 50% chance of no change (remaining in $\overline{F}$).

*Suggestion: Draw the MDP transition diagram. Draw each transition with action, associated probability, and associated reward.*

Since the angler cannot directly observe whether there is a fish on the line, they receive a noisy observation about the state. This observation can be:

- $o_1$: The signal suggests that a fish might be on the line.
- $o_2$: The signal suggests that there is no fish on the line.

The observation model, which defines the probability of receiving each observation given the true state is as follows:

| | $\mathbb{P}(o_1 \mid \cdot)$ | $\mathbb{P}(o_2 \mid \cdot)$ |
|---|---|---|
| $F$ | 0.8 | 0.2 |
| $\overline{F}$ | 0.3 | 0.7 |

The angler's goal is to choose actions that maximize their overall reward, balancing the chances of catching a fish against the cost of waiting and unsuccessful pulls.

1. Given an initial belief $b_0(F) = b_0(\overline{F}) = 0.5$, the angler chooses to wait and observes $o_1$. Compute the updated belief $b_1$ using the observation model and belief update equation (10.42).
2. Given belief $b_1(F) \approx 0.765$ and $b_1(\overline{F}) \approx 0.235$, compute the updated belief $b_2$ for both actions $P$ (pull) and $W$ (wait), both in the case where you observe $o_1$ (fish likely) and $o_2$ (fish unlikely).

# 11

# *Tabular Reinforcement Learning*

## *11.1 The Reinforcement Learning Problem*

Reinforcement learning is concerned with probabilistic planning in unknown environments. This extends our study of known environments in the previous chapter. Those environments are still modeled by Markov decision processes, but in reinforcement learning, we do not know the dynamics $p$ and rewards $r$ in advance. Hence, reinforcement learning is at the intersection of the theories of probabilistic planning (i.e., Markov decision processes) and learning (e.g., multi-armed bandits), which we covered extensively in the previous chapters.

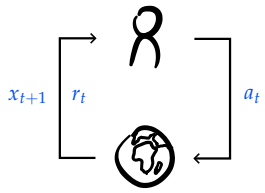



Figure 11.1: In reinforcement learning, an agent interacts with its environment in a sequence of rounds. After playing an action $a_t$, it observes rewards $r_t$ and its new state $x_{t+1}$. The agent then uses this information to learn how to act to maximize reward.

We will continue to focus on the fully observed setting, where the agent knows its current state. As we have seen in the previous section, the partially observed setting corresponds to a fully observed setting with an enlarged state space. In this chapter, we will begin by considering reinforcement learning with small state and action spaces. This setting is often called the *tabular setting*, as the value functions can be computed exhaustively for all states and stored in a table.

Clearly, the agent needs to trade exploring and learning about the environment with exploiting its knowledge to maximize rewards. Thus, the exploration-exploitation dilemma, which was at the core of Bayesian optimization (see Section 9.1), also plays a crucial role in reinforcement learning. In fact, Bayesian optimization can be viewed as reinforcement learning with a fixed state: In each round, the agent plays an action, aiming to find the action that maximizes the reward. However, playing the same action multiple times yields the same reward, implying that we remain in a single state. In the context of Bayesian optimization, we used "regret" as performance metric: in the jargon of planning, minimizing regret corresponds to maximizing the cumulative reward.

Another key challenge of reinforcement learning is that the observed data is dependent on the played actions. This is in contrast to the setting of supervised learning that we have been considering in earlier chapters, where the data is sampled independently.

### 11.1.1 Trajectories

The data that the agent collects is modeled using so-called trajectories.

**Definition 11.1** (Trajectory)**.** A *trajectory* $\tau$ is a (possibly infinite) sequence,

$$\tau \doteq (\tau_0, \tau_1, \tau_2, \dots), \tag{11.1}$$

of *transitions*,

$$\tau_i \doteq (x_i, a_i, r_i, x_{i+1}), \tag{11.2}$$

where $x_i \in X$ is the starting state, $a_i \in A$ is the played action, $r_i \in \mathbb{R}$ is the attained reward, and $x_{i+1} \in X$ is the ending state.

In the context of learning a dynamics and rewards model, $x_i$ and $a_i$ can be understood as inputs, and $r_i$ and $x_{i+1}$ can be understood as labels of a regression problem.

Crucially, the newly observed states $x_{t+1}$ and the rewards $r_t$ (across multiple transitions) are conditionally independent given the previous states $x_t$ and actions $a_t$. This follows directly from the Markovian structure of the underlying Markov decision process.[1] Formally, we have,

$$X_{t+1} \perp X_{t'+1} \mid X_t, X_{t'}, A_t, A_{t'}, \tag{11.3a}$$
$$R_t \perp R_{t'} \mid X_t, X_{t'}, A_t, A_{t'}, \tag{11.3b}$$

for any $t, t' \in \mathbb{N}_0$. In particular, if $x_t = x_{t'}$ and $a_t = a_{t'}$, then $x_{t+1}$ and $x_{t'+1}$ are independent samples according to the transition model $p(X_{t+1} \mid x_t, a_t)$. Analogously, if $x_t = x_{t'}$ and $a_t = a_{t'}$, then $r_t$ and $r_{t'}$ are independent samples of the reward model $r(x_t, a_t)$. As we will see later in this chapter and especially in Chapter 13, this independence property is crucial for being able to learn about the underlying Markov decision process. Notably, this implies that we can apply the law of large numbers (A.36) and Hoeffding's inequality (A.41) to our estimators of both quantities.

[1] Recall the Markov property (6.6), which assumes that in the underlying Markov decision process (i.e., in our environment) the future state of an agent is independent of past states given the agent's current state. This is commonly called a Markovian structure. From this Markovian structure, we gather that repeated encounters of state-action pairs result in independent trials of the transition model and rewards.

The collection of data is commonly classified into two settings. In the *episodic setting*, the agent performs a sequence of "training" rounds (called *episodes*). In the beginning of each episode, the agent is reset to some initial state. In contrast, in the *continuous setting* (or non-episodic /

online setting), the agent learns online. Especially, every action, every reward, and every state transition counts.

The episodic setting is more applicable to an agent playing a computer game. That is, the agent is performing in a simulated environment that is easy to reset. The continuous setting is akin to an agent that is deployed to the "real world". In principle, real-world agents can be trained in simulated environments before being deployed. However, this bears the risk of learning to exploit or rely on features of the simulated environment that are not present in the real environment. Sometimes, using a simulated environment for training is downright impossible, as the real environment is too complex.

### 11.1.2 On-policy and Off-policy Methods

Another important distinction in how data is collected, is the distinction between on-policy and off-policy methods. As the names suggest, *on-policy* methods are used when the agent has control over its own actions, in other words, the agent can freely choose to follow any policy. Being able to follow a policy is helpful, for example because it allows the agent to experiment with trading exploration and exploitation.

In contrast, *off-policy* methods can be used even when the agent cannot freely choose its actions. Off-policy methods are therefore able to make use of purely observational data. This might be data that was collected by another agent, a fixed policy, or during a previous episode. Off-policy methods are therefore more *sample-efficient* than on-policy methods. This is crucial, especially in settings where conducting experiments (i.e., collecting new data) is expensive.

## 11.2 Model-based Approaches

Approaches to reinforcement learning are largely categorized into two classes. *Model-based* approaches aim to learn the underlying Markov decision process. More concretely, they learn models of the dynamics $p$ and rewards $r$. They then use these models to perform planning (i.e., policy optimization) in the underlying Markov decision process. In contrast, *model-free* approaches learn the value function directly. We begin by discussing model-based approaches to the tabular setting. In Section 11.4, we cover model-free approaches.

### 11.2.1 Learning the Underlying Markov Decision Process

Recall that the underlying Markov decision process was specified by its dynamics $p(x' \mid x, a)$ that correspond to the probability of entering

state $x' \in X$ when playing action $a \in A$ from state $x \in X$, and its rewards $r(x, a)$ for playing action $a \in A$ in state $x \in X$. A natural first idea is to use maximum likelihood estimation to approximate these quantities.

We can think of each transition $x' \mid x, a$ as sampling from a categorical random variable of which we want to estimate the success probabilities for landing in each of the states. Therefore, as we have seen in Example A.7, the MLE of the dynamics model coincides with the sample mean,

$$\hat{p}(x' \mid x, a) = \frac{N(x' \mid x, a)}{N(a \mid x)} \tag{11.4}$$

where $N(x' \mid x, a)$ counts the number of transitions from state $x$ to state $x'$ when playing action $a$ and $N(a \mid x)$ counts the number of transitions that start in state $x$ and play action $a$ (regardless of the next state). Similarly, for the rewards model, we obtain the following maximum likelihood estimate (i.e., sample mean),

$$\hat{r}(x, a) = \frac{1}{N(a \mid x)} \sum_{\substack{t=0 \\ x_t = x \\ a_t = a}}^{\infty} r_t. \tag{11.5}$$

It is immediate that both estimates are unbiased as both correspond to a sample mean.

Still, for the models of our environment to become accurate, our agent needs to visit *each* state-action pair $(x, a)$ numerous times. Note that our estimators for dynamics and rewards are only well-defined when we visit the corresponding state-action pair at least once. However, in a stochastic environment, a single visit will likely not result in an accurate model. We can use Hoeffding's inequality (A.41) to gauge how accurate the estimates are after only a limited number of visits.

## 11.3 Balancing Exploration and Exploitation

The next natural question is how to use our current model of the environment to pick actions such that exploration and exploitation are traded effectively. This is what we will consider next.

Given the estimated MDP given by $\hat{p}$ and $\hat{r}$, we can compute the optimal policy using either policy iteration or value iteration. For example, using value iteration, we can compute the optimal state-action value function $Q^\star$ within the *estimated* MDP, and then employ the greedy policy

$$\pi(x) = \arg\max_{a \in A} Q^\star(x, a). \tag{11.6}$$

Recall from Equation (10.31) that this corresponds to always picking the best action under the *current* model (that is, $\pi$ is the optimal policy). But since the model is inaccurate, while potentially quickly generating some reward, we will likely get stuck in a suboptimal state.

### 11.3.1 $\varepsilon$-greedy

Consider the other extreme: If we always pick a random action, we will eventually(!) estimate the dynamics and rewards correctly, yet we will do extremely poorly in terms of maximizing rewards along the way. To trade exploration and exploitation, a natural idea is to balance these two extremes.

Arguably, the simplest idea is the following: At each time step, throw a biased coin. If this coin lands heads, we pick an action uniformly at random among all actions. If the coin lands tails, we pick the best action under our current model. This algorithm is called *$\varepsilon$-greedy*, where the probability of a coin landing heads at time $t$ is $\varepsilon_t$.

**Algorithm 11.2:** $\varepsilon$-greedy

```
for t = 0 to ∞ do
    sample u ∈ Unif([0, 1])
    if u ≤ ε_t then pick action uniformly at random among all actions
    else pick best action under the current model
```

The $\varepsilon$-greedy algorithm provides a general framework for addressing the exploration-exploitation dilemma. When the underlying MDP is learned using Monte Carlo estimation as we discussed in Section 11.2.1, the resulting algorithm is known as *Monte Carlo control*. However, the same framework can also be used in the model-free setting where we pick the best action without estimating the full underlying MDP. We discuss this approach in greater detail in Section 11.4.

Amazingly, this simple algorithm already works quite well. Nevertheless, it can clearly be improved. The key problem of $\varepsilon$-greedy is that it explores the state space in an uninformed manner. In other words, it explores ignoring all past experience. It thus does not eliminate clearly suboptimal actions. This is a problem, especially as we typically have many state-action pairs and recalling that we have to explore each such pair many times to learn an accurate model.

**Remark 11.3: Asymptotic convergence**

It can be shown that Monte Carlo control converges to an optimal policy (albeit slowly) almost surely when the learned policy is "greedy in the limit with infinite exploration".

**Definition 11.4** (Greedy in the limit with infinite exploration, GLIE)**.** A sequence of policies $\pi_t$ is said to be *greedy in the limit with infinite exploration* if

1. all state-action pairs are explored infinitely many times,[2]

$$\lim_{t\to\infty} N_t(x,a) = \infty \quad \text{and} \tag{11.7}$$

2. the policy converges to a greedy policy,

$$\lim_{t\to\infty} \pi_t(a \mid x) = \mathbb{1}\{a = \arg\max_{a'\in A} Q_t^\star(x,a')\} \tag{11.8}$$

where we denote by $N_t(x,a)$ the number of transitions from state $x$ playing action $a$ until time $t$, and $Q_t^\star$ is the optimal state-action value function in the estimated MDP at time $t$.

Note that $\varepsilon$-greedy is GLIE with probability 1 if the sequence $(\varepsilon_t)_{t\in\mathbb{N}_0}$ satisfies the Robbins-Monro (RM) conditions (A.60),

$$\varepsilon_t \geq 0 \quad \forall t, \quad \sum_{t=0}^{\infty} \varepsilon_t = \infty \quad \text{and} \quad \sum_{t=0}^{\infty} \varepsilon_t^2 < \infty.$$

The RM-conditions are satisfied, for example, if $\varepsilon_t = 1/t$.

**Theorem 11.5** (Convergence of Monte Carlo control)**.** *GLIE Monte Carlo control converges to an optimal policy with probability* 1*.*

Intuitively, the probability of exploration converges to zero, and hence, the policy will "eventually coincide" with the greedy policy. Moreover, the greedy policy will "eventually coincide" with the optimal policy due to an argument akin to the convergence of policy iteration,[3] and using that each state-action pair is visited infinitely often.

[2] That all state-action pairs are chosen is a fundamental requirement. There is no reason why any algorithm would converge to the true value function for *all* states when it only sees some state-action pairs finitely many times, or even not at all.

[3] see Theorems 10.15 and 10.16

### 11.3.2 *Softmax Exploration*

An alternative to using $\varepsilon$-greedy for trading between greedy exploitation and uniform exploration is the so-called *softmax exploration* or *Boltzmann exploration*. Given the agent is in state $x$, we pick action $a$ with probability,

$$\pi_\lambda(a \mid x) \propto \exp\left(\frac{1}{\lambda} Q^\star(x,a)\right), \tag{11.9}$$

which is the Gibbs distribution with temperature parameter $\lambda > 0$. Observe that for $\lambda \to 0$, softmax exploration corresponds to greedily maximizing the Q-function (i.e., greedy exploitation), whereas for $\lambda \to \infty$, softmax exploration explores uniformly at random. This can outperform $\varepsilon$-greedy as the exploration is directed towards actions with larger estimated value.

### 11.3.3 *Optimism*

Recall from our discussion of multi-armed bandits in Section 9.2.1 that a key principle in effectively trading exploration and exploitation is *optimism in the face of uncertainty*. Let us apply this principle to the reinforcement learning setting. The key idea is to assume that the dynamics and rewards model "work in our favor" until we have learned "good estimates" of the true dynamics and rewards.

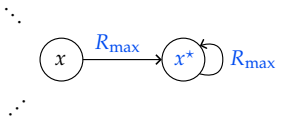



Figure 11.2: Illustration of the fairy-tale state of $R_{\max}$. If in doubt, the agent believes actions from the state $x$ to lead to the fairy-tale state $x^\star$ with maximal rewards. This encourages the exploration of unknown states.

More formally, if $r(x, a)$ is unknown, we set $\hat{r}(x, a) = R_{\max}$, where $R_{\max}$ is the maximum reward our agent can attain during a single transition. Similarly, if $p(x' \mid x, a)$ is unknown, we set $\hat{p}(x^\star \mid x, a) = 1$, where $x^\star$ is a "fairy-tale state". The fairy-tale state corresponds to everything our agent could wish for, that is,

$$\hat{p}(x^\star \mid x^\star, a) = 1 \qquad \forall a \in A, \tag{11.10}$$

$$\hat{r}(x^\star, a) = R_{\max} \qquad \forall a \in A. \tag{11.11}$$

In practice, the decision of when to assume that the learned dynamics and reward models are "good enough" has to be tuned.

In using these optimistic estimates of $p$ and $r$, we obtain an optimistic underlying Markov decision process that exhibits a bias towards exploration. In particular, the rewards attained in this MDP, are an upper bound of the true reward. The resulting algorithm is known as the $R_{\max}$ *algorithm*.

How many transitions are "enough"? We can use Hoeffding's inequality (A.41) to get a rough idea! The key here, is our observation from Equation (11.3) that the transitions and rewards are conditionally independent given the state-action pairs since, as we have discussed in Section 6.1.4 on the ergodic theorem, Hoeffding's inequality does not hold for dependent samples. In this case, Hoeffding's inequality tells us that for the absolute approximation error to be below $\epsilon$ with probability at least $1 - \delta$, we need

$$N(a \mid x) \geq \frac{R_{\max}^2}{2\epsilon^2} \log \frac{2}{\delta}. \tag{11.12}$$

see (A.42)

**Lemma 11.7** (Exploration and exploitation of $R_{\max}$)**.** *Every $T$ time steps, with high probability, $R_{\max}$ either*

**Algorithm 11.6:** $R_{\max}$ algorithm

add the fairy-tale state $x^\star$ to the Markov decision process
set $\hat{r}(x, a) = R_{\max}$ for all $x \in X$ and $a \in A$
set $\hat{p}(x^\star \mid x, a) = 1$ for all $x \in X$ and $a \in A$
compute the optimal policy $\hat{\pi}$ for $\hat{r}$ and $\hat{p}$
**for** $t = 0$ **to** $\infty$ **do**
    execute policy $\hat{\pi}$ (for some number of steps)
    for each visited state-action pair $(x, a)$, update $\hat{r}(x, a)$
    estimate transition probabilities $\hat{p}(x' \mid x, a)$
    after observing "enough" transitions and rewards, recompute the optimal policy $\hat{\pi}$ according the current model $\hat{p}$ and $\hat{r}$.

- *obtains near-optimal reward; or*
- *visits at least one unknown state-action pair.*[4]

*Here, $T$ depends on the mixing time of the Markov chain induced by the optimal policy.*

**Theorem 11.8** (Convergence of $R_{\max}$, Brafman and Tennenholtz (2002))**.** *With probability at least $1 - \delta$, $R_{\max}$ reaches an $\epsilon$-optimal policy in a number of steps that is polynomial in $|X|$, $|A|$, $T$, $1/\epsilon$, $1/\delta$, and $R_{\max}$.*

[4] Note that in the tabular setting, there are "only" polynomially many state-action pairs.

### 11.3.4 Challenges of Model-based Approaches

We have seen that the $R_{\max}$ algorithm performs remarkably well in the tabular setting. However, there are important computational limitations to the model-based approaches that we discussed so far.

First, observe that the (tabular) model-based approach requires us to store $\hat{p}(x' \mid x, a)$ and $\hat{r}(x, a)$ in a table. This table already has $O(n^2 m)$ entries. Even though polynomial in the size of the state and action spaces, this quickly becomes unmanageable.

Second, the model-based approach requires us to "solve" the learned Markov decision processes to obtain the optimal policy (using policy or value iteration). As we continue to learn over time, we need to find the optimal policy many times. $R_{\max}$ recomputes the policy after each state-action pair is observed sufficiently often, so $O(nm)$ times.

## 11.4 Model-free Approaches

In the previous section, we have seen that learning and remembering the model as well as planning within the estimated model can potentially be quite expensive in the model-based approach. We there-

fore turn to model-free methods that estimate the value function directly. Thus, they require neither remembering the full model nor planning (i.e., policy optimization) in the underlying Markov decision process. We will, however, return to model-based methods in Chapter 13 to see that promise lies in combining methods from model-based reinforcement learning with methods from model-free reinforcement learning.

A significant benefit to model-based reinforcement learning is that it is inherently off-policy. That is, any trajectory regardless of the policy used to obtain it can be used to improve the model of the underlying Markov decision process. In the model-free setting, this not necessarily true. By default, estimating the value function according to the data from a trajectory, will yield an estimate of the value function corresponding to the policy that was used to sample the data.

We will start by discussing on-policy methods and later see how the value function can be estimated off-policy.

### 11.4.1 On-policy Value Estimation

Let us suppose, our agent follows a fixed policy $\pi$. Then, the corresponding value function $v^\pi$ is given as

$$\begin{aligned} v^\pi(x) &= r(x, \pi(x)) + \gamma \sum_{x' \in X} p(x' \mid x, \pi(x)) \cdot v^\pi(x') && \text{using the definition of the value function (10.7)} \\ &= \mathbb{E}_{R_0, X_1}[R_0 + \gamma v^\pi(X_1) \mid X_0 = x, A_0 = \pi(x)] && \text{interpreting the above expression as an expectation over the random quantities } R_0 \text{ and } X_1 \end{aligned} \tag{11.13}$$

Our first instinct might be to use a Monte Carlo estimate of this expectation. Due to the conditional independence of the transitions (11.3), Monte Carlo approximation does yield an unbiased estimate,

$$\approx r + \gamma v^\pi(x'), \tag{11.14}$$

where the agent observed the transition $(x, a, r, x')$. Note that to estimate this expectation we use a single(!) sample,[5] unlike our previous applications of Monte Carlo sampling where we usually averaged over $m$ samples. However, there is one significant problem in this approximation. Our approximation of $v^\pi$ does in turn depend on the (unknown) true value of $v^\pi$!

[5] The idea is that we will use this approximation repeatedly as our agent collects new data to achieve the same effect as initially averaging over multiple samples.

The key idea is to use a bootstrapping estimate of the value function instead. That is, in place of the true value function $v^\pi$, we will use a "running estimate" $V^\pi$. In other words, whenever observing a new transition, we use our previous best estimate of $v^\pi$ to obtain a new estimate $V^\pi$. We already encountered bootstrapping briefly in Section 7.3.4 in the context of probabilistic ensembles in Bayesian deep learning. More generally, *bootstrapping* refers to approximating a true

quantity (e.g., $v^\pi$) by using an empirical quantity (e.g., $V^\pi$), which itself is constructed using samples from the true quantity that is to be approximated.

Due to its use in estimating the value function, bootstrapping is a core concept to model-free reinforcement learning. Crucially, using a bootstrapping estimate generally results in biased estimates of the value function. Moreover, due to relying on a single sample, the estimates from Equation (11.14) tend to have very large variance.

The variance of the estimate is typically reduced by mixing new estimates of the value function with previous estimates using a learning rate $\alpha_t$. This yields the *temporal-difference learning* algorithm.

**Algorithm 11.9:** Temporal-difference (TD) learning

initialize $V^\pi$ arbitrarily (e.g., as $\mathbf{0}$)
**for** $t = 0$ **to** $\infty$ **do**
  follow policy $\pi$ to obtain the transition $(x, a, r, x')$
  $V^\pi(x) \leftarrow (1 - \alpha_t) V^\pi(x) + \alpha_t (r + \gamma V^\pi(x'))$ // (11.15)

The update rule is sometimes written equivalently as

$$V^\pi(x) \leftarrow V^\pi(x) + \alpha_t (r + \gamma V^\pi(x') - V^\pi(x)). \tag{11.16}$$

Thus, the update to $V^\pi(x)$ is proportional to the learning rate and the difference between the previous estimate and the renewed estimate using the new observation.

**Theorem 11.10** (Convergence of TD-learning, Jaakkola et al. (1993))**.** *If $(\alpha_t)_{t \in \mathbb{N}_0}$ satisfies the RM-conditions* (A.60) *and all state-action pairs are chosen infinitely often, then $V^\pi$ converges to $v^\pi$ with probability* 1.

Importantly, note that due to the Monte Carlo approximation of Equation (11.13) with respect to transitions attained by following policy $\pi$, TD-learning is fundamentally on-policy. That is, for the estimates $V^\pi$ to converge to the true value function $v^\pi$, the transitions that are used for the estimation must follow policy $\pi$.

### 11.4.2 SARSA: On-policy Control

TD-learning merely estimates the value function of a fixed policy $\pi$. To find the optimal policy $\pi^\star$, we can use an analogue of policy iteration (see Algorithm 10.14). Here, it is more convenient to use an estimate of the state-action value function $q^\pi$ which can be obtained

analogously to the bootstrapping estimate of $v^\pi$ (11.14),

$$\begin{aligned} q^\pi(x,a) &= r(x,a) + \gamma \sum_{x' \in X} p(x' \mid x,a) \sum_{a' \in A} \pi(a' \mid x') q^\pi(x',a') && \text{using Bellman's expectation equation (10.15)} \\ &= \mathbb{E}_{R_0, X_1, A_1}[R_0 + \gamma q^\pi(X_1, A_1) \mid X_0 = x, A_0 = a] \qquad (11.17) && \text{interpreting the above expression as an expectation over } R_0, X_1 \text{ and } A_1 \\ &\approx r + \gamma q^\pi(x',a'), \qquad (11.18) && \text{Monte Carlo approximation with a single sample} \end{aligned}$$

where the agent observed transitions $(x, a, r, x')$ and $(x', a', r', x'')$.

The update rule from TD-learning is therefore adapted to[6]

$$Q^\pi(x,a) \leftarrow (1 - \alpha_t) Q^\pi(x,a) + \alpha_t (r + \gamma Q^\pi(x',a')). \qquad (11.19)$$

[6] Note that for deterministic policies $\pi$, $Q^\pi(x',a') = Q^\pi(x', \pi(x')) = V^\pi(x')$ if the transitions are obtained by following policy $\pi$.

This algorithm is known as *SARSA* (short for *state-action-reward-state-action*). Similar convergence guarantees to those of TD-learning can also be derived for SARSA.

**Theorem 11.11** (Convergence of SARSA, Singh et al. (2000))**.** *If* $(\alpha_t)_{t \in \mathbb{N}_0}$ *satisfies the RM-conditions* (A.60) *and all state-action pairs are chosen infinitely often, then* $Q^\pi$ *converges to* $q^\pi$ *with probability* 1.

The policy iteration scheme to identify the optimal policy can be outlined as follows: In each iteration $t$, we estimate the value function $q^{\pi_t}$ of policy $\pi_t$ with the estimate $Q^{\pi_t}$ obtained from SARSA. We then choose the greedy policy with respect to $Q^{\pi_t}$ as the next policy $\pi_{t+1}$. However, due to the on-policy nature of SARSA, we cannot reuse any data between the iterations. Moreover, it turns out that in practice, when using only finitely many samples, this form of greedily optimizing Markov decision processes does not explore enough. At least partially, this can be compensated for by injecting noise when choosing the next action, e.g., by following an $\varepsilon$-greedy policy or using softmax exploration.

### 11.4.3 *Off-policy Value Estimation*

Consider the following slight adaptation of the derivation of SARSA (11.18),

$$\begin{aligned} q^\pi(x,a) &= r(x,a) + \gamma \sum_{x' \in X} p(x' \mid x,a) \sum_{a' \in A} \pi(a' \mid x') q^\pi(x',a') && \text{using Bellman's expectation equation (10.15)} \\ &= \mathbb{E}_{R_0, X_1} \left[ R_0 + \gamma \sum_{a' \in A} \pi(a' \mid X_1) q^\pi(X_1, a') \,\middle|\, X_0 = x, A_0 = a \right] \qquad (11.20) && \text{interpreting the above expression as an expectation over } R_0 \text{ and } X_1 \\ &\approx r + \gamma \sum_{a' \in A} \pi(a' \mid x') q^\pi(x',a'), \qquad (11.21) && \text{Monte Carlo approximation with a single sample} \end{aligned}$$

where the agent observed the transition $(x, a, r, x')$. This yields the update rule,

$$Q^\pi(x,a) \leftarrow (1-\alpha_t)Q^\pi(x,a) + \alpha_t\left(r + \gamma \sum_{a' \in A} \pi(a' \mid x')Q^\pi(x',a')\right). \tag{11.22}$$

This adapted update rule *explicitly* chooses the subsequent action $a'$ according to policy $\pi$ whereas SARSA absorbs this choice into the Monte Carlo approximation. The algorithm has analogous convergence guarantees to those of SARSA.

Crucially, this algorithm is off-policy. That is, we can use transitions that were obtained according to *any* policy to estimate the value of a fixed policy $\pi$, which we may have never used! Perhaps this seems contradictory at first, but it is not. As noted, the key difference to the on-policy TD-learning and SARSA is that our estimate of the Q-function explicitly keeps track of the next-performed action. It does so for any action in any state. Moreover, note that the transitions that are due to the dynamics model and rewards are unaffected by the used policy. They merely depend on the originating state-action pair. We can therefore use the instances where other policies played action $\pi(x)$ in state $x$ to estimate the performance of $\pi$.

### 11.4.4 Q-learning: Off-policy Control

It turns out that there is a way to estimate the value function of the optimal policy directly. Recall from Bellman's theorem (10.31) that the optimal policy $\pi^\star$ can be characterized in terms of the optimal state-action value function $q^\star$,

$$\pi^\star(x) = \arg\max_{a \in A} q^\star(x,a).$$

$\pi^\star$ corresponds to greedily maximizing the value function.

Analogously to our derivation of SARSA (11.18), only using Bellman's theorem (10.32) in place of Bellman's expectation equation (10.15), we obtain,

$$\begin{aligned} q^\star(x,a) &= r(x,a) + \gamma \sum_{x' \in X} p(x' \mid x,a) \max_{a' \in A} q^\star(x',a') && \text{using that the Q-function is a fixed-point of the Bellman update, see Bellman's theorem (10.32)} \\ &= \mathbb{E}_{R_0, X_1}\left[R_0 + \gamma \max_{a' \in A} q^\star(X_1, a') \;\middle|\; X_0 = x, A_0 = a\right] && \text{interpreting the above expression as an expectation over } R_0 \text{ and } X_1 \quad (11.23) \\ &\approx r + \gamma \max_{a' \in A} q^\star(x',a'), && \text{Monte Carlo approximation with a single sample} \quad (11.24) \end{aligned}$$

where the agent observed the transition $(x, a, r, x')$. Using a bootstrapping estimate $Q^\star$ for $q^\star$, we obtain a structurally similar algorithm to

TD-learning and SARSA — only for estimating the optimal Q-function directly! This algorithm is known as *Q-learning*. Whereas we have seen that the optimal policy can be found using SARSA in a policy-iteration-like scheme, Q-learning is conceptually similar to value iteration.

**Algorithm 11.12:** Q-learning

initialize $Q^\star(x,a)$ arbitrarily (e.g., as $\mathbf{0}$)
**for** $t = 0$ **to** $\infty$ **do**
  observe the transition $(x, a, r, x')$
  $Q^\star(x,a) \leftarrow (1-\alpha_t)Q^\star(x,a) + \alpha_t(r + \gamma \max_{a' \in A} Q^\star(x',a'))$
  // (11.25)

Similarly to TD-learning, the update rule can also be expressed as

$$Q^\star(x,a) \leftarrow Q^\star(x,a) + \alpha_t \left( r + \gamma \max_{a' \in A} Q^\star(x',a') - Q^\star(x,a) \right). \quad (11.26)$$

Crucially, the Monte Carlo approximation of Equation (11.23) does not depend on the policy. Thus, Q-learning is an off-policy method.

**Theorem 11.13** (Convergence of Q-learning, Jaakkola et al. (1993))**.** *If $(\alpha_t)_{t \in \mathbb{N}_0}$ satisfies the RM-conditions* (A.60) *and all state-action pairs are chosen infinitely often (that is, the sequence of policies used to obtain the transitions is GLIE), then $Q^\star$ converges to $q^\star$ with probability* 1.

It can be shown that with probability at least $1 - \delta$, Q-learning converges to an $\epsilon$-optimal policy in a number of steps that is polynomial in $\log |X|$, $\log |A|$, $1/\epsilon$ and $\log 1/\delta$ (Even-Dar et al., 2003).

### 11.4.5 *Optimistic Q-learning*

The next natural question is how to effectively trade exploration and exploitation to both visit all state-action pairs many times, but also attain a high reward.

However, as we have seen in Section 11.3, random "uninformed" exploration like $\varepsilon$-greedy and softmax exploration explores the state space very slowly. We therefore return to the principle of *optimism in the face of uncertainty*, which already led us to the $R_{\max}$ algorithm in the model-based setting. We will now additionally assume that the rewards are non-negative, that is, $0 \leq r(x,a) \leq R_{\max}$ $(\forall x \in X, a \in A)$. It turns out that a similar algorithm to $R_{\max}$ also exists for (model-free) Q-learning: it is called *optimistic Q-learning* and shown in Algorithm 11.14.

**Algorithm 11.14:** Optimistic Q-learning

initialize $Q^\star(x,a) = V_{\max} \prod_{t=1}^{T_{\text{init}}} (1-\alpha_t)^{-1}$
**for** $t = 0$ **to** $\infty$ **do**
  pick action $a_t = \arg\max_{a \in A} Q^\star(x,a)$ and observe the transition $(x, a_t, r, x')$
  $Q^\star(x,a_t) \leftarrow (1-\alpha_t) Q^\star(x,a_t) + \alpha_t (r + \gamma \max_{a' \in A} Q^\star(x',a'))$ // (11.27)

Here,

$$V_{\max} \doteq \frac{R_{\max}}{1-\gamma} \geq \max_{x \in X, a \in A} q^\star(x,a),$$

is an upper bound on the discounted return and $T_{\text{init}}$ is some initialization time. Intuitively, the initialization of $Q^\star$ corresponds to the best-case long-term reward, assuming that all individual rewards are upper bounded by $R_{\max}$. This is shown by the following lemma.

**Lemma 11.15.** *Denote by $Q_t^\star$, the approximation of $q^\star$ attained in the $t$-th iteration of optimistic Q-learning. Then, for any state-action pair $(x,a)$ and iteration $t$ such that $N(a \mid x) \leq T_{\text{init}}$,*[7]

$$Q_t^\star(x,a) \geq V_{\max} \geq q^\star(x,a). \tag{11.28}$$

[7] $N(a \mid x)$ is the number of times action $a$ is performed in state $x$.

*Proof.* We write $\beta_\tau \doteq \prod_{i=1}^{\tau} (1-\alpha_i)$ and $\eta_{i,\tau} \doteq \alpha_i \prod_{j=i+1}^{\tau} (1-\alpha_j)$. Using the update rule of optimistic Q-learning (11.27), we have

$$Q_t^\star(x,a) = \beta_{N(a|x)} Q_0^\star(x,a) + \sum_{i=1}^{N(a|x)} \eta_{i,N(a|x)} (r + \gamma \max_{a_i \in A} Q_{t_i}^\star(x_i, a_i)) \tag{11.29}$$

where $x_i$ is the next state arrived at time $t_i$ when action $a$ is performed the $i$-th time in state $x$.

Using the assumption that the rewards are non-negative, from Equation (11.29) and $Q_0^\star(x,a) = V_{\max}/\beta_{T_{\text{init}}}$, we immediately have

$$\begin{aligned} Q_t^\star(x,a) &\geq \frac{\beta_{N(a|x)}}{\beta_{T_{\text{init}}}} V_{\max} \\ &\geq V_{\max}. \end{aligned}$$

using $N(a \mid x) \leq T_{\text{init}}$

□

Now, if $T_{\text{init}}$ is chosen large enough, it can be shown that optimistic Q-learning converges quickly to an optimal policy.

**Theorem 11.16** (Convergence of optimistic Q-learning, Even-Dar and Mansour (2001))**.** *With probability at least $1-\delta$, optimistic Q-learning obtains an $\epsilon$-optimal policy after a number of steps that is polynomial in $|X|$,*

*$|A|$, $1/\epsilon$, $\log 1/\delta$, and $R_{\max}$ where the initialization time $T_{\text{init}}$ is upper bounded by a polynomial in the same coefficients.*

Note that for Q-learning, we still need to store $Q^\star(x, a)$ for any state-action pair in memory. Thus, Q-learning requires $O(nm)$ memory. During each transition, we need to compute

$$\max_{a \in A} Q^\star(x', a')$$

once. If we run Q-learning for $T$ iterations, this yields a time complexity of $O(Tm)$. Crucially, for sparse Markov decision processes where, in most states, only few actions are permitted, each iteration of Q-learning can be performed in (virtually) constant time. This is a big improvement of the quadratic (in the number of states) performance of the model-based $R_{\max}$ algorithm.

## *Discussion*

We have seen that both the model-based $R_{\max}$ algorithm and the model-free Q-learning take time polynomial in the number of states $|X|$ and the number of actions $|A|$ to converge. While this is acceptable in small grid worlds, this is completely unacceptable for large state and action spaces.

Often, domains are continuous, for example when modeling beliefs about states in a partially observable environment. Also, in many structured domains (e.g., chess or multiagent planning), the size of the state and action space is exponential in the size of the input. In the final two chapters, we will therefore explore how model-free and model-based methods can be used (approximately) in such large domains.

## *Problems*

### 11.1. Q-learning.

Assume the following grid world, where from state $A$ the agent can go to the right and down, and from state $B$ to the left and down. From states $G_1$ and $G_2$ the only action is to exit. The agent receives a reward ($+10$ or $+1$) only when exiting.

| Rewards | | States | |
|---|---|---|---|
| 0 | 0 | $A$ | $B$ |
| $+10$ | $+1$ | $G_1$ | $G_2$ |

We assume the discount factor $\gamma = 1$ and that all actions are deterministic.

1. We observe the following two episodes:

Episode 1

| $x$ | $a$ | $x'$ | $r$ |
|---|---|---|---|
| $A$ | $\downarrow$ | $G_1$ | 0 |
| $G_1$ | exit | | 10 |

Episode 2

| $x$ | $a$ | $x'$ | $r$ |
|---|---|---|---|
| $B$ | $\leftarrow$ | $A$ | 0 |
| $A$ | $\downarrow$ | $G_1$ | 0 |
| $G_1$ | exit | | 10 |

   Assume $\alpha = 0.3$, and that Q-values of all non-terminal states are initialized to 0.5. What are the Q-values

$$Q^\star(A, \downarrow), \quad Q^\star(G_1, \text{exit}), \quad Q^\star(G_2, \text{exit})$$

   learned by executing Q-learning with the above episodes?
2. Will Q-learning converge to $q^\star$ for all state-action pairs $(x, a)$ if we repeat episode 1 and episode 2 infinitely often? If not, design a sequence of episodes that leads to convergence.
3. How does the choice of initial Q-values influence the convergence of the Q-learning algorithm when episodes are obtained off-policy?
4. Determine $v^\star$ for all states.

# 12

# *Model-free Reinforcement Learning*

In the previous chapter, we have seen methods for tabular settings. Our goal now is to extend the model-free methods like TD-learning and Q-learning to large state-action spaces $\mathcal{X}$ and $\mathcal{A}$. We have seen that a crucial bottleneck of these methods is the parameterization of the value function. If we want to store the value function in a table, we need at least $O(|\mathcal{X}|)$ space. If we learn the Q function, we even need $O(|\mathcal{X}| \cdot |\mathcal{A}|)$ space. Also, for large state-action spaces, the time required to compute the value function for every state-action pair exactly will grow polynomially in the size of the state-action space. Hence, a natural idea is to learn *approximations* of these functions with a low-dimensional parameterization. Such approximations were the focus of the first few chapters and are, in fact, the key idea behind methods for reinforcement learning in large state-action spaces.

## 12.1 *Tabular Reinforcement Learning as Optimization*

To begin with, let us reinterpret the model-free methods from the previous section, TD-learning and Q-learning, as solving an optimization problem, where each iteration corresponds to a single gradient update. We will focus on TD-learning here, but the same interpretation applies to Q-learning. Recall the update rule of TD-learning (11.15),

$$V^\pi(x) \leftarrow (1 - \alpha_t)V^\pi(x) + \alpha_t(r + \gamma V^\pi(x')).$$

Note that this looks just like the update rule of an optimization algorithm! We can parameterize our estimates $V^\pi$ with parameters $\boldsymbol{\theta}$ that are updated according to the gradient of some loss function, assuming fixed bootstrapping estimates. In particular, in the tabular setting (i.e., over a finite domain), we can parameterize the value function exactly by learning a separate parameter for each state,

$$\boldsymbol{\theta} \doteq [\boldsymbol{\theta}(1), \ldots, \boldsymbol{\theta}(n)], \quad V^\pi(x; \boldsymbol{\theta}) \doteq \boldsymbol{\theta}(x). \tag{12.1}$$

To re-derive the above update rule as a gradient update, let us consider the following loss function,

$$\bar{\ell}(\boldsymbol{\theta}; x, r) \doteq \frac{1}{2}(v^\pi(x) - \boldsymbol{\theta}(x))^2 \tag{12.2}$$

$$= \frac{1}{2}\Big(r + \gamma \mathbb{E}_{x'|x,\pi(x)}\big[v^\pi(x')\big] - \boldsymbol{\theta}(x)\Big)^2 \tag{12.3}$$

using Bellman's expectation equation (10.11)

Note that this loss corresponds to a standard squared loss of the difference between the parameter $\boldsymbol{\theta}(x)$ and the label $v^\pi(x)$ we want to learn.

We can now find the gradient of this loss. Elementary calculations yield,

$$\nabla_{\boldsymbol{\theta}(x)} \bar{\ell}(\boldsymbol{\theta}; x, r) = \boldsymbol{\theta}(x) - \Big(r + \gamma \mathbb{E}_{x'|x,\pi(x)}\big[v^\pi(x')\big]\Big). \tag{12.4}$$

Now, we cannot compute this derivative because we cannot compute the expectation. Firstly, the expectation is over the true value function which is unknown to us. Secondly, the expectation is over the transition model which we are trying to avoid in model-free methods.

To resolve the first issue, analogously to TD-learning, instead of learning the true value function $v^\pi$ which is unknown, we learn the bootstrapping estimate $V^\pi$. Recall that the core principle behind bootstrapping as discussed in Section 11.4.1 is that this bootstrapping estimate $V^\pi$ is *treated* as if it were independent of the current estimate of the value function $\boldsymbol{\theta}$. To emphasize this, we write $V^\pi(x; \boldsymbol{\theta}^{\text{old}}) \approx v^\pi(x)$ where $\boldsymbol{\theta}^{\text{old}} = \boldsymbol{\theta}$ but $\boldsymbol{\theta}^{\text{old}}$ is treated as a constant with respect to $\boldsymbol{\theta}$.[1]

[1] That is, the bootstrapping estimate $V^\pi(x; \boldsymbol{\theta}^{\text{old}})$ is assumed to be constant with respect to $\boldsymbol{\theta}(x)$ in the same way that $v^\pi(x)$ is constant with respect to $\boldsymbol{\theta}(x)$.

If we were not using the bootstrapping estimate, the following derivation of the gradient of the loss would not be this simple.

To resolve the second issue, analogously to the introduction of TD-learning in the previous chapter, we will use a Monte Carlo estimate using a single sample. Recall that this is only possible because the transitions are conditionally independent given the state-action pair.

**Remark 12.1: Sample (in)efficiency of model-free methods**

Taking these two shortcuts are two of the main reasons why model-free methods such as TD-learning and Q-learning are usually *sample inefficient*. This is because using a bootstrapping estimate leads to "(initially) incorrect" and "unstable" targets of the optimization problem,[2] and Monte Carlo estimation with a single sample leads to a large variance. Recall that the theoretical guarantees for model-free methods in the tabular setting therefore required that all state-action pairs are visited infinitely often.

[2] We explore this in some more capacity in Section 12.2.1 where we cover heuristic approaches to alleviate this problem to some degree.

Using the aforementioned shortcuts, let us define the loss $\ell$ after observing the single transition $(x, a, r, x')$,

$$\ell(\boldsymbol{\theta}; x, r, x') \doteq \frac{1}{2}\Big(r + \gamma \boldsymbol{\theta}^{\text{old}}(x') - \boldsymbol{\theta}(x)\Big)^2. \tag{12.5}$$

We define the negative gradient of this loss with respect to $\boldsymbol{\theta}(x)$ as

$$\begin{aligned}\delta_{\text{TD}} &\doteq -\boldsymbol{\nabla}_{\boldsymbol{\theta}(x)}\, \ell(\boldsymbol{\theta}; x, r, x') \\ &= \Big(r + \gamma \boldsymbol{\theta}^{\text{old}}(x')\Big) - \boldsymbol{\theta}(x). \end{aligned} \tag{12.6}$$

This error term is also called *temporal-difference (TD) error*. The temporal difference error compares the previous estimate of the value function to the bootstrapping estimate of the value function. We know from the law of large numbers (A.36) that Monte Carlo averages are unbiased.[3] We therefore have,

$$\mathbb{E}_{x'|x,\pi(x)}[\delta_{\text{TD}}] \approx -\boldsymbol{\nabla}_{\boldsymbol{\theta}(x)}\, \overline{\ell}(\boldsymbol{\theta}; x, r). \tag{12.7}$$

[3] Crucially, the samples are unbiased with respect to the approximate label in terms of the bootstrapping estimate only. Due to bootstrapping the value function, the estimates are not unbiased with respect to the true value function. Moreover, the variance of each individual estimation of the gradient is large, as we only consider a single transition.

Naturally, we can use these unbiased gradient estimates with respect to the loss $\overline{\ell}$ to perform stochastic gradient descent. This yields the update rule,

$$V^\pi(x; \boldsymbol{\theta}) = \boldsymbol{\theta}(x) \leftarrow \boldsymbol{\theta}(x) + \alpha_t \delta_{\text{TD}} \tag{12.8}$$

using stochastic gradient descent with learning rate $\alpha_t$, see Algorithm A.29

$$= (1 - \alpha_t)\boldsymbol{\theta}(x) + \alpha_t \Big(r + \gamma \boldsymbol{\theta}^{\text{old}}(x')\Big)$$

using the definition of the temporal difference error (12.6)

$$= (1 - \alpha_t) V^\pi(x; \boldsymbol{\theta}) + \alpha_t \Big(r + \gamma V^\pi(x'; \boldsymbol{\theta}^{\text{old}})\Big). \tag{12.9}$$

substituting $V^\pi(x; \boldsymbol{\theta})$ for $\boldsymbol{\theta}(x)$

Observe that this gradient update coincides with the update rule of TD-learning (11.15). Therefore, TD-learning is essentially performing stochastic gradient descent using the TD-error as an unbiased gradient estimate.[4] Crucially, TD-learning performs stochastic gradient descent with respect to the bootstrapping estimate of the value function $V^\pi$ and not the true value function $v^\pi$! Stochastic gradient descent with a bootstrapping estimate is also called *stochastic semi-gradient descent*. Importantly, the optimization target $r + \gamma V^\pi(x'; \boldsymbol{\theta}^{\text{old}})$ from the loss $\ell$ is now *moving* between iterations which introduces some practical challenges we will discuss in Section 12.2.1. We have seen in the previous chapter that using a bootstrapping estimate still guarantees (asymptotic) convergence to the true value function.

[4] An alternative interpretation is that TD-learning performs gradient descent with respect to the loss $\ell$.

## 12.2 *Value Function Approximation*

To scale to large state spaces, it is natural to approximate the value function using a parameterized model, $V(\boldsymbol{x}; \boldsymbol{\theta})$ or $Q(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})$. You may think of this as a regression problem where we map state(-action) pairs to a real number. Recall from the previous section that this is a strict generalization of the tabular setting, as we could use a separate parameter to learn the value function for each individual state-action pair. Our goal for large state-action spaces is to exploit the smoothness properties[5] of the value function to condense the representation.

[5] That is, the value function takes similar values in "similar" states.

A straightforward approach is to use a linear function approximation with the hand-designed feature map $\boldsymbol{\phi}$,

$$Q(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \doteq \boldsymbol{\theta}^\top \boldsymbol{\phi}(\boldsymbol{x}, \boldsymbol{a}). \tag{12.10}$$

A common alternative is to use a deep neural network to learn these features instead. Doing so is also known as *deep reinforcement learning*.[6]

[6] Note that often non-Bayesian deep learning (i.e., point estimates of the weights of a neural network) is applied. In the final chapter, Chapter 13, we will explore the benefits of using Bayesian deep learning.

We will now apply the derivation from the previous section directly to Q-learning. For Q-learning, after observing the transition $(\boldsymbol{x}, \boldsymbol{a}, r, \boldsymbol{x}')$, the loss function is given as

$$\ell(\boldsymbol{\theta}; \boldsymbol{x}, \boldsymbol{a}, r, \boldsymbol{x}') \doteq \frac{1}{2}\left(r + \gamma \max_{\boldsymbol{a}' \in \mathcal{A}} Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})\right)^2. \tag{12.11}$$

Here, we simply use Bellman's optimality equation (10.32) to estimate $q^\star(\boldsymbol{x}, \boldsymbol{a})$, instead of the estimation of $v^\pi(\boldsymbol{x})$ using Bellman's expectation equation for TD-learning. The difference between the current approximation and the optimization target,

$$\delta_{\text{B}} \doteq r + \gamma \max_{\boldsymbol{a}' \in \mathcal{A}} Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}), \tag{12.12}$$

is called the *Bellman error*. Analogously to TD-learning,[7] we obtain the gradient update,

[7] compare to Equation (12.8)

$$\begin{aligned}
\boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} - \alpha_t \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \boldsymbol{x}, \boldsymbol{a}, r, \boldsymbol{x}') && (12.13)\\
&= \boldsymbol{\theta} - \alpha_t \nabla_{\boldsymbol{\theta}} \frac{1}{2}\left(r + \gamma \max_{\boldsymbol{a}' \in \mathcal{A}} Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})\right)^2 && \text{using the definition of } \ell \text{ (12.11)}\\
&= \boldsymbol{\theta} + \alpha_t \delta_{\text{B}} \nabla_{\boldsymbol{\theta}} Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}). && (12.14) \quad \text{using the chain rule}
\end{aligned}$$

When using a neural network to learn $Q^\star$, we can use automatic differentiation to obtain unbiased gradient estimates. In the case of linear function approximation, we can compute the gradient exactly,

$$\begin{aligned}
&= \boldsymbol{\theta} + \alpha_t \delta_{\text{B}} \nabla_{\boldsymbol{\theta}} \boldsymbol{\theta}^\top \boldsymbol{\phi}(\boldsymbol{x}, \boldsymbol{a}) && \text{using the linear approximation of } Q^\star \text{ (12.10)}\\
&= \boldsymbol{\theta} + \alpha_t \delta_{\text{B}} \boldsymbol{\phi}(\boldsymbol{x}, \boldsymbol{a}). && (12.15)
\end{aligned}$$

In the tabular setting, this algorithm is identical to Q-learning and, in particular, converges to the true Q-function $q^\star$.[8] There are few such results in the approximate setting. Usage in practice indicates that using an approximation of the value function "should be fine" when a "rich-enough" class of functions is used.

[8] see Theorem 11.13

### 12.2.1 *Heuristics*

The vanilla stochastic semi-gradient descent is very slow. In this subsection, we will discuss some "tricks of the trade" to improve its performance.

*Stabilizing optimization targets:* There are mainly two problems. The first problem is that, as mentioned previously, the bootstrapping estimate changes after each iteration. As we are trying to learn an approximate value function that depends on the bootstrapping estimate, this means that the optimization target is "moving" between iterations. In practice, moving targets lead to stability issues. The first family of techniques we discuss here aim to "stabilize" the optimization targets.

One such technique is called *neural fitted Q-iteration* or *deep Q-networks* (DQN) (Mnih et al., 2015). DQN updates the neural network used for the approximate bootstrapping estimate infrequently to maintain a constant optimization target across multiple episodes. How this is implemented exactly varies. One approach is to clone the neural network and maintain one changing neural network ("online network") for the most recent estimate of the Q-function which is parameterized by $\boldsymbol{\theta}$, and one fixed neural network ("target network") used as the target which is parameterized by $\boldsymbol{\theta}^{\text{old}}$ and which is updated infrequently.

This can be implemented by maintaining a data set $\mathcal{D}$ of observed transitions (the so-called *replay buffer*) and then "every once in a while" (e.g., once $|\mathcal{D}|$ is large enough) solving a regression problem, where the labels are determined by the target network. This yields a loss term where the target is fixed across all transitions in the replay buffer $\mathcal{D}$,

$$\ell_{\text{DQN}}(\boldsymbol{\theta}; \mathcal{D}) \doteq \frac{1}{2} \sum_{(\boldsymbol{x},\boldsymbol{a},r,\boldsymbol{x}') \in \mathcal{D}} \left( r + \gamma \max_{\boldsymbol{a}' \in \mathcal{A}} Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \right)^2. \tag{12.16}$$

compare to the Q-learning loss (12.11)

The loss can also be interpreted (in an online sense) as performing regular Q-learning with the modification that the target network $\boldsymbol{\theta}^{\text{old}}$ is not updated to $\boldsymbol{\theta}$ after every observed transition, but instead only after observing $|\mathcal{D}|$-many transitions. This technique is known as *experience replay*. Another approach is *Polyak averaging* where the target network is gradually "nudged" by the neural network used to estimate the Q-function.

*Maximization bias:* Now, observe that the estimates $Q^\star$ are noisy estimates of $q^\star$ and consider the term,

$$\max_{\boldsymbol{a}' \in \mathcal{A}} q^\star(\boldsymbol{x}', \boldsymbol{a}') \approx \max_{\boldsymbol{a}' \in \mathcal{A}} Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}),$$

from the loss function (12.16). This term maximizes a *noisy* estimate of $q^\star$, which leads to a *biased* estimate of $\max q^\star$ as can be seen in Figure 12.1. The fact that the update rules of Q-learning and DQN are affected by inaccuracies (i.e., noise in the estimates) of the learned Q-function is known as the "*maximization bias*".

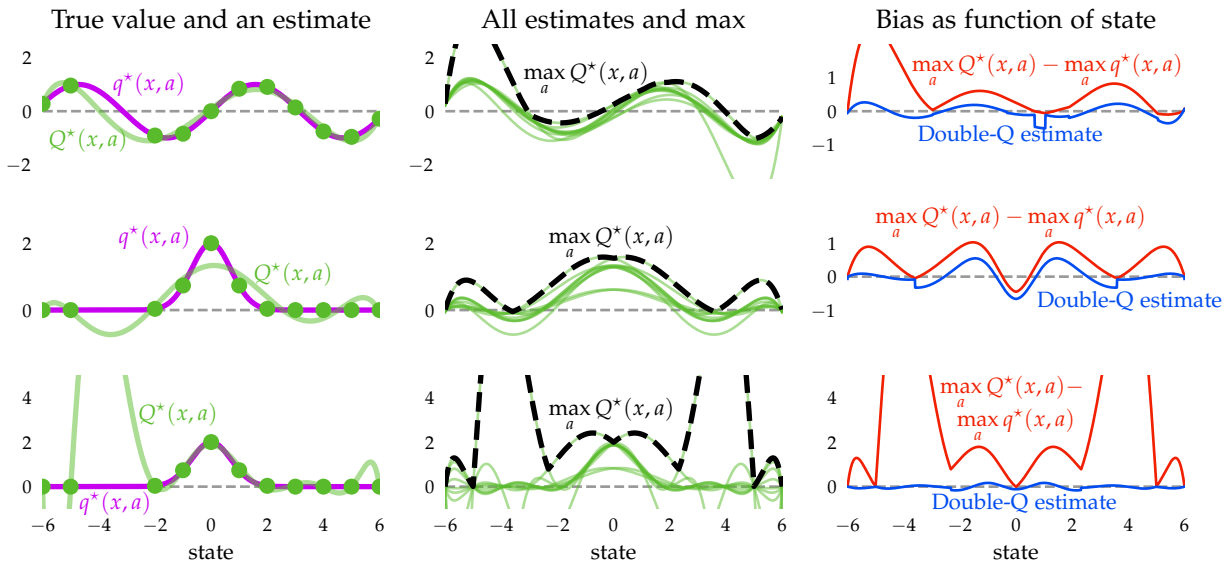



Figure 12.1: Illustration of overestimation during learning. In each state (x-axis), there are 10 actions. The left column shows the true values $v^\star(x)$ (purple line). All true action values are defined by $q^\star(x,a) \doteq v^\star(x)$. The green line shows estimated values $Q^\star(x,a)$ for one action as a function of state, fitted to the true value at several sampled states (green dots). The middle column plots show all the estimated values (green), and the maximum of these values (dashed black). The maximum is higher than the true value (purple, left plot) almost everywhere. The right column plots show the difference in red. The blue line in the right plots is the estimate used by Double Q-learning with a second set of samples for each state. The blue line is much closer to zero, indicating less bias. The three rows correspond to different true functions (left, purple) or capacities of the fitted function (left, green). Reproduced with permission from "Deep reinforcement learning with double q-learning" (Van Hasselt et al., 2016).

*Double DQN* (DDQN) is an algorithm that addresses this maximization bias (Van Hasselt et al., 2016). Instead of picking the optimal action with respect to the old network, it picks the optimal action with respect to the new network,

$$\ell_{\text{DDQN}}(\boldsymbol{\theta}; \mathcal{D}) \doteq \frac{1}{2} \sum_{(\boldsymbol{x},\boldsymbol{a},r,\boldsymbol{x}') \in \mathcal{D}} \Big( r + \gamma Q^\star(\boldsymbol{x}', \boldsymbol{a}^\star(\boldsymbol{x}'; \boldsymbol{\theta}); \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \Big)^2 \tag{12.17}$$

$$\text{where} \quad \boldsymbol{a}^\star(\boldsymbol{x}'; \boldsymbol{\theta}) \doteq \arg\max_{\boldsymbol{a}' \in \mathcal{A}} Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}). \tag{12.18}$$

Intuitively, this change ensures that the evaluation of the target network is consistent with the updated Q-function, which makes the algorithm more robust to noise.

Similarly to DQN, this can also be interpreted as the online update,

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha_t \Big( r + \gamma Q^\star(\boldsymbol{x}', \boldsymbol{a}^\star(\boldsymbol{x}'; \boldsymbol{\theta}); \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \Big) \nabla_{\boldsymbol{\theta}} Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \tag{12.19}$$

after observing a single transition $(\boldsymbol{x}, \boldsymbol{a}, r, \boldsymbol{x}')$ where while differentiating, $\boldsymbol{a}^\star(\boldsymbol{x}'; \boldsymbol{\theta})$ is treated as constant with respect to $\boldsymbol{\theta}$. $\boldsymbol{\theta}^{\text{old}}$ is then updated to $\boldsymbol{\theta}$ after observing $|\mathcal{D}|$-many transitions.

## 12.3 *Policy Approximation*

Q-learning defines a policy implicitly by

$$\pi^\star(\boldsymbol{x}) \doteq \arg\max_{\boldsymbol{a} \in \mathcal{A}} Q^\star(\boldsymbol{x}, \boldsymbol{a}). \tag{12.20}$$

Q-learning also maximizes over the set of all actions in its update step while learning the Q-function. This is intractable for large and, in particular, continuous action spaces. A natural idea to escape this limitation is to immediately learn an approximate parameterized policy,

$$\pi^\star(x) \approx \pi(x;\varphi) \doteq \pi_\varphi(x). \tag{12.21}$$

Methods that find an approximate policy are also called *policy search methods* or *policy gradient methods*.

Whereas with Q-learning, exploration can be encouraged by using an $\varepsilon$-greedy policy, softmax exploration, or an optimistic initialization, we will see later that policy gradient methods fundamentally rely on randomized policies for exploration.

**Remark 12.2: Notation**
We refer to deterministic policies $\pi$ in bold, as they can be interpreted as vector-valued functions from $\mathcal{X}$ to $\mathcal{A}$. We still refer to randomized policies by $\pi$, as for each state $x \in \mathcal{X}$ they are represented as a PDF over actions $\mathcal{A}$.

In particular, we denote by $\pi_\varphi(a \mid x)$ the probability (density) of playing action $a$ when in state $x$ according to $\pi_\varphi$.

### 12.3.1 Estimating Policy Values

We will begin by attributing a "value" to a policy. Recall the definition of the discounted payoff $G_t$ from time $t$, which we are aiming to maximize,

$$G_t = \sum_{m=0}^{\infty} \gamma^m R_{t+m}. \qquad \text{see Equation (10.5)}$$

We define $G_{t:T}$ to be the *bounded discounted payoff* until time $T$,

$$G_{t:T} \doteq \sum_{m=0}^{T-1-t} \gamma^m R_{t+m}. \tag{12.22}$$

Based on these two random variables, we can define the policy value function:

**Definition 12.3** (Policy value function)**.** The *policy value function*,

$$J(\pi) \doteq \mathbb{E}_\pi[G_0] = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R_t\right], \tag{12.23}$$

measures the expected discounted payoff of policy $\pi$.[9] We also define the bounded variant,

$$J_T(\pi) \doteq \mathbb{E}_\pi[G_{0:T}] = \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} \gamma^t R_t\right]. \tag{12.24}$$

[9] We neglect here that implicitly one also averages over the initial state if this state is not fixed.

For simplicity, we will abbreviate $J(\boldsymbol{\varphi}) \doteq J(\pi_{\boldsymbol{\varphi}})$.

**Remark 12.4: Notation of policy value function**

We adopt the more common notation $J(\pi)$ for the policy value function, as opposed to $j(\pi)$, which would be consistent with our notation of the (true) value functions $v^\pi, q^\pi$. So don't be confused by this: just like the value functions $v^\pi, q^\pi$, the policy value function $J(\pi)$ is a deterministic object, measuring the mean discounted payoff. We will use $\widehat{J}(\pi)$ to refer to our estimates of the policy value function.

Naturally, we want to maximize $J(\boldsymbol{\varphi})$. That is, we want to solve

$$\boldsymbol{\varphi}^\star \doteq \arg\max_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi}) \tag{12.25}$$

which is a non-convex optimization problem. Let us see how $J(\boldsymbol{\varphi})$ can be evaluated to understand the optimization problem better. We will again use a Monte Carlo estimate. Recall that a fixed $\boldsymbol{\varphi}$ induces a unique Markov chain, which can be simulated. In the episodic setting, each episode (also called *rollout*) of length $T$ yields an independently sampled trajectory,

$$\tau^{(i)} \doteq (\boldsymbol{x}_0^{(i)}, \boldsymbol{a}_0^{(i)}, r_0^{(i)}, \boldsymbol{x}_1^{(i)}), (\boldsymbol{x}_1^{(i)}, \boldsymbol{a}_1^{(i)}, r_1^{(i)}, \boldsymbol{x}_2^{(i)}), \dots) \tag{12.26}$$

Simulating $m$ rollouts yields the samples $\tau^{(1)}, \dots, \tau^{(m)} \sim \Pi_{\boldsymbol{\varphi}}$, where $\Pi_{\boldsymbol{\varphi}}$ is the distribution of all trajectories (i.e., rollouts) of the Markov chain induced by policy $\pi_{\boldsymbol{\varphi}}$. We denote the (bounded) discounted payoff of the $i$-th rollout by

$$g_{0:T}^{(i)} \doteq \sum_{t=0}^{T-1} \gamma^t r_t^{(i)} \tag{12.27}$$

where $r_t^{(i)}$ is the reward at time $t$ of the $i$-th rollout. Using a Monte Carlo approximation, we can then estimate $J_T(\boldsymbol{\varphi})$. Moreover, due to the exponential discounting of future rewards, it is reasonable to approximate the policy value function using bounded trajectories,

$$J(\boldsymbol{\varphi}) \approx J_T(\boldsymbol{\varphi}) \approx \widehat{J}_T(\boldsymbol{\varphi}) \doteq \frac{1}{m} \sum_{i=1}^{m} g_{0:T}^{(i)}. \tag{12.28}$$

### 12.3.2 *Policy Gradient*

Policy gradient methods solve the above optimization problem (12.25) by stochastic gradient ascent on the policy parameter $\boldsymbol{\varphi}$:

$$\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} + \eta \nabla_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi}). \tag{12.29}$$

How can we compute the policy gradient? Let us first formally define the distribution over trajectories $\Pi_{\boldsymbol{\varphi}}$ that we introduced in the previous section. We can specify the probability of a specific trajectory $\tau$ under a policy $\pi_{\boldsymbol{\varphi}}$ by

$$\Pi_{\boldsymbol{\varphi}}(\tau) = p(\boldsymbol{x}_0) \prod_{t=0}^{T-1} \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t). \tag{12.30}$$

For optimizing $J(\boldsymbol{\varphi})$ we need to obtain unbiased gradient estimates:

$$\boldsymbol{\nabla}_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi}) \approx \boldsymbol{\nabla}_{\boldsymbol{\varphi}} J_T(\boldsymbol{\varphi}) = \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}[G_{0:T}]. \tag{12.31}$$

Note that the expectation integrates over the measure $\Pi_{\boldsymbol{\varphi}}$, which depends on the parameter $\boldsymbol{\varphi}$. Thus, we cannot move the gradient operator inside the expectation as we have often done previously (cf. Section A.1.5). This should remind you of the reparameterization trick (see Equation (5.62)) that we used to solve a similar gradient in the context of variational inference. In this context, however, we cannot apply the reparameterization trick.[10] Fortunately, there is another way of estimating this gradient.

[10] This is because the distribution $\Pi_{\boldsymbol{\varphi}}$ is generally not reparameterizable. We will, however, see that reparameterization gradients are also useful in reinforcement learning. See, e.g., Sections 12.5.1 and 13.1.2.

**Theorem 12.5** (Score gradient estimator)**.** *Under some regularity assumptions, we have*

$$\boldsymbol{\nabla}_{\boldsymbol{\varphi}} \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}[G_{0:T}] = \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}\big[G_{0:T} \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau)\big]. \tag{12.32}$$

*This estimator of the gradient is called the* score gradient estimator.

*Proof.* To begin with, let us look at the so-called *score function* of the distribution $\Pi_{\boldsymbol{\varphi}}$, $\boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau)$. Using the chain rule, the score function can be expressed as

$$\boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) = \frac{\boldsymbol{\nabla}_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau)}{\Pi_{\boldsymbol{\varphi}}(\tau)} \tag{12.33}$$

and by rearranging the terms, we obtain

$$\boldsymbol{\nabla}_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau) = \Pi_{\boldsymbol{\varphi}}(\tau) \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau). \tag{12.34}$$

This is called the *score function trick*.[11]

[11] We have already applied this "trick" in Problem 6.9.

Now, assuming that state and action spaces are continuous, we obtain

$$\begin{aligned}
\boldsymbol{\nabla}_{\boldsymbol{\varphi}} \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}[G_{0:T}] &= \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \int \Pi_{\boldsymbol{\varphi}}(\tau) \cdot G_{0:T} \, d\tau && \text{using the definition of expectation (1.19)} \\
&= \int \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau) \cdot G_{0:T} \, d\tau && \text{using the regularity assumptions to swap gradient and integral} \\
&= \int G_{0:T} \cdot \Pi_{\boldsymbol{\varphi}}(\tau) \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) \, d\tau && \text{using the score function trick (12.34)} \\
&= \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}\big[G_{0:T} \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau)\big]. && \text{interpreting the integral as an expectation over } \Pi_{\boldsymbol{\varphi}}
\end{aligned}$$

□

Intuitively, maximizing $J(\boldsymbol{\varphi})$ increases the probability of trajectories with high returns and decreases the probability of trajectories with low returns.

To use the score gradient estimator for estimating the gradient, we need to compute $\nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau)$.

$$
\begin{aligned}
&\nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) \\
&= \nabla_{\boldsymbol{\varphi}} \left( \log p(\boldsymbol{x}_0) + \sum_{t=0}^{T-1} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) + \sum_{t=0}^{T-1} \log p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t) \right) && \text{using the definition of the distribution over trajectories } \Pi_{\boldsymbol{\varphi}} \\
&= \nabla_{\boldsymbol{\varphi}} \log p(\boldsymbol{x}_0) + \sum_{t=0}^{T-1} \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) + \sum_{t=0}^{T-1} \nabla_{\boldsymbol{\varphi}} \log p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t) \\
&= \sum_{t=0}^{T-1} \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t). && (12.35) \quad \text{using that the first and third term are independent of } \boldsymbol{\varphi}
\end{aligned}
$$

When using a neural network for the parameterization of the policy $\pi$, we can use automatic differentiation to compute the gradients.

The expectation of the score gradient estimator (12.32) can be approximated using Monte Carlo sampling,

$$
\nabla_{\boldsymbol{\varphi}} J_T(\boldsymbol{\varphi}) \approx \nabla_{\boldsymbol{\varphi}} \widehat{J}_T(\boldsymbol{\varphi}) \doteq \frac{1}{m} \sum_{i=1}^{m} g_{0:T}^{(i)} \sum_{t=0}^{T-1} \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t^{(i)} \mid \boldsymbol{x}_t^{(i)}). \quad (12.36)
$$

However, typically the variance of these estimates is very large. Using so-called *baselines*, we can reduce the variance dramatically (?). **Problem 12.3**

**Lemma 12.6** (Score gradients with baselines)**.** *We have,*

$$
\mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}} \left[ G_{0:T} \nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) \right] = \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}} \left[ (G_{0:T} - b) \nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) \right]. \quad (12.37)
$$

*Here, $b \in \mathbb{R}$ is called a* baseline.

*Proof.* For the term to the right, we have due to linearity of expectation (1.20),

$$
\begin{aligned}
\mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}} \left[ (G_{0:T} - b) \nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) \right] &= \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}} \left[ G_{0:T} \nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) \right] \\
&\quad - \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}} \left[ b \cdot \nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) \right].
\end{aligned}
$$

Thus, it remains to show that the second term is zero,

$$
\begin{aligned}
\mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}} \left[ b \cdot \nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) \right] &= b \cdot \int \Pi_{\boldsymbol{\varphi}}(\tau) \nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau) \, d\tau && \text{using the definition of expectation (1.19)} \\
&= b \cdot \int \Pi_{\boldsymbol{\varphi}}(\tau) \frac{\nabla_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau)}{\Pi_{\boldsymbol{\varphi}}(\tau)} \, d\tau && \text{substituting the score function (12.33), "undoing the score function trick"} \\
&= b \cdot \int \nabla_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau) \, d\tau && \Pi_{\boldsymbol{\varphi}}(\tau) \text{ cancels} \\
&= b \cdot \nabla_{\boldsymbol{\varphi}} \int \Pi_{\boldsymbol{\varphi}}(\tau) \, d\tau \\
&= b \cdot \nabla_{\boldsymbol{\varphi}} 1 = 0. && \square \quad \text{integrating a PDF over its domain is 1 and the derivative of a constant is 0}
\end{aligned}
$$

One can even show, that we can subtract arbitrary baselines depending on *previous* states (?).

**Problem 12.4**

**Example 12.7: Downstream returns**

A commonly used state-dependent baseline is

$$b(\tau_{0:t-1}) \doteq G_{0:t} = \sum_{m=0}^{t-1} \gamma^m R_m. \tag{12.38}$$

This baseline subtracts the returns of all actions before time $t$. Intuitively, using this baseline, the score gradient only considers downstream returns. Recall from Equation (12.22) that we defined $G_{t:T}$ as the bounded discounted payoff from time $t$. It is also commonly called the (bounded) *downstream return* (or *reward to go*) beginning at time $t$.

For a fixed trajectory $\tau$ that is bounded at time $T$, we have

$$G_{0:T} - b(\tau_{0:t-1}) = G_{0:T} - G_{0:t} = \gamma^t G_{t:T}, \tag{12.39}$$

yielding the gradient estimator,

$$\begin{aligned} \nabla_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi}) \approx \nabla_{\boldsymbol{\varphi}} J_T(\boldsymbol{\varphi}) &= \mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[G_{0:T} \nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau)\right] && \text{using the score gradient estimator (12.32)} \\ &= \mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[\sum_{t=0}^{T-1} \gamma^t G_{t:T} \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t)\right]. && \text{using a state-dependent baseline (12.102)} \end{aligned} \tag{12.40}$$

Performing stochastic gradient descent with the score gradient estimator and downstream returns is known as the *REINFORCE algorithm* (Williams, 1992) which is shown in Algorithm 12.8.

**Algorithm 12.8:** REINFORCE algorithm

```
initialize policy weights φ
repeat
    generate an episode (i.e., rollout) to obtain trajectory τ
    for t = 0 to T − 1 do
        set g_{t:T} to the downstream return from time t
        φ ← φ + η γ^t g_{t:T} ∇_φ log π_φ(a_t | x_t)        // (12.41)
until converged
```

The variance of REINFORCE can be reduced further. A common tech-

nique is to subtract a term $b_t$ from the downstream returns,

$$\nabla_{\boldsymbol{\varphi}} J_T(\boldsymbol{\varphi}) = \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}} \left[ \sum_{t=0}^{T-1} \gamma^t (G_{t:T} - b_t) \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) \right]. \tag{12.42}$$

For example, we can subtract the $t$-independent mean reward to go,

$$b_t \doteq b = \frac{1}{T} \sum_{t'=0}^{T-1} G_{t':T}. \tag{12.43}$$

The main advantage of policy gradient methods such as REINFORCE is that they can be used in continuous action spaces. However, REINFORCE is not guaranteed to find an optimal policy. Even when operating in very small domains, REINFORCE can get stuck in local optima.

Typically, policy gradient methods are slow due to the large variance in the score gradient estimates. Because of this, they need to take small steps and require many rollouts of a Markov chain. Moreover, we cannot reuse data from previous rollouts, as policy gradient methods are fundamentally on-policy.[12]

[12] This is because the score gradient estimator is used to obtain gradients of the policy value function with respect to the *current* policy.

Next, we will combine value approximation techniques like Q-learning and policy gradient methods, leading to an often more practical family of methods called actor-critic methods.

## 12.4 On-policy Actor-Critics

Actor-Critic methods reduce the variance of policy gradient estimates by using ideas from value function approximation. They use function approximation both to approximate value functions *and* to approximate policies. The goal for these algorithms is to scale to reinforcement learning problems, where we both have large state spaces and large action spaces.

### 12.4.1 Advantage Function

A key concept of actor-critic methods is the advantage function.

**Definition 12.9** (Advantage function)**.** Given a policy $\pi$, the *advantage function*,

$$\begin{aligned} a^\pi(\boldsymbol{x}, \boldsymbol{a}) &\doteq q^\pi(\boldsymbol{x}, \boldsymbol{a}) - v^\pi(\boldsymbol{x}) && (12.44) \\ &= q^\pi(\boldsymbol{x}, \boldsymbol{a}) - \mathbb{E}_{\boldsymbol{a}' \sim \pi(\boldsymbol{x})} \left[ q^\pi(\boldsymbol{x}, \boldsymbol{a}') \right], && (12.45) \end{aligned}$$

using Equation (10.14)

measures the advantage of picking action $\boldsymbol{a} \in \mathcal{A}$ when in state $\boldsymbol{x} \in \mathcal{X}$ over simply following policy $\pi$.

It follows immediately from Equation (12.45) that for any policy $\pi$ and state $\boldsymbol{x} \in \mathcal{X}$, there exists an action $\boldsymbol{a} \in \mathcal{A}$ such that $a^\pi(\boldsymbol{x}, \boldsymbol{a})$ is non-negative,

$$\max_{\boldsymbol{a} \in \mathcal{A}} a^\pi(\boldsymbol{x}, \boldsymbol{a}) \geq 0. \tag{12.46}$$

Moreover, it follows directly from Bellman's theorem (10.31) that

$$\pi \text{ is optimal} \iff \forall \boldsymbol{x} \in \mathcal{X}, \boldsymbol{a} \in \mathcal{A} : a^\pi(\boldsymbol{x}, \boldsymbol{a}) \leq 0. \tag{12.47}$$

In other words, quite intuitively, $\pi$ is optimal if and only if there is no action that has an advantage in any state over the action played by $\pi$.

Finally, we can re-define the greedy policy $\pi_q$ with respect to the state-action value function $q$ in terms of the associated advantage function,

$$\begin{aligned} \pi_q(\boldsymbol{x}) &\doteq \arg\max_{\boldsymbol{a} \in \mathcal{A}} q(\boldsymbol{x}, \boldsymbol{a}) \\ &= \arg\max_{\boldsymbol{a} \in \mathcal{A}} q(\boldsymbol{x}, \boldsymbol{a}) - v^q(\boldsymbol{x}) \\ &= \arg\max_{\boldsymbol{a} \in \mathcal{A}} a^q(\boldsymbol{x}, \boldsymbol{a}), \end{aligned} \tag{12.48}$$

since the state value function of $\pi_q$, $v^q(\boldsymbol{x})$, is independent of $\boldsymbol{a}$. This coincides with our initial definition of greedy policies in Equation (10.29). Intuitively, the advantage function is a shifted version of the state-action value function $q$ that is relative to 0. Using this quantity rather than $q$ often has numerical advantages.

### 12.4.2 *Policy Gradient Theorem*

Recall the score gradient estimator (12.40) that we had introduced in the previous section,

$$\nabla_{\boldsymbol{\varphi}} J_T(\boldsymbol{\varphi}) = \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}} \left[ \sum_{t=0}^{T-1} \gamma^t G_{t:T} \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) \right].$$

Previously, we have approximated the policy value function $J(\boldsymbol{\varphi})$ by the bounded policy value function $J_T(\boldsymbol{\varphi})$. We said that this approximation was "reasonable" due to the diminishing returns. Essentially, we have "cut off the tails" of the policy value function. Let us now reinterpret score gradients while taking into account the tails of $J(\boldsymbol{\varphi})$.

$$\begin{aligned} \nabla_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi}) &= \lim_{T \to \infty} \nabla_{\boldsymbol{\varphi}} J_T(\boldsymbol{\varphi}) \\ &= \sum_{t=0}^{\infty} \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}} \left[ \gamma^t G_t \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) \right]. \end{aligned} \tag{12.49}$$

substituting the score gradient estimator with downstream returns (12.40) and using linearity of expectation (1.20)

Observe that because the expectations only consider downstream returns, we can disregard all data from the trajectory prior to time $t$. Let us define

$$\tau_{t:\infty} \doteq ((\boldsymbol{x}_t, \boldsymbol{a}_t, r_t, \boldsymbol{x}_{t+1}), (\boldsymbol{x}_{t+1}, \boldsymbol{a}_{t+1}, r_{t+1}, \boldsymbol{x}_{t+2}), \dots), \tag{12.50}$$

as the trajectory from time step $t$. Then,

$$= \sum_{t=0}^{\infty} \mathbb{E}_{\tau_{t:\infty} \sim \Pi_{\boldsymbol{\varphi}}} \left[ \gamma^t G_t \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) \right].$$

We now condition on $\boldsymbol{x}_t$ and $\boldsymbol{a}_t$,

$$= \sum_{t=0}^{\infty} \mathbb{E}_{\boldsymbol{x}_t, \boldsymbol{a}_t} \left[ \gamma^t \mathbb{E}_{r_t, \tau_{t+1:\infty}} [G_t \mid \boldsymbol{x}_t, \boldsymbol{a}_t] \, \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) \right].$$

using that $\pi_{\boldsymbol{\varphi}}(a_t \mid x_t)$ is a constant given $x_t$ and $a_t$

Observe that averaging over the trajectories $\mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}[\cdot]$ that are sampled according to policy $\pi_{\boldsymbol{\varphi}}$ is equivalent to our shorthand notation $\mathbb{E}_{\pi_{\boldsymbol{\varphi}}}[\cdot]$ from Equation (10.6),

$$= \sum_{t=0}^{\infty} \mathbb{E}_{\boldsymbol{x}_t, \boldsymbol{a}_t} \left[ \gamma^t \mathbb{E}_{\pi_{\boldsymbol{\varphi}}} [G_t \mid \boldsymbol{x}_t, \boldsymbol{a}_t] \, \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) \right]$$

$$= \sum_{t=0}^{\infty} \mathbb{E}_{\boldsymbol{x}_t, \boldsymbol{a}_t} \left[ \gamma^t q^{\pi_{\boldsymbol{\varphi}}}(\boldsymbol{x}_t, \boldsymbol{a}_t) \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t) \right]. \tag{12.51}$$

using the definition of the Q-function (10.8)

It turns out that $\mathbb{E}_{\boldsymbol{x}_t, \boldsymbol{a}_t}[q^{\pi_{\boldsymbol{\varphi}}}(\boldsymbol{x}_t, \boldsymbol{a}_t)]$ exhibits much less variance than our previous estimator $\mathbb{E}_{\boldsymbol{x}_t, \boldsymbol{a}_t} \mathbb{E}_{\pi_{\boldsymbol{\varphi}}}[G_t \mid \boldsymbol{x}_t, \boldsymbol{a}_t]$. Equation (12.51) is known as the *policy gradient theorem*.

Often, the policy gradient theorem is stated in a slightly rephrased form in terms of the *discounted state occupancy measure*,[13]

$$\rho_{\boldsymbol{\varphi}}^{\infty}(\boldsymbol{x}) \doteq (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t p_{\boldsymbol{X}_t}(\boldsymbol{x}). \tag{12.52}$$

[13] Depending on the reward setting, there exist various variations of the policy gradient theorem. We derived the variant for infinite-horizon discounted payoffs. "Reinforcement learning: An introduction" (Sutton and Barto, 2018) derive the variant for undiscounted average rewards.

The factor $(1 - \gamma)$ ensures that $\rho_{\boldsymbol{\varphi}}^{\infty}$ is a probability density as

$$\int \rho_{\boldsymbol{\varphi}}^{\infty}(\boldsymbol{x}) \, d\boldsymbol{x} = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \int p_{\boldsymbol{X}_t}(\boldsymbol{x}) \, d\boldsymbol{x} = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t = 1.$$

Intuitively, $\rho_{\boldsymbol{\varphi}}^{\infty}(\boldsymbol{x})$ measures how often we visit state $\boldsymbol{x}$ when following policy $\pi_{\boldsymbol{\varphi}}$. It can be thought of as a "discounted frequency".

**Theorem 12.10** (Policy gradient theorem in terms of $\rho_{\boldsymbol{\varphi}}^{\infty}$)**.** *Policy gradients can be represented in terms of the Q-function,*

$$\boldsymbol{\nabla}_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi}) \propto \mathbb{E}_{\boldsymbol{x} \sim \rho_{\boldsymbol{\varphi}}^{\infty}} \mathbb{E}_{\boldsymbol{a} \sim \pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})} \left[ q^{\pi_{\boldsymbol{\varphi}}}(\boldsymbol{x}, \boldsymbol{a}) \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a} \mid \boldsymbol{x}) \right]. \tag{12.53}$$

*Proof.* The right hand side of Equation (12.51) can be expressed as

$$\sum_{t=0}^{\infty} \int p_{\boldsymbol{X}_t}(\boldsymbol{x}) \cdot \mathbb{E}_{\boldsymbol{a}_t \sim \pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})} \left[ \gamma^t q^{\pi_{\boldsymbol{\varphi}}}(\boldsymbol{x}, \boldsymbol{a}_t) \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}) \right] d\boldsymbol{x}$$

$$= \frac{1}{1-\gamma} \int \rho_{\varphi}^{\infty}(x) \cdot \mathbb{E}_{a \sim \pi_{\varphi}(\cdot|x)} \left[ q^{\pi_{\varphi}}(x, a) \nabla_{\varphi} \log \pi_{\varphi}(a \mid x) \right] dx$$

where we swapped the order of sum and integral and reorganized terms. □

Matching our intuition, according to the policy gradient theorem, maximizing $J(\varphi)$ corresponds to increasing the probability of actions with a large value and decreasing the probability of actions with a small value, taking into account how often the resulting policy visits certain states.

Observe that we cannot use the policy gradient to calculate the gradient exactly, as we do not know $q^{\pi_{\varphi}}$. Instead, we will use bootstrapping estimates $Q^{\pi_{\varphi}}$ of $q^{\pi_{\varphi}}$.

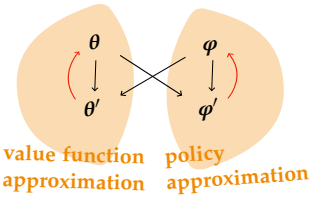



Figure 12.2: Illustration of one iteration of actor-critic methods. The dependencies between the actors and critics are shown as arrows. Methods differ in the exact order in which actor and critic are updated.

### 12.4.3 A First Actor-Critic

Actor-Critic methods consist of two components:

- a parameterized policy, $\pi(a \mid x; \varphi) \doteq \pi_{\varphi}$, which is called *actor*; and (12.54)
- a value function approximation, $q^{\pi_{\varphi}}(x, a) \approx Q^{\pi_{\varphi}}(x, a; \theta)$, which is called *critic*. In the following, we will abbreviate $Q^{\pi_{\varphi}}$ by $Q$. (12.55)

In deep reinforcement learning, neural networks are used to parameterize both actor and critic. Therefore, in principle, the actor-critic framework allows scaling to both large state spaces and large action spaces. We begin by discussing on-policy actor-critics.

One approach in the online setting (i.e., non-episodic setting), is to simply use SARSA for learning the critic. To learn the actor, we use stochastic gradient ascent with gradients obtained using single samples from

$$\nabla_{\varphi} J(\varphi) \approx \nabla_{\varphi} \widehat{J}(\varphi) \doteq \mathbb{E}_{x \sim \rho_{\varphi}^{\infty}} \mathbb{E}_{a \sim \pi_{\varphi}(\cdot|x)} \left[ Q(x, a; \theta) \nabla_{\varphi} \log \pi_{\varphi}(a \mid x) \right] \tag{12.56}$$

see Equation (12.53)

where $Q$ is a bootstrapping estimate of $q^{\pi_{\varphi}}$. This algorithm is known as *online actor-critic* or *Q actor-critic* and shown in Algorithm 12.11.

Comparing to the derivation for TD-learning from Equation (12.8), we observe that Equation (12.58) corresponds to the SARSA update rule.[14] Due to the use of SARSA for learning the critic, this algorithm is fundamentally on-policy.

Crucially, by neglecting the dependence of the bootstrapping estimate $Q$ on the policy parameters $\varphi$, we introduce bias in the gradient estimates. In other words, using the bootstrapping estimate $Q$ means that the resulting gradient direction might not be a valid ascent direction.

[14] The gradient with respect to $\theta$ appears analogously to our derivation of approximate Q-learning (12.15).

**Algorithm 12.11:** Online actor-critic

initialize parameters $\boldsymbol{\varphi}$ and $\boldsymbol{\theta}$
**repeat**
  use $\pi_{\boldsymbol{\varphi}}$ to obtain transition $(\boldsymbol{x}, \boldsymbol{a}, r, \boldsymbol{x}')$ and the next action $\boldsymbol{a}' \sim \pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}')$
  $\delta = r + \gamma Q(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}) - Q(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})$
  // actor update
  $\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} + \eta Q(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a} \mid \boldsymbol{x})$ // (12.57)
  // critic update
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \delta \nabla_{\boldsymbol{\theta}} Q(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})$ // (12.58)
**until** converged

In particular, the actor is not guaranteed to improve. Still, it turns out that under strong so-called "compatibility conditions" that are rarely satisfied in practice, a valid ascent direction can be guaranteed.

### 12.4.4 *Improved Actor-Critics*

*Reducing variance:* To further reduce the variance of the gradient estimates, it turns out that a similar approach to the baselines we discussed in the previous section on policy gradient methods is useful. A common approach is to subtract the state value function from estimates of the Q-function,

$$\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} + \eta_t (Q(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) - V(\boldsymbol{x}; \boldsymbol{\theta})) \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a} \mid \boldsymbol{x}) \tag{12.59}$$

$$= \boldsymbol{\varphi} + \eta_t A(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a} \mid \boldsymbol{x}) \tag{12.60}$$

using the definition of the advantage function (12.44)

where $A(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})$ is a bootstrapped estimate of the advantage function $a^{\pi_{\boldsymbol{\varphi}}}$. This algorithm is known as *advantage actor-critic* (A2C) (Mnih et al., 2016). Recall that the Q-function is an absolute quantity, whereas the advantage function is a relative quantity, where the sign is informative for the gradient direction. Intuitively, an absolute value is harder to estimate than the sign. Actor-Critic methods are therefore often implemented with respect to the advantage function rather than the Q-function.

Taking a step back, observe that policy gradient methods such as REINFORCE generally have *high variance* in their gradient estimates. However, due to using Monte Carlo estimates of $G_t$, the gradient estimates are *unbiased*. In contrast, using a bootstrapped Q-function to obtain gradient estimates yields estimates with a *smaller variance*, but those estimates are *biased*. We are faced with a *bias-variance tradeoff*. A natural approach is therefore to blend both gradient estimates to allow

for effectively trading bias and variance. This leads to algorithms such as *generalized advantage estimation* (GAE) (Schulman et al., 2016).

*Exploration:* Similarly to REINFORCE, actor-critic methods typically rely on randomization in the policy to encourage exploration, the idea being that if the policy is stochastic, then the agent will visit a diverse set of states. The inherent stochasticity of the policy is, however, often insufficient. A common problem is that the policy quickly "collapses" to a deterministic policy since the objective function is greedily exploitative. A common workaround is to use an $\varepsilon$-greedy policy (cf. Section 11.3.1) or to explicitly encourage the policy to exhibit uncertainty by adding an entropy term to the objective function (more on this in Section 12.6). However, note that for on-policy methods, changing the policy also changes the value function learned by the critic.

*Improving sample efficiency:* Actor-Critic methods typically suffer from low sample efficiency. When additionally using an on-policy method, actor-critics often need an extremely large number of interactions before learning a near-optimal policy, because they cannot reuse past data. Allowing to reuse past data is a major advantage of off-policy methods like Q-learning.

One well-known variant that slightly improves the sample efficiency is *trust-region policy optimization* (TRPO) (Schulman et al., 2015). TRPO uses multiple iterations, where in each iteration a fixed critic is used to optimize the policy.[15] During iteration $k$, we select

[15] Intuitively, each iteration performs a collection of gradient ascent steps.

$$\boldsymbol{\varphi}_{k+1} \leftarrow \arg\max_{\boldsymbol{\varphi}} \widehat{J}(\boldsymbol{\varphi}) \quad \text{subject to } \mathbb{E}_{\boldsymbol{x}\sim\rho^{\infty}_{\boldsymbol{\varphi}_k}} \mathrm{KL}\big(\pi_{\boldsymbol{\varphi}_k}(\cdot \mid \boldsymbol{x}) \| \pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})\big) \leq \delta \tag{12.61}$$

for some fixed $\delta > 0$ and where

$$\widehat{J}(\boldsymbol{\varphi}) \doteq \mathbb{E}_{\boldsymbol{x}\sim\rho^{\infty}_{\boldsymbol{\varphi}_k}, \boldsymbol{a}\sim\pi_{\boldsymbol{\varphi}_k}(\cdot|\boldsymbol{x})}\big[w_k(\boldsymbol{\varphi}; \boldsymbol{x}, \boldsymbol{a}) A^{\pi_{\boldsymbol{\varphi}_k}}(\boldsymbol{x}, \boldsymbol{a})\big]. \tag{12.62}$$

Notably, $\widehat{J}$ is an expectation with respect to the *previous* policy $\pi_{\boldsymbol{\varphi}_k}$ and the previous critic $A^{\pi_{\boldsymbol{\varphi}_k}}$. TRPO uses *importance sampling* where the importance weights (called "likelihood ratios"),

$$w_k(\boldsymbol{\varphi}; \boldsymbol{x}, \boldsymbol{a}) \doteq \frac{\pi_{\boldsymbol{\varphi}}(\boldsymbol{a} \mid \boldsymbol{x})}{\pi_{\boldsymbol{\varphi}_k}(\boldsymbol{a} \mid \boldsymbol{x})},$$

are used to correct for taking the expectation over the previous policy. When $w_k \approx 1$ the policies $\pi_{\boldsymbol{\varphi}}$ and $\pi_{\boldsymbol{\varphi}_k}$ are similar, whereas when $w_k \ll 1$ or $w_k \gg 1$, the policies differ significantly. To be able to assume that the fixed critic is a good approximation within a certain "trust region" (i.e., one iteration), we impose the constraint

$$\mathbb{E}_{\boldsymbol{x}\sim\rho^{\infty}_{\boldsymbol{\varphi}_k}} \mathrm{KL}\big(\pi_{\boldsymbol{\varphi}_k}(\cdot \mid \boldsymbol{x}) \| \pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})\big) \leq \delta$$

to optimize only in the "neighborhood" of the current policy. This constraint is also necessary for the importance weights not to blow up.

**Remark 12.12: Estimating the KL-divergence**

Instead of naive computation with $\mathbb{E}_{\boldsymbol{a}\sim\pi_{\boldsymbol{\varphi}_k}(\cdot|\boldsymbol{x})}[-\log w_k(\boldsymbol{\varphi};\boldsymbol{x},\boldsymbol{a})]$, the KL-divergence is commonly estimated by Monte Carlo samples of

$$\begin{aligned}&\mathrm{KL}\big(\pi_{\boldsymbol{\varphi}_k}(\cdot \mid \boldsymbol{x}) \| \pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})\big)\\ &\quad = \mathbb{E}_{\boldsymbol{a}\sim\pi_{\boldsymbol{\varphi}_k}(\cdot|\boldsymbol{x})}[w_k(\boldsymbol{\varphi};\boldsymbol{x},\boldsymbol{a}) - 1 - \log w_k(\boldsymbol{\varphi};\boldsymbol{x},\boldsymbol{a})],\end{aligned}$$

which adds the "baseline" $w_k(\boldsymbol{\varphi};\boldsymbol{x},\boldsymbol{a}) - 1$ with mean 0. Observe that this estimator is unbiased, always non-negative since $\log(x) \leq x - 1$ for all $x$, while having a lower variance than the naive estimator.

Taking the expectation with respect to the previous policy $\pi_{\boldsymbol{\varphi}_k}$ means that we can reuse data from rollouts within the same iteration. That is, TRPO allows reusing past data as long as it can still be "trusted". This makes TRPO "somewhat" off-policy. Fundamentally, though, TRPO is still an on-policy method.

*Proximal policy optimization* (PPO) is a family of heuristic variants of TRPO which replace the constrained optimization problem of Equation (12.61) by the unconstrained optimization of a regularized objective (Schulman et al., 2017; Wang et al., 2020). PPO algorithms often work well in practice. One canonical PPO method uses the modified objective

$$\boldsymbol{\varphi}_{k+1} \leftarrow \arg\max_{\boldsymbol{\varphi}} \widehat{J}(\boldsymbol{\varphi}) - \lambda \mathbb{E}_{\boldsymbol{x}\sim\rho^{\infty}_{\boldsymbol{\varphi}_k}} \mathrm{KL}\big(\pi_{\boldsymbol{\varphi}_k}(\cdot \mid \boldsymbol{x}) \| \pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})\big) \tag{12.63}$$

with some $\lambda > 0$, which regularizes towards the trust region. Another common variant of PPO is based on controlling the importance weights directly rather than regularizing by the KL-divergence. PPO is used, for example, to train large-scale language models such as GPT (Stiennon et al., 2020; OpenAI, 2023) which we will discuss in more detail in Section 12.7. There we will also see that the objective from Equation (12.63) can be cast as performing probabilistic inference.

*Improving computational efficiency:* A practical problem with the above methods is that the estimation of the advantage function $A(\boldsymbol{x},\boldsymbol{a};\boldsymbol{\theta})$ requires training a separate critic, next to the policy (i.e., the actor) parameterized by $\boldsymbol{\varphi}$. This can be computationally expensive. In particular, when both models are large neural networks (think multiple billions of parameters each), training both models is computationally

prohibitive. Now, recall that we introduced critics in the first place to reduce the variance of the policy gradient estimates.[16] *Group relative policy optimization* (GRPO) replaces the critic in PPO with simple Monte Carlo estimates of the advantage function (Shao et al., 2024):

$$\widehat{J}(\boldsymbol{\varphi}) \doteq \mathbb{E}_{\{\tau^{(i)}\}_{i=1}^{m} \sim \Pi_{\boldsymbol{\varphi}_k}(\cdot|\boldsymbol{x})} \left[ \frac{1}{m} \sum_{i=1}^{m} \sum_{t=0}^{T-1} w_k(\boldsymbol{\varphi}; \boldsymbol{a}^{(i)}) \hat{A}_{t,i}^{\pi_{\boldsymbol{\varphi}_k}} \right], \tag{12.64}$$

$$\text{where} \quad \hat{A}_{t,i}^{\pi_{\boldsymbol{\varphi}_k}} \doteq \frac{g_{t:T}^{(i)} - \text{mean}(\{\tau^{(i)}\})}{\text{std}(\{\tau^{(i)}\})}$$

estimates the advantage of action $\boldsymbol{a}^{(i)}$ at time $t$ by comparing to the mean reward and normalizing by the standard deviation of rewards from all trajectories $\tau^{(i)}$.[17] GRPO combines Monte Carlo sampling and baselines for variance reduction with the trust-region optimization of PPO, leading to a method that is more sample efficient than naive REINFORCE while being computationally more efficient than PPO.

[16] That is, we moved from the REINFORCE policy update (12.41) to the actor-critic policy update (12.57).

[17] In Equation (12.36), we have already seen that using a Monte Carlo estimate of returns is a simple approach to reduce variance without needing to learn a critic.

## 12.5 *Off-policy Actor-Critics*

In many applications, sample efficiency is crucial. Either because requiring too many interactions is computationally prohibitive or because obtaining sufficiently many samples for learning a near-optimal policy is simply impossible. We therefore now want to look at a separate family of actor-critic methods, which are off-policy, and hence, allow for the reuse of past data. These algorithms use the reparameterization gradient estimates, which we encountered before in the context of variational inference,[18] instead of score gradient estimators.

[18] see Section 5.5.1

The on-policy methods that we discussed in the previous section can be understood as performing a variant of *policy iteration*, where we use an estimate of the state-action value function of the current policy and then try to improve that policy by acting greedily with respect to this estimate. They mostly vary in how improving the policy is traded with improving the estimate of its value. Fundamentally, these methods rely on policy evaluation.[19]

[19] Policy evaluation is at the core of policy iteration. See Algorithm 10.14 for the definition of policy iteration and Section 10.2 for a summary of policy evaluation in the context of Markov decision processes.

The techniques that we will introduce in this section are much more closely related to value iteration, essentially making use of Bellman's optimality principle to learn the optimal value function directly which characterizes the optimal policy.

To begin with, let us assume that the policy $\boldsymbol{\pi}$ is deterministic. We will later lift this restriction in Section 12.5.1. Recall that our initial motivation to consider policy gradient methods and then actor-critic

methods was the intractability of the DQN loss

$$\ell_{\text{DQN}}(\boldsymbol{\theta}; \mathcal{D}) = \frac{1}{2} \sum_{(\boldsymbol{x}, \boldsymbol{a}, r, \boldsymbol{x}') \in \mathcal{D}} \left( r + \gamma \max_{\boldsymbol{a}' \in \mathcal{A}} Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \right)^2$$

see Equation (12.16)

when the action space $\mathcal{A}$ is large. What if we simply replace the exact maximum over actions by a parameterized policy?

$$\ell_{\text{DQN}}(\boldsymbol{\theta}; \mathcal{D}) \approx \frac{1}{2} \sum_{(\boldsymbol{x}, \boldsymbol{a}, r, \boldsymbol{x}') \in \mathcal{D}} \left( r + \gamma Q^\star(\boldsymbol{x}', \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x}'); \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \right)^2. \tag{12.65}$$

We want to train our parameterized policy to learn the maximization over actions, that is, to approximate the greedy policy[20]

$$\boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x}) \approx \boldsymbol{\pi}^\star(\boldsymbol{x}) = \arg\max_{\boldsymbol{a} \in \mathcal{A}} Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}). \tag{12.66}$$

[20] Here, we already apply the improvement of DDQN to use the most-recent estimate of the Q-function for action selection (see Equation (12.17)).

The key idea is that if we use a "rich-enough" parameterization of policies, selecting the greedy policy with respect to $Q^\star$ is equivalent to

$$\boldsymbol{\varphi}^\star = \arg\max_{\boldsymbol{\varphi}} \mathbb{E}_{\boldsymbol{x} \sim \mu} \left[ Q^\star(\boldsymbol{x}, \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x}); \boldsymbol{\theta}) \right] \tag{12.67}$$

where $\mu(\boldsymbol{x}) > 0$ is an *exploration distribution* over states with full support.[21] We refer to this expectation by

$$\widehat{J}_\mu(\boldsymbol{\varphi}; \boldsymbol{\theta}) \doteq \mathbb{E}_{\boldsymbol{x} \sim \mu} \left[ Q^\star(\boldsymbol{x}, \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x}); \boldsymbol{\theta}) \right]. \tag{12.68}$$

[21] We require full support to ensure that all states are explored.

Commonly, the exploration distribution $\mu$ is taken to be the distribution that samples states uniformly from a replay buffer. Note that we can easily obtain unbiased gradient estimates of $\widehat{J}_\mu$ with respect to $\boldsymbol{\varphi}$:

$$\nabla_{\boldsymbol{\varphi}} \widehat{J}_\mu(\boldsymbol{\varphi}; \boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{x} \sim \mu} \left[ \nabla_{\boldsymbol{\varphi}} Q^\star(\boldsymbol{x}, \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x}); \boldsymbol{\theta}) \right]. \tag{12.69}$$

see Section A.1.5

Analogously to on-policy actor-critics (see Equation (12.56)), we use a bootstrapping estimate of $Q^\star$. That is, we neglect the dependence of the critic $Q^\star$ on the actor $\boldsymbol{\pi}_{\boldsymbol{\varphi}}$, and in particular, the policy parameters $\boldsymbol{\varphi}$. We have seen that bootstrapping works with Q-learning, so there is reason to hope that it will work in this context too. This then allows us to use the chain rule to compute the gradient,

$$\nabla_{\boldsymbol{\varphi}} Q^\star(\boldsymbol{x}, \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x}); \boldsymbol{\theta}) = \boldsymbol{D}_{\boldsymbol{\varphi}} \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x}) \cdot \nabla_{\boldsymbol{a}} Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}) \big|_{\boldsymbol{a} = \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x})}. \tag{12.70}$$

This corresponds to evaluating the bootstrapping estimate of the Q-function at $\boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x})$ and obtaining a gradient estimate of the policy estimate (e.g., through automatic differentiation). Note that as $\boldsymbol{\pi}_{\boldsymbol{\varphi}}$ is vector-valued, $\boldsymbol{D}_{\boldsymbol{\varphi}} \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x})$ is the Jacobian of $\boldsymbol{\pi}_{\boldsymbol{\varphi}}$ evaluated at $\boldsymbol{x}$.

*Exploration:* Now that we have estimates of the gradient of our optimization target $\widehat{J}_\mu$, it is natural to ask how we should select actions (based on $\pi_{\boldsymbol{\varphi}}$) to trade exploration and exploitation. As we have seen, policy gradient techniques rely on the randomness in the policy to explore, but here we consider deterministic policies. As our method is off-policy, a simple idea in continuous action spaces is to add Gaussian noise to the action selected by $\pi_{\boldsymbol{\varphi}}$ — also known as *Gaussian noise "dithering"*.[22] This corresponds to an algorithm called *deep deterministic policy gradients* (Lillicrap et al., 2016) shown in Algorithm 12.13. This algorithm is essentially equivalent to Q-learning with function approximation (e.g., DQN),[23] with the only exception that we replace the maximization over actions with the learned policy $\pi_{\boldsymbol{\varphi}}$.

[22] Intuitively, this adds "additional randomness" to the policy $\pi_{\boldsymbol{\varphi}}$.

[23] see Equation (12.16)

**Algorithm 12.13:** Deep deterministic policy gradients, DDPG

initialize $\boldsymbol{\varphi}, \boldsymbol{\theta}$, a (possibly non-empty) replay buffer $\mathcal{D} = \emptyset$
set $\boldsymbol{\varphi}^{\text{old}} = \boldsymbol{\varphi}$ and $\boldsymbol{\theta}^{\text{old}} = \boldsymbol{\theta}$
**for** $t = 0$ **to** $\infty$ **do**
  observe state $\boldsymbol{x}$, pick action $\boldsymbol{a} = \pi_{\boldsymbol{\varphi}}(\boldsymbol{x}) + \boldsymbol{\varepsilon}$ for $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \lambda \boldsymbol{I},)$
  execute $\boldsymbol{a}$, observe $r$ and $\boldsymbol{x}'$
  add $(\boldsymbol{x}, \boldsymbol{a}, r, \boldsymbol{x}')$ to the replay buffer $\mathcal{D}$
  **if** collected "enough" data **then**
    // policy improvement step
    **for** some iterations **do**
      sample a mini-batch $B$ of $\mathcal{D}$
      for each transition in $B$, compute the label $y = r + \gamma Q^\star(\boldsymbol{x}', \pi(\boldsymbol{x}'; \boldsymbol{\varphi}^{\text{old}}); \boldsymbol{\theta}^{\text{old}})$
      // critic update
      $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \boldsymbol{\nabla}_{\boldsymbol{\theta}} \frac{1}{B} \sum_{(\boldsymbol{x},\boldsymbol{a},r,\boldsymbol{x}',y)\in B} (y - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}))^2$
      // actor update
      $\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} + \eta \boldsymbol{\nabla}_{\boldsymbol{\varphi}} \frac{1}{B} \sum_{(\boldsymbol{x},\boldsymbol{a},r,\boldsymbol{x}',y)\in B} Q^\star(\boldsymbol{x}, \pi(\boldsymbol{x}; \boldsymbol{\varphi}); \boldsymbol{\theta})$
      $\boldsymbol{\theta}^{\text{old}} \leftarrow (1 - \rho)\boldsymbol{\theta}^{\text{old}} + \rho \boldsymbol{\theta}$
      $\boldsymbol{\varphi}^{\text{old}} \leftarrow (1 - \rho)\boldsymbol{\varphi}^{\text{old}} + \rho \boldsymbol{\varphi}$

*Twin delayed DDPG* (TD3) is an extension of DDPG that uses two separate critic networks for predicting the maximum action and evaluating the policy (Fujimoto et al., 2018). This addresses the maximization bias akin to Double-DQN. TD3 also applies delayed updates to the actor network, which increases stability.

### 12.5.1 Randomized Policies

We have seen that randomized policies naturally encourage exploration. With deterministic actor-critic methods like DDPG, we had to inject Gaussian noise to enforce sufficient exploration. A natural question is therefore whether we can also handle randomized policies in this framework of off-policy actor-critics.

The key idea is to replace the squared loss of the critic,

$$\ell_{\text{DQN}}(\boldsymbol{\theta}; \boldsymbol{x}, \boldsymbol{a}, r\boldsymbol{x}') \approx \frac{1}{2}\Big(r + \gamma Q^\star(\boldsymbol{x}', \boldsymbol{\pi}(\boldsymbol{x}'; \boldsymbol{\varphi}); \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})\Big)^2, \qquad \text{refer to the squared loss of Q-learning (12.11)}$$

which only considers the fixed action $\boldsymbol{\pi}(\boldsymbol{x}'; \boldsymbol{\varphi}^{\text{old}})$ with an expected squared loss,

$$\begin{aligned} &\ell_{\text{DQN}}(\boldsymbol{\theta}; \boldsymbol{x}, \boldsymbol{a}, r\boldsymbol{x}') \qquad (12.71)\\ &\quad \approx \mathbb{E}_{\boldsymbol{a}' \sim \pi(\boldsymbol{x}'; \boldsymbol{\varphi})}\left[\frac{1}{2}\Big(r + \gamma Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})\Big)^2\right], \end{aligned}$$

which considers a distribution over actions.

It turns out that we can still compute gradients of this expectation.

$$\begin{aligned} &\nabla_{\boldsymbol{\theta}}\, \mathbb{E}_{\boldsymbol{a}' \sim \pi(\boldsymbol{x}'; \boldsymbol{\varphi})}\left[\frac{1}{2}\Big(r + \gamma Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})\Big)^2\right] \\ &= \mathbb{E}_{\boldsymbol{a}' \sim \pi(\boldsymbol{x}'; \boldsymbol{\varphi})}\left[\nabla_{\boldsymbol{\theta}} \frac{1}{2}\Big(r + \gamma Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})\Big)^2\right]. \qquad \text{see Section A.1.5} \end{aligned}$$

Similarly to our definition of the Bellman error (12.12), we define by

$$\delta_{\text{B}}(\boldsymbol{a}') \doteq r + \gamma Q^\star(\boldsymbol{x}', \boldsymbol{a}'; \boldsymbol{\theta}^{\text{old}}) - Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta}), \qquad (12.72)$$

the *Bellman error* for a fixed action $\boldsymbol{a}'$. Using the chain rule, we obtain

$$= -\mathbb{E}_{\boldsymbol{a}' \sim \pi(\boldsymbol{x}'; \boldsymbol{\varphi})}\left[\delta_{\text{B}}(\boldsymbol{a}') \nabla_{\boldsymbol{\theta}}\, Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})\right]. \qquad (12.73)$$

Note that this is identical to the gradient in DQN (12.14), except that now we have an expectation over actions. As we have done many times already, we can use automatic differentiation to obtain gradient estimates of $\nabla_{\boldsymbol{\theta}}\, Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})$. This provides us with a method of obtaining unbiased gradient estimates for the critic.

We also need to reconsider the actor update. When using a randomized policy, the objective function changes to

$$\widehat{J}_\mu(\boldsymbol{\varphi}; \boldsymbol{\theta}) \doteq \mathbb{E}_{\boldsymbol{x} \sim \mu} \mathbb{E}_{\boldsymbol{a} \sim \pi(\boldsymbol{x}; \boldsymbol{\varphi})}[Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})].$$

of which we can obtain gradients via

$$\nabla_{\boldsymbol{\varphi}}\, \widehat{J}_\mu(\boldsymbol{\varphi}; \boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{x} \sim \mu} \nabla_{\boldsymbol{\varphi}}\, \mathbb{E}_{\boldsymbol{a} \sim \pi(\boldsymbol{x}; \boldsymbol{\varphi})}[Q^\star(\boldsymbol{x}, \boldsymbol{a}; \boldsymbol{\theta})]. \qquad (12.74) \qquad \text{see Section A.1.5}$$

Note that the inner expectation is with respect to a measure that depends on the parameters $\boldsymbol{\varphi}$, which we are trying to optimize. We therefore cannot move the gradient operator inside the expectation. This is a problem that we have already encountered several times. In the previous section on policy gradients, we used the score gradient estimator.[24] Earlier, in Chapter 5 on variational inference we have already seen reparameterization gradients.[25] Here, if our policy is reparameterizable, we can use the *reparameterization trick* from Theorem 5.19!

[24] see Equation (12.32)

[25] see (5.63)

**Example 12.14: Reparameterization gradients for Gaussians**

Suppose we use a Gaussian parameterization of policies,

$$\pi(\boldsymbol{x};\boldsymbol{\varphi}) \doteq \mathcal{N}(\boldsymbol{\mu}(\boldsymbol{x};\boldsymbol{\varphi}), \boldsymbol{\Sigma}(\boldsymbol{x};\boldsymbol{\varphi})).$$

Then, using conditional linear Gaussians, our action $\boldsymbol{a}$ is given by

$$\boldsymbol{a} = \boldsymbol{g}(\boldsymbol{\varepsilon};\boldsymbol{x},\boldsymbol{\varphi}) \doteq \boldsymbol{\Sigma}^{1/2}(\boldsymbol{x};\boldsymbol{\varphi})\boldsymbol{\varepsilon} + \boldsymbol{\mu}(\boldsymbol{x};\boldsymbol{\varphi}), \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}) \tag{12.75}$$

where $\boldsymbol{\Sigma}^{1/2}(\boldsymbol{x};\boldsymbol{\varphi})$ is the square root of $\boldsymbol{\Sigma}(\boldsymbol{x};\boldsymbol{\varphi})$. This coincides with our earlier application of the reparameterization trick to Gaussians in Example 5.20.

As we have seen, not only Gaussians are reparameterizable. In general, we called a distribution (in this context, a policy) reparameterizable iff $\boldsymbol{a} \sim \pi(\boldsymbol{x};\boldsymbol{\varphi})$ is such that $\boldsymbol{a} = \boldsymbol{g}(\boldsymbol{\varepsilon};\boldsymbol{x},\boldsymbol{\varphi})$, where $\boldsymbol{\varepsilon} \sim \phi$ is an independent random variable.

Then, we have,

$$\begin{aligned}
&\nabla_{\boldsymbol{\varphi}} \mathbb{E}_{\boldsymbol{a}\sim\pi(\boldsymbol{x};\boldsymbol{\varphi})}[Q^\star(\boldsymbol{x},\boldsymbol{a};\boldsymbol{\theta})] \\
&= \mathbb{E}_{\boldsymbol{\varepsilon}\sim\phi}\left[\nabla_{\boldsymbol{\varphi}} Q^\star(\boldsymbol{x},\boldsymbol{g}(\boldsymbol{\varepsilon};\boldsymbol{x},\boldsymbol{\varphi});\boldsymbol{\theta})\right] && (12.76) && \text{using the reparameterization trick (5.63)} \\
&= \mathbb{E}_{\boldsymbol{\varepsilon}\sim\phi}\left[\boldsymbol{D}_{\boldsymbol{\varphi}}\boldsymbol{g}(\boldsymbol{\varepsilon};\boldsymbol{x},\boldsymbol{\varphi}) \cdot \nabla_{\boldsymbol{a}} Q^\star(\boldsymbol{x},\boldsymbol{a};\boldsymbol{\theta})\big|_{\boldsymbol{a}=\boldsymbol{g}(\boldsymbol{\varepsilon};\boldsymbol{x},\boldsymbol{\varphi})}\right]. && (12.77) && \text{using the chain rule analogously to Equation (12.70)}
\end{aligned}$$

In this way, we can obtain unbiased gradient estimates for reparameterizable policies. This general technique does not only apply to continuous action spaces. For discrete action spaces, there is the analogous so-called *Gumbel-max trick*, which we will not discuss in greater detail here.

The algorithm that uses Equation (12.73) to obtain gradients for the critic and reparameterization gradients for the actor is called *stochastic value gradients* (SVG) (Heess et al., 2015).

## 12.6 Maximum Entropy Reinforcement Learning

In practice, algorithms like SVG often do not explore enough. A key issue with relying on randomized policies for exploration is that they might collapse to deterministic policies. That is, the algorithm might quickly reach a local optimum, where all mass is placed on a single action.

A simple trick that encourages a little bit of extra exploration is to regularize the randomized policies "away" from putting all mass on a single action. In other words, we want to encourage the policies to exhibit some uncertainty. A natural measure of uncertainty is entropy, which we have already seen several times.[26] This approach is known as *entropy regularization* or *maximum entropy reinforcement learning* (MERL). Canonically, entropy regularization is applied to finite-horizon rewards (cf. Remark 10.5), yielding the optimization problem of maximizing

[26] see Section 5.4

$$J_\lambda(\boldsymbol{\varphi}) \doteq J(\boldsymbol{\varphi}) + \lambda \mathrm{H}\big[\Pi_{\boldsymbol{\varphi}}\big] \tag{12.78}$$

$$= \sum_{t=1}^{T} \mathbb{E}_{(\boldsymbol{x}_t, \boldsymbol{a}_t) \sim \Pi_{\boldsymbol{\varphi}}} \big[ r(\boldsymbol{x}_t, \boldsymbol{a}_t) + \lambda \mathrm{H}\big[\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t)\big] \big], \tag{12.79}$$

where we have a preference for entropy in the actor distribution to encourage exploration which is regulated by the temperature parameter $\lambda$. As $\lambda \to 0$, we recover the "standard" reinforcement learning objective (here for finite-horizon rewards):

$$J(\boldsymbol{\varphi}) = \sum_{t=1}^{T} \mathbb{E}_{(\boldsymbol{x}_t, \boldsymbol{a}_t) \sim \Pi_{\boldsymbol{\varphi}}} [r(\boldsymbol{x}_t, \boldsymbol{a}_t)]. \tag{12.80}$$

Here, for notational convenience, we begin the sum with $t = 1$ rather than $t = 0$.

### 12.6.1 Entropy Regularization as Probabilistic Inference

The entropy-regularized objective from Equation (12.79) leads us to a remarkable interpretation of reinforcement learning and, more generally, decision-making under uncertainty as solving an inference problem akin to variational inference. The framing of "control as inference" will lead us to contemporary algorithms for reinforcement learning as well as paint a path for decision-making under uncertainty beyond stationary MDPs.

Let us denote by $\Pi_\star$ the distribution over trajectories $\tau$ under the optimal policy $\pi^\star$. By framing the problem of optimal control as an inference problem in a hidden Markov model with hidden "optimality variables" $O_t \in \{0, 1\}$ indicating whether the played action $\boldsymbol{a}_t$ was

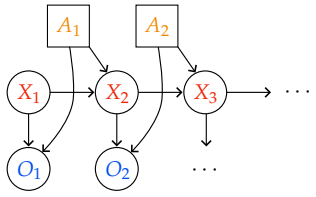



Figure 12.3: Directed graphical model of the underlying hidden Markov model with hidden states $X_t$, optimality variables $O_t$, and actions $A_t$.

optimal we can derive $\Pi_\star$ analytically. That is to say, when $O_t = 1$ and $O_{t+1:T} \equiv 1$ the policy from time $t$ onwards was optimal. To simplify the notation, we will denote the event $O_t = 1$ by $\mathcal{O}_t$.

We consider the HMM defined by the Gibbs distribution

$$p(\mathcal{O}_t \mid \boldsymbol{x}_t, \boldsymbol{a}_t) \propto \exp\left(\frac{1}{\lambda} r(\boldsymbol{x}_t, \boldsymbol{a}_t)\right), \quad \text{with } \lambda > 0 \tag{12.81}$$

which is a natural choice as we have seen in Problem 6.7 that the Gibbs distribution maximizes entropy subject to $\mathbb{E}[O_t \cdot r(\boldsymbol{x}_t, \boldsymbol{a}_t) \mid \boldsymbol{x}_t, \boldsymbol{a}_t] < \infty$.

The distribution over trajectories conditioned on optimality of actions (i.e., conditioned on $\mathcal{O}_{1:T}$) is given by

$$\Pi_\star(\tau) \doteq p(\tau \mid \mathcal{O}_{1:T}) = p(\boldsymbol{x}_1) \prod_{t=1}^{T} p(\boldsymbol{a}_t \mid \boldsymbol{x}_t, \mathcal{O}_t) p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t). \tag{12.82}$$

Using Equation (12.30). We assume here that the dynamics and initial state distribution are "fixed", that is, we assume $p(\boldsymbol{x}_1 \mid \mathcal{O}_{1:T}) = p(\boldsymbol{x}_1)$ and $p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t, \mathcal{O}_{1:T}) = p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t)$.

It remains to determine $p(\boldsymbol{a}_t \mid \boldsymbol{x}_t, \mathcal{O}_t)$ which corresponds to the optimal policy $\pi^\star(\boldsymbol{a}_t \mid \boldsymbol{x}_t)$. It is generally useful to think of the situation where the prior policy $p(\boldsymbol{a}_t \mid \boldsymbol{x}_t)$ is uniform on $\mathcal{A}$,[27] in which case by Bayes' rule (1.45), $p(\boldsymbol{a}_t \mid \boldsymbol{x}_t, \mathcal{O}_t) \propto p(\mathcal{O}_t \mid \boldsymbol{x}_t, \boldsymbol{a}_t)$, so

$$\Pi_\star(\tau) \propto \left[p(\boldsymbol{x}_1) \prod_{t=1}^{T} p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t)\right] \exp\left(\frac{1}{\lambda} \sum_{t=1}^{T} r(\boldsymbol{x}_t, \boldsymbol{a}_t)\right). \tag{12.83}$$

[27] This is not a restriction as any informative prior can be pushed into Equation (12.81).

Recall that our fundamental goal is to approximate $\Pi_\star$ with a distribution over trajectories $\Pi_{\boldsymbol{\varphi}}$ under the parameterized policy $\pi_{\boldsymbol{\varphi}}$. It is therefore a natural idea to minimize $\mathrm{KL}(\Pi_{\boldsymbol{\varphi}} \| \Pi_\star)$:[28]

[28] Observe that we cannot easily minimize forward-KL as we cannot sample from $\Pi_\star$. In the context of RL, it can be argued that the mode-seeking behavior of reverse-KL is preferable over the moment-matching behavior of forward-KL (Levine, 2018).

$$\begin{aligned}
&\arg\min_{\boldsymbol{\varphi}} \mathrm{KL}(\Pi_{\boldsymbol{\varphi}} \| \Pi_\star) \\
&= \arg\min_{\boldsymbol{\varphi}} \mathrm{H}[\Pi_{\boldsymbol{\varphi}} \| \Pi_\star] - \mathrm{H}[\Pi_{\boldsymbol{\varphi}}] && \text{using the definition of KL-divergence (5.34)} \\
&= \arg\max_{\boldsymbol{\varphi}} \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}[\log \Pi_\star(\tau) - \log \Pi_{\boldsymbol{\varphi}}(\tau)] && \text{using the definition of cross-entropy (5.32) and entropy (5.27)} \\
&= \arg\max_{\boldsymbol{\varphi}} \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}\left[\sum_{t=1}^{T} r(\boldsymbol{x}_t, \boldsymbol{a}_t) - \lambda \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t)\right] && \text{using Equations (12.30) and (12.83) and simplifying} \\
&= \arg\max_{\boldsymbol{\varphi}} \sum_{t=1}^{T} \mathbb{E}_{(\boldsymbol{x}_t, \boldsymbol{a}_t) \sim \Pi_{\boldsymbol{\varphi}}}[r(\boldsymbol{x}_t, \boldsymbol{a}_t) + \lambda \mathrm{H}[\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t)]]. && \text{using the definition of entropy (5.27) and linearity of expectation (1.20)}
\end{aligned} \tag{12.84}$$

That is, entropy regularization is equivalent to minimizing the KL-divergence from $\Pi_\star$ to $\Pi_{\boldsymbol{\varphi}}$. This highlights a very natural tradeoff between exploration and exploitation, wherein $\mathrm{H}[\Pi_{\boldsymbol{\varphi}} \| \Pi_\star]$ encourages exploitation and $\mathrm{H}[\Pi_{\boldsymbol{\varphi}}]$ encourages exploration.

It can be shown that a "softmax" version of the Bellman optimality equation (10.34) can be obtained for Equation (12.84) (?):

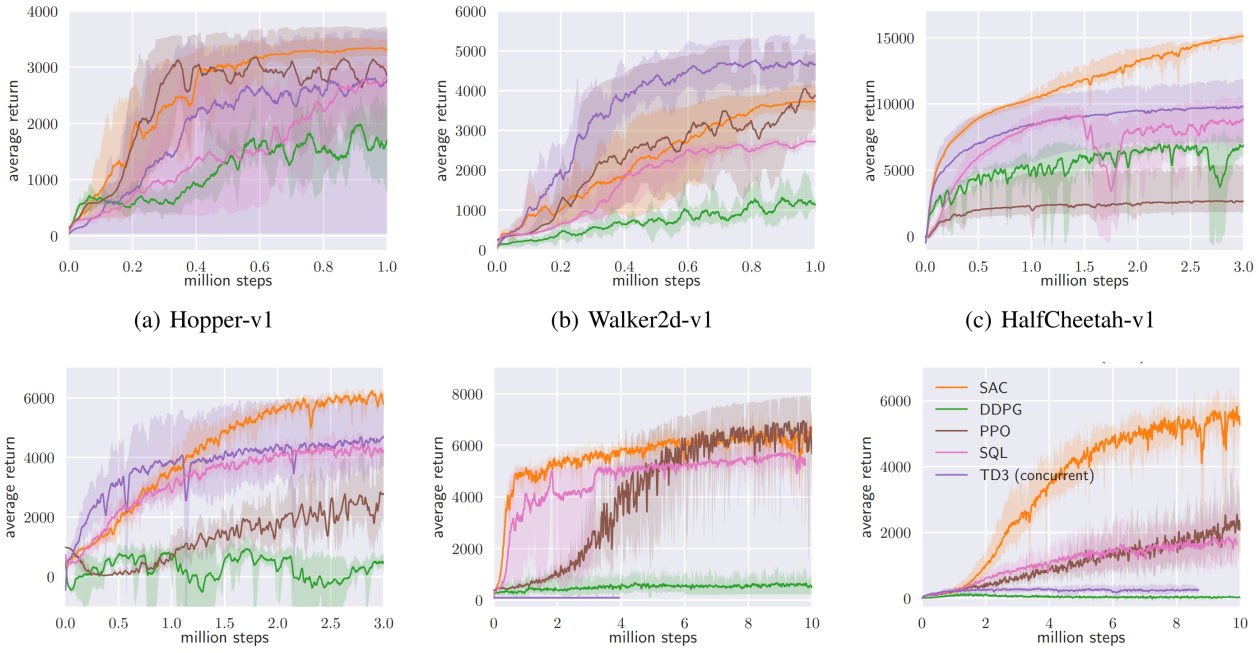
**Problem 12.7**

$$q^\star(\boldsymbol{x}, \boldsymbol{a}) = \frac{1}{\lambda} r(\boldsymbol{x}, \boldsymbol{a}) + \mathbb{E}_{\boldsymbol{x}' \sim \boldsymbol{x}, \boldsymbol{a}}\left[\log \int_{\mathcal{A}} \exp(q^\star(\boldsymbol{x}', \boldsymbol{a}'))\, d\boldsymbol{a}'\right] \tag{12.85}$$

with the convention that $q^\star(\boldsymbol{x}_T, \boldsymbol{a}) = 0$ for all $\boldsymbol{a}$.[29] Here, $q^\star$ is called a *soft value function*. As we will see in Problem 12.7, the optimal policy has the form $\pi^\star(\boldsymbol{a} \mid \boldsymbol{x}) \propto \exp(q^\star(\boldsymbol{x}, \boldsymbol{a}))$, that is, it simply corresponds to performing softmax exploration (11.9) with the soft value function. The second term of Equation (12.85) quantifies downstream rewards. In comparison to the "standard" Bellman optimality equation (10.34), the soft value function is less greedy which tends to encourage robustness.

[29] Note that Equation (10.34) was derived in the infinite horizon setting with discounted rewards, whereas here we study the finite horizon setting. Equation (12.85) is the natural extension of the standard Bellman optimality equation in the finite horizon setting where the downstream rewards are measured by a softmax rather than the greedy policy.

Analogously to Q-learning, the soft value function $q^\star$ can be approximated via a bootstrapped "critic" $Q^\star$ which is called *soft Q-learning* (Levine, 2018). Note that computing the optimal policy requires computing an integral over the action space, which is typically intractable for continuous action spaces. As discussed in Sections 12.3 to 12.5 and analogously to actor-critic methods such as DDPG and SVG, we can learn a parameterized policy (i.e., an "actor") $\pi_{\boldsymbol{\varphi}}$ to approximate the optimal policy $\pi^\star$. The resulting algorithm, *soft actor critic* (SAC) (Haarnoja et al., 2018a,b), is widely used. Due to its off-policy nature, it is also relatively sample efficient.




(a) Hopper-v1 (b) Walker2d-v1 (c) HalfCheetah-v1

(d) Ant-v1 (e) Humanoid-v1 (f) Humanoid (rllab)

Figure 12.4: Comparison of training curves of a selection of on-policy and off-policy policy gradient methods. Reproduced with permission from "Soft actor-critic algorithms and applications" (Haarnoja et al., 2018b).

We can also express this optimization in terms of an evidence lower

bound. The evidence lower bound for the observations $\mathcal{O}_{1:T}$ is

$$L(\Pi_{\boldsymbol{\varphi}}, \Pi_\star; \mathcal{O}_{1:T}) = \mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\big[\log p(\mathcal{O}_{1:T} \mid \tau) + \log \Pi(\tau) - \log \Pi_{\boldsymbol{\varphi}}(\tau)\big] \tag{12.86}$$

using the definition of the ELBO (5.55b)

where $\Pi$ denotes the distribution over trajectories (12.30) under the prior policy $p(\boldsymbol{a}_t \mid \boldsymbol{x}_t)$. This is commonly written as the variational free energy

$$-L(\Pi_{\boldsymbol{\varphi}}, \Pi_\star; \mathcal{O}_{1:T}) = \mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}[\mathrm{S}[p(\mathcal{O}_{1:T} \mid \tau)]] + \mathrm{KL}\big(\Pi_{\boldsymbol{\varphi}} \| \Pi\big) \tag{12.87}$$

$$= \underbrace{\mathrm{S}[p(\mathcal{O}_{1:T})]}_{\text{“extrinsic” value}} + \underbrace{\mathrm{KL}\big(\Pi_{\boldsymbol{\varphi}} \| \Pi_\star\big)}_{\text{“epistemic” value}}. \tag{12.88}$$

using that $\Pi_\star(\tau) = p(\tau \mid \mathcal{O}_{1:T})$

which we already encountered in Section 5.5.2 in the context of variational inference. Here, the “extrinsic” value is independent of the variational distribution $\Pi_{\boldsymbol{\varphi}}$ and can be thought of as a fixed “problem cost”, whereas the “epistemic” value can be interpreted as the approximation error or “solution cost”. To summarize, we have seen that

$$\arg\max_{\boldsymbol{\varphi}} J_\lambda(\boldsymbol{\varphi}) = \arg\min_{\boldsymbol{\varphi}} \mathrm{KL}\big(\Pi_{\boldsymbol{\varphi}} \| \Pi_\star\big) = \arg\max_{\boldsymbol{\varphi}} L(\Pi_{\boldsymbol{\varphi}}, \Pi_\star; \mathcal{O}_{1:T}).$$

Recall that the free energy $-L(\Pi_{\boldsymbol{\varphi}}, \Pi_\star; \mathcal{O}_{1:T})$ is a variational upper bound to the surprise about observations $\mathrm{S}[\Pi_\star(\mathcal{O}_{1:T})]$ when following an optimal policy.[30] For example, undesirable states incur a low reward while desirable states yield a high reward, and thus, if we expect optimality of actions (i.e., $\mathcal{O}_{1:T}$) paths leading to such states have high and low surprise, respectively.[31]

[30] see Section 5.5.2

[31] This is because, a path leading to a low reward state will include sub-optimal actions.

An agent which acts to minimize free energy with $\mathcal{O}_{1:T}$ can be thought of as hallucinating to perform optimally, and acting to minimize the surprise about having played suboptimal actions. Think, for example, about the robotics task of moving an arm to a new position. Intuitively, minimizing free energy solves this task by “hallucinating that the arm is at the goal position”, and then minimizing the surprise with respect to this perturbed world model. In this way, MERL can be understood as identifying paths of least surprise akin to the *free energy principle*.

**Remark 12.15: Towards active inference**

Maximum entropy reinforcement learning makes one fundamental assumption, namely, that the “biased” distribution about observations specifying the underlying HMM is

$$p(\mathcal{O}_t \mid \boldsymbol{x}_t, \boldsymbol{a}_t) \propto \exp\left(\frac{1}{\lambda} r(\boldsymbol{x}_t, \boldsymbol{a}_t)\right).$$

This assumption is key to the framing of optimal control (in an

unknown MDP) with a certain reward function as an inference problem. One could conceive other HMMs. For example, Fellows et al. (2019) propose an HMM defined in terms of the current value function of the agent:

$$p(\mathcal{O}_t \mid \boldsymbol{x}_t, \boldsymbol{a}_t) \propto \exp\left(\frac{1}{\lambda} Q(\boldsymbol{x}_t, \boldsymbol{a}_t; \boldsymbol{\theta})\right).$$

Crucially, the choice of $p(\mathcal{O}_t \mid \boldsymbol{x}_t, \boldsymbol{a}_t)$ is the only place where the reward enters the inference problem, and one can conceive of settings where a "stationary" (i.e., time-independent) reward is not assumed to exist. The general approach to decision-making as probabilistic inference presented in this section (but for possibly reward-independent and non-stationary HMMs) is known as *active inference* (Friston et al., 2015; Millidge et al., 2020, 2021; Parr et al., 2022).

## 12.7 *Learning from Preferences*

So far, we have been assuming that the agent is presented with a reward signal after every played action. This is a natural assumption in domains such as games and robotics — even though it often requires substantial "reward engineering" to break down complex tasks with sparse rewards to more manageable tasks (cf. Section 13.3). In many other domains such as an agent learning to drive a car or a chatbot, it is unclear how one can even quantify the reward associated with an action or a sequence of actions. For example, in the context of autonomous driving it is typically desired that agents behave "human-like" even though a different driving behavior may also reach the destination safely.

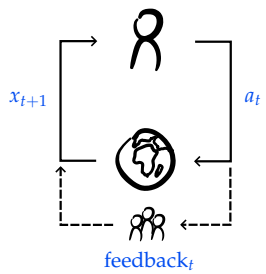



Figure 12.5: We generalize the perspective of reinforcement learning from Figure 11.1 by allowing the feedback to come from either the environment or an evaluation by other agents (e.g., humans), and by allowing the feedback to come in other forms than a numerical reward.

The task of "aligning" the behavior of an agent to human expectations is difficult in complex domains such as the physical world and language, yet crucial for their practical use. To this end, one can conceive of alternative ways for presenting "feedback" to the agent:

- The classical feedback in reinforcement learning is a numerical score. Consider, for example, a recommender system for movies. The feedback is obtained after a movie was recommended to a user by a user-rating on a given scale (often 1 to 10). This rating is informative as it corresponds to an *absolute value assessment*, allowing to place the recommendation in a complete ranking of all previous recommendations. However, numerical feedback of this type can be error-prone as it is scale-dependent (different users may ascribe different value to a recommendation rated a 7).

- An alternative feedback mechanism is comparison-based. The user is presented with $k$ alternative actions and selects their preferred action (or alternatively returns a ranking of actions). This feedback is typically easy to obtain as humans are fast in making "this-or-that" decisions. However, in contrast to numerical rewards, the feedback provides information only on the user's *relative preferences*. That is, such preference feedback encodes fewer bits of information than score feedback, and it therefore often takes longer to learn complex behavior from this feedback.

**Remark 12.16: Context-dependent feedback**

We neglect here that feedback is often context-dependent (Lindner and El-Assady, 2022; Casper et al., 2023). For example, if someone is asked whether they prefer "ice cream" over "pizza" the answer may depend on whether they are hungry and the weather.

### 12.7.1 *Language Models as Agents*

In the following, we discuss approaches to learning from preference feedback in the context of autoregressive[32] large language models and chatbots. A chatbot is an agent (often based on a transformer architecture, Vaswani et al. (2017)) parameterized by $\boldsymbol{\varphi}$ that given a *prompt* $\boldsymbol{x}$ returns a (stochastic) *response* $\boldsymbol{y}$. The autoregressive generation of the response can be understood in terms of a policy $\pi_{\boldsymbol{\varphi}}(y_{t+1} \mid \boldsymbol{x}, y_{1:t})$ which generates the next token given the prompt and all previous tokens.[33] We denote the policy over complete responses (i.e., the chatbot) by

$$\Pi_{\boldsymbol{\varphi}}(\boldsymbol{y} \mid \boldsymbol{x}) = \prod_{t=0}^{T-1} \pi_{\boldsymbol{\varphi}}(y_{t+1} \mid \boldsymbol{x}, y_{1:t}). \tag{12.89}$$

In RL jargon, the agents action corresponds to the choice of next token $y_{t+1}$ and the deterministic dynamics add this token to the current (incomplete) response $y_{1:t}$. A full trajectory $\boldsymbol{y}$ consists sequentially of all tokens $y_{1:T}$ comprising a response, and the prompt $\boldsymbol{x}$ can be interpreted as a *context* to this trajectory. Observe that Equation (12.89) is derived from the general representation of the distribution over trajectories (12.30), noting that prior and dynamics are deterministic.

The standard pipeline for applying pre-trained[34] large language models such as GPT (OpenAI, 2023) to downstream tasks consists of two main steps which are illustrated in Figure 12.6 (Stiennon et al., 2020): (1) supervised fine-tuning and (2) post-training using preference feedback.

The first step is to fine-tune the language model with supervised learning on high-quality data for the downstream task of interest. For ex-

[32] An autoregressive model predicts/-generates the next "token" as a function of previous tokens.

[33] In large language models, a "token" is usually taken to be a letter, word, or something in-between. A special token is used to terminate the response.

[34] The pre-trained language model is usually obtained by self-supervised training on a large corpus of text. *Self-supervised learning* generates labeled training data from an unlabeled data source by selectively "masking-out" parts of the data. When training an autoregressive language model, labeled training data can be obtained by repeatedly "masking-out" the next word in a sentence, and training the language model to predict this word. Such large models that can be fine-tuned and "post-trained" to various downstream tasks are also called *foundation models*.

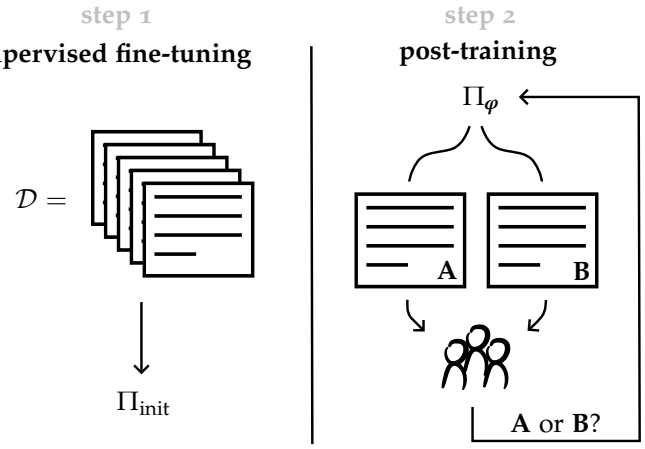



Figure 12.6: Illustration of the learning pipeline of a large language model.

ample, when the goal is to build a chatbot, this data may consist of desirable responses to some exemplary prompts. We will denote the parameters of the fine-tuned language model by $\boldsymbol{\varphi}^{\text{init}}$, and its associated policy by $\Pi_{\text{init}}$.

The second step is then to "post-train" the language model $\Pi_{\text{init}}$ from the first step using human feedback. Here, it is important that $\Pi_{\text{init}}$ is already capable of producing sensible output (i.e., with correct spelling and grammar). Learning this from scratch using only preference feedback would take far too long. Instead, the post-training step is used to align the agent to the task and user preferences.

In each iteration of post-training, the model is prompted with prompts $\boldsymbol{x}$ to produce pairs of answers $(\boldsymbol{y}_A, \boldsymbol{y}_B) \sim \Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})$. These answers are presented to human labelers who express their preference for one of the answers, denoted $\boldsymbol{y}_A \succ \boldsymbol{y}_B \mid \boldsymbol{x}$. A popular choice for modeling preferences is the *Bradley-Terry model* which stipulates that the human preference distribution is given by

$$p(\boldsymbol{y}_A \succ \boldsymbol{y}_B \mid \boldsymbol{x}, r) = \frac{\exp(r(\boldsymbol{y}_A \mid \boldsymbol{x}))}{\exp(r(\boldsymbol{y}_A \mid \boldsymbol{x})) + \exp(r(\boldsymbol{y}_B \mid \boldsymbol{x}))} \tag{12.90}$$

for some unknown latent reward model $r(\boldsymbol{y} \mid \boldsymbol{x})$ (Bradley and Terry, 1952). This can be written in terms of the logistic function $\sigma$ (5.9):

$$= \sigma(r(\boldsymbol{y}_A \mid \boldsymbol{x}) - r(\boldsymbol{y}_B \mid \boldsymbol{x})). \tag{12.91}$$

as seen in Problem 7.1 this is the Gibb's distribution with energy $-r(\boldsymbol{y} \mid \boldsymbol{x})$ in a binary classification problem

**Remark 12.17: Outcome rewards**

Note that the Bradley-Terry model attributes reward only to "complete" responses. We call such a reward an *outcome reward*.[35] While the following discussion is on outcome rewards (which is most common in the context of language models), everything translates to individual per-step rewards.

[35] Framing this in terms of an individual "per-step" rewards which we have seen so far, this corresponds to a (sparse) reward which is zero until the final action.

The aggregated human feedback $\mathcal{D} = \{\boldsymbol{y}_A^{(i)} \succ \boldsymbol{y}_B^{(i)} \mid \boldsymbol{x}^{(i)}\}_{i=1}^n$ across $n$ different prompts is then used to update the language model $\pi_{\boldsymbol{\varphi}}$. In the next two sections, we discuss two standard approaches to post-training: reinforcement learning from human feedback (RLHF) and direct preference optimization (DPO).

### 12.7.2 *Reinforcement Learning from Human Feedback*

RLHF separates the post-training step into two stages (Stiennon et al., 2020). First, the human feedback is used to learn an approximate reward model $r_{\boldsymbol{\theta}}$. This reward model is then used in the second stage to determine a refined policy $\Pi_{\boldsymbol{\varphi}}$.

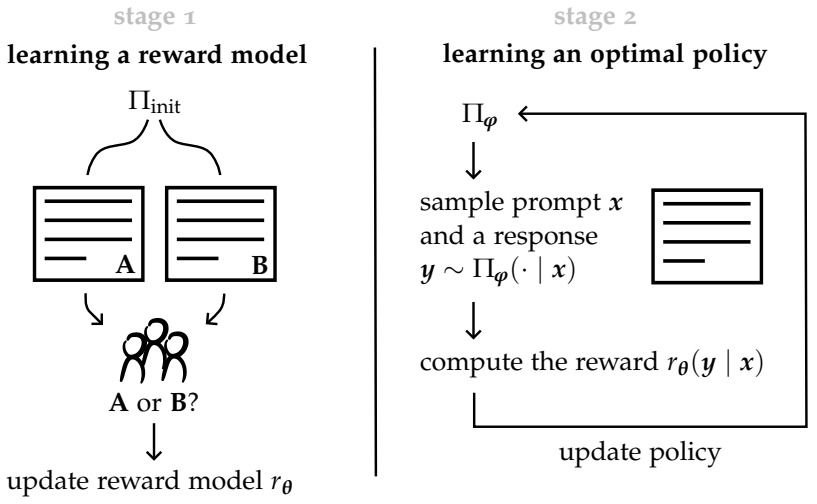



Figure 12.7: Illustration of the post-training process of RLHF.

*Learning a reward model:* During the first stage, the initial policy obtained by supervised fine-tuning $\Pi_{\text{init}}$ is used to generate proposals $(\boldsymbol{y}_A, \boldsymbol{y}_B)$ for some exemplary prompts $\boldsymbol{x}$, which are then ranked according to the preference of human labelers. This preference data can be used to learn a reward model $r_{\boldsymbol{\theta}}$ by maximum likelihood esti-

mation (or equivalently minimizing cross-entropy):

$$\begin{aligned}&\arg\max_{\boldsymbol{\theta}} p(\mathcal{D} \mid r_{\boldsymbol{\theta}}) \\ &= \arg\max_{\boldsymbol{\theta}} \mathbb{E}_{(\boldsymbol{y}_A \succ \boldsymbol{y}_B \mid \boldsymbol{x}) \sim \mathcal{D}}[\log \sigma(r_{\boldsymbol{\theta}}(\boldsymbol{y}_A \mid \boldsymbol{x}) - r_{\boldsymbol{\theta}}(\boldsymbol{y}_B \mid \boldsymbol{x}))]\end{aligned} \tag{12.92}$$

using Equation (12.91) and SGD

This is analogous to the standard maximum likelihood estimation of the reward model in model-based RL with score feedback which we discussed in Section 11.2.1. The reward model $r_{\boldsymbol{\theta}}$ is often initialized from the initial policy $\pi_{\text{init}}$ by placing a linear layer producing a scalar output on top of the final transformer layer.

*Learning an optimal policy:* One can now employ the methods from this and previous chapters to determine the optimal policy for the approximate reward $r_{\boldsymbol{\theta}}$. Due to the use of an *approximate* reward, however, simply maximizing $r_{\boldsymbol{\theta}}$ surfaces the so-called "reward gaming" problem which is illustrated in Figure 12.8. As the responses generated by the learned policy $\pi_{\boldsymbol{\varphi}}$ deviate from the distribution of $\mathcal{D}$ (i.e., the distribution induced by the initial policy $\pi_{\text{init}}$), the approximate reward model becomes inaccurate. The approximate reward may severely overestimate the true reward in regions far away from the training data. A common approach to address this problem is to regularize the policy search towards policies whose distribution does not stray away "too far" from the training distribution.

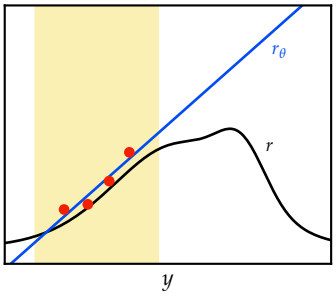



Figure 12.8: Illustration of "reward gaming". Shown in black is the true reward $r(y \mid x)$ for a fixed prompt $x$. Shown in blue is the approximation based on the feedback $\mathcal{D}$ to the responses shown in red. The yellow region symbolizes responses $y$ where the approximate reward can still be "trusted".

Analogously to the trust-region methods we discussed in Section 12.4.4, the deviation from the initial policy is typically controlled by maximizing the regularized objective

$$J_\lambda(\boldsymbol{\varphi}; \boldsymbol{\varphi}^{\text{init}} \mid \boldsymbol{x}) \doteq \mathbb{E}_{\boldsymbol{y} \sim \Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})}[r(\boldsymbol{y} \mid \boldsymbol{x})] - \lambda \mathrm{KL}(\Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}) \| \Pi_{\text{init}}(\cdot \mid \boldsymbol{x}))$$

in expectation over prompts $\boldsymbol{x}$ sampled uniformly at random from the dataset $\mathcal{D}$. Note that this coincides with the PPO objective from Equation (12.63) with an outcome reward. We can expand the regularization term to obtain

$$\begin{aligned}&= \underbrace{\mathbb{E}_{\boldsymbol{y} \sim \Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})}[r(\boldsymbol{y} \mid \boldsymbol{x})] + \lambda \mathrm{H}[\Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})]}_{\text{entropy-regularized RL}} \\ &\qquad - \lambda \mathrm{H}[\Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}) \| \Pi_{\text{init}}(\cdot \mid \boldsymbol{x})]\end{aligned} \tag{12.93}$$

using the definition of KL-divergence (5.34)

which indicates an intimate relationship to entropy-regularized RL.[36]

[36] compare to Equation (12.79)

The optimal policy maximizing $J_\lambda(\boldsymbol{\varphi}; \boldsymbol{\varphi}^{\text{init}} \mid \boldsymbol{x})$ is ⓘ

**Problem 12.8**

$$\Pi_\star(\boldsymbol{y} \mid \boldsymbol{x}) \propto \Pi_{\text{init}}(\boldsymbol{y} \mid \boldsymbol{x}) \exp\left(\frac{1}{\lambda} r(\boldsymbol{y} \mid \boldsymbol{x})\right) \tag{12.94}$$

and can be interpreted as a probabilistic update to the prior $\Pi_{\text{init}}$ where $\exp(\frac{1}{\lambda} r(\boldsymbol{y} \mid \boldsymbol{x}))$ corresponds to the "likelihood of optimality". As $\lambda \to \infty$, $\Pi_\star \to \Pi_{\text{init}}$, and as $\lambda \to 0$, the optimal policy reduces to deterministically picking the response with the highest reward.

In practice, sampling from $\Pi_\star$ explicitly is intractable, but note that any of the approximate inference methods discussed in Part I are applicable here. In the context of chatbots, it is important that the sampling from the resulting policy is efficient (i.e., the chatbot should respond quickly to prompts). Most commonly, the optimization problem of maximizing $j\boldsymbol{\varphi};\boldsymbol{\varphi}^{\text{init}} \mid \boldsymbol{x}[\lambda]$ (with the estimated reward $r_{\boldsymbol{\theta}}$) is solved approximately within a parameterized family of policies. This is typically done using policy gradient methods such as PPO (Stiennon et al., 2020) or GRPO (Guo et al., 2025).

**Remark 12.18: Other (non-preference) reward models**

Note that the post-training pipeline we described here is agnostic to the choice of reward model. That is, instead of post-training our language model to comply with human preferences, we could have also post-trained it to maximize any other reward signal. More recently, works have explored the use of reward models for challenging "reasoning problems" such as mathematical calculations, where the reward is usually based on the correctness of the answer.[37] One prominent example for this are "reasoning" models such as the DeepSeek-R1 model (Guo et al., 2025), which was trained with GRPO.

[37] On training data where the answer to problem $\boldsymbol{x}$ is known to be $y^\star(\boldsymbol{x})$, the reward is simply $r(\boldsymbol{y} \mid \boldsymbol{x}) = \mathbb{1}\{y^\star(\boldsymbol{x}) \in \boldsymbol{y}\}$. That is, the reward is 1 if the response $\boldsymbol{y}$ contains the correct answer and 0 otherwise.

### 12.7.3 *Direct Preference Optimization*

Observe that the reward model can be expressed in terms of its associated optimal policy:

$$r(\boldsymbol{y} \mid \boldsymbol{x}) = \lambda \log \frac{\Pi_\star(\boldsymbol{y} \mid \boldsymbol{x})}{\Pi_{\text{init}}(\boldsymbol{y} \mid \boldsymbol{x})} + \text{const.} \tag{12.95}$$

by reorganizing the terms of Equation (12.94)

In particular, it follows that given a fixed prior $\Pi_{\text{init}}$, *any* policy $\Pi_{\boldsymbol{\varphi}}$ has a family of associated reward models with respect to which it is optimal! We denote by

$$r_{[\boldsymbol{\varphi}]}(\boldsymbol{y} \mid \boldsymbol{x}) \doteq \lambda \log \frac{\Pi_{\boldsymbol{\varphi}}(\boldsymbol{y} \mid \boldsymbol{x})}{\Pi_{\text{init}}(\boldsymbol{y} \mid \boldsymbol{x})} \tag{12.96}$$

the "simplest" of these reward models. Remembering the characterization of the optimal policy from Equation (12.94) it follows immediately that $\Pi_{\boldsymbol{\varphi}}$ is optimal with respect to $r_{[\boldsymbol{\varphi}]}$.

Instead of first learning an approximate reward model and then finding the associated optimal policy, DPO exploits the relationship of

Equation (12.95) to learn the optimal policy directly (Rafailov et al., 2023). Substituting $r_{\boldsymbol{\theta}}$ in the maximum likelihood estimation from Equation (12.92), yields the objective

$$\begin{aligned}
&\mathbb{E}_{(\boldsymbol{y}_A \succ \boldsymbol{y}_B \mid \boldsymbol{x}) \sim \mathcal{D}} \left[ \log \sigma \Big( r_{[\boldsymbol{\varphi}]}(\boldsymbol{y}_A \mid \boldsymbol{x}) - r_{[\boldsymbol{\varphi}]}(\boldsymbol{y}_B \mid \boldsymbol{x}) \Big) \right] \\
&= \mathbb{E}_{(\boldsymbol{y}_A \succ \boldsymbol{y}_B \mid \boldsymbol{x}) \sim \mathcal{D}} \left[ \log \sigma \left( \lambda \log \frac{\Pi_{\boldsymbol{\varphi}}(\boldsymbol{y}_A \mid \boldsymbol{x})}{\Pi_{\text{init}}(\boldsymbol{y}_A \mid \boldsymbol{x})} - \lambda \log \frac{\Pi_{\boldsymbol{\varphi}}(\boldsymbol{y}_B \mid \boldsymbol{x})}{\Pi_{\text{init}}(\boldsymbol{y}_B \mid \boldsymbol{x})} \right) \right]. \qquad \text{using Equation (12.95)}
\end{aligned} \tag{12.97}$$

Gradients can be computed via automatic differentiation:

$$\begin{aligned}
\lambda \mathbb{E}_{(\boldsymbol{y}_A \succ \boldsymbol{y}_B \mid \boldsymbol{x}) \sim \mathcal{D}} \Bigg[ & \underbrace{\sigma \Big( r_{[\boldsymbol{\varphi}]}(\boldsymbol{y}_B \mid \boldsymbol{x}) - r_{[\boldsymbol{\varphi}]}(\boldsymbol{y}_A \mid \boldsymbol{x}) \Big)}_{\text{weight according to error of reward estimate}} \\
& \Big[ \underbrace{\nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\boldsymbol{y}_A \mid \boldsymbol{x})}_{\text{increase likelihood of } \boldsymbol{y}_A} - \underbrace{\nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\boldsymbol{y}_B \mid \boldsymbol{x})}_{\text{decrease likelihood of } \boldsymbol{y}_B} \Big] \Bigg].
\end{aligned} \tag{12.98}$$

Intuitively, DPO successively increases the likelihood of preferred responses $\boldsymbol{y}_A$ and decreases the likelihood of dispreferred responses $\boldsymbol{y}_B$. Examples $(\boldsymbol{y}_A \succ \boldsymbol{y}_B \mid \boldsymbol{x})$ are weighted by the strength of regularization $\lambda$ and by the degree to which the implicit reward model incorrectly orders the responses.

### *Discussion*

In this chapter, we studied central ideas in actor-critic methods. We have seen two main approaches to use policy-gradient methods. We began, in Section 12.3, by introducing the REINFORCE algorithm which uses policy gradients and Monte Carlo estimation, but suffered from large variance in the gradient estimates of the policy value function. In Section 12.4, we have then seen a number of actor-critic methods such as A2C and GAE behaving similarly to policy iteration that exhibit less variance, but are very sample inefficient due to their on-policy nature. TRPO improves the sample efficiency slightly, but not fundamentally.

In Section 12.5, we discussed a second family of policy gradient techniques that generalize Q-learning and are akin to value iteration. For reparameterizable policies, this led us to algorithms such as DDPG, TD3, SVG. Importantly, these algorithms are significantly more sample efficient than on-policy policy gradient methods, which often results in much faster learning of a near-optimal policy. In Section 12.6, we discussed entropy regularization which frames reinforcement learning as probabilistic inference, and we derived the SAC algorithm which is widely used and works quite well in practice.

Finally, in Section 12.7, we studied two canonical approaches to learning from preference feedback: RLHF which separately learns reward model and policy and DPO which learns a policy directly. We have seen that RLHF is akin to model-based RL as it explicitly learns a reward model through maximum likelihood estimation. In contrast, DPO is more closely related to model-free RL and policy gradient methods as it learns the optimal policy directly.

**Optional Readings**

- **A3C:** Mnih, Badia, Mirza, Graves, Lillicrap, Harley, Silver, and Kavukcuoglu (2016).
  *Asynchronous methods for deep reinforcement learning*.
- **GAE:** Schulman, Moritz, Levine, Jordan, and Abbeel (2016).
  *High-dimensional continuous control using generalized advantage estimation*.
- **TRPO:** Schulman, Levine, Abbeel, Jordan, and Moritz (2015).
  *Trust region policy optimization*.
- **PPO:** Schulman, Wolski, Dhariwal, Radford, and Klimov (2017).
  *Proximal policy optimization algorithms*.
- **DDPG:** Lillicrap, Hunt, Pritzel, Heess, Erez, Tassa, Silver, and Wierstra (2016).
  *Continuous control with deep reinforcement learning*.
- **TD3:** Fujimoto, Hoof, and Meger (2018).
  *Addressing function approximation error in actor-critic methods*.
- **SVG:** Heess, Wayne, Silver, Lillicrap, Erez, and Tassa (2015).
  *Learning continuous control policies by stochastic value gradients*.
- **SAC:** Haarnoja, Zhou, Abbeel, and Levine (2018a).
  *Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor*.
- **DPO:** Rafailov, Sharma, Mitchell, Ermon, Manning, and Finn (2023).
  *Direct preference optimization: Your language model is secretly a reward model*.

## *Problems*

### 12.1. Q-learning and function approximation.

Consider the MDP of Figure 12.9 and set $\gamma = 1$.

1. Using Bellman's theorem, prove that $v^\star(x) = -|x-4|$ is the optimal value function.
2. Suppose we observe the following episode:

| $x$ | $a$ | $x'$ | $r$ |
|---|---|---|---|
| 3 | $-1$ | 2 | $-1$ |
| 2 | 1 | 3 | $-1$ |
| 3 | 1 | 4 | $-1$ |
| 4 | 1 | 4 | 0 |

   We initialize all Q-values to 0. Compute the updated Q-values using Q-learning with learning rate $\alpha = 1/2$.
3. We will now approximate the Q-function with a linear function. We let

$$Q(x, a; \boldsymbol{w}) \doteq x w_0 + a w_1 + w_2$$

   where $\boldsymbol{w} = [w_0\ w_1\ w_2]^\top \in \mathbb{R}^3$.
   Suppose we have $\boldsymbol{w}^{\text{old}} = [1\ \ -1\ \ -2]^\top$ and $\boldsymbol{w} = [-1\ 1\ 1]^\top$, and we observe the transition $\tau = (2, -1, -1, 1)$. Use the learning rate $\alpha = 1/2$ to compute $\boldsymbol{\nabla}_{\boldsymbol{w}}\, \ell(\boldsymbol{w}; \tau)$ and the updated weights $\boldsymbol{w}' = \boldsymbol{w} - \alpha \boldsymbol{\nabla}_{\boldsymbol{w}}\, \ell(\boldsymbol{w}; \tau)$.

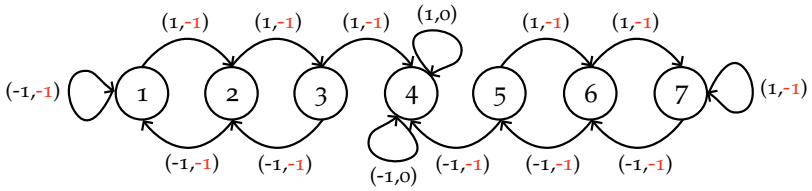



Figure 12.9: MDP studied in Problem 12.1. Each arrow marks a (deterministic) transition and is labeled with (action, reward).

### 12.2. Eligibility vector.

The vector $\boldsymbol{\nabla}_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t)$ is commonly called *eligibility vector*. In the following, we assume that the action space $\mathcal{A}$ is finite and denote it by $A$.

If we parameterize $\pi_{\boldsymbol{\varphi}}$ as a softmax distribution

$$\pi_{\boldsymbol{\varphi}}(a \mid \boldsymbol{x}) \doteq \frac{\exp(h(\boldsymbol{x}, a, \boldsymbol{\varphi}))}{\sum_{b \in A} \exp(h(\boldsymbol{x}, b, \boldsymbol{\varphi}))} \tag{12.99}$$

with linear preferences $h(\boldsymbol{x}, a, \boldsymbol{\varphi}) \doteq \boldsymbol{\varphi}^\top \boldsymbol{\phi}(\boldsymbol{x}, a)$ where $\boldsymbol{\phi}(\boldsymbol{x}, a)$ is some feature vector, what is the form of the eligibility vector?

**12.3. Variance of score gradients with baselines.**

In this exercise, we will see a sufficient condition for baselines to reduce the variance of score gradient estimators.

1. Suppose for a random vector $\mathbf{X}$, we want to estimate $\mathbb{E}[f(\mathbf{X})]$ for some function $f$. Assume that you are given a function $g$ and also its expectation $\mathbb{E}[g(\mathbf{X})]$. Instead of estimating $\mathbb{E}[f(\mathbf{X})]$ directly, we will instead estimate $\mathbb{E}[f(\mathbf{X}) - g(\mathbf{X})]$ as we know from linearity of expectation (1.20) that

$$\mathbb{E}[f(\mathbf{X})] = \mathbb{E}[f(\mathbf{X}) - g(\mathbf{X})] + \mathbb{E}[g(\mathbf{X})].$$

   Prove that if $\frac{1}{2}\mathrm{Var}[g(\mathbf{X})] \leq \mathrm{Cov}[f(\mathbf{X}), g(\mathbf{X})]$, then

$$\mathrm{Var}[f(\mathbf{X}) - g(\mathbf{X})] \leq \mathrm{Var}[f(\mathbf{X})]. \tag{12.100}$$

2. Consider estimating $\nabla_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi})$. Prove that if $b^2 \leq 2b \cdot r(\boldsymbol{x}, \boldsymbol{a})$ for every state $\boldsymbol{x} \in \mathcal{X}$ and action $\boldsymbol{a} \in \mathcal{A}$, then

$$\mathrm{Var}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[(G_{0:T} - b)\nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau)\right] \leq \mathrm{Var}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[G_{0:T}\nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau)\right]. \tag{12.101}$$

**12.4. Score gradients with state-dependent baselines.**

For a sequence of *state-dependent baselines* $\{b(\tau_{0:t-1})\}_{t=1}^{T}$ where

$$\tau_{0:t-1} \doteq (\tau_0, \tau_1, \ldots, \tau_{t-1}),$$

show that

$$\begin{aligned}&\mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[G_{0:T}\nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau)\right] \\ &= \mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[\sum_{t=0}^{T-1}(G_{0:T} - b(\tau_{0:t-1}))\nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t)\right]\end{aligned} \tag{12.102}$$

where we write $b(\tau_{0:-1}) = 0$.

**12.5. Policy gradients with downstream returns.**

Suppose we are training an agent to solve a computer game. There are only two possible actions, specifically:

1. *do nothing*; and
2. *move*.

Each episode lasts for four ($T = 3$) time steps. The policy $\pi_\theta$ is completely determined by the parameter $\theta \in [0, 1]$. Here, for simplicity, we have assumed that the policy is independent of the current state. The probability of moving (action 2) is equal to $\theta$ and the probability of doing nothing (action 1) is $1 - \theta$.

Initially, $\theta = 0.5$. One episode is played with this initial policy and the results are

$$\text{actions} = (1, 0, 1, 0), \quad \text{rewards} = (1, 0, 1, 1).$$

Compute the policy gradient estimate with downstream returns, discount factor $\gamma = 1/2$, and the provided *single* sample $\tau \sim \Pi_\theta$.

**12.6. Policy gradient with an exponential family.**

1. Suppose, we can choose between two actions $a \in \{0, 1\}$ in each state. A natural stochastic policy is induced by a Bernoulli distribution,

$$a \sim \text{Bern}(\sigma(f_{\boldsymbol{\varphi}}(\boldsymbol{x}))), \tag{12.103}$$

   where $\sigma$ is the logistic function from Equation (5.9). First, write down the expression for $\pi_{\boldsymbol{\varphi}}(a \mid \boldsymbol{x})$. Then, derive the expression for $\boldsymbol{\nabla}_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi})$ in terms of $q^{\pi_{\boldsymbol{\varphi}}}$, $\sigma(f_{\boldsymbol{\varphi}}(\boldsymbol{x}))$, and $\boldsymbol{\nabla}_{\boldsymbol{\varphi}} f_{\boldsymbol{\varphi}}(\boldsymbol{x})$ using the policy gradient theorem.
2. The Bernoulli distribution is part of a family of distributions that allows for a much simpler derivation of the gradient than was necessary in (1). A univariate *exponential family* is a family of distributions whose PDF (or PMF) can be expressed in canonical form as

$$\pi_{\boldsymbol{\varphi}}(a \mid \boldsymbol{x}) = h(a) \exp(a f_{\boldsymbol{\varphi}}(\boldsymbol{x}) - A(f_{\boldsymbol{\varphi}}(\boldsymbol{x}))) \tag{12.104}$$

   where $h$, $f$, and $A$ are known functions.[38] Derive the expression of the policy gradient $\boldsymbol{\nabla}_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi})$ for such a distribution.
3. Can you relate the results of the previous two exercises (1) and (2)? What are $h$ and $A$ in case of the Bernoulli distribution?
4. The Gaussian distribution with unit variance $\mathcal{N}(f_{\boldsymbol{\varphi}}(\boldsymbol{x}), 1)$ is of the same canonical form with

$$A(f_{\boldsymbol{\varphi}}(\boldsymbol{x})) = \frac{f_{\boldsymbol{\varphi}}(\boldsymbol{x})^2}{2}. \tag{12.105}$$

   Determine the policy gradient $\boldsymbol{\nabla}_{\boldsymbol{\varphi}} J(\boldsymbol{\varphi})$.
5. For a Gaussian policy, can we instead apply the reparameterization trick (5.65) that we have seen in the context of variational inference? If yes, how? If not, why?

[38] Observe that this form is equivalent to the form introduced in Equation (5.48) where $f_{\boldsymbol{\varphi}}(\boldsymbol{x})$ is the natural parameter, and we let $A(f) = \log Z(f)$.

**12.7. Soft value function.**

In this exercise, we derive the optimal policy solving Equation (12.84) and the soft value function from Equation (12.85).

1. We let $\beta(\boldsymbol{a}_t \mid \boldsymbol{x}_t) \doteq \exp(\frac{1}{\lambda} r(\boldsymbol{x}_t, \boldsymbol{a}_t))$, $Z(\boldsymbol{x}) \doteq \int_{\mathcal{A}} \beta(\boldsymbol{a} \mid \boldsymbol{x}) \, d\boldsymbol{a}$, and denote by $\hat{\pi}(\cdot \mid \boldsymbol{x})$ the policy $\beta(\cdot \mid \boldsymbol{x}) / Z(\boldsymbol{x})$. Show that

$$\begin{aligned} &\mathrm{KL}\big(\Pi_{\boldsymbol{\varphi}} \| \Pi_\star\big) \\ &\quad = \sum_{t=1}^{T} \mathbb{E}_{\boldsymbol{x}_t \sim \Pi_{\boldsymbol{\varphi}}} \big[\mathrm{KL}\big(\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t) \| \hat{\pi}(\cdot \mid \boldsymbol{x}_t)\big) - \log Z(\boldsymbol{x}_t)\big]. \end{aligned} \tag{12.106}$$

2. Show that if the space of policies parameterized by $\boldsymbol{\varphi}$ is sufficiently expressive, $\pi^\star(\boldsymbol{a} \mid \boldsymbol{x}) \propto \exp(q^\star(\boldsymbol{x}, \boldsymbol{a}))$ solves Equation (12.84).

**12.8. PPO as probabilistic inference.**

1. Consider the same generative model as was introduced in Section 12.6.1 when we interpreted entropy-regularized RL as probabilistic inference. Only now, assume that $T = 1$ [39] and suppose that the prior over actions is $p(\boldsymbol{y} \mid \boldsymbol{x}) = \Pi_{\text{init}}(\boldsymbol{y} \mid \boldsymbol{x})$ rather than uniform. Define the distribution over optimal trajectories $\Pi_\star(\boldsymbol{y} \mid \boldsymbol{x})$ as before in Equation (12.82). Show that for any context $\boldsymbol{x}$,

$$\arg\min_{\boldsymbol{\varphi}} \mathrm{KL}\big(\Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}) \| \Pi_\star(\cdot \mid \boldsymbol{x})\big) = \arg\max_{\boldsymbol{\varphi}} J_\lambda(\boldsymbol{\varphi}; \boldsymbol{\varphi}^{\text{init}} \mid \boldsymbol{x}). \tag{12.107}$$

2. Conclude that the policy maximizing $J_\lambda(\boldsymbol{\varphi}; \boldsymbol{\varphi}^{\text{init}} \mid \boldsymbol{x})$ is

$$\Pi_\star(\boldsymbol{y} \mid \boldsymbol{x}) \propto \Pi_{\text{init}}(\boldsymbol{y} \mid \boldsymbol{x}) \exp\left(\frac{1}{\lambda} r(\boldsymbol{y} \mid \boldsymbol{x})\right). \tag{12.108}$$

[39] since, in this section, we have been considering outcome rewards

# 13

# *Model-based Reinforcement Learning*

In this final chapter, we will revisit the model-based approach to reinforcement learning. We will see some advantages it offers over model-free methods. In particular, we will use the machinery of probabilistic inference, which we developed in the first chapters, to quantify uncertainty about our model and use this uncertainty for planning, exploration, and reliable decision-making.

To recap, in Chapter 11, we began by discussing model-based reinforcement learning which attempts to learn the underlying Markov decision process and then use it for planning. We have seen that in the tabular setting, computing and storing the entire model is computationally expensive. This led us to consider the family of model-free approaches, which learn the value function directly, and as such can be considered more economical in the amount of data that they store.

In Chapter 12, we saw that using function approximation, we were able to scale model-free methods to very large state and action spaces. We will now explore similar ideas in the model-based framework. Namely, we will use function approximation to condense the representation of our model of the environment. More concretely, we will learn an approximate dynamics model $f \approx p$ and approximate rewards $r$, which is also called a *world model*.

There are a few ways in which the model-based approach is advantageous. First, if we have an accurate model of the environment, we can use it for planning — ideally also for interacting safely with the environment. However, in practice, we will rarely have such a model. In fact, the accuracy of our model will depend on the past experience of our agent and the region of the state-action space. Understanding the uncertainty in our model of the environment is crucial for planning. In particular, quantifying uncertainty is necessary to drive safe(!) exploration and avoid undesired states.

Moreover, modeling uncertainty in our model of the environment can be extremely useful in deciding where to explore. Learning a model can therefore help to dramatically reduce the sample complexity over model-free techniques. Often times this is crucial when developing agents for real-world use as in such settings, exploration is costly and potentially dangerous.

Algorithm 13.1 describes the general approach to model-based reinforcement learning.

**Algorithm 13.1:** Model-based reinforcement learning (outline)

```
start with an initial policy π and no (or some) initial data 𝒟
for several episodes do
    roll out policy π to collect data
    learn a model of the dynamics f and rewards r from data
    plan a new policy π based on the estimated models
```

We face three main challenges in model-based reinforcement learning. First, given a fixed model, we need to perform planning to decide on which actions to play. Second, we need to learn models $f$ and $r$ accurately and efficiently. Third, we need to effectively trade exploration and exploitation. We will discuss these three topics in the following.

## *13.1 Planning*

There exists a large literature on planning in various settings. These settings can be mainly characterized as

- discrete or continuous action spaces;
- fully- or partially observable state spaces;
- constrained or unconstrained; and
- linear or nonlinear dynamics.

In Chapter 10, we have already seen algorithms such as policy iteration and value iteration, which can be used to solve planning exactly in tabular settings. In the following, we will now focus on the setting of continuous state and action spaces, fully observable state spaces, no constraints, and nonlinear dynamics.

### *13.1.1 Deterministic Dynamics*

To begin with, let us assume that our dynamics model is deterministic and known. That is, given a state-action pair, we know the subsequent state,

$$\boldsymbol{x}_{t+1} = \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{a}_t). \tag{13.1}$$

We continue to focus on the setting of infinite-horizon discounted returns (10.5), which we have been considering throughout our discussion of reinforcement learning. This yields the objective,

$$\max_{\boldsymbol{a}_{0:\infty}} \sum_{t=0}^{\infty} \gamma^t r(\boldsymbol{x}_t, \boldsymbol{a}_t) \quad \text{such that } \boldsymbol{x}_{t+1} = \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{a}_t). \tag{13.2}$$

Now, because we are optimizing over an infinite time horizon, we cannot solve this optimization problem directly. This problem is studied ubiquitously in the area of *optimal control*. We will discuss one central idea from optimal control that is widely used in model-based reinforcement learning, and will later return to using this idea for learning parametric policies in Section 13.1.3.

*Planning over finite horizons:* The key idea of a classical algorithm from optimal control called *receding horizon control* (RHC) or *model predictive control* (MPC) is to iteratively plan over finite horizons. That is, in each round, we plan over a finite time horizon $H$ and carry out the first action.

**Algorithm 13.2:** Model predictive control, MPC

1 **for** $t = 0$ **to** $\infty$ **do**
2 observe $\boldsymbol{x}_t$
3 plan over a finite horizon $H$,
$$\max_{\boldsymbol{a}_{t:t+H-1}} \sum_{\tau=t}^{t+H-1} \gamma^{\tau-t} r(\boldsymbol{x}_\tau, \boldsymbol{a}_\tau) \quad \text{such that } \boldsymbol{x}_{\tau+1} = \boldsymbol{f}(\boldsymbol{x}_\tau, \boldsymbol{a}_\tau) \tag{13.3}$$
4 carry out action $\boldsymbol{a}_t$

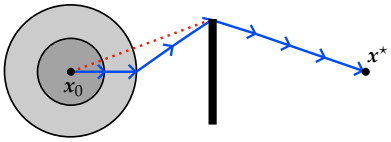



Figure 13.1: Illustration of model predictive control in a deterministic transition model. The agent starts in position $\boldsymbol{x}_0$ and wants to reach $\boldsymbol{x}^\star$ despite the black obstacle. We use the reward function $r(\boldsymbol{x}) = -\|\boldsymbol{x} - \boldsymbol{x}^\star\|$. The gray concentric circles represent the length of a single step. We plan with a time horizon of $H = 2$. Initially, the agent does not "see" the black obstacle, and therefore moves straight towards the goal. As soon as the agent sees the obstacle, the optimal trajectory is "replanned". The dotted red line corresponds to the optimal trajectory, the agent's steps are shown in blue.

Observe that the state $\boldsymbol{x}_\tau$ can be interpreted as a deterministic function $\boldsymbol{x}_\tau(\boldsymbol{a}_{t:\tau-1})$, which depends on all actions from time $t$ to time $\tau - 1$ and the state $\boldsymbol{x}_t$. To solve the optimization problem of a single iteration, we therefore need to maximize,

$$J_H(\boldsymbol{a}_{t:t+H-1}) \doteq \sum_{\tau=t}^{t+H-1} \gamma^{\tau-t} r(\boldsymbol{x}_\tau(\boldsymbol{a}_{t:\tau-1}), \boldsymbol{a}_\tau). \tag{13.4}$$

This optimization problem is in general non-convex. If the actions are continuous and the dynamics and reward models are differentiable, we can nevertheless obtain analytic gradients of $\widehat{J}_H$. This can be done using the chain rule and "backpropagating" through time, analogously to backpropagation in neural networks.[1] Especially for large horizons $H$, this optimization problem becomes difficult to solve exactly due to local optima and vanishing/exploding gradients.

[1] see Section 7.1.4

*Tree search:* Often, heuristic global optimization methods (also called "search methods") are used to optimize $\widehat{J}_H$. An example are *random shooting methods*, which find the optimal choice among a set of random proposals. Of course, obtaining a "good" set of randomly proposed action sequences is crucial. A naive way of generating the proposals is to pick them uniformly at random. This strategy, however, usually does not perform very well as it corresponds to suggesting random walks of the state space. Alternatives are to sample from a Gaussian distribution or using the *cross-entropy method* which gradually adapts the sampling distribution by reweighing samples according to the rewards they produce.

**Algorithm 13.3:** Random shooting methods

1 generate $m$ sets of random samples, $\boldsymbol{a}_{t:t+H-1}^{(i)}$

2 pick the sequence of actions $\boldsymbol{a}_{t:t+H-1}^{(i^\star)}$ where

$$i^\star = \arg\max_{i \in [m]} J_H(\boldsymbol{a}_{t:t+H-1}^{(i)}) \tag{13.5}$$

The evolution of the state can be visualized as a tree where — if dynamics are deterministic — the branching is fully determined by the played actions. For this reason, classical tree search algorithms can be employed, such as *alpha-beta pruning* or *Monte Carlo tree search* (MCTS), which was used for example by "AlphaZero" (Silver et al., 2016, 2017) and can be viewed as an advanced variant of a shooting method.

*Finite-horizon planning with sparse rewards:* A common problem of finite-horizon methods is that in the setting of sparse rewards, there is often no signal that can be followed. You can think of an agent that operates in a kitchen and tries to find a box of candy. Yet, to get to this box, it needs to perform a large number of actions. In particular, if this number of actions is larger than the horizon $H$, then the local optimization problem of MPC will not take into account the reward for choosing this long sequence of actions. Thus, the box of candy will likely never be found by our agent.

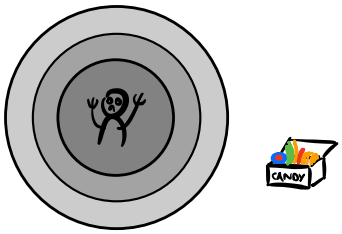



Figure 13.2: Illustration of finite-horizon planning with sparse rewards. When the finite time horizon does not suffice to "reach" a reward, the agent has no signal to follow.

A solution to this problem is to amend a long-term value estimate to the finite-horizon sum. The idea is to not only consider the rewards attained *while* following the actions $\boldsymbol{a}_{t:t_H-1}$, but to also consider the value of the final state $\boldsymbol{x}_{t+H}$, which estimates the discounted sum of

*future* rewards.

$$\widehat{J}_H(\boldsymbol{a}_{t:t+H-1}) \doteq \underbrace{\sum_{\tau=t}^{t+H-1} \gamma^{\tau-t} r(\boldsymbol{x}_\tau(\boldsymbol{a}_{t:\tau-1}), \boldsymbol{a}_\tau)}_{\text{short-term}} + \underbrace{\gamma^H V(\boldsymbol{x}_{t+H})}_{\text{long-term}}. \tag{13.6}$$

Intuitively, $\gamma^H V(\boldsymbol{x}_{t+H})$ is estimating the tail of the infinite sum.

**Remark 13.4: Planning generalizes model-free methods!**

Observe that for $H = 1$, when we use the value function estimate associated with this MPC controller, maximizing $J_H$ coincides with using the greedy policy $\pi_V$. That is,

$$\boldsymbol{a}_t = \arg\max_{\boldsymbol{a} \in \mathcal{A}} \widehat{J}_1(\boldsymbol{a}) = \pi_V(\boldsymbol{x}_t). \tag{13.7}$$

Thus, by looking ahead for a single time step, we recover the approaches from the model-free setting in this model-based setting! Essentially, if we do not plan long-term and only consider the value estimate, the model-based setting reduces to the model-free setting. However, in the model-based setting, we are now able to use our model of the transition dynamics to anticipate the downstream effects of picking a particular action $\boldsymbol{a}_t$. This is one of the fundamental reasons for why model-based approaches are typically severely more sample efficient than model-free methods.

To obtain the value estimates, we can use the approaches we discussed in detail in Section 11.4, such as TD-learning for on-policy value estimates and Q-learning for off-policy value estimates. For large state-action spaces, we can use their approximate variants, which we discussed in Section 12.1 and Section 12.2. To improve value estimates, we can obtain "artificial" data by rolling out policies within our model. This is a key advantage over model-free methods as, once a sufficiently accurate model has been learned, data for value estimation can be generated efficiently in simulation.

### *13.1.2 Stochastic Dynamics*

How can we extend this approach to planning to a stochastic transition model? A natural extension of model predictive control is to do what is called *stochastic average approximation* (SAA) or *trajectory sampling* (Chua et al., 2018). Like in MPC, we still optimize over a deterministic sequence of actions, but now we will average over all resulting trajectories.

**Algorithm 13.5:** Trajectory sampling

**for** $t = 0$ **to** $\infty$ **do**

observe $\boldsymbol{x}_t$

optimize expected performance over a finite horizon $H$,

$$\max_{\boldsymbol{a}_{t:t+H-1}} \mathbb{E}_{\boldsymbol{x}_{t+1:t+H}} \left[ \sum_{\tau=t}^{t+H-1} \gamma^{\tau-t} r_\tau + \gamma^H V(\boldsymbol{x}_{t+H}) \,\middle|\, \boldsymbol{a}_{t:t+H-1}, f \right] \tag{13.8}$$

carry out action $\boldsymbol{a}_t$

Intuitively, trajectory sampling optimizes over a much simpler object — namely a deterministic sequence of actions of length $H$ — than finding a policy, which corresponds to finding an optimal mapping from states to actions across all states. Of course, using trajectory sampling (from an arbitrary starting state) implies such a policy. However, trajectory sampling never computes this policy explicitly, and rather, in each step, only plans over a finite horizon.



Figure 13.3: Illustration of trajectory sampling. High-reward states are shown in brighter colors. The agent iteratively plans a finite number of time steps into the future and picks the best initial action.

Computing the expectation exactly is typically not possible as this involves solving a high-dimensional integral of nonlinear functions. Instead, a common approach is to use Monte Carlo estimates of this expectation. This approach is known as *Monte Carlo trajectory sampling*. The key issue with using sampling based estimation is that the sampled trajectory (i.e., sampled sequence of states) we obtain, depends on the actions we pick. In other words, the measure we average over depends on the decision variables — the actions. This is a problem that we have seen several times already! It naturally suggests using the reparameterization trick.[2]

[2] see Theorem 5.19 and Equation (12.76)

Previously, we have used the reparameterization trick to reparameterize variational distributions (see Section 5.5.1) and to reparameterize policies (see Section 12.5.1). It turns out that we can use the exact same approach for reparameterizing the transition model. We say that a (stochastic) transition model $f$ is *reparameterizable* iff $\boldsymbol{x}_{t+1} \sim f(\boldsymbol{x}_t, \boldsymbol{a}_t)$ is such that $\boldsymbol{x}_{t+1} = g(\varepsilon; \boldsymbol{x}_t, \boldsymbol{a}_t)$, where $\varepsilon \sim \phi$ is a random variable that is independent of $\boldsymbol{x}_t$ and $\boldsymbol{a}_t$. We have already seen in Example 12.14 (in the context of stochastic policies) how a Gaussian transition model can be reparameterized.

In this case, $\boldsymbol{x}_\tau$ is determined recursively by $\boldsymbol{a}_{t:\tau-1}$ and $\varepsilon_{t:\tau-1}$,

$$\begin{aligned} \boldsymbol{x}_\tau &= \boldsymbol{x}_\tau(\varepsilon_{t:\tau-1}; \boldsymbol{a}_{t:\tau-1}) \\ &\doteq g(\varepsilon_{\tau-1}; g(\dots; (\varepsilon_{t+1}; g(\varepsilon_t; \boldsymbol{x}_t, \boldsymbol{a}_t), \boldsymbol{a}_{t+1}), \dots), \boldsymbol{a}_{\tau-1}). \end{aligned} \tag{13.9}$$

This allows us to obtain unbiased estimates of $J_H$ using Monte Carlo approximation,

$$\widehat{J}_H(\boldsymbol{a}_{t:t+H-1}) \approx \frac{1}{m}\sum_{i=1}^{m}\left(\sum_{\tau=t}^{t+H-1}\gamma^{\tau-t}r(\boldsymbol{x}_\tau(\boldsymbol{\varepsilon}^{(i)}_{t:\tau-1};\boldsymbol{a}_{t:\tau-1}),\boldsymbol{a}_\tau) + \gamma^H V(\boldsymbol{x}_{t+H}(\boldsymbol{\varepsilon}^{(i)}_{t:t+H-1};\boldsymbol{a}_{t:t+H-1}))\right) \tag{13.10}$$

where $\boldsymbol{\varepsilon}^{(i)}_{t:t+H-1} \overset{\text{iid}}{\sim} \phi$ are independent samples. To optimize this approximation we can again compute analytic gradients or use shooting methods as we have discussed in Section 13.1.1 for deterministic dynamics.

### 13.1.3 Parametric Policies

When using algorithms such as model predictive control for planning, planning needs to be done online before each time we take an action. This is called *closed-loop control* and can be expensive. Especially when the time horizon is large, or we encounter similar states many times (leading to "repeated optimization problems"), it can be beneficial to "store" the planning decision in a (deterministic) policy,

$$\boldsymbol{a}_t = \boldsymbol{\pi}(\boldsymbol{x}_t;\boldsymbol{\varphi}) \doteq \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x}_t). \tag{13.11}$$

This policy can then be trained offline and evaluated cheaply online, which is known as *open-loop control*.

This is akin to a problem that we have discussed in detail in the previous chapter when extending Q-learning to large action spaces. There, this led us to discuss policy gradient and actor-critic methods. Recall that in Q-learning, we seek to follow the greedy policy,

$$\boldsymbol{\pi}^\star(\boldsymbol{x}) = \arg\max_{\boldsymbol{a}\in\mathcal{A}} Q^\star(\boldsymbol{x},\boldsymbol{a};\boldsymbol{\theta}), \qquad \text{see Equation (12.20)}$$

and therefore had to solve an optimization problem over all actions. We accelerated this by learning an approximate policy that "mimicked" this optimization,

$$\boldsymbol{\varphi}^\star = \arg\max_{\boldsymbol{\varphi}} \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\mu}\left[Q^\star(\boldsymbol{x},\boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x});\boldsymbol{\theta})\right]}_{\widehat{J}_\mu(\boldsymbol{\varphi};\boldsymbol{\theta})} \qquad \text{see Equation (12.67)}$$

where $\mu(\boldsymbol{x}) > 0$ was some exploration distribution that has full support and thus leads to the exploration of all states. The key idea was that if we use a differentiable approximation $Q$ and a differentiable parameterization of policies, which is "rich enough", then both optimizations are equivalent, and we can use the chain rule to obtain

gradient estimates of the second expression. We then used this to derive the *deep deterministic policy gradients* (DDPG) and *stochastic value gradients* (SVG) algorithms. It turns out that there is a very natural analogue to DDPG/SVG for model-based reinforcement learning.

Instead of maximizing the Q-function directly, we use finite-horizon planning to estimate the immediate value of the policy within the next $H$ time steps and simply use the Q-function to approximate the terminal value (i.e., the tails of the infinite sum). Then, our objective becomes,

$$\widehat{J}_{\mu,H}(\boldsymbol{\varphi};\boldsymbol{\theta}) \doteq \mathbb{E}_{\boldsymbol{x}_0\sim\mu,\boldsymbol{x}_{1:H}|\boldsymbol{\pi}_{\boldsymbol{\varphi}},f}\left[\sum_{\tau=0}^{H-1}\gamma^\tau r_\tau + \gamma^H Q^\star(\boldsymbol{x}_H, \boldsymbol{\pi}_{\boldsymbol{\varphi}}(\boldsymbol{x}_H);\boldsymbol{\theta})\right] \tag{13.12}$$

This approach naturally extends to randomized policies using reparameterization gradients, which we have discussed in Section 12.5.1. Analogously to Remark 13.4, for $H = 0$, this coincides with the DDPG objective! For larger time horizons, the look-ahead takes into account the transition model for planning next time steps. This tends to help dramatically in improving policies *much more rapidly* between episodes. Instead of just gradually improving policies a little by slightly adapting the policy to the learned value function estimates (as in model-free RL), we use the model to anticipate the consequences of actions multiple time steps ahead. This is at the heart of model-based reinforcement learning.

Essentially, we are using methods such as Q-learning and DDPG/SVG as subroutines within the framework of model predictive control to do much bigger steps in policy improvement than to slightly improve the next picked action. To encourage exploration, it is common to extend the objective in Equation (13.12) by an additional entropy regularization term as seen in Section 12.6.

**Remark 13.6: Planning as inference**

We have been using *Monte Carlo rollouts* to estimate the expectation of Equation (13.12). That is, we have been using a Monte Carlo approximation (e.g., a sample mean) using samples obtained by "rolling out" the induced Markov chain of a fixed policy.

It would certainly be preferable to compute the expectation exactly, however, this is generally not possible as this involves solving a high dimensional integral. Recall that we faced the same problem when studying inference in the first half of the manuscript. In both problems, we need to approximate high-dimensional integrals (i.e., expectations). This suggests a deep connection between

the problems of planning and inference. It is therefore not surprising that many techniques for approximate inference that we have seen earlier can also be applied to planning.

- *Monte Carlo approximation* which we have been focusing on during our discussion of planning is a very simple inference algorithm — approximating an expectation by sampling from the distribution that is averaged over. This allowed us to obtain unbiased gradient estimates (which may have high variance).
- An alternative approach is *moment matching* (cf. Section 5.4.6). Instead of approximating the expectation, here, we approximate the distribution over trajectories using a tractable distribution (e.g., a Gaussian) and "matching" their moments. This then allows us to analytically compute gradients of the expectation in Equation (13.12). A prominent example of this approach is *probabilistic inference for learning control* (PILCO).
- In Section 12.6, we have used *variational inference* for planning, and seen that it coincides with entropy regularization as implemented by the *soft actor critic* (SAC) algorithm.

## 13.2 Learning

Thus far, we have considered known environments. That is, we assumed that the transition model $f$ and the rewards $r$ are known. In reinforcement learning, $f$ and $r$ are (of course!) not known. Instead, we have to estimate them from data. This will also be crucial in our later discussion of exploration in Section 13.3 where we explore methods of driving data collection to learn what we need to learn about the world.

First, let us revisit one of our key observations when we first introduced the reinforcement learning problem. Namely, that the observed transitions $\boldsymbol{x}'$ and rewards $r$ are conditionally independent given the state-action pairs $(\boldsymbol{x}, \boldsymbol{a})$.[3] This is due to the Markovian structure of the underlying Markov decision process.

[3] see Equation (11.3)

This is the key observation that allows us to treat the estimation of the dynamics and rewards as a simple regression problem (or a density estimation problem when the quantities are stochastic rather than deterministic). More concretely, we can estimate the dynamics and rewards off-policy using the standard supervised learning techniques we discussed in earlier chapters, from a replay buffer

$$\mathcal{D} = \{(\underbrace{\boldsymbol{x}_t, \boldsymbol{a}_t}_{\text{"input"}}, \underbrace{r_t, \boldsymbol{x}_{t+1}}_{\text{"label"}})\}_t. \tag{13.13}$$

Here, $\boldsymbol{x}_t$ and $\boldsymbol{a}_t$ are the "inputs", and $r_t$ and $\boldsymbol{x}_{t+1}$ are the "labels" of the

regression problem. Due to the conditional independence of the labels given the inputs, we have independent label noise (i.e., "independent training data") which is the basic assumption that we have been making throughout our discussion of techniques for probabilistic machine learning in Part I.

The key difference to supervised learning is that the set of inputs depends on how we act. That is, the current inputs arise from previous policies, and the inputs which we will observe in the future will depend on the model (and policy) obtained from the current data: we have feedback loops! We will come back to this aspect of reinforcement learning in the next section on exploration. For now, recall we only assume that we have used an arbitrary policy to collect some data, which we then stored in a replay buffer, and which we now want to use to learn the "best-possible" model of our environment.

### *13.2.1 Probabilistic Inference*

In the following, we will discuss how we can use the techniques from probabilistic inference, which we have seen in Part I, to learn the dynamics and reward models. Thereby, we will focus on learning the transition model $f$ as learning the reward model $r$ is completely analogous. For learning deterministic dynamics or rewards, we can use for example Gaussian processes (cf. Chapter 4) or deep neural networks (cf. Chapter 7). We will now focus on the setting where the dynamics are stochastic, that is,

$$\boldsymbol{x}_{t+1} \sim f(\boldsymbol{x}_t, \boldsymbol{a}_t; \boldsymbol{\psi}). \tag{13.14}$$

**Example 13.7: Conditional Gaussian dynamics**

We could, for example, use a conditional Gaussian for the transition model,

$$\boldsymbol{x}_{t+1} \sim \mathcal{N}(\boldsymbol{\mu}(\boldsymbol{x}_t, \boldsymbol{a}_t; \boldsymbol{\psi}), \boldsymbol{\Sigma}(\boldsymbol{x}_t, \boldsymbol{a}_t; \boldsymbol{\psi})). \tag{13.15}$$

As we have seen in Equation (A.7), we can rewrite the covariance matrix $\boldsymbol{\Sigma}$ as a product of a lower-triangular matrix $\mathcal{L}$ and its transpose using the Cholesky decomposition $\boldsymbol{\Sigma} = \mathcal{L}\mathcal{L}^\top$ of $\boldsymbol{\Sigma}$. This allows us to represent the model by only $n(n+1)/2$ parameters. Moreover, we have learned that Gaussians are reparameterizable, which we have seen to be useful for planning.

Note that this model reduces to a deterministic model if the covariance is zero. So the stochastic transition models encompass all deterministic models. Moreover, in many applications (such as robotics), it is often useful to use stochastic models to attribute

slight inaccuracies and measurement noise to a small uncertainty in the model.

A first approach might be to obtain a point estimate for $f$, either through maximum likelihood estimation (which we have seen to overfit easily) or through maximum a posteriori estimation. If, for example, $f$ is represented as a deep neural network, we have already seen how to find the MAP estimate of its weights in Section 7.2.1.

**Remark 13.8: The key pitfall of point estimates**

However, using point estimates leads to a *key pitfall* of model-based reinforcement learning. Using a point estimate of the model for planning — even if this point estimate is very accurate — often performs *very poorly*. The reason is that planning is very good at overfitting (i.e., exploiting) small errors in the transition model. Moreover, the errors in the model estimate compound over time when using a longer time horizon $H$. The key to remedy this pitfall lies in being robust to misestimated models. This naturally suggests quantifying the uncertainty in our model estimate and taking it into account during planning.[4] In the following section, we will rediscover that estimates of epistemic uncertainty are also extremely useful for driving (safe) exploration — something that we have already encountered in our discussion of Bayesian optimization.

[4] Quantifying the uncertainty of an estimate is a problem that we have spent the first few chapters exploring. Notably, refer to

- Section 2.2 for a description of epistemic and aleatoric uncertainty;
- Chapter 4 for our use of uncertainty estimates in the context of Gaussian processes;
- Chapter 7 for our use of uncertainty estimates in the context of Bayesian deep learning; and
- Chapters 8 and 9 for our use of epistemic uncertainty estimates to drive exploration.

In the following, we will differentiate between the epistemic uncertainty and the aleatoric uncertainty. Recall from Section 2.2 that epistemic uncertainty corresponds to our uncertainty about the model, $p(f \mid \mathcal{D})$, while aleatoric uncertainty corresponds to the uncertainty of the transitions in the underlying Markov decision process (which can be thought of as "irreducible" noise), $p(\boldsymbol{x}_{t+1} \mid f, \boldsymbol{x}_t, \boldsymbol{a}_t)$.

Intuitively, probabilistic inference of dynamics models corresponds to learning a distribution over possible models $f$ and $r$ given prior *beliefs*, where $f$ and $r$ characterize the underlying Markov decision process. This goes to show another benefit of the model-based over the model-free approach to reinforcement learning. Namely, that it is much easier to encode prior knowledge about the transition and rewards model.

**Example 13.9: Inference with conditional Gaussian dynamics**

Let us revisit inference with our conditional Gaussian dynamics

model from Equation (13.15),

$$\boldsymbol{x}_{t+1} \sim \mathcal{N}(\boldsymbol{\mu}(\boldsymbol{x}_t, \boldsymbol{a}_t; \boldsymbol{\psi}), \boldsymbol{\Sigma}(\boldsymbol{x}_t, \boldsymbol{a}_t; \boldsymbol{\psi})).$$

Recall that in the setting of Bayesian deep learning, most approximate inference techniques represented the approximate posterior using some form of a mixture of Gaussians,[5]

$$p(\boldsymbol{x}_{t+1} \mid \mathcal{D}, \boldsymbol{x}_t, \boldsymbol{a}_t) \approx \frac{1}{m} \sum_{i=1}^{m} \mathcal{N}(\boldsymbol{\mu}(\boldsymbol{x}_t, \boldsymbol{a}_t; \boldsymbol{\psi}^{(i)}), \boldsymbol{\Sigma}(\boldsymbol{x}_t, \boldsymbol{a}_t; \boldsymbol{\psi}^{(i)})). \tag{13.16}$$

Hereby, the epistemic uncertainty is represented by the variance between mixture components, and the aleatoric uncertainty by the average variance within the components.[6]

[5] see
- Equation (7.18) for variational inference;
- Equation (7.21a) for Markov chain Monte Carlo;
- Equation (7.28) for dropout regularization; and
- Equation (7.29) for probabilistic ensembles.

[6] see Section 7.3.1

In supervised learning, we often conflated the notions of epistemic and aleatoric uncertainty. In the context of planning, there is an important consequence of the decomposition into epistemic and aleatoric uncertainty. Recall that the epistemic uncertainty corresponds to a distribution over Markov decision processes $f$, whereas the aleatoric uncertainty corresponds to the randomness in the transitions within one such MDP $f$. Crucially, this randomness in the transitions must be consistent within a single MDP! That is, once we selected a single MDP for planning, we should disregard the epistemic uncertainty and solely focus on the randomness of the transitions. Then, to take into account epistemic uncertainty, we should average our plan across the different realizations of $f$. This yields the following Monte Carlo estimate of our reward $\widehat{J}_H$,

$$\widehat{J}_H(\boldsymbol{a}_{t:t+H-1}) \approx \frac{1}{m} \sum_{i=1}^{m} \widehat{J}_H(\boldsymbol{a}_{t:t+H-1}; f^{(i)}) \qquad \text{where} \tag{13.17}$$

$$\widehat{J}_H(\boldsymbol{a}_{t:t+H-1}; f) \doteq \sum_{\tau=t}^{t+H-1} \gamma^{\tau-t} r(\boldsymbol{x}_\tau(\boldsymbol{\varepsilon}_{t:\tau-1}^{(i)}; \boldsymbol{a}_{t:\tau-1}, f), \boldsymbol{a}_\tau) + \gamma^H V(\boldsymbol{x}_{t+H}). \tag{13.18}$$

Here, $f^{(i)} \overset{\text{iid}}{\sim} p(f \mid \mathcal{D})$ are independent samples of the transition model, and $\boldsymbol{\varepsilon}_{t:t+H-1}^{(i)} \overset{\text{iid}}{\sim} \phi$ parameterizes the dynamics analogously to Equation (13.9):

$$\begin{aligned} &\boldsymbol{x}_\tau(\boldsymbol{\varepsilon}_{t:\tau-1}; \boldsymbol{a}_{t:\tau-1}, f) \\ &\doteq f(\boldsymbol{\varepsilon}_{\tau-1}; f(\dots; (\boldsymbol{\varepsilon}_{t+1}; f(\boldsymbol{\varepsilon}_t; \boldsymbol{x}_t, \boldsymbol{a}_t), \boldsymbol{a}_{t+1}), \dots), \boldsymbol{a}_{\tau-1}). \end{aligned} \tag{13.19}$$

Observe that the epistemic and aleatoric uncertainty are treated differently. Within a particular MDP $f$, we ensure that randomness (i.e., ale-

atoric uncertainty) is simulated consistently using our previous framework from our discussion of planning.[7] The Monte Carlo samples of $f$ take into account the epistemic uncertainty about the transition model. In our previous discussion of planning, we assumed the Markov decision process $f$ to be fixed. Essentially, in Equation (13.17) we are now using Monte Carlo trajectory sampling as a subroutine and average over an "ensemble" of Markov decision processes.

[7] see Equation (13.10)

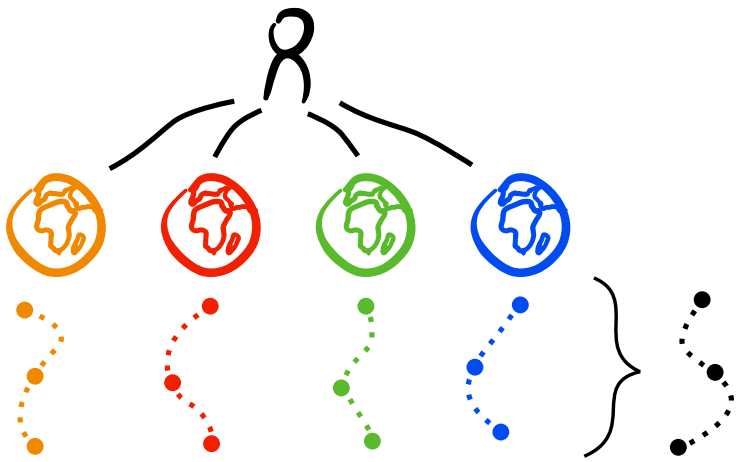


Figure 13.4: Illustration of planning with epistemic uncertainty and Monte Carlo sampling. The agent considers $m$ alternative "worlds". Within each world, it plans a sequence of actions over a finite time horizon. Then, the agent averages all optimal initial actions from all worlds. Crucially, each world by itself is *consistent*. That is, its transition model (i.e., the aleatoric uncertainty of the model) is constant.

The same approach that we have seen in Section 13.1.3 can be used to "compile" these plans into a parametric policy that can be trained offline, in which case, we write $\widehat{J}_H(\pi)$ instead of $\widehat{J}_H(\boldsymbol{a}_{t:t+H-1})$.

This leads us to a first greedily exploitative algorithm for model-based reinforcement learning, which is shown in Algorithm 13.10. This algorithm is purely exploitative as it greedily maximizes the expected reward with respect to the transition model, taking into account epistemic uncertainty.

**Algorithm 13.10:** Greedy exploitation for model-based RL

start with (possibly empty) data $\mathcal{D} = \emptyset$ and a prior $p(f) = p(f \mid \mathcal{D})$
**for** several episodes **do**
  plan a new policy $\pi$ to (approximately) maximize,

$$\max_{\pi} \mathbb{E}_{f \sim p(\cdot \mid \mathcal{D})} \left[ \widehat{J}_H(\pi; f) \right] \tag{13.20}$$

  roll out policy $\pi$ to collect more data
  update posterior $p(f \mid \mathcal{D})$

In the context of Gaussian process models, this algorithm is called

*probabilistic inference for learning control* (PILCO) (Deisenroth and Rasmussen, 2011), which was the first demonstration of how sample efficient model-based reinforcement learning can be. As was mentioned in Remark 13.6, the originally proposed PILCO uses moment matching instead of Monte Carlo averaging.

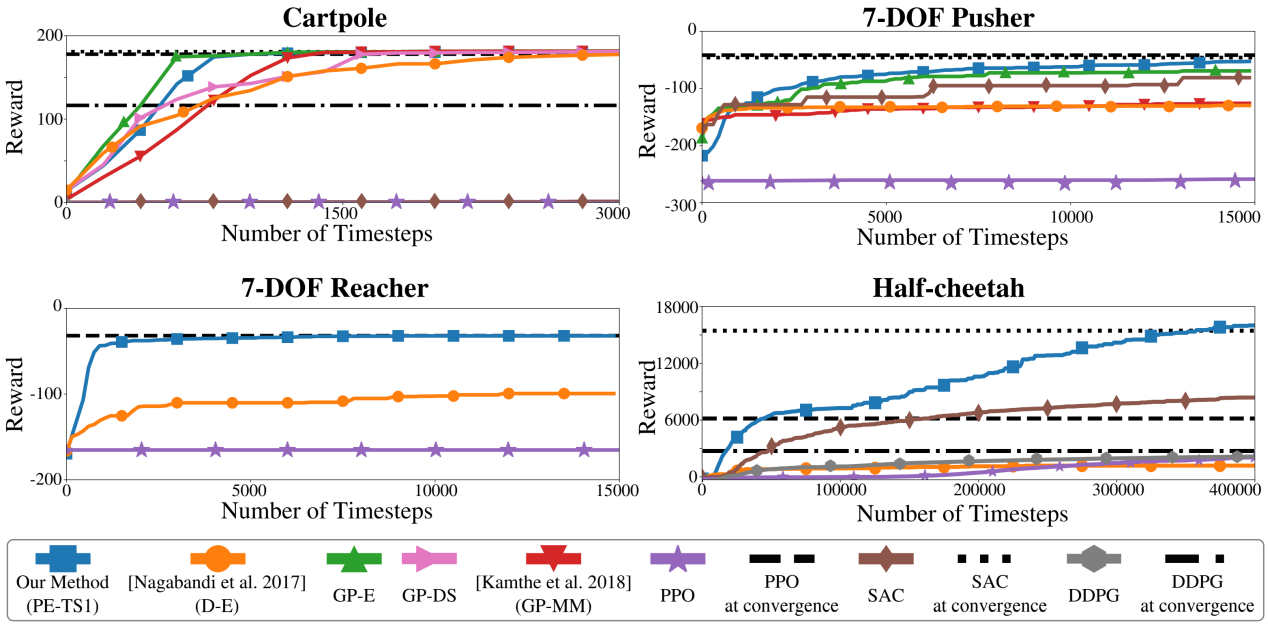



Figure 13.5: Sample efficiency of model-free and model-based reinforcement learning. DDPG and SAC are shown as constant (black) lines, because they take an order of magnitude more time steps before learning a good model. They also compare the PETS algorithm (blue) to deterministic ensembles (orange), where the transition models are assumed to be deterministic (or to have covariance 0). Reproduced with permission from "Deep reinforcement learning in a handful of trials using probabilistic dynamics models" (Chua et al., 2018).

In the context of neural networks, this algorithm is called *probabilistic ensembles with trajectory sampling* (PETS) (Chua et al., 2018), which is one of the state-of-the-art algorithms. PETS uses an ensemble of conditional Gaussian distributions over weights, trajectory sampling for evaluating the performance and model predictive control for planning.

Notably, PETS does not explicitly explore. Exploration only happens due to the uncertainty in the model, which already drives exploration to some extent. In many settings, however, this incentive is not sufficient for exploration — especially when rewards are sparse.

**Optional Readings**

- **PILCO:** Deisenroth and Rasmussen (2011).
  *PILCO: A model-based and data-efficient approach to policy search.*
- **PETS:** Chua, Calandra, McAllister, and Levine (2018).
  *Deep reinforcement learning in a handful of trials using probabilistic dynamics models.*

### 13.2.2 *Partial Observability*

Depending on the task, it may be difficult to learn the observed dynamics directly. For example, when learning an artificial player for a computer game based only on the games' visual interface it may be difficult to predict the next frame in pixel space. Instead, a common approach is to treat the visual interface as an observation in a POMDP with a hidden latent space (cf. Section 10.4), and to learn the dynamics within the latent space and the observation model separately.

An example of this approach is the *deep planning network* (PlaNet) algorithm which learns such a POMDP via variational inference and uses the cross-entropy method for closed-loop planning of action sequences (Hafner et al., 2019). The *Dreamer* algorithm replaces the closed-loop planning of PlaNet by open-loop planning with entropy regularization (Hafner et al., 2020, 2021). Notably, neither PlaNet nor Dreamer explicitly take into account epistemic model uncertainty during planning but rather use point estimates.

## 13.3 *Exploration*

Exploration is critical when interacting with unknown environments, such as in reinforcement learning. We first encountered the exploration-exploitation dilemma in our discussion of Bayesian optimization, where we aimed at maximizing an unknown function in as little time as possible by making noisy observations.[8] Within the framework of Bayesian optimization, we used so-called acquisition functions for selecting the next point at which to observe the unknown function. Observe that these acquisition functions are analogous to policies in the setting of reinforcement learning. In particular, the policy that uses greedy exploitation like we have seen in the previous section is analogous to simply picking the point that maximizes the mean of the posterior distribution. In the context of Bayesian optimization, we have already seen that this is insufficient for exploration and can easily get stuck in locally optimal solutions. As reinforcement learning is a strict generalization of Bayesian optimization, there is no reason why such a strategy should be sufficient now.

[8] see Chapter 9

Recall from our discussion of Bayesian optimization that we "solved" this problem by using the epistemic uncertainty in our model of the unknown function to pick the next point to explore. This is what we will now explore in the context of reinforcement learning.

One simple strategy that we already investigated is the addition of some *exploration noise*. In other words, one follows a greedy exploitative strategy, but every once in a while, one chooses a random action

(like in $\varepsilon$-greedy); or one adds additional noise to the selected actions (known as Gaussian noise "dithering"). We have already seen that in difficult exploration tasks, these strategies are not systematic enough.

Two other approaches that we have considered before are Thompson sampling (cf. Section 9.3.3) and, more generally, the principle of *optimism in the face of uncertainty* (cf. Section 9.2.1).

### 13.3.1 Thompson Sampling

Recall from Section 9.3.3 the main idea behind Thompson sampling: namely, that the randomness in the realizations of $f$ from the posterior distribution is already enough to drive exploration. That is, instead of picking the action that performs best on average across all realizations of $f$, Thompson sampling picks the action that performs best for a *single realization* of $f$. The epistemic uncertainty in the realizations of $f$ leads to variance in the picked actions and provides an additional incentive for exploration. This yields Algorithm 13.11 which is an immediate adaptation of greedy exploitation and straightforward to implement.

**Algorithm 13.11:** Thompson sampling

start with (possibly empty) data $\mathcal{D} = \emptyset$ and a prior $p(f) = p(f \mid \mathcal{D})$
**for** several episodes **do**
  sample a model $f \sim p(\cdot \mid \mathcal{D})$
  plan a new policy $\pi$ to (approximately) maximize,

$$\max_{\pi} \widehat{J}_H(\pi; f) \tag{13.21}$$

  roll out policy $\pi$ to collect more data
  update posterior $p(f \mid \mathcal{D})$

### 13.3.2 Optimistic Exploration

We have already seen in the context of Bayesian optimization and tabular reinforcement learning that optimism is a central pillar for exploration. But how can we explore optimistically in model-based reinforcement learning?

Let us consider a set $\mathcal{M}(\mathcal{D})$ of *plausible models* given some data $\mathcal{D}$. Optimistic exploration would then optimize for the most advantageous model among all models that are plausible given the seen data.

**Example 13.12: Plausible conditional Gaussians**

In the context of conditional Gaussians, we can consider the set of all models such that the prediction of a single dimension $i$ lies in some confidence interval,

$$f_i(\boldsymbol{x}, \boldsymbol{a}) \in \mathcal{C}_i \doteq [\mu_i(\boldsymbol{x}, \boldsymbol{a} \mid \mathcal{D}) - \beta_i \sigma_i(\boldsymbol{x}, \boldsymbol{a} \mid \mathcal{D}), \\ \mu_i(\boldsymbol{x}, \boldsymbol{a} \mid \mathcal{D}) + \beta_i \sigma_i(\boldsymbol{x}, \boldsymbol{a} \mid \mathcal{D})], \tag{13.22}$$

where $\beta_i$ tunes the width of the confidence interval, analogously to Section 9.3.1. The set of plausible models is then given as

$$\mathcal{M}(\mathcal{D}) \doteq \{f \mid \forall \boldsymbol{x} \in \mathcal{X}, \boldsymbol{a} \in \mathcal{A}, i \in [d] : f_i(\boldsymbol{x}, \boldsymbol{a}) \in \mathcal{C}_i\}. \tag{13.23}$$

When compared to greedy exploitation, instead of taking the optimal step on average with respect to all realizations of the transition model $f$, optimistic exploration as shown in Algorithm 13.13 takes the optimal step with respect to the most optimistic model among all transition models that are consistent with the data.

**Algorithm 13.13:** Optimistic exploration

start with (possibly empty) data $\mathcal{D} = \emptyset$ and a prior $p(f) = p(f \mid \mathcal{D})$
**for** several episodes **do**
  plan a new policy $\pi$ to (approximately) maximize,

$$\max_{\pi} \max_{f \in \mathcal{M}(\mathcal{D})} \widehat{J}_H(\pi; f) \tag{13.24}$$

  roll out policy $\pi$ to collect more data
  update posterior $p(f \mid \mathcal{D})$

In general, the joint maximization over $\pi$ and $f$ is very difficult to solve computationally. Yet, remarkably, it turns out that this complex optimization problem can be reduced to standard model-based reinforcement learning with a fixed model. The key idea is to consider an agent that can control its "luck". In other words, we assume that the agent believes it can control the outcome of its actions — or rather choose which of the plausible dynamics it follows. The "luck" of the agent can be considered as additional decision variables. Consider the optimization problem,

$$\pi_{t+1} \doteq \arg\max_{\pi} \max_{\boldsymbol{\eta}(\cdot) \in [-1,1]^d} \widehat{J}_H(\pi; \widehat{f}_t) \tag{13.25}$$

with "optimistic" dynamics

$$\widehat{f}_{t,i}(\boldsymbol{x}, \boldsymbol{a}) \doteq \mu_{t,i}(\boldsymbol{x}, \boldsymbol{a}) + \beta_{t,i} \eta_i(\boldsymbol{x}, \boldsymbol{a}) \sigma_{t,i}(\boldsymbol{x}, \boldsymbol{a}). \tag{13.26}$$

Here the decision variables $\eta_i$ control the variance of an action. That is, within the confidence bounds of the transition model, the agent can freely choose the state that is reached by playing an action $\boldsymbol{a}$ from state $\boldsymbol{x}$. Essentially, this corresponds to maximizing expected reward in an augmented (optimistic) MDP with *known* dynamics $\widehat{f}$ and with a larger action space that also includes the decision variables $\boldsymbol{\eta}$. This is a known MDP for which we can use our toolbox for planning which we developed in Section 13.1.

The algorithm that maximizes expected reward in this augmented MDP is called *hallucinated upper confidence reinforcement learning* (H-UCRL) (Curi et al., 2020; Treven et al., 2024). H-UCRL can be seen as the natural extension of the UCB acquisition function from Bayesian optimization to reinforcement learning. An illustration of the algorithm is given in Figure 13.6.

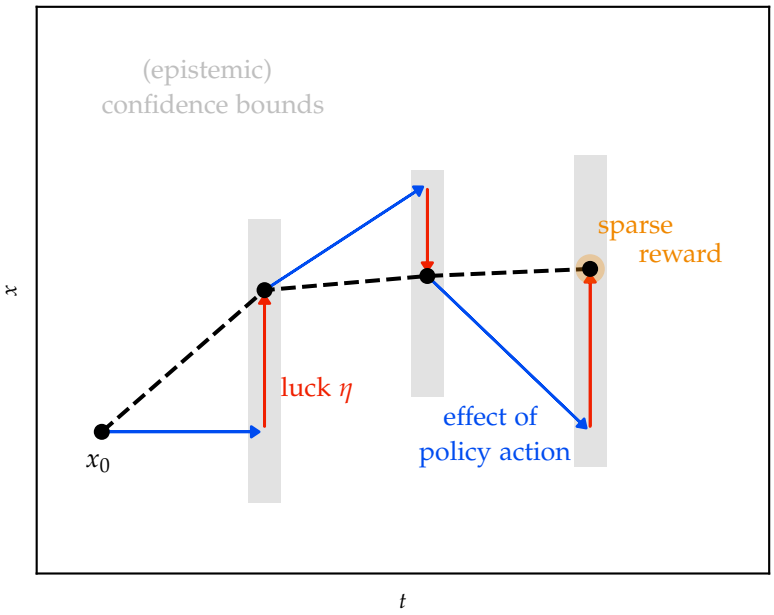



Figure 13.6: Illustration of H-UCRL in a one-dimensional state space. The agent "hallucinates" that it takes the black trajectory when, in reality, the outcomes of its actions are as shown in blue. The agent can hallucinate to land anywhere within the gray confidence regions (i.e., the epistemic uncertainty in the model) using the luck decision variables $\boldsymbol{\eta}$. This allows agents to discover long sequences of actions leading to sparse rewards.

Intuitively, the agent has the tendency to believe that it can achieve much more than it actually can. As more data is collected, the confidence bounds shrink and the optimistic policy rewards converge to the actual rewards. Yet, crucially, we only collect data in regions of the state-action space that are more promising than the regions that we have already explored. That is, we only collect data in the most promising regions.

*Scalable Optimistic Exploration.* While H-UCRL provides a principled solution to the optimistic exploration objective in Equation (13.24), it

does so at the cost of substantially enlarging the action space. In particular, the introduction of the auxiliary luck variables $\boldsymbol{\eta}$ adds $d_x$ additional control dimensions, which quickly becomes computationally prohibitive in high-dimensional state spaces such as those encountered in vision-based or robotic control tasks.

To overcome this limitation, we instead consider an approach that incorporates model uncertainty directly into the reward function via an exploration bonus. Concretely, we define the modified reward

$$\widehat{r}_t(\boldsymbol{x}, \boldsymbol{a}) \doteq r(\boldsymbol{x}, \boldsymbol{a}) + \sum_{i=1}^{d} \beta_{t,i} \sigma_{t,i}(\boldsymbol{x}, \boldsymbol{a}), \tag{13.27}$$

where the bonus term incentivizes the agent to visit state–action pairs with high epistemic uncertainty in the dynamics model.

By embedding optimism directly into the reward, we can leverage standard model-based reinforcement learning techniques without modifying the action space. At each episode, the policy is obtained by solving

$$\pi_{t+1} \doteq \arg\max_{\pi} \widehat{J}_{H,t}(\pi; \boldsymbol{\mu}_t), \tag{13.28}$$

where $\widehat{J}_{H,t}(\pi; \boldsymbol{\mu}_t)$ is defined as in Equation (13.17), but evaluated using the modified reward function $\widehat{r}_t$ and the mean dynamics model $\boldsymbol{\mu}_t$.

The resulting algorithm, called *scalable and optimistic model-based reinforcement learning* (SOMBRL, Sukhija et al., 2025), remains optimistic while scaling gracefully to high-dimensional state spaces.

Notably, the SOMBRL framework naturally extends to purely exploratory settings without extrinsic rewards, i.e., when $r(\boldsymbol{x}, \boldsymbol{a}) = 0$ for all $(\boldsymbol{x}, \boldsymbol{a})$ (Sukhija et al., 2023a; Zheng et al., 2025). In this regime, exploration is driven entirely by the uncertainty bonus in Equation (13.27), yielding an intrinsically motivated agent that actively seeks out informative regions of the environment.

Optimistic exploration yields the strongest effects for hard exploration tasks, for example, settings with large penalties associated with performing certain actions and settings with sparse rewards.[9] In those settings, most other strategies (i.e., those that are not sufficiently explorative), learn not to act at all. However, even in settings of "ordinary rewards", optimistic exploration often learns good policies faster.

[9] Action penalties are often used to discourage the agent from exhibiting unwanted behavior. However, increasing the action penalty increases the difficulty of exploration. Therefore, optimistic exploration is especially useful in settings where we want to practically disallow many actions by attributing large penalties to them.

### 13.3.3 *Constrained Exploration*

Besides making exploration more efficient, another use of uncertainty is to make exploration more safe. Today, reinforcement learning is still

far away from being deployed directly to the real world. In practice, reinforcement learning is almost always used together with a simulator, in which the agent can "safely" train and explore. Yet, in many domains, it is not possible to simulate the training process, either because we are lacking a perfect model of the environment, or because simulating such a model is too computationally inefficient. This is where we can make use of uncertainty estimates of our model to avoid unsafe states.

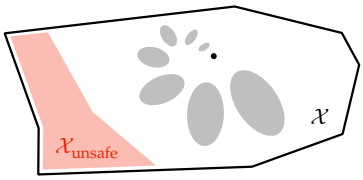



Figure 13.7: Illustration of planning with confidence bounds. The unsafe set of states is shown as the red region. The agent starts at the position denoted by the black dot and plans a sequence of actions. The confidence bounds on the subsets of states that are reached by this action sequence are shown in gray.

Let us denote by $\mathcal{X}_{\text{unsafe}}$ the subset of unsafe states of our state space $\mathcal{X}$. A natural idea is to perform planning using confidence bounds of our epistemic uncertainty. This allows us to pessimistically forecast the plausible consequences of our actions, given what we have already learned about the transition model. As we collect more data, the confidence bounds will shrink, requiring us to be less conservative over time. This idea is also at the heart of fields like *robust control*.

A general formalism for planning under constraints is the notion of *constrained Markov decision processes* (Altman, 1999). Given dynamics $f$, planning in constrained MDPs amounts to the following optimization problem:

$$\max_{\pi} \quad J_\mu(\pi; f) \doteq \mathbb{E}_{\boldsymbol{x}_0 \sim \mu, \boldsymbol{x}_{1:\infty} | \pi, f} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \right] \tag{13.29a}$$

$$\text{subject to} \quad J_\mu^c(\pi; f) \leq \delta \tag{13.29b}$$

where $\mu$ is a distribution over the initial state and

$$J_\mu^c(\pi; f) \doteq \mathbb{E}_{\boldsymbol{x}_0 \sim \mu, \boldsymbol{x}_{1:\infty} | \pi, f} \left[ \sum_{t=0}^{\infty} \gamma^t c(\boldsymbol{x}_t) \right] \tag{13.30}$$

are expected discounted costs with respect to a cost function $c : \mathcal{X} \to \mathbb{R}_{\geq 0}$.[10] Observe that for the cost function $c(\boldsymbol{x}) \doteq \mathbb{1}\{\boldsymbol{x} \in \mathcal{X}_{\text{unsafe}}\}$, the value $J_\mu^c(\pi; f)$ can be interpreted as an upper bound on the (discounted) probability of visiting unsafe states under dynamics $f$,[11] and hence, the constraint $J_\mu^c(\pi; f) \leq \delta$ bounds the probability of visiting an unsafe state when following policy $\pi$.

[10] It is straightforward to extend this framework to allow for multiple constraints.

[11] This follows from a simple union bound (1.73).

The *augmented Lagrangian method* can be used to solve the optimization problem of Equation (13.29b).[12] Thereby, one solves

$$\max_{\pi} \min_{\lambda \geq 0} \quad J_\mu(\pi; f) - \lambda (J_\mu^c(\pi; f) - \delta) \tag{13.31}$$

$$= \max_{\pi} \begin{cases} J_\mu(\pi; f) & \text{if } \pi \text{ is feasible} \\ -\infty & \text{otherwise.} \end{cases} \tag{13.32}$$

[12] For an introduction, read chapter 17 of "Numerical optimization" (Wright et al., 1999).

Observe that if $\pi$ is feasible, then $J_\mu^c(\pi; f) \leq \delta$ and so the minimum over $\lambda$ is satisfied if $\lambda = 0$. Conversely, if $\pi$ is infeasible, $\lambda$ can be

made arbitrarily large to solve the optimization problem. In practice, an additional penalty term is added to smooth the objective when transitioning between feasible and infeasible policies.

Note that solving constrained optimization problems such as Equation (13.29b) yields an optimal safe policy. However, it is not ensured that constraints are not violated during the search for the optimal policy. Generally, exterior penalty methods such as the augmented Lagrangian method allow for generating infeasible points during the search, and are therefore not suitable when constraints have to be strictly enforced at all times. Thus, this method is more applicable in the episodic setting (e.g., when an agent is first "trained" in a simulated environment and then "deployed" to the actual task) rather than in the continuous setting where the agent has to operate in the "real world" from the beginning and cannot easily be reset.

**Remark 13.14: Barrier functions**

The augmented Lagrangian method is merely one example of optimizing a joint objective of maximizing rewards and minimizing costs to find a policy with bounded costs. Another example of this approach are so-called *Barrier functions* which augment the reward objective with a smoothed approximation of the boundary of a set $\mathcal{X}_{\text{unsafe}}$. That is, one solves

$$\max_{\pi} J_\mu(\pi; f) - \lambda \cdot B^c_\mu(\pi; f) \tag{13.33}$$

for some $\lambda > 0$ where the barrier function $B^c_\mu(\pi; f)$ goes to infinity as a state on the boundary of $\mathcal{X}_{\text{unsafe}}$ is approached. Examples for barrier functions are $-\log c(\boldsymbol{x})$ and $\frac{1}{c(\boldsymbol{x})}$.

Barrier functions are an interior penalty method which ensure that points are feasible during the search for the optimal solution.

So far, we have assumed knowledge of the true dynamics to solve the optimization problem of Equation (13.29b). If we do not know the true dynamics but instead have access to a set of plausible models $\mathcal{M}(\mathcal{D})$ given data $\mathcal{D}$ (cf. Section 13.3.2) which encompasses the true dynamics, then a natural strategy is to be optimistic with respect to future rewards and pessimistic with respect to future constraint violations. More specifically, we solve the optimization problem,

$$\max_{\pi} \max_{f \in \mathcal{M}(\mathcal{D})} J_\mu(\pi; f) \tag{13.34a}$$

*optimistic*

$$\text{subject to} \quad \max_{f \in \mathcal{M}(\mathcal{D})} J^c_\mu(\pi; f) \leq \delta. \tag{13.34b}$$

*pessimistic*

Intuitively, jointly maximizing $J_\mu(\pi; f)$ with respect to $\pi$ and (plausi-

ble) $f$ can lead the agent to try behaviors with potentially high reward due to *optimism* (i.e., the agent "dreams" about potential outcomes). On the other hand, being *pessimistic* with respect to constraint violations enforces the safety constraints (i.e., the agent has "nightmares" about potential outcomes).

If the policy values $J_\mu(\pi; f)$ and $J_\mu^c(\pi; f)$ are modeled as a distribution (e.g., using Bayesian deep learning), then the inner maximization over plausible dynamics can be approximated using samples from the posterior distributions. Thus, the augmented Lagrangian method can also be used to solve the general optimization problem of Equation (13.34b). The resulting algorithm is known as *Lagrangian model-based agent* (LAMBDA) (As et al., 2022).

### *13.3.4 Safe Exploration*

As noted in the previous section, in many settings we do not only want to eventually find a safe policy, but we also want to ensure that we act safely while searching for an optimal policy. To this end, recall the general approach outlined in the beginning of the previous section wherein we plan using (pessimistic) confidence bounds of the plausible consequences of our actions.

One key challenge of this approach is that we need to forecast plausible trajectories. The confidence bounds of such trajectories cannot be nicely represented anymore. One approach is to over-approximate the confidence bounds over reachable states from trajectories.

**Theorem 13.15** (informal, Koller et al. (2018))**.** *For conditional Gaussian dynamics, the reachable states of trajectories can be over-approximated with high probability.*

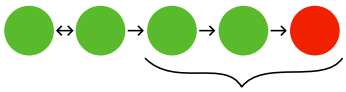
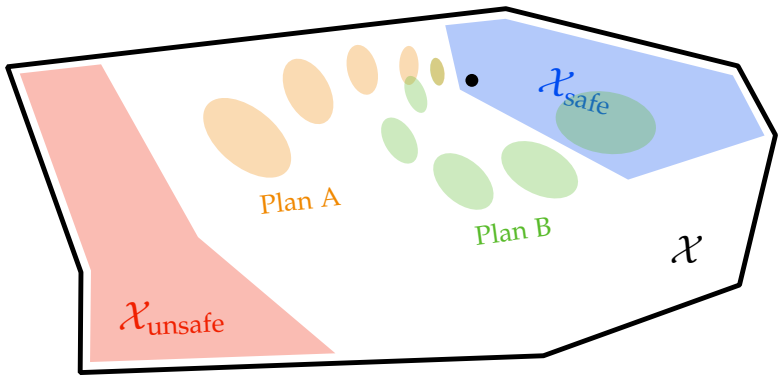



Figure 13.8: Illustration of long-term consequence when planning a finite number of steps. Green dots are to denote safe states and the red dot is to denote an unsafe state. After performing the first action, the agent is still able to return to the previous state. Yet, after reaching the third state, the agent is already guaranteed to end in an unsafe state. When using only a finite horizon of $H = 2$ for planning, the agent might make this transition regardless.

Another key challenge is that actions might have consequences that exceed the time horizon used for planning. In other words, by performing an action now, our agent might put itself into a state that is not yet unsafe, but out of which it cannot escape and which will eventually lead to an unsafe state. You may think of a car driving towards a wall. When a crash with the wall is designated as an unsafe state, there are quite a few states in advance at which it is already impossible to avoid the crash. Thus, looking ahead a finite number of steps is not sufficient to prevent entering unsafe states.

It turns out that it is still possible to use the epistemic uncertainty about the model and short-term plausible behaviors to make guarantees about certain long-term consequences. One idea is to also consider a set of safe states $\mathcal{X}_{\text{safe}}$ alongside the set of unsafe states $\mathcal{X}_{\text{unsafe}}$, for which our agent knows how to stay inside (i.e., remain safe). In other

words, for states $x \in \mathcal{X}_{\text{safe}}$, we know that our agent can always behave in such a way that it will not reach an unsafe state.[13] An illustration of this approach is shown in Figure 13.9.




[13] In the example of driving a car, the set of safe states corresponds to those states where we know that we can still safely brake before hitting the wall.

Figure 13.9: Illustration of long-term consequence when planning a finite number of steps. The unsafe set of states is shown in red and the safe set of states is shown in blue. The confidence intervals corresponding to actions of the performance plan and safety plan are shown in orange and green, respectively.

The problem is that this set of safe states might be very conservative. That is to say, it is likely that rewards are mostly attained *outside* of the set of safe states. The key idea is to plan two sequences of actions, instead of only one. "Plan A" (the *performance plan*) is planned with the objective to solve the task, that is, attain maximum reward. "Plan B" (the *safety plan*) is planned with the objective to return to the set of safe states $\mathcal{X}_{\text{safe}}$. In addition, we enforce that both plans must agree on the first action to be played.

Under the assumption that the confidence bounds are conservative estimates, this guarantees that after playing this action, our agent will still be in a state of which it can return to the set of safe states. In this way, we can gradually increase the set of states that are safe to explore. It can be shown that under suitable conditions, returning to the safe set can be guaranteed (Koller et al., 2018).

### 13.3.5 *Safety Filters*

An alternative approach is to (slightly) adjust a potentially unsafe policy $\pi$ to obtain a policy $\hat{\pi}$ which avoids entering unsafe states with high probability.

Following our interpretation of constrained policy optimization in terms of optimism and pessimism from Section 13.3.3, to pessimistically evaluate the safety of a policy with respect to a cost function $c$ given a set of plausible models $\mathcal{M}(\mathcal{D})$, we can use

$$C^{\pi}(x) \doteq \max_{f \in \mathcal{M}(\mathcal{D})} J^{c}_{\delta_x}(\pi; f) \tag{13.35}$$

$\delta_x$ is the point density at $x$

where the initial-state distribution $\delta_{\boldsymbol{x}}$ is to denote that the initial state is $\boldsymbol{x}$. Observe that Equation (13.35) permits a reparameterization in terms of additional decision variables $\boldsymbol{\eta}$ which is analogous to our discussion in Section 13.3.2. Concretely, we have

$$C^\pi(\boldsymbol{x}) = \max_{\boldsymbol{\eta}} J^c_{\delta_{\boldsymbol{x}}}(\pi; \widehat{f}) \tag{13.36}$$

where $\widehat{f}$ are the adjusted dynamics (13.26) which are based on the "luck" variables $\boldsymbol{\eta}$. In the context of estimating costs which we aim to minimize (as opposed to rewards which we aim to maximize), $\boldsymbol{\eta}$ can be interpreted as the "bad luck" of the agent.

When our only objective is to act safely, that is, we only aim to minimize cost and are indifferent to rewards, then this reparameterization allows us to find a "maximally safe" policy,

$$\pi_{\text{safe}} \doteq \arg\min_\pi \mathbb{E}_{\boldsymbol{x}\sim\mu}[C^\pi(\boldsymbol{x})] = \arg\min_\pi \max_{\boldsymbol{\eta}} J^c_\mu(\pi; \widehat{f}). \tag{13.37}$$

Under some conditions $\pi_{\text{safe}}$ can be shown to satisfy $\pi_{\text{safe}}(\boldsymbol{x}) = \pi_{\boldsymbol{x}}(\boldsymbol{x})$ for any $\boldsymbol{x}$ where $\pi_{\boldsymbol{x}} \doteq \arg\min_\pi C^\pi(\boldsymbol{x})$ (Curi et al., 2022, proposition 4.2). On its own, the policy $\pi_{\text{safe}}$ is rather useless for exploring the state space. In particular, when in a state that we already deem safe, following policy $\pi_{\text{safe}}$, the agent aims simply to "stay" within the set of safe states which means that it has no incentive to explore / maximize reward.

Instead, one can interpret $\pi_{\text{safe}}$ as a "backup" policy in case we realize upon exploring that a certain trajectory is too dangerous, akin to our notion of the "safety plan" in Section 13.3.4. That is, given any (potentially explorative and dangerous) policy $\pi$, we can find the adjusted policy,

$$\hat{\pi}(\boldsymbol{x}) \doteq \arg\min_{\boldsymbol{a}\in\mathcal{A}} \quad d(\pi(\boldsymbol{x}), \boldsymbol{a}) \tag{13.38a}$$

$$\text{subject to} \quad \max_{\boldsymbol{\eta}} C^{\pi_{\text{safe}}}(\widehat{f}(\boldsymbol{x}, \boldsymbol{a})) \leq \delta \tag{13.38b}$$

for some metric $d(\cdot, \cdot)$ on $\mathcal{A}$. The constraint ensures that the pessimistic next state $\widehat{f}(\boldsymbol{x}, \boldsymbol{a})$ is recoverable by following policy $\pi_{\text{safe}}$. In this way,

$$\tilde{\pi}(\boldsymbol{x}) \doteq \begin{cases} \hat{\pi}(\boldsymbol{x}) & \text{if } C^{\pi_{\text{safe}}}(\boldsymbol{x}) \leq \delta \\ \pi_{\text{safe}}(\boldsymbol{x}) & \text{if } C^{\pi_{\text{safe}}}(\boldsymbol{x}) > \delta \end{cases} \tag{13.39}$$

is the policy "closest" to $\pi$ which is $\delta$-safe with respect to the pessimistic cost estimates $C^{\pi_{\text{safe}}}$ (Curi et al., 2022).[14] This is also called a *safety filter*. Similar approaches using backup policies also allow safe exploration in the model-free setting (Sukhija et al., 2023b; Widmer et al., 2023).

[14] The policy $\tilde{\pi}$ is required as, a priori, it is not guaranteed that the state $\boldsymbol{x}_t$ will satisfy $C^{\pi_{\text{safe}}}(\boldsymbol{x}_t) \leq \delta$ for all $t$ unless $\hat{\pi}$ is replanned after every step.

**Optional Readings**

- Curi, Berkenkamp, and Krause (2020).
  *Efficient model-based reinforcement learning through optimistic policy search and planning*.
- Berkenkamp, Turchetta, Schoellig, and Krause (2017).
  *Safe model-based reinforcement learning with stability guarantees*.
- Koller, Berkenkamp, Turchetta, and Krause (2018).
  *Learning-based model predictive control for safe exploration*.
- As, Usmanova, Curi, and Krause (2022).
  *Constrained Policy Optimization via Bayesian World Models*.
- Curi, Lederer, Hirche, and Krause (2022).
  *Safe Reinforcement Learning via Confidence-Based Filters*.
- Turchetta, Berkenkamp, and Krause (2019).
  *Safe exploration for interactive machine learning*.

## *Discussion*

In this final chapter, we learned that leveraging a learned "world model" for planning can be dramatically more sample-efficient than model-free reinforcement learning. Additionally, such world models allow for effective multitask learning since they can be reused across tasks and only the reward function needs to be swapped out.[15] Finally, we explored how to use probabilistic inference of our world model to drive exploration more effectively and safely.

[15] A similar idea, though often classified as model-free, is to learn a goal-conditioned policy which is studied extensively in *goal-conditioned reinforcement learning* (Andrychowicz et al., 2017; Plappert et al., 2018; Park et al., 2024).

## *Problems*

**13.1. Bounding the regret of H-UCRL.**

Analogously to our analysis of the UCB acquisition function for Bayesian optimization, we can use optimism to bound the regret of H-UCRL. We assume for simplicity that $d = 1$ and drop the index $i$ in the following. Let us denote by $x_{t,k}$ the $k$-th state visited during episode $t$ when following policy $\pi_t$. We denote by $\widehat{x}_{t,k}$ the corresponding state but under the optimistic dynamics $\widehat{f}$ rather than the true dynamics $f$. The instantaneous regret during episode $t$ is given by

$$r_t = J_H(\pi^\star; f) - J_H(\pi_t; f) \tag{13.40}$$

where we take the objective to be $J_H(\pi; f) = \sum_{k=0}^{H-1} r(x_{t,k}, \pi(x_{t,k}))$.

You may assume that $f(x, \pi(x))$, $r(x, \pi(x))$, and $\sigma(x, \pi(x))$ are Lipschitz continuous in $x$.

1. Show by induction that for any $k \geq 0$, with high probability,

$$\left\|\widehat{x}_{t,k} - x_{t,k}\right\| \leq 2\beta_t \sum_{l=0}^{k-1} \alpha_t^{k-1-l} \sigma_{t-1}(x_{t,l}, \pi_t(x_{t,l})) \tag{13.41}$$

   where $\alpha_t$ depends on the Lipschitz constants and $\beta_t$.
2. Assuming w.l.o.g. that $\alpha_t \geq 1$, show that with high probability,

$$r_t \leq O\left(\beta_t H \alpha_t^{H-1} \sum_{k=0}^{H-1} \sigma_{t-1}(x_{t,k}, \pi_t(x_{t,k}))\right) \tag{13.42}$$

3. Let $\Gamma_T \doteq \max_{\pi_1,\dots\pi_T} \sum_{t=1}^{T} \sum_{k=0}^{H-1} \sigma_{t-1}^2(x_{t,k}, \pi_t(x_{t,k}))$. Analogously to Problem 9.3, it can be derived that $\Gamma_T \leq O(H\gamma_T)$ if the dynamics are modeled by a Gaussian process.[16] Bound the cumulative regret $R_T = \sum_{t=1}^{T} r_t \leq O\left(\beta_T H^{\frac{3}{2}} \alpha_T^{H-1} \sqrt{T\Gamma_T}\right)$.

Thus, if the dynamics are modeled by a Gaussian process with kernel such that $\gamma_T$ is sublinear, the regret of H-UCRL is sublinear.

[16] For details, see appendix A of Treven et al. (2024).

# A

# Mathematical Background

## A.1 Probability

### A.1.1 Common discrete distributions

- *Bernoulli* $\text{Bern}(p)$ describes (biased) coin flips. Its domain is $\Omega = \{0, 1\}$ where 1 corresponds to heads and 0 corresponds to tails. $p \in [0, 1]$ is the probability of the coin landing heads, that is, $\mathbb{P}(X = 1) = p$ and $\mathbb{P}(X = 0) = 1 - p$.
- *Binomial* $\text{Bin}(n, p)$ counts the number of heads in $n$ independent Bernoulli trials, each with probability of heads $p$.
- *Categorical* $\text{Cat}(m, p_1, \ldots, p_m)$ is a generalization of the Bernoulli distribution and represents throwing a (biased) $m$-sided die. Its domain is $\Omega = [m]$ and we have $\mathbb{P}(X = i) = p_i$. We require $p_i \geq 0$ and $\sum_{i \in [m]} p_i = 1$.
- *Multinomial* $\text{Mult}(n, m, p_1, \ldots, p_m)$ counts the number of outcomes of each side in $n$ independent Categorical trials.
- *Uniform* $\text{Unif}(S)$ assigns identical probability mass to all values in the set $S$. That is, $\mathbb{P}(X = x) = \frac{1}{|S|}$ $(\forall x \in S)$.

### A.1.2 Probability simplex

We denote by $\Delta^m$ the set of all categorical distributions on $m$ classes. Observe that $\Delta^m$ is an $(m-1)$-dimensional convex polytope.

To see this, let us consider the $m$-dimensional space of probabilities $[0, 1]^m$. It follows from our characterization of the categorical distribution in Section A.1.1 that there is a one-to-one correspondence between probability distributions over $m$ classes and points in the space $[0, 1]^m$ where all coordinates sum to one. This $(m-1)$-dimensional subspace of $[0, 1]^m$ is also known as the *probability simplex*.

### A.1.3 Universality of the Uniform and Sampling

Sampling from a continuous random variable is a common and non-trivial problem. A family of techniques is motivated by the following general property of the uniform distribution, colloquially known as the "universality of the uniform".

**Uniform distribution** The (continuous) *uniform distribution* $\text{Unif}([a,b])$ is the only distribution that assigns constant density to all points in the support $[a,b]$. That is, it has PDF

$$p(u) = \begin{cases} \frac{1}{b-a} & \text{if } u \in [a,b] \\ 0 & \text{otherwise} \end{cases}$$

and CDF

$$P(u) = \begin{cases} \frac{u-a}{b-a} & \text{if } u \in [a,b] \\ 0 & \text{otherwise.} \end{cases}$$

First, given a random variable $X \sim P$, we call

$$P^{-1}(u) \doteq \min\{x \mid P(x) \geq u\} \quad \text{for all } 0 < u < 1. \tag{A.1}$$

the *quantile function* of $X$. That is, $P^{-1}(u)$ corresponds to the value $x$ such that the probability of $X$ being at most $x$ is $u$. If the CDF $P$ is invertible, then $P^{-1}$ coincides with the inverse of $P$.

**Theorem A.1** (Universality of the uniform)**.** *If $U \sim \text{Unif}([0,1])$ and $P$ is invertible, then $P^{-1}(U) \sim P$.*

*Proof.* Let $U \sim \text{Unif}([0,1])$. Then,

$$\mathbb{P}\Big(P^{-1}(U) \leq x\Big) = \mathbb{P}(U \leq P(x)) = P(x) \qquad \text{using } \mathbb{P}(U \leq u) = u \text{ if } U \sim \text{Unif}([0,1])$$

This implies that if we are able to sample from $\text{Unif}([0,1])$,[1] then we are able to sample from any distribution with invertible CDF. This method is known as *inverse transform sampling*.

[1] This is done in practice using so-called *pseudo-random number generators*.

In the case of Gaussians, we learned that the CDF cannot be expressed in closed-form (and hence, is not invertible), however, for practical purposes, the quantile function of Gaussians can be approximated well.

### A.1.4 Point densities and Dirac's delta function

The *Dirac delta function* $\delta_\alpha$ is a function satisfying $\delta_\alpha(x) = 0$ for any $x \neq \alpha$ and $\int \delta_\alpha(x)\,dx = 1$. $\delta_\alpha$ is also called a *point density* at $\alpha$.

As an example, let us consider the random variable $Y = g(X)$, which is defined in terms of another random variable $X$ and a continuously differentiable function $g : \mathbb{R} \to \mathbb{R}$. Using the sum rule and product rule, we can express the PDF of $Y$ as

$$p(y) = \int p(y \mid x) \cdot p(x)\,dx = \int \delta_{g(x)}(y) \cdot p(x)\,dx. \tag{A.2}$$

In Section 1.1.11, we discuss how one can obtain an explicit representation of $p_Y$ using the "change of variables" formula.

### A.1.5 Gradients of expectations

We often encounter gradients of expectations, $\nabla_{\boldsymbol{\theta}}\, \mathbb{E}_{\mathbf{X}}[f(\mathbf{X}, \boldsymbol{\theta})]$.

**Gradient** The *gradient* of a function $f : \mathbb{R}^n \to \mathbb{R}$ at a point $\boldsymbol{x} \in \mathbb{R}^n$ is

$$\nabla f(\boldsymbol{x}) \doteq \begin{bmatrix} \frac{\partial f(\boldsymbol{x})}{\partial \boldsymbol{x}(1)} & \cdots & \frac{\partial f(\boldsymbol{x})}{\partial \boldsymbol{x}(n)} \end{bmatrix}^\top. \tag{A.3}$$

**Fact A.2** (Differentiation under the integral sign)**.** *Let $f(\boldsymbol{x}, t)$ be differentiable in $\boldsymbol{x}$, integrable in $t$, and be such that*

$$\left|\frac{\partial f(\boldsymbol{x}, t)}{\partial t}\right| \leq g(\boldsymbol{x})$$

*for some integrable function $g$. Then, if the distribution of $\mathbf{X}$ is not parameterized by $t$,*[2]

$$\frac{\partial \mathbb{E}[f(\mathbf{X}, t)]}{\partial t} = \mathbb{E}\left[\frac{\partial f(\mathbf{X}, t)}{\partial t}\right]. \tag{A.4}$$

[2] That is, $\mathbb{E}[f(\mathbf{X}, t)] = \int p(\boldsymbol{x}) \cdot f(\boldsymbol{x}, t)\, d\boldsymbol{x}$ and $p(\boldsymbol{x})$ does not depend on $t$.

Therefore, if the distribution of $\mathbf{X}$ is not parameterized by $\boldsymbol{\theta}$,

$$\boldsymbol{\nabla}_{\boldsymbol{\theta}} \mathbb{E}[f(\mathbf{X}, \boldsymbol{\theta})] = \mathbb{E}[\boldsymbol{\nabla}_{\boldsymbol{\theta}} f(\mathbf{X}, \boldsymbol{\theta})]. \tag{A.5}$$

## A.2 Quadratic Forms and Gaussians

**Definition A.3** (Quadratic form)**.** Given a symmetric matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, the *quadratic form* induced by $\boldsymbol{A}$ is defined as

$$f_{\boldsymbol{A}} : \mathbb{R}^n \to \mathbb{R}, \quad \boldsymbol{x} \mapsto \boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x} = \sum_{i=1}^{n} \sum_{j=1}^{n} \boldsymbol{A}(i, j) \cdot \boldsymbol{x}(i) \cdot \boldsymbol{x}(j). \tag{A.6}$$

We call $\boldsymbol{A}$ a *positive definite* matrix if all eigenvalues of $\boldsymbol{A}$ are positive. Equivalently, we have $f_{\boldsymbol{A}}(\boldsymbol{x}) > 0$ for all $\boldsymbol{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ and $f_{\boldsymbol{A}}(\mathbf{0}) = 0$. Similarly, $\boldsymbol{A}$ is called *positive semi-definite* if all eigenvalues of $\boldsymbol{A}$ are non-negative, or equivalently, if $f_{\boldsymbol{A}}(\boldsymbol{x}) \geq 0$ for all $\boldsymbol{x} \in \mathbb{R}^n$. In particular, if $\boldsymbol{A}$ is positive definite then $\sqrt{f_{\boldsymbol{A}}(\boldsymbol{x})}$ is a norm (called the *Mahalanobis norm* induced by $\boldsymbol{A}$), and is often denoted by $\|\boldsymbol{x}\|_{\boldsymbol{A}}$.

If $\boldsymbol{A}$ is positive definite, then the sublevel sets of its induced quadratic form $f_{\boldsymbol{A}}$ are convex ellipsoids. Not coincidentally, the same is true for the sublevel sets of the PDF of a normal distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, which we have seen an example of in Figure 1.6. Hereby, the axes of the ellipsoid and their corresponding squared lengths are the eigenvectors and eigenvalues of $\boldsymbol{\Sigma}$, respectively.

**Remark A.4: Correspondence of quadratic forms and Gaussians**

Observe that the PDF of a zero-mean multivariate Gaussian $\mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$ is an exponentiated and appropriately scaled quadratic form induced by the positive definite precision matrix $\boldsymbol{\Sigma}^{-1}$. The constant factor is chosen such that the resulting function is a valid probability density function, that is, sums to one.

One important property of positive definiteness of $\boldsymbol{\Sigma}$ is that $\boldsymbol{\Sigma}$ can be decomposed into the product of a lower-triangular matrix with its transpose. This is known as the *Cholesky decomposition*.

**Fact A.5** (Cholesky decomposition, symmetric matrix-form)**.** *For any symmetric and positive (semi-)definite matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, there is a decomposition of the form*

$$\boldsymbol{A} = \mathcal{L}\mathcal{L}^\top \tag{A.7}$$

*where $\mathcal{L} \in \mathbb{R}^{n \times n}$ is lower triangular and positive (semi-)definite.*

We will not prove this fact, but it is not hard to see that a decomposition exists (it takes more work to show that $\mathcal{L}$ is lower-triangular).

Let $\boldsymbol{A}$ be a symmetric and positive (semi-)definite matrix. By the spectral theorem of symmetric matrices, $\boldsymbol{A} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^\top$, where $\boldsymbol{\Lambda}$ is a diagonal matrix of eigenvalues and $\boldsymbol{V}$ is an orthonormal matrix of corresponding eigenvectors. Consider the matrix

$$\boldsymbol{A}^{1/2} \doteq \boldsymbol{V}\boldsymbol{\Lambda}^{1/2}\boldsymbol{V}^\top \tag{A.8}$$

where $\boldsymbol{\Lambda}^{1/2} \doteq \operatorname{diag}\{\sqrt{\boldsymbol{\Lambda}(i,i)}\}$, also called the *square root* of $\boldsymbol{A}$. Then,

$$\begin{aligned} \boldsymbol{A}^{1/2}\boldsymbol{A}^{1/2} &= \boldsymbol{V}\boldsymbol{\Lambda}^{1/2}\boldsymbol{V}^\top\boldsymbol{V}\boldsymbol{\Lambda}^{1/2}\boldsymbol{V}^\top \\ &= \boldsymbol{V}\boldsymbol{\Lambda}^{1/2}\boldsymbol{\Lambda}^{1/2}\boldsymbol{V}^\top \\ &= \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^\top = \boldsymbol{A}. \end{aligned} \tag{A.9}$$

It is immediately clear from the definition that $\boldsymbol{A}^{1/2}$ is also symmetric and positive (semi-)definite.

Quadratic forms of positive semi-definite matrices are a generalization of the Euclidean norm, as

$$\|\boldsymbol{x}\|_{\boldsymbol{A}}^2 = \boldsymbol{x}^\top\boldsymbol{A}\boldsymbol{x} = \boldsymbol{x}^\top\boldsymbol{A}^{1/2}\boldsymbol{A}^{1/2}\boldsymbol{x} = (\boldsymbol{A}^{1/2}\boldsymbol{x})^\top\boldsymbol{A}^{1/2}\boldsymbol{x} = \|\boldsymbol{A}^{1/2}\boldsymbol{x}\|_2^2, \tag{A.10}$$

and in particular,

$$\log \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{1}{2}\|\boldsymbol{x} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}^{-1}}^2 + \text{const} \tag{A.11}$$

$$= -\frac{1}{2}\left[\boldsymbol{x}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{x} - 2\boldsymbol{\mu}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{x}\right] + \text{const}, \tag{A.12}$$

$$\log \mathcal{N}(\boldsymbol{x}; \boldsymbol{0}, \boldsymbol{I}) = -\frac{1}{2}\|\boldsymbol{x}\|_2^2 + \text{const}. \tag{A.13}$$

## *A.3 Parameter Estimation*

In this section, we provide a more detailed account of parameter estimation as outlined in Section 1.3.

### *A.3.1 Estimators*

Suppose we are given a collection of independent samples $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ from some random vector $\mathbf{X}$. Often, the exact distribution of $\mathbf{X}$ is unknown to us, but we still want to "estimate" some property of this

distribution, for example its mean. We denote the property that we aim to estimate from our sample by $\theta^\star$. For example, if our goal is estimating the mean of $\mathbf{X}$, then $\theta^\star \doteq \mathbb{E}[\mathbf{X}]$.

An *estimator* for a parameter $\theta^\star$ is a random vector $\hat{\theta}_n$ that is a function of $n$ sample variables $\mathbf{X}_1, \ldots, \mathbf{X}_n$ whose distribution is identical to the distribution of $\mathbf{X}$. Any concrete sample $\{x_i \sim \mathbf{X}_i\}_{i=1}^n$ yields a concrete estimate of $\theta^\star$.

**Example A.6: Estimating expectations**

The most common estimator for $\mathbb{E}[\mathbf{X}]$ is the *sample mean*

$$\overline{\mathbf{X}}_n \doteq \frac{1}{n}\sum_{i=1}^{n} \mathbf{X}_i \tag{A.14}$$

where $\mathbf{X}_i \overset{\text{iid}}{\sim} \mathbf{X}$. Using a sample mean to estimate an expectation is often also called *Monte Carlo sampling*, and the resulting estimate is referred to as a *Monte Carlo average*.

**Example A.7: MLE of Bernoulli random variables**

Let us consider an i.i.d. sample $x_{1:n}$ of $\text{Bern}(p)$ distributed random variables. We want to estimate the parameter $p$ using a MLE. We have,

$$\begin{aligned}
\hat{p}_{\text{MLE}} &= \arg\max_p \mathbb{P}(x_{1:n} \mid p) \\
&= \arg\max_p \sum_{i=1}^{n} \log \mathbb{P}(x_i \mid p) \\
&= \arg\max_p \sum_{i=1}^{n} \log p^{x_i}(1-p)^{1-x_i} \qquad \text{using the Bernoulli PMF, see Section A.1.1} \\
&= \arg\max_p \sum_{i=1}^{n} x_i \log p + (1-x_i)\log(1-p).
\end{aligned}$$

Computing the first derivative with respect to $p$, we see that the objective is maximized by

$$= \frac{1}{n}\sum_{i=1}^{n} x_i. \tag{A.15}$$

Thus, the maximum likelihood estimate for the parameter $p$ of $X \sim \text{Bern}(p)$ coincides with the sample mean $\overline{X}_n$.

What does it mean for $\hat{\theta}_n$ to be a "good estimate" of $\theta^\star$? There are two canonical measures of goodness of an estimator: its bias and its variance.

Clearly, we want $\mathbb{E}\left[\hat{\boldsymbol{\theta}}_n\right] = \boldsymbol{\theta}^\star$. Estimators that satisfy this property are called *unbiased*. The *bias*, $\mathbb{E}_{\hat{\boldsymbol{\theta}}_n}\left[\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}^\star\right]$, of an unbiased estimator is $\mathbf{0}$. It follows directly from linearity of expectation (1.20) that the sample mean is unbiased. In Section A.3.3, we will see that the variance of the sample mean is small for reasonably large $n$ for "light-tailed" distributions.

**Example A.8: Estimating variances**

Analogously to the sample mean, the most common estimator for the covariance matrix $\mathrm{Var}[\mathbf{X}]$ is the *sample variance* (also called *sample covariance matrix*)

$$\boldsymbol{S}_n^2 \doteq \frac{1}{n-1}\sum_{i=1}^{n}(\mathbf{X}_i - \overline{\mathbf{X}}_n)(\mathbf{X}_i - \overline{\mathbf{X}}_n)^\top \tag{A.16}$$

$$= \frac{n}{n-1}\left(\frac{1}{n}\sum_{i=1}^{n}\mathbf{X}_i\mathbf{X}_i{}^\top - \overline{\mathbf{X}}_n\overline{\mathbf{X}}_n^\top\right) \tag{A.17}$$

where $\mathbf{X}_i \overset{\text{iid}}{\sim} \mathbf{X}$. It can be shown that the sample variance is unbiased (?). Intuitively, the normalizing factor is $1/n-1$ because $\boldsymbol{S}_n^2$ depends on the sample mean $\overline{\mathbf{X}}_n$, which is obtained using the same samples. For this reason, $\boldsymbol{S}_n^2$ has $n-1$ degrees of freedom.[3]

**Problem A.1**

[3] That is, any sample $\mathbf{X}_i$ can be recovered using all other samples and the sample mean.

The second desirable property of an estimator $\hat{\boldsymbol{\theta}}_n$ is that its variance is small.[4] A common measure for the variance of an estimator of $\theta$ is the *mean squared error*,

$$\mathrm{MSE}(\hat{\theta}_n) \doteq \mathbb{E}_{\hat{\theta}_n}\left[(\hat{\theta}_n - \theta^\star)^2\right]. \tag{A.18}$$

[4] The variance of estimators vector-valued estimators $\hat{\boldsymbol{\theta}}_n$ is typically studied component wise.

The mean squared error can be decomposed into the estimator's bias and variance:

$$\mathrm{MSE}(\hat{\theta}_n) = \mathbb{E}_{\hat{\theta}_n}\left[(\hat{\theta}_n - \theta^\star)^2\right] = \mathrm{Var}\left[\hat{\theta}_n\right] + (\mathbb{E}\left[\hat{\theta}_n\right] - \theta^\star)^2, \tag{A.19}$$

using (1.35) and $\mathrm{Var}_{\hat{\theta}_n}\left[\hat{\theta}_n - \theta^\star\right] = \mathrm{Var}\left[\hat{\theta}_n\right]$

the mean squared error of an estimator can be written as the sum of its variance and its squared bias.

A desirable property is for $\hat{\theta}_n$ to converge to $\theta^\star$ in probability (A.28):

$$\forall \epsilon > 0: \quad \lim_{n\to\infty} \mathbb{P}\left(\left|\hat{\theta}_n - \theta^\star\right| > \epsilon\right) = 0.$$

Such estimators are called *consistent*, and a sufficient condition for consistency is that the mean squared error converges to zero as $n \to \infty$.[5]

[5] It follows from Chebyshev's inequality (A.72) that $\mathbb{P}\left(\left|\hat{\theta}_n - \theta^\star\right| > \epsilon\right) \leq \frac{\mathrm{MSE}(\hat{\theta}_n)}{\epsilon^2}$.

Consistency is an asymptotic property. In practice, one would want to know "how quickly" the estimator converges as $n$ grows. To this end, an estimator $\hat{\theta}_n$ is said to be *sharply concentrated* around $\theta^\star$ if

$$\forall \epsilon > 0: \quad \mathbb{P}\left(\left|\hat{\theta}_n - \theta^\star\right| > \epsilon\right) \leq \exp(-\Omega(\epsilon)), \tag{A.20}$$

where $\Omega(\epsilon)$ denotes the class of functions that grow at least linearly in $\epsilon$.[6] Thus, if an estimator is sharply concentrated, its absolute error is bounded by an exponentially quickly decaying error probability.

[6] That is, $h \in \Omega(\epsilon)$ if and only if $\lim_{\epsilon\to\infty} \frac{h(\epsilon)}{\epsilon} > 0$. With slight abuse of notation, we force $h$ to be positive (so as to ensure that the argument to the exponential function is negative) whereas in the traditional definition of Landau symbols, $h$ is only required to grow linearly in absolute value.

### A.3.2 Heavy Tails

It is often said that a sharply concentrated estimator $\hat{\theta}_n$ has *small tails*, where "tails" refer to the "ends" of a PDF. Let us examine the difference between a *light-tailed* and a *heavy-tailed* distribution more closely.

**Definition A.9.** A distribution $P_X$ is said to have a *heavy (right) tail* if its *tail distribution function*

$$\overline{P}_X(x) \doteq 1 - P_X(x) = \mathbb{P}(X > x) \tag{A.21}$$

decays slower than that of the exponential distribution, that is,

$$\limsup_{x\to\infty} \frac{\overline{P}_X(x)}{e^{-\lambda x}} = \infty \tag{A.22}$$

for all $\lambda > 0$. When $\limsup_{x\to\infty} \frac{\overline{P}_X(x)}{\overline{P}_Y(x)} > 1$, the (right) tail of $X$ is said to be *heavier* than the (right) tail of $Y$.

It is immediate from the definitions that the distribution of an unbiased estimator is light-tailed if and only if the estimator is sharply concentrated, so both notions are equivalent.

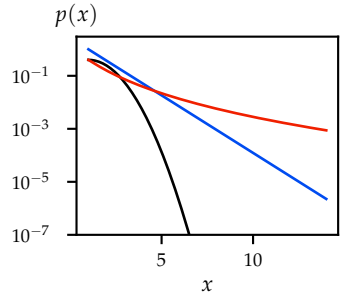



Figure A.1: Shown are the right tails of the PDFs of a **Gaussian** with mean 1 and variance 1, a **exponential distribution** with mean 1 and parameter $\lambda = 1$, and a **log-normal distribution** with mean 1 and variance 1 on a log-scale.

**Example A.10: Light- and heavy-tailed distributions**

- A Gaussian $X \sim \mathcal{N}(0,1)$ is light-tailed since its tail distribution function is bounded by

$$\overline{P}_X(x) = \int_x^\infty \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\, dt \leq \int_x^\infty \frac{t}{x}\frac{1}{\sqrt{2\pi}} e^{-t^2/2}\, dt = \frac{e^{-x^2/2}}{\sqrt{2\pi}x}. \tag{A.23}$$

using $\frac{t}{x} \geq 1$

- The *Laplace distribution* with PDF

$$\text{Laplace}(x;\mu,h) \propto \exp\left(-\frac{|x-\mu|}{h}\right) \tag{A.24}$$

is light-tailed, but its tails decay slower than those of the Gaussian. It is also "more sharply peaked" at its mean than the Gaussian.

Heavy-tailed distributions frequently occur in many domains, the following just serving as a few examples.

- A well-known example of a heavy-tailed distribution is the *log-normal distribution*. A random variable $X$ is logarithmically normal distributed with parameters $\mu$ and $\sigma^2$ if $\log X \sim \mathcal{N}(\mu, \sigma^2)$.

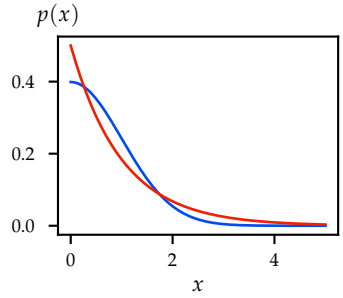



Figure A.2: Shown are the right tails of the PDFs of a **Laplace distribution** with mean 0 and length scale 1 and a **Gaussian** with mean 0 and variance 1.

The log-normal arises when modeling natural growth phenomena or stock prices which are multiplicative, and hence, become additive on a logarithmic scale.

- The *Pareto distribution* was originally used to model the distribution of wealth in a society, but it is also used to model many other phenomena such as the size of cities, the frequency of words, and the returns on stocks. Formally, the Pareto distribution is defined by the following PDF,

$$\text{Pareto}(x; \alpha, c) = \frac{\alpha c^\alpha}{x^{\alpha+1}} \mathbb{1}\{x \geq c\}, \quad x \in \mathbb{R} \tag{A.25}$$

  where the *tail index* $\alpha > 0$ models the "weight" of the right tail of the distribution (larger $\alpha$ means lighter tail) and $c > 0$ corresponds to a cutoff threshold. The distribution is supported on $[c, \infty)$, and as $\alpha \to \infty$ it approaches a point density at $c$. The Pareto's right tail is $\overline{P}(x) = (\frac{c}{x})^\alpha$ for all $x \geq c$.
- The *Cauchy distribution* arises in the description of numerous physical phenomena. The CDF of a Cauchy distribution is

$$P_X(x) \doteq \frac{1}{\pi} \arctan\left(\frac{x - m}{\tau}\right) + \frac{1}{2} \tag{A.26}$$

  where $X \sim \text{Cauchy}(m, \tau)$ with location $m$ and scale $\tau > 0$.

Light- and heavy-tailed distributions exhibit dramatically different behavior. For example, consider the sample mean $\overline{X}_n$ of $n$ i.i.d. random variables $X_i$ with mean 1, and we are told that the sample mean is 5. What does this tell us about the (conditional) distribution of $X_1$? There are many possible explanations for this observation, but they can be largely grouped into two categories:

1. either *many* $X_i$ are *slightly* larger than 1,
2. or *very few* $X_i$ are *much* larger than 1.

Nair et al. (2022) term interpretation (1) the *conspiracy principle* and interpretation (2) the *catastrophe principle*. It turns out that which of the two principles applies depends on the tails of the distribution of $X^{(1)}$. If the tails are light, then the conspiracy principle applies, and if the tails are heavy, then the catastrophe principle applies as is illustrated in Figure A.3.

When working with light-tailed distributions, the conspiracy principle tells us that outliers are rare, and hence, we can usually ignore them. In contrast, when working with heavy-tailed distributions, the catastrophe principle tells us that outliers are not just common but a defining feature, and hence, we need to be careful to not ignore them.

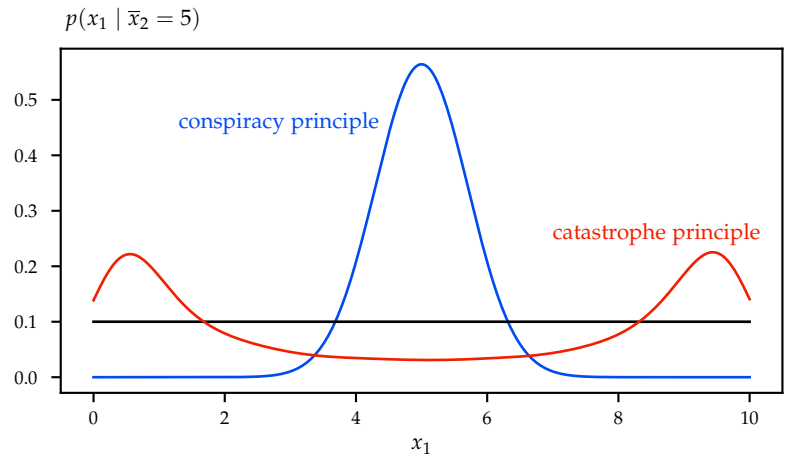



Figure A.3: Shown are the conditional distributions of $X_1$ given the event that the sample mean across two samples is surprisingly large: $\overline{X}_2 = 5$. We plot the examples where $X$ is a **light-tailed** Gaussian with mean 1, $X$ is a **heavy-tailed** log-normal with mean 1, and $X$ is **exponential** with mean 1. When $X$ is *light-tailed*, the large mean is explained by "conspiratory" samples $X_1$ and $X_2$. In contrast, when $X$ is *heavy-tailed*, the large mean is explained by a single "catastrophic" event. See also Nair et al. (2022).

**Readings**

For more details and examples of heavy-tailed distributions, refer to "The Fundamentals of Heavy Tails" (Nair et al., 2022).

### *A.3.3 Mean Estimation and Concentration*

We have seen that we desire two properties in estimators: namely, (1) that they are unbiased; and (2) that their variance is small.[7] For estimating expectations with the sample mean $\overline{X}_n$ which is also known as a *Monte Carlo approximation*, we will see that both properties follow from standard results in statistics.

[7] That is, they are consistent and, ideally, their variance converges quickly.

We have already concluded that the sample mean (A.14) is an unbiased estimator for $\mathbb{E}X$. We will now see that the sample mean is also consistent, that its limiting distribution can usually be determined explicitly, and in some cases it is even sharply concentrated. First, let us recall the common notions of convergence of sequences of random variables.

**Definition A.11** (Convergence of random variables)**.** Let $\{X_n\}_{n\in\mathbb{N}}$ be a sequence of random variables and $X$ another random variable. We say that,

1. $X_n$ converges to $X$ *almost surely* (also called *convergence with probability* 1) if

$$\mathbb{P}\left(\left\{\omega \in \Omega : \lim_{n\to\infty} X_n(\omega) = X(\omega)\right\}\right) = 1, \tag{A.27}$$

   and we write $X_n \overset{a.s.}{\to} X$ as $n \to \infty$.
2. $X_n$ converges to $X$ *in probability* if for any $\epsilon > 0$,

$$\lim_{n\to\infty} \mathbb{P}(|X_n - X| > \epsilon) = 0, \tag{A.28}$$

and we write $X_n \xrightarrow{\mathbb{P}} X$ as $n \to \infty$.

3. $X_n$ converges to $X$ *in distribution* if for all points $x \in X(\Omega)$ at which $P_X$ is continuous,

$$\lim_{n\to\infty} P_{X_n}(x) = P_X(x), \tag{A.29}$$

and we write $X_n \xrightarrow{\mathcal{D}} X$ as $n \to \infty$.

It can be shown that as $n \to \infty$,

$$X_n \xrightarrow{a.s.} X \implies X_n \xrightarrow{\mathbb{P}} X \implies X_n \xrightarrow{\mathcal{D}} X. \tag{A.30}$$

**Remark A.12: Convergence in mean square**

Mean square continuity and mean square differentiability are a generalization of continuity and differentiation to random processes, where the limit is generalized to a limit in the "mean square sense".

A sequence of random variables $\{X_n\}_{n\in\mathbb{N}}$ is said to converge to the random variable $X$ in *mean square* if

$$\lim_{n\to\infty} \mathbb{E}\left[(X_n - X)^2\right] = 0. \tag{A.31}$$

Using Markov's inequality (A.71) it can be seen that convergence in mean square implies convergence in probability. Moreover, convergence in mean square implies

$$\lim_{n\to\infty} \mathbb{E}X_n = \mathbb{E}X \quad \text{and} \tag{A.32}$$

$$\lim_{n\to\infty} \mathbb{E}X_n^2 = \mathbb{E}X^2. \tag{A.33}$$

Whereas a deterministic function $f(\boldsymbol{x})$ is said to be continuous at a point $\boldsymbol{x}^\star$ if $\lim_{\boldsymbol{x}\to\boldsymbol{x}^\star} f(\boldsymbol{x}) = f(\boldsymbol{x}^\star)$, a random process $f(\boldsymbol{x})$ is *mean square continuous* at $\boldsymbol{x}^\star$ if

$$\lim_{\boldsymbol{x}\to\boldsymbol{x}^\star} \mathbb{E}\left[(f(\boldsymbol{x}) - f(\boldsymbol{x}^\star))^2\right] = 0. \tag{A.34}$$

It can be shown that a random process is mean square continuous at $\boldsymbol{x}^\star$ iff its kernel function $k(\boldsymbol{x}, \boldsymbol{x}')$ is continuous at $\boldsymbol{x} = \boldsymbol{x}' = \boldsymbol{x}^\star$.

Similarly, a random process $f(\boldsymbol{x})$ is *mean square differentiable* at a point $\boldsymbol{x}$ in direction $i$ if $(f(\boldsymbol{x} + \delta\boldsymbol{e}_i) - f(\boldsymbol{x}))/\delta$ converges in mean square as $\delta \to 0$ where $\boldsymbol{e}_i$ is the unit vector in direction $i$. This notion can be extended to higher-order derivatives.

For the precise notions of mean square continuity and mean square differentiability in the context of Gaussian processes, refer to section 4.1.1 of "Gaussian processes for machine learning" (Williams and Rasmussen, 2006).

Given a random variable $X : \Omega \to \mathbb{R}$ with finite variance, it can be shown that

$$\overline{X}_n \xrightarrow{\mathbb{P}} \mathbb{E}X \tag{A.35}$$

which is known as the *weak law of large numbers* (WLLN) and which establishes consistency of the sample mean (?). Using more advanced tools, it is possible to show almost sure convergence of the sample mean even when the variance is infinite:

**Problem A.2**

**Fact A.13** (Strong law of large numbers, SLLN)**.** *Given the random variable $X : \Omega \to \mathbb{R}$ with finite mean. Then, as $n \to \infty$,*

$$\overline{X}_n \xrightarrow{a.s.} \mathbb{E}X. \tag{A.36}$$

To get an idea of how quickly the sample mean converges, we can look at its variance:

$$\mathrm{Var}\left[\overline{X}_n\right] = \mathrm{Var}\left[\frac{1}{n}\sum_{i=1}^{n} X_i\right] = \frac{\mathrm{Var}[X]}{n}. \tag{A.37}$$

Remarkably, one cannot only compute its variance, but also its limiting distribution. This is known as the *central limit theorem* (CLT) which states that the prediction error of the sample mean tends to a normal distribution as the sample size goes to infinity (even if the samples themselves are not normally distributed).

**Fact A.14** (Central limit theorem by Lindeberg-Lévy)**.** *Given the random variable $X : \Omega \to \mathbb{R}$ with finite mean and finite variance. Then,*

$$\frac{\overline{X}_n - \mathbb{E}X}{\sqrt{\mathrm{Var}\left[\overline{X}_n\right]}} \xrightarrow{\mathcal{D}} \mathcal{N}(0,1) \tag{A.38}$$

*as $n \to \infty$.*

$\frac{\overline{X}_n - \mathbb{E}X}{\sqrt{\mathrm{Var}\left[\overline{X}_n\right]}}$ is the standardized sample mean, subtracting its mean and dividng by its standard deviation

The central limit theorem makes the critical assumption that the variance of $X$ is finite. This is not the case for many heavy-tailed distributions such as the Pareto or Cauchy distributions. One can generalize the central limit theorem to distributions with infinite variance, with the important distinction that their limiting distribution is no longer a Gaussian but also a heavy-tailed distribution with infinite variance (Nair et al., 2022).

For a subclass of light-tailed distributions it is also possible to show that the sample mean $\overline{X}_n$ is sharply concentrated, which is a much stronger and much more practical property than consistency. We will consider the class of sub-Gaussian random variables which encompass

random variables whose tail probabilities decay at least as fast as those of a Gaussian.

**Definition A.15** (Sub-Gaussian random variable)**.** A random variable $X : \Omega \to \mathbb{R}$ is called *$\sigma$-sub-Gaussian* for $\sigma > 0$ if for all $\lambda \in \mathbb{R}$,

$$\mathbb{E}\left[e^{\lambda(X-\mathbb{E}X)}\right] \leq \exp\left(\frac{\sigma^2\lambda^2}{2}\right). \tag{A.39}$$

**Example A.16: Examples of sub-Gaussian random variables**

- If a random variable $X$ is $\sigma$-sub-Gaussian, then so is $-X$.
- If $X \sim \mathcal{N}(\mu, \sigma^2)$ then a simple calculation shows that

$$\varphi_X(\lambda) \doteq \mathbb{E}\left[e^{\lambda X}\right] = \exp\left(\mu\lambda + \frac{\sigma^2\lambda^2}{2}\right) \qquad \text{for any } \lambda \in \mathbb{R} \tag{A.40}$$

  which is called the "moment-generating function" of the normal distribution, and implying that $X$ is $\sigma$-sub-Gaussian.
- If the random variable $X : \Omega \to \mathbb{R}$ satisfies $a \leq X \leq b$ with probability 1 then $X$ is $\frac{b-a}{2}$-sub-Gaussian. This is also known as *Hoeffding's lemma*.

**Theorem A.17** (Hoeffding's inequality)**.** *Let $X : \Omega \to \mathbb{R}$ be a $\sigma$-sub-Gaussian random variable. Then, for any $\epsilon > 0$,*

$$\mathbb{P}\left(\left|\overline{X}_n - \mathbb{E}X\right| \geq \epsilon\right) \leq 2\exp\left(-\frac{n\epsilon^2}{2\sigma^2}\right). \tag{A.41}$$

In words, the absolute error of the sample mean is bounded by an exponentially quickly decaying error probability $\delta$. Solving for $n$, we obtain that for

$$n \geq \frac{2\sigma^2}{\epsilon^2}\log\frac{2}{\delta} \tag{A.42}$$

the probability that the absolute error is greater than $\epsilon$ is at most $\delta$.

*Proof of Theorem A.17.* Let $S_n \doteq n\overline{X}_n = X_1 + \cdots + X_n$. We have for any $\lambda, \epsilon > 0$ that

$$\begin{aligned}
\mathbb{P}\left(\overline{X}_n - \mathbb{E}X \geq \epsilon\right) &= \mathbb{P}(S_n - \mathbb{E}S_n \geq n\epsilon) \\
&= \mathbb{P}\left(e^{\lambda(S_n - \mathbb{E}S_n)} \geq e^{n\epsilon\lambda}\right) && \text{using that } z \mapsto e^{\lambda z} \text{ is increasing} \\
&\leq e^{-n\epsilon\lambda}\mathbb{E}\left[e^{\lambda(S_n - \mathbb{E}S_n)}\right] && \text{using Markov's inequality (A.71)} \\
&= e^{-n\epsilon\lambda}\prod_{i=1}^{n}\mathbb{E}\left[e^{\lambda(X_i - \mathbb{E}X)}\right] && \text{using independence of the } X_i
\end{aligned}$$

$$\leq e^{-n\epsilon\lambda} \prod_{i=1}^{n} e^{\sigma^2\lambda^2/2} \qquad \text{using the characterizing property of a } \sigma\text{-sub-Gaussian random variable (A.39)}$$

$$= \exp\left(-n\epsilon\lambda + \frac{n\sigma^2\lambda^2}{2}\right).$$

Minimizing the expression with respect to $\lambda$, we set $\lambda = \epsilon/\sigma^2$, and obtain

$$\mathbb{P}\left(\overline{X}_n - \mathbb{E}X \geq \epsilon\right) \leq \min_{\lambda>0}\left\{\exp\left(-n\epsilon\lambda + \frac{n\sigma^2\lambda^2}{2}\right)\right\} = \exp\left(-\frac{n\epsilon^2}{2\sigma^2}\right).$$

The theorem then follows from

$$\mathbb{P}\left(\left|\overline{X}_n - \mathbb{E}X\right| \geq \epsilon\right) = \mathbb{P}\left(\overline{X}_n - \mathbb{E}X \geq \epsilon\right) + \mathbb{P}\left(\overline{X}_n - \mathbb{E}X \leq -\epsilon\right)$$

and noting that the second term can be bounded analogously to the first term by considering the random variables $-X_1, \ldots, -X_n$. □

The law of large numbers and Hoeffding's inequality tell us that when $X$ is light-tailed, we can estimate $\mathbb{E}X$ very precisely with "few" samples using a sample mean. Crucially, the sample mean requires *independent* samples $x_i$ from $X$ which are often hard to obtain.

**Remark A.18: When the sample mean fails**

While we have seen that the sample mean works well for light-tailed distributions, it fails for heavy-tailed distributions. For example, if we only assume that the variance of $X$ is finite, then the best known error rate is obtained by applying Chebyshev's inequality (A.72),

$$\mathbb{P}\left(\left|\overline{X}_n - \mathbb{E}X\right| \geq \epsilon\right) \leq \frac{\mathrm{Var}[X]}{n\epsilon^2} \tag{A.43}$$

which decays only linearly in $n$. This is a result of the catastrophe principle, namely, that outliers may be likely and therefore need to be accounted for.

Robust methods of estimation in the presence of outliers or corruptions have been studied extensively in *robust statistics*. On the subject of mean estimation, approaches such as trimming outliers before taking the sample mean (called the *truncated sample mean*) or using the *median-of-means* which selects the median among $k$ sample means — each computed on a subset of the data — have been shown to yield sharply concentrated estimates, even when the variance of $X$ is infinite (Bubeck et al., 2013).

### A.3.4 Asymptotic Efficiency of Maximum Likelihood Estimation

In Section 1.3.1, we briefly discussed the asymptotic behavior of the MLE, and we mentioned that the MLE can be shown to be "asymptotically efficient".

Let us first address the question of how the asymptotic covariance matrix $\boldsymbol{S}_n$ looks like? It can be shown that $\boldsymbol{S}_n = \boldsymbol{I}_n(\boldsymbol{\theta})^{-1}$ where

$$\boldsymbol{I}_n(\boldsymbol{\theta}) \doteq \mathbb{E}_{\mathcal{D}_n}[\boldsymbol{H}_{\boldsymbol{\theta}} \ell_{\text{nll}}(\boldsymbol{\theta}; \mathcal{D}_n)] \tag{A.44}$$

is the so-called *Fisher information* which captures the curvature of the negative log-likelihood around $\boldsymbol{\theta}$. The Fisher information $\boldsymbol{I}_n(\boldsymbol{\theta})$ can be used to measure the "difficulty" of estimating $\boldsymbol{\theta}$ as shown by the *Cramér-Rao lower bound*:

**Fact A.19** (Cramér-Rao lower bound)**.** *Let $\hat{\boldsymbol{\theta}}_n$ be an unbiased estimator of $\boldsymbol{\theta}$. Then,*[8]

$$\text{Var}\big[\hat{\boldsymbol{\theta}}_n\big] \succeq \boldsymbol{I}_n(\boldsymbol{\theta})^{-1}. \tag{A.45}$$

[8] We use $\boldsymbol{A} \succeq \boldsymbol{B}$ as shorthand for $\boldsymbol{A} - \boldsymbol{B}$ being positive semi-definite. This partial ordering of positive semi-definite matrices is called the *Loewner order*.

An estimator is called *efficient* if it achieves equality in the Cramér-Rao lower bound, and the MLE is therefore *asymptotically efficient*.

### A.3.5 Population Risk and Empirical Risk

The notion of "error" mentioned in Section 1.3 is typically captured by a loss function $\ell(\hat{y}; y) \in \mathbb{R}$ which is small when the prediction $\hat{y} = \hat{f}(\boldsymbol{x})$ is "close" to the true label $y = f^\star(\boldsymbol{x})$ and large otherwise. Let us fix a distribution $\mathcal{P}_{\mathcal{X}}$ over inputs, which together with the likelihood from Equation (1.56) induces an unknown joint distribution over input-label pairs $(\boldsymbol{x}, y)$ by $\mathcal{P}$. The canonical objective is to best-approximate the mappings $(\boldsymbol{x}, y) \sim \mathcal{P}$, that is, to minimize

$$\mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{P}}\Big[\ell(\hat{f}(\boldsymbol{x}); y)\Big]. \tag{A.46}$$

This quantity is also called the *population risk*. However, the underlying distribution $\mathcal{P}$ is unknown to us. All that we can work with is the training data for which we assume $\mathcal{D}_n \overset{\text{iid}}{\sim} \mathcal{P}$. It is therefore natural to consider minimizing

$$\frac{1}{n}\sum_{i=1}^{n} \ell(\hat{f}(\boldsymbol{x}_i); y_i), \quad \mathcal{D}_n = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n, \tag{A.47}$$

which is known as the *empirical risk*.

However, selecting $\hat{f}$ by minimizing the empirical risk can be problematic. The reason is that in this case the model $\hat{f}$ and the empirical risk depend on the same data $\mathcal{D}_n$, implying that the empirical risk will not

be an unbiased estimator of the population risk. This can result in a model which fits the training data too closely, and which is failing to generalize to unseen data — a problem called *overfitting*. We will discuss some (probabilistic) solutions to this problem in Section 4.4 when covering model selection.

## *A.4 Optimization*

Finding parameter estimates is one of the many examples where we seek to minimize some function $\ell$.[9] The field of optimization has a rich history, which we will not explore in much detail here. What will be important for us is that given that the function to be optimized (called the *objective* or *loss*) fulfills certain smoothness properties, optimization is a well-understood problem and can often be solved exactly (e.g., when the objective is convex) or "approximately" when the objective is non-convex. In fact, we will see that it is often advantageous to frame problems as optimization problems when suitable because the machinery to solve these problems is so extensive.

[9] W.l.o.g. we assume that we want to minimize $\ell$. If we wanted to maximize the objective, we can simply minimize its negation.

**Readings**

For a more thorough reminder of optimization methods, read chapter 7 of "Mathematics for machine learning" (Deisenroth et al., 2020).

### *A.4.1 Stationary Points*

In this section, we derive some basic facts about unconstrained optimization problems. Given some function $f : \mathbb{R}^n \to \mathbb{R}$, we want to find

$$\min_{\boldsymbol{x} \in \mathbb{R}^n} f(\boldsymbol{x}). \tag{A.48}$$

We say that a point $\boldsymbol{x}^\star \in \mathbb{R}^n$ is a *(global) optimum* of $f$ if $f(\boldsymbol{x}^\star) \leq f(\boldsymbol{x})$ for any $\boldsymbol{x} \in \mathbb{R}^n$.

Consider the more general problem of minimizing $f$ over some subset $S \subseteq \mathbb{R}^n$, that is, to minimize the function $f_S : S \to \mathbb{R}, \boldsymbol{x} \mapsto f(\boldsymbol{x})$. If there exists some open subset $S \subseteq \mathbb{R}^n$ including $\boldsymbol{x}^\star$ such that $\boldsymbol{x}^\star$ is optimal with respect to the function $f_S$, then $\boldsymbol{x}^\star$ is called a *local optimum* of $f$.

**Remark A.20: Differentiability**

We will generally assume that $f$ is continuously (Fréchet) differentiable on $\mathbb{R}^n$. That is, at any point $\boldsymbol{x} \in \mathbb{R}^n$, there exists $\boldsymbol{\nabla} f(\boldsymbol{x})$

such that for any $\boldsymbol{\delta} \in \mathbb{R}^n$,

$$f(\boldsymbol{x}+\boldsymbol{\delta}) = f(\boldsymbol{x}) + \boldsymbol{\nabla} f(\boldsymbol{x})^\top \boldsymbol{\delta} + o(\|\boldsymbol{\delta}\|_2), \tag{A.49}$$

where $\lim_{\boldsymbol{\delta}\to\mathbf{0}} \frac{o(\|\boldsymbol{\delta}\|_2)}{\|\boldsymbol{\delta}\|_2} = 0$, and $\boldsymbol{\nabla} f(\boldsymbol{x})$ is continuous on $\mathbb{R}^n$.

Equation (A.49) is also called a *first-order expansion* of $f$ at $\boldsymbol{x}$.

**Definition A.21** (Stationary point)**.** Given a function $f : \mathbb{R}^n \to \mathbb{R}$, a point $\boldsymbol{x} \in \mathbb{R}^n$ where $\boldsymbol{\nabla} f(\boldsymbol{x}) = \mathbf{0}$ is called a *stationary point* of $f$.

Being a stationary point is not sufficient for optimality. Take for example the point $x \doteq 0$ of $f(x) \doteq x^3$. Such a point that is stationary but not (locally) optimal is called a *saddle point*.

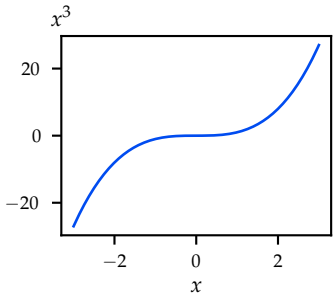



Figure A.4: Example of a saddle point at $x = 0$.

**Theorem A.22** (First-order optimality condition)**.** *If $\boldsymbol{x} \in \mathbb{R}^n$ is a local extremum of a differentiable function $f : \mathbb{R}^n \to \mathbb{R}$, then $\boldsymbol{\nabla} f(\boldsymbol{x}) = \mathbf{0}$.*

*Proof.* Assume $\boldsymbol{x}$ is a local minimum of $f$. Then, for all $\boldsymbol{d} \in \mathbb{R}^n$ and for all small enough $\lambda \in \mathbb{R}$, we have $f(\boldsymbol{x}) \leq f(\boldsymbol{x} + \lambda \boldsymbol{d})$, so

$$\begin{aligned} 0 &\leq f(\boldsymbol{x} + \lambda \boldsymbol{d}) - f(\boldsymbol{x}) \\ &= \lambda \boldsymbol{\nabla} f(\boldsymbol{x})^\top \boldsymbol{d} + o(\lambda \|\boldsymbol{d}\|_2). \end{aligned}$$

using a first-order expansion of $f$ around $\boldsymbol{x}$

Dividing by $\lambda$ and taking the limit $\lambda \to 0$, we obtain

$$0 \leq \boldsymbol{\nabla} f(\boldsymbol{x})^\top \boldsymbol{d} + \lim_{\lambda \to 0} \frac{o(\lambda \|\boldsymbol{d}\|_2)}{\lambda} = \boldsymbol{\nabla} f(\boldsymbol{x})^\top \boldsymbol{d}.$$

Take $\boldsymbol{d} \doteq -\boldsymbol{\nabla} f(\boldsymbol{x})$. Then, $0 \leq -\|\boldsymbol{\nabla} f(\boldsymbol{x})\|_2^2$, so $\boldsymbol{\nabla} f(\boldsymbol{x}) = \mathbf{0}$. □

### A.4.2 Convexity

Convex functions are a subclass of functions where finding global minima is substantially easier than for general functions.

**Definition A.23** (Convex function)**.** A function $f : \mathbb{R}^n \to \mathbb{R}$ is *convex* if

$$\forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n : \forall \theta \in [0,1] : f(\theta \boldsymbol{x} + (1-\theta)\boldsymbol{y}) \leq \theta f(\boldsymbol{x}) + (1-\theta) f(\boldsymbol{y}). \tag{A.50}$$

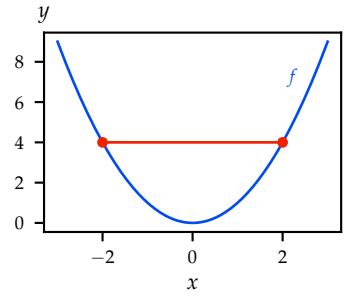



Figure A.5: Example of a convex function. Any line between two points on $f$ lies "above" $f$.

That is, any line between two points on $f$ lies "above" $f$. If the inequality of Equation (A.50) is strict, we say that $f$ is *strictly convex*.

If the function $f$ is convex, we say that the function $-f$ is *concave*. The above is also known as the 0*th-order characterization of convexity*.

**Theorem A.24** (First-order characterization of convexity)**.** *Suppose that $f : \mathbb{R}^n \to \mathbb{R}$ is differentiable, then $f$ is convex if and only if*

$$\forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n : f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \boldsymbol{\nabla} f(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x}). \tag{A.51}$$

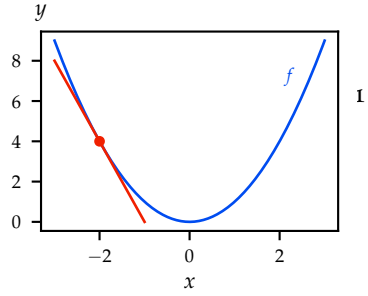



Figure A.6: The first-order characterization characterizes convexity in terms of affine lower bounds. Shown is an affine lower bound based at $x = -2$.

Observe that the right-hand side of the inequality is an affine function with slope $\nabla f(x)$ based at $f(x)$.

*Proof.* In the following, we will make use of directional derivatives. In particular, we will use that given a function $f : \mathbb{R}^n \to \mathbb{R}$ that is differentiable at $x \in \mathbb{R}^n$,

$$\lim_{\lambda \to 0} \frac{f(x + \lambda d) - f(x)}{\lambda} = \nabla f(x)^\top d \tag{A.52}$$

for any direction $d \in \mathbb{R}^n$ (?).

**Problem A.3**

- "⇒": Fix any $x, y \in \mathbb{R}^n$. As $f$ is convex,

  $$f((1-\theta)x + \theta y) \leq (1-\theta)f(x) + \theta f(y),$$

  for all $\theta \in [0, 1]$. We can rearrange to

  $$f(\underbrace{(1-\theta)x + \theta y}_{x+\theta(y-x)}) - f(x) \leq \theta(f(y) - f(x)).$$

  Dividing by $\theta$ yields,

  $$\frac{f(x + \theta(y-x)) - f(x)}{\theta} \leq f(y) - f(x).$$

  Taking the limit $\theta \to 0$ on both sides gives the directional derivative at $x$ in direction $y - x$,

  $$\nabla f(x)^\top (y - x) = Df(x)[y - x] \leq f(y) - f(x).$$

- "⇐": Fix any $x, y \in \mathbb{R}^n$ and let $z \doteq \theta y + (1-\theta)x$. We have,

  $$f(y) \geq f(z) + \nabla f(z)^\top (y - z), \quad \text{and}$$
  $$f(x) \geq f(z) + \nabla f(z)^\top (x - z).$$

  We also have $y - z = (1-\theta)(y - x)$ and $x - z = \theta(x - y)$. Hence,

  $$\theta f(y) + (1-\theta)f(x) \geq f(z) + \nabla f(z)^\top (\underbrace{\theta(y - z) + (1-\theta)(x - z)}_{0})$$
  $$= f(\theta y + (1-\theta)x).$$

□

**Theorem A.25.** *Let $f : \mathbb{R}^n \to R$ be a convex and differentiable function. Then, if $x^\star \in \mathbb{R}^n$ is a stationary point of $f$, $x^\star$ is a global minimum of $f$.*

*Proof.* By the first-order characterization of convexity, we have for any $y \in \mathbb{R}^n$,

$$f(y) \geq f(x^\star) + \underbrace{\nabla f(x^\star)^\top}_{0}(y - x^\star) = f(x^\star).$$

□

Generally, the main difficulty in solving convex optimization problems lies therefore in finding stationary points (or points that are sufficiently close to stationary points).

**Remark A.26: Second-order characterization of convexity**

We say that $f : \mathbb{R}^n \to \mathbb{R}$ is twice continuously (Fréchet) differentiable if for any point $\boldsymbol{x} \in \mathbb{R}^n$, there exists $\boldsymbol{\nabla} f(\boldsymbol{x})$ and $\boldsymbol{H}_f(\boldsymbol{x})$ such that for any $\boldsymbol{\delta} \in \mathbb{R}^n$,

$$f(\boldsymbol{x}+\boldsymbol{\delta}) = f(\boldsymbol{x}) + \boldsymbol{\nabla} f(\boldsymbol{x})^\top \boldsymbol{\delta} + \frac{1}{2}\boldsymbol{\delta}^\top \boldsymbol{H}_f(\boldsymbol{x})\boldsymbol{\delta} + o(\|\boldsymbol{\delta}\|_2^2), \quad \text{(A.53)}$$

where $\lim_{\boldsymbol{\delta}\to\mathbf{0}} \frac{o(\|\boldsymbol{\delta}\|_2^2)}{\|\boldsymbol{\delta}\|_2^2} = 0$, and $\boldsymbol{\nabla} f(\boldsymbol{x})$ and $\boldsymbol{H}_f(\boldsymbol{x})$ are continuous on $\mathbb{R}^n$. Equation (A.53) is also called a *second-order expansion* of $f$ at $\boldsymbol{x}$.

Twice continuously differentiable functions admit a natural characterization of convexity in terms of the Hessian.

**Fact A.27** (Second-order characterization of convexity)**.** *Consider a twice continuously differentiable function $f : \mathbb{R}^n \to R$. Then, $f$ is convex if and only if $\boldsymbol{H}_f(\boldsymbol{x})$ is positive semi-definite for all $\boldsymbol{x} \in \mathbb{R}^n$.*

It follows that $f$ is concave if and only if $\boldsymbol{H}_f(\boldsymbol{x})$ is negative semi-definite for all $\boldsymbol{x} \in \mathbb{R}^n$.

### *A.4.3 Stochastic Gradient Descent*

In this course, we primarily employ so-called first-order methods, which rely on (estimates of) the gradient of the objective function to determine a direction of local improvement. The main idea behind these methods is to repeatedly take a step in the opposite direction of the gradient scaled by a learning rate $\eta_t$, which may depend on the current iteration $t$.

We will often want to minimize a stochastic optimization objective

$$L(\boldsymbol{\theta}) \doteq \mathbb{E}_{\boldsymbol{x}\sim p}[\ell(\boldsymbol{\theta}; \boldsymbol{x})] \quad \text{(A.58)}$$

where $\ell$ and its gradient $\boldsymbol{\nabla}\ell$ are known.

Based on our discussion in previous subsections, it is a natural first step to look for stationary points of $L$, that is, the roots of $\boldsymbol{\nabla} L$.

**Fact A.28** (Robbins-Monro (RM) algorithm)**.** *Let $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^m$ be an unknown function of which we want to find a root and suppose that*

**Jacobian** Given a vector-valued function,

$$\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^m, \quad \boldsymbol{x} \mapsto \begin{bmatrix} g_1(\boldsymbol{x}) \\ \vdots \\ g_m(\boldsymbol{x}) \end{bmatrix},$$

where $g_i : \mathbb{R}^n \to \mathbb{R}$, the *Jacobian* of $\boldsymbol{g}$ at $\boldsymbol{x} \in \mathbb{R}^n$ is

$$\boldsymbol{D}\boldsymbol{g}(\boldsymbol{x}) \doteq \begin{bmatrix} \frac{\partial g_1(\boldsymbol{x})}{\partial x(1)} & \cdots & \frac{\partial g_1(\boldsymbol{x})}{\partial x(n)} \\ \vdots & \ddots & \vdots \\ \frac{\partial g_m(\boldsymbol{x})}{\partial x(1)} & \cdots & \frac{\partial g_m(\boldsymbol{x})}{\partial x(n)} \end{bmatrix}. \quad \text{(A.54)}$$

Observe that for a function $f : \mathbb{R}^n \to \mathbb{R}$,

$$\boldsymbol{D} f(\boldsymbol{x}) = \boldsymbol{\nabla} f(\boldsymbol{x})^\top. \quad \text{(A.55)}$$

**Hessian** The *Hessian* of a function $f : \mathbb{R}^n \to \mathbb{R}$ at a point $\boldsymbol{x} \in \mathbb{R}^n$ is

$$\begin{aligned} \boldsymbol{H}_f(\boldsymbol{x}) &\doteq \boldsymbol{H} f(\boldsymbol{x}) \\ &\doteq \begin{bmatrix} \frac{\partial^2 f(\boldsymbol{x})}{\partial x(1)\,\partial x(1)} & \cdots & \frac{\partial^2 f(\boldsymbol{x})}{\partial x(1)\,\partial x(n)} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\boldsymbol{x})}{\partial x(n)\,\partial x(1)} & \cdots & \frac{\partial^2 f(\boldsymbol{x})}{\partial x(n)\,\partial x(n)} \end{bmatrix} \quad \text{(A.56)} \\ &= (\boldsymbol{D}\boldsymbol{\nabla} f(\boldsymbol{x}))^\top \quad \text{(A.57)} \\ &\in \mathbb{R}^{n\times n}. \end{aligned}$$

Thus, a Hessian captures the curvature of $f$. If the Hessian of $f$ is positive definite at a point $\boldsymbol{x}$, then $f$ is "curved up around $\boldsymbol{x}$".

Hessians are symmetric when the second partial derivatives are continuous, due to Schwartz's theorem.

*we have access to unbiased noisy observations* $\mathbf{G}(\boldsymbol{\theta})$ *of* $\boldsymbol{g}(\boldsymbol{\theta})$*. The scheme*

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t - \eta_t \boldsymbol{g}^{(t)}(\boldsymbol{\theta}_t), \tag{A.59}$$

*where* $\boldsymbol{g}^{(t)}(\boldsymbol{\theta}_t) \sim \mathbf{G}(\boldsymbol{\theta}_t)$ *are independent and unbiased estimates of* $\boldsymbol{g}(\boldsymbol{\theta}_t)$*, is known as the* Robbins-Monro algorithm.

*It can be shown that if the sequence of learning rates* $\{\eta_t\}_{t=0}^{\infty}$ *is chosen such that the* Robbins-Monro conditions,[10]

$$\eta_t \geq 0 \quad \forall t, \quad \sum_{t=0}^{\infty} \eta_t = \infty \quad \textit{and} \quad \sum_{t=0}^{\infty} \eta_t^2 < \infty, \tag{A.60}$$

*and additional regularity assumptions are satisfied,*[11] *then we have that*

$$\boldsymbol{g}(\boldsymbol{\theta}_t) \overset{a.s.}{\to} \mathbf{0} \quad \textit{as } t \to \infty. \tag{A.61}$$

*That is, the RM-algorithm converges to a root almost surely.*[12]

[10] $\eta_t = 1/t$ is an example of a learning rate satisfying the RM-conditions.

[11] For more details, refer to "Online learning and stochastic approximations" (Bottou, 1998).

[12] See Equation (A.27) for a definition of almost sure convergence of a sequence of random variables.

Using Robbins-Monro to find a root of $\nabla L$ is known as *stochastic gradient descent*. In particular, when $\ell$ is convex and the RM-conditions are satisfied, Robbins-Monro converges to a stationary point (and hence, a global minimum) of $L$ almost surely. Moreover, it can be shown that for general $\ell$ and satisfied RM-conditions, stochastic gradient descent converges to a local minimum almost surely. Intuitively, the randomness in the gradient estimates allows the algorithm to "jump past" saddle points.

A commonly used strategy to obtain unbiased gradient estimates is to take the sample mean of the gradient with respect to some set of samples $B$ (also called a *batch*) as is shown in Algorithm A.29.

**Algorithm A.29:** Stochastic gradient descent, SGD

initialize $\boldsymbol{\theta}$
**repeat**
  draw mini-batch $B \doteq \{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}, \boldsymbol{x}^{(i)} \sim p$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta_t \frac{1}{m} \sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \boldsymbol{x}^{(i)})$
**until** converged

**Example A.30: Minimizing training loss**

A common application of SGD in the context of machine learning is the following: $p \doteq \text{Unif}(\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\})$ is a uniform distribution

over the training inputs, yielding the objective

$$L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \ell(\boldsymbol{\theta}; \boldsymbol{x}_i), \tag{A.62}$$

where $\ell(\boldsymbol{\theta}; \boldsymbol{x}_i)$ is the loss of a model parameterized by $\boldsymbol{\theta}$ for training input $\boldsymbol{x}_i$ and $m$ is a fixed *batch size*. Here, computing the gradients exactly (i.e., choosing $m = n$) is expensive when the size of the training set is large.

A commonly used alternative to sampling each batch independently, is to split the training set into equally sized batches and perform a gradient descent step with respect to each of them. Typically, the optimum is not found after a single pass through the training data, so the same procedure is repeated multiple times. One such iteration is called an *epoch*.

**Remark A.31: Regularization via weight decay**

A common technique to improve the "stability" of minima found through gradient-based methods is to "regularize" the loss by adding an explicit bias favoring "simpler" solutions. That is, given a loss $\ell$ measuring the quality of fit, we consider the regularized loss

$$\ell'(\boldsymbol{\theta}; \boldsymbol{x}) \doteq \underbrace{\ell(\boldsymbol{\theta}; \boldsymbol{x})}_{\text{quality of fit}} + \underbrace{r(\boldsymbol{\theta})}_{\text{regularization}} \tag{A.63}$$

where $r(\boldsymbol{\theta})$ is large for "complex" and small for "simple" choices of $\boldsymbol{\theta}$, respectively. A common choice is $r(\boldsymbol{\theta}) = \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2$ for some $\lambda > 0$, which is known as *$L_2$-regularization*.

Recall from Section 1.3.2 that in the context of likelihood maximization, this choice of $r$ corresponds to imposing the Gaussian prior $\mathcal{N}(\mathbf{0}, \lambda \boldsymbol{I})$, and finding the MAP estimate. Imposing, for example, a Laplace prior leads to *$L_1$-regularization*.

Using $L_2$-regularization, we obtain for the gradient,

$$\boldsymbol{\nabla}_{\boldsymbol{\theta}} \ell'(\boldsymbol{\theta}; \boldsymbol{x}) = \boldsymbol{\nabla}_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \boldsymbol{x}) + \lambda \boldsymbol{\theta}. \tag{A.64}$$

Thus, a gradient descent step changes to

$$\boldsymbol{\theta} \leftarrow (1 - \lambda \eta_t) \boldsymbol{\theta} - \eta_t \boldsymbol{\nabla}_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \boldsymbol{x}). \tag{A.65}$$

That is, in each step, $\boldsymbol{\theta}$ decays towards zero at the rate $\lambda \eta_t$. This regularization method is also called *weight decay*.

**Remark A.32: Adaptive learning rates and momentum**

Commonly, in optimization, constant learning rates are used to accelerate mixing. The performance can be further improved by taking an adaptive gradient step with respect to the geometry of the cost function, which is done by commonly used algorithms such as *Adagrad* and *Adam*. The methods employed by these algorithms are known as *adaptive learning rates* and *momentum*.

For more details on momentum read section 7.1.2 of "Mathematics for machine learning" (Deisenroth et al., 2020) and for an overview of the aforementioned optimization algorithms refer to "An overview of gradient descent optimization algorithms" (Ruder, 2016).

## A.5 Useful Matrix Identities and Inequalities

- *Woodbury's matrix identity* states that for any matrices $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, $\boldsymbol{C} \in \mathbb{R}^{m\times m}$, and $\boldsymbol{U}, \boldsymbol{V}^\top \in \mathbb{R}^{n\times m}$ where $\boldsymbol{A}$ and $\boldsymbol{C}$ are invertible,

$$(\boldsymbol{A}+\boldsymbol{UCV})^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}\boldsymbol{U}(\boldsymbol{C}^{-1}+\boldsymbol{VA}^{-1}\boldsymbol{U})^{-1}\boldsymbol{VA}^{-1}. \quad \text{(A.66)}$$

  The following identity, called the *Sherman-Morrison formula*, is a direct consequence:

$$(\boldsymbol{A}+\boldsymbol{x}\boldsymbol{x}^\top)^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}\boldsymbol{x}(1+\boldsymbol{x}^\top\boldsymbol{A}^{-1}\boldsymbol{x})^{-1}\boldsymbol{x}^\top\boldsymbol{A}^{-1} \quad \text{(A.67a)}$$

$$= \boldsymbol{A}^{-1} - \frac{(\boldsymbol{A}^{-1}\boldsymbol{x})(\boldsymbol{A}^{-1}\boldsymbol{x})^\top}{1+\boldsymbol{x}^\top\boldsymbol{A}^{-1}\boldsymbol{x}} \quad \text{(A.67b)}$$

  for any symmetric and invertible matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{x} \in \mathbb{R}^n$.
- The *matrix inversion lemma* states that for matrices $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{n\times n}$,

$$(\boldsymbol{A}+\boldsymbol{B})^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}(\boldsymbol{A}^{-1}+\boldsymbol{B}^{-1})^{-1}\boldsymbol{A}^{-1}. \quad \text{(A.68)}$$

- *Hadamard's inequality* states that

$$\det(\boldsymbol{M}) \leq \prod_{i=0}^{d} \boldsymbol{M}(i,i) \quad \text{(A.69)}$$

  for any positive definite matrix $\boldsymbol{M} \in \mathbb{R}^{d\times d}$.
- The *Weinstein-Aronszajn identity* for positive definite matrices $\boldsymbol{A} \in \mathbb{R}^{d\times n}$ and $\boldsymbol{B} \in \mathbb{R}^{n\times d}$ states that

$$\det(\boldsymbol{I}_{d\times d} + \boldsymbol{AB}) = \det(\boldsymbol{I}_{n\times n} + \boldsymbol{BA}). \quad \text{(A.70)}$$

### Problems

**A.1. The sample variance is unbiased.**

Let $X$ be a zero-mean random variable. Confirm that the sample variance of $X$ is indeed unbiased.

**A.2. Consistency of the sample mean.**

We will now show that the sample mean is consistent. To this end, let us first derive two classical concentration inequalities.

1. Prove *Markov's inequality* which says that if $X$ is a non-negative random variable, then for any $\epsilon > 0$,

$$\mathbb{P}(X \geq \epsilon) \leq \frac{\mathbb{E}X}{\epsilon}. \tag{A.71}$$

2. Prove *Chebyshev's inequality* which says that if $X$ is a random variable with finite and non-zero variance, then for any $\epsilon > 0$,

$$\mathbb{P}(|X - \mathbb{E}X| \geq \epsilon) \leq \frac{\mathrm{Var}X}{\epsilon^2}. \tag{A.72}$$

3. Prove the weak law of large numbers from Equation (A.35).

**A.3. Directional derivatives.**

Given a function $f : \mathbb{R}^n \to \mathbb{R}$ that is differentiable at $\boldsymbol{x} \in \mathbb{R}^n$, the *directional derivative* of $f$ at $\boldsymbol{x}$ in the direction $\boldsymbol{d} \in \mathbb{R}^n$ is

$$Df(\boldsymbol{x})[\boldsymbol{d}] \doteq \lim_{\lambda \to 0} \frac{f(\boldsymbol{x} + \lambda \boldsymbol{d}) - f(\boldsymbol{x})}{\lambda}. \tag{A.73}$$

Show that

$$Df(\boldsymbol{x})[\boldsymbol{d}] = \boldsymbol{\nabla} f(\boldsymbol{x})^\top \boldsymbol{d}. \tag{A.74}$$

*Hint: Consider a first-order expansion of $f$ at $\boldsymbol{x}$ in direction $\lambda \boldsymbol{d}$.*

# B

# Solutions

## *Fundamentals of Inference*

**Solution to Problem 1.1.**

1. By the third axiom and $B = A \cup (B \setminus A)$,

$$\mathbb{P}(B) = \mathbb{P}(A) + \mathbb{P}(B \setminus A).$$

   Noting from the first axiom that $\mathbb{P}(B \setminus A) \geq 0$ completes the proof.
2. By the second axiom,

$$\mathbb{P}\big(A \cup \overline{A}\big) = \mathbb{P}(\Omega) = 1$$

   and by the third axiom,

$$\mathbb{P}\big(A \cup \overline{A}\big) = \mathbb{P}(A) + \mathbb{P}\big(\overline{A}\big).$$

   Reorganizing the equations completes the proof.
3. Define the countable sequence of events

$$A_i' \doteq A_i \setminus \left( \bigcup_{j=1}^{i-1} A_j \right).$$

   Note that the sequence of events $\{A_i'\}_i$ is disjoint. Thus, we have by the third axiom and then using (1) that

$$\mathbb{P}\left( \bigcup_{i=1}^{\infty} A_i \right) = \mathbb{P}\left( \bigcup_{i=1}^{\infty} A_i' \right) = \sum_{i=1}^{\infty} \mathbb{P}\big(A_i'\big) \leq \sum_{i=1}^{\infty} \mathbb{P}(A_i).$$

**Solution to Problem 1.2.** We show that any vertex $v$ is visited eventually with probability 1.

We denote by $w \to v$ the event that the random walk starting at vertex $w$ visits the vertex $v$ eventually, we denote by $\Gamma(w)$ the neighborhood

of $w$, and we write $\deg(w) \doteq |\Gamma(w)|$. We have,

$$\begin{aligned}
\mathbb{P}(w \to v) &= \sum_{w' \in \Gamma(w)} \mathbb{P}\big(\text{the random walk first visits } w'\big) \cdot \mathbb{P}\big(w' \to v\big) && \text{using the law of total probability (1.12)} \\
&= \frac{1}{\deg(w)} \sum_{w' \in \Gamma(w)} \mathbb{P}\big(w' \to v\big). && \text{using that the random walk moves to a neighbor uniformly at random}
\end{aligned}$$

Take $u$ to be the vertex such that $\mathbb{P}(u \to v)$ is minimized. Then,

$$\mathbb{P}(u \to v) = \frac{1}{\deg(u)} \sum_{u' \in \Gamma(u)} \underbrace{\mathbb{P}\big(u' \to v\big)}_{\geq \mathbb{P}(u \to v)} \geq \mathbb{P}(u \to v).$$

That is, for all neighbors $u'$ of $u$, $\mathbb{P}(u \to v) = \mathbb{P}(u' \to v)$. Using that the graph is connected and finite, we conclude $\mathbb{P}(u \to v) = \mathbb{P}(w \to v)$ for any vertex $w$. Finally, note that $\mathbb{P}(v \to v) = 1$, and hence, the random walk starting at any vertex $u$ visits the vertex $v$ eventually with probability 1.

**Solution to Problem 1.3.** Let $\mathbb{1}\{A_i\}$ be the indicator random variable for the event $A_i$. Then,

$$\begin{aligned}
\mathbb{E}[\mathbf{X} \cdot \mathbb{1}\{A_i\}] &= \mathbb{E}[\mathbb{E}[\mathbf{X} \cdot \mathbb{1}\{A_i\} \mid \mathbb{1}\{A_i\}]] && \text{by the tower rule (1.25)} \\
&= \mathbb{E}[\mathbf{X} \cdot \mathbb{1}\{A_i\} \mid \mathbb{1}\{A_i\} = 1] \cdot \mathbb{P}(\mathbb{1}\{A_i\} = 1) && \text{expanding the outer expectation} \\
&\qquad + 0 \cdot \mathbb{P}(\mathbb{1}\{A_i\} = 0) \\
&= \mathbb{E}[\mathbf{X} \mid A_i] \cdot \mathbb{P}(A_i). && \text{the event } \mathbb{1}\{A_i\} = 1 \text{ is equivalent to } A_i
\end{aligned}$$

Summing up for all $i$, the left-hand side becomes

$$\mathbb{E}[\mathbf{X}] = \sum_{i=1}^{k} \mathbb{E}[\mathbf{X} \cdot \mathbb{1}\{A_i\}].$$

**Solution to Problem 1.4.** Let $\boldsymbol{\Sigma} \doteq \mathrm{Var}[\mathbf{X}]$ be a covariance matrix of the random vector $\mathbf{X}$ and fix any $z \in \mathbb{R}^n$. Then,

$$\begin{aligned}
z^\top \boldsymbol{\Sigma} z &= z^\top \mathbb{E}\Big[(\mathbf{X} - \mathbb{E}[\mathbf{X}])(\mathbf{X} - \mathbb{E}[\mathbf{X}])^\top\Big] z && \text{using the definitiion of variance (1.34)} \\
&= \mathbb{E}\Big[z^\top (\mathbf{X} - \mathbb{E}[\mathbf{X}])(\mathbf{X} - \mathbb{E}[\mathbf{X}])^\top z\Big]. && \text{using linearity of expectation (1.20)}
\end{aligned}$$

Define the random variable $Z \doteq z^\top (\mathbf{X} - \mathbb{E}[\mathbf{X}])$. Then,

$$= \mathbb{E}\Big[Z^2\Big] \geq 0.$$

**Solution to Problem 1.5.** Let us start by defining some events that we will reason about later. For ease of writing, let us call the person in question X.

$$D = \text{X has the disease},$$

$P =$ The test shows a positive response.

Now we can translate the information in the question to formal statements in terms of $D$ and $P$,

$$\mathbb{P}(D) = 10^{-4} \qquad \text{the disease is rare}$$
$$\mathbb{P}(P \mid D) = \mathbb{P}\big(\overline{P} \mid \overline{D}\big) = 0.99. \qquad \text{the test is accurate}$$

We want to determine $\mathbb{P}(D \mid P)$. One can find this probability by using Bayes' rule (1.45),

$$\mathbb{P}(D \mid P) = \frac{\mathbb{P}(P \mid D) \cdot \mathbb{P}(D)}{\mathbb{P}(P)}.$$

From the quantities above, we have everything except for $\mathbb{P}(P)$. This, however, we can compute using the law of total probability,

$$\begin{aligned}\mathbb{P}(P) &= \mathbb{P}(P \mid D) \cdot \mathbb{P}(D) + \mathbb{P}\big(P \mid \overline{D}\big) \cdot \mathbb{P}\big(\overline{D}\big) \\ &= 0.99 \cdot 10^{-4} + 0.01 \cdot (1 - 10^{-4}) \\ &= 0.010098.\end{aligned}$$

Hence, $\mathbb{P}(D \mid P) = 0.99 \cdot 10^{-4}/0.010098 \approx 0.0098 = 0.98\%$.

**Solution to Problem 1.6.**

1. Suppose that $\mathbf{X}$ is not linearly independent. Then, there exists some $\boldsymbol{\alpha} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ such that $\boldsymbol{\alpha}^\top \mathbf{X} = 0$. We have that $\boldsymbol{\alpha}$ is an eigenvector of $\mathrm{Var}[\mathbf{X}]$ with zero eigenvalue since

$$\boldsymbol{\alpha}^\top \mathrm{Var}[\mathbf{X}]\boldsymbol{\alpha} = \mathrm{Var}\Big[\boldsymbol{\alpha}^\top \mathbf{X}\Big] = \mathrm{Var}[0] = 0. \qquad \text{using Equation (1.38)}$$

2. Suppose that $\mathrm{Var}[\mathbf{X}]$ has a zero eigenvalue. Let $\boldsymbol{\lambda}$ be the corresponding eigenvector and consider the suggested linear combination $Y \doteq \boldsymbol{\lambda}^\top \mathbf{X}$. We have

$$\begin{aligned}\mathrm{Var}[Y] &= \mathrm{Var}\Big[\boldsymbol{\lambda}^\top \mathbf{X}\Big] \\ &= \boldsymbol{\lambda}^\top \mathrm{Var}[\mathbf{X}]\boldsymbol{\lambda} && \text{using Equation (1.38)} \\ &= \sum_{i=1}^{n} \boldsymbol{\lambda}(i) \cdot 0 && \text{using the property of a zero eigenvalue:} \\ &= 0 && \boldsymbol{\lambda}^\top \mathrm{Var}[\mathbf{X}]\boldsymbol{\lambda} = 0\end{aligned}$$

   which implies that $Y$ must be a constant. Since $\mathbf{X}$ is centered $\mathbb{E}[Y] = \mathbb{E}\big[\boldsymbol{\lambda}^\top \mathbf{X}\big] = \boldsymbol{\lambda}^T \mathbb{E}[\mathbf{X}] = 0$, so this constant must be 0. Hence, $\boldsymbol{\lambda}^\top \mathbf{X} = 0$ almost surely. Taking $\boldsymbol{\alpha} = \boldsymbol{\lambda}$ shows that $\mathbf{X}$ is not linearly independent.

**Solution to Problem 1.7.** We need to find $\boldsymbol{\mu} \in \mathbb{R}^n$ and $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$ such that for all $\boldsymbol{x} \in \mathbb{R}^n$,

$$\begin{aligned}&(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) \\ &\quad = (\boldsymbol{x}-\boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}_1^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_1) + (\boldsymbol{x}-\boldsymbol{\mu}_2)^\top \boldsymbol{\Sigma}_2^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_2) + \text{const.}\end{aligned}$$

The left-hand side of Equation (B.1) is equal to

$$\boldsymbol{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{x} - 2\boldsymbol{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}.$$

The right-hand side of Equation (B.1) is equal to

$$\begin{aligned}&\left(\boldsymbol{x}^\top \boldsymbol{\Sigma}_1^{-1}\boldsymbol{x} + \boldsymbol{x}^\top \boldsymbol{\Sigma}_2^{-1}\boldsymbol{x}\right) - 2\left(\boldsymbol{x}^\top \boldsymbol{\Sigma}_1^{-1}\boldsymbol{\mu}_1 + \boldsymbol{x}^\top \boldsymbol{\Sigma}_2^{-1}\boldsymbol{\mu}_2\right) \\ &\qquad\qquad + \left(\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}_1^{-1}\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2^\top \boldsymbol{\Sigma}_2^{-1}\boldsymbol{\mu}_2\right) \\ &= \boldsymbol{x}^\top \left(\boldsymbol{\Sigma}_1^{-1} + \boldsymbol{\Sigma}_2^{-1}\right)\boldsymbol{x} - 2\boldsymbol{x}^\top \left(\boldsymbol{\Sigma}_1^{-1}\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_2^{-1}\boldsymbol{\mu}_2\right) \\ &\qquad\qquad + \left(\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}_1^{-1}\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2^\top \boldsymbol{\Sigma}_2^{-1}\boldsymbol{\mu}_2\right).\end{aligned}$$

We observe that both sides are equal up to constant terms if

$$\boldsymbol{\Sigma}^{-1} = \boldsymbol{\Sigma}_1^{-1} + \boldsymbol{\Sigma}_2^{-1} \quad \text{and} \quad \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} = \boldsymbol{\Sigma}_1^{-1}\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_2^{-1}\boldsymbol{\mu}_2.$$

**Solution to Problem 1.8.** Recall that independence of any random vectors $\mathbf{X}$ and $\mathbf{Y}$ implies that they are uncorrelated,[1] the converse implication is a special property of Gaussian random vectors.

[1] This is because the expectation of their product factorizes, see Equation (1.21).

Consider the joint Gaussian random vector $\mathbf{Z} \doteq [\mathbf{X}\ \mathbf{Y}] \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and assume that $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}_\mathbf{X}, \boldsymbol{\Sigma}_\mathbf{X})$ and $\mathbf{Y} \sim \mathcal{N}(\boldsymbol{\mu}_\mathbf{Y}, \boldsymbol{\Sigma}_\mathbf{Y})$ are uncorrelated. Then, $\boldsymbol{\Sigma}$ can be expressed as

$$\boldsymbol{\Sigma} = \begin{bmatrix} \boldsymbol{\Sigma}_\mathbf{X} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_\mathbf{Y} \end{bmatrix},$$

implying that the PDF of $\mathbf{Z}$ factorizes,

$$\begin{aligned}&\mathcal{N}([\boldsymbol{x}\ \boldsymbol{y}]; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \\ &= \frac{1}{\sqrt{\det(2\pi\boldsymbol{\Sigma}_\mathbf{X}) \cdot \det(2\pi\boldsymbol{\Sigma}_\mathbf{Y})}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_\mathbf{X})^\top \boldsymbol{\Sigma}_\mathbf{X}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_\mathbf{X})\right. \\ &\qquad\qquad \left. -\frac{1}{2}(\boldsymbol{y}-\boldsymbol{\mu}_\mathbf{Y})^\top \boldsymbol{\Sigma}_\mathbf{Y}^{-1}(\boldsymbol{y}-\boldsymbol{\mu}_\mathbf{Y})\right) \\ &= \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_\mathbf{X}, \boldsymbol{\Sigma}_\mathbf{X}) \cdot \mathcal{N}(\boldsymbol{y}; \boldsymbol{\mu}_\mathbf{Y}, \boldsymbol{\Sigma}_\mathbf{Y}).\end{aligned}$$

**Solution to Problem 1.9.** Let $\boldsymbol{x} \sim \mathbf{X}$. The joint distribution can be expressed as

$$p(\boldsymbol{x}) = p(\boldsymbol{x}_A, \boldsymbol{x}_B)$$

$$= \frac{1}{Z} \exp\left( -\frac{1}{2} \begin{bmatrix} \boldsymbol{x}_A - \boldsymbol{\mu}_A \\ \boldsymbol{x}_B - \boldsymbol{\mu}_B \end{bmatrix}^\top \begin{bmatrix} \boldsymbol{\Lambda}_{AA} & \boldsymbol{\Lambda}_{AB} \\ \boldsymbol{\Lambda}_{BA} & \boldsymbol{\Lambda}_{BB} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_A - \boldsymbol{\mu}_A \\ \boldsymbol{x}_B - \boldsymbol{\mu}_B \end{bmatrix} \right)$$

where $Z$ denotes the normalizing constant. To simplify the notation, we write

$$\begin{bmatrix} \boldsymbol{\Delta}_A \\ \boldsymbol{\Delta}_B \end{bmatrix} \doteq \begin{bmatrix} \boldsymbol{x}_A - \boldsymbol{\mu}_A \\ \boldsymbol{x}_B - \boldsymbol{\mu}_B \end{bmatrix}.$$

1. We obtain

$$\begin{aligned}
&p(\boldsymbol{x}_A) \\
&= \frac{1}{Z} \int \exp\left( -\frac{1}{2} \begin{bmatrix} \boldsymbol{\Delta}_A \\ \boldsymbol{\Delta}_B \end{bmatrix}^\top \begin{bmatrix} \boldsymbol{\Lambda}_{AA} & \boldsymbol{\Lambda}_{AB} \\ \boldsymbol{\Lambda}_{BA} & \boldsymbol{\Lambda}_{BB} \end{bmatrix} \begin{bmatrix} \boldsymbol{\Delta}_A \\ \boldsymbol{\Delta}_B \end{bmatrix} \right) d\boldsymbol{x}_B && \text{using the sum rule (1.7)} \\
&= \frac{1}{Z} \exp\left( -\frac{1}{2} \left[ \boldsymbol{\Delta}_A^\top (\boldsymbol{\Lambda}_{AA} - \boldsymbol{\Lambda}_{AB} \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA}) \boldsymbol{\Delta}_A \right] \right) && \text{using the first hint} \\
&\quad \cdot \int \exp\left( -\frac{1}{2} \left[ (\boldsymbol{\Delta}_B + \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} \boldsymbol{\Delta}_A)^\top \boldsymbol{\Lambda}_{BB} (\boldsymbol{\Delta}_B + \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} \boldsymbol{\Delta}_A) \right] \right) d\boldsymbol{x}_B.
\end{aligned}$$

Observe that the integrand is an unnormalized Gaussian PDF, and hence, the integral evaluates to $\sqrt{\det\left(2\pi \boldsymbol{\Lambda}_{BB}^{-1}\right)}$ (so is constant with respect to $\boldsymbol{x}_A$). We can therefore simplify to

$$\begin{aligned}
&= \frac{1}{Z'} \exp\left( -\frac{1}{2} \left[ \boldsymbol{\Delta}_A^\top (\boldsymbol{\Lambda}_{AA} - \boldsymbol{\Lambda}_{AB} \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA}) \boldsymbol{\Delta}_A \right] \right) \\
&= \frac{1}{Z'} \exp\left( -\frac{1}{2} \left[ \boldsymbol{\Delta}_A^\top \boldsymbol{\Sigma}_{AA}^{-1} \boldsymbol{\Delta}_A \right] \right). && \text{using the second hint}
\end{aligned}$$

2. We obtain

$$\begin{aligned}
&p(\boldsymbol{x}_B \mid \boldsymbol{x}_A) \\
&= \frac{p(\boldsymbol{x}_A, \boldsymbol{x}_B)}{p(\boldsymbol{x}_A)} && \text{using the definition of conditional distributions (1.10)} \\
&= \frac{1}{Z'} \exp\left( -\frac{1}{2} \begin{bmatrix} \boldsymbol{\Delta}_A \\ \boldsymbol{\Delta}_B \end{bmatrix}^\top \begin{bmatrix} \boldsymbol{\Lambda}_{AA} & \boldsymbol{\Lambda}_{AB} \\ \boldsymbol{\Lambda}_{BA} & \boldsymbol{\Lambda}_{BB} \end{bmatrix} \begin{bmatrix} \boldsymbol{\Delta}_A \\ \boldsymbol{\Delta}_B \end{bmatrix} \right) && \text{noting that } p(\boldsymbol{x}_A) \text{ is constant with respect to } \boldsymbol{x}_B \\
&= \frac{1}{Z'} \exp\left( -\frac{1}{2} \left[ \boldsymbol{\Delta}_A^\top (\boldsymbol{\Lambda}_{AA} - \boldsymbol{\Lambda}_{AB} \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA}) \boldsymbol{\Delta}_A \right] \right) && \text{using the first hint} \\
&\quad \cdot \exp\left( -\frac{1}{2} \left[ (\boldsymbol{\Delta}_B + \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} \boldsymbol{\Delta}_A)^\top \boldsymbol{\Lambda}_{BB} (\boldsymbol{\Delta}_B + \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} \boldsymbol{\Delta}_A) \right] \right) \\
&= \frac{1}{Z''} \exp\left( -\frac{1}{2} \left[ (\boldsymbol{\Delta}_B + \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} \boldsymbol{\Delta}_A)^\top \boldsymbol{\Lambda}_{BB} (\boldsymbol{\Delta}_B + \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} \boldsymbol{\Delta}_A) \right] \right). && \text{observing that the first exponential is constant with respect to } \boldsymbol{x}_B
\end{aligned}$$

Finally, observe that

$$\begin{aligned}
\boldsymbol{\Delta}_B + \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} \boldsymbol{\Delta}_A = \boldsymbol{\mu}_B - \boldsymbol{\Lambda}_{BB}^{-1} \boldsymbol{\Lambda}_{BA} (\boldsymbol{x}_A - \boldsymbol{\mu}_A) = \boldsymbol{\mu}_{B|A} \quad \text{and} && \text{using the third hint} \\
\boldsymbol{\Lambda}_{BB}^{-1} = \boldsymbol{\Sigma}_{BB} - \boldsymbol{\Sigma}_{BA} \boldsymbol{\Sigma}_{AA}^{-1} \boldsymbol{\Sigma}_{AB} = \boldsymbol{\Sigma}_{B|A}. && \text{using the second hint}
\end{aligned}$$

Thus, $\boldsymbol{x}_B \mid \boldsymbol{x}_A \sim \mathcal{N}(\boldsymbol{\mu}_{B|A}, \boldsymbol{\Sigma}_{B|A})$.

**Solution to Problem 1.10.**

1. Let $\mathbf{Y} \doteq \boldsymbol{A}\mathbf{X} + \boldsymbol{b}$ and define $\boldsymbol{s} \doteq \boldsymbol{A}^\top \boldsymbol{t}$. We have for the MGF of $\mathbf{Y}$ that

$$\begin{aligned}
\varphi_{\mathbf{Y}}(\boldsymbol{t}) &= \mathbb{E} \exp\left(\boldsymbol{t}^\top \mathbf{Y}\right) \\
&= \mathbb{E} \exp\left(\boldsymbol{t}^\top \boldsymbol{A}\mathbf{X}\right) \cdot \exp\left(\boldsymbol{t}^\top \boldsymbol{b}\right) \\
&= \mathbb{E} \exp\left(\boldsymbol{s}^\top \mathbf{X}\right) \cdot \exp\left(\boldsymbol{t}^\top \boldsymbol{b}\right) \\
&= \varphi_{\mathbf{X}}(\boldsymbol{s}) \cdot \exp\left(\boldsymbol{t}^\top \boldsymbol{b}\right) \\
&= \exp\left(\boldsymbol{s}^\top \boldsymbol{\mu} + \frac{1}{2} \boldsymbol{s}^\top \boldsymbol{\Sigma} \boldsymbol{s}\right) \cdot \exp\left(\boldsymbol{t}^\top \boldsymbol{b}\right) \\
&= \exp\left(\boldsymbol{t}^\top (\boldsymbol{A}\boldsymbol{\mu} + \boldsymbol{b}) + \frac{1}{2} \boldsymbol{t}^\top \boldsymbol{A} \boldsymbol{\Sigma} \boldsymbol{A}^\top \boldsymbol{t}\right),
\end{aligned}$$

   which implies that $\mathbf{Y} \sim \mathcal{N}(\boldsymbol{A}\boldsymbol{\mu} + \boldsymbol{b}, \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\top)$.

2. We have for the MGF of $\mathbf{Y} \doteq \mathbf{X} + \mathbf{X}'$ that

$$\begin{aligned}
\varphi_{\mathbf{Y}}(\boldsymbol{t}) &= \mathbb{E} \exp\left(\boldsymbol{t}^\top \mathbf{Y}\right) \\
&= \mathbb{E} \exp\left(\boldsymbol{t}^\top \mathbf{X}\right) \cdot \mathbb{E} \exp\left(\boldsymbol{t}^\top \mathbf{X}'\right) && \text{using the independence of } \mathbf{X} \text{ and } \mathbf{X}' \\
&= \varphi_{\mathbf{X}}(\boldsymbol{t}) \cdot \varphi_{\mathbf{X}'}(\boldsymbol{t}) \\
&= \exp\left(\boldsymbol{t}^\top \boldsymbol{\mu} + \frac{1}{2} \boldsymbol{t}^\top \boldsymbol{\Sigma} \boldsymbol{t}\right) \cdot \exp\left(\boldsymbol{t}^\top \boldsymbol{\mu}' + \frac{1}{2} \boldsymbol{t}^\top \boldsymbol{\Sigma}' \boldsymbol{t}\right) \\
&= \exp\left(\boldsymbol{t}^\top (\boldsymbol{\mu} + \boldsymbol{\mu}') + \frac{1}{2} \boldsymbol{t}^\top (\boldsymbol{\Sigma} + \boldsymbol{\Sigma}') \boldsymbol{t}\right),
\end{aligned}$$

   which implies that $\mathbf{Y} \sim \mathcal{N}(\boldsymbol{\mu} + \boldsymbol{\mu}', \boldsymbol{\Sigma} + \boldsymbol{\Sigma}')$.

**Solution to Problem 1.11.** Note that if $Y \sim \mathcal{N}(0,1)$ is a (univariate) standard normal random variable, then

$$\begin{aligned}
\mathbb{E}[Y] &= \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} y \cdot \exp\left(-\frac{1}{2} y^2\right) dy && \text{using the PDF of the standard normal distribution (1.5)} \\
&= \frac{1}{\sqrt{2\pi}} \left( \int_{-\infty}^{0} y \cdot \exp\left(-\frac{1}{2} y^2\right) dy + \int_{0}^{\infty} y \cdot \exp\left(-\frac{1}{2} y^2\right) dy \right) \\
&= \frac{1}{\sqrt{2\pi}} \left( \left[ -\exp\left(-\frac{1}{2} y^2\right) \right]_{-\infty}^{0} + \left[ -\exp\left(-\frac{1}{2} y^2\right) \right]_{0}^{\infty} \right) \\
&= \frac{1}{\sqrt{2\pi}} ([-1+0] + [0+1]) = 0, \\
\mathrm{Var}[Y] &= \mathbb{E}\left[Y^2\right] - \underbrace{\mathbb{E}[Y]^2}_{0} && \text{using the definition of variance (1.35)} \\
&= \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} y^2 \cdot \exp\left(-\frac{1}{2} y^2\right) dy && \text{using the PDF of the standard normal distribution (1.5)} \\
&= \frac{1}{\sqrt{2\pi}} \left( \int_{-\infty}^{0} y^2 \cdot \exp\left(-\frac{1}{2} y^2\right) dy + \int_{0}^{\infty} y^2 \cdot \exp\left(-\frac{1}{2} y^2\right) dy \right)
\end{aligned}$$

$$
\begin{aligned}
&= \frac{1}{\sqrt{2\pi}} \left( \left[ -y \cdot \exp\left(-\frac{1}{2}y^2\right) \right]_{-\infty}^{0} + \int_{-\infty}^{0} \exp\left(-\frac{1}{2}y^2\right) dy \right. && \text{integrating by parts} \\
&\qquad \left. + \left[ -y \cdot \exp\left(-\frac{1}{2}y^2\right) \right]_{0}^{\infty} + \int_{0}^{\infty} \exp\left(-\frac{1}{2}y^2\right) dy \right) \\
&= \frac{1}{\sqrt{2\pi}} \left( \int_{-\infty}^{0} \exp\left(-\frac{1}{2}y^2\right) dy + \int_{0}^{\infty} \exp\left(-\frac{1}{2}y^2\right) dy \right) \\
&= \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} \exp\left(-\frac{1}{2}y^2\right) dy = 1. && \text{a PDF integrates to 1}
\end{aligned}
$$

Recall from Equation (1.54) that we can express $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ as

$$\mathbf{X} = \boldsymbol{\Sigma}^{1/2}\mathbf{Y} + \boldsymbol{\mu}$$

where $\mathbf{Y} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$. Using that $\mathbf{Y}$ is a vector of independent univariate standard normal random variables, we conclude that $\mathbb{E}[\mathbf{Y}] = \mathbf{0}$ and $\mathrm{Var}[\mathbf{Y}] = \boldsymbol{I}$. We obtain

$$
\begin{aligned}
\mathbb{E}[\mathbf{X}] &= \mathbb{E}\left[\boldsymbol{\Sigma}^{1/2}\mathbf{Y} + \boldsymbol{\mu}\right] = \boldsymbol{\Sigma}^{1/2} \underbrace{\mathbb{E}[\mathbf{Y}]}_{\mathbf{0}} + \boldsymbol{\mu} = \boldsymbol{\mu}, \quad \text{and} && \text{using linearity of expectation (1.20)} \\
\mathrm{Var}[\mathbf{X}] &= \mathrm{Var}\left[\boldsymbol{\Sigma}^{1/2}\mathbf{Y} + \boldsymbol{\mu}\right] = \boldsymbol{\Sigma}^{1/2} \underbrace{\mathrm{Var}[\mathbf{Y}]}_{\boldsymbol{I}} \boldsymbol{\Sigma}^{1/2\top} = \boldsymbol{\Sigma}. && \text{using Equation (1.38)}
\end{aligned}
$$

**Solution to Problem 1.12.**

1. **Yes**. In the following, we give one example of a non-affine transformation that preserves the Gaussianity of a random vector.
   Let $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ be a Gaussian random vector in $\mathbb{R}^d$. Define the coordinate-wise transformation $\boldsymbol{\phi} : \mathbb{R}^d \to \mathbb{R}^d$ as $(\boldsymbol{\phi}(\boldsymbol{x}))_i = \phi_i(x_i)$ with

   $$\phi_i(x_i) \doteq \begin{cases} -(x_i - \mu_i) + \mu_i & |x_i - \mu_i| < 1 \\ x_i & \text{otherwise.} \end{cases}$$

   Intuitively, $\phi_i$ simply flips all the "mass" in a neighborhood of $\mu_i$ on the $i$-th coordinate. Due to the symmetry of the Gaussian distribution around its mean, you could easily imagine that the transformation still preserves the Gaussian property. Also, this function cannot be an affine transformation since it is not continuous.
   To prove Gaussianity, we can use the change of variables formula (1.43). Let

   $$\mathbf{Y} \doteq \boldsymbol{\phi}(\mathbf{X}), \quad p_{\mathbf{Y}}(\boldsymbol{y}) = p_{\mathbf{X}}(\boldsymbol{\phi}^{-1}(\boldsymbol{y})) \cdot \left|\det\left(\boldsymbol{D}\boldsymbol{\phi}^{-1}(\boldsymbol{y})\right)\right| \quad (\forall \boldsymbol{y} \in \mathbb{R}^d).$$

   There are two cases for $\boldsymbol{y}$:
   (a) If $|y_i - \mu_i| < 1$, then $x_i = \phi_i^{-1}(y_i) = -(y_i - \mu_i) + \mu_i$. Also note that in this case, the Jacobian matrix is simply a diagonal

matrix with $-1$ in the $i$-th position and 1 elsewhere. Hence, its determinant is $-1$. We have

$$\begin{aligned}
p_{\mathbf{Y}}(\boldsymbol{y}) &= p_{\mathbf{X}}(\boldsymbol{\phi}^{-1}(\boldsymbol{y})) \cdot \left|\det\left(\boldsymbol{D}\boldsymbol{\phi}^{-1}(\boldsymbol{y})\right)\right| \\
&= p_{\mathbf{X}}(\boldsymbol{\phi}^{-1}(\boldsymbol{y})) \\
&= p_{\mathbf{X}}\left(\begin{bmatrix} \vdots \\ -(y_i - \mu_i) + \mu_i \\ \vdots \end{bmatrix}\right) \\
&= p_{\mathbf{X}}\left(\begin{bmatrix} \vdots \\ y_i \\ \vdots \end{bmatrix}\right) && \text{since } p_{\mathbf{X}} \text{ is symmetric w.r.t. } \mu_i \\
&= p_{\mathbf{X}}(\boldsymbol{y}).
\end{aligned}$$

(b) If $|y_i - \mu_i| \geq 1$, then $p_{\mathbf{Y}}(\boldsymbol{y}) = p_{\mathbf{X}}(\boldsymbol{y})$ since $\boldsymbol{\phi}$ is the identity function in this case.

Thus, $\mathbf{Y}$ has an identical PDF to $\mathbf{X}$ and is therefore Gaussian.

2. **Yes**. It is difficult to calculate the distribution of $W \doteq \frac{X+YZ}{\sqrt{1+Z^2}}$ directly since $Z$ is in the denominator. The trick here is to first condition $W$ on $Z$, and then integrate over the distribution of $Z$. For all $z \in \mathbb{R}$, we have

$$\begin{aligned}
p(W \mid Z = z) &= p\left(\frac{X + YZ}{\sqrt{1+Z^2}} \middle| Z = z\right) \\
&= p\left(\frac{X + Yz}{\sqrt{1+z^2}}\right) && \text{since } X, Y \perp Z \\
&= \mathcal{N}(W; 0, 1) && \text{as } X + Yz \sim \mathcal{N}(0, 1+z^2)
\end{aligned}$$

which means that $W \mid Z = z \sim \mathcal{N}(0,1)$ which is *independent* of $z$! Using the law of total probability (1.12), we can now write

$$\begin{aligned}
p(W) &= \int p(W \mid Z = z) \cdot p_Z(z)\, dz \\
&= \mathcal{N}(W; 0, 1) \int p_Z(z)\, dz \\
&= \mathcal{N}(W; 0, 1).
\end{aligned}$$

**Solution to Problem 1.13.**

1. We have

$$\mathbb{E}_y[(y-a)^2 \mid \boldsymbol{x}] = a^2 - 2a\mathbb{E}_y[y \mid \boldsymbol{x}] + \text{const},$$

which is easily seen to be minimized when $a = \mathbb{E}_y[y \mid \boldsymbol{x}]$.

2. We have

$$\mathbb{E}_y[r(y,a) \mid \boldsymbol{x}] = c_1 \int_a^\infty (y-a) p(y \mid \boldsymbol{x})\, dy + c_2 \int_{-\infty}^a (a-y) p(y \mid \boldsymbol{x})\, dy.$$

We can differentiate this expression with respect to $a$:

$$\begin{aligned}\frac{d}{da}\mathbb{E}_y[r(y,a) \mid \boldsymbol{x}] &= -c_1 \int_a^\infty p(y \mid \boldsymbol{x})\, dy + c_2 \int_{-\infty}^a p(y \mid \boldsymbol{x})\, dy \\ &= -c_1 \mathbb{P}(y \geq a \mid \boldsymbol{x}) + c_2 \mathbb{P}(y \leq a \mid \boldsymbol{x}) \\ &= -c_1(1 - \mathbb{P}(y \leq a \mid \boldsymbol{x})) + c_2 \mathbb{P}(y \leq a \mid \boldsymbol{x}) \\ &= -c_1 + (c_1 + c_2)\mathbb{P}(y \leq a \mid \boldsymbol{x}).\end{aligned}$$

Setting this equal to zero, we find the critical point condition

$$\mathbb{P}(y \leq a \mid \boldsymbol{x}) \overset{!}{=} \frac{c_1}{c_1 + c_2}.$$

We obtain $a^\star(\boldsymbol{x}) = \mu_{\boldsymbol{x}} + \sigma_{\boldsymbol{x}} \cdot \Phi^{-1}\left(\frac{c_1}{c_1+c_2}\right)$ by transforming to a standard normal random variable.

## *Linear Regression*

**Solution to Problem 2.1.**

1. We begin by deriving the gradient and Hessian of the least squares and ridge losses. For least squares,

$$\begin{aligned}\nabla_{\boldsymbol{w}} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2 &= \nabla_{\boldsymbol{w}} \left(\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} - 2\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{y} + \|\boldsymbol{y}\|_2^2\right) \\ &= 2(\boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} - \boldsymbol{X}^\top \boldsymbol{y}), \\ \boldsymbol{H}_{\boldsymbol{w}} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2 &= 2\boldsymbol{X}^\top \boldsymbol{X}.\end{aligned}$$

   Similarly, for ridge regression,

$$\begin{aligned}\nabla_{\boldsymbol{w}} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2 + \lambda \|\boldsymbol{w}\|_2^2 &= 2(\boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} - \boldsymbol{X}^\top \boldsymbol{y}) + 2\lambda \boldsymbol{w}, \\ \boldsymbol{H}_{\boldsymbol{w}} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2 + \lambda \|\boldsymbol{w}\|_2^2 &= 2\boldsymbol{X}^\top \boldsymbol{X} + 2\lambda \boldsymbol{I}.\end{aligned}$$

   From the assumption that the Hessians are positive definite, we know that any minimizer is a unique globally optimal solution (due to strict convexity), and that $\boldsymbol{X}^\top \boldsymbol{X}$ and $\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I}$ are invertible.

   Using the first-order optimality condition for convex functions, we attain the solutions to least squares and ridge regression by setting the respective gradient to $\boldsymbol{0}$.

2. We choose $\hat{\boldsymbol{w}}_{\text{ls}}$ such that $\boldsymbol{X}\hat{\boldsymbol{w}}_{\text{ls}} = \boldsymbol{\Pi}_{\boldsymbol{X}}\boldsymbol{y}$. This implies that $\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}}_{\text{ls}} \perp \boldsymbol{X}\boldsymbol{w}$ for all $\boldsymbol{w} \in \mathbb{R}^d$. In other words, $(\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}}_{\text{ls}})^\top \boldsymbol{X}\boldsymbol{w} = 0$ for all $\boldsymbol{w}$, which implies that $(\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}}_{\text{ls}})^\top \boldsymbol{X} = \boldsymbol{0}$. By simple algebraic manipulation it can be seen that this condition is equivalent to the gradient condition of the previous problem.

**Solution to Problem 2.2.** To prevent confusion with the length of the data set $n$, we drop the subscript from the noise variance $\sigma_{\text{n}}^2$ in the

following. We have

$$\begin{aligned}\hat{\sigma} &= \arg\max_{\sigma} \sum_{i=1}^{n} \log p(y_i \mid \boldsymbol{x}_i, \sigma) \\ &= \arg\min_{\sigma} \sum_{i=1}^{n} \frac{1}{2}\log(2\sigma^2\pi) + \frac{1}{2\sigma^2}(y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 \\ &= \arg\min_{\sigma} \frac{n}{2}\log\sigma^2 + \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2.\end{aligned}$$

We can solve this optimization problem by differentiating and setting to zero:

$$\begin{aligned}\frac{\partial}{\partial\sigma^2}\left[\frac{n}{2}\log\sigma^2 + \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2\right] &= 0 \\ \frac{n}{2\sigma^2} - \frac{1}{2\sigma^4}\sum_{i=1}^{n}(y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 &= 0 \\ n - \frac{1}{\sigma^2}\sum_{i=1}^{n}(y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 &= 0. \qquad \text{multiplying through by } 2\sigma^2\end{aligned}$$

The desired result follows by solving for $\sigma^2$.

**Solution to Problem 2.3.** Let us first derive the variance of the least squares estimate:

$$\begin{aligned}\mathrm{Var}[\hat{\boldsymbol{w}}_{\mathrm{ls}} \mid \boldsymbol{X}] &= \mathrm{Var}\left[(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y} \;\middle|\; \boldsymbol{X}\right] && \text{using Equation (2.4)} \\ &= (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\mathrm{Var}[\boldsymbol{y} \mid \boldsymbol{X}]\,((\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top)^\top && \text{using Equation (1.38)} \\ &= (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\mathrm{Var}[\boldsymbol{y} \mid \boldsymbol{X}]\,\boldsymbol{X}(\boldsymbol{X}^\top\boldsymbol{X})^{-1}. \qquad \text{(B.1)} && \text{using } (\boldsymbol{A}^\top)^{-1} = (\boldsymbol{A}^{-1})^\top\end{aligned}$$

Due to the Gaussian likelihood (2.7), $\mathrm{Var}[\boldsymbol{y} \mid \boldsymbol{X}] = \sigma_{\mathrm{n}}^2\boldsymbol{I}$, so

$$= \sigma_{\mathrm{n}}^2(\boldsymbol{X}^\top\boldsymbol{X})^{-1}. \qquad \text{(B.2)}$$

In the two-dimensional setting, i.e., data is of the form $\boldsymbol{x}_i = [1\ x_i]^\top$ ($x_i \in \mathbb{R}$), we have

$$\boldsymbol{X}^\top\boldsymbol{X} = \begin{bmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix}.$$

Thus,

$$\mathrm{Var}[\hat{\boldsymbol{w}}_{\mathrm{ls}} \mid \boldsymbol{X}] = \sigma_{\mathrm{n}}^2(\boldsymbol{X}^\top\boldsymbol{X})^{-1} = \frac{\sigma_{\mathrm{n}}^2}{Z}\begin{bmatrix} \sum x_i^2 & -\sum x_i \\ -\sum x_i & n \end{bmatrix} \qquad \text{using Equation (B.2)}$$

where $Z \doteq n(\sum x_i^2) - (\sum x_i)^2$.

Therefore, the predictive variance at a point $[1\ x^\star]^\top$ is

$$\begin{bmatrix}1 & x^\star\end{bmatrix}\mathrm{Var}[\hat{\boldsymbol{w}}_{\mathrm{ls}}]\begin{bmatrix}1 \\ x^\star\end{bmatrix} = \frac{\sigma_{\mathrm{n}}^2}{Z}\sum x_i^2 - 2x_ix^\star + (x^\star)^2 = \frac{\sigma_{\mathrm{n}}^2}{Z}\sum(x_i - x^\star)^2.$$

Thus, the predictive variance is minimized for $x^\star = \frac{1}{n}\sum x_i$.

**Solution to Problem 2.4.**

1. Recall from Section 2.0.1 that the MLE and least squares estimate coincide if the noise is zero-mean Gaussian. We therefore have,

$$\hat{\boldsymbol{w}}_{\text{MLE}} = \hat{\boldsymbol{w}}_{\text{ls}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y} = \begin{bmatrix} 0.63 \\ 1.83 \end{bmatrix}.$$

2. Recall from Equation (2.10c) that $\boldsymbol{w} \mid \boldsymbol{X}, \boldsymbol{y} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with

$$\boldsymbol{\Sigma} = \left(\sigma_{\text{n}}^{-2} \boldsymbol{X}^\top \boldsymbol{X} + \sigma_{\text{p}}^{-2} \boldsymbol{I}\right)^{-1} \quad \text{and}$$
$$\boldsymbol{\mu} = \sigma_{\text{n}}^{-2} \boldsymbol{\Sigma} \boldsymbol{X}^\top \boldsymbol{y}.$$

   Inserting $\sigma_{\text{n}}^2 = 0.1$ and $\sigma_{\text{p}}^2 = 0.05$ yields,

$$\boldsymbol{\Sigma} = \begin{bmatrix} 0.019 & -0.014 \\ -0.014 & 0.019 \end{bmatrix}.$$

   Then,

$$\hat{\boldsymbol{w}}_{\text{MAP}} = \boldsymbol{\mu} = \begin{bmatrix} 0.91 \\ 1.31 \end{bmatrix}.$$

3. Recall from Equation (2.16) that $y^\star \mid \boldsymbol{X}, \boldsymbol{y}, \boldsymbol{x}^\star \sim \mathcal{N}(\mu_y^\star, \sigma_{y^\star}^2)$ with

$$\mu_y^\star = \boldsymbol{\mu}^\top \boldsymbol{x}^\star \quad \text{and} \quad \sigma_{y^\star}^2 = \boldsymbol{x}^{\star\top} \boldsymbol{\Sigma} \boldsymbol{x}^\star + \sigma_{\text{n}}^2.$$

   Inserting $\boldsymbol{x}^\star = [3\ 3]^\top$, and $\sigma_{\text{n}}^2, \boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ from above yields,

$$\mu_y^\star = 6.66 \quad \text{and} \quad \sigma_{y^\star}^2 = 0.186.$$

4. One would have to let $\sigma_{\text{p}}^2 \to \infty$.

**Solution to Problem 2.5.** We denote by $\boldsymbol{X}_t$ the design matrix and by $\boldsymbol{y}_t$ the vector of observations including the first $t$ data points.

1. Note that

$$\boldsymbol{X}_t^\top \boldsymbol{X}_t = \sum_{i=1}^{t} \boldsymbol{x}_i \boldsymbol{x}_i^\top \quad \text{and} \quad \boldsymbol{X}_t^\top \boldsymbol{y}_t = \sum_{i=1}^{t} y_i \boldsymbol{x}_i.$$

   This means that after observing the $(t+1)$-st data point, we have that

$$\boldsymbol{X}_{t+1}^\top \boldsymbol{X}_{t+1} = \boldsymbol{X}_t^\top \boldsymbol{X}_t + \boldsymbol{x}_{t+1} \boldsymbol{x}_{t+1}^\top \quad \text{and}$$
$$\boldsymbol{X}_{t+1}^\top \boldsymbol{y}_{t+1} = \boldsymbol{X}_t^\top \boldsymbol{y}_t + y_{t+1} \boldsymbol{x}_{t+1}.$$

   Hence, by just keeping $\boldsymbol{X}_t^\top \boldsymbol{X}_t$ (which is a $d \times d$ matrix) and $\boldsymbol{X}_t^\top \boldsymbol{y}_t$ (which is a vector in $\mathbb{R}^d$) in memory, and updating them as above, we do not need to keep the whole data in memory.

2. One has to compute $(\sigma_\mathrm{n}^{-2}\boldsymbol{X}_t^\top \boldsymbol{X}_t + \sigma_\mathrm{p}^{-2}\boldsymbol{I})^{-1}$ for finding $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ in every round. We can write

$$\begin{aligned}(\sigma_\mathrm{n}^{-2}\boldsymbol{X}_{t+1}^\top \boldsymbol{X}_{t+1} + \sigma_\mathrm{p}^{-2}\boldsymbol{I})^{-1} &= \sigma_\mathrm{n}^2(\boldsymbol{X}_{t+1}^\top \boldsymbol{X}_{t+1} + \sigma_\mathrm{n}^2\sigma_\mathrm{p}^{-2}\boldsymbol{I})^{-1} \\ &= \sigma_\mathrm{n}^2(\underbrace{\boldsymbol{X}_t^\top \boldsymbol{X}_t + \sigma_\mathrm{n}^2\sigma_\mathrm{p}^{-2}\boldsymbol{I}}_{\boldsymbol{A}_t} + \boldsymbol{x}_{t+1}\boldsymbol{x}_{t+1}^\top)^{-1}\end{aligned}$$

where $\boldsymbol{A}_t \in \mathbb{R}^{d\times d}$. Using the Woodbury matrix identity (A.67) and that we know the inverse of $\boldsymbol{A}_t$ (from the previous iteration), the computation of the inverse of $(\boldsymbol{A}_t + \boldsymbol{x}_{t+1}\boldsymbol{x}_{t+1}^\top)$ is of $O(d^2)$, which is much better than computing the inverse of $(\boldsymbol{A}_t + \boldsymbol{x}_{t+1}\boldsymbol{x}_{t+1}^\top)$ from scratch.

**Solution to Problem 2.6.** The law of total variance (2.18) yields the following decomposition of the predictive variance,

$$\begin{aligned}\mathrm{Var}_{y^\star}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, y_{1:n}] = \mathbb{E}_{\boldsymbol{w}}\left[\mathrm{Var}_{y^\star}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{w}] \mid \boldsymbol{x}_{1:n}, y_{1:n}\right] \\ + \mathrm{Var}_{\boldsymbol{w}}\left[\mathbb{E}_{y^\star}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{w}] \mid \boldsymbol{x}_{1:n}, y_{1:n}\right]\end{aligned}$$

wherein the first term corresponds to the aleatoric uncertainty and the second term corresponds to the epistemic uncertainty.

The aleatoric uncertainty is given by

$$\mathbb{E}_{\boldsymbol{w}}\left[\mathrm{Var}_{y^\star}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{w}] \mid \boldsymbol{x}_{1:n}, y_{1:n}\right] = \mathbb{E}_{\boldsymbol{w}}\left[\sigma_\mathrm{n}^2 \,\middle|\, \boldsymbol{x}_{1:n}, y_{1:n}\right] = \sigma_\mathrm{n}^2$$

using the definition of $\sigma_\mathrm{n}^2$. (2.7)

For the epistemic uncertainty,

$$\begin{aligned}\mathrm{Var}_{\boldsymbol{w}}\left[\mathbb{E}_{y^\star}[y^\star \mid \boldsymbol{x}^\star, \boldsymbol{w}] \mid \boldsymbol{x}_{1:n}, y_{1:n}\right] &= \mathrm{Var}_{\boldsymbol{w}}\left[\boldsymbol{w}^\top \boldsymbol{x}^\star \,\middle|\, \boldsymbol{x}_{1:n}, y_{1:n}\right] && \text{using that } y^\star = \boldsymbol{w}^\top \boldsymbol{x}^\star + \varepsilon \text{ where } \varepsilon \text{ is zero-mean noise} \\ &= \boldsymbol{x}^{\star\top} \mathrm{Var}_{\boldsymbol{w}}[\boldsymbol{w} \mid \boldsymbol{x}_{1:n}, y_{1:n}]\, \boldsymbol{x}^\star && \text{using Equation (1.38)} \\ &= \boldsymbol{x}^{\star\top} \boldsymbol{\Sigma} \boldsymbol{x}^\star && \text{using Equation (2.10c)}\end{aligned}$$

where $\boldsymbol{\Sigma}$ is the posterior covariance matrix.

**Solution to Problem 2.7.**

1. Let $\boldsymbol{f} \doteq \boldsymbol{X}\boldsymbol{w}$. Since $\boldsymbol{w} \mid \boldsymbol{\mu}, \lambda \sim \mathcal{N}(\boldsymbol{\mu}, \lambda^{-1}\boldsymbol{I}_d)$, we have

$$\boldsymbol{f} \mid \boldsymbol{X}, \boldsymbol{\mu}, \lambda \sim \mathcal{N}(\boldsymbol{X}\boldsymbol{\mu}, \lambda^{-1}\boldsymbol{X}\boldsymbol{X}^\top).$$

using Equation (1.38)

Thus,

$$\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\mu}, \lambda \sim \mathcal{N}(\boldsymbol{X}\boldsymbol{\mu}, \lambda^{-1}\boldsymbol{X}\boldsymbol{X}^\top + \lambda^{-1}\boldsymbol{I}_n).$$

2. We will denote the MLE by $\widehat{\boldsymbol{\mu}}_1$. We have

$$\widehat{\boldsymbol{\mu}}_1 = \arg\max_{\boldsymbol{\mu}} p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\mu}, \lambda) = \arg\max_{\boldsymbol{\mu}} \log \mathcal{N}(\boldsymbol{y}; \boldsymbol{X}\boldsymbol{\mu}, \boldsymbol{\Sigma}_{\boldsymbol{y}}).$$

We can simplify

$$\begin{aligned}-\log \mathcal{N}(\boldsymbol{y}; \boldsymbol{X}\boldsymbol{\mu}, \boldsymbol{\Sigma}_y) &= \frac{1}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})^\top \boldsymbol{\Sigma}_y^{-1}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}) + \text{const} \\ &= \frac{1}{2}\boldsymbol{\mu}^\top \boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{X}\boldsymbol{\mu} - \boldsymbol{\mu}^\top \boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{y} + \text{const}.\end{aligned}$$

Taking the gradient with respect to $\boldsymbol{\mu}$, we obtain

$$\boldsymbol{\nabla}_\mu \left(\frac{1}{2}\boldsymbol{\mu}^\top \boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{X}\boldsymbol{\mu} - \boldsymbol{y}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{X}\boldsymbol{\mu} + \text{const}\right) = \boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{X}\boldsymbol{\mu} - \boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{y}.$$

This is zero iff $\boldsymbol{\mu} = (\boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{X})^{-1}\boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{y}$.

3. By Bayes' rule (1.45),

$$p(\boldsymbol{\mu} \mid \boldsymbol{X}, \boldsymbol{y}, \lambda) \propto p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\mu}, \lambda) \cdot p(\boldsymbol{\mu}).$$

Taking the negative logarithm,

$$-\log p(\boldsymbol{\mu} \mid \boldsymbol{X}, \boldsymbol{y}, \lambda) = -\log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\mu}, \lambda) - \log p(\boldsymbol{\mu}) + \text{const}.$$

Analogously to the previous question, we can simplify

$$\begin{aligned}&= \frac{1}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})^\top \boldsymbol{\Sigma}_y^{-1}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}) + \frac{1}{2}\boldsymbol{\mu}^\top \boldsymbol{\mu} + \text{const} \\ &= \frac{1}{2}\boldsymbol{\mu}^\top (\boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{X} + \boldsymbol{I}_d)\boldsymbol{\mu} - \boldsymbol{\mu}^\top \boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{y} + \text{const}.\end{aligned}$$

This is a quadratic in $\boldsymbol{\mu}$, so the posterior distribution must be Gaussian. By matching the terms with

$$\log \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}', \boldsymbol{\Sigma}') = -\frac{1}{2}\boldsymbol{x}^\top \boldsymbol{\Sigma}'^{-1}\boldsymbol{x} + \boldsymbol{x}^\top \boldsymbol{\Sigma}'^{-1}\boldsymbol{\mu}' + \text{const},$$

we obtain the covariance matrix $(\boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{X} + \boldsymbol{I}_d)^{-1}$ and the mean vector $\boldsymbol{\Sigma}_\mu \boldsymbol{X}^\top \boldsymbol{\Sigma}_y^{-1}\boldsymbol{y}$.

4. Let $\widetilde{\boldsymbol{\Sigma}}_y \doteq \lambda \boldsymbol{\Sigma}_y$. By Bayes' rule (1.45),

$$\begin{aligned}p(\lambda \mid \boldsymbol{X}, \boldsymbol{y}, \boldsymbol{\mu}) &\propto p(\lambda) \cdot p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\mu}, \lambda) \\ &= e^{-\lambda} \cdot \mathcal{N}(\boldsymbol{y}; \boldsymbol{X}\boldsymbol{\mu}, \boldsymbol{\Sigma}_y) \\ &\propto e^{-\lambda} \cdot \det(\boldsymbol{\Sigma}_y)^{-1/2} \exp\left(-\frac{1}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})^\top \boldsymbol{\Sigma}_y^{-1}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})\right) \\ &= e^{-\lambda} \cdot \det\left(\lambda^{-1}\widetilde{\boldsymbol{\Sigma}}_y\right)^{-1/2} \exp\left(-\frac{1}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})^\top (\lambda^{-1}\widetilde{\boldsymbol{\Sigma}}_y)^{-1}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})\right) \\ &\propto e^{-\lambda} \cdot \lambda^{n/2} \exp\left(-\frac{\lambda}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})^\top \widetilde{\boldsymbol{\Sigma}}_y^{-1}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})\right) \qquad \text{using that } \lambda^{-1}\widetilde{\boldsymbol{\Sigma}}_y \text{ is independent of } \lambda \\ &= \lambda^{n/2} \exp\left(-\lambda\left[1 + \frac{1}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})^\top \widetilde{\boldsymbol{\Sigma}}_y^{-1}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})\right]\right) \\ &\propto \text{Gamma}(\alpha, \beta)\end{aligned}$$

with $\alpha = 1 + \frac{n}{2}$ and $\beta = 1 + \frac{1}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})^\top \widetilde{\boldsymbol{\Sigma}}_y^{-1}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu})$.

## *Filtering*

**Solution to Problem 3.1.** Recall from Equation (3.8) that

$$p(x_{t+1} \mid y_{1:t+1}) = \frac{1}{Z} p(x_{t+1} \mid y_{1:t}) p(y_{t+1} \mid x_{t+1}). \tag{B.3}$$

Using the sensor model (3.12b),

$$p(y_{t+1} \mid x_{t+1}) = \frac{1}{Z'} \exp\left( -\frac{1}{2} \frac{(y_{t+1} - x_{t+1})^2}{\sigma_y^2} \right).$$

It remains to compute the predictive distribution,

$$\begin{aligned}
p(x_{t+1} \mid y_{1:t}) &= \int p(x_{t+1} \mid x_t) p(x_t \mid y_{1:t}) \, dx_t && \text{using Equation (3.9)} \\
&= \frac{1}{Z''} \int \exp\left( -\frac{1}{2} \left[ \frac{(x_{t+1} - x_t)^2}{\sigma_x^2} + \frac{(x_t - \mu_t)^2}{\sigma_t^2} \right] \right) dx_t && \text{using the motion model (3.12a) and previous update} \\
&= \frac{1}{Z''} \int \exp\left( -\frac{1}{2} \left[ \frac{\sigma_t^2 (x_{t+1} - x_t)^2 + \sigma_x^2 (x_t - \mu_t)^2}{\sigma_t^2 \sigma_x^2} \right] \right) dx_t.
\end{aligned}$$

The exponent is the sum of two expressions that are quadratic in $x_t$. *Completing the square* allows rewriting any quadratic $ax^2 + bx + c$ as the sum of a squared term $a(x + \frac{b}{2a})^2$ and a residual term $c - \frac{b^2}{4a}$ that is independent of $x$. In this case, we have $a = (\sigma_t^2 + \sigma_x^2)/(\sigma_t^2 \sigma_x^2)$, $b = -2(\sigma_t^2 x_{t+1} + \sigma_x^2 \mu_t)/(\sigma_t^2 \sigma_x^2)$, and $c = (\sigma_t^2 x_{t+1}^2 + \sigma_x^2 \mu_t^2)/(\sigma_t^2 \sigma_x^2)$. The residual term can be taken outside the integral, giving

$$= \frac{1}{Z''} \exp\left( -\frac{1}{2} \left[ c - \frac{b^2}{4a} \right] \right) \int \exp\left( -\frac{a}{2} \left[ x_t + \frac{b}{2a} \right]^2 \right) dx_t.$$

The integral is simply the integral of a Gaussian over its entire support, and thus evaluates to 1. We are therefore left with only the residual term from the quadratic. Plugging back in the expressions for $a$, $b$, and $c$ and simplifying, we obtain

$$= \frac{1}{Z''} \exp\left( -\frac{1}{2} \frac{(x_{t+1} - \mu_t)^2}{\sigma_t^2 + \sigma_x^2} \right).$$

That is, $X_{t+1} \mid y_{1:t} \sim \mathcal{N}(\mu_t, \sigma_t^2 + \sigma_x^2)$.

Plugging our results back into Equation (B.3), we obtain

$$p(x_{t+1} \mid y_{1:t+1}) = \frac{1}{Z'''} \exp\left( -\frac{1}{2} \left[ \frac{(x_{t+1} - \mu_t)^2}{\sigma_t^2 + \sigma_x^2} + \frac{(y_{t+1} - x_{t+1})^2}{\sigma_y^2} \right] \right).$$

Completing the square analogously to our derivation of the predictive distribution yields,

$$= \frac{1}{Z'''} \exp\left( -\frac{1}{2} \frac{\left( x_{t+1} - \frac{(\sigma_t^2 + \sigma_x^2) y_{t+1} + \sigma_y^2 \mu_t}{\sigma_t^2 + \sigma_x^2 + \sigma_y^2} \right)^2}{\frac{(\sigma_t^2 + \sigma_x^2)\sigma_y^2}{\sigma_t^2 + \sigma_x^2 + \sigma_y^2}} \right).$$

Hence, $X_{t+1} \mid y_{t+1} \sim \mathcal{N}(\mu_{t+1}, \sigma_{t+1}^2)$ as defined in Equation (3.13).

**Solution to Problem 3.2.** We prove the equivalence by induction. Note that the base case is satisfied trivially since the priors are identical.

Assume after $t-1$ steps that $(\boldsymbol{\mu}_{t-1}, \boldsymbol{\Sigma}_{t-1})$ coincide with the BLR posterior for the first $t-1$ data points. We will show that the Kalman filter update equations yield the BLR posterior for the first $t$ data points.

*Covariance update:*

$$
\begin{aligned}
\boldsymbol{\Sigma}_t &= \boldsymbol{\Sigma}_{t-1} - \boldsymbol{k}_t \boldsymbol{x}_t^\top \boldsymbol{\Sigma}_{t-1} \\
&= \boldsymbol{\Sigma}_{t-1} - \frac{(\boldsymbol{\Sigma}_{t-1}\boldsymbol{x}_t)(\boldsymbol{\Sigma}_{t-1}\boldsymbol{x}_t)^\top}{\boldsymbol{x}_t^\top \boldsymbol{\Sigma}_{t-1}\boldsymbol{x}_t + 1} && \text{using the symmetry of } \boldsymbol{\Sigma}_{t-1} \\
&= \left(\boldsymbol{\Sigma}_{t-1}^{-1} + \boldsymbol{x}_t \boldsymbol{x}_t^\top\right)^{-1} && \text{using the Sherman-Morrison formula (A.67) with } \boldsymbol{A}^{-1} = \boldsymbol{\Sigma}_{t-1} \\
&= \left(\boldsymbol{X}_{1:t-1}^\top \boldsymbol{X}_{1:t-1} + \boldsymbol{x}_t \boldsymbol{x}_t^\top + \boldsymbol{I}\right)^{-1} && \text{by the inductive hypothesis and using Equation (2.10c)} \\
&= \left(\boldsymbol{X}_{1:t}^\top \boldsymbol{X}_{1:t} + \boldsymbol{I}\right)^{-1} \\
&= \boldsymbol{\Sigma}_t^{\text{BLR}}. && \text{using Equation (2.10c)}
\end{aligned}
$$

*Mean update:*

$$
\begin{aligned}
\boldsymbol{\Sigma}_t^{-1}\boldsymbol{\mu}_t &= \boldsymbol{\Sigma}_t^{-1}\boldsymbol{\mu}_{t-1} + \boldsymbol{\Sigma}_t^{-1}\boldsymbol{k}_t(y_t - \boldsymbol{x}_t^\top \boldsymbol{\mu}_{t-1}) \\
&= \boldsymbol{\Sigma}_{t-1}^{-1}\boldsymbol{\mu}_{t-1} + \boldsymbol{x}_t \boldsymbol{x}_t^\top \boldsymbol{\mu}_{t-1} + \boldsymbol{\Sigma}_t^{-1}\boldsymbol{k}_t(y_t - \boldsymbol{x}_t^\top \boldsymbol{\mu}_{t-1}) && \text{using } \boldsymbol{\Sigma}_t^{-1} = \boldsymbol{\Sigma}_{t-1}^{-1} + \boldsymbol{x}_t \boldsymbol{x}_t^\top \\
&= \boldsymbol{\Sigma}_{t-1}^{-1}\boldsymbol{\mu}_{t-1} + \boldsymbol{x}_t \boldsymbol{x}_t^\top \boldsymbol{\mu}_{t-1} + \boldsymbol{x}_t(y_t - \boldsymbol{x}_t^\top \boldsymbol{\mu}_{t-1}) && \text{using } \boldsymbol{\Sigma}_t^{-1}\boldsymbol{k}_t = \boldsymbol{x}_t \\
&= \boldsymbol{\Sigma}_{t-1}^{-1}\boldsymbol{\mu}_{t-1} + \boldsymbol{x}_t y_t && \text{canceling terms} \\
&= \boldsymbol{X}_{1:t-1}^\top \boldsymbol{y}_{1:t-1} + \boldsymbol{x}_t y_t && \text{by the inductive hypothesis and using Equation (2.10c)} \\
&= \boldsymbol{X}_{1:t}^\top \boldsymbol{y}_{1:t} \\
&= \boldsymbol{\Sigma}_t^{-1}\boldsymbol{\Sigma}_t \boldsymbol{X}_{1:t}^\top \boldsymbol{y}_{1:t} \\
&= \boldsymbol{\Sigma}_t^{-1}\boldsymbol{\Sigma}_t^{\text{BLR}} \boldsymbol{X}_{1:t}^\top \boldsymbol{y}_{1:t} && \text{using the covariance update above} \\
&= \boldsymbol{\Sigma}_t^{-1}\boldsymbol{\mu}_t^{\text{BLR}}. && \text{using Equation (2.10c)}
\end{aligned}
$$

This completes the induction.

**Solution to Problem 3.3.**

1. The given parameter estimation problem can be formulated as a Kalman filter in the following way:

$$
\begin{aligned}
x_t &= \pi && \forall t, \\
y_t &= x_t + \eta_t && \eta_t \sim \mathcal{N}(0, \sigma_y^2).
\end{aligned}
$$

Thus, in terms of Kalman filters, this yields $f = h = 1, \varepsilon_t = \sigma_x^2 = 0$.
Using Equation (3.14), the *Kalman gain* is given by

$$k_{t+1} = \frac{\sigma_t^2}{\sigma_t^2 + \sigma_y^2},$$

whereas the *variance of the estimation error* $\sigma_t^2$ satisfies

$$\sigma_{t+1}^2 = \sigma_y^2 k_{t+1} = \frac{\sigma_t^2 \sigma_y^2}{\sigma_t^2 + \sigma_y^2}. \qquad \text{using Equation (3.17)}$$

To get the closed form, observe that

$$\frac{1}{\sigma_{t+1}^2} = \frac{1}{\sigma_t^2} + \frac{1}{\sigma_y^2} = \frac{1}{\sigma_{t-1}^2} + \frac{2}{\sigma_y^2} = \cdots = \frac{1}{\sigma_0^2} + \frac{t+1}{\sigma_y^2},$$

yielding,

$$\sigma_{t+1}^2 = \frac{\sigma_0^2 \sigma_y^2}{(t+1)\sigma_0^2 + \sigma_y^2} \quad \text{and} \quad k_{t+1} = \frac{\sigma_0^2}{(t+1)\sigma_0^2 + \sigma_y^2}.$$

2. When $t \to \infty$, we get $k_{t+1} \to 0$ and $\sigma_{t+1}^2 \to 0$, giving

$$\mu_{t+1} = \mu_t + k_{t+1}(y_{t+1} - \mu_t) = \mu_t, \qquad \text{using Equation (3.16)}$$

thus resulting in a stationary sequence.
3. Observe that $\sigma_0^2 \to \infty$ implies $k_{t+1} = \frac{1}{t+1}$. Therefore,

$$\begin{aligned}
\mu_{t+1} &= \mu_t + \frac{1}{t+1}(y_{t+1} - \mu_t) && \text{using Equation (3.16)} \\
&= \frac{t}{t+1}\mu_t + \frac{y_{t+1}}{t+1} \\
&= \frac{t}{t+1}\left(\frac{t-1}{t}\mu_{t-1} + \frac{y_t}{t}\right) + \frac{y_{t+1}}{t+1} && \text{using Equation (3.16)} \\
&= \frac{t-1}{t+1}\mu_{t-1} + \frac{y_t + y_{t+1}}{t+1} \\
&\vdots \\
&= \frac{y_1 + \cdots + y_{t+1}}{t+1},
\end{aligned}$$

which is simply the *sample mean*.

## *Gaussian Processes*

**Solution to Problem 4.1.**

1. We have for any $j \in \mathbb{N}_0$ and $x, y \in \mathbb{R}$,

$$\frac{1}{\sqrt{j!}} e^{-\frac{x^2}{2}} x^j \cdot \frac{1}{\sqrt{j!}} e^{-\frac{y^2}{2}} y^j = e^{-\frac{x^2+y^2}{2}} \frac{(xy)^j}{j!}.$$

Summing over all $j$, we obtain

$$\begin{aligned}
\boldsymbol{\phi}(x)^\top \boldsymbol{\phi}(y) &= \sum_{j=0}^{\infty} \phi_j(x)\phi_j(y) \\
&= e^{-\frac{x^2+y^2}{2}} \sum_{j=0}^{\infty} \frac{(xy)^j}{j!} \\
&= e^{-\frac{x^2+y^2}{2}} e^{xy} && \text{using the Taylor series expansion for the exponential function} \\
&= e^{-\frac{(x-y)^2}{2}} \\
&= k(x,y).
\end{aligned}$$

2. As we have seen in Section 2.4, the effective dimension is $n$. The crucial difference of kernelized regression (e.g., Gaussian processes) to linear regression is that the effective dimension *grows* with the sample size, whereas it is fixed for linear regression. Models where the effective dimension may depend on the sample size are called *non-parametric* models, and models where the effective dimension is fixed are called *parametric* models.

**Solution to Problem 4.2.**

1. **Yes**. This is merely a restriction of the standard Gaussian kernel in $\mathbb{R}^2$ onto its subset $\mathbb{S}$.
2. We have

$$\begin{aligned}
&\underbrace{\begin{pmatrix} 1 & e^{-\pi^2/32} & e^{-\pi^2/8} & e^{-\pi^2/32} \\ e^{-\pi^2/32} & 1 & e^{-\pi^2/32} & e^{-\pi^2/8} \\ e^{-\pi^2/8} & e^{-\pi^2/32} & 1 & e^{-\pi^2/32} \\ e^{-\pi^2/32} & e^{-\pi^2/8} & e^{-\pi^2/32} & 1 \end{pmatrix}}_{K} \begin{pmatrix} -1 \\ 1 \\ -1 \\ 1 \end{pmatrix} \\
&\qquad = \underbrace{\left(1 + e^{-\pi^2/8} - 2e^{-\pi^2/32}\right)}_{\approx -0.18} \begin{pmatrix} -1 \\ 1 \\ -1 \\ 1 \end{pmatrix}
\end{aligned}$$

3. **No**. (2) gives an example of a negative eigenvalue in a covariance matrix. This implies that $k_{\mathrm{i}}$ cannot be positive semi-definite.
4. **Yes**. We have $k_h(\theta, \theta') = \langle \boldsymbol{\phi}(\theta), \boldsymbol{\phi}(\theta') \rangle_{\mathbb{R}^L}$ with

$$\boldsymbol{\phi}(\theta) \doteq \frac{1}{\sqrt{C_\kappa}} \begin{pmatrix} 1 \\ e^{-\frac{\kappa^2}{4}} \sqrt{2} \cos(\theta) \\ e^{-\frac{\kappa^2}{4}} \sqrt{2} \sin(\theta) \\ \vdots \\ e^{-\frac{\kappa^2}{4}(L-1)^2} \sqrt{2} \cos((L-1)\theta) \\ e^{-\frac{\kappa^2}{4}(L-1)^2} \sqrt{2} \sin((L-1)\theta) \end{pmatrix}.$$

**Solution to Problem 4.3.** First we look at the mean,

$$\mu(t) = \mathbb{E}[X_t] = \mathbb{E}[X_{t-1} + \varepsilon_{t-1}] = \mathbb{E}[X_{t-1}] = \mu(t-1).$$

Knowing that $\mu(0) = 0$, we can derive that $\mu(t) = 0$ $(\forall t)$.

Now we look at the variance of $X_t$,

$$\mathrm{Var}[X_t] = \mathrm{Var}\left[X_0 + \sum_{\tau=0}^{t-1} \varepsilon_\tau\right] = \sigma_0^2 + t\sigma_x^2. \qquad \text{using that the noise is independent}$$

Finally, we look at the distribution of $[f(t)\ f(t')]^\top$ for arbitrary $t \leq t'$.

$$\begin{bmatrix} X_t \\ X_{t'} \end{bmatrix} = \begin{bmatrix} X_t \\ X_t \end{bmatrix} + \sum_{\tau=t}^{t'-1} \begin{bmatrix} 0 \\ \varepsilon_\tau \end{bmatrix}.$$

Therefore, we get that

$$\begin{aligned} \begin{bmatrix} X_t \\ X_{t'} \end{bmatrix} &\sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mathrm{Var}[X_t] \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + (t'-t) \begin{bmatrix} 0 & 0 \\ 0 & \sigma_x^2 \end{bmatrix}\right) \\ &= \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} \mathrm{Var}[X_t] & \mathrm{Var}[X_t] \\ \mathrm{Var}[X_t] & \mathrm{Var}[X_t] + (t'-t)\sigma_x^2 \end{bmatrix}\right). \end{aligned}$$

We take the kernel $k_{\mathrm{KF}}(t, t')$ to be the covariance between $f(t)$ and $f(t')$, which is $\mathrm{Var}[X_t] = \sigma_0^2 + \sigma_x^2 t$. Notice, however, that we assumed $t \leq t'$. Thus, overall, the kernel is described by

$$k_{\mathrm{KF}}(t, t') = \sigma_0^2 + \sigma_x^2 \min\{t, t'\}.$$

**Solution to Problem 4.4.**

1. As $f \in \mathcal{H}_k(\mathcal{X})$ we can express $f(\boldsymbol{x})$ for some $\beta_i \in \mathbb{R}$ and $\boldsymbol{x}_i \in \mathcal{X}$ as

$$\begin{aligned} f(\boldsymbol{x}) &= \sum_{i=1}^{n} \beta_i k(\boldsymbol{x}, \boldsymbol{x}_i) \\ &= \sum_{i=1}^{n} \beta_i \left\langle k(\boldsymbol{x}_i, \cdot), k(\boldsymbol{x}, \cdot) \right\rangle_k \\ &= \left\langle \sum_{i=1}^{n} \beta_i k(\boldsymbol{x}_i, \cdot), k(\boldsymbol{x}, \cdot) \right\rangle_k \\ &= \left\langle f(\cdot), k(\boldsymbol{x}, \cdot) \right\rangle_k. \end{aligned}$$

2. By applying Cauchy-Schwarz,

$$\begin{aligned} |f(\boldsymbol{x}) - f(\boldsymbol{y})| &= |\left\langle f, k(\boldsymbol{x}, \cdot) - k(\boldsymbol{y}, \cdot) \right\rangle_k| \\ &\leq \|f\|_k \, \|k(\boldsymbol{x}, \cdot) - k(\boldsymbol{y}, \cdot)\|_k \end{aligned}$$

**Solution to Problem 4.5.** We denote by $f_A \doteq \Pi_A f$ the orthogonal projection of $f$ onto $\operatorname{span}\{k(\boldsymbol{x}_1, \cdot), \ldots, k(\boldsymbol{x}_n, \cdot)\}$ which implies that $\hat{f}_A$ is a linear combination of $k(\boldsymbol{x}_1, \cdot), \ldots, k(\boldsymbol{x}_n, \cdot)$.

We then have that $f_A^\perp \doteq f - f_A$ is orthogonal to $\operatorname{span}\{k(\boldsymbol{x}_1, \cdot), \ldots, k(\boldsymbol{x}_n, \cdot)\}$. Therefore, for any $i \in [n]$,

$$f(\boldsymbol{x}) = \langle f, k(\boldsymbol{x}_i, \cdot)\rangle_k = \left\langle f_A + f_A^\perp, k(\boldsymbol{x}_i, \cdot)\right\rangle_k = \langle f_A, k(\boldsymbol{x}_i, \cdot)\rangle_k = f_A(\boldsymbol{x})$$

which implies

$$\mathcal{L}(f(\boldsymbol{x}_1), \ldots, f(\boldsymbol{x}_n)) = \mathcal{L}(f_A(\boldsymbol{x}_1), \ldots, f_A(\boldsymbol{x}_n)).$$

Denoting the objective of Equation (4.19) by $j(f)$ and noting that $\|f_A\|_k \leq \|f\|_k$, we have that $j(f_A) \leq j(f)$. Therefore, if $\hat{f}$ minimizes $j(f)$, then $\hat{f}_A \doteq \Pi_A \hat{f}$ is also a minimizer since $j(\hat{f}_A) \leq j(\hat{f})$. Thus, we conclude that there exists some $\hat{\boldsymbol{\alpha}} \in \mathbb{R}^n$ such that $\hat{f}_A(\boldsymbol{x}) = \sum_{i=1}^n \hat{\alpha}_i k(\boldsymbol{x}_i, \boldsymbol{x})$ minimizes $j(f)$.

**Solution to Problem 4.6.**

1. By the representer theorem (4.20), $\hat{f}(\boldsymbol{x}) = \hat{\boldsymbol{\alpha}}^\top \boldsymbol{k}_{\boldsymbol{x},A}$. In particular, we have $\boldsymbol{f} = \boldsymbol{K}\hat{\boldsymbol{\alpha}}$ and therefore

$$\begin{aligned} -\log p(y_{1:n} \mid \boldsymbol{x}_{1:n}, f) &= \frac{1}{2\sigma_{\mathrm{n}}^2} \|\boldsymbol{y} - \boldsymbol{f}\|_2^2 + \text{const} \\ &= \frac{1}{2\sigma_{\mathrm{n}}^2} \|\boldsymbol{y} - \boldsymbol{K}\hat{\boldsymbol{\alpha}}\|_2^2 + \text{const}. \end{aligned}$$

   The regularization term simplifies to

$$\|f\|_k^2 = \langle f, f\rangle_k = \hat{\boldsymbol{\alpha}}^\top \boldsymbol{K}\hat{\boldsymbol{\alpha}} = \|\hat{\boldsymbol{\alpha}}\|_{\boldsymbol{K}}^2 .$$

   Combining, we have that

$$\hat{\boldsymbol{\alpha}} = \operatorname*{arg\,min}_{\boldsymbol{\alpha} \in \mathbb{R}^n} \frac{1}{2\sigma_{\mathrm{n}}^2} \|\boldsymbol{y} - \boldsymbol{K}\boldsymbol{\alpha}\|_2^2 + \frac{1}{2} \|\boldsymbol{\alpha}\|_{\boldsymbol{K}}^2$$

   as desired. It follows by multiplying through with $2\sigma_{\mathrm{n}}^2$ that $\lambda = \sigma_{\mathrm{n}}^2$.
2. Expanding the objective determined in (1), we are looking for the minimizer of

$$\boldsymbol{\alpha}^\top (\sigma_{\mathrm{n}}^2 \boldsymbol{K} + \boldsymbol{K}^2)\boldsymbol{\alpha} - 2\boldsymbol{y}^\top \boldsymbol{K}\boldsymbol{\alpha} + \boldsymbol{y}^\top \boldsymbol{y}.$$

   Differentiating with respect to the coefficients $\boldsymbol{\alpha}$, we obtain the minimizer $\hat{\boldsymbol{\alpha}} = (\boldsymbol{K} + \sigma_{\mathrm{n}}^2 \boldsymbol{I})^{-1}\boldsymbol{y}$. Thus, the prediction at a point $\boldsymbol{x}^\star$ is $\boldsymbol{k}_{\boldsymbol{x}^\star,A}^\top (\boldsymbol{K} + \sigma_{\mathrm{n}}^2 \boldsymbol{I})^{-1}\boldsymbol{y}$ which coincides with the MAP estimate.

**Solution to Problem 4.7.**

1. Applying the two hints Equations (4.54) and (4.55) to Equation (4.29) yields,

$$\frac{\partial}{\partial \theta_j} \log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{y}^\top \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \boldsymbol{y} - \frac{1}{2} \operatorname{tr}\left( \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} \right).$$

We can simplify to

$$\begin{aligned}
&= \frac{1}{2} \operatorname{tr}\left( \boldsymbol{y}^\top \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \boldsymbol{y} \right) - \frac{1}{2} \operatorname{tr}\left( \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} \right) && \text{using that } \boldsymbol{y}^\top \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \tfrac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \boldsymbol{y} \text{ is a scalar} \\
&= \frac{1}{2} \operatorname{tr}\left( \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \boldsymbol{y} \boldsymbol{y}^\top \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} - \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} \right) && \text{using the cyclic property and linearity of the trace} \\
&= \frac{1}{2} \operatorname{tr}\left( \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \boldsymbol{y} (\boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \boldsymbol{y})^\top \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} - \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} \right) && \text{using that } \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1} \text{ is symmetric} \\
&= \frac{1}{2} \operatorname{tr}\left( (\alpha \alpha^\top - \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}^{-1}) \frac{\partial \boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}}}{\partial \theta_j} \right).
\end{aligned}$$

2. We denote by $\tilde{\boldsymbol{K}}$ the covariance matrix of $\boldsymbol{y}$ for the covariance function $\tilde{k}$, so $\boldsymbol{K}_{\boldsymbol{y},\boldsymbol{\theta}} = \theta_0 \tilde{\boldsymbol{K}}$. Then,

$$\frac{\partial}{\partial \theta_0} \log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\theta}) = \frac{1}{2} \operatorname{tr}\Big( (\theta_0^{-2} \tilde{\boldsymbol{K}}^{-1} \boldsymbol{y} (\tilde{\boldsymbol{K}}^{-1} \boldsymbol{y})^\top - \theta_0^{-1} \tilde{\boldsymbol{K}}^{-1}) \tilde{\boldsymbol{K}} \Big). \qquad \text{using Equation (4.30)}$$

Simplifying the terms and using linearity of the trace, we obtain that

$$\frac{\partial}{\partial \theta_0} \log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\theta}) = 0 \quad \iff \quad \theta_0 = \frac{1}{n} \operatorname{tr}\Big( \boldsymbol{y} \boldsymbol{y}^\top \tilde{\boldsymbol{K}}^{-1} \Big).$$

If we define $\tilde{\boldsymbol{\Lambda}} \doteq \tilde{\boldsymbol{K}}^{-1}$ as the precision matrix associated to $\boldsymbol{y}$ for the covariance function $\tilde{k}$, we can express $\theta_0^\star$ in closed form as

$$\theta_0^\star = \frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{n} \tilde{\boldsymbol{\Lambda}}(i,j) y_i y_j. \tag{B.4}$$

3. We immediately see from Equation (B.4) that $\theta_0^\star$ scales by $s^2$ if $\boldsymbol{y}$ is scaled by $s$.

**Solution to Problem 4.8.**

1. We have that $s(\cdot) \in [-\sqrt{2}, \sqrt{2}]$, and hence, $s(\boldsymbol{\Delta}_i)$ is $\sqrt{2}$-sub-Gaussian.[2] It then follows from Hoeffding's inequality (A.41) that

[2] see Example A.16

$$\mathbb{P}(|f(\boldsymbol{\Delta}_i)| \geq \epsilon) \leq 2 \exp\left( -\frac{m \epsilon^2}{4} \right).$$

2. We can apply Markov's inequality (A.71) to obtain

$$\mathbb{P}\Big( \|\nabla f(\boldsymbol{\Delta}^\star)\|_2 \geq \frac{\epsilon}{2r} \Big) = \mathbb{P}\left( \|\nabla f(\boldsymbol{\Delta}^\star)\|_2^2 \geq \left( \frac{\epsilon}{2r} \right)^2 \right)$$

$$\leq \left(\frac{2r\mathbb{E}\left\|\nabla f(\boldsymbol{\Delta}^\star)\right\|_2}{\epsilon}\right)^2.$$

It remains to bound the expectation. We have

$$\begin{aligned}\mathbb{E}\left\|\nabla f(\boldsymbol{\Delta}^\star)\right\|_2^2 &= \mathbb{E}\left\|\nabla s(\boldsymbol{\Delta}^\star) - \nabla k(\boldsymbol{\Delta}^\star)\right\|_2^2 \\ &= \mathbb{E}\left\|\nabla s(\boldsymbol{\Delta}^\star)\right\|_2^2 - 2\nabla k(\boldsymbol{\Delta}^\star)^\top \mathbb{E}\nabla s(\boldsymbol{\Delta}^\star) + \mathbb{E}\left\|\nabla k(\boldsymbol{\Delta}^\star)\right\|_2^2. && \text{using linearity of expectation (1.20)}\end{aligned}$$

Note that $\mathbb{E}\nabla s(\boldsymbol{\Delta}) = \nabla k(\boldsymbol{\Delta})$ using Equation (A.5) and using that $s(\boldsymbol{\Delta})$ is an unbiased estimator of $k(\boldsymbol{\Delta})$. Therefore,

$$\begin{aligned}&= \mathbb{E}\left\|\nabla s(\boldsymbol{\Delta}^\star)\right\|_2^2 - \mathbb{E}\left\|\nabla k(\boldsymbol{\Delta}^\star)\right\|_2^2 \\ &\leq \mathbb{E}\left\|\nabla s(\boldsymbol{\Delta}^\star)\right\|_2^2 \\ &\leq \mathbb{E}_{\boldsymbol{\omega}\sim p}\left\|\boldsymbol{\omega}\right\|_2^2 = \sigma_{\mathrm{p}}^2. && \text{using that } s \text{ is the cos of a linear function in } \boldsymbol{\omega}\end{aligned}$$

3. Using a union bound (1.73) and then the result of (1),

$$\mathbb{P}\left(\bigcup_{i=1}^{T} |f(\boldsymbol{\Delta}_i)| \geq \frac{\epsilon}{2}\right) \leq \sum_{i=1}^{T} \mathbb{P}\left(|f(\boldsymbol{\Delta}_i)| \geq \frac{\epsilon}{2}\right) \leq 2T\exp\left(-\frac{m\epsilon^2}{16}\right).$$

4. First, note that by contraposition,

$$\sup_{\boldsymbol{\Delta}\in\mathcal{M}_\Delta} |f(\boldsymbol{\Delta})| \geq \epsilon \implies \exists i.\ |f(\boldsymbol{\Delta}_i)| \geq \frac{\epsilon}{2} \text{ or } \left\|\nabla f(\boldsymbol{\Delta}^\star)\right\|_2 \geq \frac{\epsilon}{2r},$$

and therefore,

$$\begin{aligned}\mathbb{P}\left(\sup_{\boldsymbol{\Delta}\in\mathcal{M}_\Delta} |f(\boldsymbol{\Delta})| \geq \epsilon\right) &\leq \mathbb{P}\left(\bigcup_{i=1}^{T} |f(\boldsymbol{\Delta}_i)| \geq \frac{\epsilon}{2} \cup \left\|\nabla f(\boldsymbol{\Delta}^\star)\right\|_2 \geq \frac{\epsilon}{2r}\right) \\ &\leq \mathbb{P}\left(\bigcup_{i=1}^{T} |f(\boldsymbol{\Delta}_i)| \geq \frac{\epsilon}{2}\right) + \mathbb{P}\left(\left\|\nabla f(\boldsymbol{\Delta}^\star)\right\|_2 \geq \frac{\epsilon}{2r}\right) && \text{using a union bound (1.73)} \\ &\leq 2T\exp\left(-\frac{m\epsilon^2}{2^4}\right) + \left(\frac{2r\sigma_{\mathrm{p}}}{\epsilon}\right)^2 && \text{using the results from (2) and (3)} \\ &\leq \alpha r^{-d} + \beta r^2 && \text{using } T \leq (4\,\mathrm{diam}(\mathcal{M})/r)^d\end{aligned}$$

with $\alpha = 2(4\,\mathrm{diam}(\mathcal{M}))^d \exp\left(-\frac{m\epsilon^2}{2^4}\right)$ and $\beta = \left(\frac{2\sigma_{\mathrm{p}}}{\epsilon}\right)^2$. Using the hint, we obtain

$$\begin{aligned}&= 2\beta^{\frac{d}{d+2}}\alpha^{\frac{2}{d+2}} \\ &= 2\left(2\left(\frac{2^3\sigma_{\mathrm{p}}\mathrm{diag}(\mathcal{M})}{\epsilon}\right)^{2d}\right)^{\frac{1}{d+2}} \exp\left(-\frac{m\epsilon^2}{2^3(d+2)}\right) \\ &\leq 2^8\left(\frac{\sigma_{\mathrm{p}}\mathrm{diag}(\mathcal{M})}{\epsilon}\right)^2 \exp\left(-\frac{m\epsilon^2}{2^3(d+2)}\right) && \text{using } \tfrac{\sigma_{\mathrm{p}}\mathrm{diam}(\mathcal{M})}{\epsilon} \geq 1\end{aligned}$$

5. We have

$$\sigma_{\mathrm{p}}^2 = \mathbb{E}_{\boldsymbol{\omega}\sim p}\left[\boldsymbol{\omega}^\top\boldsymbol{\omega}\right]$$

$$
\begin{aligned}
&= \int \boldsymbol{\omega}^\top \boldsymbol{\omega} \cdot p(\boldsymbol{\omega})\, d\boldsymbol{\omega} \\
&= \int \boldsymbol{\omega}^\top \boldsymbol{\omega} \cdot e^{i\boldsymbol{\omega}^\top \mathbf{0}} p(\boldsymbol{\omega})\, d\boldsymbol{\omega}.
\end{aligned}
$$

Now observe that $\frac{\partial^2}{\partial \Delta_j^2} \int e^{i\boldsymbol{\omega}^\top \boldsymbol{\Delta}} p(\boldsymbol{\omega})\, d\boldsymbol{\omega} = -\int \omega_j^2 e^{i\boldsymbol{\omega}^\top \boldsymbol{\Delta}} p(\boldsymbol{\omega})\, d\boldsymbol{\omega}$. Thus,

$$
\begin{aligned}
&= -\mathrm{tr}\left( \boldsymbol{H}_{\boldsymbol{\Delta}} \int p(\boldsymbol{\omega}) e^{i\boldsymbol{\omega}^\top \boldsymbol{\Delta}}\, d\boldsymbol{\omega} \Big|_{\boldsymbol{\Delta}=\mathbf{0}} \right) \\
&= -\mathrm{tr}(\boldsymbol{H}_{\boldsymbol{\Delta}} k(\mathbf{0})). && \text{using that } p \text{ is the Fourier transform of } k \text{ (4.38)}
\end{aligned}
$$

Finally, we have for the Gaussian kernel that

$$
\frac{\partial^2}{\partial \Delta_j^2} \exp\left( -\frac{\boldsymbol{\Delta}^\top \boldsymbol{\Delta}}{2h^2} \right) \Big|_{\Delta_j = 0} = -\frac{1}{h^2}.
$$

**Solution to Problem 4.9.**

1. We write $\tilde{\boldsymbol{f}} \doteq [\boldsymbol{f}\ f^\star]^\top$. From the definition of SoR (4.48), we gather

$$
q_{\mathrm{SoR}}(\tilde{\boldsymbol{f}} \mid \boldsymbol{u}) = \mathcal{N}\left( \tilde{\boldsymbol{f}};\ \begin{bmatrix} \boldsymbol{K}_{AU} \boldsymbol{K}_{UU}^{-1} \\ \boldsymbol{K}_{\star U} \boldsymbol{K}_{UU}^{-1} \end{bmatrix} \boldsymbol{u}, \mathbf{0} \right). \tag{B.5}
$$

   We know that $\tilde{\boldsymbol{f}}$ and $\boldsymbol{u}$ are jointly Gaussian, and hence, the marginal distribution of $\tilde{\boldsymbol{f}}$ is also Gaussian. We have for the mean and variance that

$$
\begin{aligned}
\mathbb{E}\left[\tilde{\boldsymbol{f}}\right] &= \mathbb{E}\left[\mathbb{E}\left[\tilde{\boldsymbol{f}} \mid \boldsymbol{u}\right]\right] && \text{using the tower rule (1.25)} \\
&= \mathbb{E}_{\boldsymbol{u}}\left[ \begin{bmatrix} \boldsymbol{K}_{AU} \boldsymbol{K}_{UU}^{-1} \\ \boldsymbol{K}_{\star U} \boldsymbol{K}_{UU}^{-1} \end{bmatrix} \boldsymbol{u} \right] && \text{using Equation (B.5)} \\
&= \begin{bmatrix} \boldsymbol{K}_{AU} \boldsymbol{K}_{UU}^{-1} \\ \boldsymbol{K}_{\star U} \boldsymbol{K}_{UU}^{-1} \end{bmatrix} \mathbb{E}[\boldsymbol{u}] = \mathbf{0} && \text{using linearity of expectation (1.20)} \\
\mathrm{Var}\left[\tilde{\boldsymbol{f}}\right] &= \mathbb{E}\underbrace{\mathrm{Var}\left[\tilde{\boldsymbol{f}} \mid \boldsymbol{u}\right]}_{\mathbf{0}} + \mathrm{Var}\,\mathbb{E}\left[\tilde{\boldsymbol{f}} \mid \boldsymbol{u}\right] && \text{using the law of total variance (1.41)} \\
&= \mathrm{Var}_{\boldsymbol{u}}\left[ \begin{bmatrix} \boldsymbol{K}_{AU} \boldsymbol{K}_{UU}^{-1} \\ \boldsymbol{K}_{\star U} \boldsymbol{K}_{UU}^{-1} \end{bmatrix} \boldsymbol{u} \right] && \text{using Equation (B.5)} \\
&= \begin{bmatrix} \boldsymbol{K}_{AU} \boldsymbol{K}_{UU}^{-1} \\ \boldsymbol{K}_{\star U} \boldsymbol{K}_{UU}^{-1} \end{bmatrix} \underbrace{\mathrm{Var}[\boldsymbol{u}]}_{\boldsymbol{K}_{UU}} \begin{bmatrix} \boldsymbol{K}_{AU} \boldsymbol{K}_{UU}^{-1} \\ \boldsymbol{K}_{\star U} \boldsymbol{K}_{UU}^{-1} \end{bmatrix}^\top && \text{using Equation (1.38)} \\
&= \begin{bmatrix} \boldsymbol{Q}_{AA} & \boldsymbol{Q}_{A\star} \\ \boldsymbol{Q}_{\star A} & \boldsymbol{Q}_{\star\star} \end{bmatrix}.
\end{aligned}
$$

   Having determined $q_{\mathrm{SoR}}(\boldsymbol{f}, f^\star)$, $q_{\mathrm{SoR}}(f^\star \mid \boldsymbol{y})$ follows directly using the formulas for finding the Gaussian process predictive posterior (4.6).

2. The given covariance function follows directly from inspecting the derived covariance matrix, $\mathrm{Var}[\boldsymbol{f}] = \boldsymbol{Q}_{AA}$.

## *Variational Inference*

**Solution to Problem 5.1.**

1. We have

$$\begin{aligned}
\nabla_w \ell_{\log}(w^\top x; y) &= \nabla_w \log(1+\exp(-y w^\top x)) && \text{using the definition of the logistic loss (5.13)}\\
&= \nabla_w \log \frac{1+\exp(y w^\top x)}{\exp(y w^\top x)}\\
&= \nabla_w \log(1+\exp(y w^\top x)) - y w^\top x\\
&= \frac{1}{1+\exp(-y w^\top x)} \cdot \exp(-y w^\top x) \cdot y x - y x && \text{using the chain rule}\\
&= -y x \cdot \left(1 - \frac{\exp(y w^\top x)}{1+\exp(y w^\top x)}\right)\\
&= -y x \cdot \sigma(-y w^\top x). && \text{using the definition of the logistic function (5.9)}
\end{aligned}$$

2. As suggested in the hint, we compute the first derivative of $\sigma$,

$$\begin{aligned}
\sigma'(z) \doteq \frac{d}{dz}\sigma(z) = \frac{d}{dz}\frac{1}{1+\exp(-z)} && \text{using the definition of the logistic function (5.9)}\\
= \frac{\exp(-z)}{(1+\exp(-z))^2} && \text{using the quotient rule}\\
= \sigma(z)\cdot(1-\sigma(z)). && \text{using the definition of the logistic function (5.9)}
\end{aligned}$$

We get for the Hessian of $\ell_{\log}$,

$$\begin{aligned}
H_w \ell_{\log}(w^\top x; y) &= D_w \nabla_w \ell_{\log}(w^\top x; y) && \text{using the definition of a Hessian (A.57) and symmetry}\\
&= -y x\, D_w \sigma(-y w^\top x). && \text{using the gradient of the logistic loss from (1)}
\end{aligned}$$

We have $D_w - y w^\top x = -y x^\top$ and $D_z \sigma(z) = \sigma'(z)$, and therefore, using the chain rule of multivariate calculus (5.74),

$$\begin{aligned}
&= -y x \cdot \sigma'(-y w^\top x) \cdot (-y x^\top)\\
&= x x^\top \cdot \sigma'(-y w^\top x). && \text{using } y^2 = 1
\end{aligned}$$

Finally, recall that

$$\begin{aligned}
\sigma(w^\top x) &= \mathbb{P}(y = +1 \mid x, w) \quad \text{and}\\
\sigma(-w^\top x) &= \mathbb{P}(y = -1 \mid x, w).
\end{aligned}$$

Thus,

$$\begin{aligned}
\sigma'(-y w^\top x) &= \mathbb{P}(Y \neq y \mid x, w) \cdot (1 - \mathbb{P}(Y \neq y \mid x, w))\\
&= \mathbb{P}(Y \neq y \mid x, w) \cdot \mathbb{P}(Y = y \mid x, w)\\
&= (1 - \mathbb{P}(Y = y \mid x, w)) \cdot \mathbb{P}(Y = y \mid x, w)\\
&= \sigma'(w^\top x).
\end{aligned}$$

3. By the second-order characterization of convexity (cf. Remark A.26), a twice differentiable function $f$ is convex if and only if its Hessian is positive semi-definite. Writing $c \doteq \sigma'(\boldsymbol{w}^\top \boldsymbol{x})$, we have for any $\boldsymbol{\delta} \in \mathbb{R}^n$ that

$$\boldsymbol{\delta}^\top \boldsymbol{H}_{\boldsymbol{w}} \ell_{\log}(\boldsymbol{w}^\top \boldsymbol{x}; y)\boldsymbol{\delta} = \boldsymbol{\delta}^\top \left(c \cdot \boldsymbol{x}\boldsymbol{x}^\top\right)\boldsymbol{\delta} = c(\boldsymbol{\delta}^\top \boldsymbol{x})^2 \geq 0, \qquad \text{using } c \geq 0 \text{ and } (\cdot)^2 \geq 0$$

and hence, the logistic loss is convex in $\boldsymbol{w}$.

**Solution to Problem 5.2.**

1. Using the law of total probability (1.12),

$$\begin{aligned} &p(y^\star = +1 \mid \boldsymbol{x}_{1:n}, y_{1:n}, \boldsymbol{x}^\star) \\ &= \int p(y^\star = +1 \mid f^\star) p(f^\star \mid \boldsymbol{x}_{1:n}, y_{1:n}, \boldsymbol{x}^\star)\, df^\star \\ &= \int \sigma(f^\star) p(f^\star \mid \boldsymbol{x}_{1:n}, y_{1:n}, \boldsymbol{x}^\star)\, df^\star. \qquad \text{using } y^\star \sim \text{Bern}(\sigma(f^\star)) \end{aligned}$$

   Due to the non-Gaussian likelihood, the integral is analytically intractable. However, as the integral is one-dimensional, numerical approximations such as the Gauss-Legendre quadrature can be used.

2. (a) According to Bayes' rule (1.45), we know that

$$\begin{aligned} \psi(\boldsymbol{f}) &= \log p(\boldsymbol{f} \mid \boldsymbol{x}_{1:n}, y_{1:n}) && \text{using Equation (5.2)} \\ &= \log p(y_{1:n} \mid \boldsymbol{f}) + \log p(\boldsymbol{f} \mid \boldsymbol{x}_{1:n}) - \log p(y_{1:n} \mid \boldsymbol{x}_{1:n}) && \text{using Bayes' rule (1.45)} \\ &= \log p(y_{1:n} \mid \boldsymbol{f}) + \log p(\boldsymbol{f} \mid \boldsymbol{x}_{1:n}) + \text{const.} \end{aligned}$$

   Note that $p(\boldsymbol{f} \mid \boldsymbol{x}_{1:n}) = \mathcal{N}(\boldsymbol{f}; \boldsymbol{0}, \boldsymbol{K}_{AA})$. Plugging in this closed-form Gaussian distribution of the GP prior gives

$$= \log p(y_{1:n} \mid \boldsymbol{f}) - \frac{1}{2}\boldsymbol{f}^\top \boldsymbol{K}_{AA}^{-1} \boldsymbol{f} + \text{const}$$

   Differentiating with respect to $\boldsymbol{f}$ yields

$$\begin{aligned} \boldsymbol{\nabla}\psi(\boldsymbol{f}) &= \boldsymbol{\nabla} \log p(y_{1:n} \mid \boldsymbol{f}) - \boldsymbol{K}_{AA}^{-1}\boldsymbol{f} \\ \boldsymbol{H}\psi(\boldsymbol{f}) &= \boldsymbol{H}_{\boldsymbol{f}} \log p(y_{1:n} \mid \boldsymbol{f}) - \boldsymbol{K}_{AA}^{-1}. \end{aligned}$$

   Hence, $\boldsymbol{\Lambda} = \boldsymbol{K}_{AA}^{-1} + \boldsymbol{W}$ where $\boldsymbol{W} \doteq -\boldsymbol{H}_{\boldsymbol{f}} \log p(y_{1:n} \mid \boldsymbol{f})\big|_{\boldsymbol{f}=\hat{\boldsymbol{f}}}$.
   It remains to derive $\boldsymbol{W}$ for the probit likelihood. Using independence of the training examples,

$$\log p(y_{1:n} \mid \boldsymbol{f}) = \sum_{i=1}^{n} \log p(y_i \mid f_i),$$

   and hence, the Hessian of this expression is diagonal. Using the symmetry of $\Phi(z; 0, \sigma_{\text{n}}^2)$ around zero, we can write

$$\log p(y_i \mid f_i) = \log \Phi(y_i f_i; 0, \sigma_{\text{n}}^2).$$

In the following, we write $\mathcal{N}(z) \doteq \mathcal{N}(z; 0, \sigma_n^2)$ and $\Phi(z) \doteq \Phi(z; 0, \sigma_n^2)$ to simplify the notation. Differentiating with respect to $f_i$, we obtain

$$\frac{\partial}{\partial f_i} \log \Phi(y_i f_i) = \frac{y_i \mathcal{N}(f_i)}{\Phi(y_i f_i)} \qquad \text{using } \mathcal{N}(y_i f_i) = \mathcal{N}(f_i) \text{ since } \mathcal{N} \text{ is an even function and } y_i \in \{\pm 1\}$$

$$\frac{\partial^2}{\partial f_i^2} \log \Phi(y_i f_i) = -\frac{\mathcal{N}(f_i)^2}{\Phi(y_i f_i)^2} - \frac{y_i f_i \mathcal{N}(f_i)}{\sigma_n^2 \Phi(y_i f_i)},$$

and $\boldsymbol{W} = -\operatorname{diag}\{\frac{\partial^2}{\partial f_i^2} \log \Phi(y_i f_i)\}_{i=1}^n \Big|_{f=\hat{f}}$.

(b) Note that $\boldsymbol{\Lambda}'$ is a precision matrix over weights $\boldsymbol{w}$ and $\boldsymbol{f} = \boldsymbol{X}\boldsymbol{w}$, so by Equation (1.38) the corresponding variance over latent values $\boldsymbol{f}$ is $\boldsymbol{X}\boldsymbol{\Lambda}'^{-1}\boldsymbol{X}^\top$. The two precision matrices are therefore equivalent if $\boldsymbol{\Lambda}^{-1} = \boldsymbol{X}\boldsymbol{\Lambda}'^{-1}\boldsymbol{X}^\top$.
Analogously to Example 5.2 and Problem 5.1, we have that

$$\begin{aligned}
\boldsymbol{W} &= -\boldsymbol{H}_{\boldsymbol{f}} \log p(\boldsymbol{y}_{1:n} \mid \boldsymbol{f})\big|_{\boldsymbol{f}=\hat{\boldsymbol{f}}} \\
&= \operatorname{diag}_{i \in [n]}\{\sigma(\hat{f}_i)(1 - \sigma(\hat{f}_i))\} \\
&= \operatorname{diag}_{i \in [n]}\{\pi_i(1 - \pi_i)\}.
\end{aligned}$$

By Woodbury's matrix identity (A.66),

$$\begin{aligned}
\boldsymbol{\Lambda}'^{-1} &= (\boldsymbol{I} + \boldsymbol{X}^\top \boldsymbol{W} \boldsymbol{X})^{-1} \\
&= \boldsymbol{I} - \boldsymbol{X}^\top (\boldsymbol{W}^{-1} + \boldsymbol{X}\boldsymbol{X}^\top)^{-1} \boldsymbol{X}.
\end{aligned}$$

Thus,

$$\begin{aligned}
\boldsymbol{X}\boldsymbol{\Lambda}'^{-1}\boldsymbol{X}^\top &= \boldsymbol{X}\boldsymbol{X}^\top - \boldsymbol{X}\boldsymbol{X}^\top (\boldsymbol{W}^{-1} + \boldsymbol{X}\boldsymbol{X}^\top)^{-1} \boldsymbol{X}\boldsymbol{X}^\top \\
&= \boldsymbol{K}_{AA} - \boldsymbol{K}_{AA}(\boldsymbol{W}^{-1} + \boldsymbol{K}_{AA})^{-1} \boldsymbol{K}_{AA} && \text{using that } \boldsymbol{K}_{AA} = \boldsymbol{X}\boldsymbol{X}^\top \\
&= (\boldsymbol{K}_{AA}^{-1} + \boldsymbol{W})^{-1} && \text{by the matrix inversion lemma (A.68)} \\
&= \boldsymbol{\Lambda}^{-1}.
\end{aligned}$$

(c) Using the formulas for the conditional distribution of a Gaussian (1.53), evaluating a conditional GP at a test point $\boldsymbol{x}^\star$ yields,

$$f^\star \mid \boldsymbol{x}^\star, \boldsymbol{f} \sim \mathcal{N}(\mu^\star, k^\star), \quad \text{where} \tag{B.6a}$$

$$\mu^\star \doteq \boldsymbol{k}_{\boldsymbol{x}^\star, A}^\top \boldsymbol{K}_{AA}^{-1} \boldsymbol{f}, \tag{B.6b}$$

$$k^\star \doteq k(\boldsymbol{x}^\star, \boldsymbol{x}^\star) - \boldsymbol{k}_{\boldsymbol{x}^\star, A}^\top \boldsymbol{K}_{AA}^{-1} \boldsymbol{k}_{\boldsymbol{x}^\star, A}. \tag{B.6c}$$

Then, using the tower rule (1.25),

$$\begin{aligned}
\mathbb{E}_q[f^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}] &= \mathbb{E}_{\boldsymbol{f} \sim q}\Big[\mathbb{E}_{f^\star \sim p}[f^\star \mid \boldsymbol{x}^\star, \boldsymbol{f}] \;\Big|\; \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}\Big] \\
&= \boldsymbol{k}_{\boldsymbol{x}^\star, A}^\top \boldsymbol{K}_{AA}^{-1} \mathbb{E}_q[\boldsymbol{f} \mid \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}] && \text{using Equation (B.6b) and linearity of expectation (1.20)}
\end{aligned}$$

$$= \boldsymbol{k}_{\boldsymbol{x}^\star,A}^\top \boldsymbol{K}_{AA}^{-1} \hat{\boldsymbol{f}}. \tag{B.7}$$

Observe that any maximum $\hat{\boldsymbol{f}}$ of $\psi(\boldsymbol{f})$ needs to satisfy $\boldsymbol{\nabla}\psi(\boldsymbol{f}) = \boldsymbol{0}$. Hence, $\hat{\boldsymbol{f}} = \boldsymbol{K}_{AA}(\boldsymbol{\nabla}_{\boldsymbol{f}} \log p(\boldsymbol{y}_{1:n} \mid \boldsymbol{f}))$, and the expectation simplifies to

$$= \boldsymbol{k}_{\boldsymbol{x}^\star,A}^\top (\boldsymbol{\nabla}_{\boldsymbol{f}} \log p(\boldsymbol{y}_{1:n} \mid \boldsymbol{f})).$$

Using the law of total variance (1.41),

$$\begin{aligned}
&\operatorname{Var}_q[f^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}] \\
&= \mathbb{E}_{\boldsymbol{f}\sim q}\left[\operatorname{Var}_{f^\star\sim p}[f^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, \boldsymbol{f}] \;\middle|\; \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}\right] \\
&\quad + \operatorname{Var}_{\boldsymbol{f}\sim q}\left[\mathbb{E}_{f^\star\sim p}[f^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, \boldsymbol{f}] \;\middle|\; \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}\right] \\
&= \mathbb{E}_q[k^\star \mid \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}] + \operatorname{Var}_q\left[\boldsymbol{k}_{\boldsymbol{x}^\star,A}^\top \boldsymbol{K}_{AA}^{-1} \boldsymbol{f} \;\middle|\; \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}\right] && \text{using Equations (B.6b) and (B.6c)} \\
&= k^\star + \boldsymbol{k}_{\boldsymbol{x}^\star,A}^\top \boldsymbol{K}_{AA}^{-1} \operatorname{Var}_q[\boldsymbol{f} \mid \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}] \, \boldsymbol{K}_{AA}^{-1} \boldsymbol{k}_{\boldsymbol{x}^\star,A}. && \text{using that } k^\star \text{ is independent of } \boldsymbol{f},
\end{aligned}$$

Equation (1.38), and symmetry of $\boldsymbol{K}_{AA}^{-1}$

Recall from (a) that $\operatorname{Var}_q[\boldsymbol{f} \mid \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}] = (\boldsymbol{K}_{AA} + \boldsymbol{W}^{-1})^{-1}$, so

$$\begin{aligned}
&= k(\boldsymbol{x}^\star, \boldsymbol{x}^\star) - \boldsymbol{k}_{\boldsymbol{x}^\star,A}^\top \boldsymbol{K}_{AA}^{-1} \boldsymbol{k}_{\boldsymbol{x}^\star,A} && \text{plugging in the expression for } k^\star \text{ (B.6c)} \\
&\quad + \boldsymbol{k}_{\boldsymbol{x}^\star,A}^\top \boldsymbol{K}_{AA}^{-1} (\boldsymbol{K}_{AA} + \boldsymbol{W}^{-1})^{-1} \boldsymbol{K}_{AA}^{-1} \boldsymbol{k}_{\boldsymbol{x}^\star,A} \\
&= k(\boldsymbol{x}^\star, \boldsymbol{x}^\star) - \boldsymbol{k}_{\boldsymbol{x}^\star,A}^\top (\boldsymbol{K}_{AA} + \boldsymbol{W}^{-1})^{-1} \boldsymbol{k}_{\boldsymbol{x}^\star,A}. && \text{using the matrix inversion lemma (A.68)}
\end{aligned} \tag{B.8}$$

Notice the similarity of Equations (B.7) and (B.8) to Equations (B.6b) and (B.6c). The latter is the conditional mean and variance if $\boldsymbol{f}$ is known whereas the former is the conditional mean and variance given the noisy observation $\boldsymbol{y}_{1:n}$ of $\boldsymbol{f}$. The matrix $\boldsymbol{W}$ quantifies the noise in the observations.

(d) Recall that

$$\begin{aligned}
p(y^\star = +1 \mid \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}, \boldsymbol{x}^\star) &\approx \int \sigma(f^\star) q(f^\star \mid \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n}, \boldsymbol{x}^\star) \, df^\star && \text{using the Laplace-approximated latent predictive posterior} \\
&= \mathbb{E}_q[\sigma(f^\star)]. && \text{using LOTUS (1.22)}
\end{aligned}$$

This quantity can be interpreted as the *averaged prediction* over all latent predictions $f^\star$. In contrast, $\sigma(\mathbb{E}_q[f^\star])$ can be understood as the *"MAP" prediction*, which is obtained using the MAP estimate of $f^\star$.[3] As $\sigma$ is nonlinear, the two quantities are not identical, and generally the averaged prediction is preferred.

[3] As $q$ is a Gaussian, its mode (i.e., the MAP estimate) and its mean coincide.

3. We have

$$\begin{aligned}
\mathbb{E}_\varepsilon \mathbb{1}\{f(\boldsymbol{x}) + \varepsilon \geq 0\} &= \mathbb{P}_\varepsilon(f(\boldsymbol{x}) + \varepsilon \geq 0) && \text{using } \mathbb{E}[X] = p \text{ if } X \sim \operatorname{Bern}(p) \\
&= \mathbb{P}_\varepsilon(-\varepsilon \leq f(\boldsymbol{x})) \\
&= \mathbb{P}_\varepsilon(\varepsilon \leq f(\boldsymbol{x})) && \text{using that the distribution of } \varepsilon \text{ is symmetric around } 0 \\
&= \Phi(f(\boldsymbol{x}); 0, \sigma_{\text{n}}^2).
\end{aligned}$$

**Solution to Problem 5.3.**

1. Recall that as $f$ is convex,

   $$\forall \boldsymbol{x}_1, \boldsymbol{x}_2, \forall \lambda \in [0,1]: \quad f(\lambda \boldsymbol{x}_1 + (1-\lambda)\boldsymbol{x}_2) \leq \lambda f(\boldsymbol{x}_1) + (1-\lambda) f(\boldsymbol{x}_2).$$

   We prove the statement by induction on $k$. The base case, $k = 2$, follows trivially from the convexity of $f$. For the induction step, suppose that the statement holds for some fixed $k \geq 2$ and assume w.l.o.g. that $\theta_{k+1} \in (0,1)$. We then have,

   $$\begin{aligned}
   \sum_{i=1}^{k+1} \theta_i f(\boldsymbol{x}_i) &= (1-\theta_{k+1}) \left( \sum_{i=1}^{k} \frac{\theta_i}{1-\theta_{k+1}} f(\boldsymbol{x}_i) \right) + \theta_{k+1} f(\boldsymbol{x}_{k+1}) \\
   &\geq (1-\theta_{k+1}) \cdot f\left( \sum_{i=1}^{k} \frac{\theta_i}{1-\theta_{k+1}} \boldsymbol{x}_i \right) + \theta_{k+1} f(\boldsymbol{x}_{k+1}) && \text{using the induction hypothesis} \\
   &\geq f\left( \sum_{i=1}^{k+1} \theta_i \boldsymbol{x}_i \right). && \text{using the convexity of } f
   \end{aligned}$$

2. Noting that $\log_2$ is concave, we have by Jensen's inequality,

   $$\begin{aligned}
   \mathrm{H}[p] &= \mathbb{E}_{x \sim p}\left[ \log_2 \left( \frac{1}{p(x)} \right) \right] && \text{by definition of entropy (5.28a)} \\
   &\leq \log_2 \mathbb{E}_{x \sim p}\left[ \frac{1}{p(x)} \right] && \text{by Jensen's inequality (5.29)} \\
   &= \log_2 n.
   \end{aligned}$$

**Solution to Problem 5.4.** If $y = 1$ then

$$\ell_{\mathrm{bce}}(\hat{y}; y) = -\log \hat{y} = \log(1 + e^{-\hat{f}}) = \ell_{\log}(\hat{F}; y).$$

If $y = -1$ then

$$\ell_{\mathrm{bce}}(\hat{y}; y) = -\log(1 - \hat{y}) = \log(1 + e^{\hat{f}}) = \ell_{\log}(\hat{f}; y).$$

Here the second equality follows from the simple algebraic fact

$$1 - \frac{1}{1+e^{-z}} = 1 - \frac{e^z}{e^z + 1} = \frac{1}{1+e^z}. \qquad \text{multiplying by } \tfrac{e^z}{e^z}$$

**Solution to Problem 5.5.**

1. Let $p$ and $q$ be two (continuous) distributions. The KL-divergence between $p$ and $q$ is

   $$\begin{aligned}
   \mathrm{KL}(p \| q) &= \mathbb{E}_{\boldsymbol{x} \sim p}\left[ \log \frac{p(\boldsymbol{x})}{q(\boldsymbol{x})} \right] && \text{using the definition of KL-divergence (5.34)} \\
   &= \mathbb{E}_{\boldsymbol{x} \sim p}\left[ \mathrm{S}\left[ \frac{q(\boldsymbol{x})}{p(\boldsymbol{x})} \right] \right].
   \end{aligned}$$

Note that the surprise $\mathrm{S}[u] = -\log u$ is a convex function in $u$, and hence,

$$\begin{aligned}
&\geq \mathrm{S}\left[\mathbb{E}_{\boldsymbol{x}\sim p}\left[\frac{q(\boldsymbol{x})}{p(\boldsymbol{x})}\right]\right] && \text{using Jensen's inequality (5.29)}\\
&= \mathrm{S}\left[\int q(\boldsymbol{x})\, d\boldsymbol{x}\right] \\
&= \mathrm{S}[1] = 0. && \text{a probability density integrates to 1}
\end{aligned}$$

2. We observe from the derivation of (1) that $\mathrm{KL}(p\|q) = 0$ iff equality holds for Jensen's inequality. Now, if $p$ and $q$ are discrete with final and identical support, we can follow from the hint that Jensen's inequality degenerates to an equality iff $p$ and $q$ are point wise identical.

**Solution to Problem 5.6.**

1. We have

$$\begin{aligned}
\mathrm{H}[f\|g] &= -\int_{\mathbb{R}} f(x) \log g(x)\, dx && \text{using the definition of cross-entropy (5.32)}\\
&= -\int_{\mathbb{R}} f(x) \cdot \left(\log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \frac{(x-\mu)^2}{2\sigma^2}\right) dx && \text{using that } g(x) = \mathcal{N}(x;\mu,\sigma^2)\\
&= -\log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) \underbrace{\int_{\mathbb{R}} f(x)\, dx}_{1} + \frac{1}{2\sigma^2}\int_{\mathbb{R}} f(x)(x-\mu)^2\, dx \\
&= \log(\sigma\sqrt{2\pi}) + \frac{1}{2\sigma^2}\underbrace{\mathbb{E}_{x\sim f}\left[(x-\mu)^2\right]}_{\sigma^2} \\
&= \log(\sigma\sqrt{2\pi}) + \frac{1}{2} \\
&= \mathrm{H}[g]. && \text{using the entropy of Gaussians (5.30)}
\end{aligned}$$

2. We have shown that

$$\mathrm{H}[g] - \mathrm{H}[f] = \mathrm{KL}(f\|g) \geq 0,$$

and hence, $\mathrm{H}[g] \geq \mathrm{H}[f]$. That is, for a fixed mean $\mu$ and variance $\sigma^2$, the distribution that has maximum entropy among all distributions on $\mathbb{R}$ is the normal distribution.

**Solution to Problem 5.7.**

1. This sample solution follows the works of Caticha and Giffin (2006); Caticha (2021). Writing down the Lagrangian with dual variables

$\lambda_0$ and $\lambda_1(\boldsymbol{y})$ for the normalization and data constraints yields

$$
\begin{aligned}
L(q, \lambda_0, \lambda_1) = & \int q(\boldsymbol{x}, \boldsymbol{y}) \log \frac{q(\boldsymbol{x}, \boldsymbol{y})}{p(\boldsymbol{x}, \boldsymbol{y})} \, d\boldsymbol{x} \, d\boldsymbol{y} && \text{the objective, using (5.34)} \\
& + \lambda_0 \left(1 - \int q(\boldsymbol{x}, \boldsymbol{y}) \, d\boldsymbol{x} \, d\boldsymbol{y}\right) && \text{the normalization constraint} \\
& + \int \lambda_1(\boldsymbol{y}) \left[\delta_{\boldsymbol{y}'}(\boldsymbol{y}) - \int q(\boldsymbol{x}, \boldsymbol{y}) \, d\boldsymbol{x}\right] d\boldsymbol{y} && \text{the data constraint} \\
= & \int q(\boldsymbol{x}, \boldsymbol{y}) \left[\log \frac{q(\boldsymbol{x}, \boldsymbol{y})}{p(\boldsymbol{x}, \boldsymbol{y})} - \lambda_0 - \lambda_1(\boldsymbol{y})\right] d\boldsymbol{x} \, d\boldsymbol{y} + \text{const.}
\end{aligned}
$$

Note that $L(q, \lambda_0, \lambda_1) = \mathrm{KL}(q_{\mathbf{X},\mathbf{Y}} \| p_{\mathbf{X},\mathbf{Y}})$ if the constraints are satisfied. Thus, we simply need to solve the (dual) optimization problem

$$
\min_{q(\cdot,\cdot) \geq 0} \max_{\lambda_0, \lambda_1(\cdot) \in \mathbb{R}} L(q, \lambda_0, \lambda_1).
$$

We have

$$
\frac{\partial L(q, \lambda_0, \lambda_1)}{\partial q(\boldsymbol{x}, \boldsymbol{y})} = \log \frac{q(\boldsymbol{x}, \boldsymbol{y})}{p(\boldsymbol{x}, \boldsymbol{y})} - \lambda_0 - \lambda_1(\boldsymbol{y}) + 1. \qquad \text{using the product rule of differentiation}
$$

Setting the partial derivatives to zero, we obtain

$$
q(\boldsymbol{x}, \boldsymbol{y}) = \exp(\lambda_0 + \lambda_1(\boldsymbol{y}) - 1) p(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{Z} \exp(\lambda_1(\boldsymbol{y})) p(\boldsymbol{x}, \boldsymbol{y})
$$

where $Z \doteq \exp(\lambda_0 - 1)$ denotes the normalizing constant. We can determine $\lambda_1(\boldsymbol{y})$ from the data constraint:

$$
\begin{aligned}
\int q(\boldsymbol{x}, \boldsymbol{y}) \, d\boldsymbol{x} &= \frac{1}{Z} \exp(\lambda_1(\boldsymbol{y})) \int p(\boldsymbol{x}, \boldsymbol{y}) \, d\boldsymbol{x} \\
&= \frac{1}{Z} \exp(\lambda_1(\boldsymbol{y})) \cdot p(\boldsymbol{y}) && \text{using the sum rule (1.7)} \\
&\overset{!}{=} \delta_{\boldsymbol{y}'}(\boldsymbol{y}).
\end{aligned}
$$

It follows that $q(\boldsymbol{x}, \boldsymbol{y}) = \delta_{\boldsymbol{y}'}(\boldsymbol{y}) \cdot \frac{p(\boldsymbol{x}, \boldsymbol{y})}{p(\boldsymbol{y})} = \delta_{\boldsymbol{y}'}(\boldsymbol{y}) \cdot p(\boldsymbol{x} \mid \boldsymbol{y})$. using the definition of conditional distributions (1.10)

2. From the sum rule (1.7), we obtain

$$
q(\boldsymbol{x}) = \int q(\boldsymbol{x}, \boldsymbol{y}) \, d\boldsymbol{y} = \int \delta_{\boldsymbol{y}'}(\boldsymbol{y}) \cdot p(\boldsymbol{x} \mid \boldsymbol{y}) \, d\boldsymbol{y} = p(\boldsymbol{x} \mid \boldsymbol{y}').
$$

**Solution to Problem 5.8.** We can rewrite the KL-divergence as

$$
\begin{aligned}
\mathrm{KL}(p \| q) &= \mathbb{E}_{\boldsymbol{x} \sim p}[\log p(\boldsymbol{x}) - \log q(\boldsymbol{x})] && \text{using the definition of KL-divergence (5.34)} \\
&= \mathbb{E}_{\boldsymbol{x} \sim p} \left[\frac{1}{2} \log \frac{\det(\boldsymbol{\Sigma}_q)}{\det(\boldsymbol{\Sigma}_p)} - \frac{1}{2} (\boldsymbol{x} - \boldsymbol{\mu}_p)^\top \boldsymbol{\Sigma}_p^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_p)\right. && \text{using the Gaussian PDF (1.51)} \\
&\qquad \left. + \frac{1}{2} (\boldsymbol{x} - \boldsymbol{\mu}_q)^\top \boldsymbol{\Sigma}_q^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_q)\right]
\end{aligned}
$$

$$= \frac{1}{2}\log\frac{\det(\boldsymbol{\Sigma}_q)}{\det(\boldsymbol{\Sigma}_p)} - \frac{1}{2}\mathbb{E}_{\boldsymbol{x}\sim p}\Big[(\boldsymbol{x}-\boldsymbol{\mu}_p)^\top \boldsymbol{\Sigma}_p^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_p)\Big] \qquad \text{using linearity of expectation (1.20)}$$
$$+ \frac{1}{2}\mathbb{E}_{\boldsymbol{x}\sim p}\Big[(\boldsymbol{x}-\boldsymbol{\mu}_q)^\top \boldsymbol{\Sigma}_q^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_q)\Big]$$

As $(\boldsymbol{x}-\boldsymbol{\mu}_p)^\top \boldsymbol{\Sigma}_p^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_p) \in \mathbb{R}$, we can rewrite the second term as

$$\begin{aligned}
&\frac{1}{2}\mathbb{E}_{\boldsymbol{x}\sim p}\Big[\mathrm{tr}\Big((\boldsymbol{x}-\boldsymbol{\mu}_p)^\top \boldsymbol{\Sigma}_p^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_p)\Big)\Big] \\
&= \frac{1}{2}\mathbb{E}_{\boldsymbol{x}\sim p}\Big[\mathrm{tr}\Big((\boldsymbol{x}-\boldsymbol{\mu}_p)(\boldsymbol{x}-\boldsymbol{\mu}_p)^\top \boldsymbol{\Sigma}_p^{-1}\Big)\Big] && \text{using the cyclic property of the trace} \\
&= \frac{1}{2}\mathrm{tr}\Big(\mathbb{E}_{\boldsymbol{x}\sim p}\Big[(\boldsymbol{x}-\boldsymbol{\mu}_p)(\boldsymbol{x}-\boldsymbol{\mu}_p)^\top\Big]\boldsymbol{\Sigma}_p^{-1}\Big) && \text{using linearity of the trace and linearity of expectation (1.20)} \\
&= \frac{1}{2}\mathrm{tr}\Big(\boldsymbol{\Sigma}_p\boldsymbol{\Sigma}_p^{-1}\Big) && \text{using the definition of the covariance matrix (1.34)} \\
&= \frac{1}{2}\mathrm{tr}(\boldsymbol{I}) = \frac{d}{2}.
\end{aligned}$$

For the third term, we use the hint (5.81) to obtain

$$\begin{aligned}
&\frac{1}{2}\mathbb{E}_{\boldsymbol{x}\sim p}\Big[(\boldsymbol{x}-\boldsymbol{\mu}_q)^\top \boldsymbol{\Sigma}_q^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_q)\Big] \\
&= \frac{1}{2}\Big((\boldsymbol{\mu}_p-\boldsymbol{\mu}_q)^\top \boldsymbol{\Sigma}_q^{-1}(\boldsymbol{\mu}_p-\boldsymbol{\mu}_q) + \mathrm{tr}\Big(\boldsymbol{\Sigma}_q^{-1}\boldsymbol{\Sigma}_p\Big)\Big).
\end{aligned}$$

Putting all terms together we get

$$\mathrm{KL}(p\|q) = \frac{1}{2}\left(\log\frac{\det(\boldsymbol{\Sigma}_q)}{\det(\boldsymbol{\Sigma}_p)} - d + (\boldsymbol{\mu}_p-\boldsymbol{\mu}_q)^\top \boldsymbol{\Sigma}_q^{-1}(\boldsymbol{\mu}_p-\boldsymbol{\mu}_q) + \mathrm{tr}\Big(\boldsymbol{\Sigma}_q^{-1}\boldsymbol{\Sigma}_p\Big)\right).$$

**Solution to Problem 5.9.**

1. Let $p$ and $q$ be discrete distributions. The derivation is analogous if $p$ and $q$ are taken to be continuous. First, we write the KL-divergence between $p$ and $q$ as

$$\begin{aligned}
&\mathrm{KL}(p\|q) \\
&= \sum_x\sum_y p(x,y)\log_2\frac{p(x,y)}{q(x)q(y)} && \text{using the definition of KL-divergence (5.34)} \\
&= \sum_x\sum_y p(x,y)\log_2 p(x,y) - \sum_x\sum_y p(x,y)\log_2 q(x) \\
&\qquad - \sum_x\sum_y p(x,y)\log_2 q(y) \\
&= \sum_x\sum_y p(x,y)\log_2 p(x,y) - \sum_x p(x)\log_2 q(x) - \sum_y p(y)\log_2 q(y) && \text{using the sum rule (1.7)} \\
&= -\mathrm{H}[p(x,y)] + \mathrm{H}[p(x)\|q(x)] + \mathrm{H}[p(y)\|q(y)] && \text{using the definitions of entropy (5.27) and cross-entropy (5.32)} \\
&= -\mathrm{H}[p(x,y)] + \mathrm{H}[p(x)] + \mathrm{H}[p(y)] + \mathrm{KL}(p(x)\|q(x)) && \text{using Equation (5.33)} \\
&\qquad + \mathrm{KL}(p(y)\|q(y))
\end{aligned}$$

$= \mathrm{KL}(p(x)\|q(x)) + \mathrm{KL}(p(y)\|q(y)) + \text{const.}$

Hence, to minimize $\mathrm{KL}(p\|q)$ with respect to the variational distributions $q(x)$ and $q(y)$ we should set $\mathrm{KL}(p(x)\|q(x)) = 0$ and $\mathrm{KL}(p(y)\|q(y)) = 0$, respectively. This is obtained when

$$q(x) = p(x) \qquad \text{and} \qquad q(y) = p(y).$$

2. The reverse KL-divergence $\mathrm{KL}(q\|p)$ on the finite domain $x, y \in \{1, 2, 3, 4\}$ is defined as

$$\mathrm{KL}(q\|p) = \sum_x \sum_y q(x)q(y) \log_2 \frac{q(x)q(y)}{p(x,y)}.$$

We can easily observe from the above formula that the support of $q$ must be a subset of the support of $p$. In other words, if $q(x, y)$ is positive outside the support of $p$ (i.e., when $p(x, y) = 0$) then $\mathrm{KL}(q\|p) = \infty$. Hence, the reverse KL-divergence has an infinite value except when the support of $q$ is either $\{1, 2\} \times \{1, 2\}$ or $\{(3, 3)\}$ or $\{(4, 4)\}$. Thus, it has three local minima.
For the first case, the minimum is achieved when $q(x) = q(y) = (\frac{1}{2}, \frac{1}{2}, 0, 0)$. The corresponding KL-divergence is $\mathrm{KL}(q\|p) = \log_2 2 = 1$. For the second case and the third case, $q(x) = q(y) = (0, 0, 1, 0)$ and $q(x) = q(y) = (0, 0, 0, 1)$, respectively. The KL-divergence in both cases is $\mathrm{KL}(q\|p) = \log_2 4 = 2$.

3. Let us compute $p(x = 4)$ and $p(y = 1)$:

$$p(x = 4) = \sum_y p(x = 4, y) = \frac{1}{4},$$
$$p(y = 1) = \sum_x p(x, y = 1) = \frac{1}{4}.$$

Hence, $q(x = 4, y = 1) = p(x = 4)p(y = 1) = \frac{1}{16}$, however, $p(x = 4, y = 1) = 0$. We therefore have for the reverse KL-divergence that $\mathrm{KL}(q\|p) = \infty$.

**Solution to Problem 5.10.**

1. Recall from Section 5.4.6 that $q^\star$ matches the first and second *moments* of $p$. In contrast, the Laplace approximation matches the *mode* of $p$ and the second derivative of $-\log p$. In general, the mean is different from the mode, and so is the second moment from the second derivative.
2. We have

$$\begin{aligned} &\arg\min_{q \in \mathcal{Q}} \mathrm{KL}(q\|p(\cdot \mid \mathcal{D})) \\ &= \arg\max_{q \in \mathcal{Q}} L(q, p; \mathcal{D}) && \text{using Equation (5.53)} \end{aligned}$$

$$= \arg\max_{q \in \mathcal{Q}} \mathbb{E}_{\boldsymbol{\theta} \sim q}[\log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})] + \mathrm{H}[q] \qquad \text{using Equation (5.54)}$$

$$= \arg\max_{q \in \mathcal{Q}} \mathbb{E}_{\boldsymbol{\theta} \sim q}[\log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})] + \frac{n}{2}\log(2\pi e) + \frac{1}{2}\log\det(\boldsymbol{\Sigma}). \qquad \text{using Equation (5.31)}$$

Differentiating with respect to $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$, we have that $\tilde{q}$ must satisfy

$$\begin{aligned} \mathbf{0} &= \boldsymbol{\nabla}_{\boldsymbol{\mu}} \mathbb{E}_{\boldsymbol{\theta} \sim \tilde{q}}[\log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})] \\ \boldsymbol{\Sigma}^{-1} &= -2\boldsymbol{\nabla}_{\boldsymbol{\Sigma}} \mathbb{E}_{\boldsymbol{\theta} \sim \tilde{q}}[\log p(y_{1:n}, \boldsymbol{\theta} \mid \boldsymbol{x}_{1:n})]. \qquad \text{using the first hint} \end{aligned}$$

The result follows by Equation (5.82).

**Solution to Problem 5.11.** To simplify the notation, we write $\boldsymbol{\Sigma} \doteq \mathrm{diag}\{\sigma_1^2, \dots, \sigma_d^2\}$. The reverse KL-divergence can be expressed as

$$\begin{aligned} \mathrm{KL}(q_{\boldsymbol{\lambda}} \| p(\cdot)) &= \frac{1}{2}\left(\mathrm{tr}\left(\sigma_{\mathrm{p}}^{-2}\boldsymbol{\Sigma}\right) + \sigma_{\mathrm{p}}^{-2}\boldsymbol{\mu}^\top\boldsymbol{\mu} - d + \log\frac{(\sigma_{\mathrm{p}}^2)^d}{\det(\boldsymbol{\Sigma})}\right) \qquad \text{using the expression for the KL-divergence of Gaussians (5.41)} \\ &= \frac{1}{2}\left(\sigma_{\mathrm{p}}^{-2}\sum_{i=1}^d \sigma_i^2 + \sigma_{\mathrm{p}}^{-2}\boldsymbol{\mu}^\top\boldsymbol{\mu} - d + d\log\sigma_{\mathrm{p}}^2 - \sum_{i=1}^d \log\sigma_i^2\right). \end{aligned}$$

It follows immediately that $\boldsymbol{\nabla}_{\boldsymbol{\mu}} \mathrm{KL}(q_{\boldsymbol{\lambda}} \| p(\cdot)) = \sigma_{\mathrm{p}}^{-2}\boldsymbol{\mu}$. Moreover,

$$\frac{\partial}{\partial\sigma_i}\mathrm{KL}(q_{\boldsymbol{\lambda}} \| p(\cdot)) = \frac{1}{2}\left(\sigma_{\mathrm{p}}^{-2}\underbrace{\frac{\partial}{\partial\sigma_i}\sigma_i^2}_{2\sigma_i} - \underbrace{\frac{\partial}{\partial\sigma_i}\log\sigma_i^2}_{2/\sigma_i}\right) = \frac{\sigma_i}{\sigma_{\mathrm{p}}^2} - \frac{1}{\sigma_i}.$$

**Solution to Problem 5.12.**

1. Let $Y \sim \mathrm{Unif}([0,1])$. Then, using the two hints,

$$(b-a)Y + a \sim \mathrm{Unif}([a,b]).$$

2. Let $Z_1 \sim \mathcal{N}(\mu, \sigma^2)$ and $Z_2 \sim \mathcal{N}(0,1)$. We have, $X = e^{Z_1}$. Recall from Equation (1.54) that $Z_1$ can equivalently be expressed in terms of $Z_2$ as $Z_1 = \sigma Z_2 + \mu$. This yields,

$$X = e^{Z_1} = e^{\sigma Z_2 + \mu}.$$

3. Denote by $F$ the CDF of $\mathrm{Cauchy}(0,1)$. Observe that $F$ is invertible with inverse $F^{-1}(y) = \tan(\pi(y - \frac{1}{2}))$. Let $Y \sim \mathrm{Unif}([0,1])$ and write $X = F^{-1}(Y)$. Then,

$$\begin{aligned} P_X(x) &= \mathbb{P}(X \leq x) \\ &= \mathbb{P}\left(F^{-1}(Y) \leq x\right) \\ &= \mathbb{P}(Y \leq F(x)) \\ &= F(x). \end{aligned}$$

This reparameterization works for any distribution with invertible CDF (not just Cauchy) and is known as the *universality of the uniform* (cf. Section A.1.3). The universality of the uniform is commonly used in pseudo-random number generators as it allows "lifting" samples from a uniform distribution to countless other well-known distributions.

4. The derivative of $\mathrm{ReLU}(z)$ is $\mathbb{1}\{z > 0\}$. Applying the reparameterization trick gives

$$\begin{aligned}
\frac{d}{d\mu}\mathbb{E}_{x\sim\mathcal{N}(\mu,1)}\mathrm{ReLU}(wx) &= \mathbb{E}_{\varepsilon\sim\mathcal{N}(0,1)}\frac{d}{d\mu}\mathrm{ReLU}(w(\mu+\varepsilon)) \\
&= w\mathbb{E}_\varepsilon \mathbb{1}\{w(\mu+\varepsilon) > 0\} && \text{using the chain rule} \\
&= w\mathbb{E}_\varepsilon \mathbb{1}\{\mu+\varepsilon > 0\} \\
&= w\mathbb{P}_\varepsilon(\mu+\varepsilon > 0) \\
&= w\mathbb{P}_\varepsilon(\varepsilon < \mu) \\
&= w\mathbb{P}_\varepsilon(\varepsilon \leq \mu) \\
&= w\Phi(\mu).
\end{aligned}$$

## *Markov Chain Monte Carlo Methods*

**Solution to Problem 6.1.** We have

$$q_{t+1}(x') = \mathbb{P}\big(X_{t+1} = x'\big) = \sum_x \underbrace{\mathbb{P}(X_t = x)}_{q_t(x)} \underbrace{\mathbb{P}\big(X_{t+1} = x' \mid X_t = x\big)}_{p(x'|x)}. \qquad \text{using the sum rule (1.7)}$$

Noting that $p(x' \mid x) = \boldsymbol{P}(x, x')$, we conclude $\boldsymbol{q}_{t+1} = \boldsymbol{q}_t\boldsymbol{P}$.

**Solution to Problem 6.2.** It follows directly from the definition of matrix multiplication that

$$\begin{aligned}
\boldsymbol{P}^k(x, x') &= \sum_{x_1,\ldots,x_{k-1}} \boldsymbol{P}(x, x_1) \cdot \boldsymbol{P}(x_1, x_2) \cdots \boldsymbol{P}(x_{k-1}, x') \\
&= \sum_{x_1,\ldots,x_{k-1}} \mathbb{P}(X_1 = x_1 \mid X_0 = x) \cdots \mathbb{P}\big(X_k = x' \mid X_{k-1} = x_{k-1}\big) && \text{using the definition of the transition matrix (6.8)} \\
&= \sum_{x_1,\ldots,x_{k-1}} \mathbb{P}\big(X_1 = x_1, \cdots, X_{k-1} = x_{k-1}, X_k = x' \mid X_0 = x\big) && \text{using the product rule (1.11)} \\
&= \mathbb{P}\big(X_k = x' \mid X_0 = x\big). && \text{using the sum rule (1.7)}
\end{aligned}$$

**Solution to Problem 6.3.** We consider the transition matrix

$$\boldsymbol{P} = \begin{bmatrix} 0.60 & 0.30 & 0.10 \\ 0.50 & 0.25 & 0.25 \\ 0.20 & 0.40 & 0.40 \end{bmatrix}.$$

We note that the entries of $\boldsymbol{P}$ are all different from 0, thus the Markov chain corresponding to this transition matrix is ergodic.[4] Thus, there

[4] All elements of the transition matrix being strictly greater than 0 is a sufficient, but not necessary, condition for ergodicity.

exists a unique stationary distribution $\pi$ to which the Markov chain converges irrespectively of the distribution over initial states $q_0$.

We know that $\boldsymbol{P}^\top \boldsymbol{\pi} = \boldsymbol{\pi}$ (where we write $\boldsymbol{\pi}$ as a column vector), therefore, to find the stationary distribution $\pi$, we need to find the normalized eigenvector associated with eigenvalue 1 of the matrix $\boldsymbol{P}^\top$. That is, we want to solve $(\boldsymbol{P}^\top - \boldsymbol{I})\boldsymbol{\pi} = \boldsymbol{0}$ for $\boldsymbol{\pi}$. We obtain the linear system of equations,

$$\begin{aligned} -0.40\pi_1 + 0.50\pi_2 + 0.20\pi_3 &= 0 \\ 0.30\pi_1 - 0.75\pi_2 + 0.40\pi_3 &= 0 \\ 0.10\pi_1 + 0.25\pi_2 - 0.60\pi_3 &= 0. \end{aligned}$$

Note that the left hand side of equation $i$ corresponds to the probability of entering state $i$ at stationarity minus $\pi_i$. Quite intuitively, this difference should be 0, that is, after one iteration the random walk is at state $i$ with the same probability as before the iteration.

Solving this system of equations, for example, using the Gaussian elimination algorithm, we obtain the normalized eigenvector

$$\boldsymbol{\pi} = \frac{1}{72} \begin{bmatrix} 35 \\ 22 \\ 15 \end{bmatrix}.$$

Thus, we conclude that in the long run, the percentage of news days that will be classified as "good" is $35/72$.

**Solution to Problem 6.4.** Observe that the described proposal distribution is symmetric. Therefore, the acceptance probability simplifies to

$$\alpha(x' \mid x) = \min\left\{1, \frac{p(x')}{p(x)}\right\}.$$

If we denote the number of 1s in a bit string by $w(x)$, we have the requirement that $p(x) \propto w(x)$. Therefore, the acceptance probability becomes

$$\alpha(x' \mid x) = \begin{cases} \min\left\{1, \frac{w(x')}{w(x)}\right\} & \text{if } w(x) \neq 0 \\ 1 & \text{otherwise.} \end{cases}$$

**Solution to Problem 6.5.**

1. We have to compute the conditional distributions. Notice that for $x \in \{0, \ldots, n\}$ and $y \in [0, 1]$,

$$p(x, y) = \binom{n}{x} y^x (1-y)^{n-x} \cdot y^{\alpha-1}(1-y)^{\beta-1} = \mathrm{Bin}(x; n, y) \cdot C_y,$$

where $\text{Bin}(n, y)$ is the PMF of the binomial distribution (1.49) with $n$ trials and success probability $y$, and $C_y$ is a constant depending on $y$. It is clear that

$$\begin{aligned} p(x \mid y) &= \frac{p(x,y)}{p(y)} && \text{using the definition of conditional probability (1.8)} \\ &= \frac{\text{Bin}(x; n, y) \cdot C_y}{p(y)} \\ &= \text{Bin}(x; n, y). && \text{using that } p(x \mid y) \text{ is a probability distribution over } x \text{ and } \text{Bin}(x; n, y) \text{ already sums to 1, so } C_y = p(y) \end{aligned}$$

So in short, sampling from $p(x \mid y)$ is equivalent to sampling from a binomial distribution, which amounts to $n$ times throwing a coin with bias $y$, and outputting the number of heads.
For the other conditional distribution, recall the PMF of the beta distribution with parameters $\alpha, \beta$,

$$\text{Beta}(y; \alpha, \beta) = C \cdot y^{\alpha-1}(1-y)^{\beta-1}$$

where $C$ is some constant depending on $\alpha$ and $\beta$ only. We then have

$$p(x, y) = \text{Beta}(y; x + \alpha, n - x + \beta) \cdot C_x,$$

where $C_x$ is some constant depending on $x, \alpha, \beta$. This shows (analogously to above), that

$$p(y \mid x) = \text{Beta}(y; x + \alpha, n - x + \beta).$$

So for sampling $y$ given $x$, one can sample from the beta distribution. There are several methods for sampling from a beta distribution, and we refer the reader to the corresponding Wikipedia page.

2. We first derive the posterior distribution of $\mu$. We have

$$\begin{aligned} \log p(\mu \mid \lambda, x_{1:n}) &= \log p(\mu) + \log p(x_{1:n} \mid \mu, \lambda) + \text{const} \\ &= -\frac{\lambda_0}{2}(\mu - \mu_0)^2 - \frac{\lambda}{2}\sum_{i=1}^{n}(x_i - \mu)^2 + \text{const} \\ &= -\frac{1}{2}(\underbrace{\lambda_0 + n\lambda}_{l_\lambda})\mu^2 + \Big(\underbrace{\lambda_0\mu_0 + \lambda\sum_{i=1}^{n} x_i}_{m_\lambda l_\lambda}\Big)\mu + \text{const}. && \text{by expanding the squares} \end{aligned}$$

It follows that $l_\lambda = \lambda_0 + n\lambda, m_\lambda = (\lambda_0\mu_0 + \lambda\sum_{i=1}^{n} x_i)/l_\lambda$, and $\mu \mid \lambda, x_{1:n} \sim \mathcal{N}(\mu_\lambda, l_\lambda^{-1})$. That is, the posterior precision is the sum of prior precision and the precisions of each observation, and the posterior mean is a weighted average of prior mean and observations (where the weights are the precisions).

For the posterior distribution of $\lambda$, we have

$$\begin{aligned} p(\lambda \mid \mu, x_{1:n}) &\propto p(\lambda) \cdot p(x_{1:n} \mid \mu, \lambda) \\ &= \lambda^{\alpha-1} e^{-\beta\lambda} \cdot \lambda^{\frac{n}{2}} e^{-\frac{\lambda}{2}\sum_{i=1}^n (x_i-\mu)^2} \\ &= \lambda^{\alpha+\frac{n}{2}-1} e^{-\lambda\left(\beta+\frac{1}{2}\sum_{i=1}^n (x_i-\mu)^2\right)} \\ &= \text{Gamma}(\lambda; a_\mu, b_\mu) \end{aligned}$$

where $a_\mu = \alpha + \frac{n}{2}$ and $b_\mu = \beta + \frac{1}{2}\sum_{i=1}^n (x_i - \mu)^2$.

3. We have

$$\begin{aligned} p(\alpha, c \mid x_{1:n}) &\propto p(\alpha, c) \cdot p(x_{1:n} \mid \alpha, c) \\ &\propto \mathbb{1}\{\alpha, c > 0\} \prod_{i=1}^n \frac{\alpha c^\alpha}{x_i^{\alpha+1}} \mathbb{1}\{x_i \geq c\} \\ &= \frac{\alpha^n c^{n\alpha}}{(\prod_{i=1}^n x_i)^{\alpha+1}} \mathbb{1}\{c < x^\star\} \mathbb{1}\{\alpha, c > 0\} \end{aligned}$$

where $x^\star \doteq \min\{x_1, \ldots, x_n\}$. It is not clear how one could sample from this distribution directly. Instead, we use Gibbs sampling. For the posterior distribution of $\alpha$, we have

$$\begin{aligned} p(\alpha \mid c, x_{1:n}) &\propto p(\alpha, c \mid x_{1:n}) \\ &\propto \frac{\alpha^n c^{n\alpha}}{(\prod_{i=1}^n x_i)^{\alpha+1}} \mathbb{1}\{\alpha > 0\} \\ &= \alpha^n \exp\left(-\alpha\left(\sum_{i=1}^n \log x_i - n \log c\right)\right) \mathbb{1}\{\alpha > 0\} \\ &\propto \text{Gamma}(\alpha; a, b) \end{aligned}$$

where $a \doteq n + 1$ and $b \doteq \sum_{i=1}^n \log x_i - n \log c$. On the other hand,

$$p(c \mid \alpha, x_{1:n}) \propto p(\alpha, c \mid x_{1:n}) \propto c^{n\alpha} \mathbb{1}\{0 < c < x^\star\}.$$

It remains to show that it is easy to sample from a random variable $X$ with $p_X(x) \propto x^a \mathbb{1}\{0 < x < b\}$ for $a, b > 0$. We have that the normalizing constant is given by $\int_0^b x^a \, dx = \frac{b^{a+1}}{a+1}$. Therefore, the CDF of $X$ is

$$\begin{aligned} P_X(x) &= \int_0^x p(y) \, dy \\ &= \frac{a+1}{b^{a+1}} \int_0^x y^a \, dy \\ &= \frac{a+1}{b^{a+1}} \frac{x^{a+1}}{a+1} \\ &= \left(\frac{x}{b}\right)^{a+1}. \end{aligned}$$

The inverse CDF is given by $P_X^{-1}(y) = by^{\frac{1}{a+1}}$. Therefore, we can sample from $X$ using inverse transform sampling (cf. Section A.1.3) by sampling $Y \sim \text{Unif}([0, 1])$ and setting $X = bY^{\frac{1}{a+1}}$.

**Solution to Problem 6.6.** First, note that the sum of convex functions is convex, hence, we consider each term individually.

The Hessian of the regularization term is $\lambda \boldsymbol{I}$, and thus, by the second-order characterization of convexity, this term is convex in $\boldsymbol{w}$.

Finally, note that the second term is a sum of logistic losses $\ell_{\log}$ (5.13), and we have seen in Problem 5.1 that $\ell_{\log}$ is convex in $\boldsymbol{w}$.

**Solution to Problem 6.7.**

1. Our goal is to solve the optimization problem

$$\begin{aligned} &\max_{p \in \Delta^{\mathcal{T}}} -\sum_{x \in \mathcal{T}} p(x) \log_2 p(x) \\ &\text{subject to} \sum_{x \in \mathcal{T}} p(x) f(x) = \mu \end{aligned} \tag{B.9}$$

   for some $\mu \in \mathbb{R}$. The Lagrangian with dual variables $\lambda_0$ and $\lambda_1$ is given by

$$\begin{aligned} L(p, \lambda_0, \lambda_1) = &-\sum_{x \in \mathcal{T}} p(x) \log_2 p(x) + \lambda_0 \left(1 - \sum_{x \in \mathcal{T}} p(x)\right) + \\ &\lambda_1 \left(\mu - \sum_{x \in \mathcal{T}} p(x) f(x)\right) \\ = &-\sum_{x \in \mathcal{T}} p(x)(\log_2 p(x) + \lambda_0 + \lambda_1 f(x)) + \text{const.} \end{aligned}$$

   Note that $L(p, \lambda_0, \lambda_1) = \mathrm{H}[p]$ if the constraints are satisfied. Thus, we simply need to solve the (dual) optimization problem

$$\max_{\boldsymbol{p} \geq \boldsymbol{0}} \min_{\lambda_0, \lambda_1 \in \mathbb{R}} L(\boldsymbol{p}, \lambda_0, \lambda_1).$$

   We have

$$\frac{\partial}{\partial p(x)} L(p, \lambda_0, \lambda_1) = -\log_2 p(x) - \lambda_0 - \lambda_1 f(x) - 1.$$

   Setting the partial derivatives to zero, we obtain

$$\begin{aligned} p(x) &= 2 \exp(-\lambda_0 - \lambda_1 f(x) - 1) \\ &\propto \exp(-\lambda_1 f(x)). \end{aligned}$$

   using $\log_2(\cdot) = \frac{\log(\cdot)}{\log(2)}$

   Clearly, $p$ is a valid probability mass function when normalized (i.e., for an appropriate choice of $\lambda_0$). We complete the proof by setting $T \doteq \frac{1}{\lambda_1}$.

2. As $T \to \infty$ (and $\lambda_1 \to 0$), the optimization problem reduces to picking the maximum entropy distribution without the first-moment constraint. This distribution is the uniform distribution over $\mathcal{T}$. Conversely, as $T \to 0$ (and $\lambda_1 \to \infty$), the Gibbs distribution reduces to a point density around its mode.

**Solution to Problem 6.8.** Recall the following two facts:

1. Gibbs sampling is an instance of Metropolis-Hastings with proposal distribution

$$r(\boldsymbol{x}' \mid \boldsymbol{x}) = \begin{cases} p(x_i' \mid \boldsymbol{x}'_{-i}) & \text{if } \boldsymbol{x}' \text{ differs from } \boldsymbol{x} \text{ only in entry } i \\ 0 & \text{otherwise} \end{cases}$$

   and acceptance distribution $\alpha(\boldsymbol{x}' \mid \boldsymbol{x}) = 1$.
2. The acceptance distribution of Metropolis-Hastings where the stationary distribution $p$ is a Gibbs distribution with energy function $f$ is

$$\alpha(\boldsymbol{x}' \mid \boldsymbol{x}) = \min\left\{1, \frac{r(\boldsymbol{x} \mid \boldsymbol{x}')}{r(\boldsymbol{x}' \mid \boldsymbol{x})} \exp(f(\boldsymbol{x}) - f(\boldsymbol{x}'))\right\}.$$

We therefore know that

$$\frac{r(\boldsymbol{x} \mid \boldsymbol{x}')}{r(\boldsymbol{x}' \mid \boldsymbol{x})} \exp(f(\boldsymbol{x}) - f(\boldsymbol{x}')) \geq 1.$$

We remark that this inequality even holds with equality using our derivation of Theorem 6.19. Taking the logarithm and reorganizing the terms, we obtain

$$f(\boldsymbol{x}') \leq f(\boldsymbol{x}) + \log r(\boldsymbol{x} \mid \boldsymbol{x}') - \log r(\boldsymbol{x}' \mid \boldsymbol{x}). \tag{B.10}$$

By the definition of the proposal distribution of Gibbs sampling,

$$r(\boldsymbol{x}' \mid \boldsymbol{x}) = p(x_i' \mid \boldsymbol{x}_{-i}) \quad \text{and} \quad r(\boldsymbol{x} \mid \boldsymbol{x}') = p(x_i \mid \boldsymbol{x}_{-i}). \qquad \text{using } \boldsymbol{x}'_{-i} = \boldsymbol{x}_{-i}$$

Taking the expectation of Equation (B.10),

$$\begin{aligned} \mathbb{E}_{x_i' \sim p(\cdot \mid \boldsymbol{x}_{-i})}\left[f(\boldsymbol{x}')\right] &\leq f(\boldsymbol{x}) + \log p(x_i \mid \boldsymbol{x}_{-i}) - \mathbb{E}_{x_i' \sim p(\cdot \mid \boldsymbol{x}_{-i})}\left[\log p(x_i' \mid \boldsymbol{x}_{-i})\right] \\ &= f(\boldsymbol{x}) - \mathrm{S}[p(x_i \mid \boldsymbol{x}_{-i})] + \mathrm{H}[p(\cdot \mid \boldsymbol{x}_{-i})]. \end{aligned}$$

That is, the energy is expected to decrease if the expected surprise of the new sample $x_i' \mid \boldsymbol{x}_{-i}$ is smaller than the surprise of the current sample $x_i \mid \boldsymbol{x}_{-i}$.

**Solution to Problem 6.9.**

1. We take the minimum of Equation (6.56) with respect to $\boldsymbol{y}$ on both sides. The minimum of the left-hand side is $f(\mathbf{0}) = 0$. To find the minimum of the right-hand side, we differentiate with respect to $\boldsymbol{y}$:

$$\frac{\partial}{\partial \boldsymbol{y}}\left[f(\boldsymbol{x}) + \boldsymbol{\nabla} f(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x}) + \frac{\alpha}{2} \|\boldsymbol{y} - \boldsymbol{x}\|_2^2\right] = \boldsymbol{\nabla} f(\boldsymbol{x}) + \alpha(\boldsymbol{y} - \boldsymbol{x}).$$

Setting the partial derivative to zero, we obtain

$$\boldsymbol{y} = \boldsymbol{x} - \frac{1}{\alpha}\boldsymbol{\nabla} f(\boldsymbol{x}).$$

Plugging this $\boldsymbol{y}$ into Equation (6.56), we have

$$\begin{aligned} 0 &\geq f(\boldsymbol{x}) - \frac{1}{\alpha}\left\|\boldsymbol{\nabla} f(\boldsymbol{x})\right\|_2^2 + \frac{1}{2\alpha}\left\|\boldsymbol{\nabla} f(\boldsymbol{x})\right\|_2^2 \\ &= f(\boldsymbol{x}) - \frac{1}{2\alpha}\left\|\boldsymbol{\nabla} f(\boldsymbol{x})\right\|_2^2. \end{aligned}$$

2. Using the chain rule,

$$\frac{d}{dt} f(\boldsymbol{x}_t) = \boldsymbol{\nabla} f(\boldsymbol{x}_t)^\top \frac{d}{dt}\boldsymbol{x}_t.$$

Note that $\frac{d}{dt}\boldsymbol{x}_t = -\boldsymbol{\nabla} f(\boldsymbol{x}_t)$ by Equation (6.55), so

$$\begin{aligned} &= -\left\|\boldsymbol{\nabla} f(\boldsymbol{x}_t)\right\|_2^2 \\ &\leq -2\alpha f(\boldsymbol{x}_t) \end{aligned}$$

where the last step follows from the PL-inequality (6.57).

3. Follows directly from (2) and Grönwall's inequality (6.58) by letting $g(t) = f(\boldsymbol{x}_t)$ and noting that $-\int_0^t 2\alpha\, ds = -2\alpha t$.
4. It suffices to show $\boldsymbol{\nabla} q_t = q_t \boldsymbol{\nabla} \log q_t$. By the chain rule,

$$\boldsymbol{\nabla} \log q_t = \frac{\boldsymbol{\nabla} q_t}{q_t}.$$

We obtain the desired result by rearranging the terms.

5. We have

$$\frac{d}{dt}\mathrm{KL}(q_t \| p) = \frac{d}{dt}\int q_t \log \frac{q_t}{p}\, d\boldsymbol{\theta}.$$

By the chain rule,

$$= \int \frac{\partial q_t}{\partial t} \log \frac{q_t}{p}\, d\boldsymbol{\theta} + \int q_t \frac{\partial}{\partial t} \log \frac{q_t}{p}\, d\boldsymbol{\theta}.$$

For the second term, we have

$$\int q_t \frac{\partial}{\partial t} \log \frac{q_t}{p}\, d\boldsymbol{\theta} = \int \frac{\partial q_t}{\partial t}\, d\boldsymbol{\theta} = \frac{d}{dt} \underbrace{\int q_t\, d\boldsymbol{\theta}}_{1} = 0.$$

Plugging in the Fokker-Planck equation (6.60) into the first term, we obtain

$$= \int \boldsymbol{\nabla} \cdot \left( q_t \boldsymbol{\nabla} \log \frac{q_t}{p} \right) \log \frac{q_t}{p}\, d\boldsymbol{\theta}.$$

Letting $\varphi \doteq \log \frac{q_t}{p}$ and $\mathbf{F} \doteq \nabla\varphi$, and then applying the hint, we have

$$\begin{aligned}
&= \int (\nabla \cdot q_t \mathbf{F}) \varphi \, d\boldsymbol{\theta} \\
&= -\int q_t \, \|\nabla\varphi\|_2^2 \, d\boldsymbol{\theta} \\
&= -\mathbb{E}_{\boldsymbol{\theta}\sim q_t}\left[\left\|\nabla \log \frac{q_t(\boldsymbol{\theta})}{p(\boldsymbol{\theta})}\right\|_2^2\right] \\
&= -\mathrm{J}(q_t \| p).
\end{aligned}$$

6. Noting that $p$ satisfies the LSI with constant $\alpha$ and combining with (5), we have that $\frac{d}{dt}\mathrm{KL}(q_t\|p) \leq -2\alpha\mathrm{KL}(q_t\|p)$. Observe that this result is analogous to the result derived in (2) and the LSI can intuitively be seen as the PL-inequality, but in the space of distributions. Analogously to (3), we obtain the desired convergence result by applying Grönwall's inequality (6.58).
7. By Pinsker's inequality (6.21), $\|q_t - p\|_{\mathrm{TV}} \leq e^{-\alpha t}\sqrt{2\mathrm{KL}(q_0\|p)}$. It follows from elementary algebra that $\|q_t - p\|_{\mathrm{TV}} \leq \epsilon$ if $t \geq \frac{1}{\alpha}\log\frac{C}{\epsilon}$ where $C \doteq \sqrt{2\mathrm{KL}(q_0\|p)}$. Thus, $\tau_{\mathrm{TV}}(\epsilon) \in O(\log(1/\epsilon))$.

**Solution to Problem 6.10.**

1. We show that under the dynamics (6.50), the Hamiltonian $H(\boldsymbol{x}, \boldsymbol{y})$ is a constant. In particular, $H(\boldsymbol{x}', \boldsymbol{y}') = H(\boldsymbol{x}, \boldsymbol{y})$. This directly implies that

$$\alpha((\boldsymbol{x}', \boldsymbol{y}') \mid (\boldsymbol{x}, \boldsymbol{y})) = \min\{1, \exp(H(\boldsymbol{x}', \boldsymbol{y}') - H(\boldsymbol{x}, \boldsymbol{y}))\} = 1.$$

To see why $H(\boldsymbol{x}, \boldsymbol{y})$ is constant, we compute

$$\begin{aligned}
\frac{d}{dt}H(\boldsymbol{x}, \boldsymbol{y}) &= \nabla_{\boldsymbol{x}} H \cdot \frac{d\boldsymbol{x}}{dt} + \nabla_{\boldsymbol{y}} H \cdot \frac{d\boldsymbol{y}}{dt} && \text{using the chain rule} \\
&= \nabla_{\boldsymbol{x}} H \cdot \nabla_{\boldsymbol{y}} H - \nabla_{\boldsymbol{x}} H \cdot \nabla_{\boldsymbol{y}} H && \text{using the Hamiltonian dynamics (6.50)} \\
&= 0.
\end{aligned}$$

2. By applying one Leapfrog step, we have for $\boldsymbol{x}$,

$$\begin{aligned}
\boldsymbol{x}(t+\tau) &= \boldsymbol{x}(t) + \tau \boldsymbol{y}(t + \tau/2) && \text{using Equation (6.51b)} \\
&= \boldsymbol{x}(t) + \tau\Big(\boldsymbol{y}(t) - \frac{\tau}{2}\nabla_{\boldsymbol{x}} f(\boldsymbol{x}(t))\Big) && \text{using Equation (6.51a)} \\
&= \boldsymbol{x}(t) - \frac{\tau^2}{2}\nabla_{\boldsymbol{x}} f(\boldsymbol{x}(t)) + \tau \boldsymbol{y}(t).
\end{aligned}$$

Now observe that $\boldsymbol{y}(t)$ is a Gaussian random variable, independent of $\boldsymbol{x}$ (because we sample $\boldsymbol{y}$ freshly at the beginning of the $L$ Leapfrog steps, and we are doing just one Leapfrog step). By renaming $\tau' \doteq \tau^2/2$ and $\epsilon \doteq \boldsymbol{y}(t)$, we get

$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \tau'\nabla_{\boldsymbol{x}} f(\boldsymbol{x}_t) + \sqrt{2\tau'}\epsilon$$

which coincides with the proposal distribution of Langevin Monte Carlo (6.44).

## *Deep Learning*

**Solution to Problem 7.1.** We have

$$\begin{aligned}\sigma_1(\boldsymbol{f}) &= \frac{\exp(f_1)}{\exp(f_0)+\exp(f_1)} && \text{using the definition of the softmax function (7.5)}\\ &= \frac{1}{\exp(f_0-f_1)+1} && \text{multiplying by } \tfrac{\exp(-f_1)}{\exp(-f_1)}\\ &= \sigma(-(f_0-f_1)). && \text{using the definition of the logistic function (5.9)}\end{aligned}$$

$\sigma_0(\boldsymbol{f}) = 1-\sigma(f)$ follows from the fact that $\sigma_0(\boldsymbol{f})+\sigma_1(\boldsymbol{f}) = 1$.

## *Active Learning*

**Solution to Problem 8.1.** We have

$$\begin{aligned}\mathrm{I}(\mathbf{X};\mathbf{Y}) &= \mathrm{H}[\mathbf{X}] - \mathrm{H}[\mathbf{X}\mid\mathbf{Y}] && \text{using the definition of mutual information (8.9)}\\ &= \mathbb{E}_x[-\log p(\boldsymbol{x})] - \mathbb{E}_{(x,y)}[-\log p(\boldsymbol{x}\mid\boldsymbol{y})] && \text{using the definitions of entropy (5.27) and conditional entropy (8.2)}\\ &= \mathbb{E}_{(x,y)}[-\log p(\boldsymbol{x})] - \mathbb{E}_{(x,y)}[-\log p(\boldsymbol{x}\mid\boldsymbol{y})] && \text{using the law of total expectation (1.12)}\\ &= \mathbb{E}_{(x,y)}\left[\log\frac{p(\boldsymbol{x}\mid\boldsymbol{y})}{p(\boldsymbol{x})}\right].\end{aligned}$$

From this we get directly that

$$\begin{aligned}\mathrm{I}(\mathbf{X};\mathbf{Y}) &= \mathbb{E}_y\mathbb{E}_{x|y}\left[\log\frac{p(\boldsymbol{x}\mid\boldsymbol{y})}{p(\boldsymbol{x})}\right]\\ &= \mathbb{E}_y[\mathrm{KL}(p(\boldsymbol{x}\mid\boldsymbol{y})\|p(\boldsymbol{x}))], && \text{using the definition of KL-divergence (5.34)}\end{aligned}$$

and we also conclude

$$\begin{aligned}\mathrm{I}(\mathbf{X};\mathbf{Y}) &= \mathbb{E}_{(x,y)}\left[\log\frac{p(\boldsymbol{x},\boldsymbol{y})}{p(\boldsymbol{x})p(\boldsymbol{y})}\right] && \text{using the definition of conditional probability (1.8)}\\ &= \mathrm{KL}(p(\boldsymbol{x},\boldsymbol{y})\|p(\boldsymbol{x})p(\boldsymbol{y})). && \text{using the definition of KL-divergence (5.34)}\end{aligned}$$

**Solution to Problem 8.2.** Symmetry of conditional mutual information (8.17) and Equation (8.16) give the following relationship,

$$\mathrm{I}(\mathbf{X};\mathbf{Y},\mathbf{Z}) = \mathrm{I}(\mathbf{X};\mathbf{Y}) + \mathrm{I}(\mathbf{X};\mathbf{Z}\mid\mathbf{Y}) = \mathrm{I}(\mathbf{X};\mathbf{Z}) + \mathrm{I}(\mathbf{X};\mathbf{Y}\mid\mathbf{Z}). \tag{B.11}$$

1. $\mathbf{X}\perp\mathbf{Z}$ implies $\mathrm{I}(\mathbf{X};\mathbf{Z}) = 0$. Thus, Equation (B.11) simplifies to

$$\mathrm{I}(\mathbf{X};\mathbf{Y}) + \mathrm{I}(\mathbf{X};\mathbf{Z}\mid\mathbf{Y}) = \mathrm{I}(\mathbf{X};\mathbf{Y}\mid\mathbf{Z}).$$

Using that $\mathrm{I}(\mathbf{X};\mathbf{Z}\mid\mathbf{Y}) \geq 0$, we conclude $\mathrm{I}(\mathbf{X};\mathbf{Y}\mid\mathbf{Z}) \geq \mathrm{I}(\mathbf{X};\mathbf{Y})$.

2. $\mathbf{X} \perp \mathbf{Z} \mid \mathbf{Y}$ implies $\mathrm{I}(\mathbf{X}; \mathbf{Z} \mid \mathbf{Y}) = 0$. Equation (B.11) simplifies to

$$\mathrm{I}(\mathbf{X}; \mathbf{Y}) = \mathrm{I}(\mathbf{X}; \mathbf{Z}) + \mathrm{I}(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}).$$

   Using that $\mathrm{I}(\mathbf{X}; \mathbf{Z}) \geq 0$, we conclude $\mathrm{I}(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}) \leq \mathrm{I}(\mathbf{X}; \mathbf{Y})$.
3. Again, Equation (B.11) simplifies to

$$\mathrm{I}(\mathbf{X}; \mathbf{Y}) = \mathrm{I}(\mathbf{X}; \mathbf{Z}) + \mathrm{I}(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}).$$

   Using that $\mathrm{I}(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}) \geq 0$, we conclude $\mathrm{I}(\mathbf{X}; \mathbf{Z}) \leq \mathrm{I}(\mathbf{X}; \mathbf{Y})$.

**Solution to Problem 8.3.**

1. Expanding the definition of interaction information, one obtains

$$\begin{aligned}\mathrm{I}(\mathbf{X}; \mathbf{Y}; \mathbf{Z}) = &(\mathrm{H}[\mathbf{X}] + \mathrm{H}[\mathbf{Y}] + \mathrm{H}[\mathbf{Z}]) \\ &- (\mathrm{H}[\mathbf{X}, \mathbf{Y}] + \mathrm{H}[\mathbf{X}, \mathbf{Z}] + \mathrm{H}[\mathbf{Y}, \mathbf{Z}]) \\ &+ \mathrm{H}[\mathbf{X}, \mathbf{Y}, \mathbf{Z}],\end{aligned}$$

   and hence, interaction information is symmetric.
2. Conditional on either one of $X_1$ or $X_2$, the distribution of $Y$ remains unchanged, and hence $\mathrm{I}(Y; X_1 \mid X_2) = \mathrm{I}(Y; X_2 \mid X_1) = 0$. Conversely, conditional on both $X_1$ and $X_2$, $Y$ is fully determined, and hence $\mathrm{I}(Y; X_1, X_2) = 1$ noting that $Y$ encodes one bit worth of information. Thus, $\mathrm{I}(Y; X_1; X_2) = -1$ meaning that there is synergy between $X_1$ and $X_2$ with respect to $Y$.

**Solution to Problem 8.4.** As suggested, we derive the result in two steps.

1. First, we have

$$\begin{aligned}\Delta_I(\boldsymbol{x} \mid A) &= I(A \cup \{\boldsymbol{x}\}) - I(A) && \text{using the definition of marginal gain (8.22)} \\ &= \mathrm{I}(\boldsymbol{f}_{A\cup\{\boldsymbol{x}\}}; \boldsymbol{y}_A, y_{\boldsymbol{x}}) - \mathrm{I}(\boldsymbol{f}_A; \boldsymbol{y}_A) && \text{using the definition of } I \text{ (8.20)} \\ &= \mathrm{I}(\boldsymbol{f}_{A\cup\{\boldsymbol{x}\}}; \boldsymbol{y}_A, y_{\boldsymbol{x}}) - \mathrm{I}(\boldsymbol{f}_{A\cup\{\boldsymbol{x}\}}; \boldsymbol{y}_A) && \text{using } \boldsymbol{y}_A \perp f_{\boldsymbol{x}} \mid \boldsymbol{f}_A \\ &= \mathrm{H}[\boldsymbol{f}_{A\cup\{\boldsymbol{x}\}} \mid \boldsymbol{y}_A] - \mathrm{H}[\boldsymbol{f}_{A\cup\{\boldsymbol{x}\}} \mid \boldsymbol{y}_A, y_{\boldsymbol{x}}] && \text{using the definition of MI (8.9)} \\ &= \mathrm{I}(\boldsymbol{f}_{A\cup\{\boldsymbol{x}\}}; y_{\boldsymbol{x}} \mid \boldsymbol{y}_A) && \text{using the definition of cond. MI (8.14)} \\ &= \mathrm{I}(\boldsymbol{f}_A; y_{\boldsymbol{x}} \mid f_{\boldsymbol{x}}, \boldsymbol{y}_A) + \mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{y}_A) && \text{using Equation (8.16)} \\ &= \mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{y}_A). && \text{using } \mathrm{I}(\boldsymbol{f}_A; y_{\boldsymbol{x}} \mid f_{\boldsymbol{x}}, \boldsymbol{y}_A) = 0 \text{ as } y_{\boldsymbol{x}} \perp \boldsymbol{f}_A \mid f_{\boldsymbol{x}}\end{aligned}$$

2. For the second part, we get

$$\begin{aligned}\mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{y}_A) &= \mathrm{I}(y_{\boldsymbol{x}}; f_{\boldsymbol{x}} \mid \boldsymbol{y}_A) && \text{using symmetry of conditional MI (8.17)} \\ &= \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{y}_A] - \mathrm{H}[y_{\boldsymbol{x}} \mid f_{\boldsymbol{x}}, \boldsymbol{y}_A] && \text{using the definition of cond. MI (8.14)} \\ &= \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{y}_A] - \mathrm{H}[y_{\boldsymbol{x}} \mid f_{\boldsymbol{x}}] && \text{using that } y_{\boldsymbol{x}} \perp \boldsymbol{\varepsilon}_A \text{ so } y_{\boldsymbol{x}} \perp \boldsymbol{y}_A \mid f_{\boldsymbol{x}} \\ &= \mathrm{H}[y_{\boldsymbol{x}} \mid \boldsymbol{y}_A] - \mathrm{H}[\varepsilon_{\boldsymbol{x}}]. && \text{given } f_{\boldsymbol{x}}, \text{ the only randomness in } y_{\boldsymbol{x}} \text{ originates from } \varepsilon_{\boldsymbol{x}}\end{aligned}$$

**Solution to Problem 8.5.** We have

$$\begin{aligned}
\mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}}; \boldsymbol{y}_{B\setminus A} \mid \boldsymbol{y}_A) &= \mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{y}_A) - \mathrm{I}(f_{\boldsymbol{x}}; y_{\boldsymbol{x}} \mid \boldsymbol{y}_B) && \text{using the definitiion of interaction information (8.18)} \\
&= \Delta_I(\boldsymbol{x} \mid A) - \Delta_I(\boldsymbol{x} \mid B) && \text{using Equation (8.23)} \\
&\geq 0
\end{aligned}$$

where the final inequality follows from submodularity of $I$.

**Solution to Problem 8.6.**

1. **Yes**. We have

   $$\Delta(j \mid S) = F(S \cup \{j\}) - F(S) = \mathrm{H}[Z \mid Y_S] - \mathrm{H}[Z \mid Y_{S\cup\{j\}}].$$

   This is non-negative iff $\mathrm{H}[Z \mid Y_S] \geq \mathrm{H}[Z \mid Y_{S\cup\{j\}}]$ which is the "information never hurts" property (8.8) of conditional entropy.
2. **No**, our acquisition function $F$ is not equivalent to uncertainty sampling. Note that all $X_i$ have identical prior variance. It suffices to show that not all $i_1 \in \{1, \ldots, 100\}$ maximize the marginal information gain $\Delta(i_1 \mid \emptyset)$.
   The prior variance of $Z$ is $\mathrm{Var}[Z] = \sum_{i=1}^{100} i^2$ since all $X_i$ are independent. Since the $x_{1:100}$ and $Z$ are jointly Gaussian, the posterior variance of $Z$ after observing $Y_1$ is

   $$\mathrm{Var}[Z \mid Y_1] = \mathrm{Var}[Z] - \frac{\mathrm{Cov}[Z, Y_1]^2}{\mathrm{Var}[Y_1]}.$$

   The marginal information gain $\Delta(i_1 \mid \emptyset)$ is equal up to constants to $-\frac{1}{2}\log \mathrm{Var}[Z \mid Y_1]$. The variance of the observation is $\mathrm{Var}[Y_1] = \mathrm{Var}\left[X_{i_1} + \varepsilon_1\right] = 2$ and the covariance is $\mathrm{Cov}[Z, Y_1] = i_1$. Therefore,

   $$\mathrm{Var}[Z \mid Y_1] = \sum_{i=1}^{100} i^2 - \frac{1}{2} i_1,$$

   which is uniquely minimized for $i_1 = 100$.
3. The acquisition function is submodular if the previous observation of an input $X_i$ does not imply that the information conveyed by observing another input $X_j$ about the value of $Z$ is larger than it would have been without having previously observed $X_i$. Clearly, if $i = j$ then the information conveyed by the second observation is always 0 bit, since the first observation was noiseless. However, depending on $Z$ previous observations of inputs other than $X_j$ may be crucial for the informativeness of observing $X_j$.
   (a) **Yes**. If $X_j = 0$ then we know that $Z = 0$ also, no matter the previously observed inputs. If $X_j = 1$ then it becomes slightly more probable that $Z = 1$, however, this does not depend on which other points have been observed previously.

(b) **Yes**. The case of OR is symmetric to AND. If $X_j = 1$ then we know that $Z = 1$ also, no matter the previously observed inputs. If $X_j = 0$ then it becomes slightly more probable that $Z = 0$, however, this does not depend on which other points have been observed previously.

(c) **No**. Consider the case where we have observed $X_{1:99}$ and are now observing $X_{100}$. The marginal information gain of all previous observations was 0 bit, since the observed values for $X_{1:99}$ did not affect the distribution of $Z$: the prior of $Z$ was $\text{Bern}(0.5)$ and the current posterior is still the same Bernoulli distribution. Now, observing $X_{100}$ will deterministically determine the value of $f$, therefore conveying 1 bit of information. However, just observing $X_{100}$ alone, without any prior observations, conveys no information. This shows that the marginal information gain can increase after conditioning on more observations, and therefore the acquisition function $F$ is not submodular with this choice of $Z$.

The key intuition behind the above arguments is that AND and OR are monotonic functions whereas XOR is not.

## *Bayesian Optimization*

**Solution to Problem 9.1.** First, observe that

$$\begin{aligned}\lim_{T\to\infty} \frac{R_T}{T} &= \max_{\boldsymbol{x}} f^\star(\boldsymbol{x}) - \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} f^\star(\boldsymbol{x}_t) && \text{using the definition of regret (9.4)}\\ &= \max_{\boldsymbol{x}} f^\star(\boldsymbol{x}) - \lim_{t\to\infty} f^\star(\boldsymbol{x}_t). && \text{(B.12)} \quad \text{using the Cesàro mean (9.38)}\end{aligned}$$

Now, suppose that the algorithm converges to the static optimum,

$$\lim_{t\to\infty} f^\star(\boldsymbol{x}_t) = \max_{\boldsymbol{x}} f^\star(\boldsymbol{x}).$$

Together with Equation (B.12) we conclude that the algorithm achieves sublinear regret.

For the other direction, we prove the contrapositive. That is, we assume that the algorithm does not converge to the static optimum and show that it has (super-)linear regret. We distinguish between two cases. Our assumption is formalized by

$$\lim_{t\to\infty} f^\star(\boldsymbol{x}_t) < \max_{\boldsymbol{x}} f^\star(\boldsymbol{x}).$$

Together with Equation (B.12) we conclude $\lim_{T\to\infty} R_T/T > 0$.

**Solution to Problem 9.2.**

1. As suggested by the hint, we let $Z \sim \mathcal{N}(0,1)$ and $c > 0$, and bound

$$\begin{aligned}\mathbb{P}(Z > c) &= \mathbb{P}(Z - c > 0) \\ &= \int_0^\infty \frac{1}{\sqrt{2\pi}} e^{-(z+c)^2/2}\, dz. && \text{using the PDF of the univariate normal distribution (1.5)}\end{aligned}$$

Note that for $z \geq 0$,

$$\frac{(z+c)^2}{2} = \frac{z^2}{2} + zc + \frac{c^2}{2} \geq \frac{z^2}{2} + \frac{c^2}{2}.$$

Thus,

$$\begin{aligned}&\leq e^{-c^2/2} \int_0^\infty \frac{1}{\sqrt{2\pi}} e^{-z^2/2}\, dz \\ &= e^{-c^2/2} \mathbb{P}(Z > 0) \\ &= \frac{1}{2} e^{-c^2/2}. && \text{(B.13)} && \text{using symmetry of the standard normal distribution around 0}\end{aligned}$$

Since we made the Bayesian assumption $f^\star(\boldsymbol{x}) \sim \mathcal{N}(\mu_0(\boldsymbol{x}), \sigma_0^2(\boldsymbol{x}))$ and assumed Gaussian observation noise $y_t \sim \mathcal{N}(f^\star(\boldsymbol{x}_t), \sigma_\mathrm{n}^2)$, the posterior is also Gaussian:

$$f^\star(\boldsymbol{x}) \mid \boldsymbol{x}_{1:t}, y_{1:t} \sim \mathcal{N}(\mu_t(\boldsymbol{x}), \sigma_t^2(\boldsymbol{x})).$$

Hence, writing $\mathbb{P}_t(\cdot) \doteq \mathbb{P}(\cdot \mid \boldsymbol{x}_{1:t}, y_{1:t})$,

$$\begin{aligned}\mathbb{P}_{t-1}(f^\star(\boldsymbol{x}) \notin \mathcal{C}_t(\boldsymbol{x})) &= \mathbb{P}_{t-1}(|f^\star(\boldsymbol{x}) - \mu_{t-1}(\boldsymbol{x})| > \beta_t \sigma_{t-1}(\boldsymbol{x})) \\ &= 2\mathbb{P}_{t-1}\left(\frac{f^\star(\boldsymbol{x}) - \mu_{t-1}(\boldsymbol{x})}{\sigma_{t-1}(\boldsymbol{x})} > \beta_t\right) && \text{using symmetry of the Gaussian distribution} \\ &\leq e^{-\beta_t^2/2}. && \text{using Equation (B.13) with } c = \beta_t\end{aligned}$$

2. We have

$$\begin{aligned}\mathbb{P}\left(\bigcup_{\boldsymbol{x}\in\mathcal{X}} \bigcup_{t\geq 1} f^\star(\boldsymbol{x}) \notin \mathcal{C}_t(\boldsymbol{x})\right) &\leq \sum_{\boldsymbol{x}\in\mathcal{X}} \sum_{t\geq 1} \mathbb{P}(f^\star(\boldsymbol{x}) \notin \mathcal{C}_t(\boldsymbol{x})) && \text{using a union bound (1.73)} \\ &\leq |\mathcal{X}| \sum_{t\geq 1} e^{-\beta_t^2/2}. && \text{using (1)}\end{aligned}$$

Letting $\beta_t^2 \doteq 2\log(|\mathcal{X}|\,(\pi t)^2/(6\delta))$, we get

$$\begin{aligned}&= \frac{6\delta}{\pi^2} \sum_{t\geq 1} \frac{1}{t^2} \\ &= \delta. && \text{using } \textstyle\sum_{t\geq 1} \frac{1}{t^2} = \frac{\pi^2}{6}\end{aligned}$$

**Solution to Problem 9.3.**

1. We denote the static optimum by $\boldsymbol{x}^\star$. By the definition of $\boldsymbol{x}_t$,

$$\mu_{t-1}(\boldsymbol{x}_t) + \beta_t \sigma_{t-1}(\boldsymbol{x}_t) \geq \mu_{t-1}(\boldsymbol{x}^\star) + \beta_t \sigma_{t-1}(\boldsymbol{x}^\star)$$

$$\geq f^\star(\boldsymbol{x}^\star). \qquad \text{using Equation (9.7)}$$

Thus,

$$\begin{aligned} r_t &= f^\star(\boldsymbol{x}^\star) - f^\star(\boldsymbol{x}_t) \\ &\leq \beta_t \sigma_{t-1}(\boldsymbol{x}_t) + \mu_{t-1}(\boldsymbol{x}_t) - f^\star(\boldsymbol{x}_t) \\ &\leq 2\beta_t \sigma_{t-1}(\boldsymbol{x}_t). && \text{again using Equation (9.7)} \end{aligned}$$

2. We have for any fixed $T$,

$$\begin{aligned} \mathrm{I}(\boldsymbol{f}_{T+1}; \boldsymbol{y}_{T+1}) &= \mathrm{H}[\boldsymbol{y}_{T+1}] - \mathrm{H}[\boldsymbol{\varepsilon}_{T+1}] && \text{analogously to Equation (8.24)} \\ &= \mathrm{H}[\boldsymbol{y}_T] - \mathrm{H}[\boldsymbol{\varepsilon}_T] + \mathrm{H}\big[y_{\boldsymbol{x}_{T+1}} \mid \boldsymbol{y}_T\big] - \mathrm{H}\big[\varepsilon_{\boldsymbol{x}_{T+1}}\big] && \text{using the chain rule for entropy (8.5) and the mutual independence of } \boldsymbol{\varepsilon}_{T+1} \\ &= \mathrm{I}(\boldsymbol{f}_T; \boldsymbol{y}_T) + \mathrm{I}\big(f_{\boldsymbol{x}_{T+1}}; y_{\boldsymbol{x}_{T+1}} \mid \boldsymbol{y}_T\big) && \text{using the definition of MI (8.9)} \\ &= \mathrm{I}(\boldsymbol{f}_T; \boldsymbol{y}_T) + \frac{1}{2} \log\left(1 + \frac{\sigma_T^2(\boldsymbol{x}_{T+1})}{\sigma_\mathrm{n}^2}\right). && \text{using Equation (8.13)} \end{aligned}$$

Note that $\mathrm{I}(\boldsymbol{f}_0; \boldsymbol{y}_0) = 0$. The result then follows by induction.

3. By Cauchy-Schwarz, $R_T^2 \leq T \sum_{t=1}^T r_t^2$, and hence, it suffices to show $\sum_{t=1}^T r_t^2 \leq O\big(\beta_T^2 \gamma_T\big)$. We have

$$\begin{aligned} \sum_{t=1}^T r_t^2 &\leq 4\beta_T^2 \sum_{t=1}^T \sigma_{t-1}^2(\boldsymbol{x}_t) && \text{using part (1)} \\ &= 4\sigma_\mathrm{n}^2 \beta_T^2 \sum_{t=1}^T \frac{\sigma_{t-1}^2(\boldsymbol{x}_t)}{\sigma_\mathrm{n}^2}. \end{aligned}$$

Observe that $\sigma_{t-1}^2(\boldsymbol{x}_t)/\sigma_\mathrm{n}^2$ is bounded by $M \doteq \max_{x \in \mathcal{X}} \sigma_0^2(\boldsymbol{x})/\sigma_\mathrm{n}^2$ as variance is monotonically decreasing (cf. Section 1.2.3). Applying the hint, we obtain

$$\begin{aligned} &\leq 4C\sigma_\mathrm{n}^2 \beta_T^2 \sum_{t=1}^T \log\left(1 + \frac{\sigma_{t-1}^2(\boldsymbol{x}_t)}{\sigma_\mathrm{n}^2}\right) \\ &= 8C\sigma_\mathrm{n}^2 \beta_T^2 \mathrm{I}(\boldsymbol{f}_T; \boldsymbol{y}_T) && \text{using part (2)} \\ &\leq 8C\sigma_\mathrm{n}^2 \beta_T^2 \gamma_T. && \text{using the definition of } \gamma_T \text{ (9.10)} \end{aligned}$$

**Solution to Problem 9.4.**

1. Let $S \subseteq \mathcal{X}$ be such that $|S| \leq T$. Recall from Equation (8.13) that $\mathrm{I}(\boldsymbol{f}_S; \boldsymbol{y}_S) = \frac{1}{2} \log\det\big(\boldsymbol{I} + \sigma_\mathrm{n}^{-2} \boldsymbol{K}_{SS}\big)$. Using that the kernel is linear we can rewrite $\boldsymbol{K}_{SS} = \boldsymbol{X}_S^\top \boldsymbol{X}_S$. Using Weinstein-Aronszajn's identity (A.70) we have

$$\mathrm{I}(\boldsymbol{f}_S; \boldsymbol{y}_S) = \frac{1}{2} \log\det\Big(\boldsymbol{I} + \sigma_\mathrm{n}^{-2} \boldsymbol{X}_S^\top \boldsymbol{X}_S\Big) = \frac{1}{2} \log\det\Big(\boldsymbol{I} + \sigma_\mathrm{n}^{-2} \boldsymbol{X}_S \boldsymbol{X}_S^\top\Big).$$

If we define $\boldsymbol{M} \doteq \boldsymbol{I} + \sigma_\mathrm{n}^{-2} \boldsymbol{X}_S \boldsymbol{X}_S^\top$ as a sum of symmetric positive definite matrices, $\boldsymbol{M}$ itself is symmetric positive definite. Thus, we have from Hadamard's inequality (A.69),

$$\det(\boldsymbol{M}) \leq \det\Big(\operatorname{diag}\{\boldsymbol{I} + \sigma_\mathrm{n}^{-2} \boldsymbol{X}_S \boldsymbol{X}_S^\top\}\Big) \qquad \operatorname{diag}\{\boldsymbol{A}\} \text{ refers to the diagonal matrix whose elements are those of } \boldsymbol{A}$$

$$= \det\Big(\boldsymbol{I} + \sigma_{\mathrm{n}}^{-2}\mathrm{diag}\{\boldsymbol{X}_S\boldsymbol{X}_S^\top\}\Big).$$

Note that

$$\begin{aligned}\mathrm{diag}\{\boldsymbol{X}_S\boldsymbol{X}_S^\top\}(i,i) &= \sum_{t=1}^{|S|}\boldsymbol{x}_t(i)^2 \\ &\leq \sum_{i=1}^{d}\sum_{t=1}^{|S|}\boldsymbol{x}_t(i)^2 = \sum_{t=1}^{|S|}\underbrace{\|\boldsymbol{x}_t\|_2^2}_{\leq 1} \leq |S| \leq T.\end{aligned}$$

If we denote by $\lambda \leq T$ the largest term of $\mathrm{diag}\{\boldsymbol{X}_S\boldsymbol{X}_S^\top\}$ then we have

$$\det(\boldsymbol{M}) \leq (1+\sigma_{\mathrm{n}}^{-2}\lambda)^d \leq (1+\sigma_{\mathrm{n}}^{-2}T)^d,$$

yielding,

$$\mathrm{I}(\boldsymbol{f}_S;\boldsymbol{y}_S) \leq \frac{d}{2}\log(1+\sigma_{\mathrm{n}}^{-2}T)$$

implying that $\gamma_T = O(d\log T)$.

2. Using the regret bound (cf. Theorem 9.5) and the Bayesian confidence intervals (cf. Theorem 9.4), and then $\gamma_T = O(d\log T)$, we have

$$R_T = O\Big(\beta_T\sqrt{\gamma_T T}\Big) = \widetilde{O}\Big(\sqrt{dT}\Big),$$

and hence, $\lim_{T\to\infty}\frac{R_T}{T} = 0$.

**Solution to Problem 9.5.**

1. Note that $f$ is a Gaussian process, and hence, our posterior distribution after round $t$ is entirely defined by the mean function $\mu_t$ and the covariance function $k_t$. Reparameterizing the posterior distribution using a standard Gaussian (1.54), we obtain

$$f(\boldsymbol{x}) \mid \boldsymbol{x}_{1:t}, y_{1:t} = \mu_t(\boldsymbol{x}) + \sigma_t(\boldsymbol{x})\varepsilon$$

for $\varepsilon \sim \mathcal{N}(0,1)$. We get

$$\begin{aligned}\mathrm{EI}_t(\boldsymbol{x}) &= \mathbb{E}_{f(\boldsymbol{x})\sim\mathcal{N}(\mu_t(\boldsymbol{x}),\sigma_t^2(\boldsymbol{x}))}[I_t(\boldsymbol{x})] && \text{using the definition of expected improvement (9.19)} \\ &= \mathbb{E}_{f(\boldsymbol{x})\sim\mathcal{N}(\mu_t(\boldsymbol{x}),\sigma_t^2(\boldsymbol{x}))}\Big[(f(\boldsymbol{x})-\hat{f}_t)_+\Big] && \text{using the definition of improvement (9.15)} \\ &= \mathbb{E}_{\varepsilon\sim\mathcal{N}(0,1)}\Big[(\mu_t(\boldsymbol{x})+\sigma_t(\boldsymbol{x})\varepsilon-\hat{f}_t)_+\Big] && \text{using the reparameterization} \\ &= \int_{-\infty}^{+\infty}(\mu_t(\boldsymbol{x})+\sigma_t(\boldsymbol{x})\varepsilon-\hat{f}_t)_+\cdot\phi(\varepsilon)\,d\varepsilon.\end{aligned}$$

For $\varepsilon < \frac{\hat{f}_t-\mu_t(\boldsymbol{x})}{\sigma_t(\boldsymbol{x})} \doteq z_t(\boldsymbol{x})$ we have $(\mu_t(\boldsymbol{x})+\sigma_t(\boldsymbol{x})\varepsilon-\hat{f}_t)_+ = 0$. Thus, we obtain

$$\mathrm{EI}_t(\boldsymbol{x}) = \int_{z_t(\boldsymbol{x})}^{+\infty}(\mu_t(\boldsymbol{x})+\sigma_t(\boldsymbol{x})\varepsilon-\hat{f}_t)\cdot\phi(\varepsilon)\,d\varepsilon. \tag{B.14}$$

2. By splitting the integral from Equation (B.14) into two distinct terms, we obtain

$$\begin{aligned}\mathrm{EI}_t(\boldsymbol{x}) = (\mu_t(\boldsymbol{x}) - \hat{f}_t) \int_{z_t(\boldsymbol{x})}^{+\infty} \phi(\varepsilon)\, d\varepsilon \\ - \sigma_t(\boldsymbol{x}) \int_{z_t(\boldsymbol{x})}^{+\infty} (-\varepsilon) \cdot \phi(\varepsilon)\, d\varepsilon.\end{aligned}$$

For the first term, we use the symmetry of $\mathcal{N}(0,1)$ around 0 to write the integral in terms of the CDF. For the second term, we notice that $(-\varepsilon) \cdot \phi(\varepsilon) = \frac{1}{\sqrt{2\pi}} \frac{d}{d\varepsilon} e^{-\varepsilon^2/2}$. Thus, we can derive this integral directly,

$$\begin{aligned}= (\mu_t(\boldsymbol{x}) - \hat{f}_t)\Phi(-z_t(\boldsymbol{x})) \\ - \sigma_t(\boldsymbol{x}) \Big( \lim_{\varepsilon \to \infty} \phi(\varepsilon) - \phi(z_t(\boldsymbol{x})) \Big).\end{aligned}$$

Using the symmetry of $\phi$ around 0, we obtain

$$\mathrm{EI}_t(\boldsymbol{x}) = (\mu_t(\boldsymbol{x}) - \hat{f}_t)\Phi\left( \frac{\mu_t(\boldsymbol{x}) - \hat{f}_t}{\sigma_t(\boldsymbol{x})} \right) + \sigma_t(\boldsymbol{x})\phi\left( \frac{\mu_t(\boldsymbol{x}) - \hat{f}_t}{\sigma_t(\boldsymbol{x})} \right).$$

**Solution to Problem 9.6.**

1. As $\boldsymbol{x}_t^{\mathrm{UCB}}$ is the UCB action,

$$\hat{\Delta}_t(\boldsymbol{x}_t^{\mathrm{UCB}}) = u_t(\boldsymbol{x}_t^{\mathrm{UCB}}) - l_t(\boldsymbol{x}_t^{\mathrm{UCB}}) = 2\beta_{t+1}\sigma_t(\boldsymbol{x}_t^{\mathrm{UCB}}).$$

2. We first bound $I_t(\boldsymbol{x}_t^{\mathrm{UCB}})$:

$$\begin{aligned}I_t(\boldsymbol{x}_t^{\mathrm{UCB}}) &= \mathrm{I}\Big( f_{\boldsymbol{x}_t^{\mathrm{UCB}}}; y_{\boldsymbol{x}_t^{\mathrm{UCB}}} \;\Big|\; \boldsymbol{x}_{1:t}, y_{1:t} \Big) \\ &= \frac{1}{2} \log\left( 1 + \frac{\sigma_t^2(\boldsymbol{x}_t^{\mathrm{UCB}})}{\sigma_{\mathrm{n}}^2} \right). \qquad \text{using Equation (8.13)}\end{aligned}$$

Note that $\sigma_t^2(\boldsymbol{x})/\sigma_{\mathrm{n}}^2 \leq C$ for some constant $C$ since variance is decreasing monotonically. So, applying the hint,

$$\geq \frac{\sigma_t^2(\boldsymbol{x}_t^{\mathrm{UCB}})}{2C\sigma_{\mathrm{n}}^2}.$$

Combining this with (1), we obtain

$$\widehat{\Psi}_t(\boldsymbol{x}_t^{\mathrm{UCB}}) = \frac{\hat{\Delta}_t(\boldsymbol{x}_t^{\mathrm{UCB}})^2}{I_t(\boldsymbol{x}_t^{\mathrm{UCB}})} \leq 8C\sigma_{\mathrm{n}}^2\beta_{t+1}^2.$$

3. With high probability,

$$\Psi_t(\boldsymbol{x}_{t+1}) \leq \widehat{\Psi}_t(\boldsymbol{x}_{t+1}) \leq \widehat{\Psi}_t(\boldsymbol{x}_t^{\mathrm{UCB}}) \leq 8C\sigma_{\mathrm{n}}^2\beta_{t+1}^2,$$

where the first inequality is due to $\Delta(\boldsymbol{x}) \leq \hat{\Delta}_t(\boldsymbol{x})$ with high probability, the second inequality due to the definition of the IDS algorithm (9.29), and the third inequality is from (2). Invoking Theorem 9.9 with $\overline{\Psi}_T \doteq 8C\sigma_{\mathrm{n}}^2\beta_T^2$, we obtain that

$$R_T \leq \sqrt{\gamma_T \overline{\Psi}_T T} = \beta_T \sqrt{8C\sigma_{\mathrm{n}}^2 \gamma_T T}$$

with high probability.

**Solution to Problem 9.7.** First notice that by the definition of S′, we obtain the simpler objective:

$$\mathcal{W}(\pi) = \sum_{x \in \mathcal{X}} \pi(x) \Big( \mu_t(x) + \phi(\Phi^{-1}(\pi(x)))\sigma_t(x) \Big).$$

Next, we show that $\mathcal{W}(\pi)$ is concave by computing the Hessian:

$$\begin{aligned}
\frac{\partial}{\partial \pi(x)} \mathcal{W}(\pi) &= \mu_t(x) - \sigma_t(x)\Phi^{-1}(\pi(x))\phi(\Phi^{-1}(\pi(x))) \frac{d}{d\pi(x)} \Phi^{-1}(\pi(x)) \\
&= \mu_t(x) - \sigma_t(x)\Phi^{-1}(\pi(x)) \\
\frac{\partial^2}{\partial \pi(x) \partial \pi(z)} \mathcal{W}(r) &= -\sigma_t(x) \mathbb{1}\{x = z\} \frac{1}{\phi(\Phi^{-1}(\pi(x)))} \begin{cases} < 0 & x = z \\ = 0 & x \neq z \end{cases},
\end{aligned}$$

where we used the inverse function rule twice. From negative definiteness of the Hessian, it follows that $\mathcal{W}(\cdot)$ is strictly concave.

We show next that the optimum lies in the relative interior of the probability simplex, $\pi^* \in \operatorname{relint}(\Delta^{\mathcal{X}})$. Indeed, at the border of the probability simplex the partial derivatives explode:

$$\frac{\partial}{\partial \pi(x)} \mathcal{W}(r) = \mu_t(x) - \sigma_t(x)\Phi^{-1}(\pi(x)) = \begin{cases} \infty & \pi(x) \to 0^+ \\ \text{finite} & \pi(x) \in (0,1) \\ -\infty & \pi(x) \to 1^- \end{cases}.$$

Together with the concavity of $\mathcal{W}(\cdot)$ this ensures that $r^* \in \operatorname{relint}(\Delta^{\mathcal{X}})$. Hence, $\pi^*$ is a local optimizer of $\mathcal{W}(r)$ on the plane defined by $\sum_{x \in \mathcal{X}} \pi(x) = 1$. Consequently, we obtain the Lagrangian

$$\mathcal{L}(\pi, \kappa) : (0,1)^{|\mathcal{X}|} \times \mathbb{R} \to \mathbb{R}, \quad \pi \mapsto \mathcal{W}(\pi) + \kappa \left( 1 - \sum_{x \in \mathcal{X}} \pi(x) \right).$$

Setting its partial derivatives equal to zero, we derive the closed-form solution:

$$0 = \mu_t(x) - \sigma_t(x)\Phi^{-1}(\pi^*(x)) - \kappa^* \iff \pi^*(x) = \Phi\left( \frac{\mu_t(x) - \kappa^*}{\sigma_t(x)} \right),$$

where $\kappa^*$ ensures a valid distribution, i.e., $\sum_{x \in \mathcal{X}} \pi^*(x) = 1$.

**Solution to Problem 9.8.** Since $R_1$ and $R_2$ are independent, we have

$$\begin{aligned}\mathbb{P}(\max\{R_1, R_2\} \leq x) &= \mathbb{P}(\{R_1 \leq x\} \cap \{R_2 \leq x\}) \\ &= \mathbb{P}(R_1 \leq x) \cdot \mathbb{P}(R_2 \leq x) && \text{using independence} \\ &= F_{R_1}(x) \cdot F_{R_2}(x) \\ &= \Phi^2\Big(\frac{x}{10}\Big)\end{aligned}$$

where $\Phi$ is the CDF of the standard Gaussian. We are looking for the probability that either $R_1$ or $R_2$ are larger than $S = 1$:

$$\mathbb{P}(\max\{R_1, R_2\} > 1) = 1 - \mathbb{P}(\max\{R_1, R_2\} \leq 1) = 1 - \Phi^2\Big(\frac{1}{10}\Big).$$

We have $\Phi^2(\frac{1}{10}) \approx 0.29$. Thus, the probability that either $R_1$ or $R_2$ are larger than $S = 1$ is approximately 0.71. Due to the symmetry in the problem, we know

$$\mathbb{P}(R_1 > 1) = \mathbb{P}(R_2 > 1) \approx 0.35.$$

## *Markov Decision Processes*

**Solution to Problem 10.1.** We can use Equation (10.15) to write the state-action values as a linear system of equations (i.e., as a "table"). This linear system can be solved, for example, using Gaussian elimination to yield the desired result.

**Solution to Problem 10.2.** It follows directly from the definition of the state-action value function (10.9) that

$$\arg\max_{a \in A} q(x, a) = \arg\max_{a \in A} r(x, a) + \gamma \sum_{x' \in X} p(x' \mid x, a) \cdot v(x').$$

**Solution to Problem 10.3.**

1. Recall from Bellman's theorem (10.31) that a policy is optimal iff it is greedy with respect to its state-action value function. Now, observe that in the "poor, unknown" state, the policy $\pi$ is not greedy.
2. Analogously to Problem 10.1, we write the state-action values as a linear system of equations and solve the system using, e.g., Gaussian elimination.
3. Observe from the result of (2) that $\pi'$ is greedy with respect to its state-action value function, and hence, it follows from Bellman's theorem that $\pi'$ is optimal.

**Solution to Problem 10.4.** Using the hint and $\boldsymbol{v}^\star \geq \boldsymbol{v}^\pi$ for any policy $\pi$,

$$\|\boldsymbol{v}^{\pi_t} - \boldsymbol{v}^\star\|_\infty \leq \|\boldsymbol{B}^\star \boldsymbol{v}^{\pi_{t-1}} - \boldsymbol{v}^\star\|_\infty$$

$$
\begin{aligned}
&= \left\| \boldsymbol{B}^\star \boldsymbol{v}^{\pi_{t-1}} - \boldsymbol{B}^\star \boldsymbol{v}^\star \right\|_\infty && \text{using that } \boldsymbol{v}^\star \text{ is a fixed-point of } \boldsymbol{B}^\star, \text{ that is, } \boldsymbol{B}^\star \boldsymbol{v}^\star = \boldsymbol{v}^\star \\
&\leq \gamma \left\| \boldsymbol{v}^{\pi_{t-1}} - \boldsymbol{v}^\star \right\|_\infty && \text{using that } \boldsymbol{B}^\star \text{ is a contraction, see Theorem 10.18} \\
&\leq \gamma^t \left\| \boldsymbol{v}^{\pi_0} - \boldsymbol{v}^\star \right\|_\infty . && \text{by induction}
\end{aligned}
$$

**Solution to Problem 10.5.**

1. Recall that the value function $v_M^\pi$ for an MDP $M$ is defined as $v_M^\pi(x) = \mathbb{E}_\pi\left[\sum_{t=0}^\infty \gamma^t R_t \mid X_0 = x\right]$. Given an optimal policy $\pi^\star$ for $M$ and any policy $\pi$, we know that for any $x \in X$,

$$
\begin{aligned}
& v_M^{\pi^\star}(x) \geq v_M^\pi(x) \\
\iff \quad & \mathbb{E}_{\pi^\star}\left[\sum_{t=0}^\infty \gamma^t R_t \mid X_0 = x\right] \geq \mathbb{E}_\pi\left[\sum_{t=0}^\infty \gamma^t R_t \mid X_0 = x\right] \\
\iff \quad & \mathbb{E}_{\pi^\star}\left[\sum_{t=0}^\infty \gamma^t \alpha R_t \mid X_0 = x\right] \geq \mathbb{E}_\pi\left[\sum_{t=0}^\infty \gamma^t \alpha R_t \mid X_0 = x\right] && \text{multiplying both sides by } \alpha \\
\iff \quad & \mathbb{E}_{\pi^\star}\left[\sum_{t=0}^\infty \gamma^t R'_t \mid X_0 = x\right] \geq \mathbb{E}_\pi\left[\sum_{t=0}^\infty \gamma^t R'_t \mid X_0 = x\right] \\
\iff \quad & v_{M'}^{\pi^\star}(x) \geq v_{M'}^\pi(x).
\end{aligned}
$$

   Thus, $\pi^\star$ is an optimal policy for $M'$.

2. We give an example where the optimal policies differ when rewards are shifted.
   Consider an MDP with three states $\{1, 2, 3\}$ where 1 is the initial state and 3 is a terminal state. If one plays action $A$ in states 1 or 2 one transitions directly to the terminal state. Additionally, in state 1 one can play action $B$ which leads to state 2. Let every transition give a deterministic reward of $r \doteq -1$. Then it is optimal to traverse the shortest path to the terminal state, in particular, to choose action $A$ when in state 1.
   If we consider the reward $r' \doteq r + 2 = 1$, then it is optimal to traverse the longest path to the terminal state, in particular, to choose action $B$ when in state 1.

3. For an MDP $M$, we know that is optimal state-action value function satisfies Bellman's optimality equation (10.34),

$$
q_M^\star(x, a) = \mathbb{E}_{x' \mid x, a}\left[ r(x, x') + \gamma \max_{a' \in A} q_M^\star(x', a') \right].
$$

   For the MDP $M'$, we have

$$
\begin{aligned}
q_{M'}^\star(x, a) &= \mathbb{E}_{x' \mid x, a}\left[ r'(x, x') + \gamma \max_{a' \in A} q_{M'}^\star(x', a') \right] \\
&= \mathbb{E}_{x' \mid x, a}\left[ r(x, x') + f(x, x') + \gamma \max_{a' \in A} q_{M'}^\star(x', a') \right] \\
&= \mathbb{E}_{x' \mid x, a}\left[ r(x, x') + \gamma \phi(x') - \phi(x) + \gamma \max_{a' \in A} q_{M'}^\star(x', a') \right].
\end{aligned}
$$

   Reorganizing the terms, we obtain

$$
q_{M'}^\star(x, a) + \phi(x) = \mathbb{E}_{x' \mid x, a}\left[ r(x, x') + \gamma \max_{a' \in A} \left( q_{M'}^\star(x', a') + \phi(x') \right) \right].
$$

If we now define $q(x,a) \doteq q^\star_{M'}(x,a) + \phi(x)$, we have

$$q(x,a) = \mathbb{E}_{x'|x,a}\left[r(x,x') + \gamma \max_{a' \in A} q(x',a')\right].$$

This is exactly Bellman's optimality equation for the MDP $M$ with reward function $r$, and hence, $q \equiv q^\star_M$.
If we take $\pi^\star$ to be an optimal policy for $M$, then it satisfies

$$\begin{aligned}
\pi^\star(x) &\in \arg\max_{a \in A} q^\star_M(x,a) \\
&= \arg\max_{a \in A} q^\star_{M'}(x,a) + \phi(x) && \text{using the above characterization of } q^\star_M \\
&= \arg\max_{a \in A} q^\star_{M'}(x,a). && \text{using that } \phi(x) \text{ is independent of } a
\end{aligned}$$

**Solution to Problem 10.6.**

1. We compute the answer in three steps:
   (a) **Predict step**: We compute the predicted belief after action $W$ but before observing $o_1$.

$$\begin{aligned}
b'_0(F) &= b_0(F) \cdot 0.6 + b_0(\overline{F}) \cdot 0.5 = 0.55 \\
b'_0(\overline{F}) &= b_0(F) \cdot 0.4 + b_0(\overline{F}) \cdot 0.5 = 0.45.
\end{aligned}$$

   (b) **Update step**: Using the observation $o_1$ and the observation model, we update the belief.

$$\begin{aligned}
b_1(F) &= \frac{1}{Z} b'_0(F) \mathbb{P}(o_1 \mid F) = \frac{1}{Z} \cdot 0.55 \cdot 0.8 \\
b_1(\overline{F}) &= \frac{1}{Z} b'_0(\overline{F}) \mathbb{P}\big(o_1 \mid \overline{F}\big) = \frac{1}{Z} \cdot 0.45 \cdot 0.3.
\end{aligned}$$

   (c) **Normalization**: We compute the normalization constant $Z$.

$$Z = 0.55 \cdot 0.8 + 0.45 \cdot 0.3 = 0.575.$$

   Therefore,

$$\begin{aligned}
b_1(F) &= \frac{0.55 \cdot 0.8}{0.575} \approx 0.765, \\
b_1(\overline{F}) &= \frac{0.45 \cdot 0.3}{0.575} \approx 0.235.
\end{aligned}$$

2. We observe that if $A_1 = P$, then $\mathbb{P}(X_2 = F) = 0$ given any starting state, which will result in a zero in the belief update formula. Therefore,
   - $b_2 = (0,1)$ given $A_1 = P$ and $O_1 = o_1$,
   - $b_2 = (0,1)$ given $A_1 = P$ and $O_1 = o_2$.

   (a) **Case with $A_1 = W$ and $O_1 = o_1$**: Using the belief update formula (10.42),

$$b_2(F) \propto \mathbb{P}(o_1 \mid F) \cdot \big(b_1(F) \cdot p(F \mid F, W) + b_1(\overline{F}) \cdot p(F \mid \overline{F}, W)\big) \approx 0.461$$

$$b_2(\overline{F}) \propto \mathbb{P}\big(o_1 \mid \overline{F}\big) \cdot \big(b_1(F) \cdot p(\overline{F} \mid F, W) + b_1(\overline{F}) \cdot p(\overline{F} \mid \overline{F}, W)\big) \approx 0.127.$$

By rescaling the probabilities, we get $b_2 \approx (0.784, 0.216)$.

(b) **Case with $A_1 = W$ and $O_1 = o_2$**: Using the belief update formula (10.42),

$$b_2(F) \propto \mathbb{P}(o_2 \mid F) \cdot \big(b_1(F) \cdot p(F \mid F, W) + b_1(\overline{F}) \cdot p(F \mid \overline{F}, W)\big) \approx 0.115$$
$$b_2(\overline{F}) \propto \mathbb{P}\big(o_2 \mid \overline{F}\big) \cdot \big(b_1(F) \cdot p(\overline{F} \mid F, W) + b_1(\overline{F}) \cdot p(\overline{F} \mid \overline{F}, W)\big) \approx 0.296.$$

By rescaling the probabilities, we get $b_2 \approx (0.279, 0.721)$.

## *Tabular Reinforcement Learning*

**Solution to Problem 11.1.**

1. $Q^\star(A, \downarrow) = 1.355, Q^\star(G_1, \text{exit}) = 5.345, Q^\star(G_2, \text{exit}) = 0.5$
2. Repeating the given episodes infinitely often will not lead to convergence to the optimal Q-function because not all state-action pairs are visited infinitely often.
   Let us assume we observe the following episode instead of the first episode.

   Episode 3

   | $x$ | $a$ | $x'$ | $r$ |
   |---|---|---|---|
   | $A$ | $\rightarrow$ | $B$ | 0 |
   | $B$ | $\downarrow$ | $G_2$ | 0 |
   | $G_2$ | exit | | 1 |

   If we repeat episodes 2 and 3 infinitely often, Q-learning will converge to the optimal Q-function as all state-action pairs will be visited infinitely often.
3. First, recall that Q-learning is an off-policy algorithm, and hence, even if episodes are obtained off-policy, Q-learning will still converge to the optimal Q-function (if the other convergence conditions are met). Note that it only matters which state-action pairs are observed and not which policies were followed to obtain these observations.
   The "closer" the initial Q-values are to the optimal Q-function, the faster the convergence of Q-learning. However, if the convergence conditions are met, Q-learning will converge to the optimal Q-function regardless of the initial Q-values.
4. $v^\star(A) = 10$, $v^\star(B) = 10$, $v^\star(G_1) = 10$, $v^\star(G_2) = 1$

## *Model-free Reinforcement Learning*

**Solution to Problem 12.1.**

1. We have to show that

$$v^\star(x) = \max_{a \in A} r(x,a) + \gamma \mathbb{E}_{x'|x,a}\big[v^\star(x')\big]$$

for every $x \in \{1,2,\dots,7\}$. We give a derivation here for $x = 1$ and $x = 2$.

- For $x = 1$,

$$\begin{aligned} v^\star(1) &= -3 \\ \max_{a \in A} r(1,a) + \gamma \mathbb{E}_{x'|1,a}\big[v^\star(x')\big] &= -3 \end{aligned}$$

since

$$r(1,a) + \gamma \mathbb{E}_{x'|1,a}\big[v^\star(x')\big] = \begin{cases} -1 + -2 = -3 & \text{if } a = 1 \\ -1 + -3 = -4 & \text{if } a = -1. \end{cases}$$

- Likewise, for $x = 2$,

$$\begin{aligned} v^\star(2) &= -2 \\ \max_{a \in A} r(2,a) + \gamma \mathbb{E}_{x'|2,a}\big[v^\star(x')\big] &= -2 \end{aligned}$$

since

$$r(2,a) + \gamma \mathbb{E}_{x'|2,a}\big[v^\star(x')\big] = \begin{cases} -1 + -1 = -2 & \text{if } a = 1 \\ -1 + -3 = -4 & \text{if } a = -1. \end{cases}$$

2. We have

$$\begin{aligned} Q(3,-1) &= 0 + \frac{1}{2}\left(-1 + \max_{a' \in A} Q(2,a')\right) = \frac{1}{2}(-1+0) = -\frac{1}{2} \\ Q(2,1) &= 0 + \frac{1}{2}\left(-1 + \max_{a' \in A} Q(3,a')\right) = \frac{1}{2}(-1+0) = -\frac{1}{2} \\ Q(3,1) &= 0 + \frac{1}{2}\left(-1 + \max_{a' \in A} Q(4,a')\right) = \frac{1}{2}(-1+0) = -\frac{1}{2} \\ Q(4,1) &= 0 + \frac{1}{2}\left(0 + \max_{a' \in A} Q(4,a')\right) = \frac{1}{2}(0+0) = 0. \end{aligned}$$

3. We compute

$$\begin{aligned} \nabla_{\boldsymbol{w}}\, \ell(\boldsymbol{w};\tau) &= -\left(r + \gamma \max_{a' \in A} Q(x',a';\boldsymbol{w}^{\text{old}}) - Q(x,a;\boldsymbol{w})\right) \begin{bmatrix} x \\ a \\ 1 \end{bmatrix} && \text{using the derivation of Equation (12.15)} \\ &= -\left(-1 + \max_{a' \in A}\{1 - a' - 2\} - (-2 - 1 + 1)\right) \begin{bmatrix} 2 \\ -1 \\ 1 \end{bmatrix} \end{aligned}$$

$$= \begin{bmatrix} -2 \\ 1 \\ -1 \end{bmatrix}.$$

This gives

$$\begin{aligned} \boldsymbol{w}' &= \boldsymbol{w} - \alpha \nabla_{\boldsymbol{w}} \ell(\boldsymbol{w}; \tau) \\ &= \begin{bmatrix} -1 \\ 1 \\ 1 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} -2 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1/2 \\ 3/2 \end{bmatrix}. \end{aligned}$$

**Solution to Problem 12.2.** We have

$$\begin{aligned} \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(a \mid \boldsymbol{x}) &= \frac{\nabla_{\boldsymbol{\varphi}} \pi_{\boldsymbol{\varphi}}(a \mid \boldsymbol{x})}{\pi_{\boldsymbol{\varphi}}(a \mid \boldsymbol{x})} && \text{using the chain rule} \\ &= \boldsymbol{\phi}(\boldsymbol{x}, a) - \frac{\sum_{b \in A} \boldsymbol{\phi}(\boldsymbol{x}, b) \exp(\boldsymbol{\varphi}^\top \boldsymbol{\phi}(\boldsymbol{x}, b))}{\sum_{b \in A} \exp(\boldsymbol{\varphi}^\top \boldsymbol{\phi}(\boldsymbol{x}, b))} && \text{using elementary calculus} \\ &= \boldsymbol{\phi}(\boldsymbol{x}, a) - \sum_{b \in A} \pi_{\boldsymbol{\varphi}}(b \mid \boldsymbol{x}) \cdot \boldsymbol{\phi}(\boldsymbol{x}, b). \end{aligned}$$

**Solution to Problem 12.3.**

1. The result follows directly using that

$$\operatorname{Var}[f(\mathbf{X}) - g(\mathbf{X})] = \operatorname{Var}[f(\mathbf{X})] + \operatorname{Var}[g(\mathbf{X})] - 2\operatorname{Cov}[f(\mathbf{X}), g(\mathbf{X})]. \qquad \text{using Equation (1.39)}$$

2. Denote by $r(\tau)$ the discounted rewards attained by trajectory $\tau$. Let $f(\tau) \doteq r(\tau) \nabla_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau)$ and $g(\tau) \doteq b \nabla_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau)$. Recall that $\mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}[g(\tau)] = 0$, implying that

$$\begin{aligned} \operatorname{Var}[g(\tau)] &= \operatorname{Var}\left[b \nabla_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau)\right] \\ &= \mathbb{E}\left[\left(b \nabla_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau)\right)^2\right]. && \text{using the definition of variance (1.34)} \end{aligned}$$

On the other hand,

$$\begin{aligned} \operatorname{Cov}[f(\tau), g(\tau)] &= \mathbb{E}[(f(\tau) - \mathbb{E}[f(\tau)]) g(\tau)] && \text{using the definition of covariance (1.26)} \\ &= \mathbb{E}[f(\tau) g(\tau)] - \mathbb{E}[f(\tau)] \underbrace{\mathbb{E}[g(\tau)]}_{0} && \text{using linearity of expectation (1.20)} \\ &= \mathbb{E}\left[b \cdot r(\tau) \cdot \left(\nabla_{\boldsymbol{\varphi}} \Pi_{\boldsymbol{\varphi}}(\tau)\right)^2\right]. \end{aligned}$$

Therefore, if $b^2 \leq 2b \cdot r(\boldsymbol{x}, \boldsymbol{a})$ for every state $\boldsymbol{x} \in \mathcal{X}$ and action $\boldsymbol{a} \in \mathcal{A}$, then the result follows from Equation (12.100).

**Solution to Problem 12.4.** First, observe that

$$\mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}\left[G_{0:T} \nabla_{\boldsymbol{\varphi}} \log \Pi_{\boldsymbol{\varphi}}(\tau)\right] = \mathbb{E}_{\tau \sim \Pi_{\boldsymbol{\varphi}}}\left[\sum_{t=0}^{T-1} G_{0:T} \nabla_{\boldsymbol{\varphi}} \log \pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t)\right], \qquad \text{using Equation (12.30)}$$

and hence, it suffices to show

$$\begin{aligned}&\mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[\sum_{t=0}^{T-1} G_{0:T}\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t\mid\boldsymbol{x}_t)\right]\\&=\mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[\sum_{t=0}^{T-1}(G_{0:T}-b(\tau_{0:t-1}))\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t\mid\boldsymbol{x}_t)\right].\end{aligned}\tag{B.15}$$

We prove Equation (B.15) with an induction on $T$. The base case ($T=0$) is satisfied trivially. Fixing any $T$ and assuming Equation (B.15) holds for $T$, we have,

$$\begin{aligned}&\mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[\sum_{t=0}^{T}(G_{0:T}-b(\tau_{0:t-1}))\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t\mid\boldsymbol{x}_t)\right]\\&=\mathbb{E}_{\tau_{0:T-1}}\left[\mathbb{E}_{\tau_T}\left[\sum_{t=0}^{T}(G_{0:T}-b(\tau_{0:t-1}))\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t\mid\boldsymbol{x}_t)\;\middle|\;\tau_{0:T-1}\right]\right] && \text{using the tower rule (1.25)}\\&=\mathbb{E}_{\tau_{0:T-1}}\left[\sum_{t=0}^{T-1} G_{0:T}\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t\mid\boldsymbol{x}_t)\right. && \text{using the induction hypothesis}\\&\qquad\left.+\mathbb{E}_{\tau_T}\left[(G_{0:T}-b(\tau_{0:T-1}))\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_T\mid\boldsymbol{x}_T)\;\middle|\;\tau_{0:T-1}\right]\right].\end{aligned}$$

Using the score function trick for the score function $\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t\mid\boldsymbol{x}_t)$ analogously to the proof of Theorem 12.6, we have,

$$\begin{aligned}&\mathbb{E}_{\tau_T}\left[b(\tau_{0:T-1})\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_T\mid\boldsymbol{x}_T)\;\middle|\;\tau_{0:T-1}\right]\\&=\mathbb{E}_{\boldsymbol{a}_T}\left[b(\tau_{0:T-1})\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_T\mid\boldsymbol{x}_T)\;\middle|\;\tau_{0:T-1}\right]\\&=b(\tau_{0:T-1})\int\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_T\mid\boldsymbol{x}_T)\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_T\mid\boldsymbol{x}_T)\,d\boldsymbol{a}_T\\&=b(\tau_{0:T-1})\int\nabla_{\boldsymbol{\varphi}}\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_T\mid\boldsymbol{x}_T)\,d\boldsymbol{a}_T && \text{using the score function trick,}\\&=0. && \nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t\mid\boldsymbol{x}_t)=\nabla_{\boldsymbol{\varphi}}\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t|\boldsymbol{x}_t)/\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t|\boldsymbol{x}_t)\end{aligned}$$

Thus,

$$\begin{aligned}&=\mathbb{E}_{\tau_{0:T-1}}\left[\mathbb{E}_{\tau_T}\left[\sum_{t=0}^{T} G_{0:T}\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t\mid\boldsymbol{x}_t)\;\middle|\;\tau_{0:T-1}\right]\right]\\&=\mathbb{E}_{\tau\sim\Pi_{\boldsymbol{\varphi}}}\left[\sum_{t=0}^{T} G_{0:T}\nabla_{\boldsymbol{\varphi}}\log\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t\mid\boldsymbol{x}_t)\right]. && \text{using the tower rule (1.25) again}\end{aligned}$$

**Solution to Problem 12.5.** Each trajectory $\tau$ is described by four transitions,

$$\tau=(x_0,a_0,r_0,x_1,a_1,r_1,x_2,a_2,r_2,x_3,a_3,r_3,x_4).$$

Moreover, we have given

$$\pi_\theta(2\mid x)=\theta,\qquad\qquad\frac{\partial\pi_\theta(2\mid x)}{\partial\theta}=+1$$

$$\pi_\theta(1 \mid x) = 1 - \theta, \qquad \frac{\partial \pi_\theta(1 \mid x)}{\partial \theta} = -1.$$

We first compute the downstream returns for the given episode,

$$\begin{aligned}
G_{0:4} &= r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r^3 = 1 + \frac{1}{2} \cdot 0 + \frac{1}{4} \cdot 1 + \frac{1}{8} \cdot 1 = \frac{11}{8} \\
G_{1:4} &= r_1 + \gamma r_2 + \gamma^2 r_3 = 0 + \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 1 = \frac{3}{4} \\
G_{2:4} &= r_2 + \gamma r_3 = 1 + \frac{1}{2} \cdot 1 = \frac{3}{2} \\
G_{3:4} &= r_3 = 1.
\end{aligned}$$

Lastly, we can combine them to compute the policy gradient,

$$\begin{aligned}
\nabla_\theta j(\theta) &\approx \sum_{t=0}^{3} \gamma^t G_{t:4} \nabla_\theta \log \pi_\theta(a_t \mid x_t) && \text{using Monte Carlo approximation of Equation (12.40) with a single sample} \\
&= 1 \cdot \frac{11}{8} - 1 \cdot \frac{1}{2} \cdot \frac{3}{4} + 1 \cdot \frac{1}{4} \cdot \frac{3}{2} - 1 \cdot \frac{1}{8} \cdot 1 = \frac{5}{4}.
\end{aligned}$$

**Solution to Problem 12.6.**

1. The distribution on actions is given by

$$\pi_{\boldsymbol{\varphi}}(a \mid x) = \sigma(f_{\boldsymbol{\varphi}}(x))^a \cdot (1 - \sigma(f_{\boldsymbol{\varphi}}(x)))^{1-a}.$$

   To simplify the notation, we write $\mathbb{E}$ for $\mathbb{E}_{(\boldsymbol{x},a)\sim\pi_{\boldsymbol{\varphi}}}$ and $q$ for $q^{\pi_{\boldsymbol{\varphi}}}$. We get

$$\begin{aligned}
&\nabla_{\boldsymbol{\varphi}} j(\boldsymbol{\varphi}) \\
&= \mathbb{E}\left[q(\boldsymbol{x},a) \nabla_{\boldsymbol{\varphi}} \left(a \log \sigma(f_{\boldsymbol{\varphi}}(\boldsymbol{x})) + (1-a) \log(1 - \sigma(f_{\boldsymbol{\varphi}}(\boldsymbol{x})))\right)\right] && \text{using the policy gradient theorem (12.53)} \\
&= \mathbb{E}\left[q(\boldsymbol{x},a) \nabla_f \left(a \log \sigma(f) + (1-a) \log(1 - \sigma(f))\right) \nabla_{\boldsymbol{\varphi}} f_{\boldsymbol{\varphi}}(\boldsymbol{x})\right] && \text{using the chain rule} \\
&= \mathbb{E}\left[q(\boldsymbol{x},a) \nabla_f \left(-a \log(1 + e^{-f}) + (1-a) \log\left(\frac{e^{-f}}{1+e^{-f}}\right)\right) \nabla_{\boldsymbol{\varphi}} f_{\boldsymbol{\varphi}}(\boldsymbol{x})\right] && \text{using the definition of the logistic function (5.9)} \\
&= \mathbb{E}\left[q(\boldsymbol{x},a) \nabla_f \left(-f + af - \log(1 + e^{-f})\right) \nabla_{\boldsymbol{\varphi}} f_{\boldsymbol{\varphi}}(\boldsymbol{x})\right] \\
&= \mathbb{E}\left[q(\boldsymbol{x},a) \left(a - 1 + \frac{e^{-f}}{1+e^{-f}}\right) \nabla_{\boldsymbol{\varphi}} f_{\boldsymbol{\varphi}}(\boldsymbol{x})\right] \\
&= \mathbb{E}\left[q(\boldsymbol{x},a)(a - \sigma(f)) \nabla_{\boldsymbol{\varphi}} f_{\boldsymbol{\varphi}}(\boldsymbol{x})\right]. && \text{using the definition of the logistic function (5.9)}
\end{aligned}$$

   The term $a - \sigma(f)$ can be understood as a residual as it corresponds to the difference between the target action $a$ and the expected action $\sigma(f)$.

2. We have

$$\nabla_{\boldsymbol{\varphi}} j(\boldsymbol{\varphi}) = \mathbb{E}\left[q(\boldsymbol{x},a) \nabla_f \log \pi_f(a \mid \boldsymbol{x}) \nabla_{\boldsymbol{\varphi}} f(\boldsymbol{x})\right] \qquad \text{using Equation (12.53) and the chain rule}$$

$$= \mathbb{E}\Big[q(\boldsymbol{x}, a)\boldsymbol{\nabla}_f \left(\log h(a) + af - A(f)\right)\boldsymbol{\nabla}_{\boldsymbol{\varphi}} f(\boldsymbol{x})\Big]$$
$$= \mathbb{E}\Big[q(\boldsymbol{x}, a)(a - \boldsymbol{\nabla}_f A(f))\boldsymbol{\nabla}_{\boldsymbol{\varphi}} f(\boldsymbol{x})\Big].$$

3. We have $\boldsymbol{\nabla}_f A(f) = \sigma(f)$. We are therefore looking for a function $A(f)$ whose derivative is $\sigma(f) = \frac{1}{1+e^{-f}} = \frac{e^f}{1+e^f}$. With this equality of the sigmoid we can find the integral, and we have $A(f) = \log(1 + e^f) + c$. Let us confirm that this gives us the Bernoulli distribution with $c = 0$:

$$\begin{aligned}\pi_f(a \mid \boldsymbol{x}) &= h(a)\exp(af - \log(1 + e^f)) \\ &= h(a)\frac{e^{af}}{1 + e^f} \\ &= h(a)\begin{cases}\sigma(f) & \text{if } a = 1 \\ 1 - \sigma(f) & \text{if } a = 0.\end{cases}\end{aligned}$$

   This is the Bernoulli distribution with parameter $\sigma(f)$ where we have $h(a) = 1$.
4. Using that $\boldsymbol{\nabla}_f A(f) = f$, we immediately get

$$\boldsymbol{\nabla}_{\boldsymbol{\varphi}} j(\boldsymbol{\varphi}) = \mathbb{E}\big[q(\boldsymbol{x}, a)(a - f)\boldsymbol{\nabla}_{\boldsymbol{\varphi}} f(\boldsymbol{x})\big].$$

5. No, we cannot use the reparameterization trick since we do not know how the states $\boldsymbol{x}$ depend on action $a$. These dependencies are determined by the unknown dynamics of the environment. Nonetheless, we can apply it after sampling an episode according to a policy and then updating policy parameters in hindsight. This is for example done by the soft actor-critic (SAC) algorithm (cf. Section 12.6).

**Solution to Problem 12.7.**

1. Equation (12.84) can be written as

$$\begin{aligned}&\mathrm{KL}(\Pi_{\boldsymbol{\varphi}} \| \Pi_\star) \\ &= \sum_{t=1}^{T} \mathbb{E}_{(\boldsymbol{x}_t, \boldsymbol{a}_t) \sim \Pi_{\boldsymbol{\varphi}}}\left[-\frac{1}{\lambda} r(\boldsymbol{x}_t, \boldsymbol{a}_t) - \mathrm{H}\big[\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t)\big]\right] \\ &= \sum_{t=1}^{T} \mathbb{E}_{(\boldsymbol{x}_t, \boldsymbol{a}_t) \sim \Pi_{\boldsymbol{\varphi}}}\big[-\log \beta(\boldsymbol{a}_t \mid \boldsymbol{x}_t) - \mathrm{H}\big[\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t)\big]\big].\end{aligned}$$

   Adding and subtracting $\log Z(\boldsymbol{x}_t)$ gives

$$\begin{aligned}&= \sum_{t=1}^{T} \mathbb{E}_{(\boldsymbol{x}_t, \boldsymbol{a}_t) \sim \Pi_{\boldsymbol{\varphi}}}\big[\mathrm{S}[\hat{\pi}(\boldsymbol{a}_t \mid \boldsymbol{x}_t)] - \log Z(\boldsymbol{x}_t) - \mathrm{H}\big[\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t)\big]\big] \\ &= \sum_{t=1}^{T} \mathbb{E}_{\boldsymbol{x}_t \sim \Pi_{\boldsymbol{\varphi}}}\big[\mathrm{H}\big[\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t) \| \hat{\pi}(\cdot \mid \boldsymbol{x}_t)\big] - \log Z(\boldsymbol{x}_t) - \mathrm{H}\big[\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t)\big]\big]\end{aligned}$$

   using the definition of cross-entropy (5.32)

$$= \sum_{t=1}^{T} \mathbb{E}_{\boldsymbol{x}_t \sim \Pi_{\boldsymbol{\varphi}}} \big[ \mathrm{KL}\big(\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t) \| \hat{\pi}(\cdot \mid \boldsymbol{x}_t)\big) - \log Z(\boldsymbol{x}_t) \big].$$

using the definition of KL-divergence (5.34)

2. We prove the statement by (reverse) induction on $t$. For the base case, note that the term

$$\mathbb{E}_{\boldsymbol{x}_T \sim \Pi_{\boldsymbol{\varphi}}} \big[ \mathrm{KL}\big(\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_T) \| \hat{\pi}(\cdot \mid \boldsymbol{x}_T)\big) - \log Z(\boldsymbol{x}_T) \big]$$

is minimized for $\pi_{\boldsymbol{\varphi}} \equiv \hat{\pi}$. The KL-divergence then evaluates to zero, and we are left only with the $\log Z(\boldsymbol{x}_T)$ term.
For the inductive step, fix any $1 \leq t < T$ and $\pi^\star(\boldsymbol{a}_t \mid \boldsymbol{x}_t)$ must minimize the two terms

$$\begin{aligned} &\mathbb{E}_{\boldsymbol{x}_t \sim \Pi_{\boldsymbol{\varphi}}} \big[ \mathrm{KL}\big(\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t) \| \hat{\pi}(\cdot \mid \boldsymbol{x}_t)\big) - \log Z(\boldsymbol{x}_t) \big] \\ &\quad + \mathbb{E}_{(\boldsymbol{x}_t, \boldsymbol{a}_t) \sim \Pi_{\boldsymbol{\varphi}}} \Big[ \mathbb{E}_{\boldsymbol{x}_{t+1} \sim p(\cdot \mid \boldsymbol{x}_t, \boldsymbol{a}_t)} \big[ -\log Z(\boldsymbol{x}_{t+1}) \big] \Big] \end{aligned}$$

where the first term stems directly from the objective (12.106) and the second term represents the contribution of $\pi_{\boldsymbol{\varphi}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t)$ to all subsequent terms. Letting $\beta^\star(\boldsymbol{a} \mid \boldsymbol{x}) \doteq \exp(q^\star(\boldsymbol{x}, \boldsymbol{a}))$, $Z^\star(\boldsymbol{x}) \doteq \int_{\mathcal{A}} \beta^\star(\boldsymbol{a} \mid \boldsymbol{x})\, d\boldsymbol{a}$, and recalling that we denote by $\pi^\star(\cdot \mid \boldsymbol{x})$ the policy $\beta^\star(\cdot \mid \boldsymbol{x}) / Z^\star(\boldsymbol{x})$, this objective can be reexpressed as

$$= \mathbb{E}_{\boldsymbol{x}_t \sim \Pi_{\boldsymbol{\varphi}}} \big[ \mathrm{KL}\big(\pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}_t) \| \pi^\star(\cdot \mid \boldsymbol{x}_t)\big) - \log Z^\star(\boldsymbol{x}_t) \big]$$

which is minimized for $\pi_{\boldsymbol{\varphi}} \equiv \pi^\star$, leaving only the $\log Z^\star(\boldsymbol{x}_t)$ term. It remains only to observe that $\beta^\star(\boldsymbol{a}_T \mid \boldsymbol{x}_T) = \beta(\boldsymbol{a}_T \mid \boldsymbol{x}_T)$, so in the final state $\boldsymbol{x}_T$, $\log Z^\star(\boldsymbol{x}_T) = \log Z(\boldsymbol{x}_T)$ and the policies $\pi^\star$ and $\hat{\pi}$ coincide.

**Solution to Problem 12.8.**

1. Let $\mathcal{O}$ denote the event that the response $\boldsymbol{y}$ is optimal. Since the prior over actions (i.e., responses) is not uniform, we have

$$\begin{aligned} \Pi_\star(\boldsymbol{y} \mid \boldsymbol{x}) &= p(\boldsymbol{y} \mid \boldsymbol{x}, \mathcal{O}) \\ &\propto p(\boldsymbol{y} \mid \boldsymbol{x}) \cdot p(\mathcal{O} \mid \boldsymbol{x}, \boldsymbol{y}) && \text{using Bayes' rule (1.45)} \\ &= \Pi_{\boldsymbol{\varphi}^{\text{init}}}(\boldsymbol{y} \mid \boldsymbol{x}) \exp\left(\frac{1}{\lambda} r(\boldsymbol{y} \mid \boldsymbol{x})\right). \end{aligned} \tag{B.16}$$

The derivation is then analogous to Equation (12.84):

$$\begin{aligned} &\arg\min_{\boldsymbol{\varphi}} \mathrm{KL}\big(\Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}) \| \Pi_\star(\cdot \mid \boldsymbol{x})\big) \\ &= \arg\min_{\boldsymbol{\varphi}} \mathrm{H}\big[\Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x}) \| \Pi_\star(\cdot \mid \boldsymbol{x})\big] - \mathrm{H}\big[\Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})\big] && \text{using the definition of KL-divergence (5.34)} \\ &= \arg\max_{\boldsymbol{\varphi}} \mathbb{E}_{\boldsymbol{y} \sim \Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})} \big[\log \Pi_\star(\boldsymbol{y} \mid \boldsymbol{x}) - \log \Pi_{\boldsymbol{\varphi}}(\boldsymbol{y} \mid \boldsymbol{x})\big] && \text{using the definition of cross-entropy (5.32) and entropy (5.27)} \\ &= \arg\max_{\boldsymbol{\varphi}} \mathbb{E}_{\boldsymbol{y} \sim \Pi_{\boldsymbol{\varphi}}(\cdot \mid \boldsymbol{x})} \Big[r(\boldsymbol{y} \mid \boldsymbol{x}) + \lambda \log \Pi_{\boldsymbol{\varphi}^{\text{init}}}(\boldsymbol{y} \mid \boldsymbol{x}) - \lambda \log \Pi_{\boldsymbol{\varphi}}(\boldsymbol{y} \mid \boldsymbol{x})\Big] && \text{using Equation (B.16) and simplifying} \\ &= \arg\max_{\boldsymbol{\varphi}} J_\lambda(\boldsymbol{\varphi}; \boldsymbol{\varphi}^{\text{init}} \mid \boldsymbol{x}). && \text{using the definition of cross-entropy (5.32) and entropy (5.27)} \end{aligned}$$

2. Recall that the KL-divergence is minimized at 0 if and only if the two distributions are identical. The desired result follows from (1) and Equation (B.16).

## *Model-based Reinforcement Learning*

**Solution to Problem 13.1.**

1. To simplify the notation, we use $z_{t,k}$ as shorthand for $(x_{t,k}, \pi_t(x_{t,k}))$ (and similarly $\widehat{z}_{t,k}$ for $(\widehat{x}_{t,k}, \pi_t(\widehat{x}_{t,k}))$). The base case is implied trivially. For the induction step, assume that Equation (13.41) holds at iteration $k$. We have

$$\begin{aligned}\left\|\widehat{x}_{t,k+1} - x_{t,k+1}\right\| &= \left\|\widehat{f}_{t-1}(\widehat{z}_{t,k}) - f(z_{t,k})\right\| \\ &\leq \left\|\widehat{f}_{n-1}(\widehat{z}_{t,k}) - f(\widehat{z}_{n,k})\right\| + \left\|f(\widehat{z}_{t,k}) - f(z_{t,k})\right\| \\ &\leq 2\beta_t \sigma_{t-1}(\widehat{z}_{t,k}) + L_1 \left\|\widehat{x}_{t,k} - x_{t,k}\right\|\end{aligned}$$

adding and subtracting $f(\widehat{z}_{n,k})$ and using Cauchy-Schwarz

where the final inequality follows with high probability from the assumption that the confidence intervals are well-calibrated (cf. Equation (13.22)) and the assumed Lipschitzness.

$$\begin{aligned}&= 2\beta_t[\sigma_{t-1}(z_{t,k}) + \sigma_{t-1}(\widehat{z}_{t,k}) - \sigma_{t-1}(z_{t,k})] \\ &\qquad + L_1 \left\|\widehat{x}_{t,k} - x_{t,k}\right\|\end{aligned}$$

Once more using Lipschitz continuity, we obtain

$$\begin{aligned}&\leq 2\beta_t \left[\sigma_{t-1}(z_{t,k}) + L_2 \left\|\widehat{x}_{t,k} - x_{t,k}\right\|\right] \\ &\qquad + L_1 \left\|\widehat{x}_{t,k} - x_{t,k}\right\| \\ &= 2\beta_t \sigma_{t-1}(z_{t,k}) + \alpha_t \left\|\widehat{x}_{t,k} - x_{t,k}\right\|\end{aligned}$$

where $\alpha_t \doteq (L_1 + 2\beta_t L_2)$. By the induction hypothesis,

$$\leq 2\beta_t \sum_{l=0}^{k} \alpha_t^{k-l} \sigma_{t-1}(z_{t,l}).$$

This is identical to the analysis of UCB from Problem 9.3, only that here errors compound along the trajectory.

2. The assumption that $\alpha_t \geq 1$ implies that

$$\left\|\widehat{x}_{t,k} - x_{t,k}\right\| \leq 2\beta_t \alpha_t^{H-1} \sum_{l=0}^{k-1} \sigma_{t-1}(z_{t,l}). \tag{B.17}$$

Moreover, by definition of $\pi_t$, we have with high probability that $J_H(\pi_t; \widehat{f}) \geq J_H(\pi^\star; f)$. This is because $\pi_t$ maximizes reward under *optimistic* dynamics. Thus,

$$r_t = \sum_{k=0}^{H-1} r(x_{t,k}, \pi^\star(x_{t,k})) - r(x_{t,k}, \pi_t(x_{t,k}))$$

$$
\begin{aligned}
&\leq \sum_{k=0}^{H-1} r(\widehat{x}_{t,k}, \pi_t(\widehat{x}_{t,k})) - r(x_{t,k}, \pi_t(x_{t,k})) \\
&\leq L_3 \sum_{k=0}^{H-1} \left\| \widehat{x}_{t,k} - x_{t,k} \right\| && \text{using Lipschitz-continuity of } r \\
&\leq 2\beta_t \alpha_t^{H-1} L_3 \sum_{k=0}^{H-1} \sum_{l=0}^{k-1} \sigma_{t-1}(z_{t,l}) && \text{using Equation (B.17)} \\
&\leq 2\beta_t H \alpha_t^{H-1} L_3 \sum_{k=0}^{H-1} \sigma_{t-1}(z_{t,k}).
\end{aligned}
$$

3. Let us first bound $R_T^2$. By the Cauchy-Schwarz inequality,

$$
\begin{aligned}
R_T^2 &\leq T \sum_{t=1}^{T} r_t^2 \\
&\leq O\left( T \sum_{t=1}^{T} \beta_t^2 H^2 \alpha_t^{2(H-1)} \left( \sum_{k=0}^{H-1} \sigma_{t-1}(z_{t,k}) \right)^2 \right) && \text{using (2)} \\
&\leq O\left( T \beta_T^2 H^3 \alpha_T^{2(H-1)} \sum_{t=1}^{T} \sum_{k=0}^{H-1} \sigma_{t-1}^2(z_{t,k}) \right). && \text{using Cauchy-Schwarz and assuming w.l.o.g. that } \beta_t \text{ is monotonically increasing}
\end{aligned}
$$

Taking the square root, we obtain

$$
\begin{aligned}
R_T &\leq O\left( \beta_T H^{\frac{3}{2}} \alpha_T^{H-1} \sqrt{T \sum_{t=1}^{T} \sum_{k=0}^{H-1} \sigma_{t-1}^2(z_{t,k})} \right) \\
&\leq O\Big( \beta_T H^{\frac{3}{2}} \alpha_T^{H-1} \sqrt{T \Gamma_T} \Big).
\end{aligned}
$$

## *Mathematical Background*

**Solution to Problem A.1.** Using that $X$ is zero-mean, we have that

$$
\mathbb{E}\left[ \overline{X}_n^2 \right] = \mathrm{Var}\left[ \overline{X}_n \right] = \frac{1}{n} \mathrm{Var}[X] \qquad \text{and}
$$

$$
\mathbb{E}\left[ \frac{1}{n} \sum_{i=1}^{n} {X_i}^2 \right] = \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}\left[ {X_i}^2 \right] = \mathrm{Var}[X].
$$

Thus,

$$
\mathbb{E}\left[ S_n^2 \right] = \frac{n}{n-1} \left( \mathbb{E}\left[ \frac{1}{n} \sum_{i=1}^{n} {X_i}^2 \right] - \mathbb{E}\left[ \overline{X}_n^2 \right] \right) = \mathrm{Var}[X].
$$

**Solution to Problem A.2.**

1. W.l.o.g. we assume that $X$ is continuous. We have

$$
\begin{aligned}
\mathbb{E}X &= \int_0^\infty x \cdot p(x)\, dx && \text{using the definition of expectation (1.19)} \\
&= \underbrace{\int_0^\epsilon x \cdot p(x)\, dx}_{\geq 0} + \int_\epsilon^\infty x \cdot p(x)\, dx \geq \epsilon \underbrace{\int_\epsilon^\infty \cdot p(x)\, dx}_{\mathbb{P}(X \geq \epsilon)}.
\end{aligned}
$$

2. Consider the non-negative random variable $Y \doteq (X - \mathbb{E}X)^2$. We have

$$\begin{aligned}\mathbb{P}(|X - \mathbb{E}X| \geq \epsilon) &= \mathbb{P}\Big((X - \mathbb{E}X)^2 \geq \epsilon^2\Big) \\ &\leq \frac{\mathbb{E}\big[(X - \mathbb{E}X)^2\big]}{\epsilon^2} && \text{using Markov's inequality (A.71)} \\ &= \frac{\mathrm{Var}X}{\epsilon^2}. && \text{using the definition of variance (1.34)}\end{aligned}$$

3. Fix any $\epsilon > 0$. Applying Chebyshev's inequality and noting that $\mathbb{E}\overline{X}_n = \mathbb{E}X$, we obtain

$$\mathbb{P}\big(\big|\overline{X}_n - \mathbb{E}X\big| \geq \epsilon\big) \leq \frac{\mathrm{Var}\overline{X}_n}{\epsilon^2}.$$

We further have for the variance of the sample mean that

$$\mathrm{Var}\overline{X}_n = \mathrm{Var}\left[\frac{1}{n}\sum_{i=1}^{n} X_i\right] = \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}[X_i] = \frac{\mathrm{Var}X}{n}.$$

Thus,

$$\lim_{n\to\infty}\mathbb{P}\big(\big|\overline{X}_n - \mathbb{E}X\big| \geq \epsilon\big) \leq \lim_{n\to\infty}\frac{\mathrm{Var}X}{\epsilon^2 n} = 0$$

which is precisely the definition of $\overline{X}_n \xrightarrow{\mathbb{P}} \mathbb{E}X$.

**Solution to Problem A.3.** Fix a $\lambda > 0$. Using a first-order expansion, we have

$$f(\boldsymbol{x} + \lambda\boldsymbol{d}) = f(\boldsymbol{x}) + \lambda\boldsymbol{\nabla}f(\boldsymbol{x})^\top\boldsymbol{d} + o(\lambda\,\|\boldsymbol{d}\|_2).$$

Dividing by $\lambda$ yields,

$$\frac{f(\boldsymbol{x} + \lambda\boldsymbol{d}) - f(\boldsymbol{x})}{\lambda} = \boldsymbol{\nabla}f(\boldsymbol{x})^\top\boldsymbol{d} + \underbrace{\frac{o(\lambda\,\|\boldsymbol{d}\|_2)}{\lambda}}_{\to 0}.$$

Taking the limit $\lambda \to 0$ gives the desired result.

# *Bibliography*


Eitan Altman. *Constrained Markov decision processes: stochastic modeling*. Routledge, 1999.

Sebastian Ament, Samuel Daulton, David Eriksson, Maximilian Balandat, and Eytan Bakshy. Unexpected improvements to expected improvement for bayesian optimization. In *NeurIPS*, volume 36, 2024.

Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *NeurIPS*, volume 30, 2017.

Javier Antorán, David Janz, James U Allingham, Erik Daxberger, Riccardo Rb Barbano, Eric Nalisnick, and José Miguel Hernández-Lobato. Adapting the linearised laplace model evidence for modern deep learning. In *ICML*, 2022.

Yarden As, Ilnura Usmanova, Sebastian Curi, and Andreas Krause. Constrained policy optimization via bayesian world models. In *ICLR*, 2022.

Marco Bagatella, Jonas Hübotter, Georg Martius, and Andreas Krause. Active fine-tuning of generalist policies. *arXiv preprint arXiv:2410.05026*, 2024.

Dominique Bakry and Michel Émery. Diffusions hypercontractives. In *Séminaire de Probabilités XIX 1983/84: Proceedings*. Springer, 2006.

Felix Berkenkamp, Matteo Turchetta, Angela Schoellig, and Andreas Krause. Safe model-based reinforcement learning with stability guarantees. In *NeurIPS*, volume 30, 2017.

Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural network. In *ICML*, 2015.

Léon Bottou. Online learning and stochastic approximations. *On-line learning in neural networks*, 17(9), 1998.

Ralph Allan Bradley and Milton E Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4), 1952.

Ronen I Brafman and Moshe Tennenholtz. R-max-a general polynomial time algorithm for near-optimal reinforcement learning. *JMLR*, 3, 2002.

Sébastien Bubeck, Nicolo Cesa-Bianchi, and Gábor Lugosi. Bandits with heavy tail. *IEEE Transactions on Information Theory*, 59(11), 2013.

Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, Jérémy Scheurer, Javier Rando, Rachel Freedman, Tomasz Korbak, David Lindner, Pedro Freire, et al. Open problems and fundamental limitations of reinforcement learning from human feedback. *arXiv preprint arXiv:2307.15217*, 2023.

Ariel Caticha. Entropy, information, and the updating of probabilities. *Entropy*, 23(7), 2021.

Ariel Caticha and Adom Giffin. Updating probabilities. In *AIP Conference Proceedings*, volume 872. American Institute of Physics, 2006.

Kathryn Chaloner and Isabella Verdinelli. Bayesian experimental design: A review. *Statistical Science*, 1995.

Tianqi Chen, Emily Fox, and Carlos Guestrin. Stochastic gradient hamiltonian monte carlo. In *ICML*, 2014.

Sayak Ray Chowdhury and Aditya Gopalan. On kernelized multi-armed bandits. In *ICML*, 2017.

Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *NeurIPS*, volume 31, 2018.

Richard T Cox. *Algebra of probable inference*. Johns Hopkins Press, 1961.

Sebastian Curi, Felix Berkenkamp, and Andreas Krause. Efficient model-based reinforcement learning through optimistic policy search and planning. In *NeurIPS*, volume 33, 2020.

Sebastian Curi, Armin Lederer, Sandra Hirche, and Andreas Krause. Safe reinforcement learning via confidence-based filters. In *CDC*, 2022.

Erik Daxberger, Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, Matthias Bauer, and Philipp Hennig. Laplace redux-effortless bayesian deep learning. In *NeurIPS*, volume 34, 2021.

Bruno De Finetti. *Theory of probability: A critical introductory treatment*. John Wiley, 1970.

Marc Deisenroth and Carl E Rasmussen. Pilco: A model-based and data-efficient approach to policy search. In *ICML*, 2011.

Marc Peter Deisenroth, A Aldo Faisal, and Cheng Soon Ong. *Mathematics for machine learning*. Cambridge University Press, 2020.

Joseph Leo Doob. Application of the theory of martingales. *Actes du Colloque International Le Calcul des Probabilités et ses applications (Lyon, 28 Juin – 3 Juillet, 1948)*, 1949.

Simon Duane, Anthony D Kennedy, Brian J Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics letters B*, 195 (2), 1987.

Nikita Durasov, Timur Bagautdinov, Pierre Baque, and Pascal Fua. Masksembles for uncertainty estimation. In *CVPR*, 2021.

David Duvenaud. *Automatic model construction with Gaussian processes*. PhD thesis, University of Cambridge, 2014.

David Duvenaud and Ryan P Adams. Black-box stochastic variational inference in five lines of python. In *NeurIPS Workshop on Black-box Learning and Inference*, 2015.

Eyal Even-Dar and Yishay Mansour. Convergence of optimistic and incremental q-learning. In *NeurIPS*, volume 14, 2001.

Eyal Even-Dar, Yishay Mansour, and Peter Bartlett. Learning rates for q-learning. *JMLR*, 5(1), 2003.

Matthew Fellows, Anuj Mahajan, Tim GJ Rudner, and Shimon Whiteson. Virel: A variational inference framework for reinforcement learning. In *NeurIPS*, volume 32, 2019.

Karl Friston. The free-energy principle: a unified brain theory? *Nature reviews neuroscience*, 11(2), 2010.

Karl Friston, Francesco Rigoli, Dimitri Ognibene, Christoph Mathys, Thomas Fitzgerald, and Giovanni Pezzulo. Active inference and epistemic value. *Cognitive neuroscience*, 6(4), 2015.

Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *ICML*, 2018.

Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Insights and applications. In *ICML Workshop on Deep Learning*, volume 1, 2015.

Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *ICML*, 2016.

Yarin Gal, Riashat Islam, and Zoubin Ghahramani. Deep bayesian active learning with image data. In *ICML*, 2017.

Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, Greg Kochanski, John Karro, and David Sculley. Google vizier: A service for black-box optimization. In *ACM SIGKDD*, 2017.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

Jackson Gorham and Lester Mackey. Measuring sample quality with stein's method. In *NeurIPS*, volume 28, 2015.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *ICML*, 2017.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *ICML*, 2018a.

Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018b.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *ICML*, 2019.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *ICLR*, 2020.

Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *ICLR*, 2021.

Nicolas Heess, Gregory Wayne, David Silver, Timothy Lillicrap, Tom Erez, and Yuval Tassa. Learning continuous control policies by stochastic value gradients. In *NeurIPS*, volume 28, 2015.

Philipp Hennig and Christian J Schuler. Entropy search for information-efficient global optimization. *JMLR*, 13(6), 2012.

James Hensman, Alexander Matthews, and Zoubin Ghahramani. Scalable variational gaussian process classification. In *AISTATS*, 2015.

Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.

Jonas Hübotter, Bhavya Sukhija, Lenart Treven, Yarden As, and Andreas Krause. Transductive active learning: Theory and applications. In *NeurIPS*, 2024.

Jonas Hübotter, Sascha Bongni, Ido Hakimi, and Andreas Krause. Efficiently learning at test-time: Active fine-tuning of llms. In *ICLR*, 2025.

Carl Hvarfner, Frank Hutter, and Luigi Nardi. Joint entropy search for maximally-informed bayesian optimization. In *NeurIPS*, volume 35, 2022.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In *UAI*, 2018.

Tommi Jaakkola, Michael Jordan, and Satinder Singh. Convergence of stochastic iterative dynamic programming algorithms. In *NeurIPS*, volume 6, 1993.

Edwin T Jaynes. Prior probabilities. *IEEE Transactions on systems science and cybernetics*, 4(3), 1968.

Edwin T Jaynes. *Probability theory: The logic of science*. Cambridge university press, 2002.

Kirthevasan Kandasamy, Akshay Krishnamurthy, Jeff Schneider, and Barnabás Póczos. Parallelised bayesian optimisation via thompson sampling. In *AISTATS*, 2018.

Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? In *NeurIPS*, volume 30, 2017.

Johannes Kirschner and Andreas Krause. Information directed sampling and bandits with heteroscedastic noise. In *COLT*, 2018.

Torsten Koller, Felix Berkenkamp, Matteo Turchetta, and Andreas Krause. Learning-based model predictive control for safe exploration. In *CDC*, 2018.

Andreas Krause and Daniel Golovin. Submodular function maximization. *Tractability*, 3, 2014.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *NeurIPS*, volume 30, 2017.

Lucien Le Cam. *Asymptotic methods in statistical decision theory*. Springer Science & Business Media, 1986.

Lucien Marie Le Cam and Grace Lo Yang. *Asymptotics in statistics: some basic concepts*. Springer Science & Business Media, 2000.

David A Levin and Yuval Peres. *Markov chains and mixing times*, volume 107. American Mathematical Soc., 2017.

Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *ICLR*, 2016.

David Lindner and Mennatallah El-Assady. Humans are not boltzmann distributions: Challenges and opportunities for modelling human feedback and interaction in reinforcement learning. *arXiv preprint arXiv:2206.13316*, 2022.

Qiang Liu and Dilin Wang. Stein variational gradient descent: A general purpose bayesian inference algorithm. In *NeurIPS*, volume 29, 2016.

Jianfeng Lu, Yulong Lu, and James Nolen. Scaling limit of the stein variational gradient descent: The mean field regime. *SIAM Journal on Mathematical Analysis*, 51(2), 2019.

Yi-An Ma, Yuansi Chen, Chi Jin, Nicolas Flammarion, and Michael I Jordan. Sampling can be faster than optimization. *Proceedings of the National Academy of Sciences*, 116(42), 2019.

David JC MacKay. Information-based objective functions for active data selection. *Neural computation*, 4(4), 1992.

Nicolas Menet, Jonas Hübotter, Parnian Kassraie, and Andreas Krause. Lite: Efficiently estimating gaussian probability of maximality. In *AISTATS*, 2025.

Jeffrey W Miller. A detailed treatment of doob's theorem. *arXiv preprint arXiv:1801.03122*, 2018.

W Jeffrey Miller. Lecture notes on advanced stochastic modeling. *Duke University, Durham, NC*, 2016.

Beren Millidge, Alexander Tschantz, Anil K Seth, and Christopher L Buckley. On the relationship between active inference and control as inference. In *IWAI Workshop on Active Inference*. Springer, 2020.

Beren Millidge, Alexander Tschantz, and Christopher L Buckley. Whence the expected free energy? *Neural Computation*, 33(2), 2021.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540), 2015.

Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *ICML*, 2016.

Shakir Mohamed, Mihaela Rosca, Michael Figurnov, and Andriy Mnih. Monte carlo gradient estimation in machine learning. *JMLR*, 21(132), 2020.

Andrew W. Moore. Prediction and search in probabilistic worlds: Markov systems, markov decision processes, and dynamic programming. `https://autonlab.org/assets/tutorials/mdp09.pdf`, 2002.

Kevin P Murphy. Conjugate bayesian analysis of the gaussian distribution. *def*, 1(2$\sigma$2):16, 2007.

Mojmír Mutnỳ. *Modern Adaptive Experiment Design: Machine Learning Perspective*. PhD thesis, ETH Zurich, 2024.

Jayakrishnan Nair, Adam Wierman, and Bert Zwart. *The Fundamentals of Heavy Tails*, volume 53. Cambridge University Press, 2022.

George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for maximizing submodular set functions. *Mathematical programming*, 14, 1978.

Vu Nguyen, Sunil Gupta, Santu Rana, Cheng Li, and Svetha Venkatesh. Regret for expected improvement over the best-observed value and stopping condition. In *ACML*, 2017.

OpenAI. Gpt-4 technical report. Technical report, OpenAI, 2023.

Manfred Opper and Cédric Archambeau. The variational gaussian approximation revisited. *Neural computation*, 21(3), 2009.

Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. Ogbench: Benchmarking offline goal-conditioned rl. *arXiv preprint arXiv:2410.20092*, 2024.

Thomas Parr, Giovanni Pezzulo, and Karl J Friston. *Active inference: the free energy principle in mind, brain, and behavior*. MIT Press, 2022.

Kaare Brandt Petersen, Michael Syskind Pedersen, et al. The matrix cookbook. *Technical University of Denmark*, 7(15), 2008.

Matthias Plappert, Marcin Andrychowicz, Alex Ray, Bob McGrew, Bowen Baker, Glenn Powell, Jonas Schneider, Josh Tobin, Maciek Chociej, Peter Welinder, et al. Multi-goal reinforcement learning: Challenging robotics environments and request for research. *arXiv preprint arXiv:1802.09464*, 2018.

Simon JD Prince. *Understanding Deep Learning*. MIT press, 2023.

Joaquin Quinonero-Candela and Carl Edward Rasmussen. A unifying view of sparse approximate gaussian process regression. *JMLR*, 6, 2005.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*, volume 36, 2023.

Maxim Raginsky, Alexander Rakhlin, and Matus Telgarsky. Non-convex learning via stochastic gradient langevin dynamics: a nonasymptotic analysis. In *COLT*, 2017.

Ali Rahimi, Benjamin Recht, et al. Random features for large-scale kernel machines. In *NeurIPS*, 2007.

Tom Rainforth, Adam Foster, Desi R Ivanova, and Freddie Bickford Smith. Modern bayesian experimental design. *Statistical Science*, 39(1), 2024.

Rajesh Ranganath, Sean Gerrish, and David Blei. Black box variational inference. In *AISTATS*, 2014.

Gareth O. Roberts and Jeffrey S. Rosenthal. General state space Markov chains and MCMC algorithms. *Probability Surveys*, 1, 2004.

Philip A Romero, Andreas Krause, and Frances H Arnold. Navigating the protein fitness landscape with gaussian processes. *Proceedings of the National Academy of Sciences*, 110(3), 2013.

Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.

Stuart Russell and Peter Norvig. *Artificial intelligence: a modern approach*. Prentice Hall, 2002.

Daniel Russo and Benjamin Van Roy. Learning to optimize via information-directed sampling. In *NeurIPS*, volume 27, 2014.

Daniel Russo and Benjamin Van Roy. An information-theoretic analysis of thompson sampling. *JMLR*, 17(1), 2016.

Grant Sanderson. But what is the fourier transform? a visual introduction, 2018. URL `https://www.youtube.com/watch?v=spUNpyF58BY`.

Bernhard Schölkopf, Ralf Herbrich, and Alex J Smola. A generalized representer theorem. In *COLT*. Springer, 2001.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *ICML*, 2015.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *ICLR*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Guy Shani, Joelle Pineau, and Robert Kaplow. A survey of point-based pomdp solvers. *Autonomous Agents and Multi-Agent Systems*, 27(1), 2013.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587), 2016.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

Satinder Singh, Tommi Jaakkola, Michael L Littman, and Csaba Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine learning*, 38(3), 2000.

Niranjan Srinivas, Andreas Krause, Sham M Kakade, and Matthias Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. In *ICML*, 2010.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *JMLR*, 15(1), 2014.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. In *NeurIPS*, volume 33, 2020.

Bhavya Sukhija, Lenart Treven, Cansu Sancaktar, Sebastian Blaes, Stelian Coros, and Andreas Krause. Optimistic active exploration of dynamical systems. In *NeurIPS*, volume 36, pages 38122–38153, 2023a.

Bhavya Sukhija, Matteo Turchetta, David Lindner, Andreas Krause, Sebastian Trimpe, and Dominik Baumann. Gosafeopt: Scalable safe exploration for global optimization of dynamical systems. *Artificial Intelligence*, 320, 2023b.

Bhavya Sukhija, Lenart Treven, Carmelo Sferrazza, Florian Dörfler, Pieter Abbeel, and Andreas Krause. Sombrl: Scalable and optimistic model-based rl. In *NeurIPS*, 2025.

Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

Martin A Tanner. *Tools for statistical inference*. Springer, 1991.

Yee Whye Teh, Alexandre H Thiery, and Sebastian J Vollmer. Consistency and fluctuations for stochastic gradient langevin dynamics. *JMLR*, 17, 2016.

Michalis Titsias and Miguel Lázaro-Gredilla. Doubly stochastic variational bayes for non-conjugate inference. In *ICML*, 2014.

Lenart Treven, Jonas Hübotter, Bhavya Sukhija, Florian Dörfler, and Andreas Krause. Efficient exploration in continuous-time model-based reinforcement learning. In *NeurIPS*, volume 36, 2024.

Matteo Turchetta, Felix Berkenkamp, and Andreas Krause. Safe exploration for interactive machine learning. In *NeurIPS*, volume 32, 2019.

Sattar Vakili, Kia Khezeli, and Victor Picheny. On information gain and regret bounds in gaussian process bandits. In *AISTATS*, 2021.

Aad W Van der Vaart. *Asymptotic statistics*, volume 3. Cambridge university press, 2000.

Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *AAAI*, volume 30, 2016.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, volume 30, 2017.

Santosh Vempala and Andre Wibisono. Rapid convergence of the unadjusted langevin algorithm: Isoperimetry suffices. In *NeurIPS*, volume 32, 2019.

Li Wan, Matthew Zeiler, Sixin Zhang, Yann Le Cun, and Rob Fergus. Regularization of neural networks using dropconnect. In *ICML*, 2013.

Yuhui Wang, Hao He, and Xiaoyang Tan. Truly proximal policy optimization. In *UAI*, 2020.

Zi Wang and Stefanie Jegelka. Max-value entropy search for efficient bayesian optimization. In *ICML*, 2017.

Max Welling and Yee W Teh. Bayesian learning via stochastic gradient langevin dynamics. In *ICML*, 2011.

Justin Whitehouse, Zhiwei Steven Wu, and Aaditya Ramdas. On the sublinear regret of gp-ucb. In *NeurIPS*, volume 36, 2024.

Daniel Widmer, Dongho Kang, Bhavya Sukhija, Jonas Hübotter, Andreas Krause, and Stelian Coros. Tuning legged locomotion controllers via safe bayesian optimization. In *CoRL*, 2023.

Christopher K Williams and Carl Edward Rasmussen. *Gaussian processes for machine learning*, volume 2. MIT press, 2006.

Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3), 1992.

Stephen Wright, Jorge Nocedal, et al. Numerical optimization. *Springer Science*, 35(67-68), 1999.

Pan Xu, Jinghui Chen, Difan Zou, and Quanquan Gu. Global convergence of langevin dynamics based algorithms for nonconvex optimization. In *NeurIPS*, volume 31, 2018.

Yinyu Ye. The simplex and policy-iteration methods are strongly polynomial for the markov decision problem with a fixed discount rate. *Mathematics of Operations Research*, 36(4), 2011.

Hehui Zheng, Bhavya Sukhija, Chenhao Li, Klemens Iten, Andreas Krause, and Robert K Katzschmann. Learning soft robotic dynamics with active exploration. *arXiv preprint arXiv:2510.27428*, 2025.

# *Syllabus*

1. *Introduction* Chapter 1

Part I: Probabilistic Machine Learning

2. *Bayesian Linear Regression (& Filtering)* Chapter 2 through to Section 2.2 (and Chapter 3)
3. *Gaussian Processes 1* rest of Chapter 2 and Chapter 4 through to Section 4.3
4. *Gaussian Processes 2* rest of Chapter 4
5. *Variational Inference* Chapter 5 up to Section 5.5.1
6. *Markov Chain Monte Carlo* Section 5.5.1 and Chapter 6
7. *Bayesian Deep Learning* Chapter 7

Part II: Sequential Decision-Making

8. *Active Learning & Bayesian Optimization* Chapter 8 and Chapter 9
9. *Markov Decision Processes* Chapter 10
10. *Reinforcement Learning 1* Chapter 11
11. *Reinforcement Learning 2* Chapter 12 through to Section 12.3
12. *Reinforcement Learning 3* Section 12.4 through to Section 12.7.3
13. *Reinforcement Learning 4* Chapter 13

# *Summary of Notation*

We follow these general rules:

- uppercase italic for constants $N$
- lowercase italic for indices $i$ and scalar variables $x$
- lowercase italic bold for vectors $\boldsymbol{x}$, entries are denoted $\boldsymbol{x}(i)$
- uppercase italic bold for matrices $\boldsymbol{M}$, entries are denoted $\boldsymbol{M}(i, j)$
- uppercase italic for random variables $X$
- uppercase bold for random vectors $\mathbf{X}$, entries are denoted $\mathbf{X}(i)$
- uppercase italic for sets $A$
- uppercase calligraphy for spaces (usually infinite sets) $\mathcal{A}$

| | |
|---|---|
| $\doteq$ | equality by definition |
| $\approx$ | approximately equals |
| $\propto$ | proportional to (up to multiplicative constants), $f \propto g$ iff $\exists k.\ \forall x.\ f(x) = k \cdot g(x)$ |
| const | an (additive) constant |
| $\mathbb{N}$ | set of natural numbers $\{1, 2, \dots\}$ |
| $\mathbb{N}_0$ | set of natural numbers, including 0, $\mathbb{N} \cup \{0\}$ |
| $\mathbb{R}$ | set of real numbers |
| $[m]$ | set of natural numbers from 1 to $m$, $\{1, 2, \dots, m-1, m\}$ |
| $i : j$ | subset of natural numbers between $i$ and $j$, $\{i, i+1, \dots, j-1, j\}$ |
| $(a, b]$ | real interval between $a$ and $b$ including $b$ but not including $a$ |
| $f : A \to B$ | function $f$ from elements of set $A$ to elements of set $B$ |
| $f \circ g$ | function composition, $f(g(\cdot))$ |
| $(\cdot)_+$ | $\max\{0, \cdot\}$ |
| log | logarithm with base $e$ |
| $\mathcal{P}(A)$ | power set (set of all subsets) of $A$ |
| $\mathbb{1}\{\mathit{predicate}\}$ | indicator function ($\mathbb{1}\{\mathit{predicate}\} \doteq 1$ if the *predicate* is true, else 0) |
| $\odot$ | Hadamard (element-wise) product |
| $\leftarrow$ | assignment |

### ANALYSIS

| | |
|---|---|
| $\nabla f(\boldsymbol{x}) \in \mathbb{R}^n$ | gradient of a function $f : \mathbb{R}^n \to \mathbb{R}$ at a point $\boldsymbol{x} \in \mathbb{R}^n$ |
| $\boldsymbol{D}\boldsymbol{g}(\boldsymbol{x}) \in \mathbb{R}^{m \times n}$ | Jacobian of a function $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^m$ at a point $\boldsymbol{x} \in \mathbb{R}^n$ |
| $\boldsymbol{H}f(\boldsymbol{x}) \in \mathbb{R}^{n \times n}$ | Hessian of a function $f : \mathbb{R}^n \to \mathbb{R}$ at a point $\boldsymbol{x} \in \mathbb{R}^n$ |
| $\nabla \cdot \mathbf{F}$ | divergence operation on vector field $\mathbf{F}$ |
| $\Delta f$ | Laplacian of a scalar field $f : \mathbb{R}^n \to \mathbb{R}$ |
| $f \in O(g)$ | $f$ grows at most as fast as $g$ (up to constant factors), $0 \leq \limsup_{n \to \infty} \left\lvert \frac{f(n)}{g(n)} \right\rvert < \infty$ |
| $f \in \widetilde{O}(g)$ | $f$ grows at most as fast as $g$ up to constant and logarithmic factors |
| $\lVert \cdot \rVert_\alpha$ | $\alpha$-norm |
| $\lVert \cdot \rVert_{\boldsymbol{A}}$ | Mahalanobis norm induced by matrix $\boldsymbol{A}$ |

### LINEAR ALGEBRA

| | |
|---|---|
| $\boldsymbol{I}$ | identity matrix |
| $\boldsymbol{A}^\top$ | transpose of matrix $\boldsymbol{A}$ |
| $\boldsymbol{A}^{-1}$ | inverse of invertible matrix $\boldsymbol{A}$ |
| $\boldsymbol{A}^{1/2}$ | square root of a symmetric and positive semi-definite matrix $\boldsymbol{A}$ |
| $\det(\boldsymbol{A})$ | determinant of $\boldsymbol{A}$ |
| $\operatorname{tr}(\boldsymbol{A})$ | trace of $\boldsymbol{A}$, $\sum_i \boldsymbol{A}(i,i)$ |
| $\operatorname{diag}_{i \in I}\{a_i\}$ | diagonal matrix with elements $a_i$, indexed according to the set $I$ |

### PROBABILITY

| | |
|---|---|
| $\Omega$ | sample space |
| $\mathcal{A}$ | event space |
| $\mathbb{P}$ | probability measure |
| $X \sim P$ | random variable $X$ follows the distribution $P$ |
| $X_{1:n} \overset{\text{iid}}{\sim} P$ | random variables $X_{1:n}$ are independent and identically distributed according to distribution $P$ |
| $x \sim P$ | value $x$ is sampled according to distribution $P$ |
| $P_X$ | cumulative distribution function of a random variable $X$ |
| $\overline{P}_X$ | tail distribution function of a random variable $X$ |
| $P_X^{-1}$ | quantile function of a random variable $X$ |
| $p_X$ | probability mass function (if discrete) or probability density function (if continuous) of a random variable $X$ |
| $\Delta^{\mathcal{A}}$ | set of all probability distributions over the set $\mathcal{A}$ |
| $\delta_\alpha$ | Dirac delta function, point density at $\alpha$ |
| $\boldsymbol{g}_\sharp p$ | pushforward of a density $p$ under perturbation $\boldsymbol{g}$ |
| $X \perp Y$ | random variable $X$ is independent of random variable $Y$ |

| | |
|---|---|
| $X \perp Y \mid Z$ | random variable $X$ is conditionally independent of random variable $Y$ given random variable $Z$ |
| $\mathbb{E}[X]$ | expected value of random variable $X$ |
| $\mathbb{E}_{x \sim X}[f(x)]$ | expected value of the random variable $f(X)$, $\mathbb{E}[f(X)]$ |
| $\mathbb{E}[X \mid Y]$ | conditional expectation of random variable $X$ given random variable $Y$ |
| $\text{Cov}[X, Y]$ | covariance of random variable $X$ and random variable $Y$ |
| $\text{Cor}[X, Y]$ | correlation of random variable $X$ and random variable $Y$ |
| $\text{Var}[X]$ | variance of random variable $X$ |
| $\text{Var}[X \mid Y]$ | conditional variance of random variable $X$ given random variable $Y$ |
| $\boldsymbol{\Sigma}_{\mathbf{X}}$ | covariance matrix of random vector $\mathbf{X}$ |
| $\boldsymbol{\Lambda}_{\mathbf{X}}$ | precision matrix of random vector $\mathbf{X}$ |
| $\text{MSE}(X)$ | mean squared error of random variable $X$ |
| $\overline{X}_n$ | sample mean of random variable $X$ with $n$ samples |
| $S_n^2$ | sample variance of random variable $X$ with $n$ samples |
| $X_n \xrightarrow{a.s.} X$ | the sequence of random variables $X_n$ converges almost surely to $X$ |
| $X_n \xrightarrow{\mathbb{P}} X$ | the sequence of random variables $X_n$ converges to $X$ in probability |
| $X_n \xrightarrow{\mathcal{D}} X$ | the sequence of random variables $X_n$ converges to $X$ in distribution |
| $\text{S}[u]$ | surprise associated with an event of probability $u$ |
| $\text{H}[p], \text{H}[X]$ | entropy of distribution $p$ (or random variable $X$) |
| $\text{H}[p \Vert q]$ | cross-entropy of distribution $q$ with respect to distribution $p$ |
| $\text{KL}(p \Vert q)$ | KL-divergence of distribution $q$ with respect to distribution $p$ |
| $\text{J}(p \Vert q)$ | relative Fisher information of distribution $q$ with respect to distribution $p$ |
| $\text{H}[X \mid Y]$ | conditional entropy of random variable $X$ given random variable $Y$ |
| $\text{H}[X, Y]$ | joint entropy of random variables $X$ and $Y$ |
| $\text{I}(X; Y)$ | mutual information of random variables $X$ and $Y$ |
| $\text{I}(X; Y \mid Z)$ | conditional mutual information of random variables $X$ and $Y$ given random variable $Z$ |
| $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ | normal distribution with mean $\boldsymbol{\mu}$ covariance $\boldsymbol{\Sigma}$ |
| $\text{Laplace}(\boldsymbol{\mu}, h)$ | Laplace distribution with mean $\boldsymbol{\mu}$ scale $h$ |
| $\text{Unif}(S)$ | uniform distribution on the set $S$ |
| $\text{Bern}(p)$ | Bernoulli distribution with success probability $p$ |
| $\text{Bin}(n, p)$ | binomial distribution with $n$ trials and success probability $p$ |
| $\text{Beta}(\alpha, \beta)$ | beta distribution with shape parameters $\alpha$ and $\beta$ |
| $\text{Gamma}(\alpha, \beta)$ | gamma distribution with shape $\alpha$ and rate $\beta$ |
| $\text{Cauchy}(m, \tau)$ | Cauchy distribution with location $m$ and scale $\tau$ |
| $\text{Pareto}(c, \alpha)$ | Pareto distribution with cutoff threshold $c$ and shape $\alpha$ |

## SUPERVISED LEARNING

| | |
|---|---|
| $\boldsymbol{\theta}$ | parameterization of a model |
| $\mathcal{X}$ | input space |
| $\mathcal{Y}$ | label space |

| | |
|---|---|
| $\boldsymbol{x} \in \mathcal{X}$ | input |
| $\epsilon(\boldsymbol{x})$ | zero-mean noise, sometimes assumed to be independent of $\boldsymbol{x}$ |
| $y \in \mathcal{Y}$ | (noisy) label, $f(\boldsymbol{x}) + \epsilon(\boldsymbol{x})$ where $f$ is unknown |
| $\mathcal{D} \subseteq \mathcal{X} \times \mathcal{Y}$ | labeled training data, $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ |
| $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ | design matrix when $\mathcal{X} = \mathbb{R}^d$ |
| $\boldsymbol{\Phi} \in \mathbb{R}^{n \times e}$ | design matrix in feature space $\mathbb{R}^e$ |
| $\boldsymbol{y} \in \mathbb{R}^n$ | label vector when $\mathcal{Y} = \mathbb{R}$ |
| $p(\boldsymbol{\theta})$ | prior belief about $\boldsymbol{\theta}$ |
| $p(\boldsymbol{\theta} \mid \boldsymbol{x}_{1:n}, y_{1:n})$ | posterior belief about $\boldsymbol{\theta}$ given training data |
| $p(y_{1:n} \mid \boldsymbol{x}_{1:n}, \boldsymbol{\theta})$ | likelihood of training data under the model parameterized by $\boldsymbol{\theta}$ |
| $p(y_{1:n} \mid \boldsymbol{x}_{1:n})$ | marginal likelihood of training data |
| $\hat{\boldsymbol{\theta}}_{\text{MLE}}$ | maximum likelihood estimate of $\boldsymbol{\theta}$ |
| $\hat{\boldsymbol{\theta}}_{\text{MAP}}$ | maximum a posteriori estimate of $\boldsymbol{\theta}$ |
| $\ell_{\text{nll}}(\boldsymbol{\theta}; \mathcal{D})$ | negative log-likelihood of the training data $\mathcal{D}$ under model $\boldsymbol{\theta}$ |

## BAYESIAN LINEAR MODELS

| | |
|---|---|
| $\boldsymbol{w} \in \mathbb{R}^d$ | weights of linear function $f(\boldsymbol{x}; \boldsymbol{w}) = \boldsymbol{w}^\top \boldsymbol{x}$ |
| $\hat{\boldsymbol{w}}_{\text{ls}}$ | least squares estimate of $\boldsymbol{w}$ |
| $\hat{\boldsymbol{w}}_{\text{ridge}}$ | ridge estimate of $\boldsymbol{w}$ |
| $\hat{\boldsymbol{w}}_{\text{lasso}}$ | lasso estimate of $\boldsymbol{w}$ |
| $\mathcal{N}(\boldsymbol{0}, \sigma_{\text{p}}^2 \boldsymbol{I})$ | prior |
| $\mathcal{N}(\boldsymbol{w}^\top \boldsymbol{x}, \sigma_{\text{n}}^2)$ | likelihood |
| $\boldsymbol{\mu} \in \mathbb{R}^d$ | posterior mean, $\sigma_{\text{n}}^{-2} \boldsymbol{\Sigma} \boldsymbol{X}^\top \boldsymbol{y}$ |
| $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$ | posterior covariance matrix, $(\sigma_{\text{n}}^{-2} \boldsymbol{X}^\top \boldsymbol{X} + \sigma_{\text{p}}^{-2} \boldsymbol{I})^{-1}$ |
| $\boldsymbol{K} \in \mathbb{R}^{n \times n}$ | kernel matrix, $\sigma_{\text{p}}^2 \boldsymbol{X} \boldsymbol{X}^\top$ |
| $\sigma$ | logistic function |
| $\text{Bern}(\sigma(\boldsymbol{w}^\top \boldsymbol{x}))$ | logistic likelihood |
| $\ell_{\log}(\cdot; \boldsymbol{x}, y)$ | logistic loss of a single training example $(\boldsymbol{x}, y)$ |

## KALMAN FILTERS

| | |
|---|---|
| $\mathbf{X}_t$ | sequence of hidden states in $\mathbb{R}^d$ |
| $\mathbf{Y}_t$ | sequence of observations in $\mathbb{R}^m$ |
| $\boldsymbol{F} \in \mathbb{R}^{d \times d}$ | motion model |
| $\boldsymbol{H} \in \mathbb{R}^{m \times d}$ | sensor model |
| $\boldsymbol{\varepsilon}_t$ | zero-mean motion noise with covariance matrix $\boldsymbol{\Sigma}_x$ |
| $\boldsymbol{\eta}_t$ | zero-mean sensor noise with covariance matrix $\boldsymbol{\Sigma}_y$ |
| $\boldsymbol{K}_t \in \mathbb{R}^{d \times m}$ | Kalman gain |

### GAUSSIAN PROCESSES

| | |
|---|---|
| $\mu : \mathcal{X} \to \mathbb{R}$ | mean function |
| $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ | kernel function / covariance function |
| $f \sim \mathcal{GP}(\mu, k)$ | $f$ is a Gaussian process with mean function $\mu$ and kernel function $k$ |
| $\mathcal{H}_k(\mathcal{X})$ | reproducing kernel Hilbert space associated with kernel function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ |

### DEEP MODELS

| | |
|---|---|
| $\boldsymbol{W}_l \in \mathbb{R}^{n_l \times n_{l-1}}$ | weight matrix of layer $l$ |
| $\boldsymbol{v}^{(l)} \in \mathbb{R}^{n_l}$ | activations of layer $l$ |
| $\varphi$ | activation function |
| Tanh | hyperbolic tangent activation function |
| ReLU | rectified linear unit activation function |
| $\sigma_i(\boldsymbol{f})$ | softmax function computing the probability mass of class $i$ given outputs $\boldsymbol{f}$ |

### VARIATIONAL INFERENCE

| | |
|---|---|
| $\mathcal{Q}$ | variational family |
| $\boldsymbol{\lambda} \in \Lambda$ | variational parameters |
| $q_{\boldsymbol{\lambda}}$ | variational posterior parameterized by $\boldsymbol{\lambda}$ |
| $L(q, p; \mathcal{D})$ | evidence lower bound for data $\mathcal{D}$ of variational posterior $q$ and true posterior $p(\cdot \mid \mathcal{D})$ |

### MARKOV CHAINS

| | |
|---|---|
| $S$ | set of $n$ states |
| $X_t$ | sequence of states |
| $p(x' \mid x)$ | transition function, probability of going from state $x$ to state $x'$ |
| $p^{(t)}(x' \mid x)$ | probability of reaching $x'$ from $x$ in exactly $t$ steps |
| $\boldsymbol{P} \in \mathbb{R}^{n \times n}$ | transition matrix |
| $q_t$ | distribution over states at time $t$ |
| $\pi$ | stationary distribution |
| $\|\mu - \nu\|_{\text{TV}}$ | total variation distance between two distributions $\mu$ and $\nu$ |
| $\tau_{\text{TV}}$ | mixing time with respect to total variation distance |

### MARKOV CHAIN MONTE CARLO METHODS

| | |
|---|---|
| $r(\boldsymbol{x}' \mid \boldsymbol{x})$ | proposal distribution, probability of proposing $\boldsymbol{x}'$ when in $\boldsymbol{x}$ |
| $\alpha(\boldsymbol{x}' \mid \boldsymbol{x})$ | acceptance distribution, probability of accepting the proposal $\boldsymbol{x}'$ when in $\boldsymbol{x}$ |
| $f$ | energy function |

### ACTIVE LEARNING

| | |
|---|---|
| $S \subseteq \mathcal{X}$ | set of observations |
| $I(S)$ | maximization objective, quantifying the "information value" of $S$ |
| $\Delta_I(\boldsymbol{x} \mid A)$ | marginal gain of observation $\boldsymbol{x}$ with respect to objective $I$ given prior observations $A$ |

### BAYESIAN OPTIMIZATION

| | |
|---|---|
| $R_T$ | cumulative regret for time horizon $T$ |
| $F(\boldsymbol{x}; \mu, \sigma)$ | acquisition function |
| $\mathcal{C}_t(\boldsymbol{x})$ | confidence interval of $f^\star(\boldsymbol{x})$ after round $t$ |
| $\beta_t(\delta)$ | scale of confidence interval to achieve confidence level $\delta$ |
| $\gamma_T$ | maximum information gain after $T$ rounds |

### REINFORCEMENT LEARNING

| | |
|---|---|
| $X, \mathcal{X}$ | set of states |
| $A, \mathcal{A}$ | set of actions |
| $p(x' \mid x, a)$ | dynamics model, probability of transitioning from state $x$ to state $x'$ when playing action $a$ |
| $r$ | reward function |
| $X_t$ | sequence of states |
| $A_t$ | sequence of actions |
| $R_t$ | sequence of rewards |
| $\pi(a \mid x)$ | policy, probability of playing action $a$ when in state $x$ |
| $G_t$ | discounted payoff from time $t$ |
| $\gamma$ | discount factor |
| $v_t^\pi(x)$ | state value function, average discounted payoff from time $t$ starting from state $x$ |
| $q_t^\pi(x, a)$ | state-action value function, average discounted payoff from time $t$ starting from state $x$ playing action $a$ |
| $a_t^\pi(x, a)$ | advantage function, $q_t^\pi(x, a) - v_t^\pi(x)$ |
| $J(\pi)$ | policy value function, expected reward of policy $\pi$ |

# *Acronyms*

*a.s.* almost surely, with high probability, with probability 1

*A2C* advantage actor-critic

*BALD* Bayesian active learning by disagreement

*BLR* Bayesian linear regression

*CDF* cumulative distribution function

*CLT* central limit theorem

*DDPG* deep deterministic policy gradients

*DDQN* double deep Q-networks

*DPO* direct preference optimization

*DQN* deep Q-networks

*ECE* expected calibration error

*EI* expected improvement

*ELBO* evidence lower bound

*ES* entropy search

*FITC* fully independent training conditional

*GAE* generalized advantage estimation

*GD* gradient descent

*GLIE* greedy in the limit with infinite exploration

*GP* Gaussian process

*GPC* Gaussian process classification

*GRPO* group relative policy optimization

*GRV* Gaussian random vector

*H-UCRL* hallucinated upper confidence reinforcement learning

*HMC* Hamiltonian Monte Carlo

*HMM* hidden Markov model

*i.i.d.* independent and identically distributed

*IDS* information-directed sampling

*iff* if and only if

*JES* joint entropy search

*KL* Kullback-Leibler

*LAMBDA* Lagrangian model-based agent

*LASSO* least absolute shrinkage and selection operator

*LD* Langevin dynamics

*LITE* linear-time independence-based estimators (of probability of maximality)

*LMC* Langevin Monte Carlo

*LOTE* law of total expectation

*LOTP* law of total probability

*LOTUS* law of the unconscious statistician

*LOTV* law of total variance

*LSI* log-Sobolev inequality

*MAB* multi-armed bandits

*MALA* Metropolis adjusted Langevin algorithm

*MAP* maximum a posteriori

*MC* Monte Carlo

*MCE* maximum calibration error

*MCMC* Markov chain Monte Carlo

*MCTS* Monte Carlo tree search

*MDP* Markov decision process

*MERL* maximum entropy reinforcement learning

*MES* max-value entropy search

*MGF* moment-generating function

*MI* mutual information

*MLE* maximum likelihood estimate

*MLL* marginal log likelihood

*MPC* model predictive control

*MSE* mean squared error

*NLL* negative log likelihood

*ODE* ordinary differential equation

*OPES* output-space predictive entropy search

*PBPI* point-based policy iteration

*PBVI* point-based value iteration

*PDF* probability density function

*PES* predictive entropy search

*PETS* probabilistic ensembles with trajectory sampling
*PI* probability of improvement
*PI* policy iteration
*PILCO* probabilistic inference for learning control
*PL* Polyak-Łojasiewicz
*PlaNet* deep planning network
*PMF* probability mass function
*POMDP* partially observable Markov decision process
*PPO* proximal policy optimization
*RBF* radial basis function
*ReLU* rectified linear unit
*RHC* receding horizon control
*RKHS* reproducing kernel Hilbert space
*RL* reinforcement learning
*RLHF* reinforcement learning from human feedback
*RM* Robbins-Monro
*SAA* stochastic average approximation
*SAC* soft actor-critic
*SARSA* state-action-reward-state-action
*SDE* stochastic differential equation
*SG-HMC* stochastic gradient Hamiltonian Monte Carlo
*SGD* stochastic gradient descent
*SGLD* stochastic gradient Langevin dynamics
*SLLN* strong law of large numbers
*SOMBRL* scalable and optimistic model-based reinforcement learning
*SoR* subset of regressors
*SVG* stochastic value gradients
*SVGD* Stein variational gradient descent
*SWA* stochastic weight averaging
*SWAG* stochastic weight averaging-Gaussian
*Tanh* hyperbolic tangent
*TD* temporal difference
*TD3* twin delayed DDPG
*TRPO* trust-region policy optimization
*UCB* upper confidence bound
*ULA* unadjusted Langevin algorithm
*VI* variational inference
*VI* value iteration
*w.l.o.g.* without loss of generality
*w.r.t.* with respect to
*WLLN* weak law of large numbers

# *Index*


$L_1$-regularization, 324
$L_2$-regularization, 324
$L_\infty$ norm, 208
$R_{\max}$ algorithm, 230
$\sigma$-algebra, 3
$\varepsilon$-greedy, **227**, 294

acceptance distribution, 121
acquisition function, **186**, 293
activation, 140
activation function, 140
active inference, 266
actor, 253
actor-critic method, 253
Adagrad, 325
Adam, 325
adaptive learning rate, 325
additive white Gaussian noise, 40
advantage actor-critic, 254
advantage function, 250
aleatoric uncertainty, 44
almost sure convergence, 313
alpha-beta pruning, 282
aperiodic Markov chain, 117
approximation error, 22
augmented Lagrangian method, 298
automatic differentiation (auto-diff), 142

backpropagation, 142
bagging, 150
Banach fixed-point theorem, 208
Barrier function, 299
baseline, 248
batch, 323
batch size, 324
Bayes by Backprop, 145
Bayes' rule, 15
Bayes' rule for entropy, 168
Bayesian active learning by disagreement, 177
Bayesian experimental design, 180
Bayesian filtering, 51
Bayesian linear regression, 43
Bayesian logistic regression, 85
Bayesian networks, 9
Bayesian neural network, 143
Bayesian optimization, 186
Bayesian reasoning, 2
Bayesian smoothing, 53
belief, 52, **216**
belief-state Markov decision process, 217
Bellman error, **242**, 260
Bellman expectation equation, 206
Bellman optimality equation, 211
Bellman update, 211
Bellman's optimality principle, 211
Bellman's theorem, 210
Bernoulli distribution, 305
Bernstein-von Mises theorem, 26
bias, 310
bias-variance tradeoff, 254
binary cross-entropy loss, 98
binomial distribution, 305
black box stochastic variational inference, 104
Bochner's theorem, 73
Boltzmann distribution, 124
Boltzmann exploration, 228

bootstrapping, 150, **231**
Borel $\sigma$-algebra, 4
bounded discounted payoff, 245
Bradley-Terry model, 268
Brownian motion, 127
burn-in time, 120

calibration, 152
catastrophe principle, 312
categorical distribution, 305
Cauchy distribution, 312
central limit theorem, 315
Cesàro mean, 199
chain rule for entropy, 168
chain rule of mutual information, 193
chain rule of probability, 8
change of variables formula, 14
Chebyshev's inequality, 326
Cholesky decomposition, 308
classification, 23
closed-loop control, 285
competitive ratio, 186
completing the square, 340
computation graph, 139
concave function, 320
conditional distribution, 7
conditional entropy, 168
conditional expectation, 11
conditional independence, 8
conditional likelihood, 15
conditional linear Gaussian, 22
conditional mutual information, 170
conditional probability, 7
conditional variance, 13
conditioning, 8
conjugate prior, 19
consistent estimator, 310
conspiracy principle, 312
constrained Markov decision processes, 298
context, 267
continuous setting, 224
contraction, 208
contrapositive, 16
control, 186
convergence in distribution, 314
convergence in probability, 313
convergence with probability 1, 313
convex body chasing, 185
convex function, 320
convex function chasing, 185
correlation, 12
cosine similarity, 12
covariance, 11
covariance function, 60
covariance matrix, 13
Cramér-Rao lower bound, 318
credible set, 28
critic, 253
Cromwell's rule, 25
cross-entropy, 92
cross-entropy loss, 142
cross-entropy method, 282
curse of dimensionality, 189

d-separation, 9
data processing inequality, 181
deep deterministic policy gradients, **259**, 286
deep planning network, 293
deep Q-networks, 243
deep reinforcement learning, 242
density, 6
design, 180
design criterion, 180
design matrix, 39
detailed balance equation, 119
differentiation under the integral sign, 307
diffusion, 128
diffusion process, 127
Dirac delta function, 306
direct preference optimization, 271
directed graphical model, 9
directional derivative, 326
Dirichlet distribution, 19
discounted payoff, **204**, 246, 281
discounted state occupancy measure, 252
distillation, 128
Doob's consistency theorem, 25
double DQN, 244

downstream return, 249
Dreamer, 293
dropconnect regularization, 147
dropout regularization, 147
dynamics model, 203

efficient, 318
eligibility vector, 274
empirical Bayes, 69
empirical risk, 318
energy function, 124
entropy, 18, **91**
entropy regularization, 196, **262**
entropy search, 180
episode, 224
episodic setting, 224
epistemic uncertainty, 44
epoch, 324
equilibrium, 135
ergodic theorem, 120
ergodicity, 117
estimation, 26
estimation error, 22
estimator, 309
Euler's formula, 72
event, 3
event space, 3
evidence, 100
exclusive KL-divergence, 95
expectation, 10
expected calibration error, 154
expected improvement, 191
experience replay, 243
experimental design, 180
exploration distribution, 258
exploration noise, 293
exploration-exploitation dilemma, **183**, 223, 293
exponential family, 98, 276
exponential kernel, 63

feature space, 45
feed-forward neural network, 142
filtering, 37, **51**, 215, 216
finite measure, 114
finite-horizon, 205
first-order characterization of convexity, 320
first-order expansion, 320
first-order optimality condition, 320
Fisher information, 318
fixed-point iteration, 208
Fokker-Planck equation, 136
forward KL-divergence, 95
forward sampling, 61
forward-backward algorithm, 54
foundation model, 267
Fourier transform, 73
free energy, 105
free energy principle, **105**, 265
fully independent training conditional, 76
fully observable environment, 204
function class, 22
function-space view, 45
fundamental theorem of ergodic Markov chains, 117

gamma distribution, 133
Gaussian, 6, **20**
Gaussian kernel, 63
Gaussian noise "dithering", 259, **294**
Gaussian process, 59
Gaussian process classification, 85, **106**
Gaussian random vector, 20
generalized advantage estimation, 255
generalized variance, 92
generative model, 7
Gibbs distribution, 124
Gibbs sampling, 123
Gibbs' inequality, 108
global optimum, 319
goal-conditioned reinforcement learning, 303
gradient, 306
gradient flow, 135
greedy in the limit with infinite exploration, 228
greedy policy, 210
group relative policy optimization, 257
Grönwall's inequality, 135
Gumbel-max trick, 261

Hadamard's inequality, 325
hallucinated upper confidence reinforcement learning, 296
Hamiltonian, 130
Hamiltonian Monte Carlo, 129
heat kernel, 78
heavy-tailed distribution, 311
Hessian, 322
heteroscedastic, **23**
heteroscedastic noise, 144
hidden layer, 140
hidden Markov model, 215
histogram binning, 154
Hoeffding's inequality, 316
Hoeffding's lemma, 316
homoscedastic, **23**
homoscedastic noise, 144
hyperbolic tangent, 140
hyperprior, 50, **70**, 186

importance sampling, 255
improper prior, 18
improvement, 190
inclusive KL-divergence, 95
independence, 8
inducing points method, 75
inductive learning, 178
information gain, 169
information never hurts, **168**, 169
information ratio, 192
information-directed sampling, 194
informative prior, 17
input layer, 139
inputs, 22
instantaneous, 205
instantaneous regret, 185
interaction information, 170
intrinsic reward, 171
inverse Fourier transform, 73
inverse transform sampling, 114, 306
irreducible Markov chain, 116
isotonic regression, 154
isotropic Gaussian, 20
isotropic kernel, 65

Jacobian, 322
Jensen's inequality, 91
joint distribution, 7
joint entropy, 168
joint likelihood, 16

Kalman filter, 52
Kalman gain, 55, **56**
Kalman smoothing, 53
Kalman update, 55
kernel function, 46, 60, **62**
kernel matrix, 46
kernel ridge regression, 79
kernel trick, 46
kernelization, 46
Kolmogorov axioms, 3
Kullback-Leibler divergence, 93

labels, 22
Lagrangian model-based agent, 300
Langevin dynamics, 127
Langevin Monte Carlo, 126
Laplace approximation, 84
Laplace distribution, 311
Laplace kernel, 63
Laplace's method, 83
lasso, 42
law of large numbers, 315
law of the unconscious statistician, 10
law of total expectation, 11
law of total probability, 8
law of total variance, 13
layer, 139
Leapfrog method, 130
learning, 27
least squares estimator, 39
length scale, 63
light-tailed distribution, 311
likelihood, 15
linear kernel, 62
linear regression, 39
linearity of expectation, 10
local optimum, 319
Loewner order, 318

log-concave distribution, 124
log-likelihood, 23
log-normal distribution, 311
log-prior, 24
log-Sobolev inequality, 137
logical inference, 1
logistic function, 85
logistic loss, 86
logits, 140
loss function, 319
Lyapunov function, 135

Mahalanobis norm, 307
marginal gain, 172
marginal likelihood, 16
marginalization, 7
Markov chain, 114
Markov decision process, 203
Markov property, 52, **114**
Markov's inequality, 326
masksemble, 149
matrix inversion lemma, 325
Matérn kernel, 63
maximization bias, 243
maximizing the marginal likelihood, 68
maximum a posteriori estimate, 24
maximum calibration error, 154
maximum entropy principle, 18
maximum entropy reinforcement learning, 262
maximum likelihood estimate, 23
mean, 10
mean function, 60
mean payoff, 205
mean square continuity, 314
mean square convergence, 314
mean square differentiability, 314
mean squared error, 141, **310**
mean-field distribution, 89
median-of-means, 317
method of moments, 98
metrical task systems, 185
Metropolis adjusted Langevin algorithm, 126
Metropolis-Hastings algorithm, 121
Metropolis-Hastings theorem, 122
mixing time, 118
mode, 6
model predictive control, 281
model-based reinforcement learning, **225**, 280
model-free reinforcement learning, 225
moment matching, **98**, 287
moment-generating function, 33
momentum, 325
monotone submodularity, 172
monotonicity of mutual information, 193
Monte Carlo approximation, 287, **313**
Monte Carlo control, 227
Monte Carlo rollouts, 286
Monte Carlo trajectory sampling, 284
Monte Carlo tree search, 282
most likely explanation, 216
multi-armed bandits, 184
multicollinearity, 40
multilayer perceptron, 139
multinomial distribution, 19, 305
mutual information, 167, **169**

natural parameters, 98
negative log-likelihood, 23, **142**
neural fitted Q-iteration, 243
neural network, 139
noninformative prior, 17
normal distribution, 6, **20**
normalizing constant, 16, 27

objective function, 319
Occam's razor, 18
off-policy, 225
on-policy, 225
online, 52
online actor-critic, 254
online algorithms, 185
online Bayesian linear regression, 49
online learning, 184
open-loop control, 285
optimal control, 281
optimal decision rule, 29
optimal design, 180

optimism in the face of uncertainty, 30, **185**, 187, 191, 196, 229, 235, 294
optimistic exploration, 295
optimistic Q-learning, 236
outcome reward, 269
output layer, 140
output scale, 62, **64**
outputs, 22
overfitting, 24, **319**

Pareto distribution, 312
partially observable Markov decision process, 215
partition function, 98
perception-action loop, 165
performance plan, 301
Pinsker's inequality, 118
planning, **203**, 223
plate notation, 10
Platt scaling, 155
plausible inference, 16
plausible model, 294
point density, 306
point estimate, 26
point-based policy iteration, 218
point-based value iteration, 218
pointwise convergence, 80
policy, 204
policy gradient method, 245
policy gradient theorem, 252
policy iteration, **212**, 257
policy search method, 245
policy value function, 245
Polyak averaging, 243
Polyak-Łojasiewicz inequality, 135
population risk, 318
positive definite, 307
positive definite kernel, 62
positive semi-definite, 307
posterior, 15, **28**
precision matrix, 20
prediction, 27
predictive posterior, 28
principle of curiosity and conformity, **105**, 128, 152, 184
principle of indifference, 17
principle of insufficient reason, 17
principle of parsimony, 18
prior, 15
probabilistic artificial intelligence, 2
probabilistic ensembles with trajectory sampling, 292
probabilistic inference, 1, **15**, 27, 37
probabilistic inference for learning control, 287, **292**
probability, 4
probability density function, 5
probability matching, 192
probability measure, 3
probability of improvement, 190
probability of maximality, 192, **195**
probability simplex, 218, **305**
probability space, 4
probit likelihood, 107
product rule, 8
proposal distribution, 121
proximal policy optimization, 256
pseudo-random number generators, 306
pushforward, 15

Q actor-critic, 253
Q-function, 205
Q-learning, 235
quadratic form, 307
quantile function, 306

radial basis function kernel, 63
random Fourier features, 72
random shooting methods, 282
random variable, 4
random vector, 7
rapidly mixing Markov chain, 118
realizations of a random variable, 5
recall, 195
receding horizon control, 281
rectified linear unit, 140
recursive Bayesian estimation, 51
redundancy, 170
regression, 23

regret, 185
Reichenbach's common cause principle, 9
REINFORCE algorithm, 249
reinforcement learning, 223
reinforcement learning from human feedback, 269
relative entropy, 93
relative Fisher information, 137
reliability diagram, 153
reparameterizable distribution, 103
reparameterization trick, **103**, 261, 284
replay buffer, 243
representer theorem, 66
reproducing kernel Hilbert space, 66
reproducing property, 66
reverse KL-divergence, 95
reversible Markov chain, 119
reward shaping, 220
reward to go, 249
ridge regression, 40
Robbins-Monro algorithm, 323
Robbins-Monro conditions, 323
robust control, 298
robust statistics, 317
rollout, 246

saddle point, 320
safety filter, 302
safety plan, 301
sample covariance matrix, 310
sample mean, 309
sample space, 3
sample variance, 310
SARSA, 233
scalable and optimistic model-based reinforcement learning, 297
score function, **102**, 247
score function trick, 247
score gradient estimator, 102, **247**
second-order characterization of convexity, 322
second-order expansion, 322
self-conjugacy, 19
self-supervised learning, 267
semi-conjugate prior, 133
sharply concentrated, 310
Sherman-Morrison formula, 325
shift-invariant kernel, 65
small tails, 311
snapshot, 146
soft actor critic, **264**, 287
soft Q-learning, 264
soft value function, 264
softmax exploration, 228
softmax function, 141
spectral density, 73
square root, 308
squared exponential kernel, 63
standard deviation, 13
standard normal distribution, 6, **20**
state of a random variable, 5
state space model, 51
state value function, 205
state-action value function, 205
state-dependent baseline, 275
stationary distribution, 115
stationary kernel, 65
stationary point, 320
Stein variational gradient descent, 151
Stein's method, 152
stochastic average approximation, 283
stochastic environment, 203
stochastic gradient descent, 323
stochastic gradient Langevin dynamics, 129
stochastic matrix, 115
stochastic process, 114
stochastic semi-gradient descent, 241
stochastic value gradients, **261**, 286
stochastic weight averaging, 147
stochastic weight averaging-Gaussian, 147
strict convexity, 320
sub-Gaussian random variable, 316
sublinear regret, 185
submodularity, 172
subsampling, 146
subset of regressors, 76
sufficient statistic, 98, 146, **170**
sum rule, 7
supervised learning, **22**, 167
support, 5

supremum norm, 208
surprise, 89
symbolic artificial intelligence, 1
synergy, 171

tabular setting, 223
tail distribution function, 311
tail index, 312
target space, 4
temperature scaling, 155
temporal models, 57
temporal-difference error, 241
temporal-difference learning, 232
testing conditional, 76
Thompson sampling, **192**, 294
time-homogeneous process, 114
total variance, 92
total variation distance, 118
tower rule, 11
training conditional, 76
trajectory, 224
trajectory sampling, 283
transductive learning, 178
transition, 224
transition function, 115
transition graph, 115
transition matrix, 115
truncated sample mean, 317
trust-region policy optimization, 255
twin delayed DDPG, 259
two-filter smoothing, 54

unadjusted Langevin algorithm, 127
unbiased estimator, 310
uncertainty sampling, 175
uncorrelated, 11
uniform convergence, 80
uniform distribution, 305, **306**
union bound, 31
universal approximation theorem, 140
universality of the uniform, **306**, 359
upper confidence bound, 187

value iteration, 214
variance, 12
variational family, 88
variational inference, **88**, 145, 150, 287
variational parameters, 83
variational posterior, 83
Viterbi algorithm, 216

weak law of large numbers, 315
weight decay, 324
weight-space view, 40
Weinstein-Aronszajn identity, 325
well-calibrated confidence interval, 188
well-specified prior, 25
Wiener process, 78, 114, **127**
Woodbury matrix identity, 325
world model, 279

# *Errata*

*No errata so far.*